

PrimeSim™ HSPICE® User Guide: Basic Simulation and Analysis

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

New in This Release	28
Related Products, Publications, and Trademarks	28
Conventions	29
Customer Support	30
Statement on Inclusivity and Diversity	30

Part 1: Introduction to PrimeSim HSPICE

1. PrimeSim HSPICE Overview	32
Features	33
Custom CMI	35
TSMC Model Interface	35
Si2 Compact Model Coalition Model Interface - OMI	36
PrimeSim HSPICE Features for Running Higher-Level Simulations	37
Simulation Structure	38
Experimental Methods Supported by PrimeSim HSPICE	38
PrimeSim HSPICE Basic Analysis Types	39
DC Analysis of an Inverter	39
AC Analysis of an RC Network	41
Transient Analysis of an RC Network	44
Transient Analysis of an Inverter	45
Measurement System in PrimeSim HSPICE	47
Simulation Process Overview	48
Parser Syntax Requirements (Unsupported Formats)	49
Use of Example Syntax	49
Recommended PrimeSim HSPICE Resources	50
2. Setup	52
Setting Environment Variables	52
License Variables	53
License Queuing Variable	53

Contents

Controlling the License Queueing Interval	54
License Server Down Iterations	55
Using the FlexLM MAX Option	55
PrimeSim HSPICE License Scheme	55
Limiting the Number of Licenses for Your Group	56
Temporary Directory Variable	56
Shared Libraries Environment Variable	56
Compiled Function Library Environment Variable	57
Using Environment Variables as Include Statements	57
Setting Distributed Processing Environment Variable	57
Setting Array Job Submitting Environment Variable	58
Verilog-A (pVA) Environment Variables	58
Standard Input Files	58
Design and File Naming Conventions	59
Initialization File (hspice.ini)	59
DC Operating Point Initial Conditions File	60
Input Netlist File	60
Library Input File	60
Standard Output Files	60
AC Analysis Results File	62
AC Analysis Measurement Results File	62
DC Analysis Results File	62
DC Analysis Measurement Results File	62
FFT Analysis Graph Data File	63
Operating Point Information File	63
Operating Point Node Voltages File	63
Output Listing File	63
Output Status File	64
Output Tables	65
Subcircuit Cross-Listing File	65
Transient Analysis Measurement Results File	65
Transient Analysis Results File	65
Waveform Viewing File	65
Working Directory Path Character Limit	66
Continuing PrimeSim HSPICE Simulations after a Power-down	66
 3. Startup and Simulation	 68
Invoking PrimeSim HSPICE Simulations	68

Syntax to Invoke PrimeSim HSPICE	69
Sequence of Events	78
Starting PrimeSim HSPICE — Examples	82
Viewing Online Help Topics From the Command-Line	83
List of Keywords for Online Help Topics	84
Interpreting Default Values of .OPTION in PrimeSim HSPICE	87
Using the Example Syntax	87
Using PrimeSim HSPICE for Calculating New Measurements	87
Using Isomorphic Analyses in Subcircuit Blocks	89
Data Inputs and Outputs	90
Limitations	91
Running PrimeSim HSPICE Simulations on Windows	91
Running PrimeSim HSPICE Interactively	91
Starting Interactive Mode	92
Running a Command File in Interactive Mode	92
Quitting Interactive Mode	92
Using PrimeSim HSPICE in Client-Server Mode	92
Starting Client-Server Mode	93
Simulating a Netlist in Client-Server Mode	94
To Quit Client-Server Mode	95
Launching the Advanced Client-Server Mode (-CC)	95
Advanced Client-Server Command Syntax	95
Application Instances	96
Running PrimeSim HSPICE to Calculate New Measurements	99
4. Distributed Processing and Multithreading	100
Predicting the Simulation Time for Monte Carlo Cases	101
Simulation Time Prediction Utility	101
Example	101
Limitations	101
Running Multiple Simulations	102
Running Distributed Processing on a Network Grid	102
Distributed Processing in PrimeSim HSPICE	102
Examples	104
Feature Support for DP	105
Output Files	107

Limitations	112
Running Distributed Processing on One Machine	112
Running Multithreaded Simulations	113
Performance Improvement Estimations	114
Multithreading on Windows	114
Impact of Intel's Hyper-Threading Technology	115
Running Multithreading and Distributed Processing Concurrently	115
PrimeSim HSPICE Precision Parallel	117
Introduction to HPP	117
HPP Status Reporting in *.lis File	118
HPP Limitations	118
5. Using Interactive Mode	120
Invoking PrimeSim HSPICE in Interactive Mode	120
Executing an Interactive Script	121
Multiple Runs in Interactive Mode	121
Using Interactive Mode on the Windows Platform	122
Examples	122
Getting Help	123
Creating a Netlist	123
Specifying an Analysis	124
Running an Analysis	124
Viewing a Netlist	124
Loading and Running an Existing Netlist	124
Using Environment Commands	125
Recording and Saving Interactive Commands to a File	126
Printing a Voltage Value During Simulation	126
Using a Command File to Run in Interactive Mode	127
Running Multiple Testcases	127
Multiple Runs Without Re-parsing the Input Netlist	128
6. PrimeSim HSPICE GUI for Windows	129
Working With Designs	129
Configuring the PrimeSim HSPICE GUI for Windows	130
HSPUI Options	131

Launching WaveView in HSPUI	132
Running Multiple Simulations	132
Building the Batch Job List	134
Simulating a Batch Job	134
1. Using a Batch File	134
2. Using HSPUI	134
Typical Batch Workflow	135
SPutil, Metaencrypt and Converter Utilities, Client/Server Operation	136
CMI Directory Structure	136
7. Input Netlist and Data Entry	137
Input Netlist File Guidelines	137
Input Line Format	139
Case Sensitivity	141
Special Characters	141
First Character	145
Delimiters	145
Instance Names	146
Hierarchy Paths	147
Numbers	147
Parameters and Expressions	148
Reserved Keywords	150
Input Netlist File Structure	151
Schematic Netlists	151
Compressed Netlist Guidelines	153
Input Netlist File Composition	153
PrimeSim HSPICE Topology Rules	156
Title of Simulation	157
Comments and Line Continuation	157
Element and Source Statements	158
Defining Subcircuits	160
Node Name (or Node Identifier) Conventions	161
Using Wildcards on Node Names	162
Wildcard Applications and Examples	163
Element, Instance, and Subcircuit Naming Conventions	164
Subcircuit Node Names	164
Path Names of Subcircuit Nodes	165
Abbreviated Subcircuit Node Names	165

Contents

Automatic Node Name Generation	166
Global Node Names	166
Circuit Temperature	167
Data-Driven Analysis	167
Library Calls and Definitions	167
Library Building Rules	168
Automatic Library Selection (PrimeSim HSPICE)	168
Defining Parameters	169
Predefined Analysis	169
Measurement Parameters	169
Altering Design Variables and Subcircuits	169
Using Multiple .ALTER Blocks	170
Adding or Changing Analyses in .ALTER Blocks	171
Connecting Nodes	172
Deleting a Library	172
Ending a Netlist	172
Condition-Controlled Netlists (IF-ELSE)	173
Using Subcircuits	175
Hierarchical Parameters	176
M (Multiply) Parameter	176
S (Scale) Parameter	177
Using Hierarchical Parameters to Simplify Simulations	178
Undefined Subcircuit Search (PrimeSim HSPICE)	179
Troubleshooting Subcircuit Node Issues	179
Subcircuit Call Statement Discrete Device Libraries	182
DDL Library Access	182
Vendor Libraries	183
Subcircuit Library Structure	184

Part 2: Parameters, Functions, and Output

8. Parameters and Functions	186
9. Simulation Output	187
Overview of Output Statements	188
Output Commands	188
Output Variables	189

Contents

Output Analysis Type	190
Displaying Simulation Results	191
.PRINT Statement	191
Example	191
Statement Order	192
.PROBE Statement	192
Using Wildcards in PRINT and PROBE Statements	192
Supported Wildcard Templates	193
Using filter in .PRINT and .PROBE Statements	193
Using level in .PRINT and .PROBE Statements	194
Switching to .PROBE to Output Subcircuit Port Names	194
Print Control Options	195
Changing the File Descriptor Limit (PrimeSim HSPICE Only)	195
Printing the Subcircuit Output	195
Using .MODEL_INFO to Print Model Parameters	197
Progress Message Time Estimates	198
Selecting Simulation Output Parameters	198
DC and Transient Output Variables	199
Nodal Capacitance Output	200
Nodal Voltage	200
Current: Independent Voltage Sources	200
Terminal Voltage: MOS Instance	201
Current: Element Branches	201
Current: Subcircuit Pin	204
Independent Source Power Output	204
Wildcard Support	205
User-defined Element Length and Width	205
Power Output	205
Total Dissipation Power	206
Element Dissipated Power	206
Element Total Power	206
Operating Point Variables	206
AC Analysis Output Variables	207
Algebraic Expressions	208
Nodal Capacitance Output	209
Nodal Voltage	209
Current: Independent Voltage Sources	210
Current: Element Branches	210
Current: Subcircuit Pin	210
Group Time Delay	211
Network	211

Contents

Noise and Distortion	212
Element Template Output (PrimeSim HSPICE Only)	213
Specifying User-Defined Analysis (.MEASURE)	214
.MEASURE Statement Order	214
.MEASURE Parameter Types	215
FIND and WHEN Functions	217
Continuous Measurement	217
Continuous Measure Output Files (Default)	218
Continuous Measure Output Files (With .OPTION MEASFORM=2)	218
Equation Evaluation	219
Average, RMS, MIN, MAX, INTEG, PP, and EM_AVG	219
Measuring Recovered Electromigration	220
INTEGRAL Function	220
DERIVATIVE Function	221
ERROR Function	221
Error Equations	221
Generating a Measure File (*.mt0, *.ac0, etc.) with All Values in a Single Row	223
Outputting Pass/Fail Measure Data	223
Measurements in MOSRA Analysis	223
Expected State of Digital Output Signal (.DOUT)	224
Reusing Simulation Output as Input Stimuli (PrimeSim HSPICE Only)	226
Output Files	227
Element Template Listings	228
Vdmargin for Vdsat Characterization	235
G-Function Based Vdsat, Gvdsat	235
Output Listing (*.lis) File with .OPTION LIS_NEW Set	236
For MOSFET Information Use .OPTION LIST	238
Verilog-A Simulation Output	239
Verilog-A Output Directory	240
Field Solver *.str File	240
Redirecting the Simulation Output Results Files to a Different Directory	241
Directing .PRINT Output to a Separate File	242
Getting Data Out of PrimeSim HSPICE Plot Files	242
Compressing Waveform Files	244
Using the PrimeSim HSPICE Output Converter Utility	244

PSF Converter	245
PWL/DATA/VEC Converter	246
Syntax	246
Prompt User Mode	247
Input Line Dependencies	248
Running the Converter Utility in Batch Mode	248
Troubleshooting Issues	249
Resolving Inductor/Voltage Source Loop Errors	249
Using Verilog-A	250

Part 3: Analyses and Simulation

10. Initializing DC-Operating Point Analysis	252
Simulation Flow—Initialization and Analysis	252
DC Initialization and Operating Point Calculation	255
.OP Statement — Operating Point	256
Output	256
Element Statement IC Parameter	257
Initial Conditions and UIC Parameters	257
.SAVE and .LOAD Statements (PrimeSim HSPICE Only)	258
.DC Statement—DC Sweeps	259
Multi-dimensional DC Sweep	259
Circuit and Model Temperature Simulation	260
Example	261
Temperature Variation Analysis	261
Other DC Analysis Statements	263
DC Initialization Control Options	263
Accuracy and Convergence	264
Accuracy Tolerances	264
Autoconverge Process	266
Effects of GMINDC	267
Reducing DC Errors	269
Shorted Element Nodes	271
Floating-Point Overflow	271
Diagnosing Convergence Problems	272
Non-Convergence Diagnostic Table	272

Traceback of Non-Convergence Source	274
Solutions for Non-Convergent Circuits	274
Poor Initial Conditions	275
Inappropriate Model Parameters	275
PN Junctions (Diodes, MOSFETs, BJTs)	277
Troubleshooting DC Bias Point and DC Sweep Non-Convergence	277
Convergence Failure: Too Many Current Probes in Netlist	278
Troubleshooting: Nodes set to initial conditions with .IC may not always begin at those voltage values	279
11. AC Small-Signal and Noise Analysis	281
Using the .AC Statement	282
.AC Control Options	282
.AC Command Examples	282
AC Small Signal Analysis	283
Using .NOISE for Small-Signal Noise Analysis	285
Basic Syntax	286
.NOISE Measurements for MOSFETs	288
.NOISE Measurements for BJTs	288
.NOISE Measurements for Diodes	289
Using .AC/.NOISE Analyses with .TRAN	289
Measuring Total Noise	290
Using .LSTB for Loop Stability Analysis	290
Output Formats for Loop Stability (.LSTB) Analysis	291
Loop Stability Analysis Usage	292
Multiple Loop Stability Analysis	292
Single-Ended Mode Example: Ideal Inverter	293
Differential Mode Example: Bandgap	294
Controlling LSTB Warning Messages	296
Other AC Analysis Statements - .DISTO, .NET, and .SAMPLE	298
Using .DISTO for Small-Signal Distortion Analysis	298
.NET Parameter Analysis	298
Using .SAMPLE for Noise Folding Analysis	301
12. Transient Analysis	302
Simulation Flow	303
Overview of Transient Analysis	303

Contents

Example	304
Data-Driven vs. Outer Parameter Sweeps	304
Transient Analysis Output	306
Transient Control Options	307
Simulation Speed and Accuracy Using the RUNLVL Option	308
The RUNLVL Algorithm	308
RUNLVL Features	308
Guidelines for RUNLVL Setting	309
Interactions Between .OPTION RUNLVL and Other Options	309
Numerical Integration Algorithm Controls	310
TRAP	311
GEAR and Backward-Euler	311
BDF	312
Dynamic Check Using the .BIASCHK Statement	313
Limit and Noise Method	314
Maximum Method	314
Minimum Method	315
Region Method	315
Checking and Detecting Dynamic Switching, Power, IR drops, and Surge Currents	316
Using CHECK Statements	316
POWER DC Analysis	320
Power DC Analysis Output Format	321
POWER Analysis	322
Detecting and Reporting Surge Currents	323
Storing and Restoring Checkpoint Files	323
Store by Using the .STORE Command	324
Store by Interrupting the Simulation Process	325
Store by Terminating the Simulation Process	325
Restore Operation	326
Example	326
Storing and Restoring Using HSPUI	327
Usage Requirements for type=IC	327
Usage Requirements for type=SNAPSHOT	327
Specifying Monte Carlo or Temperature Sweeps	328
Golden Reference for Control Options	328
Troubleshooting: Internal Time Step, Measurement Errors	329

Troubleshooting 'Time step too small' Errors	329
How TSTEP Affects a Transient Simulation	330
Troubleshooting .MEASUREMENT Issues	331
13. Spectrum Analysis	333
Spectrum Analysis (Fourier Transform)	333
Using the Fourier-Related Statements and Options	335
Fourier Accuracy	335
Fourier Equation	335
.FFT Analysis	337
Using Windows in FFT Analysis	338
Examining the FFT Output	341
Notes:	342
Measuring FFT Output Information	344
AM Modulation	345
Graphical Output	346
Balanced Modulator and Demodulator	347
Signal Detection Test Circuit	352
Input Listing	352
Output	352
14. Pole-Zero Analysis	358
Overview of Pole-Zero Analysis	358
Using Pole-Zero Analysis	359
Matrix Approach	359
Muller Method	360
How PrimeSim HSPICE Calculates Poles and Zeros	360
Pole/Zero Analysis Examples	361
Example 1 – Low-Pass Filter	361
Example 2 – Kerwin's Circuit	364
Example 3 – High-Pass Butterworth Filter	364
Example 4 – CMOS Differential Amplifier	365
Example 5 – Simple Amplifier	367
Example 6 – Active Low-Pass Filter	368
References	373

15. MOSFET Reliability Analysis (MOSRA)	375
MOSRA Overview	375
Reliability Analysis Use Model	376
Example Setup	377
MOSRA Model Overview	377
Level 1 MOSRA BTI and HCI Model	378
Synopsys Level 1 MOSRA Model	378
Synopsys LEVEL1 MOSRA model, BTI Vth degradation	378
Mobility Degradation BTI Parameters	380
Synopsys LEVEL1 MOSRA model, HCI for NMOS and PMOS	382
Level 3 MOSRA BTI and HCI Model	384
BTI Model	384
HCI Model	386

16. Post-Layout Simulation: RC Network Reduction and Back-Annotation	388
Linear Acceleration	389
Using the SIM_LA Option	389
PACT Algorithm	390
LNE Algorithm	390
Control Options for Linear Acceleration	391
Supporting Parasitic L- and K-elements	392
Pruning Parasitics from a Post-Layout Flat Netlist	392
Using the .PRUNE Command	392
Example	393
Limitation of .PRUNE	394
Post-Layout Back-Annotation	395
Full Back-Annotation	396
Flow for Full Back-Annotation	396
Example	397
Selective Net Back-Annotation	397
Flow for Selective Net Back-Annotation	398
Example	398
Active Net Generation	399
Warnings/Error Messages	401
Listing of Back-Annotation Commands and Options	402
Back-Annotation Cases	403
Full Back-Annotation	403

	Selective Back-Annotation	409
	Recommended Settings for STAR-RC-XT Command File	411
	DSPF and SPEF File Structures	411
	DSPF File Structure	412
	SPEF File Structure	413
17.	Multi-Technology Simulation of 3D Integrated Circuit	415
18.	Simulation of Stacked Devices	416
	Stacked Devices Structure	416
	Syntax	417
	MOSFET Stacked Devices	418
	.OPTION DEV_STACK	419
	.OPTION OUT_DEV_STACK	419
	Options SSDP and SSSP	420
	Instance Parameters SSDP and SSSP of MOSFET Stacked Device	421
	Parameter DLSX for IVTH Calculation	422
	Supported Output Templates	422
	Calculation of IVTH	422
	Limitations	423
	Error Handling and Recovery	423
19.	Random Telegraph Noise Model and Simulation	424
	Random Telegraph Noise Overview	424
	Example	425
20.	Data-To-Design (D2D)	429
21.	Simulation of Thermal Effect in Macro Models	430
	Macro Model Extension to Enable Thermal Effect	430
	Syntax	430
	.SET_NODE_TYPE	430
	.OPTION ENABLE_SUBCKT_SHE	431
	SUB_SHE_TEMP()	432

Example	432
---------------	-----

Part 4: Variation, Optimization, and Statistical Analysis

22. Performing Digital Cell Characterization	434
Performing Basic Cell Measurements	435
Rise, Fall, and Delay Calculations	435
Delay versus Fanout	437
Pin Capacitance Measurement	438
Op-amp Characterization of LM124	440
Performing Advanced Cell Characterization	441
Monte Carlo Plus Data Sweep in Transient Simulations	442
MC+Data Sweep	443
MC+Data Sweep+Bisection	443
Output of Multi-Dimensional Sweeps	444
Limitations of Multi-Dimensional Sweeps	444
Cell Characterization Examples	445
 23. Timing Analysis Using Bisection	 448
Overview of Bisection	449
Bisection Methodology	451
Measurement	451
Optimization	451
Using Bisection	451
Examples	452
Examining the Command Syntax	453
Performing Transient Analyses with Bisections	455
Setup Time Analysis	455
Input Listing	455
Results	457
Minimum Pulse Width Analysis	459
Input Listing Directory	459
Results	460
Pushout Bisection Methodology	460

Example	461
Using RELOUT and RELIN to Affect PrimeSim HSPICE Bisection Optimization ..	463
.MODEL Optimization Syntax	463
RELOUT Parameter	464
RELIN Parameter	464
Example	464
Using Bisection with Monte Carlo Analysis	465
Setting Up Monte Carlo Analysis with Bisection	466
Performing Bisection with Monte Carlo Sweep	466
.TRAN Syntax for Bisection with Monte Carlo Sweep	467
Example	467
Output Files — Monte Carlo and Bisection with Monte Carlo	467
Monte Carlo Output Files	467
Output List File for Bisection with Monte Carlo Sweep	468
Output Measurement File	469
Output Waveform File	470
Bisection Optimization Application Example	470
24. Variation Analysis	472
Overview of Variation on Silicon	473
Application of Statistical Analysis	474
Analytical Model Types	475
Traditional Monte Carlo Simulations	476
Basic Syntax	477
Local and Global Parameter Variation	478
Exception for Model Parameters	478
Starting Values and Seeds	478
Other Monte Carlo Control Options	479
Worst-case Analysis	479
Model Skew Parameters	480
Using Skew Parameters	481
Skew File Interface to Device Models	483
Traditional Monte Carlo Analysis Characteristics	484
Monte Carlo Setup	485
Examples	487
Monte Carlo Output	487
.PARAM Distribution Function	487

Contents

Syntax	488
Examples	489
Monte Carlo Parameter Distribution	490
Global and Local Variations with Monte Carlo	491
Key to Demonstration Examples for Monte Carlo	491
Variation on Model Parameters as a Function of Device Geometry	497
Non-Gaussian Probability Distribution Functions	497
Traditional and Variation Block Monte Carlo	499
Simple Random Sampling and Other Sampling Methods	499
Traditional SRS Versus VB SRS	499
Random Number Generators	500
Pseudo-Random Number Generators	500
Data Mining	501
Invoking Advanced Sampling Methods	501
Monte Carlo Examples	503
Delay and Power Consumption Analysis of Inverters	503
Netlist	503
Transient Sigma Sweep Results	504
Operating-Point Results in Transient Analysis	504
Gaussian, Uniform, and Limit Functions	506
Major and Minor Distribution	507
RC Time Constant	508
Switched-Capacitor Filter Design	509
Physical Approach	510
Electrical Approach	511
Monte Carlo Results	511
Troubleshooting Monte Carlo Issues	516
Perturbation Information Missing from Output Listing in Monte Carlo and Subcircuit Local Variables	516
25. Statistical Representation Using the Variation Block	518
Overview: Monte Carlo Using the Variation Block Flow	518
Variability in PrimeSim HSPICE	520
Overview of the Variation Block	522
Variation Block Structure	523
General Section	524
Subblocks for Global, Local, and Spatial Variations	524
Independent Random Variables	525

Contents

Dependent Random Variables	527
Absolute Versus Relative Variation	529
Variations on Model Parameters	530
Variations on Subcircuit Parameters	531
Variations on Top-Level Parameters	532
Variations on Temperature	533
Access Functions, Get_E(), Get_P(), Get_M(), Get_O()	533
Variations of Element Parameters	535
Spatial Variation	539
Special Rules Regarding Variation Block Usage	539
Variation Block Examples	539
Simple VB Example with Global and Local Variations	540
Principal Component-based Global Variation Modeling	540
Local Variation Example for Submicron Technology	541
Spatial Variation Example	541
Group Operator {...} and Subexpressions	541
Syntax	542
Syntax Extension with Bins	542
Example	542
Rules for Using the Group Operator	543
Syntax and Examples	544
Model Card in the Header Section	544
Syntax	544
Example	544
Parasitic Section	544
Examples	545
Control Options and Syntax	545
References	548
26. Monte Carlo Analysis—Flow and Outputs	550
Monte Carlo Analysis in PrimeSim HSPICE	551
Input Syntax	553
DC Sweep Examples	553
Options for Monte Carlo Analysis	553
Output for Variation Block Monte Carlo	561
Simulation Listing	561
Measurement Output File	561
Parameter File	562

Contents

Sampling Options	566
Simple Random Sampling (SRS)	566
One-Factor-at-a-Time (OFAT) Sampling	567
Factorial Sampling	568
Latin Hypercube Sampling (LHS)	568
Monte Carlo with firstrun and list Options	569
Sobol and Niederreiter Sampling (LDS)	570
External Sampling	571
Usage Model for External Sampling	572
Syntax	572
Controlling the Read-in of an External File	573
Using the MC_File_Only Option	573
Allowing Zero Variations in External Sampling for Missing IRVs	574
Monte Carlo	575
Comparison of Sampling Methods	575
Gaussian Style Random Variable Definition	584
Example 1	585
Example 2	585
Input/Output With New Capability	585
Example 1: Variation Duplicated in Traditional Format and Variation Block ...	587
Example 2: Subcircuit and Macro Models	589
Application Considerations	593
Output Files for Monte Carlo Simulations	593
Content in *.mpp0 File	594
Post-Processing of Monte Carlo Results	596
Summary Statistics	597
Variable Screening	608
Corner Analysis - DC Monte Carlo/Transient Analysis	613
Back-Annotation — *.annotate(.ava.annotate) File	615
The Tail Samples File - .ava.details.json	615
More on Debugging - .ava.dsvc.json, .ava.critical and .ava.critical.m(s/d/t)0 ..	615
Standalone Data Mining in PrimeSim HSPICE	616
Post Processing Utility in PrimeSim HSPICE	616
Incremental Monte Carlo Simulation in Standalone Data Mining Mode	617
CFG File Configuration for Data Mining in DP Mode	618
Large Scale Monte Carlo Simulation – Traditional and Variation Block	619
Automatic Incremental DP Monte Carlo	621
Modifying the Built-in Distribution for Monte Carlo Analysis	622

Contents

Syntax for Modifying the Distribution	622
Yield Computation in Monte Carlo Analysis	623
Syntax for Yield Specification	625
Yield Specification for Standalone Data Mining	626
Input Example for Yield Analysis	626
Output of Yield Analysis	628
Sigma Amplification MC and Outputs	630
Activating Sigma Amplification MC	631
Determining the Amplification Factor	632
Sigma Amplification MC Outputs	633
Q-Q Plot Files in PDF and WaveView Formats	634
The *.amp_mpp0 File	636
Frequently-Asked Questions	637
Defining Correlated Normal Distributed Random Variables in Monte Carlo Simulation	638
Output	641
External Sampling	641
Troubleshooting Monte Carlo-VB Issues	641
Independent Random Variable Assignments	641
References	643
27. Mismatch Analyses	644
Mismatch	645
DCMatch Analysis	645
Input Syntax	646
Examples	647
Example 1	647
Example 2	647
DCMatch Table Output	648
Example: Simple Op-Amp and DCMatch Output Table Described	649
PrimeSim HSPICE Ignores Limit Distributions in DCMatch	651
Output Using .PROBE and .MEASURE Commands	651
Syntax for .PROBE Command for DCMatch	652
Syntax for .MEASURE Command	652
DCMatch Example Netlist	653
DC Sensitivity Analysis	654
Sensitivity Block Using the Variation Block Construct	655

Contents

Sensitivity to Local Variations	655
Sensitivity to Global Variations	655
Input Syntax	655
ACMatch Analysis	656
Parasitic Capacitor Sensitivity	657
Input Syntax	657
Examples	658
ACMatch Table Output	658
Example 1: Parasitic Capacitance Sensitivity	659
Example 2: Sensitivity Not Calculated	660
Output from .PROBE and .MEASURE Commands for ACMatch	661
Syntax for .PROBE Command	661
Syntax for .MEASURE Command	662
ACMatch Analysis Examples	662
Example 1	662
Example 2	663
Application Considerations	664
Mismatch Compared to Monte Carlo Analysis	665
Supported Models with DCMatch and ACMatch	665
References	665
28. Exploration Block	666
Exploration Block Functions	666
Usage Guidelines	667
Multiple Instantiations of the Same Cell or Subcircuit	667
Specifying Relationships between Devices	668
Examples	668
Specifying Relationships between Device Properties	668
Example	668
Subcircuits and Elements Supported for Exploration	668
Examples	669
Flow Using an External Exploration Tool	670
Information Extraction and Export Phase	670
Definition Phase	670
Exploration Phase	671
Exploration Block Structure	672
Syntax	673

Contents

Control Options	674
Example: Option secondary_param=yes	675
Parameters Section	676
Device Relationships	677
Property Relationships	677
Get_E() Function In Exploration Block	678
Parameters Defined Outside the Exploration Block	679
Area Measurement	679
Syntax for Device Area Calculation	679
Rules for Area Measurement using the design_area Keyword	680
Specifying Constraints	681
The Processing of Netlist Parameters	683
Export File Syntax	683
Syntax Structure	684
Example of an Export File	685
Execution of Exploration in PrimeSim HSPICE	685
Exploration Data Block Syntax	686
Exploration and Variation Block Interactions	686
Limitations	687
29. Optimization	688
Overview	688
Optimization Control	689
Simulation Accuracy	690
Curve Fit Optimization	690
Goal Optimization	690
Timing Analysis	691
Optimization Statements	691
Syntax for Optimizing Analysis (.DC, .TRAN, .AC)	692
Optimization Examples	693
MOS Level 3 Model DC Optimization	693
Input Netlist File for Level 3 Model DC Optimization	694
Results	695
MOS Level 13 Model DC Optimization	696
DC Optimization Input Netlist File for Level 13 Model	696
RC Network Optimization	696
Optimization Results	697
Optimized Parameters OPTRC	698

Optimizing CMOS Tristate Buffer	700
Input Netlist File to Optimize a CMOS Tristate Buffer	701
BJT S-parameters Optimization	703
Key PrimeSim HSPICE Features Used	704
Input Netlist File for Optimizing BJT S-parameters	704
Optimization Results	704
BJT Model DC Optimization	705
BJT Model DC Optimization Input Netlist File	706
Results	706
Optimizing GaAsFET Model DC	707
GaAsFET Model DC Optimization Input Netlist File	708
Results	708
Optimizing MOS Op-amp	708
MOS Op-amp Optimization Input Netlist File	709
Results	710
30. Model Parameter Optimization	711
Overview	711
Examples	712
Example 1: IV Data Fitting	712
MPO Input Files	713
Example 2: Measurement Data Fitting	716
Input File	717
Part 5: Encryption and Error and Warning Messages	
31. Library and Data Encryption	720
32. Warning/Error Messages	721
Warning Messages	721
Topology Warnings	722
Topology Integrity	722
No DC Path to Ground	722
Duplicate Initialization	723
Model Warnings	723
Zero or Negative Conductance	723
Encryption-Related Warnings	723

Contents

Model Binning Warnings	724
Key Model Parameter Checking	724
Parameter Expression Warning	724
Control Option Warnings	724
ACCURATE	724
GMIN, GMINDC	724
Device Warnings	724
Device Geometry Check	724
Device Parameter Check	725
Analysis Warnings	725
Transient	725
Bisection	725
Measure	725
.DC and .OP Analysis Warnings	726
Error Messages	727
Topology Errors	727
Model Errors	728
Analysis Errors	728
.DC and Operating Convergence Errors	728
Convergence Termination Criteria	729
Non-Convergence: Possible Causes	729
DC/OP Convergence Aids	730
DC Bias Point Convergence Actions	730
DC Bias Point Troubleshooting with .NODESET and .IC	730
Troubleshooting Model-Related DC Bias Point Issues	731
Impact of Conductance on Convergence	731
Transient Analysis Errors and Solutions	734
Transient Analysis Error	734
Transient Non-Convergence	734
Transient Convergence Aids	735
Safe Operating Area (SOA) Warnings	736
MOSFET / JFET	737
SOI MOSFET (LEVEL 70)	739
BJT	748
Diodes	749
Resistor	749
Capacitor	750
Verilog-A (pVA) Messages	750
Warning Message Index [00001-11146]	751

Contents

Error Message Index [20001-20084] 764

Exit Codes769

About the PrimeSim HSPICE User Guide: Basic Simulation and Analysis

The *PrimeSim HSPICE User Guide: Basic Simulation and Analysis* describes how to use Synopsys PrimeSim HSPICE® to simulate and analyze your circuit designs.

This preface includes the following sections:

- [New in This Release](#)
- [Related Products, Publications, and Trademarks](#)
- [Conventions](#)
- [Customer Support](#)
- [Statement on Inclusivity and Diversity](#)

New in This Release

Information about new features, enhancements, changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in the Release Notes on the SolvNetPlus site.

Related Products, Publications, and Trademarks

For additional information about the PrimeSim HSPICE simulator, see the documentation on the Synopsys SolvNetPlus support site at the following address:

<https://solvnetplus.synopsys.com>

You might also want to see the documentation for the following related Synopsys products:

Synopsys PrimeWave™ Design Environment

Synopsys PrimeSim™ XA

Synopsys PrimeSim™

Synopsys WaveView™

This manual is organized into the following Parts:

- [Introduction to PrimeSim HSPICE](#)
- [Parameters, Functions, and Output](#)
- [Analyses and Simulation](#)
- [Variation, Optimization, and Statistical Analysis](#)
- [Encryption and Error and Warning Messages](#)

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
Courier	Indicates syntax, such as <code>write_file</code> .
<i>Courier italic</i>	Indicates a user-defined value in syntax, such as <code>write_file design_list</code>
Courier bold	Indicates user input—text you type verbatim—in examples, such as <code>prompt> write_file top</code>
Purple	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as <code>include_enclosing = true false</code>
[]	Denotes optional arguments in syntax, such as <code>write_file [-format fmt]</code>
...	Indicates that arguments can be repeated as many times as needed, such as <code>pin1 pin2 ... pinN</code> .
	Indicates a choice among alternatives, such as <code>low medium high</code>
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy .

Convention	Description
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

<https://solvnetplus.synopsys.com>

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to <https://solvnetplus.synopsys.com>.

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

Part 1: Introduction to PrimeSim HSPICE

Describes PrimeSim HSPICE features and the simulation process.

Part 1 presents the following topics:

- [Chapter 1, PrimeSim HSPICE Overview](#)
- [Chapter 2, Setup](#)
- [Chapter 3, Startup and Simulation](#)
- [Chapter 4, Distributed Processing and Multithreading](#)
- [Chapter 5, Using Interactive Mode](#)
- [Chapter 6, PrimeSim HSPICE GUI for Windows](#)
- [Chapter 7, Input Netlist and Data Entry](#)

1

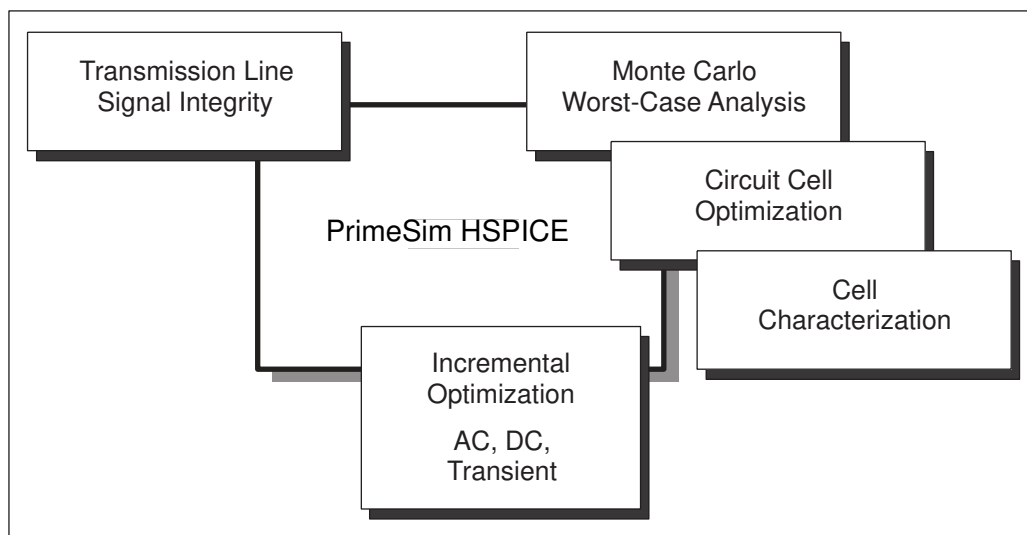
PrimeSim HSPICE Overview

Describes PrimeSim HSPICE features and the simulation process.

PrimeSim HSPICE is an optimizing analog circuit simulator. You can use it to simulate electrical circuits in steady-state, transient, and frequency domains.

PrimeSim HSPICE is unequalled for fast, accurate circuit and behavioral simulation. It facilitates circuit-level analysis of performance and yield by using Monte Carlo, worst-case, parametric sweep, and data-table sweep analyses, and employs the most reliable automatic-convergence capability (see [Figure 1](#)).

Figure 1 *Synopsys PrimeSim HSPICE Design Features*



PrimeSim HSPICE forms the cornerstone of a suite of Synopsys tools and services that allow accurate calibration of logic and circuit model libraries to actual silicon performance.

Only memory can limit the size of the circuits that PrimeSim HSPICE can simulate. PrimeSim HSPICE can address a maximum of 4GB memory on UNIX/Linux depending on your system. On Windows, PrimeSim HSPICE normally can address a maximum of 2GB memory or with Windows large memory mode enabled, a maximum of 3GB memory. For details, consult with Microsoft regarding application memory limitation on Windows.

For a description of commands and options that you can include in your PrimeSim HSPICE netlist, see PrimeSim HSPICE Netlist Commands and PrimeSim HSPICE Netlist Simulation Control Options, in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

PrimeSim HSPICE ships hundreds of examples for your use; see Listing of Demonstration Input Files in the *PrimeSim HSPICE User Guide: Demonstration Netlists* for paths to the demo files.

The following sections discuss these topics:

- [Features](#)
- [PrimeSim HSPICE Features for Running Higher-Level Simulations](#)
- [Simulation Structure](#)
- [Parser Syntax Requirements \(Unsupported Formats\)](#)
- [Use of Example Syntax](#)
- [Recommended PrimeSim HSPICE Resources](#)

Features

PrimeSim HSPICE is compatible with most SPICE variations and provides the following additional features:

- Superior convergence
- Accurate modeling, including many foundry models
- Hierarchical node names and reference
- Circuit optimization for models and cells with incremental or simultaneous multiparameter optimizations in AC, DC, and transient simulations
- Interpreted Monte Carlo and worst-case design support
- Input, output, and behavioral algebraics for cells with parameters
- Cell characterization tools to characterize standard cell libraries
- Geometric lossy-coupled transmission lines for PCB, multichip, package, and IC technologies
- Discrete component, pin, package, and vendor IC libraries
- Interactive graphing and analysis of multiple simulation waveforms with waveform viewers such as WaveView

- Flexible license manager that allocates licenses intelligently based on run status and user-specified job priorities

If you suspend a simulation job (Ctrl-Z), the load sharing facility (LSF) license manager signals PrimeSim HSPICE to release that job's license. This frees the license for another simulation job, or the stopped job can reclaim the license and resume.

You can also prioritize the simulation jobs you submit; LSF automatically suspends low-priority simulation jobs to run high-priority jobs. When the high-priority job completes, LSF releases the license back to the lower-priority job, which resumes at the point of suspension. To resume the LSF job on the same terminal, type either **fg** or **bg**.

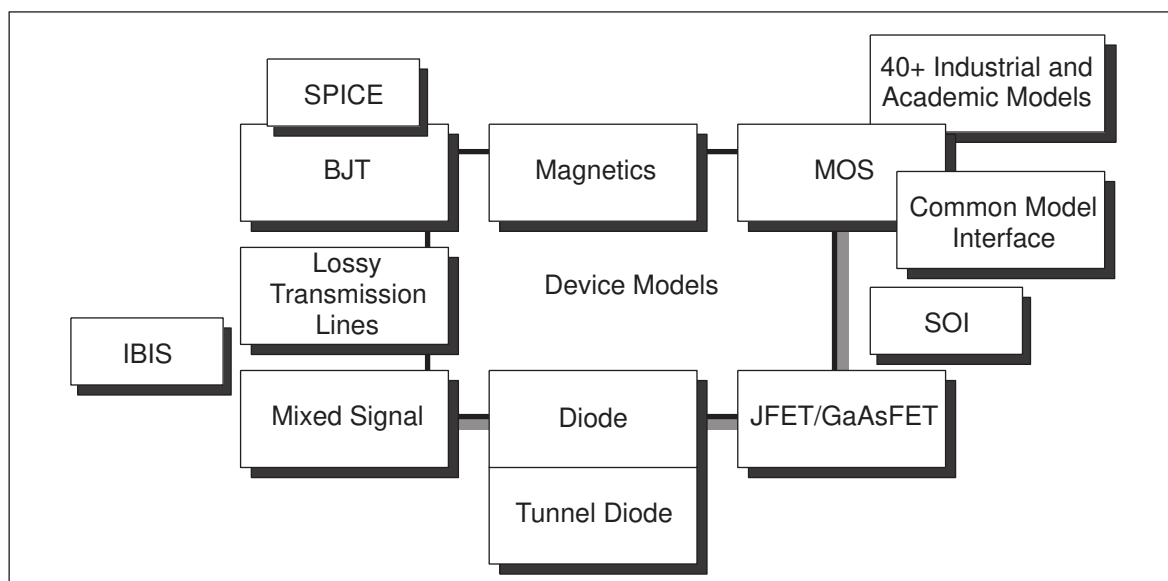
- A number of circuit analysis types and device modeling technologies. See [Figure 2](#)
- Support for the Compiled Function Library (CFL) function.

This enables dynamic linking to PrimeSim HSPICE during run time through a built-in mathematical function or user-defined function written in CFL. You can include multiple CFL functions in each library.

A general CFL function input argument can come from a predefined parameter value, a mathematical expression of multiple predefined parameter values, a built-in mathematical function in the standard library, or an output of another evaluated CFL function. The CFL function allows users to initialize a data structure and return its address as an input argument of another CFL function. CFL is a static function and is usable only for parameter evaluations.

See [Compiled Function Library Environment Variable](#) to set the required environment variable and file path. See also .OPTION CFLFLAG in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

Figure 2 *Synopsys PrimeSim HSPICE Modeling Technologies*



The following sections discuss these topics:

- [Custom CMI](#)
- [TSMC Model Interface](#)
- [Si2 Compact Model Coalition Model Interface - OMI](#)

Custom CMI

PrimeSim HSPICE provides the ability to integrate models with the Custom Common Model Interface (CMI) for which PrimeSim HSPICE uses a dynamically linked shared library. Consult your PrimeSim HSPICE technical support team for access to the PrimeSim HSPICE CMI API application note and source code.

TSMC Model Interface

PrimeSim HSPICE provides the ability to invoke the TSMC Model Interface (TMI) flow using proprietary TSMC model files and compiled libraries. Jointly developed by Synopsys and TSMC, the TMI technology and API provides a compact model with additional instance parameters and equations for an advanced modeling approach. The approach

supports TSMC's extension of the standard CMC core model (such as BSIM4) with additional instance parameters and equations for advanced process technology.

- Because Modeling API code is in C, it is available in a compiled format for PrimeSim HSPICE and Synopsys FastSPICE products to link to during the simulation. TMI settings to invoke the flow and the location of a `.so` file require TSMC model libraries. Use `.OPTION TMIPATH` and `.OPTION TMIFLAG` to access these libraries.

The simulators enable automatic platform selection on the `.so` file. Both PrimeSim HSPICE and the Synopsys FastSPICE products provide the tool binaries and support the same `*.so` file.

Use the existing PrimeSim HSPICE and FastSPICE commands to run the simulation.

The API also performs automatic platform selection on the `.so` file. Both PrimeSim HSPICE and HSPICE provide the tool binaries and support the same `*.so` file.

Use the existing PrimeSim HSPICE and HSPICE commands to run the simulation.

- PrimeSim HSPICE supports hybrid simulation with both TMI models and non-TMI models.
- A TMI model parameter, `tmimodel` (available in TSMC's SPICE model parameter libraries), enables you to switch between the built-in model only and the TMI model:
 - `tmimodel = 1` turns on the TMI model
 - `tmimodel = 0` (default) turns on the native/built-in models without calling TMI

For further information, contact Synopsys Technical Support.

See also the *PrimeSim HSPICE Reference Manual: Commands and Control Options* for `.OPTION TMIFLAG`.

Si2 Compact Model Coalition Model Interface - OMI

PrimeSim HSPICE provides the ability to invoke simulation using custom model implemented in the OMI model interface. OMI model usage is defined in Si2 Compact Model Coalition OMI Functional Specification and foundry or IDMs OMI model usage guide.

- Since Modeling API code is in C, it is available in a compiled format for PrimeSim HSPICE and Synopsys FastSPICE products to link it during the simulation. Use `.OPTION OMIPATH` and `.OPTION OMIFLAG` to access these libraries.
- Use the existing PrimeSim HSPICE and FastSPICE commands to run the simulation.
- PrimeSim HSPICE supports hybrid simulation with both OMI models and non-OMI models.

- The OMI model parameter `omimodel` enables users to switch between the built-in model only or the OMI model:
 - `omimodel = 1` turns on the OMI model
 - `omimodel = 0` (default) turns on the native/built-in models without calling OMI

For more information, contact Synopsys Technical Support. Also, see the *PrimeSim HSPICE Reference Manual: Commands and Control Options* for `.OPTION OMIFLAG`, `.OPTION TMIFLAG` and `.OPTION OMIPATH`.

PrimeSim HSPICE Features for Running Higher-Level Simulations

Simulations at the integrated circuit level and at the system level require careful planning of the organization and interaction between transistor models and subcircuits. Methods that work for small circuits may have too many limitations for higher-level simulations.

You can use the following PrimeSim HSPICE features to organize how simulation circuits and models run:

- Explicit include files – `.INCLUDE` statement
- Implicit include files – `.OPTION SEARCH='lib_directory'`
- Algebraics and parameters for devices and models – `.PARAM` statement
- Parameter library files – `.LIB` statement
- Automatic model selector – `LMIN`, `LMAX`, `WMIN`, and `WMAX` model parameters
- Parameter sweep – sweep analysis statements
- Statistical analysis – sweep monte analysis statements
- Multiple alternative – `.ALTER` statement
- Automatic measurements – `.MEASURE` statement
- Condition-controlled netlists (`IF-ELSEIF-ELSE-ENDIF` statements in PrimeSim HSPICE only)

Simulation Structure

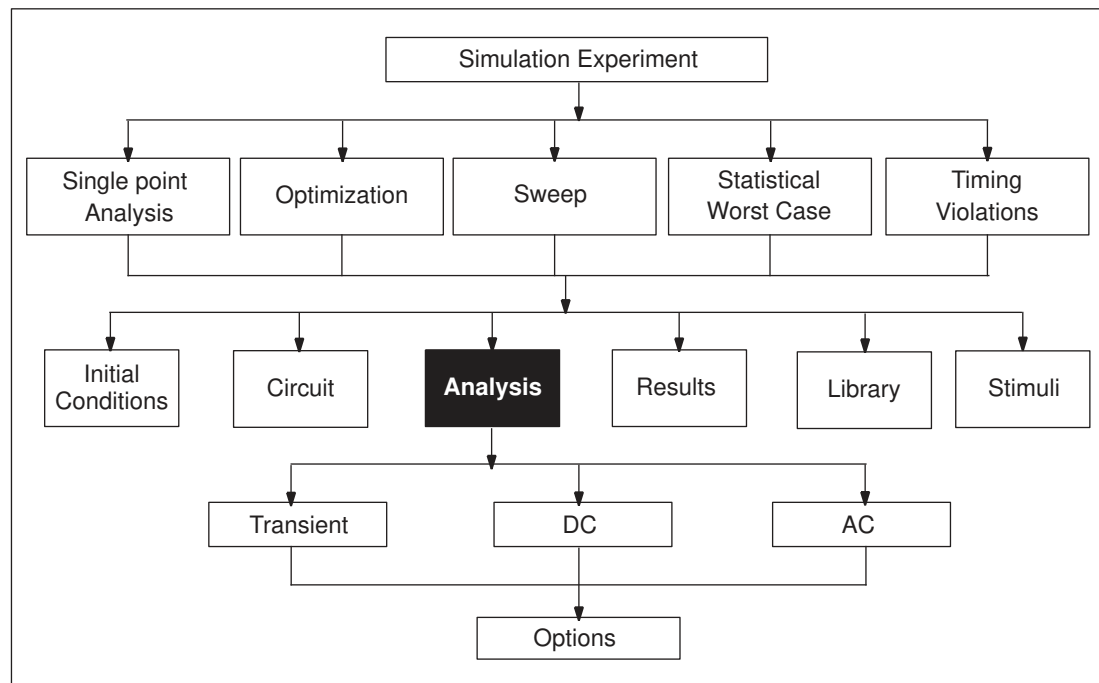
The following sections discuss these topics:

- [Experimental Methods Supported by PrimeSim HSPICE](#)
- [PrimeSim HSPICE Basic Analysis Types](#)
- [Measurement System in PrimeSim HSPICE](#)
- [Simulation Process Overview](#)

Experimental Methods Supported by PrimeSim HSPICE

Typically, you use experiments to analyze and verify complex designs. These experiments can be simple sweeps, more complex Monte Carlo and optimization analyses, or setup and hold violation analyses of DC, AC, and transient conditions.

Figure 3 Simulation Program Structure



For each simulation experiment, you must specify tolerances and limits to achieve the desired goals, such as optimizing or centering a design. Common factors for each experiment are:

- Process
- Voltage
- Temperature
- Parasitics

PrimeSim HSPICE supports two experimental methods:

- Single point – simple procedure that produces a single result, or a single set of output data
- Multipoint – Performs an analysis (single point) sweep for each value in an outer loop (multipoint) sweep

The following are examples of multipoint experiments:

- Process variation – Monte Carlo or worst-case model parameter variation
- Element variation – Monte Carlo or element parameter sweeps
- Voltage variation – VCC, VDD, or substrate supply variation
- Temperature variation – Design temperature sensitivity
- Timing analysis – Basic timing, jitter, and signal integrity analysis
- Parameter optimization – Balancing complex constraints, such as speed versus power, or frequency versus slew rate versus offset (analog circuits)

PrimeSim HSPICE Basic Analysis Types

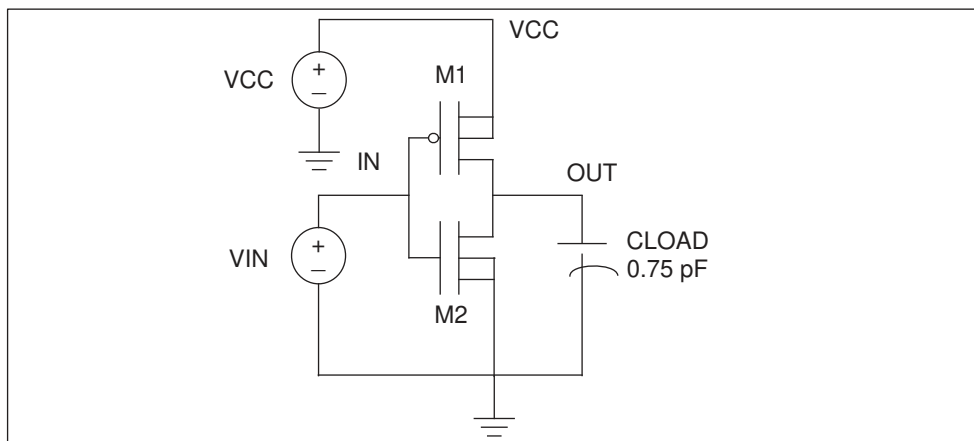
The following sections discuss these topics.

- [DC Analysis of an Inverter](#)
- [AC Analysis of an RC Network](#)
- [Transient Analysis of an RC Network](#)
- [Transient Analysis of an Inverter](#)

DC Analysis of an Inverter

You can analyze the DC behavior of the simple MOS inverter shown in the following figure:

Figure 4 MOS Inverter Circuit



To analyze the DC behavior, perform the following steps:

1. Type the following netlist data into a file named `quickDC.sp`.

```
Inverter Circuit
.OPTION POST
.DC VIN 0 5 0.1
.PRINT DC V(IN) V(OUT)
M1 OUT IN VCC VCC PCH L=1U W=20U
M2 OUT IN 0 0 NCH L=1U W=20U
VCC VCC 0 5
VIN IN 0 0 PULSE .2 4.8 2N 1N 1N 5N 20N
CLOAD OUT 0 .75P
.MODEL PCH PMOS LEVEL=1
.MODEL NCH NMOS LEVEL=1
.END
```

You can find the complete netlist for this example in `$installdir/demo/hspice/apps/quickDC.sp`.

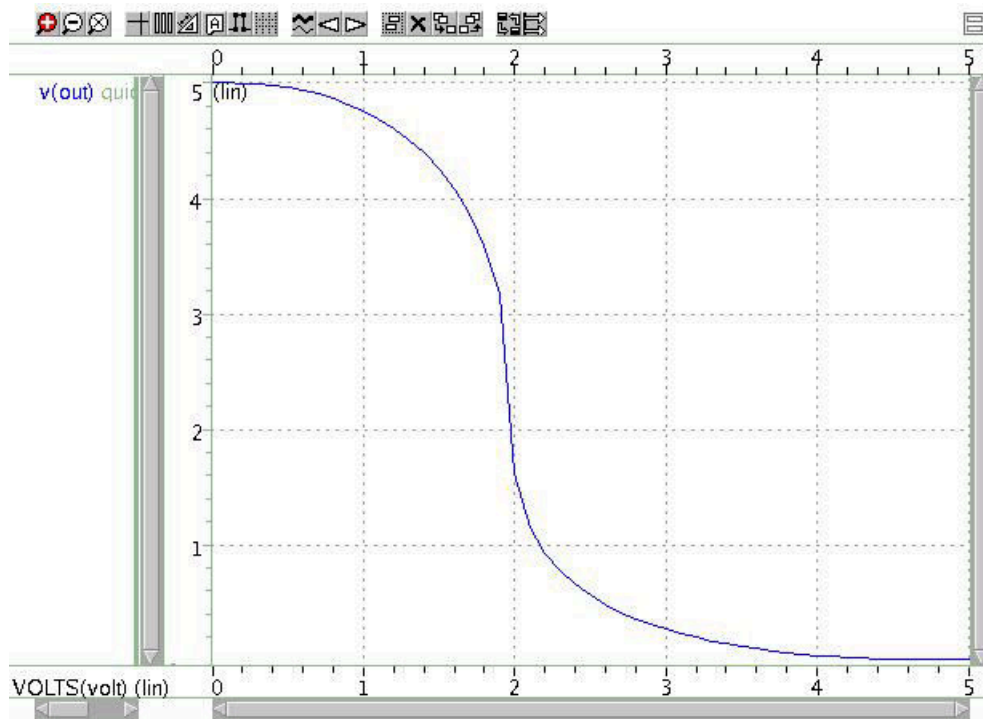
2. Run the PrimeSim HSPICE analysis by typing the following command:

```
hspice -i quickDC.sp -o quickDC.lis
```

3. Use WaveView to examine the voltage waveform at the inverter OUT node.

[Figure 5 on page 41](#) shows the waveforms.

Figure 5 Voltage at Inverter Node v(out)

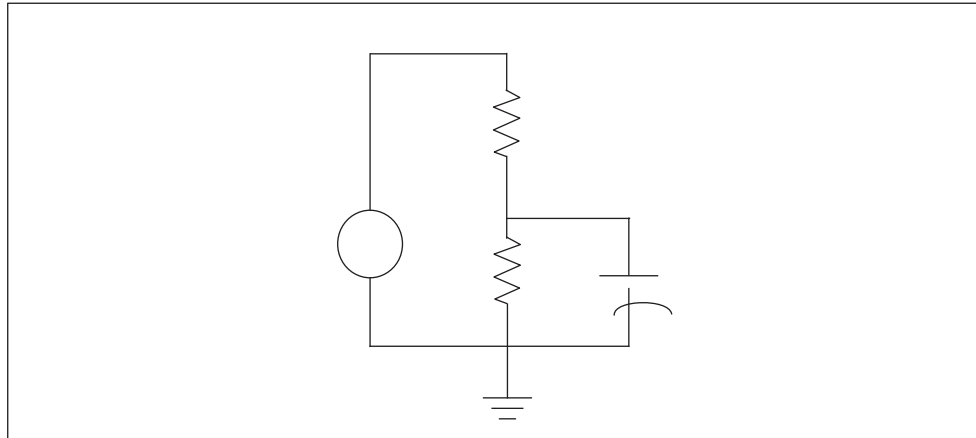


AC Analysis of an RC Network

[Figure 6 on page 42](#) shows a simple RC network with a DC and AC source applied. The circuit consists of:

- Two resistors, R1 and R2.
- Capacitor C1.
- Voltage source V1.
- Node 1, the connection between the source positive terminal and R1.
- Node 2, where R1, R2, and C1 are connected.
- PrimeSim HSPICE ground (always node 0).

Figure 6



The netlist for this RC network is based on demonstration netlist `quickAC.sp`, which is available in directory `$installdir/demo/hspice/apps`:

```
A SIMPLE AC RUN
.OPTION LIST NODE POST
.OP
.AC DEC 10 1K 1MEG
.PRINT AC V(1) V(2) I(R2) I(C1)
V1 1 0 10 AC 1
R1 1 2 1K
R2 2 0 1K
C1 2 0 .001U
.END
```

To perform AC analysis for an RC network circuit, perform the following steps:

1. Type the above netlist into a file named `quickAC.sp`.
2. To run a PrimeSim HSPICE analysis, type:

```
hspice quickAC.sp > quickAC.lis
```

When the run finishes, PrimeSim HSPICE displays the following message:

```
>info:      ***** hspice job concluded
```

This is followed by a line that shows the amount of real time, user time, and system time needed for the analysis.

Your run directory includes the following new files:

- quickAC.ac0
- quickAC.ic0
- quickAC.lis
- quickAC.st0

3. Use an editor to view the .lis and .st0 files to examine the simulation results and status.
4. Run WaveView.
5. From the **File** menu, select **File > Import > Waveform File**.
6. Select the quickAC.ac0 file from the Open: Waveform Files window.
7. Display the voltage at node 2 by using a log scale on the x-axis.

Figure 7 on page 43 shows the waveform that PrimeSim HSPICE produces if you sweep the response of node 2, while varying the frequency of the input from 1 kHz to 1 MHz.

Figure 7 RC Network Node 2 Frequency Response



As you sweep the input from 1 kHz to 1 MHz, the `quickAC.lis` file displays:

- Input netlist.
- Details about the elements and topology.
- Operating point information.
- Table of requested data.

The `quickAC.ic0` file contains information about DC operating point conditions. The `quickAC.st0` file contains information about the simulation run status.

To use the operating point conditions for subsequent simulation runs, execute the `.LOAD` statement.

Transient Analysis of an RC Network

To run a transient analysis of a RC network with a pulse source, a DC source, and an AC source, perform the following steps:

1. Type the following netlist into a file named `quickTRAN.sp`.

```
A SIMPLE TRANSIENT RUN
.OPTION LIST NODE POST
.OP
.TRAN 10N 2U
.PRINT TRAN V(1) V(2) I(R2) I(C1)
V1 1 0 10 AC 1 PULSE 0 5 10N 20N 20N 500N 2U
R1 1 2 1K
R2 2 0 1K
C1 2 0 .001U
.END
```

This example uses demonstration netlist `quickTRAN.sp`, which is available in directory `$installdir/demo/hspice/apps`.

Note:

The V1 source specification includes a pulse source. For the syntax of pulse sources and other types of sources, see Sources and Stimuli in the *PrimeSim HSPICE User Guide: Elements*.

2. To run PrimeSim HSPICE, type the following:

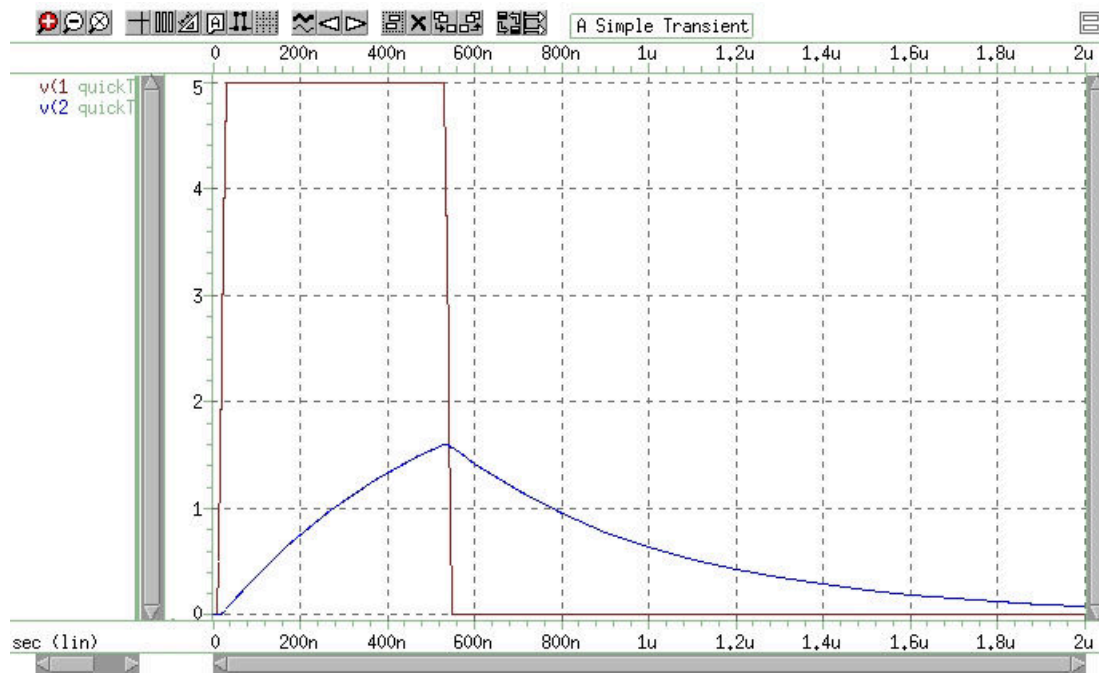
```
hspice quickTRAN.sp > quickTRAN.lis
```

3. To examine the simulation results and status, use an editor and view the `.lis` and `.st0` files.
4. Run WaveView and open the `.sp` file.
5. From the File menu, select **File > Import Waveform > File**.

6. Select the `quickTRAN.tr0` file from the Open: Waveform Files window.
7. Display the voltage at nodes 1 and 2 on the x-axis.

Figure 8 shows the waveforms.

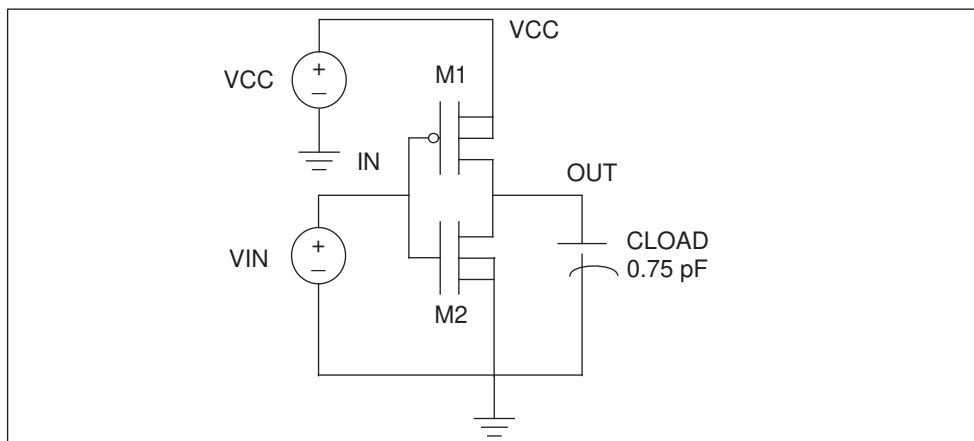
Figure 8 Voltages at RC Network Circuit Node 1 and Node 2



Transient Analysis of an Inverter

As a final example, you can analyze the transient behavior of the simple MOS inverter shown in Figure 9.

Figure 9 MOS Inverter Circuit



To analyze the transient behavior of an inverter, perform the following steps:

1. Type the following netlist data into a file named `quickINV.sp`.

```
Inverter Circuit
.OPTION LIST NODE POST
.TRAN 200P 20N
.PRINT TRAN V(IN) V(OUT)
M1 OUT IN VCC VCC PCH L=1U W=20U
M2 OUT IN 0 0 NCH L=1U W=20U
VCC VCC 0 5
VIN IN 0 0 PULSE .2 4.8 2N 1N 1N 5N 20N
CLOAD OUT 0 .75P
.MODEL PCH PMOS LEVEL=1
.MODEL NCH NMOS LEVEL=1
.END
```

You can find the complete netlist for this example in directory `$installdir/demo/hspice/apps/quickINV.sp`.

2. To run PrimeSim HSPICE, type the following:

```
hspice quickINV.sp > quickINV.lis
```

3. Use WaveView to examine the voltage waveforms, at the inverter IN and OUT nodes.

[Figure 10 on page 47](#) shows the waveforms.

Figure 10 Voltage at MOS Inverter Nodes $v(in)$ and $v(out)$



Measurement System in PrimeSim HSPICE

In this manual, the measurements always refer to MKS units (meter, kilogram, second measurement), unless otherwise stated. PrimeSim HSPICE expects length and width to be represented in meters. However, PrimeSim HSPICE does directly support units of “mil” (0.001inch, 25.4e-06 meters) as input.

This example defines a transmission line with a length of 0.4 inches:

```
T1 IN 0 OUT 0 Z0=50 f=1meg L=400mil
```

To get the results you expect, use caution when mixing units of measure.

For reference, some other “m” units can become confusing. Mega, sometimes expressed as “meg” or “x”, may look like “m” (milli):

1m = 1e-3 (milli)

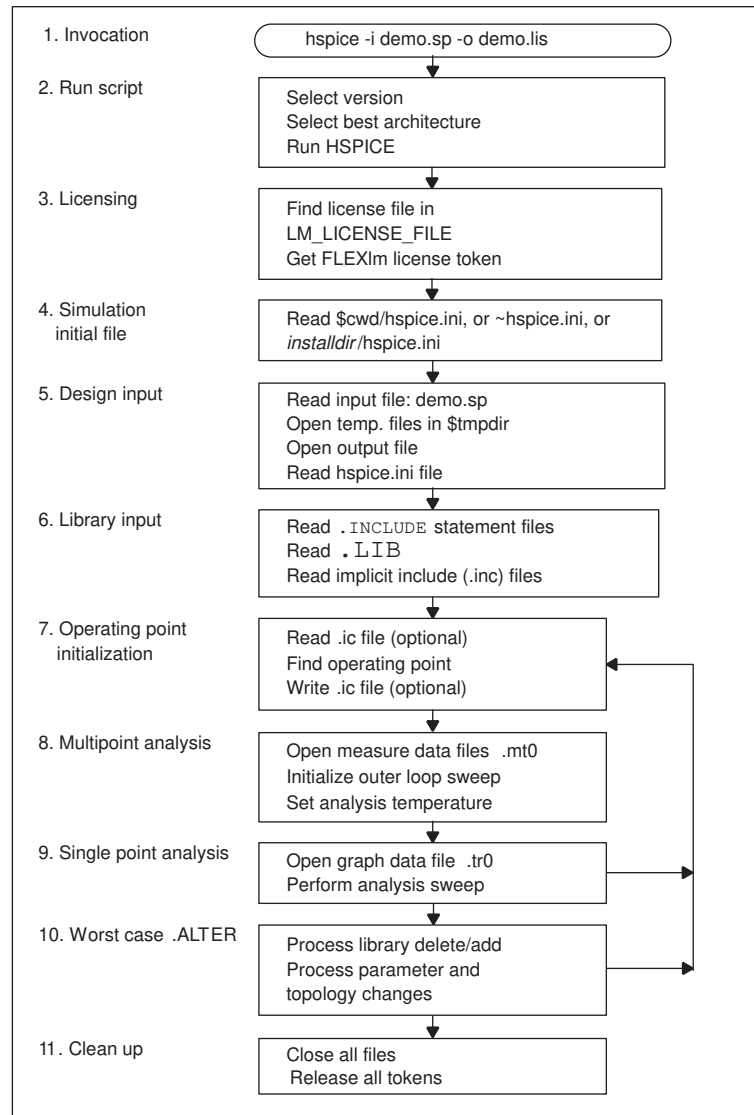
1meg = 1x = 1e6 (mega)

1u = 1e-6 (micro)

Simulation Process Overview

Figure 11 shows the PrimeSim HSPICE simulation process.

Figure 11 Simulation Process



Parser Syntax Requirements (Unsupported Formats)

The following table defines the unsupported formats:

	Unsupported Format/Rule	Examples	Result
1.	Unsupported format/syntax in any line	<code>".temp 55"</code>	Error message issued
2.	Unbalanced brackets, quotes	<code>+ P_CGDL'</code>	Error message issued
3.	Unbalanced brackets, quotes in one line	<code>.probe v(1</code>	Warning message issued
4.	In model definition, model parameter value written with quotes instead of equal sign ("a=20" instead of a=20)	<code>+ "XTIS = 3"</code>	Error message issued
5.	Strictly following brackets (parentheses)/equals wherever necessary	<code>.measure dc VT find + par('v(2)*pnorp') + when I1(MAIN) pspvtl VSOURCE in 0 PWL TD= + 10ps R 0ps</code>	Error message issued (should be:) <code>I1(MAIN)=pspvtl (should be:) VSOURCE in 0 PWL + TD = 10ps R=0ps</code>
6.	Incomplete model parameter definition in model	<code>XW= or + CGDL =</code>	Error message issued
7.	Entry name must exactly match in both the .lib call and the .lib definition	<code>.lib 'typical.lib' 'slow' or .lib 'typical.lib' slow ...matches: .lib slow</code>	<code>slow=slow 'slow'='slow' otherwise, error issued</code>

See [Reserved Keywords on page 150](#) for additional information.

Use of Example Syntax

To copy and paste proven syntax, use the demonstration files shipped with your installation of PrimeSim HSPICE (see Listing of Demonstration Input Files in the *PrimeSim HSPICE User Guide: Demonstration Netlists*). Do not attempt to copy and paste from the documentation as this can produce unexpected results. Documentation often includes hidden characters, white space, and other formatting elements for visual clarity.

Recommended PrimeSim HSPICE Resources

In addition to the PrimeSim HSPICE product documentation, you can access the following resources:

- [Synopsys SolvNetPlus](#)
- [PrimeSim HSPICE Product Website](#)
- [Analog Insights Blog](#)
- [SNUG Presentations and Papers](#)
- [PrimeSim HSPICE Demonstration Netlists](#)

Synopsys SolvNetPlus

A wealth of useful information relating to PrimeSim HSPICE in the form of application notes, articles, troubleshooting, workarounds, recorded trainings, white papers, and time-saving scripts is available through Synopsys SolvNetPlus (<https://solvnetplus.synopsys.com>).

If you do not already have a SolvNetPlus account, sign up for one, and enter “PrimeSim HSPICE” to search for supplementary information, from getting started to in-depth articles of Synopsys User Group (SNUG) presentations.

PrimeSim HSPICE Product Website

The PrimeSim HSPICE website <http://www.PrimeSim HSPICE.com> includes presentations and mini-demos of recent PrimeSim HSPICE features.

This website provides links that you can use to:

- Access PrimeSim HSPICE white papers
- Download a library of PrimeSim HSPICE compatible Verilog-A modules
- Sign up for PrimeSim HSPICE feature webinars
- Learn about product feature including integration with Custom Compiler

Analog Insights Blog

Synopsys presents valuable technical discussions in this interactive blog site at: <https://blogs.synopsys.com/analoginsights/>.

You can read and participate in discussions, use model tips and follow links to other useful sites. This blog is a lively collection of recent posts, Q&As, and reports from the field moderated by the Staff Technical Marketing Manager for circuit simulation products.

SNUG Presentations and Papers

Synopsys User Group events are held in multiple cities and countries each year. The proceedings include tutorials, presentations, and papers written by PrimeSim HSPICE product CAEs, ACs, and customers. See [https://www.synopsys.com/Community/SNUG/Pages/snugresults.aspx?qry=PrimeSim HSPICE](https://www.synopsys.com/Community/SNUG/Pages/snugresults.aspx?qry=PrimeSim%20HSPICE).

Articles and presentations require a SolvNetPlus password.

PrimeSim HSPICE Demonstration Netlists

PrimeSim HSPICE delivers nearly 350 demonstration files with every release. These demo netlists, many with “readme” files, illustrate the use of the PrimeSim HSPICE commands and control options for multiple applications.

You can find the example directories at:

- For PrimeSim HSPICE: `$installdir/demo/hspice/`
- For PrimeSim HSPICE advanced analyses: `$installdir/demo/hspice/rf_examples/`

For a full listing of the demo file categories, brief descriptions of the netlists, and links to detailed information on the demos, see:

- For PrimeSim HSPICE examples: Listing of Demonstration Input Files in the *PrimeSim HSPICE User Guide: Demonstration Netlists*.
- For PrimeSim HSPICE advanced analyses: Advanced Analog Demonstration Input Files in the *PrimeSim HSPICE User Guide: Advanced Analog Simulation and Analysis*.

2

Setup

Describes the environment variables, standard I/O files, invocation commands, simulation modes, and current parser notation.

This chapter discusses the following topics:

- [Setting Environment Variables](#)
- [Standard Input Files](#)
- [Standard Output Files](#)
- [Working Directory Path Character Limit](#)
- [Continuing PrimeSim HSPICE Simulations after a Power-down](#)

PrimeSim HSPICE ships hundreds of examples for your use; see Listing of Demonstration Input Files in the *PrimeSim HSPICE User Guide: Demonstration Netlists* for paths to the demo files.

For descriptions of individual PrimeSim HSPICE commands mentioned in this chapter, see the *PrimeSim™ Continuum Reference Manual: Commands and Control Options*.

Setting Environment Variables

This section discusses the following topics:

- [License Variables](#)
- [Temporary Directory Variable](#)
- [Shared Libraries Environment Variable](#)
- [Compiled Function Library Environment Variable](#)
- [Using Environment Variables as Include Statements](#)
- [Setting Distributed Processing Environment Variable](#)

- [Setting Array Job Submitting Environment Variable](#)
- [Verilog-A \(pVA\) Environment Variables](#)

License Variables

PrimeSim HSPICE requires you to set the `LM_LICENSE_FILE` environment variable. This variable specifies the location of the `license.dat` license file. Set the `LM_LICENSE_FILE` environment variable to `port@hostname` to point to a license file on a server.

- If you are using the C shell, add the following line to the `.cshrc` file:

```
setenv LM_LICENSE_FILE port@hostname
```

- If you are using the Bash or Bourne shell, add these lines to the `.bashrc` or `.profile` file:

```
LM_LICENSE_FILE=port@hostname  
export LM_LICENSE_FILE
```

The `port` and `hostname` variables correspond to the TCP port and license server host name specified in the `SERVER` line of the Synopsys license file.

Each license file can contain licenses for many packages from multiple vendors. You can specify multiple license files by separating each entry. For UNIX/Linux, use a colon (:), and for Windows, use a semicolon (;).

For details about setting the license file environment variable, see *Setting Up PrimeSim HSPICE for Each User* in *Synopsys® PrimeSim HSPICE® Installation Notes*.

The following sections discuss these topics:

- [License Queuing Variable](#)
- [Controlling the License Queueing Interval](#)
- [License Server Down Iterations](#)
- [Using the FlexLM MAX Option](#)
- [PrimeSim HSPICE License Scheme](#)
- [Limiting the Number of Licenses for Your Group](#)

License Queuing Variable

The optional `PRIMESIM_WAIT_LICENSE` environment variable causes PrimeSim HSPICE to wait for an available license. It is particularly helpful in environments where the tool runs sequentially from batch files and a license checkout failure could result in the loss of important data.

To enable queueing of PrimeSim HSPICE licenses, set:

```
setenv PRIMESIM_WAIT_LICENSE 1
```

If you have five PrimeSim HSPICE or advanced analog feature floating licenses and have checked out all five licenses, then with the `PRIMESIM_WAIT_LICENSE` environment variable enabled, the next job submitted waits in the queue until a license is available. When you have enabled `PRIMESIM_WAIT_LICENSE` and all available licenses are in use, PrimeSim HSPICE issues another message advising that no licenses are available.

When there are multiple license servers, the PrimeSim HSPICE tool queries all of the servers listed in the `LM_LICENSE_FILE` variable and, if no licenses are available, queues the jobs on the servers, when `PRIMESIM_WAIT_LICENSE` is enabled.

For example:

```
LM_LICENSE_FILE = 27000@server1:27000@server2:27000@server3
```

Controlling the License Queueing Interval

As described in the previous section, setting the environment variable `PRIMESIM_WAIT_LICENSE` causes PrimeSim HSPICE to wait for an available license from the license server. If no license becomes available, PrimeSim HSPICE waits for 2 minutes by default. There may be times where it is desirable to set different longer queueing time, such as during scripted operations.

To do this, you must set the environment variable: `PRIMESIM_WAIT_LICENSE_TIMEOUT`. The argument to `PRIMESIM_WAIT_LICENSE_TIMEOUT` is in minutes.

For example:

```
#!/bin/csh -f
setenv PRIMESIM_WAIT_LICENSE 1
setenv PRIMESIM_WAIT_LICENSE_TIMEOUT 60
hspice -i myjob.sp -o outfile
```

Here, `PRIMESIM_WAIT_LICENSE` enables PrimeSim HSPICE to wait for an available license. Next, it sets to 60 minutes. The license request de-queues if no license is available in 60 minutes and the script can continue. This feature is available on all platforms.

Note:

- The `META_QUEUE` and `META_QUEUE_TIME` environment variables are obsolete since S-2021.09 release. They are replaced with `PRIMESIM_WAIT_LICENSE` and `PRIMESIM_WAIT_LICENSE_TIMEOUT` appropriately.

License Server Down Iterations

By default, when the license server is down, PrimeSim HSPICE attempts to reconnect to the license server indefinitely. This behavior is consistent with other Synopsys tools.

Using the FlexLM MAX Option

The FlexLM MAX option allows you to limit the number of licenses available to a user, user group, host or host group.

For example, to limit the number of PrimeSim HSPICE licenses to the user group *designers* to 5, specify the option file as:

```
MAX 5 hspice GROUP designers
```

If a user in the group starts a PrimeSim HSPICE job that exceeds the MAX option value and if `PRIMESIM_WAIT_LICENSE` is set, then the job is queued instead of being terminated.

PrimeSim HSPICE License Scheme

The license scheme of multi-threading (-mt) in standalone license mode is that each license copy serves two threads as listed in the following table.

Number of Cores	1-2	3-4	5-6	7-8	9-10	11-12	13-14	15-16
All PrimeSim HSPICE analysis including HB, SN, Transient Noise, and StatEye.	1	2	3	4	5	6	7	8

The license scheme of multi-threading (-mt) in PrimeSim Continuum token-license mode is that wherein for one set of SPICE tokens enables up to 2 cores. Each additional set of tokens multiplies the cores by 2.

Tool	-mt cores	PrimeSim Tokens
PrimeSim HSPICE	1-2	1(1x1)
	3-4	2(1x2)
	5-8	3(1x3)
	9-16	4(1x4)

Limiting the Number of Licenses for Your Group

You can create a license option file named `snpslmd.opt` in the directory where the license file is located. In the following example:

```
HOST_GROUP HSPICEGR 1.160.10.240 238.17.159.4  
MAX 2 hspice HOST_GROUP HSPICEGR
```

The first line creates a group named `HSPICEGR` followed by a list of the IP addresses of the machines in that group. The second line defines (for the license server) that the group is allowed a maximum of two PrimeSim HSPICE licenses.

You can change the name of the file and its directory. The option file also allows you to reserve licenses for a group and create groups by user name instead of IP address. For more information about license option files, see the `Administration_Guide.pdf` (installed with the Synopsys Common Licensing product).

Temporary Directory Variable

To specify the location to deposit scratch files, set the `tmpdir` (UNIX/ Linux), `TEMP` or `TMP` (Windows) environment variable. PrimeSim HSPICE opens three scratch files in the `/tmp` directory.

In the Windows environment, PrimeSim HSPICE opens three scratch files in the `c:\path\TEMP` (or `\TMP`) directory. To change this directory, reset the `tmpdir` environment variable in the PrimeSim HSPICE command script.

For example:

```
setenv tmpdir /my_area/scratch/project
```

Shared Libraries Environment Variable

If you receive a message of the form “error while loading shared libraries”, it is due to an OS issue relating to a missing library on your machine. For example, if you are using PrimeSim HSPICE 64-bit machine, verify that the library exists in the `/usr/lib64` directory. The PrimeSim HSPICE 64-bit (`usr/lib64`) shared libraries require the setting of the following environment variable:

```
setenv LD_LIBRARY_PATH /usr/lib64
```

Verify that your OS is compatible with the Synopsys Platform recommendations at: <http://www.synopsys.com/Support/Licensing/SupportPlatform/ReleaseSupport/Pages/default.aspx>

Compiled Function Library Environment Variable

Set the CFL environment variable and the specified path and *.so file as follows:

```
setenv CFL_COMPILED_LIB CFL library_file_name
```

For example:

```
% setenv CFL_COMPILED_LIB /path/libcfl.so
```

For a discussion of the CFL capability, see [Features](#).

Using Environment Variables as Include Statements

To use environment variables as include statements:

1. Set an environment variable to the name of the file that you want to include.

```
% setenv MY_OWN_INCLUDE_FILE inv.inc
```

2. Use the environment variable in the netlist:

```
*
.inc '$MY_OWN_INCLUDE_FILE'    $ include the file inv.inc
.option list
.op
X1 in out inv
c1 out 0 0.1p
.global vdd gnd
.model n nmos level= 49
.model p pmos level= 49
vdd vdd 0 5
vin in 0 pulse 0 5 0 1n 1n 5n 10n
.tran 10p 100n
.option post=2
.end
```

PrimeSim HSPICE includes the file defined by the environment variable in the netlist.

Setting Distributed Processing Environment Variable

Only if you run the PrimeSim HSPICE binary directly, set the following environment variable before invoking Distributed Processing (DP): `setenv CDPL_HOME $INSTALLDIR/hspice/cdpl`

For Synopsys distributed processing, the PrimeSim HSPICE wrapper sets up `$CDPL_HOME` automatically. The Synopsys Common Distributed Processing Library (CDPL) provides the engine for the `-dp` command-line invocation. The `-mp` (multiprocessing) calls are interchangeable with DP.

Setting Array Job Submitting Environment Variable

The `PRIMESIM_CDPL_ARRAY_JOB` environment variable enables array job submitting support for distributed processing. This feature supports array job for SGE, LSF, PBS, and SLURM grids.

You can activate this feature by setting `PRIMESIM_CDPL_ARRAY_JOB` to 1.

For `csh` users:

```
setenv PRIMESIM_CDPL_ARRAY_JOB 1
```

For `bash` users:

```
export PRIMESIM_CDPL_ARRAY_JOB=1
```

By default, PrimeSim HSPICE does not automatically submit Distributed Processing (DP) worker as array job.

Verilog-A (pVA) Environment Variables

The pVA library allows both absolute path and relative paths in `.hdl` statements. Alternatively, you can use environment variables. For example:

```
.hdl "/tmp/design01/lib/INV_linux4020110.pvalib"  
.hdl "${PVA_PATH}/${LIBNAME}_${PLATFORM}4020110.pvalib"  
.hdl "../../lib/${PVA_LIBRARY}.pvalib"
```

You can define `${PVA_PATH}`, `${LIBNAME}`, `${PLATFORM}`, and `${PVA_LIBRARY}` using the `setenv` UNIX command.

In addition, to prevent re-compiling, you can set the `PVA_DIR` environment variable to the name of the directory that you want to share the compiled Verilog-A modules. In the following example, the environment variable redirects `*.pvadir` to directory `${PVA_DIR}.pvadir`.

```
setenv PVA_DIR $abs|relative_path/xyz
```

Standard Input Files

This section describes the standard input files to PrimeSim HSPICE.

The following sections discuss these topics:

- [Design and File Naming Conventions](#)
- [Initialization File \(hspice.ini\)](#)
- [DC Operating Point Initial Conditions File](#)

- [Input Netlist File](#)
- [Library Input File](#)

Design and File Naming Conventions

The design name identifies the circuit and any related files, including:

- Schematic and netlist files.
- Simulator input and output files.
- Design configuration files.
- Hardcopy files

PrimeSim HSPICE extracts the design name from their input files, and performs actions based on that name. PrimeSim HSPICE reads and writes files related to the current circuit design. Files related to a design usually reside in a single directory. The output file is stdout on UNIX platforms; you can redirect the output.

[Table 1](#) lists the input file types and their standard names. The subsequent sections describe these files.

Table 1 *Input Files*

Input File Type	File Name
Initialization file	hspice.ini
DC operating point initial conditions file	<i>design.ic#</i>
Input netlist file	<i>design.sp</i>
Library input file	<i>library_name</i>

Initialization File (hspice.ini)

The initialization file enables you to specify user defaults. If PrimeSim HSPICE reads one `hspice.ini` file, PrimeSim HSPICE includes its contents at the top of the input file. All PrimeSim HSPICE simulations look for *one* implicit `hspice.ini` file. The default search order for the `hspice.ini` file is:

1. `cwd/hspice.ini`: The current working directory
2. `$HOME/hspice.ini`: The HOME directory of the user
3. `$installdir/hspice.ini`: The PrimeSim HSPICE installation directory

You can use an initialization file to set options (`.OPTION` statements) and to access libraries. You can define a customized initialization file and include it in your netlist using the `.INC` command.

DC Operating Point Initial Conditions File

The DC operating point initial conditions file `design.ic#` is an optional input file that contains initial DC conditions for particular nodes. You can use this file to initialize DC conditions, by using either a `.NODESET` or an `.IC` statement.

A `.SAVE` statement can also create a `design.ic#` file. A subsequent `.LOAD` statement initializes the circuit to the DC operating point values that you specified in this file.

Input Netlist File

The input netlist file `design.sp` contains the design netlist. Optionally, it can also contain statements that specify the type of analysis to run, the type of output desired, and what library to use.

Library Input File

You use the `library_name` files to identify libraries and macros that need inclusion for simulating `design.sp`.

Standard Output Files

Table 2 lists the various types of output files produced by PrimeSim HSPICE.

For information about the standard output file while performing advanced analog function, see PrimeSim HSPICE Advanced Analog Output Files in the *PrimeSim HSPICE User Guide: Advanced Analog Simulation and Analysis*.

Table 2 *PrimeSim HSPICE Output Files and Extensions*

Output File Type	Extension
AC analysis measurement results	.ma# ¹
AC analysis results (from <code>.POST</code> statement)	.ac#
Data mining results	.mpp0
DC analysis measurement results	.ms#

1. # can be either a sweep number or a hardcopy file number. For `.ac#`, `.dp#`, `.dm#`, `.ic#`, `.st#`, `.sw#`, and `.tr#` files, # is from 0 through 9999.

Table 2 *PrimeSim HSPICE Output Files and Extensions (Continued)*

Output File Type	Extension
DC analysis results (from <code>.POST</code> statement)	.sw#
Error message file generated when outputs do not match the expected outputs defined in the <code>.VEC</code> file	.err#
FFT analysis graph data (from <code>FFT</code> statement)	.ft#
Monte Carlo results	.mc#
Operating point information (from <code>.OPTION OPFILE</code> statement)	.dp#
Operating point node voltages (initial conditions)	.ic#
Output listing	.lis, or user-specified
Output status	.st#
Output tables (from <code>.DCMATCH OUTVAR</code> statement)	.dm#
Pole-zero analysis results	.pz#
StatEye analysis measurement results	.mste#
StatEye analysis initial transient measurement results	.mtNp{f}# ²
StatEye analysis initial transient results	.trNp{f}#
StatEye analysis time-based results (from <code>.POST</code> statement)	.stet#
StatEye analysis voltage based results (from <code>.POST</code> statement)	.stev#
Subcircuit cross-listing	.pa#
Transient analysis measurement results	.mt#
Transient analysis results (from <code>.POST</code> statement)	.tr#
Waveform viewing files from <code>.OPTION WDF</code> argument for use with WaveView/SX tools	*_wdf.tr#, *_wdf.sw#, or *_wdf.ac#

The following sections describe these output files:

- [AC Analysis Results File](#)
- [AC Analysis Measurement Results File](#)
- [DC Analysis Results File](#)
- [DC Analysis Measurement Results File](#)

2. *N* is the incident port index, *f* is the edge index when edge is greater than 1, and # is the alter index.

- [FFT Analysis Graph Data File](#)
- [Operating Point Information File](#)
- [Operating Point Node Voltages File](#)
- [Output Listing File](#)
- [Output Status File](#)
- [Output Tables](#)
- [Subcircuit Cross-Listing File](#)
- [Transient Analysis Measurement Results File](#)
- [Transient Analysis Results File](#)
- [Waveform Viewing File](#)

AC Analysis Results File

PrimeSim HSPICE writes AC analysis results to the *output_file.ac#* file, where # is 0–9999, according to your specifications following the `.AC` statement. These results list the output variables as a function of frequency.

AC Analysis Measurement Results File

PrimeSim HSPICE writes measurement results of an AC analysis to the *output_file.ma#* file, when the input file includes a `.MEASURE AC` statement.

DC Analysis Results File

PrimeSim HSPICE writes DC analysis results to the *output_file.sw#* file, where # is 0–9999, when the input file includes a `.DC` statement. This file contains the results of the applied stepped or swept DC parameters that the statement defines. The results can include noise, distortion, or network analysis.

DC Analysis Measurement Results File

PrimeSim HSPICE writes DC analysis measurement results to the *output_file.ms#* file, when the input file includes a `.MEASURE DC` statement.

FFT Analysis Graph Data File

The FFT analysis graph data file, *output_file.ft#*, contains the graphical data to display the FFT analysis waveforms.

Operating Point Information File

PrimeSim HSPICE writes operating point information to the *design.dp#* file, when the input file includes an `.OPTION OPFILE=1` statement.

Operating Point Node Voltages File

PrimeSim HSPICE writes operating point node voltages to the *output_file.ic#* file, where # is 0-9999, when the input file includes a `.SAVE` statement. These node voltages are the DC operating point initial conditions.

Output Listing File

The output listing is a text file. You can name it *output_file* (no file extension), *output_file.lis*, or a file extension that you specify, depending on the format that you use to start the simulation.

The output listing file includes the following information:

- Name of the simulator used.
- Version of the PrimeSim HSPICE simulator used.
- The Synopsys message block.
- Input filename.
- User name.
- License details.
- Node count.
- Operating point parameters.
- Actual control option values that PrimeSim HSPICE uses for the present simulation. (This is useful when options such as RUNLVL override user-specified values.)
- Details of the voltage drop, current, and power for each source and subcircuit.
- Results of a `.PRINT` statement.
- Results of the `.OPTION` statements.

- **Total CPU time** (the sum of `op point`, `transient`, `readin`, `errchk`, `setup`, and `output` times).

In the following snippet of a `*.lis` file, you can see that the total CPU time is the sum of `op point`, `transient`, `readin`, `errchk`, `setup`, and `output` times.

```
analysis   time      # points   tot. iter   conv.iter
op point           0.00             1           4
transient          0.07          446328        64       32 rev= 3
readin             0.01
errchk             0.01
setup              0.00
output             0.00
total cpu time     0.09 seconds
```

Where:

- `op point`: Is the time required to perform operating point analysis.
- `transient`: Is the time required to perform transient analysis.
- `readin`: Is the time required to read the user data file and any additional library files, and generate an internal representation of the information.
- `errchk`: Is the time required to check the errors and evaluate the models.
- `setup`: Is the time required to construct a sparse matrix pointer system.
- `output`: Is the time required to prepare the output files and to process all prints and plots.
- `total cpu time`: Is the time taken for the simulation only.

This time differs slightly from run to run, even though runs are identical. It does not include memory/disk allocation time or disk I/O time. You can calculate this by subtracting job end-time from job start-time.

Output Status File

The output status file, `output_file.st#`, where `#` is 0-9999, contains the following runtime reports:

- Start and end times for each CPU phase.
- Option settings, with warnings for obsolete options.
- Status of preprocessing checks for licensing, input syntax, models, and circuit topology.
- Convergence strategies that PrimeSim HSPICE uses on difficult circuits.

You can use the information in this file to diagnose problems, particularly when communicating with Synopsys Customer Support.

Output Tables

The `.DCMATCH` output tables file, `output_file.dm#`, contains the variability data from analysis.

Subcircuit Cross-Listing File

If the input netlist includes subcircuits, PrimeSim HSPICE automatically generates a subcircuit cross-listing file, `output_file.pa#`, where # is 0-9999. This file relates the subcircuit node names, in the subcircuit call, to the node names used in the corresponding subcircuit definitions. In PrimeSim HSPICE advanced analyses, you cannot replicate output commands within subcircuit (`.subckt`) definitions.

Transient Analysis Measurement Results File

PrimeSim HSPICE writes transient analysis measurement results to the `output_file.mt#` file, when the input file includes an `.MEASURE TRAN` statement.

Transient Analysis Results File

PrimeSim HSPICE places the results of transient analysis in the `output_file.tr#` file, where # is 0-9999, which you set in the `-n` command-line argument. This file lists the numerical results of transient analysis. A `.TRAN` statement in the input file, together with an `.OPTION POST` statement, creates this post-analysis file. If the input file includes an `.OPTION POST` statement, then the output file contains simulation output suitable for a waveform display tool.

Waveform Viewing File

After you use `.OPTION WDF` for transient, DC, or AC analyses, for the WDF waveform file, PrimeSim HSPICE automatically appends `_wdf` into the output file root name to specify that it is in WDF format. The file names appear as: `*_wdf.tr#`, `*_wdf.sw#`, or `*_wdf.ac#`.

For example, PrimeSim HSPICE names the WDF waveform output file `design_wdf.tr0`.

Working Directory Path Character Limit

In PrimeSim HSPICE, the number of characters in a path name, including the file name, is limited to a maximum of 1024 characters.

For example:

```
hspice -i path_name/input_file -o out_file
```

When specifying a path and file name using `-i` or `-o`, the length must not exceed 1024 characters. If the working directory path is greater than 1024 characters, PrimeSim HSPICE aborts with a signal 11 error.

To check the length of the working directory path, use the UNIX command:

```
% pwd | wc -c
```

Workaround

Because this can be an issue for automated programs that create pathnames based on appending long design and cell names, there is a workaround. On Linux/UNIX platforms, you can create a soft link in your local design directory to the file at the end of the long path.

```
% ln -s /long/directory/path/target.inc target.inc
```

Then, include the link instead of the actual file:

```
.inc target.inc
```

This way, the file created by the automated program does not require relocation.

Continuing PrimeSim HSPICE Simulations after a Power-down

When a power-down occurs, one of the following situations can occur:

1. The license server shuts down.
2. The machine running the PrimeSim HSPICE job shut down.
3. Both the license server and the machine running the PrimeSim HSPICE job shuts down.

For case 1 (the power down happens only to the license server), PrimeSim HSPICE gives you a 30-minute grace period to reconnect to the license server. If power is restored and the license server is up within 30 minutes, then any interrupted PrimeSim HSPICE jobs start up from where they stopped.

For cases 2 and 3 (the machine running the PrimeSim HSPICE jobs powers down), you can make use of the `.STORE` and `-restore` features in PrimeSim HSPICE.

The `.STORE` command can save the simulation state to a file at a certain time or set of times. For more information on using the `.STORE` command, see [Storing and Restoring Checkpoint Files](#).

3

Startup and Simulation

Describes the invocation commands and simulation modes.

PrimeSim HSPICE ships numerous examples for your use; see Listing of Demonstration Input Files in the *PrimeSim HSPICE User Guide: Demonstration Netlists* for paths to the demo files.

For descriptions of individual PrimeSim HSPICE commands mentioned in this chapter, see the *PrimeSim™ Continuum Reference Manual: Commands and Control Options*.

For simulations involving multiple cores, distributed processing, and multithreading, see [Chapter 4, Distributed Processing and Multithreading](#).

The following sections discuss these topics:

- [Invoking PrimeSim HSPICE Simulations](#)
- [Using Isomorphic Analyses in Subcircuit Blocks](#)
- [Running PrimeSim HSPICE Simulations on Windows](#)
- [Running PrimeSim HSPICE Interactively](#)
- [Using PrimeSim HSPICE in Client-Server Mode](#)
- [Running PrimeSim HSPICE to Calculate New Measurements](#)

Invoking PrimeSim HSPICE Simulations

The following sections discuss these topics:

- [Syntax to Invoke PrimeSim HSPICE](#)
- [Sequence of Events](#)

Syntax to Invoke PrimeSim HSPICE

Use the following syntax to invoke PrimeSim HSPICE:

```
hspice [-dpredict]
        [-i | -hspice path/input_file]
        [-o path/output_file]
        [-arch COMPATIBLE]
        [-n number]
        [-gz]
        [-d]
        [-C path/input_file]
        [-CC path/input_file]
        [-I]
        [-K]
        [-L command_file]
        [-include_first ins_filename]
        [-include_last app_filename]
        [-S]
        [-case 0|1]
        [-datamining -i datamining.cfg [-o outname]]
        [-dp [num]
            [-dpconfig dp_configuration_file]
            [-dplocation NFS|TMP]
            [-merge]
            [-dpgui]
            [-dpmode alter|sweep]
            [-dpincremental original_path]
            [-dscale output.dynamic] ]
        [-dpworkerchk none | mem | load | runtime | disk | all ]]
        [-dplicense 1|0]
        [-mp process_count]
        [-mt thread_count]
        [-hpp]
        [-meas measure_file]
        [-mrasm [0|1|2|3]]
        [-top subcktname]
        [-restore checkpoint_file]
        [-auto_restore | -auto_restart]
        [-hdl file_name]
        [-hdlpath pathname]
        [-vamodel name]
        [-vamodel name2...]
        [-sae]
        [-search | -inc path1[:path2:path3...]]
        [-alter_select num | num1:num2 | alter_name]
        [-help | -webhelp]
        [-doc]
        [-h]
        [-v | -version]
        [-t | time_value]
        [-time]
```

```
[ -stop time_value ]
[ -wavefmt | -format fsdb|wdf|psf|tr0 ]
```

Argument	Description
<code>-dpredict</code>	Predicts the simulation time for Monte Carlo cases.
<code>-i path/input_file</code>	Input netlist file name for which an extension <code>*.ext</code> is optional. - If you do not specify an input file name extension in the command, PrimeSim HSPICE searches for: - A <code>*.sp#</code> file or: - A <code>*.tr#</code>, <code>*.ac#</code>, or <code>*.sw#</code> file (PSF files are not supported). PrimeSim HSPICE uses the input file name as the root for the output files. To exceed 256 characters, use the <code>-i longpath_exceed256/filename</code> command. PrimeSim HSPICE also checks for an initial conditions file (<code>.ic</code>) that has the input file root name. The following is an example of an input file name: <code>/usr/sim/work/rb_design.sp</code>. In this file name: <code>/usr/sim/work/</code> is the directory path to the design <code>rb_design</code> is the design root name. <code>.sp</code> is the file name suffix.
<code>-o path/output_file</code>	Name of the output file. Here, <code>output_file</code> is the root name of the output file. PrimeSim HSPICE appends the <code>.lis</code> extension to all output files. For example: - For the output log file: <code>output_file.lis</code> - For transient waveform: <code>output_file.tr0</code> - For transient measurement: <code>output_file.mt0</code> Everything up to the last period is the root file name and everything after the last period is the file name extension. - If you include the <code>.lis</code> extension in the file name that you enter using this option, then PrimeSim HSPICE does not append another <code>.lis</code> extension to the root file name of the output file. - If you do not specify an output file name, PrimeSim HSPICE directs output to stdout. For the <code>.meas</code> option, some case results differ from the measure result PrimeSim HSPICE produces. To exceed 256 characters, use the <code>-o longpath_exceed256/filename</code> command.
<code>-arch COMPATIBLE</code>	Switches to PrimeSim HSPICE's AVX2 compatible math lib, instead of the CPU's default math lib.
<code>-t</code>	Supports the overriding <code>.tran</code> end time. Default unit is second (s).
<code>-n number</code>	Starting number for numbering output data file revisions (<code>output_file.tr#</code>, <code>output_file.ac#</code>, <code>output_file.sw#</code>, and <code>output_file.ft#</code>, where <code>#</code> is between 0 and 9999.).
<code>-gz</code>	Generates compression output on analysis results for these output types: <code>.tr#</code> , <code>.ac#</code> , <code>.sw#</code> , <code>.ma#</code> , <code>.mt#</code> , <code>.ms#</code> , <code>.mc#</code> , <code>.print*</code> .
<code>-d</code>	(UNIX) Displays the content of <code>.st0</code> files on screen while running PrimeSim HSPICE, for example, to show the status during simulation.

Argument	Description
<code>-C path/input_file</code>	Client/Server (C/S) mode. - Entering <code>hspice -C</code> checks out a PrimeSim HSPICE license and starts client/server mode. - Entering <code>hspice -C path/input_file</code> simulates your netlist. - Entering <code>hspice -C -K</code> releases the PrimeSim HSPICE license and exits. For additional information, see Using PrimeSim HSPICE in Client-Server Mode
<code>-CC path/input_file</code>	Advanced Client/Server mode. - Entering <code>hspice -CC</code> checks out PrimeSim HSPICE license and starts the advanced client/server mode. - Entering <code>hspice -CC path/input_file</code> simulates your netlist. - Entering <code>hspice -CC -share common.sp -o output</code> redirects share file to avoid issues with *.lis file for the shared model file while running the multiple servers on a farm or multi-CPU machine. - Entering <code>hspice -CC -K</code> releases the PrimeSim HSPICE license and exits. For additional information, see Launching the Advanced Client-Server Mode (-CC)
<code>-I</code>	Interactive mode. - Entering <code>hspice -I</code> invokes interactive mode. - Entering <code>help</code> at the PrimeSim HSPICE prompt, lists supported commands. - Entering <code>hspice -I -L file_name</code> runs a command file. - Entering <code>quit</code> at the <code>hspice</code> prompt, exits interactive mode. For additional information, see Using Interactive Mode.
<code>-K</code>	Used with <code>-C</code> option to terminate client/server mode and exit.
<code>-L file_name</code>	Used with <code>-I</code> option to run commands contained in a command file.
<code>-include_first ins_filename</code>	Used to insert the specified file from the command line.
<code>-include_last app_filename</code>	Used to append the specified file from the command line.
<code>-S</code>	Performs as a server. Accepts data from SPEED2000, simulates the circuit, and returns results to SPEED2000. - On UNIX and Linux, PrimeSim HSPICE waits for successive simulations after invocation. - On Windows, you must re-invoke for each successive simulation.

Argument	Description
<code>-case 0 1</code>	• 0: (default) case sensitivity disabled • 1: case sensitivity enabled Enables case sensitivity only for the following items (PrimeSim HSPICE commands and control options continue to be case-insensitive): • Parameter Names • Node Names • Instance Names • Model Names • Subcircuit Names • Data Names • Measure Names • File Names and Paths (case sensitive by default) • Library Entry Names
<code>-datamining -i datamining.cfg [-o outname]</code>	PrimeSim HSPICE skips netlist <code>readin</code>, <code>errchk</code>, and simulation, and only does standalone data mining. The configuration file content includes: • comments/description • Required records • <code>.sampleFile input.mct</code> • <code>.measFile input .mt0 input .mt0A input.mt0B...</code> • <code>.Option Screening_Method [= Pearson Spearman]</code> See Data Mining for more information.
<code>[-dscale output.dynamic]</code>	Supports dynamic adjustment of the number of distributed processing (DP) workers. During DP, the simulator creates a file <code>output.dynamic</code> in the DP output directory. Now, you can launch another PrimeSim HSPICE simulation with the following command: <pre>hspice -dscale output.dynamic -dp dp_num</pre> to change/adjust the current number of DP workers to another value. In this manner, you can decrease and increase the DP worker numbers, as many times as required For details, see Running Distributed Processing on a Network Grid.
<code>[-dpconfig dp_configuration_file]</code>	Specifies the configuration file for DP. Refer to the <i>CDPL User Guide</i> for details about the configuration file. Note: If <code>-dp</code> is triggered without specifying the <code>-dpconfig dp_configuration_file</code> option, all the distributions are run on the same machine.
<code>[-dplocation NFS TMP]</code>	By default, all the results files are written to the <code>/tmp</code> folder, and then move whatever output is needed to the <code>-o HSPICE output</code>. Otherwise, specify the location of where to write all the intermediate files during simulation such as <code>/my_location/tmp</code>.
<code>[-merge]</code>	Merges the output files from PrimeSim HSPICE only if you specify this option.

Argument	Description
<code>[-dpgui]</code>	Launches the DP manager to monitor the status of the DP run. For more information on using DP Manager, see <i>DPmanagerUserGuide</i> in location <code>\$installdir/hspice/cdpl/doc/DPManagerUserGuide.pdf</code>.
<code>[-dpmode alter sweep]</code>	Changes the priority for distributed processing. The default DP mode is automatically determined by tool based on the <code>dp_num</code>, <code>.ALTER</code> blocks, and Monte Carlo (data block or parametric sweep). If the number of alter blocks is smaller than the <code>dp_num</code> and the number of Monte Carlo is more than ten times greater than the number of alter blocks; the <code>dpmode</code> switches to <code>[sweep]</code> automatically for optimal use of <code>dp_worker</code>. If DP mode is set to <code>sweep</code>, DP is based on transient, dc, and ac sweep commands respectively, instead of <code>.alter</code>. Note: When both the transient command and the dc/ac sweep command are in a netlist, DP for transient analysis has the highest priority.
<code>[-dpincremental original_path]</code>	Invokes the automatic incremental DP Monte Carlo simulation. <i>original_path</i> specifies the "sub_dpdir" of the original run, in the incremental run mode. For additional information, see Automatic Incremental DP Monte Carlo.
<code>[-dscale dp_num]</code>	Supports dynamic adjustment of the number of DP workers. During DP processing, the simulator creates a file <i>output.dynamic</i> in the DP output directory. Now, you can launch another PrimeSim HSPICE simulation with the following command: <pre>hspice -i output.dynamic -dscale dp_num</pre> to change/adjust the current number of DP workers to another value. In this manner, you can decrease and increase the DP worker numbers, as many times as required.
<code>-dplicense 1 0]</code>	Specifies the DP license optimization mode. 1: Turns on DP license optimization. Based on the DP number specified (that is, in <code>-dp num</code>), the PrimeSim HSPICE supervisor process checks out the required copies of licenses and launches the workers . 0 (default): Turns off DP license optimization. Based on the DP number specified, the PrimeSim HSPICE supervisor process launches the workers. Each worker checks out a license for its own simulation. Note: If you do not specify <code>-dplicense</code>, the PrimeSim HSPICE tool uses the <code>-dplicense 0</code> default mode.

Argument	Description
<pre>-dpworkerchk none mem load runtime disk all</pre>	DP monitor level. Default is <i>all</i>. <i>none</i>: no feature is monitored. <i>mem</i>: memory monitor. DP worker shuts down if memory consumption is too high on remote host. <i>load</i>: load monitor. DP worker shuts down if host is in heavy load. <i>runtime</i>: task runtime monitor. Supported in Monte Carlo. Some samples may be skipped if it takes too long to conclude in comparison to others. <i>disk</i>: disk monitor. DP worker shuts down if disk usage is too high on remote host. DP master shuts down if disk usage is too high on output path. <i>all</i>: memory, load, task runtime and disk usage are all monitored. Note: Multiple monitoring types for memory, load, runtime and disk options are supported by combining two options with "+" syntax.
<pre>-mp [process_count]</pre>	Activates multiprocessing while running ALTER cases, transient sweeps, and Monte Carlo analyses on one machine with multiple processors/cores. If you specify the number of CPUs, you can limit the number of CPUs to avoid overtaxing performance scalability. If a CPU number is not specified, PrimeSim HSPICE auto-determines the child processes by the number of available CPUs. For details, see the <i>PrimeSim HSPICE User Guide: Basic Simulation and Analysis</i>. For additional information, see Running Multithreading and Distributed Processing Concurrently in the <i>PrimeSim HSPICE User Guide: Basic Simulation and Analysis</i>.
<pre>-mt thread_count</pre>	Invokes multithreading and specifies the number of processors for a multi-threaded simulation. If <i>thread_count</i> is not entered, PrimeSim HSPICE issues an error. For additional information, see Running Multithreading and Distributed Processing Concurrently in the <i>PrimeSim HSPICE User Guide: Basic Simulation and Analysis</i>. See also .OPTION MTTHRESH in this manual.

Argument	Description
<code>-meas <i>measure_file</i></code>	Re-invokes the measure file to calculate new measurements from a previous simulation. The format of <i>measure_file</i> is similar to the PrimeSim HSPICE netlist format. The first line is a comment line and the last line is an <code>.END</code> command. <code>-meas</code> also supports double precision waveform (post_version=2001) data. The following netlist commands are supported: <code>.MEASURE</code> <code>.PARAM</code> <code>.TEMP</code> <code>.OPTION</code> <code>.DATA</code> <code>.ENDDATA</code> <code>.FFT</code> <code>.MEASURE FFT</code> <code>.CREATE_EYE</code> <code>.MEASURE TRANEYE</code> <code>.END</code> Note: The <code>.DATA</code> command in the measure file must be consistent with the <code>.DATA</code> command in the wavefile. - The following types of <code>.OPTION</code> commands are supported: <code>MEASFAIL</code> <code>MEASFORM</code> <code>NUMDGT</code> <code>INGOLD</code> <code>MEASDGT</code> <code>EM_RECOVERY</code> Warnings are issued if other options or commands are used. Syntax to perform spectrum analysis measurements from previous simulation results: <code>hspice -i *.tr0 -meas <i>measure_file</i></code>
<code>-mrasim [0 1 2 3]</code>	Overwrites the value of <code>SimMode</code> in a <code>.MOSRA</code> command card: 0: Selects pre-stress simulation only 1: Selects post-stress simulation only 2: Selects both pre- and post-stress simulation 3: Selects continual degradation integration through <code>.ALTERS</code> For example: <code>hspice -i input.sp -o run1 -mrasim 2</code>
<code>-top <i>subcktname</i></code>	Top level subcircuit name. Effectively eliminates <code>.subckt <i>subcktname</i></code> and corresponding <code>.ends</code> statements. Users do not need to instantiate top-level SUBCKT using "X" syntax of PrimeSim HSPICE.

Argument	Description
<pre>-restore checkpoint_file/-auto_restore auto_restart</pre>	The <i>checkpoint_file</i> specifies from which simulation the checkpoint data is to be restored. The <i>restore</i> operation should be submitted on a machine that has the same kernel version as the machine used for store. Otherwise, a failure may occur. Any output files generated by the previous simulation should not be removed. After the restore simulation is done, the output files will be updated. For example: <code>hspice -i test.sp -restore test.1e-7.ic0 -o test.</code> <code>hspice -i test.sp -restore test_100ns-@Linux:3.10.0.el7.x86_64.isnapshot -o test.</code> The simulation starts from the time point that data was stored at, in the previously interrupted simulation. See Storing and Restoring Checkpoint Files for full details. The <code>-auto_restore -auto_restart</code> argument automatically lets PrimeSim HSPICE detect the last saved check-point files (<code>*.<time_point>.ic0</code>, <code>*.<time_point>.sup</code>) by the <code>.store</code> statement in previous simulation. If <code>type=snapshot</code>, the <code>-auto_restore -auto_restart</code> argument automatically lets PrimeSim HSPICE detect the last saved check-point files (<code>*.<time_point>@<linux_platform_info>.isnapshot</code>, <code>*.<time_point>@<linux_platform_info>.isnapshot.FILES</code>) by the <code>.store</code> statement in previous simulation. It is similar to the <code>-restore</code> argument while it does not need you to specify the specific saved check-point files. To restore from the last snapshot and extend the transient simulation to new <code>tstop</code>: <pre>hspice input.sp -o output_name -auto_restart -stop new_tstop"</pre>
<pre>-hdl file_name</pre>	Verilog-A module. The Verilog-A file is assumed to have a <code>*.va</code> extension when only a prefix is provided. One <code>-hdl</code> option can include one Verilog-A file, use multiple <code>-hdl</code> options, if multiple Verilog-A files are needed. This example loads the <code>amp.va</code> Verilog-A source file: <pre>hspice amp.sp -hdl amp.va</pre> When a module to be loaded has the same name as a previously-loaded module or the names differ in case only, the latter one is ignored and the simulator issues a warning message. If a Verilog-A module file is not found or the Compiled Model Library file has an incompatible version, the simulation exits and an error message is issued.

Argument	Description
<code>-hdlpath pathname</code>	Search path for a Verilog-A file, if PrimeSim HSPICE cannot find it in the current working directory. The search order for Verilog-A files is: 1. Current working directory 2. Path defined by command-line argument <code>-hdlpath</code> 3. Path defined by environment variable <code>HSP_HDL_PATH</code> The path defined by either <code>-hdlpath</code> or <code>HSP_HDL_PATH</code> can consist of a set of directory names. The path separator must follow PrimeSim HSPICE conventions or platform conventions ("," on UNIX). Path entries that do not exist are ignored and no error/warning messages are issued. This example first searches the current working directory and when a *.va file is not found, the relative location <code>./my_modules</code> directory is searched: <code>hspice amp.sp -hdlpath ./my_modules</code>
<code>-vamodel name -vamodel name2..</code> <code>.</code>	Cell names for Verilog-A definitions. <i>name</i> is the cell name that uses a Verilog-A definition rather than a subcircuit definition when both exist. Each <code>-vamodel</code> option can take no more than one name. Repeat this option, if multiple Verilog-A modules are defined. If no name is supplied after <code>-vamodel</code> , then the Verilog-A definition will be used whenever it is available.
<code>-sae</code>	Outputs a new <code>results.xml</code> file.
<code>-search path1[:path2:path3...]</code>	Search paths for netlist files, if PrimeSim HSPICE cannot find it in the current working directory. You can specify multiple paths using the <code>:</code> delimiter.
<code>-alter_select num num1:num2</code> <code> alter_name</code>	Enables running one or multiple specific <code>.ALTER</code> sections by specifying its index, index range, or name, without first running the main netlist. Multiple indices can also be specified without separators. Note: The <code>ALTER</code> index starts from 1 and not 0.
<code>-help</code>	Opens a searchable browser-based help system for PrimeSim HSPICE. A html browser must be installed on your machine to access this help system. For more information, see Viewing Online Help Topics From the Command-Line section.
<code>-doc</code>	Displays the PDF documentation set of user manuals for PrimeSim HSPICE. It is recommended that you have Adobe Acrobat Reader or another PDF format reader installed on your system. You can do full text searches of the documentation set.
<code>-h</code>	Displays a help message and exits.
<code>-t</code>	Overrides <code>.tran</code> end time. Supports scale unit such as p, n, m, u, meg, and x.
<code>-time</code>	Alias of <code>-t</code> in PrimeSim HSPICE .
<code>-stop</code>	Overrides transient stop time of <code>.Tran</code> .

Argument	Description
-webhelp	Alias to -help open browser for online manual.
-version	Outputs PrimeSim HSPICE version information.
-info tmi_sec	Displays the version details of TMI/TME SPICE Model. For example: TMI/TME SPICE MODEL SPECIFICATION VERSION : v2021d01.
-hspice	Alias to -i.
-inc	Alias to search (-search).

For multiple processing, multithreading, distributed processing, and PrimeSim HSPICE Precision Parallel features, see [Chapter 4, Distributed Processing and Multithreading](#).

PrimeSim HSPICE provides a quick demo file for a simple LRC circuit to test your installation. See `demo.sp` under Benchmark Examples in the *PrimeSim HSPICE User Guide: Demonstration Netlists*.

Sequence of Events

When you invoke a PrimeSim HSPICE simulation, the following sequence of events occurs:

1. Invocation:

You invoke the PrimeSim HSPICE tool by entering the following command, for example, at the shell prompt:

```
% hspice -i demo.sp -o demo.lis &
```

This command invokes the PrimeSim HSPICE tool on the input netlist file `demo.sp` and directs the output listing to the file `demo.lis`. The `&` character at the end of the command invokes the PrimeSim HSPICE tool in the background, so that you can continue to use the command window and the keyboard while the PrimeSim HSPICE tool runs.

2. Script execution:

The `hspice` command starts the PrimeSim HSPICE executable from the appropriate architecture (machine type) directory. The UNIX `run` script launches a PrimeSim HSPICE simulation. This procedure establishes the environment for the PrimeSim HSPICE executable. The script prompts for information such as the platform that you are using and the PrimeSim HSPICE version to run. (The available versions of the PrimeSim HSPICE tool are pre determined during the installation of the tool.)

3. Licensing:

The PrimeSim HSPICE tool supports the FLEXlm licensing management system. When you use FLEXlm licensing, the PrimeSim HSPICE tool reads the `SNPSLMD_LICENSE_FILE` or the `LM_LICENSE_FILE` environment variable to find the available license server. If the PrimeSim HSPICE tool cannot authorize the access, the job terminates at this point and the PrimeSim HSPICE tool prints an error message in the output listing file.

4. Simulation configuration:

The PrimeSim HSPICE tool reads the appropriate `hspice.ini` file. The search order for the configuration file is:

- a. The current working directory.
- b. The `$HOME` directory of the user.
- c. The PrimeSim HSPICE installation directory.

Figure 12

5. Design input:

The PrimeSim HSPICE tool opens the input netlist file `demo.sp`. If this file does not exist, a `no input data` error message appears in the output listing file.

The PrimeSim HSPICE tool then opens the output listing file `demo.lis` for writing. If you do not own the current directory, the tool terminates with a file open error.

The following is an example of a simple PrimeSim HSPICE input netlist:

```
*Inverter Circuit
.OPTION LIST NODE POST
.TRAN 200P 20N SWEEP TEMP -55 75 10
.PRINT TRAN V(IN) V(OUT)
M1 VCC IN OUT VCC PCH L=1U W=20U
M2 OUT IN 0 0 NCH L=1U W=20U
VCC VCC 0 5
VIN IN 0 0 PULSE .2 4.8 2N 1N 1N 5N 20N
CLOAD OUT 0 .75P
.MODEL PCH PMOS
.MODEL NCH NMOS
.ALTER
CLOAD OUT 0 1.5P
.END
```

6. Library input:

The PrimeSim HSPICE tool reads the files, if any, that you specified in the `.INCLUDE` and `.LIB` statements.

7. Operating point initialization:

The PrimeSim HSPICE tool reads initial conditions, if any, that you specified in the `.IC` and `.NODESET` commands, finds an operating point (that you can save with a `.SAVE` command), and writes any operating point information that you requested.

8. Analysis:

The PrimeSim HSPICE tool performs the specified single or multipoint sweep of the design and produces a set of output files.

9. `.ALTER` commands:

Using `.ALTER` commands, you can vary the simulation conditions and repeat the specified single or multipoint analysis. You can activate multiprocessing while running `.ALTER` cases by entering `hspice -dp` on the command line.

10. Suspending a simulation:

To suspend a simulation job, press Ctrl+Z. The Load Sharing Facility (LSF) then frees up the PrimeSim HSPICE license for another simulation job. To resume the suspended

job, on the same terminal, type either `fg` or `bg`. This command accesses a PrimeSim HSPICE license and continues the suspended simulation.

11. Normal termination:

After you complete the simulation, the PrimeSim HSPICE tool closes the opened files and releases the license tokens used for the simulation.

Starting PrimeSim HSPICE — Examples

The following are more examples of commands to start running PrimeSim HSPICE.

- `hspice demo.sp -n 7 > demo.out`

This command redirects output to a file instead of stdout. `demo.sp` is the input netlist file. The `.sp` extension is optional. The `-n 7` starts the output data file revision numbers at 7; for example: `demo.tr7`, `demo.ac7`, `demo.sw7`, and so forth. The `>` redirects the program output listing to file `demo.out`.

- `hspice -i demo.sp`

`demo` is the root input file name. Without the `-o` argument and without redirection, PrimeSim HSPICE does not generate an output listing file.

- `hspice -i demo.sp -o demo`

`demo` is the output file root name (designated with the `-o` option). Output files are named `demo.lis`, `demo.tr0`, `demo.st0`, and `demo.ic0`.

- `hspice -i rmdir/demo.sp`

`demo` is the input root file name. PrimeSim HSPICE writes the `demo.lis`, `demo.tr0`, and `demo.st0` output files into the directory where you executed the PrimeSim HSPICE command. It also writes the `demo.ic0` output file into the same directory as the input source—that is, `rmdir`.

- `hspice -i a.b.sp`

`a.b` is the root name. The output files are `./a.b.lis`, `./a.b.tr0`, `./a.b.st0`, and `./a.b.ic0`.

- `hspice -i a.b -o d.e`

`a.b` is the root name for the input file. `d.e` is the root output file name, except for the `.ic` file to which PrimeSim HSPICE assigns the `a.b` input file root name. The output files are `d.e.lis`, `d.e.tr0`, `d.e.st0`, and `a.b.ic0`.

- `hspice -i a.b.sp -o outdir/d.e`

PrimeSim HSPICE writes the output files as: `outdir/d.e.lis`, `outdir/d.e.tr0`, `outdir/d.e.st0`, and `outdir/d.e.ic0`.

- `hspice -i indir/a.b.sp -o outdir/d.e.lis`
`a.b` is the root for the `.ic` file. PrimeSim HSPICE writes the `.ic0` file into a file named `indir/a.b.ic0`. `d.e` is the root for the output files.
- `hspice test.sp -o test.lis`
This command creates only a `.lis` output file after simulating the `test.sp` input netlist.
- `hspice -i test.sp -o out1/a.lis`
This command creates the output file in the `.lis` format. The `.lis` file is in the `out1` directory and its root file name is `a`.
- `hspice -i test.sp -o test -x`
This command launches a full parasitic back-annotation for the file named `test.sp`.

Viewing Online Help Topics From the Command-Line

You can use the `-help` option from the command-line to access the online help topics:

Topic Type	Syntax	Example	Notes
Command	<code>hspice -help <i>command</i></code>	<code>hspice -help .AC</code>	When issuing Digital Vector related commands, prefix the command with <code>.VEC_</code>
Control Option	<code>hspice -help .OPTION_<i>option</i></code>	<code>hspice -help .OPTION_DELMAX</code>	You must concatenate the <code>.OPTION</code> and control name with an underscore “ <code>_</code> ” for the help topic to launch.
Demo File	<code>hspice -help <i>demo_file_name</i></code>	<code>hspice -help clockbuf4sn.sp</code>	Specify the name of the demo file with the <code>.sp</code> extension. See Listing of Demonstration Input Files in <i>PrimeSim HSPICE User Guide: Demonstration Netlists</i> .
General	<code>hspice -help <i>keyword</i></code>	<code>hspice -help bsim3v3</code>	See List of Keywords for Online Help Topics

List of Keywords for Online Help Topics

The following table lists the keywords and the corresponding topics that are displayed when you use the given keyword in the `-help` expression:

Keyword	Topic
3DIC	Multi technology simulation of 3D integrated circuits
AC	Using the .AC Statement
AVA	Advanced Variability Analysis
ACMatch	ACMatch Analysis
back-annotation	Post-Layout Back-Annotation
bisection	Timing Analysis Using Bisection
BJT	BJT Models
BJT-L8-v2.34	BJT LEVEL=8 Version 2.34 Added Parameters
BSIM-CMG	BSIM-CMG MOSFET Model
BSIM3-V3	Level 49 and 53 BSIM3v3 MOS Models
BSIM4	BSIM4 Model
BSIM-IMG	BSIM-IMG Model
BSIM-IMG-102.9.1	BSIM-IMG Model version 102.9.1 Updates
DC	.DC Statement—DC Sweeps
DCMatch	DCMatch Analysis
Demos	The PrimeSim HSPICE Demonstration Netlists
DFE-Function	Built in DFE Function for Rx Port Element in StatEye Analysis
Diodes	Diode Models
Docs	The PrimeSim HSPICE Documentation Set
DP	Multiple Simulations, and DP
Element-Templates	Element Template Listings
Exploration-Block	Exploration Block
FFT	.FFT Analysis
Field-Solver	Using the field solver to extract transmission lines
FinFET	BSIM-CMG MOSFET/FINFET Model

Keyword	Topic
HB	Steady-State Harmonic Balance Analysis
HBAC	Multitone Harmonic Balance AC Analysis (.HBAC)
HBOSC	Harmonic Balance Oscillator Analysis (.HBOSC)
HICUM-v2.4	HICUM v2.4 Added Parameters
HiSIM-HV	PrimeSim HSPICE HiSIM-LDMOS/HiSIM-HV Model
HiSIM2	STARC HiSIM2 Model
HiSIM-HV-2.2.3	HiSIM HV Version 2.2.3 Updates
HSMC	High Sigma Monte Carlo Analysis
IBIS	Modeling Input/output Buffers Using IBIS Files
IBIS-AMI	Using IBIS-AMI Equalizer Models with StatEye
IBIS-AMI-Mid-Channel-Repeater	Using an IBIS-AMI Mid-Channel Repeater Model
JFET	JFET Models
JUNCAP2-v200.5	JUNCAP2 Model 200.5 Updates
junction-capacitance	.OPTION EXT_OP
keywords	List of Keywords for Online Help Topics
EKV3-Model	LEVEL 79: EKV3 Model
LIN	LIN Analysis
list-of-demos	Listing of PrimeSim HSPICE Demonstration Files
LSTB	Using .LSTB for Loop Stability Analysis
mc-distribution-modification	Modifying the Built-in Distribution for MC Analysis
MC-plus-corner	MC Analysis with Corners in PrimeSim HSPICE ADE
MC-plus-data-sweep	MC Analysis plus Data Sweep
MC-plus-MOSRA	MC plus MOSRA Analysis in PrimeSim HSPICE ADE
MESFET	MESFET Models
.measure_acphasenoise	.MEASURE ACPHASENOISE Command
.measure_phasenoise	.MEASURE PHASENOISE Command
.measure_ptdnoise	.MEASURE PTDNOISE Command
.measure_acmatch	.MEASURE for ACMatch Analysis

Keyword	Topic
.measure_dcmatch	.MEASURE for DCMatch Analysis
.measure_fft	.MEASURE FFT Command
.measure_1stb	.MEASURE LSTB Command
Monte-Carlo	Monte Carlo—Traditional Flow Statistical Analysis
MOSFET-output-templates	MOSFET Output Templates
MOSRA	MOSFET Reliability Analysis (MOSRA)
mquit	Multiple Runs in Interactive Mode
mruntime	Multiple Runs in Interactive Mode
NOISE	Using .NOISE for Small-Signal Noise Analysis
param	Multiple Runs in Interactive Mode
phase-noise	Phase Noise Analysis (.PHASENOISE)
pole-zero	Pole Zero Analysis
PSP	PSP100 DFM Support Series Model
PSP-103.6	PSP 103.6 Model Updates
reset	Multiple Runs in Interactive Mode
RF-demos	Listing of PrimeSim HSPICE Advanced Analog Demonstration Files
Sigma_Amplification	Sigma Amplification Simulation
S-parameter	S-parameter Modeling Using the S-element
SI	Signal integrity
SN	Steady-State Shooting Newton Analysis
SNAC	Shooting Newton AC Analysis (.SNAC)
SNOSC	Oscillator Analysis Using Shooting Newton (.SNOSC)
SPUTIL	S-parameter Standalone Manipulation Utility (SPutil)
HISIM-v2.9.1	STARC-HiSIM2 MOSFET Model 2.9.1 Updates
StatEye	Statistical Eye Analysis
TFT	TFT Model
TMI-aging-and-self-heating	TMI model aging and self-heating simulation in PrimeSim HSPICE-ADE

Keyword	Topic
TRAN	Transient Analysis
transient-noise	Transient Noise Analysis
UTSOI-v2.2.0	LETI-UTSOI MOSFET Model 2.2.0 Updates
Verilog-A	Using Verilog-A
yield-computation-in-MC	Yield Computation in MC Analysis

Interpreting Default Values of .OPTION in PrimeSim HSPICE

The typical behavior for options is:

- Option not specified: value is default value, typically “OFF” or 0.
- Option specified but without value: typically turns the option “ON” or to a value of 1.

If an option has more than two values allowed, specifying it without a value sets it to 1, if appropriate. In most cases, options without values are allowed only for flags that can be on or off, and specifying the option without a value turns it on. There are a few options (such as POST), where there are more than two values allowed, but you can still specify it without a value. Usually, you should expect it to be 1.

Using the Example Syntax

To copy and paste proven syntax, use the demonstration files shipped with your installation of PrimeSim HSPICE (see Listing of Demonstration Input Files in the *PrimeSim HSPICE User Guide: Demonstration Netlists*). Attempting to copy and paste from the book or help documentation may present unexpected results, as text used in formatting may include hidden characters, and white space for visual clarity.

Using PrimeSim HSPICE for Calculating New Measurements

When you want to calculate new measurements from previous simulation results produced by PrimeSim HSPICE, you can use the following mode to rerun PrimeSim HSPICE without having to do another simulation:

```
hspice -meas measurefile -i wavefile -o outputfile -h -v
```

See the following table for arguments and descriptions:

Argument	Description
<code>-meas</code> <code>measurefile</code>	- The <code>.OPTION</code> command supports the following types: - MEASFAIL - MEASFORM - NUMDGT - INGOLD - MEASDGT This format is similar to the PrimeSim HSPICE netlist format. The first line is a comment line and the last line is an <code>.END</code> command. The following netlist commands are supported: <code>.MEASURE</code> <code>.PARAM</code> <code>.TEMP</code> <code>.OPTION</code> <code>.DATA</code> <code>.ENDDATA</code> <code>.END</code> Note: The <code>.DATA</code> command in the measure file must be consistent with the <code>.DATA</code> command in the waveform file. Warnings are issued if other options or commands are used.
<code>-i wavefile</code>	Can be: <code>*.tr#</code>, <code>*.ac#</code>, <code>*.sw#</code>, <code>*.hb0</code>, <code>*.sn0</code>, <code>*.pn0</code>, or <code>*.snpn0</code> waveform files produced by PrimeSim HSPICE. If a plot fails to open, it is due to one of the following reasons: - Waveform file format is not supported. - File format is not understood. - File is not found. - File larger than the maximum size of x. Note: “x” depends on any file size limitation of your application. Limitations: - For <code>*.hb0</code> files, only the measures containing the keywords <code>max</code>, <code>min</code>, <code>avg</code>, and <code>rms</code> are supported. - For <code>*.sn0</code> files, only FIND-AT measures and the measures containing the keywords <code>max</code>, <code>min</code>, <code>avg</code> and <code>rms</code> are supported. - For <code>*.pn0</code> and <code>*.snpn0</code> files, only the measures containing the keyword <code>integral</code> are supported. <code>-meas</code> is not supported when the original advanced analog analysis contains a sweep analysis.

Argument	Description
<code>-o outputfile</code>	Produces the same output files as PrimeSim HSPICE. In some cases, the results are different from the measure result PrimeSim HSPICE produces due to an accuracy problem. Note: If a sweep transient, AC, or DC analysis is specified, and PSF waveform is also specified, the waveform file contains the extension <code>*@sweep_num#</code> and the measure result file will have the form <code>*mt*@sweep_num#</code> or <code>*ma*@sweep_num#</code> or <code>*ms*@sweep_num#</code> .
<code>-h</code>	Displays a help message and exits.
<code>-v</code>	Outputs version information and exits.

Using Isomorphic Analyses in Subcircuit Blocks

The isomorphic analyses feature enables you to run unrelated analyses (.DC, .AC, and .TRAN) many times during a simulation by grouping the set of analyses into a subcircuit, which performs multiple analyses in one simulation with calls to the subcircuit. The usage model is: Specify the analyses commands within the subcircuit definition block and then instantiate the subcircuit to perform the analyses. Each call of the subcircuit is an individual analysis with its own set of parameters.

The syntax is as follows:

```
.subckt analyses_sb [start=p1 stop=p2 steps=p3]
.DC ...
.AC ...
.TRAN ...
.ends analyses_sb
```

The above syntax is followed by the analysis call:

```
x1 analyses_sb [start=a1] [stop=a2] [steps=a3]
x2 analyses_sb [start=b1] [stop=b2] [steps=b3]
```

For information on the available analyses, see the following list:

- [Chapter 10, Initializing DC-Operating Point Analysis](#)
- The .DC command in the *PrimeSim™ Continuum Reference Manual: Commands and Control Options*
- [Chapter 11, AC Small-Signal and Noise Analysis](#)
- The .AC command in the *PrimeSim™ Continuum Reference Manual: Commands and Control Options*

- [Chapter 12, Transient Analysis](#)
- The .TRAN command in the *PrimeSim™ Continuum Reference Manual: Commands and Control Options*

The following sections discuss these topics:

- [Data Inputs and Outputs](#)
- [Limitations](#)

Data Inputs and Outputs

Input is in the form of a subcircuit block call. You can specify the different analyses inside the .subckt and then call the subcircuit. Each call performs all the specified analyses in the subcircuit.

You generate output files based on the analyses specified. The suffix number increases for every “same type” of analysis.

For example:

```
.subckt analyses_sb start_dc=-25 stop_dc=25 steps_dc=5
+ steps_tran=1n stop_tran=10n
.DC TEMP start_dc stop_dc steps_dc
.TRAN steps_tran stop_tran
.ends analyses_sb
...
x1 analyses_sb start_dc=25 stop_dc=75 steps_dc=10
x2 analyses_sb steps_tran=2n
```

This example creates the output files: *.sw0, *.sw1 and *.tr0, *.tr1.

Isomorphic Analyses Example

The following example shows how to specify different analyses within the .subckt block.

```
.subckt analyses_sb start_dc=-25 stop_dc=25 steps_dc=5
+ steps_tran=1n stop_tran=10n
.DC TEMP start_dc stop_dc steps_dc
.TRAN steps_tran stop_tran
.ends analyses_sb
```

This example specifies both .DC and .TRAN analyses within the .subckt. To invoke these analyses you can call the subcircuits.

```
x1 analyses_sb start_dc=25 stop_dc=75 steps_dc=10
x2 analyses_sb steps_tran=2n
x3 analyses_sb
```

- Each subcircuit call performs DC and Transient analysis.
- Parameters defined in the subcircuit calls override the default values in the subcircuit definition.
- If the subcircuit calls do not define the parameters they take the default values given in the subcircuit.

Limitations

The subcircuit block only supports DC, AC, and transient analyses inside it.

Running PrimeSim HSPICE Simulations on Windows

You can use the MS-DOS command window to run PrimeSim HSPICE in command line mode, similar to UNIX/Linux.

For example:

1. Open an MS-DOS command window (Run > cmd).
2. Enter your case directory.
3. Type the following to invoke PrimeSim HSPICE and view command line help:

```
%\synopsys\Hspice_release_version\bin\hspice
```

4. Or type the following command to run a simulation:

```
%\synopsys\Hspice_release_version\bin\hspice filename.sp -o
```

Running PrimeSim HSPICE Interactively

For a full discussion, refer to [Chapter 5, Using Interactive Mode](#). Interactive mode enables you to use these PrimeSim HSPICE commands at the PrimeSim HSPICE prompt to help you simulate circuits interactively:

ac [...statement]	cd
dc [...statement]	edit
help	info outflag
input	list [lineno]
load filename	ls [directory]
measure [statement]	op

<code>print [tran/ac/dc],v/vm/vr/vi/vp/vdb</code>	<code>pwd</code>
<code>quit</code>	<code>run</code>
<code>save netlist/command filename</code>	<code>set outflag true false</code>
<code>tran [...statement]</code>	

The following sections discuss these topics:

- [Starting Interactive Mode](#)
- [Running a Command File in Interactive Mode](#)
- [Quitting Interactive Mode](#)

Starting Interactive Mode

To invoke the interactive mode, enter:

```
hspice -I
```

You can also use the `help` command at the PrimeSim HSPICE prompt for an annotated list of the commands supported in the interactive mode.

The interactive mode also supports saving commands into a script file. To save the commands that you use and replay them later, enter:

```
hspice> save command filename
```

Running a Command File in Interactive Mode

To run the command you have saved in a command file, enter:

```
hspice> -I -L filename
```

Quitting Interactive Mode

To exit the interactive mode and return to the system prompt, enter:

```
hspice> quit
```

Using PrimeSim HSPICE in Client-Server Mode

When you run many small simulation cases, you can use the client/server mode to improve performance. This performance improvement occurs because you check out

and check in a PrimeSim HSPICE license only once. This is an effective measure when you characterize cells. (For an advanced procedure, see [Launching the Advanced Client-Server Mode \(-CC\).](#))

Table 3 *Supported Platforms*

Linux RHEL	Linux SUSE	Windows
Yes	Yes	Yes

The following sections discuss these topics:

- [Starting Client-Server Mode](#)
- [Simulating a Netlist in Client-Server Mode](#)
- [To Quit Client-Server Mode](#)
- [Launching the Advanced Client-Server Mode \(-CC\)](#)

Starting Client-Server Mode

Starting the client/server mode creates a PrimeSim HSPICE server and checks out a PrimeSim HSPICE license. To start the client/server mode, enter:

```
hspice -C
```

Server

The server name is a specific name connected with the machine on which PrimeSim HSPICE runs. When you create the server, PrimeSim HSPICE also generates a hidden `.hspicecc` directory in your home directory. PrimeSim HSPICE places some related files in this directory, and removes them when the server exits.

PrimeSim HSPICE Client/Server mode does not let one user create several servers on the same machine.

When you create a server, the output on the screen is:

```
*****
*Starting HSPICE Client/Server Mode...*
*****
Checking out HSPICE license...
HSPICE license has been checked out.
*****
*Welcome to HSPICE Client/Server Mode!*
```

After you create the server, it automatically runs in the background. If the server does not receive any request from a client for one hour, the server releases the license and exits automatically.

Client

The client can send a request to the server to determine the PrimeSim HSPICE license status, or to kill the server.

- If the request is to check the license status, the server checks and replies to the client. The syntax of this request is:

```
hspice -C casename.sp
```

Where *casename* is the name of the circuit design to simulate.

- If the client receives `ok`, it begins to simulate the circuit design.
- If the client receives `no`, it exits.
- If the server receives several requests at the same time, it queues these requests, and processes them in the order that the server received them.
- If PrimeSim HSPICE does not find a server, it creates a server first. Then the server checks out a PrimeSim HSPICE license, and simulates the circuit.
- If the request is to kill the server, the server releases the PrimeSim HSPICE license and other sources, and exits.

When you kill the server, any simulation cases in the queue on that server do not run, and the server's name disappears from the hidden `.hspicecc` directory in your home directory.

If you do not specify an output file, PrimeSim HSPICE directs output to the client terminal. Use this syntax to redirect the output to a file, instead of to the terminal:

```
% hspice -C casename.sp output_file
```

The following message is printed in the `*.lis` file when you are running a job in client-server mode:

```
Connect to server hostname:port
```

Simulating a Netlist in Client-Server Mode

Once you have started the client/server mode, which automatically checks out a PrimeSim HSPICE license, you can run simulations. To simulate a netlist in client/server mode, enter:

```
hspice -C path/input_file
```

Note:

This mode also supports other PrimeSim HSPICE command line options.

To Quit Client-Server Mode

Quitting the client/server mode releases the PrimeSim HSPICE license and exits PrimeSim HSPICE. To exit the client/server mode, enter:

```
hspice -C -K
```

Launching the Advanced Client-Server Mode (-CC)

The Advanced Client/Server Mode provides an efficient interface for cell characterization applications. The advanced C/S mode facilitates the Client/Server mode as follows:

- Checks out a PrimeSim HSPICE license once and locks it to do multiple simulations in sequence.
- Reads in the common file only once in multiple simulations with different circuits, when the common file might contain subcircuit or model definition.
- Provides an easy-to-use interface.

Note:

The Client user ID should be the same as the user ID which started the server.

Table 4 Supported Platforms

Linux RHEL	Linux SUSE	Windows
Yes	Yes	No

The following sections present these topics:

- [Advanced Client-Server Command Syntax](#)
- [Application Instances](#)

Advanced Client-Server Command Syntax

The following commands are used to start the PrimeSim HSPICE server, run simulations as a client and stop the server. The following table describes the arguments.

1. To start the server, enter:

```
hspice -CC [-share inc_file] [-port hostname:port_num]
+ [-o output]
+ [-idle idle_time]
```

2. To begin a client simulation, enter:

```
hspice -CC input_file [-port hostname:port_num]
+ [-o output_file]
```

The port information must come *after* the input file information on the command line.

3. To stop the PrimeSim HSPICE server, enter:

```
hspice -CC -K [-port hostname:port_num]
```

Argument	Description
-CC	Launches the advanced PrimeSim HSPICE C/S mode. After the server starts, it runs in background.
-port hostname:port_num	Starts server on the designated port hostname:port_num. If you do not specify this argument, it uses the default port number (25001). If the default port is not available, PrimeSim HSPICE chooses any free port. When you start the server or run a case with -port hostname:port_num, it ignores the hostname. But you can stop the server with -port hostname:port_num to implement remote control. PrimeSim HSPICE prints out port information to the screen.
-share inc_file	Specifies a common file name shared by different circuits.
-idle idle_time	The idle time unit=hour(s). "idle_time" means that the server has not received any client request. If you do not specify an "idle_time", the server quits automatically after the default idle_time of 1 hour.
-o output_file	Specifies the output file name. If you do not specify an "output_file" PrimeSim HSPICE uses the shared file root filename as the output file root filename. If there is no shared file, no output file is generated.
-K	Shuts down the client server.

Application Instances

In the following instances, assume you want to run five netlist files (t1.sp, t2.sp, ... t5.sp), and this group includes a common file. Also assume that all five files are in the same directory: /home1/user1/test/testcase.

```
t1.sp
V1 1 0 dc 1.05
V2 2 0 dc 0
.temp 125
.inc "/home1/user1/test/model/model_file"
```

```
.inc "101.spc"  
.end  
  
t2.sp  
  
V1 1 0 dc 1.05  
V2 2 0 dc 0  
.temp 125  
.inc "/home1/user1/test/model/model_file"  
.inc "102.spc"  
.end
```

...

```
t5.sp  
  
V1 1 0 dc 1.05  
V2 2 0 dc 0  
.temp 125  
.inc "/home1/user1/test/model/model_file"  
.inc "105.spc"  
.end
```

Use the following commands, to invoke the PrimeSim HSPICE C/S mode to run this group of cases. The work path is: /home1/user1/test/testcase

1. Start the PrimeSim HSPICE server on the default port and read in the common file:

```
hspice -CC -share /home1/user1/test/model/model_file
```

2. Run a simulation on the default port without reading in the common file again.

```
hspice -CC t1.sp
```

Since the `.inc "/home1/user1/test/model/model_file"` statement appears in each netlist, PrimeSim HSPICE does not read it in again because the server has already processed the information.

3. Repeat Step 2 until you simulate all cases.
4. Exit the PrimeSim HSPICE Client/Server mode.

```
hspice -CC -K
```

The sequence of commands is:

```
hspice -CC -share /home1/user1/test/model/model_file  
hspice -CC t1.sp  
hspice -CC t2.sp  
hspice -CC t3.sp  
hspice -CC t4.sp  
hspice -CC t5.sp  
hspice -CC -K
```

Notes

- If you start the server and run simulations by a Perl script, use the system (\$cmd) instead of ' \$cmd' to avoid hanging the server.

For example:

```
#!/usr/bin/perl
##Start server without designated port, redirect output
##information
$cmd = "hspice -CC >& log ";
system($cmd);
```

- To use multiple servers, you need to specify multiple ports. If you submit several scripts to start multiple servers, you need to specify multiple ports. If you do not designate port numbers to a multiple-cpu machine or to a machine in computer farm environment, only one server starts on the default port number. If the default port is not available, PrimeSim HSPICE chooses any free port. PrimeSim HSPICE also prints out port information, "Server is started on port port_num". To assure that the simulation runs successfully in a different script, add -port \$port_num.

For example:

```
#!/usr/bin/perl -w
##start server without designated port, redirect output
##information
$cmd = "hspice -CC >& log ";
system($cmd) ;
##get the port_num on which server is started
open(IN,"log");
while(<IN>){
    if(/^Server is started on/){
        @fld=split;
        $portnum=$fld[4];
    }
}
close(IN);
##do simulation
$cmd1 = "hspice -CC test1.sp -port $portnum";
system($cmd1) ;
...
##stop server
$cmdn ="hspice -CC -K -port $portnum";
system($cmdn) ;
```

- To avoid redefinition errors, verify that the common file both in "-share inc_file" and in ".inc inc_file" of every netlist that has the same absolute path and file name. For example, there are 5 netlist files, t1.sp, t2.sp, t3.sp, t4.sp, and t5.sp, to be run and this group of netlists includes a common file. Assume that all these 5 files are in the same directory /home1/user1/test/testcase.

The following is the correct usage:

```
hspice -CC -share /home1/user1/test/model/model_file
hspice -CC t1.sp
hspice -CC t2.sp
hspice -CC t3.sp
hspice -CC t4.sp
hspice -CC t5.sp
hspice -CC -K
```

Each of the netlists includes `.inc "/home1/tom/test/model/model_file"`. In every case, the absolute path name of the common file in `.inc "/home1/user1/test/model/model_file"`... is the same as the absolute path name of the common file specified by `-share /home1/user1/test/model/model_file`.

The common file `/home1/user1/test/model/model_file` will only be read in once and `.inc "/home1/user1/test/model/model_file"` will be ignored in every case.

Running PrimeSim HSPICE to Calculate New Measurements

To calculate new measurements from previous simulation results produced by PrimeSim HSPICE, you can rerun PrimeSim HSPICE.

To get new measurements from a previous simulation, enter:

```
hspice -meas measure_file -i wavefile [-o outputfile]
```

To execute the wildcard characters for input and output files, use:

```
hspice -meas meas.sp -i sim.tr* -o simout'
```

The wildcard `sim.tr*` is required for input waveform index. The `'simout` is a prefix string for output files. PrimeSim HSPICE automatically generates the correct output file from the corresponding input file with the index.

4

Distributed Processing and Multithreading

Describes the distributed processing (DP), and multithreading. Although, multiprocessing is functionally replaced by distributed processing, you can still run multiprocessing.

For descriptions of individual PrimeSim HSPICE commands mentioned in this chapter, see the *PrimeSim™ Continuum Reference Manual: Commands and Control Options*.

The following sections discuss these topics:

- [Predicting the Simulation Time for Monte Carlo Cases](#)
- [Running Multiple Simulations](#)
- [Running Distributed Processing on a Network Grid](#)
- [Running Distributed Processing on One Machine](#)
- [Running Multithreaded Simulations](#)
- [Running Multithreading and Distributed Processing Concurrently](#)
- [PrimeSim HSPICE Precision Parallel](#)

Predicting the Simulation Time for Monte Carlo Cases

The following sections discuss these topics:

- [Simulation Time Prediction Utility](#)
- [Example](#)
- [Limitations](#)

Simulation Time Prediction Utility

Use the following syntax to predict the simulation time, without finishing the whole simulation, for Monte Carlo cases:

```
hspice -dpredict -i input -o output
```

The PrimeSim HSPICE Prediction Utility then predicts the simulation time and prints the prediction summary both on the terminal and in the *.lis file.

Example

Predicting the simulation time of a Monte Carlo case:

```
hspice -dpredict -i input.sp -o input
```

The following prediction summary is printed on the terminal:

```
*** -dpredict usage is turned on ***
Input File:                      input.sp
.....
Predicted CPU Time:                533.62 hours
Predicted Elapsed Time on 100 cores: 5.34 hours
```

The prediction information in the *.lis file:

```
***** Prediction Information *****
Predicted CPU Time :    533.62(h)
Predicted Elapsed Time on 100 cores : 5.34 hours
```

Limitations

The following limitations apply to the simulation time prediction utility in PrimeSim HSPICE:

- Can be used only for Monte Carlo simulation cases.
- Cannot be used for Monte Carlo cases with external sampling method.

- Cannot be used for Monte Carlo cases with `.ALTER`.
- Cannot be used for running cases in DP mode.

Running Multiple Simulations

Use the following syntax to start PrimeSim HSPICE depending on the type of multiple or parallel processes available on your site:

```
hspice [-dp [num]
+       [-dpconfig dp_configuration_file]
+       [-dplocation NFS|TMP]
+       [-merge]
+       [-dpgui]
+       [-dpmode alter|sweep]
+       [-dpincremental original_path]

+       [-dplicense 1|0]]
+       [-mt thread_count]
+       [-hpp]
```

Caution:

Multithreading and PrimeSim HSPICE Precision Parallel are available on Windows platform, but not distributed processing.

For information on running batch file jobs from the PrimeSim HSPICE user interface, see [Running Multiple Simulations on page 132](#).

Running Distributed Processing on a Network Grid

The following sections discuss these topics:

- [Distributed Processing in PrimeSim HSPICE](#)
- [Examples](#)
- [Feature Support for DP](#)
- [Output Files](#)
- [Limitations](#)

Distributed Processing in PrimeSim HSPICE

You can submit a PrimeSim HSPICE simulation job to a network of machine for parallel processing on the following network grids: SGE, LSF, RTDA, RSH, and

SSH. These machine cluster/compute farm manager modes are controlled by the *dp_configuration_file*.

Distributed processing (DP) uses the Synopsys Common Distributed Processing Library (CDPL). For Synopsys DP, the PrimeSim HSPICE wrapper sets up `$CDPL_HOME` automatically. If you run the PrimeSim HSPICE binary directly, set the following environment variable before invoking DP:

```
setenv CDPL_HOME $INSTALLDIR/hspice/cdpl
```

The supported platforms (64-bit) are:

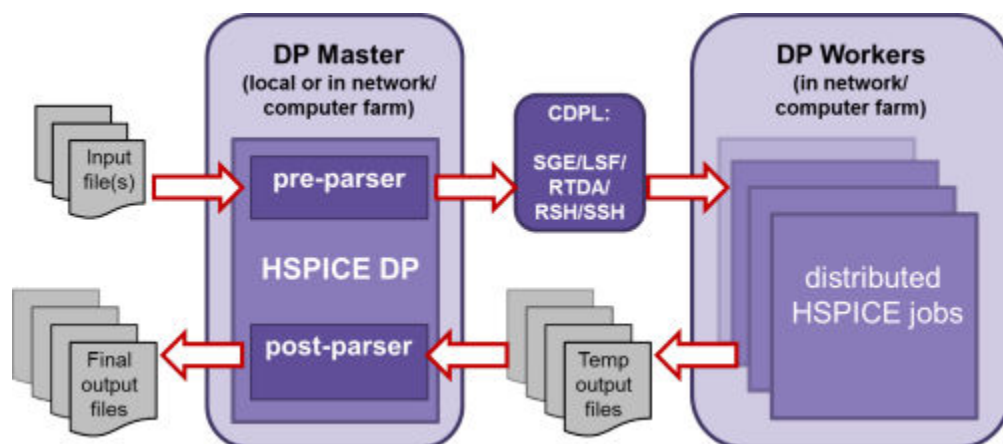
Linux RHEL	Linux SUSE	Windows
Yes	Yes	No

Typical configuration files have the following forms, depending on the type of network grid:

	#flag	hostname	slots	tmpDir	protocol	command
SGE	1		-1	/tmp	SGE	qsub -P bnormal
LSF	1		-1	/tmp	LSF	bsub -q bnormal
RTDA	1		-1	/tmp	RTDA	nc run -e "SNAPSHOT"
NetBatch	1		-1	/tmp	NB	nbgjob run
RSH	1	host1	2	/tmp	RSH	rsh
	1	host2	4	/tmp	RSH	rsh
SSH	1	host1	3	/tmp	SSH	ssh
	1	host2	3	/tmp	SSH	ssh

Where, the **flag** column posts the value **1** if the host can be used as a worker or **0** if it cannot. The **slots** column notes the number of workers that can be started on the particular host and has a value of **-1** for grid types SGE, LSF, RTDA, and NetBatch.

For more information, refer to the *CDPL User Guide*. The following is the illustration of DP flow:



CDPL flow terminology:

- A PrimeSim HSPICE job starts on the *master* (local machine or a farm machine).
- *Workers* may be on the same or different hosts than the master.
- *Master* does pre-parsing, job distributing, and post-parsing. *Workers* run the simulation jobs.
- The simulation is performed in parallel, but the initial parse and the final file merge/summary steps are executed in scalar mode.

Using DP reduces runtime for sequential simulations such as Monte Carlo, `.ALTER`, and sweep with no loss in accuracy. For example, solving 1 million Monte Carlo points takes approximately the same time as 10000 Monte Carlo points (plus the time for results integration) if 100 cores are used.

For more information on starting DP from the PrimeSim HSPICE command-line prompt, see the [Invoking PrimeSim HSPICE Simulations](#).

Caution:

PrimeSim HSPICE DP uses the same license keys as PrimeSim HSPICE. Each process running on the machine cluster checks out one copy of the license key. PrimeSim HSPICE DP turns on `META_QUEUE=1` automatically, so that, if there are no licenses available, workers will wait in queue, instead of quitting the jobs.

Examples

Example 1

Distributing a job into 10 processes on an SGE farm.

```
hspice -dp 10 -dpconfig sge.cfg -i input -o output
```

```
sge.cfg file example:  
1||-1|/tmp|SGE|qsub -P bnormal -V -cwd -o JOB_NAME.out -e JOB_NAME.err -l  
arch=glinux, os_version=WS6.0
```

Where *JOB_NAME* specifies the name of your job.

Example 2

Distributing a job into 10 processes on an LSF farm; each process runs two threads.

```
hspice -dp 10 -dpconfig lsf.cfg -mt 2 -i input -o output
```

```
lsf.cfg file example:  
1||-1|/tmp|LSF|bsub -q bnormal -n 2 -R "arch==glinux &&  
os_version==WS6_0"
```

Example 3

Distributing a job into 10 processes on two machines with RSH protocol, with four processes on host1, six processes on host2.

```
hspice -dp 10 -dpconfig rsh.cfg -i input.sp -o output
```

```
rsh.cfg file example:  
1|host1|4|/tmp|RSH|rsh  
1|host2|6|/tmp|RSH|rsh
```

Feature Support for DP

PrimeSim HSPICE DP supports the following features for .AC, .DC, and .TRAN analyses:

- .ALTER blocks
- Monte Carlo
- Bisection analysis/optimization with Monte Carlo
- Data block
- Parametric sweep
- StatEye
- .AC|.DC sweep1 sweep2
- The /tmp directory is the default location to store output files temporarily, and those files will be moved back to the current run directory when the simulation finishes. Use -dplocation NFS to change the store location to the current run directory. This option allows users to see results at an early stage, but it may slow down performance. If the /tmp directory space is not enough for a simulation, such as a large Monte Carlo job, use this option to avoid the /tmp disk from becoming full.

- Use `-dpgui` to launch DP Manager, an interactive graphic tool to monitor DP status. Refer to the DP Manager User Guide for more details.
- PrimeSim HSPICE automatically turns on `.OPTION WARN_SEP` when running DP to separate warnings into a file, while suppressing them in the `*.lis` file.
- PrimeSim HSPICE DP follows the `.OPTION MCBRIEF` set by the user; if this option is not set, PrimeSim HSPICE DP sets it to 0.

Note:

When running a Monte Carlo analysis with `Option large_scale_mc=yes`, `MCBRIEF` is automatically set to 5.

- PrimeSim HSPICE provides load balance for Monte Carlo analysis, data block, and parametric sweeps:
 - The total tasks a DP worker processed varies machine speed. Workers located on *fast* machines process more tasks. Workers located on *slow* machines process fewer tasks.
 - For Monte Carlo DP, Monte Carlo sample is divided into sub-sequences with sample count in decreasing order. This guarantees the process in full speed. Use `option dp_task_method=2` to equally divide sample count into tasks.
 - Worker has resource monitor during processing. Under following occasions worker is forced shut down:
 - Its memory usage is above 99% or less than 500MB memory left on the machine.
 - Disk is full.
 - The load on single core has been over 4 for 15 minutes. Using the `-dpworkerchk` command to turn on/off these features.
 - Assumption has been made for Monte Carlo DP that each sample takes approximately same time to conclude. DP is able to detect samples that do not conclude in long time and skip these samples, for the purpose of speeding up whole process. Use the `-dpworkerchk` command to turn on or off this feature.
- `.AC` frequency sweep

Note:

PrimeSim HSPICE DP supports running AC analysis and its sub-analyses (`.LSTB` and `.LIN`) frequency sweep. Such AC analysis cannot include `.ALTER`, data sweep, or Monte Carlo analysis. If including these, PrimeSim HSPICE automatically runs `.ALTER`, sweep, or Monte Carlo DP.

For AC + .LSTB DP, you must add .PROBE AC LSTB or .PROBE AC LSTB(*lstb_name*) (for multiple LSTB analyses) to get the LSTB measure results.

See also [Table 8](#).

- SN / SNOSC Monte Carlo analysis

Note:

Only SN/SNOSC Monte Carlo analysis is supported. Such analysis cannot include .ALTER or data sweep.

See also [Table 9](#).

- Transient noise analysis

See also [Table 10](#).

- StatEye now deploys distributed processing for the initial edge response transient.

See also [Table 11](#).

Output Files

The following table lists the names of the possible output files generated for .ALTER, Monte Carlo, data block, and parametric sweep simulations. They may not appear at the same time as the output file generation depends on the type of simulation and the -merge option. See [Table 6](#) and [Table 7](#) for more details.

Table 5 Output File Descriptions

Output File Name	Comments
output_dp/output.dp_parseonly_report	Contains the information that the DP pre-parser collects; the PrimeSim HSPICE tool controls the DP behavior according to its content.
output_dp/task#/output.lis	Without -merge, the *.lis for each task remains in the worker subdirectory.
output.lis	Merged from all the *.lis files generated by all workers.
output_dp.lis	New file created by PrimeSim HSPICE DP. It contains parts of the errors and warnings generated by the simulation, the DP progress status, Monte Carlo results, and DP run-time statistics.
output.st0	Merged from all the *.st0 files generated by all workers.
output.mt#(ALTER)	The *.mt# files generated by all the .ALTER blocks.

Table 5 *Output File Descriptions (Continued)*

Output File Name	Comments
output.mt0	Merged from all the *.mt0 files generated by all workers.
output.mc# (ALTER)	The *.mc# files generated by all the .ALTER blocks.
output.mc0	Merged from all the *.mc0 files generated by all workers.
output.tr#	The waveform files generated by all the .ALTER blocks.
output_dp/worker#/output.tr0	The waveform file generated by each worker for Monte Carlo, data block, or parametric sweep.
output.tr0@mc.grp	A waveform group file for Monte Carlo. When using WaveView to open this file, the tool can load all the output_dp/worker#/output.tr0# files automatically. Use version G-2012.06 or later of WaveView for maximum compatibility.
output.tr0@ds.grp	A waveform group file for data block or parametric sweep. When using WaveView to open this file, the tool can automatically load all the output_dp/worker#/output.tr0 files. Use version G-2012.06 or later of WaveView for maximum compatibility.
\$HOME/.synopsys/cdpl/master.hostname.pid.timestamp.bcast	A broadcasting file used by DP Monitor.
output_dp/master.hostname.pid.err	An error message file for the DP master.
output_dp/master.hostname.pid.log	A log file for the DP master.
output_dp/worker.W#.hostname.pid.log	Log files for DP workers.

Table 6 *Output Files Without “-merge” Option*

.ALTER Block	Monte Carlo, Data Block, Parametric Sweep
output_dp/output.dp_parseonly_report	output_dp/output.dp_parseonly_report
output_dp/worker#/output.lis	output_dp/worker#/output.lis
output_dp.lis	output_dp.lis
output.mc#	output.mc0
output.st#	output_dp/worker#/output.st0
output.mt#	output_dp/worker#/output.mt0 output.mt0 (for Monte Carlo DP)

Table 6 *Output Files Without “-merge” Option (Continued)*

.ALTER Block	Monte Carlo, Data Block, Parametric Sweep
output.tr#	output_dp/worker#/output.tr0 output.tr0@mc.grp (Monte Carlo only) output.tr0@ds.grp (data block or parametric sweep only)
\$HOME/.synopsys/cdpl/master.hostname.p id.timestamp.bcast	\$HOME/.synopsys/cdpl/master.hostname.pid.t imestamp.bcast
output_dp/master.hostname.pid.err	output_dp/master.hostname.pid.err
output_dp/master.hostname.pid.log	output_dp/master.hostname.pid.log
output_dp/worker.W#.hostname.pid.log	output_dp/worker.W#.hostname.pid.log

Table 7 *Output Files With “-merge” Option*

.ALTER Block	Monte Carlo, Data Block, Parametric Sweep
output_dp/output.dp_parseonly_report	output_dp/output.dp_parseonly_report
output_dp.lis	output_dp.lis
output.lis	output.lis
output.st0	output.st0
output.mt#	output.mt0
output.mc#	output.mc0
output.tr#	output.tr0
\$HOME/.synopsys/cdpl/master.hostname.p id.timestamp.bcast	\$HOME/.synopsys/cdpl/master.hostname.pid. timestamp.bcast
test_dp/master.hostname.pid.err	test_dp/master.hostname.pid.err
test_dp/master.hostname.pid.log	test_dp/master.hostname.pid.log
test_dp/worker.W#.hostname.pid.log	test_dp/worker.W#.hostname.pid.log

Table 8 *Output Files for AC Frequency Sweep*

output_dp/output.dp_parseonly_report
output_dp.lis

Table 8 *Output Files for AC Frequency Sweep (Continued)*

<code>output.lis</code>
<code>output.st0</code>
<code>output.ma0</code>
<code>output.ac0.grp</code>
<code>output.sc0 (.LIN output)</code>
<code>output.s#p (.LIN output)</code>
<code>output.citi0 (.LIN output)</code>
<code>output_dp/master.hostname.pid.log</code>
<code>output_dp/worker.W#.hostname.pid.log</code>

Table 9 *Output Files for SN/SNOSC DP*

<code>output_dp/output_dp_parseonly_report</code>
<code>output_dp.lis</code>
<code>output.mpp0</code>
<code>output.msn0</code>
<code>output.msnf0</code>
<code>output.msnnois0</code>
<code>output.sn0@mc.grp</code>
<code>output.snf0@mc.grp</code>
<code>output.snpn0@mc.grp</code>
<code>output_dp/worker#/output.tr0</code>
<code>output_dp/worker#/output.sn0</code>
<code>output_dp/worker#/output.snf0</code>
<code>output_dp/worker#/output.snpn0</code>
<code>output_dp/master.hostname.pid.log</code>

Table 10 *Output Files for Multi-Sample TRANNOISE DP*

```

output_dp/output.dp_parseonly_report

output_dp.lis

output.mt0

output.tr0@mc.grp

output.trjt0@mc.grp

output.trpn0@mc.grp

output.trac0@mc.grp

output_dp/worker#/output.tr0

output_dp/worker#/output.trjt0

output_dp/worker#/output.trpn0

output_dp/worker#/output.trac0

output_dp/master.hostname.pid.log

```

Table 11 *Output Files for StatEye DP*

```

output_dp/output.dp_parseonly_report

output_dp.lis

output.lis

output.st0

output.stet0

output.stev0

output.printSte0

output.mset0

output.p#e#.tr0

output_dp/worker#/output.save0

output_dp/worker#/output.st0

```

Table 11 *Output Files for StatEye DP (Continued)*

```
output_dp/worker#/output.warnlog
output_dp/worker#/output.lis
output_dp/worker#/output.ic0
```

Limitations

The following limitations apply to DP in the PrimeSim HSPICE tool:

- The PrimeSim HSPICE tool ignores DP when a simulation has the following:
 - Multiple `.TRAN`, `.DC`, or `.AC` statements and no `.ALTER` block
 - Temperature sweeps and no `.ALTER` block
 - `.LOAD` command with `RUN=PREVIOUS`
 - TMI ageing `.OPTION TMIAGE` to 1
 - `.OPTION RADEGFILE` (MOSRA analysis)
 - Advanced analog features
 - The PrimeSim HSPICE tool ignores `.OPTION ALTCC` during a DP run.
-

Running Distributed Processing on One Machine

DP runs not only on a network, but also on one multicore machine. When running on one multicore machine, processes are only distributed on one machine. DP is functionally the same as the earlier multiprocessing (MP) technology, but it has better efficiency.

When using DP on a network, the configuration option (`-dpconfig`) is not needed for DP on a multicore machine. This will make the DP usage exactly the same as earlier MP: simply replace `-mp` with `-dp`.

Example 4

Distributing a job into four processes on a local 8-core machine; each process uses two threads and 8 cores.

```
New Usage: hspice -dp 4 -mt 2 -i input -o output
Old Usage: hspice -mp 4 -mt 2 -i input -o output
```

Example 5

Distributing a job into all processes on a local machine; the process count can be omitted.

New Usage: `hspice -dp -i input -o output`
Old Usage: `hspice -mp -i input -o output`

Running Multithreaded Simulations

PrimeSim HSPICE can use multiple cores or processors within a single node using multithreading for model evaluations and matrix solving to significantly improve simulation performance. In circuits composed primarily of active devices, model evaluation takes the most time. For designs with many linear resistors and capacitors (such as post-layout circuits), matrix solving takes most of the time. Using multithreading will improve the performance of both model evaluation and matrix solving.

Note:

- One license is required per two cores.
- If the specified number of threads is larger than the number of cores, multithreading will reduce the number of threads to the number of cores. For example, if you specify "`-mt 12`" on an 8-core machine, PrimeSim HSPICE will issue a warning and change this to "`-mt 8`".

To determine the number of available processors and cores:

Platform	Action
Linux	Examine the processor, physical ID, and core ID flags in the <code>/proc/cpuinfo</code> file.
Windows	Invoke the Task Manager, click the Performance tab, and count the number of CPU Usage History graphs.

For example, running four threads (two licenses are needed):

```
hspice -mt 4 -i input.sp -o output
```

The following sections discuss these topics:

- [Performance Improvement Estimations](#)
- [Multithreading on Windows](#)
- [Impact of Intel's Hyper-Threading Technology](#)

Performance Improvement Estimations

Starting to understand the improvement in multithreaded performance from the following equation:

$$T_{mt} = T_{serial} + T_{parallel}/N_{cpu} + T_{overhead}$$

where:

- T_{serial} represents PrimeSim HSPICE calculations that do not thread.
- $T_{parallel}$ represents threaded PrimeSim HSPICE calculations.
- N_{cpu} is the number of CPUs used.
- $T_{overhead}$ is the overhead from multithreading. Typically, this represents a small fraction of the total run-time.

For example, for a 151-stage NAND ring oscillator using LEVEL 49, $T_{parallel}$ is about 80% of T_{1cpu} (the CPU time associated with a single CPU) if you run with two threads on a multi-CPU machine. Ideally, assuming $T_{overhead}=0$, you can achieve a speedup of:

$$T_{1cpu} / (0.2T_{1cpu} + 0.8T_{1cpu}/2_{cpus}) = 1.67$$

The typical $T_{parallel}$ value is 0.6 to 0.7 for moderate-to-large circuits.

Running more threads does not necessarily mean the simulation would be faster. If there are not enough active devices or elements, a small number of threads sometimes may run faster.

Multithreading on Windows

PrimeSim HSPICE multithreading can be used on Windows. The maximum possible number of threads is the number of cores, even if you specify a larger number in the PrimeSim HSPICE user interface. Using multithreading with hyperthreading is not recommended because no performance gain will be obtained from hyperthreading. Check with the BIOS vendor to see how to turn off hyperthreading.

On the Windows platform, from the PrimeSim HSPICE user interface (HSPUI):

1. Select the correct `hspice.exe` version in the **Version** box.
2. Select **MultiCpu Option** and select the mt number.
3. Select the input case and run.

Impact of Intel's Hyper-Threading Technology

Intel's Hyper-Threading (HT) technology is especially effective when each thread performs different types of operations and when there are under-utilized resources on the processor. However, PrimeSim HSPICE fits neither of these criteria. The threaded portions of the PrimeSim HSPICE code mostly perform identical operations of heavy floating-point computations on each thread and execute at high efficiencies using most of the available resources.

You can obtain higher performance by disabling the HT technology. If you run with HT enabled, the performance of multithreading can be impacted if you run fewer threads than the physical cores. Multithreading performance is also impacted if there are two threads assigned to one physical core, by the thread scheduler, while the other cores are ignored.

Running Multithreading and Distributed Processing Concurrently

You can run multithreading and DP at the same time. Multithreading can speed up large circuit simulation while DP can speed up multijob simulations (.ALTER, Monte Carlo, and data sweep). If you have a large circuit with multiple jobs to run, multithreading plus DP can greatly boost the performance.

Note:

You can run both DP and multithreading simultaneously only on Linux platforms.

The following is the command usage for concurrent multithreading and DP:

```
hspice -dp process_count
      [-dpconfig configuration_file]
      -mt thread_count
      -i input.sp
      -o output_file
```

where:

- *process_count* is a nonzero integer, which defines the initial value of available child process numbers to fork.
- *thread_count* is the maximum number of threads used in model evaluation and matrix solving. Every process in DP uses the same number of threads defined here.
- *-dpconfig configuration_file* is optional and can be omitted if the job is running on a local machine.

Note:

The number of licenses checked out equals the number of threads divided by 2 and multiplied by the number of DPs. For example, `-dp 10 -mt 4` will check out 20 licenses.

DP takes priority in CPU resource allocation. PrimeSim HSPICE triggers multithreading only if a remainder of CPU resources after DP's *process_count* is satisfied.

If the number of cores available for multithreading is not sufficient (after `-dp`) for the specified thread count, PrimeSim HSPICE lowers the value of the thread count to that of available cores.

Example 1:

```
hspice -dp 6 -mt 4 -i input.sp -o input
```

Core=8

Monte=1000

Total multiprocessed jobs=6

Thread count reset to 1 because only 2 cores left and must have equal threads across processes.

Example 2:

```
hspice -dp 6 -mt 4 -i input.sp -o input
```

Core=18

Monte=1000

Total multiprocessed jobs=6

Thread count reset to 3 since there are sufficient cores to have 3 threads per process.

Example 3:

```
hspice -dp 2 -mt 4 -i input.sp -o input
```

Core=4

Monte=1000

Total multiprocessed jobs=2

Thread count is reset to 2.

PrimeSim HSPICE Precision Parallel

The following sections discuss these topics:

- [Introduction to HPP](#)
- [HPP Status Reporting in *.lis File](#)
- [HPP Limitations](#)

Introduction to HPP

PrimeSim HSPICE takes advantage of the latest multicore technology to speed up prelayout and post-layout circuit transient simulations especially for medium-to-large size blocks including PLLs, ADCs, DACs, SERDES, and other full mixed-signal circuits of over 10 million elements.

PrimeSim HSPICE Precision Parallel (HPP) scales performance from one to eight cores and beyond with no loss in accuracy. For a short demonstration of HPP, go to the [PrimeSim HSPICE page](#) on the Synopsys website and click **Videos > Introducing PrimeSim HSPICE Precision Parallel Technology**.

HPP can be activated by inserting `-hpp` on the command line for a single-core run, and it can be combined with multithreading to speed up transient analysis simulations even further by inserting `-hpp` and `-mt N` on the command line, where *N* is the number of threads. HPP is a transient engine, so it will not help with `.AC`, `.DC`, or other analyses.

When running the `-hpp` multicore algorithm with multithreading, every two threads require one PrimeSim HSPICE license.

For example:

```
hspice -mt 8 -hpp -i input.sp -o output.lis
```

Note:

Four PrimeSim HSPICE licenses are used.

Table 12 Supported Platforms

Linux RHEL	Linux SUSE	Windows
Yes	Yes	Yes

Note:

On Windows platforms, HPP scalability depends on the machine configuration. Personal laptops show poor scalability, while Windows server with Xeon could achieve higher scalability.

HPP Status Reporting in *.lis File

When you run HPP, the *.lis file shows running updates for the simulation status and the CPU utilization.

For example:

```
98.8% time = 34.597003 us ( etc = 56.4 sec, ett = 2:16:41 )  
( wall = 2:15:44 cpu = 1:02:14:45 s=11.6008 )
```

where:

% = Percentage of simulation completion
time = Transient time (seconds)
etc = Estimated time to completion (seconds)
ett = Estimated total time
wall = Wall-clock time
cpu = Total CPU time
s = Number of CPUs used

HPP Limitations

The following features and commands are *not* supported:

- Transient noise analysis-HPP does not support the stochastic differential equation (SDE) method.
- IBIS: .ICM

In HPP, templates are supported only for the following models:

- BSIM3
- BSIM4
- PSP
- BSIM-CMG

In HPP, the IVTH feature and LX142() are supported only for the following models:

- BSIM4
- BSIM-CMG
- MOS101

5

Using Interactive Mode

This chapter describes how to use the interactive mode in PrimeSim HSPICE.

Note:

PrimeSim HSPICE advanced analog analyses do not support the interactive mode.

PrimeSim HSPICE ships hundreds of examples for your use; see Listing of Demonstration Input Files in the *PrimeSim HSPICE User Guide: Demonstration Netlists* for paths to the demo files.

The following sections discuss these topics:

- [Invoking PrimeSim HSPICE in Interactive Mode](#)
- [Executing an Interactive Script](#)
- [Multiple Runs in Interactive Mode](#)
- [Using Interactive Mode on the Windows Platform](#)
- [Examples](#)

Invoking PrimeSim HSPICE in Interactive Mode

Start an interactive mode by using the `hspice` executable in the `$installdir/arch` directory.

To start the interactive mode, from the command prompt, type:

```
% hspice -I  
HSPICE >
```

The interactive environment functions from a special HSPICE-shell. You can use many commands while in this environment to simplify your design work.

You quit an interactive mode session by entering:

```
HSPICE > quit
```

Executing an Interactive Script

You can also use interactive commands in a command script file. To execute an interactive script, from the command prompt, type:

```
% hspice -I -L command_script
```

Where *command_script* is the name of the file containing the interactive commands.

Multiple Runs in Interactive Mode

PrimeSim HSPICE now supports multiple runs without re-parsing the input netlist, in the interactive mode. You can enter the multiple-run mode using the `mruncmd` command. Before every run, you can update circuit parameters using the `param` command and update model and MOSFET element parameters using the `reset` command. You can quit the multiple-run mode with the `mquit` command.

PrimeSim HSPICE provides the following commands to support multiple runs in the interactive mode:

- `mruncmd`

Allows you to enter the multiple-run mode and run simulations interactively.

```
hspice > mruncmd
```

The first `mruncmd` simulation parses the netlist and runs the simulation using the original circuit parameter values. You can then update the circuit parameters using the `param` and `reset` commands, without re-parsing the netlist. Later `mruncmd` commands run the simulation using the updated circuit parameters.

- `reset`

Use to change model/instance parameters, but not re-run the netlist.

```
hspice>  
  reset model_name:model_type model_param_name1=value1 model_param_name  
2=value2  
hspice> reset instance_name  
  instance_param_name1=value1 instance_param_name2=value2
```

Note:

Currently, you can change model parameters using `reset` for models above or equal to Level 54. Additionally, the `reset` command supports changing model parameters for only the NMOS and PMOS model types.

- `param`

Use to change `.param` values, but not re-run the netlist.

```
hspice> param param_name=value
```

- `mquit`

Use to quit the multiple-run mode.

```
hspice> mquit
```

For a multiple-run example, see [Multiple Runs Without Re-parsing the Input Netlist](#).

Using Interactive Mode on the Windows Platform

To start Windows Interactive Mode, open a Windows Command Prompt Launch the command prompt window by entering `cmd` into **Start > Run** or through **Start > All Programs > Accessories**. From the command prompt, type:

```
C: \>hspice -I
```

The interactive environment functions from a special PrimeSim HSPICE shell. You can use the same commands as you do under UNIX. For example:

```
HSPICE > load case.sp  
HSPICE > run
```

To quit an interactive mode, enter:

```
HSPICE > quit
```

Examples

The examples in this section show you how to use the interactive environment commands.

- [Getting Help](#)
- [Creating a Netlist](#)
- [Specifying an Analysis](#)
- [Running an Analysis](#)
- [Viewing a Netlist](#)
- [Loading and Running an Existing Netlist](#)
- [Using Environment Commands](#)
- [Recording and Saving Interactive Commands to a File](#)

- [Printing a Voltage Value During Simulation](#)
- [Using a Command File to Run in Interactive Mode](#)
- [Running Multiple Testcases](#)
- [Multiple Runs Without Re-parsing the Input Netlist](#)

Getting Help

You use the `help` command to show the interactive mode syntax; for example,

```
% hspice -I
HSPICE > help

list [lineno]
input
edit
ls [directory]
load filename
run
pwd
cd directory
info outflag
set outflag true/false
save [netlist/command] filename
quit
help
dc [...statement]           (as in the netlist)
ac [...statement]           (as in the netlist)
tran [...statement]         (as in the netlist)
op
measure [...statement]      (as in the netlist)
print [tran/ac/dc] v/vm/vr/vi/vp/vdb
```

Creating a Netlist

Use the `input` command to create a netlist by using the `vi` text editor; for example,

```
% hspice -I
HSPICE > input
R1 1 0 2
V1 1 0 3
.print I(R1)
.end
```

Save the file and exit `vi`.

Specifying an Analysis

Use the `ac`, `dc`, or `tran` command to specify an analysis; for example,

```
HSPICE > dc v1 -5v 5v 0.5v
```

Running an Analysis

Use the `run` command to simulate a netlist; for example,

```
HSPICE > run > info:  ***** hspice job concluded
```

PrimeSim HSPICE outputs the simulation results. This output is equivalent to a `.lis` file.

Viewing a Netlist

Use the `list` command to view a netlist; for example,

```
HSPICE > list
1  * this is an interactive mode example
2
3  R1 1 0 2
4  V1 1 0 3
5  .print I(R1)
6  .end
```

Loading and Running an Existing Netlist

Use the `load` command to read an existing netlist; for example, to load a netlist named `"tt1.sp"`:

```
% hspice -I
HSPICE > load tt1
```

Use the `list` command to view a netlist; for example, to view the `tt1.sp` netlist:

```
HSPICE > list
1  * this is an interactive mode example
2
3  R1 1 0 2
4  V1 1 0 3
5  .print I(R1)
6  .end
```

Use the `dc` command to specify an analysis and then run PrimeSim HSPICE:

```
HSPICE > dc v1 -5v 5v 0.5v
HSPICE > run
> info:  ***** hspice job concluded
```


Use the `list` command to view a netlist again. Notice that the DC analysis is in the interactive mode netlist. The original netlist, `tt1.sp`, is unchanged.

```
HSPICE > list
1  * this is an interactive mode example
2
3  R1 1 0 2
4  V1 1 0 3
5  .print I(R1)
6  .dc v1 -5v 5v 0.5v
7  .end
```

Use the `run` command to simulate the netlist:

```
HSPICE > run
> info:  ***** hspice job concluded
```

Use the `cd` command to change the current working directory to `/home/usr`:

```
HSPICE > cd /home/usr
```

Use the `pwd` command to print the full pathname of the current directory:

```
HSPICE > pwd
> /home/usr
```

Use the `quit` command to terminate the interactive mode:

```
HSPICE > quit
%
```

Using Environment Commands

Use the `load` command to read netlist “`tt3.sp`” and the `list` command to view it:

```
% hspice -I
HSPICE > load tt3.sp
HSPICE > list
R1 1 0 2
V1 1 0 3
.print I(R1)
.param a=10, t=24
.dc v(1) -5v 5v 0.5v
.end
```

Use the `ls` command to list the files in the current working directory:

```
HSPICE > ls
tt.sp
tt.sw0
:
```

Use the `set outflag` command to prevent printing simulation results to `stdout` (the default is `false`):

```
HSPICE > set outflag false
HSPICE > run
```

Use the `info outflag` command to view the current setting of `outflag`:

```
HSPICE > info outflag
false
HSPICE > quit
%
```

Recording and Saving Interactive Commands to a File

Use the `load` command to read netlist “`tt6.sp`” and print the full pathname of the current directory:

```
% hspice -I
HSPICE > load tt6.sp
HSPICE > pwd
```

Write all interactive commands entered to file `int1.cmd`:

```
HSPICE > save command int1
HSPICE > ls
int1.cmd
tt6.sp
tt6.sw0
HSPICE > input
R1 1 0 2
V1 1 0 3
.print I(R1)
.end
HSPICE > save netlist ex.sp
HSPICE > ls
int1.cmd
tt6.sp
tt6.sw0
ex.sp
HSPICE > quit
```

Quit interactive mode and return to the system prompt.

Printing a Voltage Value During Simulation

This example prints a voltage value during simulation:

```
% hspice -I
HSPICE > load tt2.sp
```

```
HSPICE > list
1  * this is an interactive mode example
2
3  R1 1 0 2
4  V1 1 0 3
5  .print I(R1)
6  .dc v1 -5v 5v 0.5v
HSPICE > print dc v(1,0)
HSPICE > run
v(1, 0) 0.000000
v(1, 0) 0.000000
v(1, 0) 0.000000
v(1, 0) 0.000000
v(1, 0) 0.000000
v(1, 0) 0.000000
v(1, 0) 0.000000
v(1, 0) 0.000000
v(1, 0) 0.000000
v(1, 0) 0.000000
v(1, 0) 0.000000
v(1, 0) 0.000000
v(1, 0) 0.000000
v(1, 0) 0.000000
v(1, 0) 0.000000
v(1, 0) 0.000000
v(1, 0) 0.000000
v(1, 0) 0.000000
v(1, 0) 0.000000
v(1, 0) 0.000000
v(1, 0) 0.000000
v(1, 0) 0.000000
v(1, 0) 0.000000
>info:          ***** hspice job concluded
HSPICE > quit
%
```

Using a Command File to Run in Interactive Mode

Use the `-I` `-L` arguments to invoke interactive mode on a command file and run it; for example,

```
% hspice -I -L int1.cmd
```

Running Multiple Testcases

To run multiple testcases and save the license check-in and check-out times, you can use the command file similar the following example. Use any text editor to create a command file named `bat.cmd` containing these entries:

```
load tt.sp
run
load qq.sp
```

```
run
quit
```

Run PrimeSim HSPICE in interactive mode to execute bat.cmd:

```
% hspice -I -L bat.cmd
...
% ls
tt.sp
tt.sw0
qq.sp
qq.sw0
```

PrimeSim HSPICE checks out a license only one time and then simulates both `tt.sp` and `qq.sp` netlists.

Multiple Runs Without Re-parsing the Input Netlist

Use `mrunch` as in the following example:

```
hspice -I -L mrunch.bat
```

The contents of the `mrunch.bat` batch file:

```
load mos.sp
mrunch
reset MAIN 1 = 3.1E-07 w = 1.89E-06 as = 1.89E-06 ad = 1.89E-06 ps =
1.89E-06 pd = 1.89E-06 DELVTO = 1.89E-06
mrunch
reset nch.7:nmos toxref = 3e-008 dlcig = 2.5e-008 vth0 = 0.1 lvth0 =
8.7417778e-007 wvth0 = 1.6053333e-006 pvth0 = 4.966e-011 k1 = 0.5 k2
= -0.2
param x2.par1=9
mrunch
mquit
```

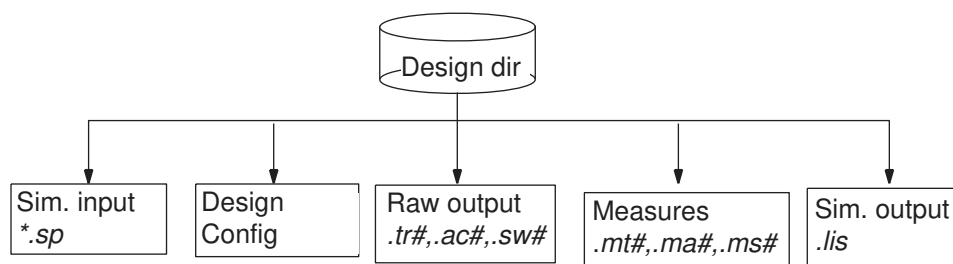
6

PrimeSim HSPICE GUI for Windows

Describes how to use the PrimeSim HSPICE GUI for Windows.

To open the PrimeSim HSPICE GUI (HSPUI) for Windows, click the HSPUI icon. [Figure 13](#) shows the directory structure for the PrimeSim HSPICE GUI for Windows (HSPUI).

Figure 13 *Directory Structure*



This following sections discuss these topics:

- [Working With Designs](#)
- [Configuring the PrimeSim HSPICE GUI for Windows](#)
- [Running Multiple Simulations](#)
- [SPutil, Metaencrypt and Converter Utilities, Client/Server Operation](#)
- [CMI Directory Structure](#)

Working With Designs

You can create a new design in several ways. HSPUI allows you to browse for an input file with the file suffix `.sp` or `.spi` for PrimeSim HSPICE.

Figure 14 HSPUI Main Window



In HSPUI, you can open a design file in one of the following ways:

- Select **File > Open** in the menu bar.
- Select the file folder icon in the icon bar.
- Click the **Browse** button next to the **Input netlist file** list.

For the preceding three cases, HSPUI opens a standard Windows file browser to help you select the design file for opening.

- Click the down arrow in the **Input netlist file** list and select one of the last-opened eight designs (to reopen the design).

Configuring the PrimeSim HSPICE GUI for Windows

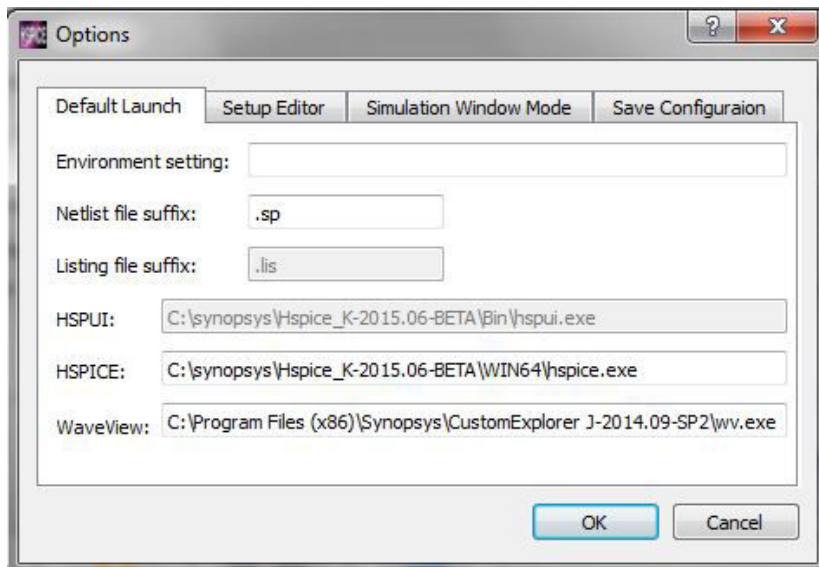
The following sections discuss these topics

- [HSPUI Options](#)
- [Launching WaveView in HSPUI](#)

HSPUI Options

You can customize the HSPUI configuration by selecting **Configuration > Options** from the menu bar or by clicking the Options icon in the icon bar. HSPUI then displays the Options dialog box (see [Figure 15 on page 131](#)) consisting of four tabs namely, **Default Launch**, **Setup Editor**, **Simulation Window Mode**, and **Save Configuration**.

Figure 15 HSPUI Options Dialog Box



You can configure options in each of these tabs (see the following table).

Table 13 HSPUI Options

Tab	Option	Description and Default Value
Default Launch	Environment setting	-
	Netlist file suffix	The file suffix of the netlist file. Default: <code>.sp</code>
	Listing file suffix	The file suffix of the listing file. Default: <code>.lis</code>
	HSPUI	HSPUI version. Default: the version defined in the <code>installdir</code> environment variable that is set up during PrimeSim HSPICE installation.
	PrimeSim HSPICE	PrimeSim HSPICE version. Default: the version defined in the <code>installdir</code> environment variable that is set up during PrimeSim HSPICE installation.

Table 13 HSPUI Options (Continued)

Tab	Option	Description and Default Value
	Waveview	Full path to the WaveView executable (vv.exe).
Setup Editor	Editor	Default: notepad.exe
Simulation Window Mode	Simulation window	• Can be one of the following: • Popup (default) • Minimized • Silent
Save Configuration	Save configuration at	• Installed Path (root permission): writes the configuration to <code>installdir\hspui.cfg</code>. • User local HOME: writes the configuration to <code>%APPDATA%\Synopsys\HSPICE\hspui.cfg</code>. Note: HSPUI saves the configuration if you select Automatic save configuration when exit and the configuration is changed, or if you select File > Save Configuration from the menu bar.

Launching WaveView in HSPUI

After you configure the path to WaveView in the HSPUI Options dialog box, you can launch Waveview in one of the following ways:

- Select **Tools > WaveView** in the menu bar.
- Click the WaveView icon in the icon bar.
- Click the WaveView action icon.

PrimeSim HSPICE automatically links the existing simulation results into WaveView, when you launch WaveView.

Note:

There should be no whitespaces in the path to the waveform file.

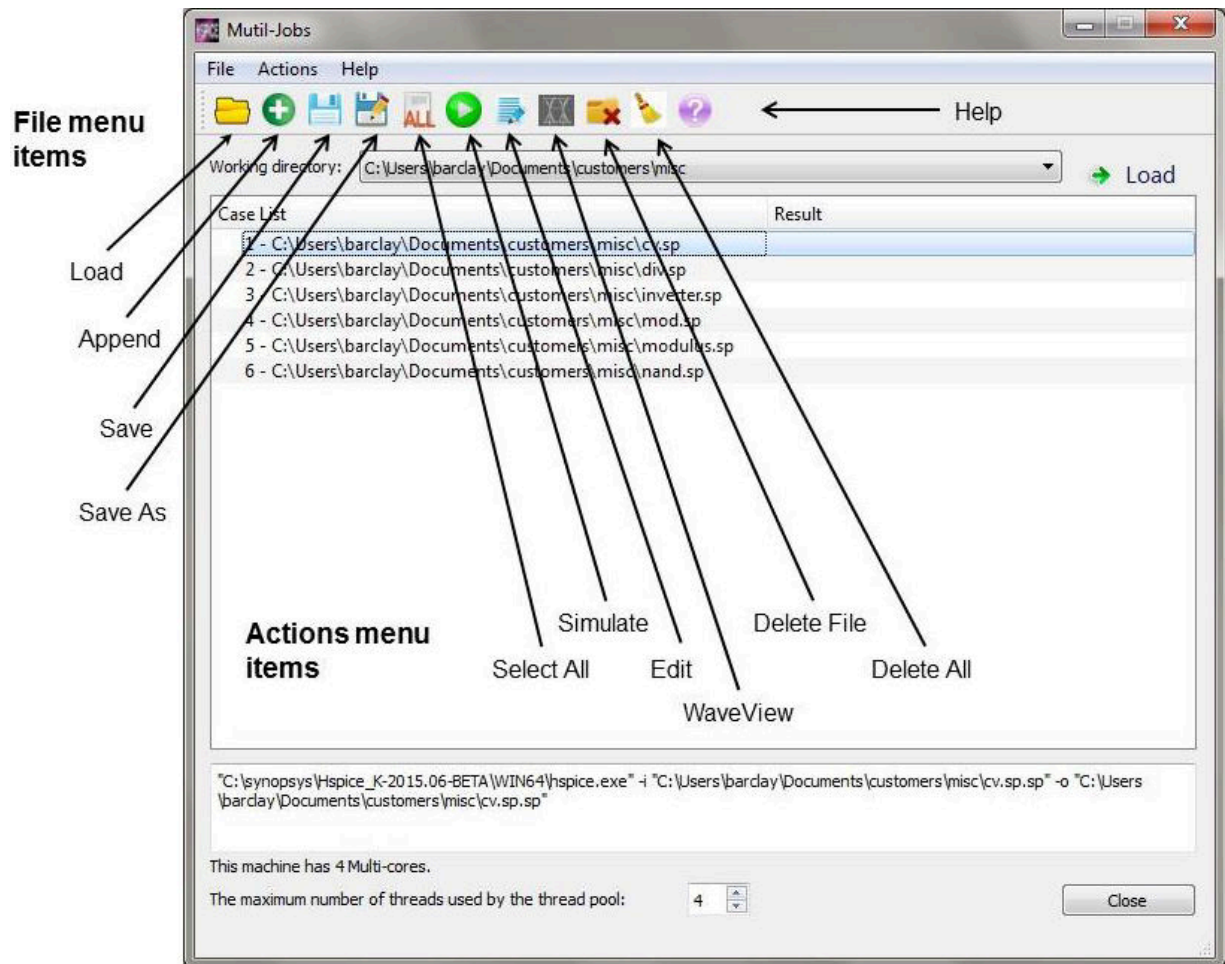
Running Multiple Simulations

You can use HSPUI to build a list of simulations, from different directories, for consecutive PrimeSim HSPICE simulation. You can use the HSPUI multi-jobs dialog box to create and run multiple simulations.

You can open the HSPUI multi-jobs dialog box in one of the following ways:

- Select **Tools > Multi-jobs** in the menu bar.
- Click the Multi-jobs icon in the icon bar.
- Click the Multi-jobs action icon.

Figure 16 HSPUI Multi-jobs Dialog Box



The following sections discuss these topics:

- [Building the Batch Job List](#)
- [Simulating a Batch Job](#)
-

Building the Batch Job List

To build a batch job list:

1. Open the HSPUI multi-jobs dialog box.
2. Click the Load icon and use the file browser to locate the directory containing the files that you want to simulate.

You can also use **File > Load** from the menu bar, or click the **Load** button next to the **Working directory** field.

3. Select any file in the directory to load all the files with the extension `.sp` or `.spi` into the Case List column in the dialog box.
4. To remove files that you do not want to simulate, select the file or files to be removed, in the Case List column, and then click the Delete File icon.

After selecting the files to be removed, you can also remove them by selecting **Actions > Delete File** from the menu bar.

5. To append files to the Case List column, from a different directory, click the Append icon and use the file browser to locate the directory containing the files you want to append.

You can also use **File > Append** from the menu bar.

Simulating a Batch Job

You can simulate the batch job list that you have created in one of the following ways:

1. Using a Batch File

After building the batch jobs list, click the Save icon to create a batch simulation file (`*.bat` file). You can also use **File > Save** from the menu bar, to create the batch simulation file.

Run the simulation by clicking the `*.bat` file in Windows Explorer or by running the `*.bat` file in an MS-DOS *cmd* window.

2. Using HSPUI

After building the batch jobs list, you can simulate the files in HSPUI.

1. To simulate all the files in the batch jobs list, select the All icon, or select **Actions > Select All** from the menu bar. Click the Simulate icon, or select **Actions > Simulate** from the menu bar, to start the consecutive simulations.
2. To simulate a single file, select the file using the left mouse button and click the Simulate icon to start the simulation.

3. To simulate a group of files, first select the files. You can use the left mouse button to select a single file. Then press and hold the Ctrl key to add another selected file (selected with the left mouse button) to the list. You can also press and hold the Shift key to select all files between the first file and the currently-selected file.

After you select the group of files to simulate, click the Simulate icon to start the consecutive simulations.

Typical Batch Workflow

A typical batch workflow using the Multi-jobs window contains the following steps:

1. Load your netlist files:

Click the Load icon. HSPUI opens the file browser. In the browser, navigate to the drive and directory where your netlist files are located and select any file.

2. Add/delete files from the run list as required:

Use the Delete file icon to delete selected files and the Append icon to add/append files.

Note:

Deleting a file only removes it from the run list and not from your hard-drive.

3. Edit the PrimeSim HSPICE files in the run list as required:

Select the file to be edited and click the Edit icon to open the file in the text editor. You can also right-click a selected file in the run list and select **Edit** from the context-sensitive menu.

4. Simulate the netlists:

HSPUI simulates only the jobs that you select in the run list.

To select a job, click the job file in the run list. You can select multiple files by holding down either the Ctrl key or the Shift key while clicking the files. To select all files in the job list, use the Select all icon.

After you select the jobs, click the Simulate icon to run the selected jobs sequentially, in the same order.

5. View simulation results:

After the simulations are complete, HSPUI displays the list of all files generated by PrimeSim HSPICE for each design. To view the contents of a file, click the down arrow near the file name.

To open a text file such as a listing (*.lis) file in the text editor, right-click the file name and select **Edit** from the context-sensitive menu.

To view the waveforms for each design, click the WaveView icon to invoke WaveView.

6. Create a batch file for running the simulation outside HSPUI:

To create a batch (*.bat) file with the list of jobs to be run (outside HSPUI), click either the File Save or File Save As icon.

The saved DOS batch file contains the full path to the PrimeSim HSPICE executable and the path and design name for each job in the run list. You can execute the *.bat file from the DOS command prompt or by double-clicking the file in Windows Explorer.

Running Multithreading and HPP

You can also run simulations using multithreading with or without HPP. You can enable both multithreading and HPP using the check boxes in the main HSPUI window. You can set the number of threads in the list box.

SPutil, Metaencrypt and Converter Utilities, Client/Server Operation

For information about use of the S-parameter utility (SPutil) refer to S-parameter Standalone Manipulation Utility in the *PrimeSim HSPICE User Guide: Signal Integrity Modeling and Analysis*.

For information about use of the `metaencrypt` utility for encryption of files, refer to Library and Data Encryption.

For information about use of the Converter utility for the conversion of output by PrimeSim HSPICE, refer to [Using the PrimeSim HSPICE Output Converter Utility](#).

The C/S mode check boxes allow you to start and use the PrimeSim HSPICE traditional client/server mode. For information about Client/Server usage, refer to [Using PrimeSim HSPICE in Client-Server Mode](#).

CMI Directory Structure

For information on the Custom Common Model Interface (CMI) directory structure for Windows platforms, contact your Synopsys technical support team to access the application note for the PrimeSim HSPICE CMI.

7

Input Netlist and Data Entry

Describes the input netlist file and methods of entering data for PrimeSim HSPICE.

PrimeSim HSPICE ships hundreds of examples for your use; for PrimeSim HSPICE demo cases, see Listing of Demonstration Input Files in the *PrimeSim HSPICE User Guide: Demonstration Netlists* for paths to the demo files. For PrimeSim HSPICE advanced analog analyses demo cases, see RF Demonstration Input Files in *PrimeSim HSPICE User Guide: Advanced Analog Simulation and Analysis*.

For descriptions of individual PrimeSim HSPICE commands referenced in this chapter, see PrimeSim HSPICE Netlist Commands in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

The following sections discuss these topics:

- [Input Netlist File Guidelines](#)
- [Input Netlist File Composition](#)
- [Using Subcircuits](#)
- [Subcircuit Call Statement Discrete Device Libraries](#)

Input Netlist File Guidelines

PrimeSim HSPICE operates on an input netlist file, and stores results in either an output listing file or a graph data file. An input file, with the name `design.sp`, contains the following:

- Design netlist (subcircuits, macros, power supplies, and so on).
- Statement naming the library to use (optional).
- The type of analysis to run (optional).
- The type of output desired (optional).

- Each token cannot exceed 1024 characters.
- An input file name can be up to 1024 characters long for all platforms except PC Windows, which has a limitation of 256 characters.
- PrimeSim HSPICE has a limitation on the number of characters in a path name plus a file name of 1024 characters (except PC Windows, 256 characters or fewer). For example:
 - `hspice -i path_name/input_file -o out_file`
- When specifying a path and file name using `-i` or `-o`, the length must be 1024 characters or fewer on all platforms. If the working directory path is greater than 1024 characters, PrimeSim HSPICE aborts with an error message.
- Model names in a netlist must begin with a letter. If you enter a model name with a leading integer PrimeSim HSPICE issues a parsing error.
- To generate input netlist and library input files, PrimeSim HSPICE uses either a schematic netlister or a text editor.
- Statements in the input netlist file can be in any order, except that the first line is a title line. In PrimeSim HSPICE, the last `.ALTER` submodule must appear at the end of the file and before the `.END` statement.
- For information on compressed netlist usage see [Compressed Netlist Guidelines on page 153](#).
- Netlist input processing is case insensitive, except for file names and their paths (see [Case Sensitivity on page 141](#)).

The following sections discuss these topics:

- [Input Line Format](#)
- [Case Sensitivity](#)
- [Special Characters](#)
- [First Character](#)
- [Delimiters](#)
- [Instance Names](#)
- [Hierarchy Paths](#)
- [Numbers](#)
- [Parameters and Expressions](#)
- [Reserved Keywords](#)

- [Input Netlist File Structure](#)
- [Schematic Netlists](#)
- [Compressed Netlist Guidelines](#)

Input Line Format

- The input reader can accept an input token, such as:
 - A statement name.
 - A node name.
 - A parameter name or value.

Any valid string of characters between two token delimiters is a token. You can use a character string as a parameter value in PrimeSim HSPICE, but not when performing PrimeSim HSPICE advanced analog analyses. See [Delimiters on page 145](#).

- A limitation of 1024 characters per token exists for an input statement or equation.
- Continuation lines:
 - Prepend a "+" character at the beginning of the continued line when the continuation is between tokens. For example:

```
R1 1 0
+ R='res1-res2'
```

- Use a double backslash "\\" at the end of the line for continuation when the continuation is inside a token or string. Whitespaces are optional to precede the string continuation. *A whitespace must not precede the token continuation, nor can a whitespace precede the double backslash.* For example:

```
*** string continuation ***
R6 4 0 R='res1-\\          res2'
R5 4 0 R='res1- \\
res2'
*** token continuation ***
R4 node1 no\\
de2 R= 'res1-res2'
```

Here, the double slash equals R4 node1 node2 R='res1-res2'.

- If you do not invoke the `-case 1` command-line switch, PrimeSim HSPICE ignores differences between upper and lower case in input lines, except in quoted filenames and or after the `.INC` or `.LIB` commands.

- PrimeSim HSPICE automatically completes parentheses and quotation marks if they carry over to (+) continued lines.
- To indicate “to the power of” in your netlist, use two asterisks (**). For example, 2**5 represents two to the fifth power (2^5).
- PrimeSim HSPICE ignores all characters after the following listed statement lines:

```
.include 'filename'

◦ .lib 'filename' corner

◦ .enddata, .end, .endl, .ends and .eom
```

For example:

```
.include 'biasckt.inc';      $ semicolon ignored
.lib 'mos251.l' tt,         $ comma ignored
```

- Parameter names must begin with an alphabetic character, but thereafter can contain numbers and some special characters. See [Special Characters](#).
 - When you use an asterisk (*) or a question mark (?) with a .PRINT, .PROBE, .LPRINT, or .CHECK statement, PrimeSim HSPICE uses the character as a wildcard. For additional information, see [Using Wildcards on Node Names on page 162](#).
 - When you use curly braces ({ }), PrimeSim HSPICE converts them to square brackets ([]) automatically.
 - Names become input tokens. Token delimiters must precede and follow names. See [Delimiters](#).
 - Names can be up to 1024 characters long and are not case-sensitive.
 - Do not use any of the time keywords as a parameter name or node name in your netlist.
 - Reserved operator keywords include these symbols: () = ‘

Do not use the symbol “=” as part of any parameter or node name that you define. Use of the symbol “+” as names causes a syntax error, and PrimeSim HSPICE stops immediately. Do not use the symbol “(” or “)” as part of any parameter name that you define. Use of the symbol “(” or “)” as part of parameter names causes a syntax error. Do not use the symbol ‘(’ or ‘)’ as part of node name that you define. The symbol ‘(’ or ‘)’ used as node names is treated as a delimiter like a space instead of part of node names.

Case Sensitivity

Where net, instance, parameter, and measure names, and any other name labels for downstream tools, require case sensitivity, PrimeSim HSPICE provides a mechanism to enable case-sensitive simulation. (Case insensitivity is unaffected for PrimeSim HSPICE keywords, commands, and options.) To invoke case sensitivity, on the PrimeSim HSPICE command-line, enter:

```
-case 1
```

For example, PrimeSim HSPICE reads a netlist with case sensitivity when you enter:

```
%hspice -case 1 -i design.sp -o design.lis
```

Case-sensitivity applies for the following instances:

- Parameter Names
- Node Names
- Instance Names
- Model Names
- Subcircuit Names
- Data Names
- Measure Names
- File Names and Paths (enabled by default)
- Library Entry Names

Mixed-case data is writable to wavefiles. To ensure backward compatibility in WaveView, in the **Waveview** tab of the Preference Settings form, Switch the **Case Sensitive PrimeSim HSPICE Waveform Files** check box.

Special Characters

[Table 14 on page 142](#) lists the special characters usable as part of node names, element parameter names, and element instance names. For details, see the appropriate sections in this chapter.

To avoid unexpected results or error messages, do not use the following mathematical characters in a parameter name in PrimeSim HSPICE: * - + ^ and /.

Table 14 PrimeSim HSPICE Netlist Special Characters

Special Character	Legal anywhere ³ =first character or any position in name "Included only" ³ =any position except first character	Node Name	Instance Name (cannot be the first character; element key letter only)	Parameter Name (cannot be the first character, element key letter only)	Delimiters
~	tilde	PrimeSim HSPICE: Legal anywhere PrimeSim HSPICE advanced analog analyses: Included only	Included only	Included only	n/a
!	exclamation point	Legal anywhere	Included only	Not recommended ³	n/a
@	at sign	Legal anywhere	Included only	Included only	n/a
#	pound sign	Legal anywhere	Included only	Included only	n/a
\$	dollar sign	Avoid using the \$ character in a node name	Included only	Included only	In-line comment character
%	percent	PrimeSim HSPICE: Legal anywhere PrimeSim HSPICE advanced analog analyses: Included only	Included only	PrimeSim HSPICE: included only Illegal in PrimeSim HSPICE advanced analog analyses	n/a
^	caret	PrimeSim HSPICE: Legal anywhere PrimeSim HSPICE advanced analog analyses: included only	Included only	PrimeSim HSPICE: included only (avoid usage), Illegal in PrimeSim HSPICE advanced analog analyses	"To the power of", i.e., 2^5, two raised to the fifth power
&	ampersand	PrimeSim HSPICE: Legal anywhere PrimeSim HSPICE advanced analog analyses Included only	Included only	Included only	n/a
*	asterisk	PrimeSim HSPICE: included only (avoid using * in node names), Illegal in PrimeSim HSPICE advanced analog analyses	Included only	PrimeSim HSPICE: included only (avoid using in parameter names), Illegal in PrimeSim HSPICE advanced analog analyses	Comment in both PrimeSim HSPICE. Wildcard character. Double asterisk (**) is "To the power of".

3. Using a parameter containing '!' in an expression will cause an error, since '!' is parsed as a part of a relational operator.

Table 14 PrimeSim HSPICE Netlist Special Characters (Continued)

Special Character	Legal anywhere="first character or any position in name Included only"=any position except first character	Node Name	Instance Name (cannot be the first character; element key letter only)	Parameter Name (cannot be the first character, element key letter only)	Delimiters
()	parentheses	Illegal	Illegal	Illegal	Token delimiter
-	minus	PrimeSim HSPICE: included only PrimeSim HSPICE advanced analog analyses: Legal anywhere	Included only	Included only (avoid usage)	n/a
_	underscore	Legal anywhere	Included only	Included only	n/a
+	plus sign	PrimeSim HSPICE: included only PrimeSim HSPICE advanced analog analyses: Legal anywhere	Included only	PrimeSim HSPICE: included only (avoid usage); Illegal in PrimeSim HSPICE advanced analog analyses	Continues previous line including expressions and algebras except for quoted strings
\	backslash	legal	legal	legal	Limits if nodes/element/param contains special characters such as () , =)
\\	double backslash (requires a whitespace before to use as a continuation)	PrimeSim HSPICE: included only PrimeSim HSPICE advanced analog analyses: Legal anywhere	Illegal	Illegal	Continuation character for quoted strings (preserves whitespace)
=	equals	Illegal	Illegal	optional in.PARAM statements	Token delimiter
< >	less/more than; '<' and '>' are treated as a relation operators in an expression	PrimeSim HSPICE: Illegal to use <...> as node name or first character; Legal to follow a first character (i.e., n<0>)	Included only	Legal as less than/ more than relational operators in an expression	n/a
?	question mark	PrimeSim HSPICE: Legal anywhere PrimeSim HSPICE advanced analog analyses: Illegal	Included only	Included only	Wildcard in character in PrimeSim HSPICE

Table 14 PrimeSim HSPICE Netlist Special Characters (Continued)

Special Character	Legal anywhere="first character or any position in name "Included only"=any position except first character	Node Name	Instance Name (cannot be the first character; element key letter only)	Parameter Name (cannot be the first character, element key letter only)	Delimiters
/	forward slash	Legal	Included only	Illegal	n/a
{ }	curly braces	PrimeSim HSPICE: included only, converts { } to [] No conversion for PrimeSim HSPICE advanced analog analyses	Included only	Included only	Auto-converts to square brackets ([]) Single ({ }) or () can be used in Variation Blocks
[]	square brackets	Included only	Included only	Included only	n/a
	pipe	PrimeSim HSPICE: Legal anywhere PrimeSim HSPICE advanced analog analyses: Included only	Included only	Included only	n/a
,	comma	Illegal	Illegal	Illegal	Token delimiter
.	period	Illegal	Included only	Included only	Netlist keyword, (i.e., .TRAN, .DC, etc.). Hierarchy delimiter when used in node names
:	colon	Included only	Included only	Illegal	Delimiter for element attributes
;	semi-colon	Included only	Included only	Included only	n/a
" "	double-quotes	Illegal	Illegal	Illegal	Expression and filename delimiter
' '	single quotes	Illegal	Illegal	Illegal	Expression and filename delimiter
	Blank (whitespace)	Use before \ or \\ line continuations			Token delimiter
Tab	Tab				Token delimiter

First Character

The first character in every line specifies how PrimeSim HSPICE interprets the remaining line. [Table 15](#) lists and describes the valid characters.

Table 15 First Character Descriptions

Line	If the First Character is...	Indicates
First line of a netlist	Any character	Title or comment line. The first line of an included file is a normal line and not a comment.
Subsequent lines of netlist, and all lines of included files	. (period)	Netlist keyword. For example, .TRAN 0.5ns 20ns
	c, C, d, D, e, E, f, F, g, G, h, H, i, I, j, J, k, K, l, L, m, M, n, N, o, O, p, P, q, Q, r, R, s, S, t, T, u, U, v, V, w, W	Element instantiation
	*(asterisk)	Comment line
	+(plus)	Continues previous line

Delimiters

- An input token is any item in the input file that PrimeSim HSPICE recognizes. Input token delimiters are: tab, blank (whitespace), comma (,), and parentheses ().
- Single (') or double quotes (") delimit expressions and filenames.
- Colons (:) delimit element attributes (for example, M1:VGS).
- Periods (.) indicate hierarchy. For example, X1.X2.n1 is the n1 node on the X2 subcircuit of the X1 circuit.

Note:

The equal sign (=) is a token delimiter in the sense only that when you define a parameter, both the parameter name and parameter value are tokens, so the '=' is a token delimiter. However, you cannot enter '=' anywhere in a line like you would a comma, space, tab, or in the case of PrimeSim HSPICE, parentheses.

For example: the following is incorrect and returns an error message:

```
xtest2====(a),,(b)=,== (mysub====r1=1000=r2=1000(((, ,
```

Instance Names

The names of element instances begin with the element key letter (see [Table 16](#)), except in subcircuits where instance names begin with X. (Subcircuits are interchangeably known as macros or modules.) Instance names can be up to 1024 characters long. In PrimeSim HSPICE, the `.OPTION LENNAM` defines the length of names in printouts (default value=8).

Table 16 *Element Identifiers*

Letter (First Char)	Element	Example Line
B	IBIS buffer	b_io_0 nd_pu0 nd_pd0 nd_out nd_in0 nd_en0 nd_outofin0 nd_pc0 nd_gc0
C	Capacitor	Cbypass 1 0 10pf
D	Diode	D7 3 9 D1
E	Voltage-controlled voltage source	Ea 1 2 3 4 K
F	Current-controlled current source	Fsub n1 n2 vin 2.0
G	Voltage-controlled current source	G12 4 0 3 0 10
H	Current-controlled voltage source	H3 4 5 Vout 2.0
I	Current source	I A 2 6 1e-6
J	JFET or MESFET	J1 7 2 3 GAASFET
K	Linear mutual inductor (general form)	K1 L1 L2 1
L	Linear inductor	LX a b 1e-9
M	MOS transistor	M834 1 2 3 4 N1
P	Port	P1 in gnd port=1 z0=50
Q	Bipolar transistor	Q5 3 6 7 8 pnp1
R	Resistor	R10 21 10 1000
S	S parameter element	S1 nd1 nd2 s_model2
V	Voltage source	V1 8 0 5
T,U,W	Transmission Line	W1 in1 0 out1 0 N=1 L=1
X	Subcircuit call	X1 2 4 17 31 MULTI WN=100 LN=5

Hierarchy Paths

- A period (.) indicates path hierarchy.
- Paths can be up to 1024 characters long.
- Path numbers compress the hierarchy for post-processing and listing files.
- You can find path number cross references in the listing and in the *design.pa0* file.
- The `.OPTION PATHNUM` controls whether the list files show full path names or path numbers.

Numbers

You can enter numbers as integer, floating point, floating point with an integer exponent, or integer or floating point with one of the scale factors listed in [Table 17](#).

Table 17 *Scale Factors*

Scale Factor	Prefix	Symbol	Multiplying Factor
T	tera	T	1e+12
G	giga	G	1e+9
ME,MEG, or X	mega	M,MEG, or X	1e+6
K	kilo	k	1e+3
MI or MIL	n/a	MI or MIL	25.4e-6
M	milli	m	1e-3
U	micro	μ	1e-6
N	nano	n	1e-9
P	pico	p	1e-12
F	femto	f	1e-15
A	atto	a	1e-18
Z	zepto	z	1e-21
Y	yocto	y	1e-24
DB	DB	db	10 ^(value/20)
MIN	MIN	min	60
HR	HR	hr	3600

Table 17 Scale Factors (Continued)

Scale Factor	Prefix	Symbol	Multiplying Factor
DAY	DAY	day	86400
YR	YR	yr	31536000

Note:

- Scale factor A is not a scale factor in a character string that contains amps. For example, PrimeSim HSPICE interprets a “20amps” string as 20 amperes of current, not as 20e-18 amps.
- Numbers can use exponential format or engineering key letter format (1e-12 or 1p), but to use both (1e-6u), you must specify `.OPTION MIXED_NUM_FORMAT = 1`.
- To designate exponents, use D or E.
- The `.OPTION EXPMAX` limits the exponent size.
- PrimeSim HSPICE interprets trailing alphabetic characters as unit comments.
- Unit comments do not go through a checking process.
- The `.OPTION INGOLD` controls the format of numbers in printouts.
- The `.OPTION NUMDGT=x` controls the listing printout accuracy.
- The `.OPTION MEASDGT=x` controls the measure file printout accuracy.

Parameters and Expressions

- Parameter names in PrimeSim HSPICE use PrimeSim HSPICE name syntax rules, except that names must begin with an alphabetic character. The other characters must be either a number, or a special character. See [Table 14 on page 142](#) in the [Special Characters](#) section for a listing of legal parameter names. For example, a “%” is legal if included in PrimeSim HSPICE, but illegal in PrimeSim HSPICE advanced analog analyses.
- To define parameter hierarchy overrides and defaults, use the `.OPTION PARHIER=global | local` statement.
- If you create multiple definitions for the same parameter or option, PrimeSim HSPICE uses the last parameter definition or `.OPTION` statement, even if that definition occurs later in the input than a reference to the parameter or option. PrimeSim HSPICE does not warn you when you redefine a parameter.

- You must define a parameter before you use that parameter to define another parameter.
- When you select design parameter names, be careful to avoid conflicts with parameterized libraries.
- To delimit expressions, *always* use single or double quotes. Otherwise, if a mathematical character (+ - * /) is within a string, PrimeSim HSPICE regards the string also as an expression.
- Expressions cannot exceed 4096 characters.
- For readability, use a double backslash preceded by a whitespace (\\) at end of a line, to continue the line.
- Use the `PAR` (expression or parameter) function to evaluate expressions in output statements.
- Limitation 1: If you define a parameter as an expression containing output signals such as `v(node)` or `i(element)`, this parameter applies only in an element value expression directly. PrimeSim HSPICE does not evaluate it to another parameter.

For example, the following is correct:

```
.param a='2*sqrt(V(p,n)) '  
r1 p n '1k+a'
```

The following definition is correct, but this definition points up the limitation and is not permitted because PrimeSim HSPICE generates an incorrect result:

```
.param a='2*sqrt(v(p,n)) '  
.param b='a+1 '  
r1 p n '1k+b'
```

It is best to use a user-defined function to replace the previous example, so that all of `r1` and `r2` are correct.

```
.param a(x)='2*sqrt(x) '  
.param b(x)='a(x)+1 '  
r1 p n '1k+a(V(p,n)) '  
r2 p n '1k+b(V(p,n)) '
```

- Limitation 2: If you use an expression containing output signals such as `v(node)` or `i(element)` is used in an element value directly, the element only can be R, C, L, E, or G.

Correct

```
G1 1 0 cur='((1-(a0*v(gate)))/b0)'
```

Incorrect

```
I1 1 0 cur='((1-(a0*v(gate)))/b0)'
```

Reserved Keywords

Reserved keywords are those which are illegal for a parameter name and/or node name. The following keywords are not “global” and are analysis/syntax-specific; that is, you can use these keywords in some specific statements, but not in other statements. For example, you cannot use *pwl* as a parameter name in the V source, but you can use it in an R-element. You can use these keywords as parameter names with single quotes anywhere. [Table 18 on page 150](#) lists the illegal keywords for the specified *element* type. [Table 19 on page 151](#) lists the illegal keywords for the specified *command*.

Table 18 *Reserved Keywords and Elements*

Elements	Illegal Keywords
B-element	FILE, MODEL, NOWARN, COMPL_TR
Bjt/Jfet element	OFF, IC, TNODEOUT
Capacitor	POLY, TC, SENS
Diode	OFF, IC
E/G-element	POLY, PWL, AND, NAND, PPWL, NPWL, OR, NOR, LAPLACE, POLE, VCCS, OPAMP, DELAY, TRANSFORMER, VCR, VCCAP, VCVS, SPUR, FREQ, ZTRANS, VMRF, FOSTER, NOISE, NOISEFILE, MNAME, PHASE, SCALE, MAX, PAR
F/H-elements	POLY, PWL, AND, NAND, OR, NOR, VMRF, CCCS, CCVS, DELAY
Independent Sources and Port element	SIN, PU, PWL, EXP, PULSE, PE, SFFM, AM, PAT, PL, HB, HBAC, DATA, AC, DC, LSAC, SNAC, PHOTO, NEUT, COS, VMRF, LFSR, PUL, HBLIN, R, BITSTREAM, PWLFILE, MOD, FILTER
Inductor	POLY, TC, SENS, RELUCTANCE, TRANSFORMER_NT, FILE
Resistor	POLY, TC, SENS
S-element	ZO, Z0, MNAME
T/U- element	IC
W-element	RLGCFILE, PRINTZO, RLGCMODEL, TABLEMODEL, FSMODEL, UMODEL, SMODEL

Table 19 *Reserved Keywords and Commands*

Commands	Illegal Keywords
.AC/.DC	UIC, MONTE, DATA, SWEEP, POI, DEC, LIN, OCT, RESULTS, LIST, OPTIMIZE, SWEEPBLOCK, EXPLORE
.DATA	MER, LAM, FILE, MERGE
.FQMODEL	DATA
.FSOPTION	COMPUTETABLE
.LAYERSTACK	LAYER
.LIN	FORMAT, LISTFREQ, FILENAME
.PAT	DATA
.PKG/.EBD/.IBIS/.ICM	FILE, MODEL, COMPONENT, NOWARN, MOD_SEL
.SAVE/.LOAD	FILE
.SHAPE	DATA, VERTEX, N
.STIM	PWL, DATA, VEC, TRAN, AC, DC, FILENAME
.TRAN	UIC, MONTE, DATA, SWEEP, POI, DEC, LIN, OCT, RESULTS, LIST, OPTIMIZE, SWEEPBLOCK, RESULT

Input Netlist File Structure

An input netlist file should consist of one main program. In PrimeSim HSPICE, an input netlist can contain one or more optional submodules. PrimeSim HSPICE uses a submodule (preceded by an `.ALTER` statement) to automatically change an input netlist file; then rerun the simulation with different options, netlist, analysis statements, and test vectors.

You can use several high-level call statements (such as `.INCLUDE` and `.LIB`) to structure the input netlist file modules. These statements can call netlists, model parameters, test vectors, analysis, and option macros into a file from library files or other files. The input netlist file also can call an external data file, which contains parameterized data for element sources and models. You must enclose the names of included or internally specified files in single or double quotation marks when they begin with a number (0-9).

Schematic Netlists

PrimeSim HSPICE typically uses netlisters to generate circuits from schematics, and accept either hierarchical or flat netlists.

The process of creating a schematic involves:

- Symbol creation with a symbol editor.
- Circuit encapsulation.
- Property creation.
- Symbol placement.
- Symbol property definition.
- Wire routing and definition

Table 20 *Input Netlist File Sections*

Sections	Examples	Definition
Title	.TITLE	The first line in the netlist is the title of the input netlist file. (PrimeSim HSPICE)
Set-up	.OPTION .IC or .NODESET, .PARAM, .GLOBAL	Sets conditions for simulation. Initial values in circuit and subcircuit. Set parameter values in the netlist. Set node name globally in netlist.
Sources	Sources and digital inputs	Sets input stimuli (I or V element).
Netlist	Circuit elements .SUBKCT, .ENDS, or .MACRO, .EOM	Circuit for simulation. Subcircuit definitions.
Analysis	.DC, .TRAN, .AC, and so on. .SAVE and .LOAD .DATA, .TEMP	Statements to perform analyses. Save and load operating point information. (PrimeSim HSPICE) Create table for data-driven analysis. Set temperature analysis.
Output	.PRINT, .PROBE, .MEASURE	Statements to output variables. Statement to evaluate and report user-defined functions of a circuit.
Library, Model and File Inclusion	.INCLUDE	General include files.
	.MALIAS	Assigns an alias to a diode, BJT, JFET, or MOSFET.
	.MODEL	Element model descriptions.
	.LIB	Library.
	.OPTION SEARCH	Search path for libraries and included files. (PrimeSim HSPICE)

Sections	Examples	Definition
Alter blocks (PrimeSim HSPICE Only)	.ALIAS, .ALTER, .DEL LIB	Renames a previous model. Sequence for in-line case analysis. Removes previous library selection.
End of netlist	.END	Required statement; end of netlist.

Compressed Netlist Guidelines

To save disk space and to improve performance on large netlists, PrimeSim HSPICE supports the `.gzip` file format for both input files (*design.sp*) and these statements:

- `.inc`
- `.lib`
- `.load`

However, PrimeSim HSPICE does not support the S-parameter and RLGC file input types in the `.gz` format.

When both *design.sp* and *design.sp.gz* exist, PrimeSim HSPICE always selects the exact match file first, if it exists. For example:

1. If `.include design.sp` is in the netlist, then PrimeSim HSPICE picks *design.sp*.
2. If `.include design.sp.gz` is in the netlist, then PrimeSim HSPICE picks *design.sp.gz*
3. If `.include design.sp` is in the netlist and only *design.sp.gz* exists, then PrimeSim HSPICE chooses *design.sp.gz*.
4. If `.include design.sp.gz` is in the netlist and only *design.sp* exists, then PrimeSim HSPICE uses *design.sp*.

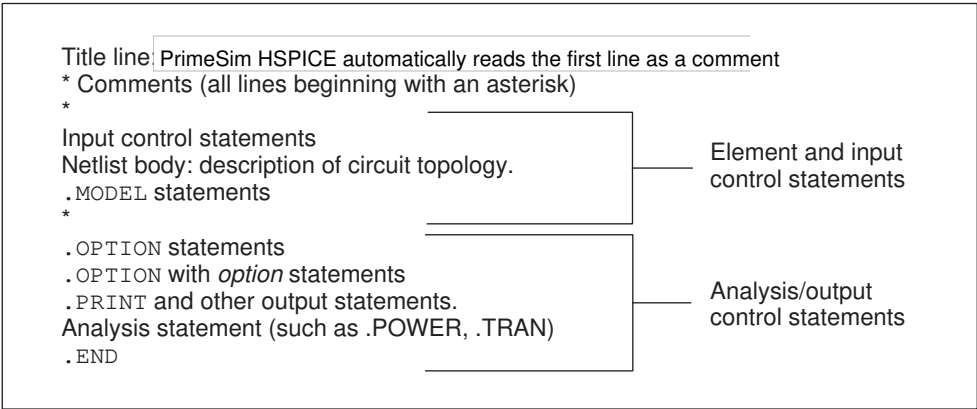
Input Netlist File Composition

The PrimeSim HSPICE circuit description syntax is compatible with the SPICE input netlist format. [Figure 17](#) and [Figure 18 on page 154](#) show the basic structure of an input netlist along with the comment and continuation characters.

Figure 17 Basic netlist structure

Netlist Structure: Overview	
Title	First line is always the title
Comment character	* - comment for a line \$ - comment after a command
Options	.option post
Print/Probe/Analysis	.print v(d) i(rl) .probe v(g) .tran .1n 5n
Initial Conditions	.ic v(b) = 0 \$ input state
Sources	Vg g 0 pulse 0 1 0 0.15 0.15 0.42 * example of a voltage source
Circuit Description	MN d g gnd n nmos RL vdd d 1K
Models	.model n nmos level = 49 + vto = 1 tox = 7n * '+' continuation character
End	.end \$ terminates the simulation

Figure 18 Sections of a netlist



The following is an example of a simple netlist file, called `inv_ckt.in`. It shows a small inverter test case that measures the timing behavior of the inverter.

To create the circuit:

1. Define MOSFET models for the PMOS/NMOS transistors of the inverter.
2. Insert the power supplies for both VDD and GND power rails.
3. Insert the pulse source to the inverter input.

This circuit uses transient analysis and produces output graphical waveform data for the input and output ports of the inverter circuit.

```
* Sample inverter circuit
* ***** MOS models *****
.MODEL n1 NMOS LEVEL=3 THETA=0.4 ...
.MODEL p1 PMOS LEVEL=3 ...
* ***** Define power supplies and sources *****
VDD VDD 0 5
VPULSE VIN 0 PULSE 0 5 2N 2N 2N 98N 200N
VGND GND 0 0
* ***** Actual circuit topology *****
M1 VOUT VIN VDD VDD p1
M2 VOUT VIN GND GND n1
* ***** Analysis statement *****
.TRAN 1n 300n
* ***** Output control statements *****
.OPTION POST PROBE
.PROBE V(VIN) V(VOUT)
.END
```

For a description of individual commands used in PrimeSim HSPICE netlists, see PrimeSim HSPICE Netlist Commands in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

The following sections discuss these topics:

- [PrimeSim HSPICE Topology Rules](#)
- [Title of Simulation](#)
- [Comments and Line Continuation](#)
- [Element and Source Statements](#)
- [Defining Subcircuits](#)
- [Node Name \(or Node Identifier\) Conventions](#)
- [Element, Instance, and Subcircuit Naming Conventions](#)
- [Subcircuit Node Names](#)
- [Path Names of Subcircuit Nodes](#)
- [Abbreviated Subcircuit Node Names](#)
- [Automatic Node Name Generation](#)
- [Global Node Names](#)
- [Circuit Temperature](#)

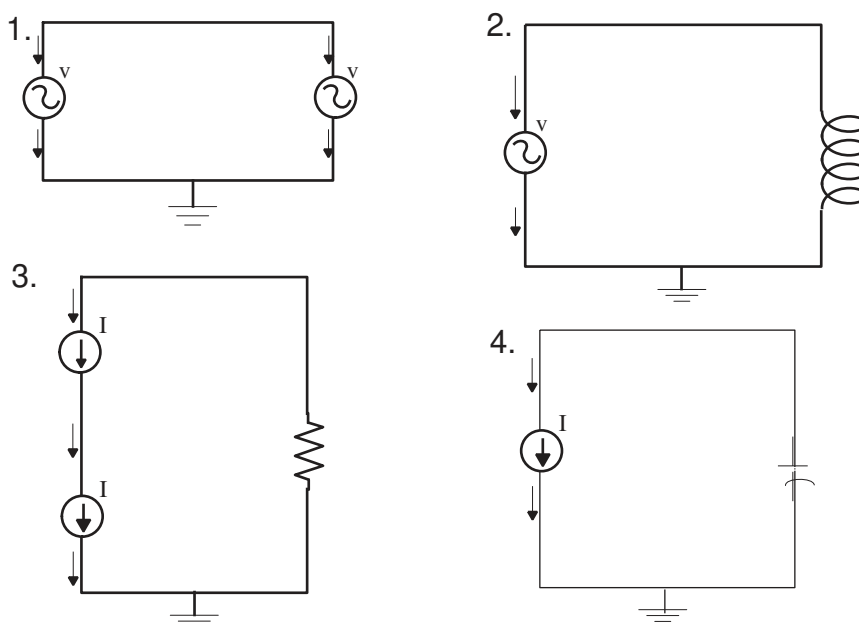
- [Data-Driven Analysis](#)
- [Library Calls and Definitions](#)
- [Defining Parameters](#)
- [Altering Design Variables and Subcircuits](#)
- [Connecting Nodes](#)
- [Deleting a Library](#)
- [Ending a Netlist](#)
- [Condition-Controlled Netlists \(IF-ELSE\)](#)

PrimeSim HSPICE Topology Rules

When constructing the circuit description PrimeSim HSPICE does not allow certain topologies. See [Figure 19 on page 157](#). The following rules apply:

1. No voltage loops (no voltage sources in parallel with no other elements).
2. No ideal voltage source in closed inductor loop.
3. No stacked current sources (no current sources in series).
4. No ideal current source in closed capacitor loop.

Figure 19 Illegal topologies in PrimeSim HSPICE



Title of Simulation

You set the simulation title in the first line of the input file. PrimeSim HSPICE always reads this line, and uses it as the title of the simulation, regardless of the line's contents. The simulation prints the title verbatim, in each section heading of the output listing file.

To set the title, you can place a `.TITLE` statement on the first line of the netlist. However, PrimeSim HSPICE does not require the `.TITLE` syntax.

The first line of the input file is always the implicit title. If any statement appears as the first line in a file, simulation interprets it as a title, and does not execute it.

An `.ALTER` statement does not support use of the `.TITLE` statement. To change a title for a `.ALTER` statement, place the title content in the `.ALTER` statement itself.

Comments and Line Continuation

The first line of a netlist is always a comment, regardless of its first character; comments that are not the first line of the netlist require an asterisk (*) as the first character in a line or a dollar sign (\$) directly in front of the comment anywhere on the line. For example,

```
* comment_on_a_line_by_itself
-or-
HSPICE_statement $ comment_following_HSPICE_input
```

You can place comment statements anywhere in the circuit description.

You must use the dollar sign (\$) comments that do *not* begin in the first character position on a line (for example, for comments that follow simulator input on the same line). If it is not the first nonblank character, then you must precede the dollar sign by either:

- Whitespace
- Comma (,)
- Valid numeric expression.

You can also place the dollar sign within node or element names.

For example,

```
* RF=1K GAIN SHOULD BE 100
$ MAY THE FORCE BE WITH MY CIRCUIT
VIN 1 0 PL 0 0 5V 5NS $ 10v 50ns
R12 1 0 1MEG $ FEED BACK
.PARAM a=1w$comment a=1, w treated as a space and ignored
.PARAM a=1k$comment a=1e3, k is a scale factor
```

A dollar sign is the preferred way to indicate comments because of the flexibility of its placement within the code.

Line continuations require a plus sign (+) as the first character in the line that follows. Here is an example of comments and line continuation in a netlist file:

```
.ABC Title Line (HSPICE ignores the netlist keyword
* on this line because the first line is always a comment)
* This is a comment line
.MODEL n1 NMOS $ this is an example of an inline comment
* This is a comment line and the following line is a continuation
+ LEVEL=3
```

Element and Source Statements

Element statements describe the netlists of devices and sources. Use nodes to connect elements to one another. Nodes can be either numbers or names. Element statements specify:

- Type of device.
- The nodes to the connected device.
- Operating electrical characteristics of the device.

Element statements can also reference model statements that define the electrical parameters of the element.

[Table 21](#) lists the parameters of an element statements.

Table 21 *Element Parameters*

Parameter	Description
elname	Element name that cannot exceed 1023 characters, and must begin with a specific letter for each element type: B - I BIS buffer (PrimeSim HSPICE Only) C - Capacitor D - Diode E,F,G,H - Dependent current and voltage sources I - Current (inductance) source J - JFET or MESFET K - Mutual inductor L - Inductor model or magnetic core mutual inductor model M - MOSFET Q - BJT P - Port R - Resistor S - S-parameter mode T, U, W - Transmission line V - Voltage source X - Subcircuit call
node1 ...	Node names identify the nodes that connect to the element. The node name begins with a letter and can contain a maximum of 1023 characters. For a listing of legal and illegal special characters usable in node names, see the Special Characters section, Table 14
mname	PrimeSim HSPICE requires a model reference name for all elements, except passive devices.
pname1 ...	An element parameter name identifies the parameter value that follows this name.
expression	Any mathematical expression containing values or parameters, such as param1 * val2
val1 ...	Value of the <i>pname1</i> parameter, or of the corresponding model node. The value can be a number or an algebraic expression.
M= <i>positive int</i>	Element multiplier. Replicates <i>val</i> element times, in parallel. Do not assign a negative value or zero as the M value.

For descriptions of element statements for the various types of supported elements, see the *PrimeSim HSPICE User Guide: Elements*.

Example 1

```
Q1234567 4000 5000 6000 SUBSTRATE BJTMODEL AREA=1.0
```

The preceding example specifies a bipolar junction transistor, with its collector connected to node 4000, its base connected to node 5000, its emitter connected to node 6000, and

its substrate connected to the `SUBSTRATE` node. The `BJTMODEL` name references the model statement, which describes the transistor parameters.

```
M1 ADDR SIG1 GND SBS N1 10U 100U
```

The preceding example specifies a MOSFET named `M1`, where:

- drain node=`ADDR`
- gate node=`SIG1`
- source node=`GND`
- substrate nodes=`SBS`

The preceding element statement calls an associated model statement, `N1`. The MOSFET dimensions are width=`100` microns and length=`10` microns.

Example 2

```
M1 ADDR SIG1 GND SBS N1 w1+w l1+l
```

The preceding example specifies a MOSFET named `M1`, where:

- drain node=`ADDR`
- gate node=`SIG1`
- source node=`GND`
- substrate nodes=`SBS`

The preceding element statement calls an associated model statement, `N1`. MOSFET dimensions are algebraic expressions (width=`w1+w`, and length=`l1+l`).

Defining Subcircuits

You can create a subcircuit description for a commonly used circuit, and include one or more references to the subcircuit in your netlist.

- Use `.SUBCKT` and `.MACRO` statements to define subcircuits within your PrimeSim HSPICE netlist.
- Use the `.ENDS` statement to terminate a `.SUBCKT` statement.
- Use the `.EOM` statement to terminate a `.MACRO` statement.

- Use `Xsubcircuit_name` (the subcircuit call statement) to call a subcircuit that you previously defined in a `.MACRO` or `.SUBCKT` command in your netlist, where `subcircuit_name` is the element name of the subcircuit that you are calling.
- Use the `.INCLUDE` statement to include another netlist as a subcircuit in the current netlist.

Node Name (or Node Identifier) Conventions

Nodes are the points of connection between elements in the input netlist. You can use either names or numbers to designate nodes. Node numbers can be from 1 to 9999999999999999 (1-1e16); node number 0 always equals ground. When the node name begins with a letter or a valid special character, the node name can contain a maximum of 1024 characters.

The following are conventions regarding node names:

- In addition to letters and digits, node names can include, but NOT always begin with some special characters. See the [Special Characters](#) section, [Table 14](#).
- If you use braces { } in node names, PrimeSim HSPICE changes them to brackets [].
- You cannot use the following characters in node names: () , = , ' , <blank>
- You should avoid using the dollar sign (\$) after a numerical digit in a node name because PrimeSim HSPICE assumes whatever follows the "\$" symbol is an in-line comment (see [Comments and Line Continuation on page 157](#) for additional information). It can cause error and warning messages depending on the location of the node with the "\$". For example, PrimeSim HSPICE generates an error to indicate that a resistor node is missing:

```
R1 1$ 2 1k
```

Also, in this example, PrimeSim HSPICE issues a warning indicating that the value of resistor R1 is limited to 1e-5 and interprets the line as "R1 2 1" without a defined value:

```
R1 2 1$ 1k
```

- The period (.) is a reserved character for use as a separator between a subcircuit name and a node name: `subcircuitName.nodeName`. If a node name contains a period, the node PrimeSim HSPICE considers it a top level node unless there is a valid match to a subcircuit name and node name in the hierarchy.
- The sorting order for operating point nodes is:

```
a-z, !, #, $, %, *, +, -, /
```

- To make node names global across all subcircuits, use a `.GLOBAL` statement (see [Global Node Names on page 166](#)).

- Simulation treats nodes with the 0, GND, GND!, and GROUND node names as a ground node, and produces v(0) into the output files. Besides these ground nodes, PrimeSim HSPICE regards all node names as separate nodes. For example, 0 and 0.3 are different nodes, 1A and 1 are different nodes, 2~ and 2 are different nodes.

Note:

Nodes named as 0, GND, GND!, and GROUND, in .subckt line, are not treated as global PrimeSim HSPICE ground. Such nodes are instantiated by the node in the corresponding .subckt instances (X elements).

The following sections discuss these topics:

- [Using Wildcards on Node Names](#)
- [Wildcard Applications and Examples](#)

Using Wildcards on Node Names

You can use wildcards to match node names.

- The ? wildcard matches any single character. For example, 9? matches 92, 9a, 9A, and 9%.
- The * wildcard matches any string of zero or more characters. For example:
 - If your netlist includes a resistor named r1 and a voltage source named vin, then .PRINT i(*) prints the current for both of these elements: i(r1) and i(vin).
 - And .PRINT v(o*) prints the voltages for all nodes whose names start with o; if your netlist contains nodes named in and out, this example prints only the v(out) voltage. For example, the following prints the results of a transient analysis for the voltage at the matched node name.

```
.PRINT TRAN V(9?t*u)
```

- Wildcards must begin with a letter or a number; wildcards must begin with output type V, I or P; for example,

.PROBE v(*)	\$ correct format
.PROBE *	\$ incorrect format
.PROBE ix*	\$ correct format

Wildcard Applications and Examples

Here are some practical applications for these wildcards:

- If your netlist includes a resistor named `r1` and a voltage source named `vin`, then `.PRINT i(*)` prints the current for both elements `i(r1)` and `i(vin)`.
- The statement `.PRINT v(o*)` prints the voltages for all nodes whose names start with `o`; if your netlist contains nodes named `in` and `out`, this example prints only the `v(out)` voltage.
- If your netlist contains nodes named `0`, `1`, `2`, and `3`, then `.PRINT v(0,*)` or `.PRINT v(0 *)` prints the voltage between node `0` and each of the other nodes: `v(0,1)`, `v(0,2)`, and `v(0,3)`.
- Wildcards can set initial conditions in `.IC` and `.NODESET` statements. Node names including wildcards in `.IC` and `.NODESET` must start with `"v( )"`. For example, `.NODESET v(a*)=5`.

Examples

The following examples use wildcards with `.PRINT`, `.PROBE`, `.LPRINT`, `.IC` and `.NODESET` statements.

- Probe node voltages for nodes at all levels.

```
.PROBE v(*)
```

- Probe all nodes whose names start with `"a"`. For example: `a1`, `a2`, `a3`, `a00`, `ayz`.

```
.PROBE v(a*)
```

- Print node voltages for nodes at the first level and all levels below the first level, where zero-level are top-level nodes. For example: `x1.A`, `x4.554`, `Xab.abc123`.

```
.PRINT v(*.*)
```

- Probe node voltages for all nodes whose name start with `"x"` at the first level and all levels below the first level, where zero-level are top-level nodes. For example: `x1.A`, `x4.554`, `xab.abc123`.

```
.PROBE v(x*.*)
```

- Print node voltages for nodes whose names start with `"x"` at the second-level and all levels below the second level. For example: `x1.x2.a`, `xab.xdff.in`.

```
.PRINT v(x*.*?)
```

- Match all first-level nodes with names that are exactly two characters long. For example: `x1.in`, `x4.12`.

```
.PRINT v(x*.*.*)
```

- Print the logic state of all top-level nodes, whose names start with **b**. For example: **b1**, **b2**, **b3**, **b56**, **bac**.

```
.LPRINT (1,4) b*
```

- Set transient initial conditions to all nodes. For example:

```
.ic v(*)=val
```

initializes nodal voltages for DC operating point analysis to nodes whose names start with “a”, such as: **a1**, **a2**, **a11**, **a21**. `.nodeset v(a*)=val`

If a specified node also matches a wildcard in `.IC` or `.NODESET`, then the specified value overrides the wildcard's value. For example, if `.ic v(a*)=5 v(a1)=10`, then `v(a1)` will be 10.

Element, Instance, and Subcircuit Naming Conventions

Instances and subcircuits are elements and as such, follow the naming conventions for elements.

Element names in PrimeSim HSPICE begin with a letter designating the element type, followed by up to 1023 alphanumeric and special characters as defined in [Table 14](#). Element type letters are R for resistor, C for capacitor, M for a MOSFET device, and so on (see [Element and Source Statements on page 158](#)).

Subcircuit Node Names

PrimeSim HSPICE assigns two subcircuit node names.

- To assign the first name, PrimeSim HSPICE uses the `(.)` extension to concatenate the circuit path name with the node name — for example, **X1.XBIAS.M5**. Node designations that start with the same number, followed by any letter, are the same node. For example, **1c** and **1d** are the same node.
- The second subcircuit node name is a unique number that PrimeSim HSPICE automatically assigns to an input netlist subcircuit. The `(:)` extension concatenates this number with the internal node name, to form the entire subcircuit's node name (for example, **10:M5**). The output listing file cross-references the node name.

Note:

PrimeSim HSPICE advanced analog analyses does not support short names for internal subcircuits, such as **10:M5**.

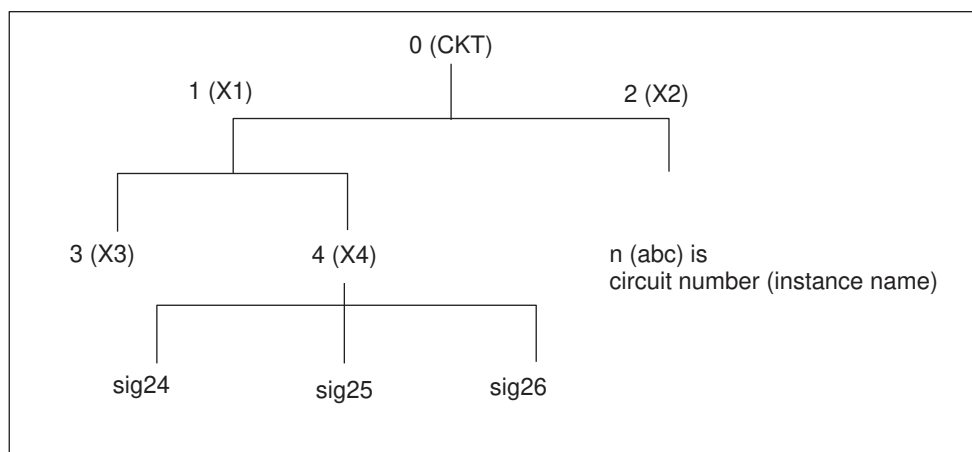
- To indicate the ground node, use either the number **0**, the name **GND**, or **!GND**. Every node should have at least two connections, except for transmission line nodes

(unterminated transmission lines are permitted) and MOSFET substrate nodes (which have two internal connections). PrimeSim HSPICE terminates floating power supply nodes with a 1Megohm resistor and a warning message.

Path Names of Subcircuit Nodes

A path name consists of a sequence of subcircuit names, starting at the highest-level subcircuit call, and ending at an element or bottom-level node. Periods separate the subcircuit names in the path name. The maximum length of the path name, including the node name, is 1024 characters. You can use path names in `.PRINT`, `.NODESET`, and `.IC` statements, as another way to reference internal nodes (that is, nodes not appearing on the parameter list). You can use the path name to reference any node, including any internal node. Subcircuit node and element names follow the rules shown in [Figure 20 on page 165](#).

Figure 20 Subcircuit Calling Tree, with Circuit Numbers and Instance Names



In [Figure 20](#), the path name of the `sig25` node in the `X4` subcircuit is `X1.X4.sig25`. You can use this path in PrimeSim HSPICE statements, such as:

```
.PRINT v(X1.X4.sig25)
```

Abbreviated Subcircuit Node Names

In PrimeSim HSPICE, you can use circuit numbers as an alternative to path names, to reference nodes or elements in `.PRINT`, `.NODESET`, or `.IC` statements. The compilation process assigns a circuit number to all subcircuits, creating an abbreviated path name:

```
subckt-num:name
```

Note:

PrimeSim HSPICE advanced analog analyses does not recognize this type of abbreviated subcircuit name.

The subcircuit name and a colon precede every occurrence of a node or element in the output listing file. For example, `4:INTNODE1` is a node named `INTNODE1`, in a subcircuit assigned the number 4.

Any node not in a subcircuit has a 0: prefix (0 references the main circuit). To identify nodes and subcircuits in the output listing file, PrimeSim HSPICE uses a circuit number that references the subcircuit where the node or element appears.

Abbreviated path names let you use DC operating point node voltage output, as input in a `.NODESET` statement for a later run.

You can copy the part of the output listing titled *Operating Point Information* or you can type it directly into the input file, preceded by a `.NODESET` statement. This eliminates recomputing the DC operating point in the second simulation.

Automatic Node Name Generation

PrimeSim HSPICE can automatically assign internal node names. To check both nodal voltages and branch currents, you can use the assigned node name when you print or plot. PrimeSim HSPICE supports several special cases for node assignment — for example, simulation automatically assigns node 0 as a ground node.

For CSOS (CMOS Silicon on Sapphire), if you assign a value of -1 to the bulk node, the name of the bulk node is `B#`. Use this name to print the voltage at the bulk node. When printing or plotting current — for example `.PRINT I(R1)` — PrimeSim HSPICE inserts a zero-valued voltage source. This source inserts an extra node in the circuit named `Vnn`, where `nn` is a number that PrimeSim HSPICE automatically generates; this number appears in the output listing file.

Global Node Names

The `.GLOBAL` statement globally assigns a node name, in PrimeSim HSPICE. This means that all references to a global node name, used at any level of the hierarchy in the circuit, connect to the same node.

The most common use of a `.GLOBAL` statement is if your netlist file includes subcircuits. This statement assigns a common node name to subcircuit nodes. Another common use of `.GLOBAL` statements is to assign power supply connections of all subcircuits. For example, `.GLOBAL VCC` connects all subcircuits with the internal node name `VCC`.

Ordinarily, in a subcircuit, the node name consists of the circuit number, concatenated to the node name. When you use a `.GLOBAL` statement, PrimeSim HSPICE does not

concatenate the node name with the circuit number, and assigns only the global name. You can then exclude the power node name in the subcircuit or macro call.

Circuit Temperature

To specify the circuit temperature for a PrimeSim HSPICE simulation, use the `.TEMP` command, the `TEMP` parameter in the `.DC`, `.AC`, and `.TRAN` statements, or the `TEMP/TEMPER` parameter in the first column of the `.DATA` statement. PrimeSim HSPICE compares the circuit simulation temperature against the reference temperature in the `TNOM` control option. PrimeSim HSPICE uses the difference between the circuit simulation temperature and the `TNOM` reference temperature to define derating factors for component values.

PrimeSim HSPICE advanced analog analyses supports only the last `.TEMP` command in a netlist, if you use multiple `.TEMP` commands.

Data-Driven Analysis

In data-driven analysis, you can modify any number of parameters, then use the new parameter values to perform an operating point, DC, AC, or transient analysis. An array of parameter values can be either inline (in the simulation input file) or stored as an external ASCII file. The `.DATA` statement associates a list of parameter names with corresponding values in the array.

PrimeSim HSPICE supports the entire functionality of the `.DATA` statement. (See `.DATA` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.) However, PrimeSim HSPICE advanced analog analyses supports `.DATA` only for:

- Data-driven analysis.
- Inline or external data files.

For more details about using the `.DATA` statement in different types of analysis, see [Chapter 10, Initializing DC-Operating Point Analysis](#) and [Chapter 12, Transient Analysis](#).

Library Calls and Definitions

To create and read from libraries of commonly used commands, device models, subcircuit analysis, and statements in library files, use the `.LIB` call statement. As PrimeSim HSPICE encounters each `.LIB` call name in the main data file, it reads the corresponding entry from the designated library file, until it finds an `.ENDL` statement. In PrimeSim HSPICE, you can also place a `.LIB` call statement in an `.ALTER` block.

The following sections discuss these topics:

- [Library Building Rules](#)
- [Automatic Library Selection \(PrimeSim HSPICE\)](#)

Library Building Rules

- A library cannot contain `.ALTER` statements.
- A library can contain nested `.LIB` calls to itself or to other libraries. If you use a relative path in a nested `.LIB` call, the path starts from the directory of the parent library, not from the work directory. If the path starts from the work directory, PrimeSim HSPICE can also find the library, but it prints a warning. Only the constraints of your system configuration limit the depth of nested calls.
- A library cannot contain a call to a library of its own entry name, within the same library file.
- A PrimeSim HSPICE library cannot contain the `.END` statement.
- `.ALTER` processing cannot change `.LIB` statements, within a file that an `.INCLUDE` statement calls.

Automatic Library Selection (PrimeSim HSPICE)

Automatic library selection searches a sequence of up to 40 directories. The `hspice.ini` file sets the default search paths.

Use this file for directories that you want PrimeSim HSPICE to always search. PrimeSim HSPICE searches for libraries in the order specified in `.OPTION SEARCH` statements.

When PrimeSim HSPICE encounters a subcircuit call, the search order is:

1. Read the input file for a `.SUBCKT` or `.MACRO` with the specified call name.
2. Read any `.INC` files or `.LIB` files for a `.SUBCKT` or `.MACRO` with the specified call name.
3. Search the directory with the input file for the `call_name.inc` file.
4. Search the directories in the `.OPTION SEARCH` list.

You can use the PrimeSim HSPICE library search and selection features to simulate process corner cases, using `.OPTION SEARCH = 'libdir'` to target different process directories. For example, if you store an input or output buffer subcircuit in a file named `iobuf.inc`, you can create three copies of the file, to simulate fast, slow and typical corner cases. Each file contains different PrimeSim HSPICE transistor models, representing the different process corners. Store these files (all named `iobuf.inc`) in separate directories.

Defining Parameters

The `.PARAM` statement defines parameters. Parameters in PrimeSim HSPICE are names that have associated numeric values. You can define parameters through predefined analysis or measurement

The following sections discuss these topics:

- [Predefined Analysis](#)
- [Measurement Parameters](#)

Predefined Analysis

PrimeSim HSPICE provides several specialized analysis types, which require a way to control the analysis. For the syntax used in these `.PARAM` commands, see the description of the `.PARAM` command in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

PrimeSim HSPICE supports the following predefined analysis parameters:

- Temperature functions (*fn*)
- Optimization guess/range
- Monte Carlo functions

PrimeSim HSPICE also supports the following predefined parameter types, that PrimeSim HSPICE advanced analog analyses does not support:

- frequency
- time

Measurement Parameters

A `.MEASURE` statement produces a measurement parameter. In general, the rules for measurement parameters are the same as those for standard parameters. While a `.PARAM` statement disallows definition of measurement parameters, the `.MEASURE` statement directly defines such parameters. For more information, see [.MEASURE Parameter Types on page 215](#).

Altering Design Variables and Subcircuits

The following rules apply when you use an `.ALTER` block to alter design variables and subcircuits in PrimeSim HSPICE. This section does not apply to PrimeSim HSPICE advanced analog analyses.

- If the name of a new element, `.MODEL` statement, or subcircuit definition is identical to the name of an original statement of the same type, then the new statement replaces the old. Add new statements in the input netlist file.
- You can alter element and `.MODEL` statements within a subcircuit definition. You can also add a new element or `.MODEL` statement to a subcircuit definition. To modify the topology in subcircuit definitions, put the element into libraries. To add a library, use `.LIB`; to delete, use `.DEL LIB`.
- If you want to change temperature points in multiple `.ALTER` statements then, parameterizing the `.TEMP` statement and updating the `.PARAM` statement in each `.ALTER` statement is recommended, instead of specifying the `.TEMP` value in the `.ALTER` blocks.
- If a parameter name in a new `.PARAM` statement in the `.ALTER` module is identical to a previous parameter name, then the new assigned value replaces the old value.
- If you used parameter (variable) values for elements (or model parameter values) when you used `.ALTER`, use the `.PARAM` statement to change these parameter values. Do not use numerical values to redescribe elements or model parameters.
- If you used an `.OPTION` statement (in an original input file or a `.ALTER` block) to turn on an option, you can turn that option off.
- Each `.ALTER` simulation run prints only the actual altered input. A special `.ALTER` title identifies the run.
- `.ALTER` processing cannot revise `.LIB` statements within a file that an `.INCLUDE` statement calls. However, `.ALTER` processing can accept `.INCLUDE` statements, within a file that a `.LIB` statement calls.

This section covers the following topics:

- [Using Multiple `.ALTER` Blocks](#)
- [Adding or Changing Analyses in `.ALTER` Blocks](#)

Using Multiple `.ALTER` Blocks

- For the first simulation run, PrimeSim HSPICE reads the input file, up to the first `.ALTER` statement, and performs the analyses up to that `.ALTER` statement.
- After it completes the first simulation, PrimeSim HSPICE reads the input between the first `.ALTER` statement, and either the next `.ALTER` statement or the `.END` statement.
- PrimeSim HSPICE then uses these statements to modify the input netlist file.
- PrimeSim HSPICE then resimulates the circuit.

- For each additional `.ALTER` statement, PrimeSim HSPICE first performs the simulation based on the netlist modified by the previous `.ALTER` statement.
- PrimeSim HSPICE then performs another simulation by using the input between the current `.ALTER` statement, and either the next `.ALTER` statement or the `.END` statement.

If you do not want to rerun the simulation that precedes the first `.ALTER` statement, every time you run an `.ALTER` simulation, then do the following:

1. Put the statements that precede the first `.ALTER` statement, into a library.
2. Use the `.LIB` statement in the main input file.
3. Put a `.DEL LIB` statement in the `.ALTER` section, to delete that library for the `.ALTER` simulation run.

Adding or Changing Analyses in .ALTER Blocks

When you add a new analysis statement (`.TRAN`, `.AC`, `.DC` and so on) in an `.ALTER` block, the added analysis statement does not replace the analysis statement previously defined in the top level. Instead, the analysis command in each `.ALTER` runs in addition to the analysis statement in the top level. This will cause PrimeSim HSPICE to output more analysis results files than expected.

Example

In the following example, a new transient analysis with a different simulation time is added in each `.ALTER` block.

```
* top level netlist
.param pw=10n per=20n
V1 1 0 pulse 0 1 0 1n 1n 'pw' 'per'
R1 1 2 1k
R2 2 0 1k
.tran 1n 20n
.alter 1
.param pw=7.5n per=15n
.tran 1n 15n
.alter 2
.param pw=5n per=10n
.tran 1n 10n
.end
```

This netlist will generate six `*.tr#` files, instead of the expected three `*.tr#` files, as described in the following list:

- **Top level:** `*.tr0 @ pw=10n, per=20n and tstop=20n`
- **Alter 1:** `*.tr1 @ pw=10n, per=20n and tstop=20n`
`*.tr2 @ pw=7.5n, per=15n and tstop=15n`

- **Alter 2:** `*.tr3 @ pw=10n, per=20n and tstop=20n`
`*.tr4 @ pw=7.5n, per=15n and tstop=15n`
`*.tr5 @ pw=5n, per=10n and tstop=10n`

To generate the desired three `*.tr#` files, you can parameterize the `tstop` value in the `.TRAN` analysis statement. Then, use only one `.TRAN` statement and change the parameterized `tstop` value in each `.ALTER` block:

```
* top level netlist
.param pw=10n per=20n
.param ts=20n $ tstop value
V1 1 0 pulse 0 1 0 1n 1n 'pw' 'per'
R1 1 2 1k
R2 2 0 1k
.tran 1n 'ts'
.alter 1
.param pw=7.5n per=15n
.param ts=15n
.alter 2
.param ts=10n
.param pw=5n per=10n
.end
```

Connecting Nodes

Use a `.CONNECT` statement to connect two nodes in your PrimeSim HSPICE netlist, so that simulation evaluates two nodes as only one node. Both nodes must be at the same level in the circuit design that you are simulating: you cannot connect nodes that belong to different subcircuits.

Deleting a Library

Use a `.DEL LIB` statement to remove library data from memory. The next time you run a simulation the `.DEL LIB` statement removes the `.LIB` call statement, with the same library number and entry name from memory. You can then use a `.LIB` statement to replace the deleted library.

You can use a `.DEL LIB` statement with a `.ALTER` statement.

Ending a Netlist

An `.END` statement must be the last statement in the input netlist file. Text that follows the `.END` statement is a comment, and has no effect on the simulation.

An input file that contains more than one simulation run must include an `.END` statement for each simulation run. You can concatenate several simulations into a single file.

Condition-Controlled Netlists (IF-ELSE)

You can use the IF-ELSE structure (PrimeSim HSPICE only) to change the circuit topology, expand the circuit, set parameter values for each device instance, select different model cards, reference subcircuits, or define subcircuits in each IF-ELSE block.

```
.if (condition1)
    statement_block1

# The following statement block in {braces} is
# optional, and you can repeat it multiple times:
{ .elseif (condition2)
    statement_block2
}

# The following statement block in [brackets]
# is optional, and you cannot repeat it:
[ .else
    statement_block3
]
.endif
```

The following example provides a quick overview of using the IF-ELSE construct:

```
*
.tran 1n 100ns
.option post
.param variable = 2
vin1 source 0 0 pwl 0ns 0v 20ns 0v 21ns 5v 30ns 5v 31ns 0v 40ns
+ 0v 41ns 5v 50ns 5v 70ns 5v 71ns 0v 80ns 0v 81ns 5v
+ 90ns 5v 91ns 0v 100ns 0v
Rin source 1 1000
x1 1 2 resistor
r1 2 0 1k
.subckt resistor 1 2
.if (variable==1)
r22 1 2 1k
.elseif (variable==2)
r22 1 2 2k
.else
r22 1 2 3k
.endif
.ends
.print v(2) i(1)
.alter
.param variable=1
.alter
.param variable=3
.end
```

Guidelines for Using IF-ELSE Blocks (PrimeSim HSPICE Only)

The following guidelines aid in usage of the .IF, .ELSE-IF, or .ELSE:

- In an .IF, .ELSEIF, or .ELSE condition statement, complex Boolean expressions must not be ambiguous. For example, change (a==b && c>=d) to ((a==b) && (c>=d)).
- In an IF, ELSEIF, or ELSE statement block, you can include most valid PrimeSim HSPICE analysis and output statements. The exceptions are:
 - .END, .ALTER, .GLOBAL, .DEL LIB, .MALIAS, .ALIAS, .LIST, .NOLIST, and .CONNECT statements.
 - .OPTIONS search, d_ibis, d_imic, d_lv56, biasfi, modsrh, cmiflag, and nxx.
- You can include IF-ELSEIF-ELSE statements in subcircuits and subcircuits in IF-ELSEIF-ELSE statements.
- You can use IF-ELSEIF-ELSE blocks to select different submodules to structure the netlist (using .INC, .LIB, and .VEC statements).
- If two or more models in an IF-ELSE block have the same model name and model type, they must also be the same revision level.
- Parameters in an IF-ELSE block do not affect the parameter value within the condition expression. PrimeSim HSPICE updates the parameter value only after it selects the IF-ELSE block.
- You can nest IF-ELSE blocks.
- You can include .SUBCKT and .MACRO statements within an IF-ELSE block.
- You can include an unlimited number of ELSEIF statements within an IF-ELSE block.
- You cannot use an IF-ELSE block within another statement. In the following example, PrimeSim HSPICE does not recognize the IF-ELSE block as part of the resistor definition:

```
r 1 0
  .if (r_val>10k)
    + 10k
  .else
    + r_val
  .endif
```

- You can use string parameters in conditional expressions in IF-ELSEIF-ELSE blocks. When using a string parameter, str() is optional.

Example:

```
X_M1 OUT1 IN1 SUB1 DEV=str('TYP1') L=1 W=1
.SUBCKT SUB1 D G S Dev=str('DT') X=L Y=W
```

```
.if ( Dev==str('TYP1') && X==1 && Y==1 )    $ Dev is equivalent to
  str(Dev)
.include 'file1'
.elseif ( Dev==str('DT') && X*Y>=2 )
.include 'file2'
.endif
.ENDS
```

Using Subcircuits

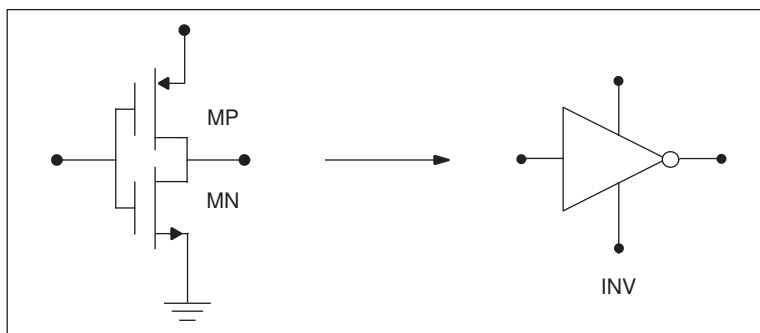
Reusable cells are the key to saving labor in any CAD system. This also applies to circuit simulation, in PrimeSim HSPICE.

- To create and simulate a reusable circuit, construct it as a subcircuit.
- Use parameters to expand the utility of a subcircuit.

Traditional SPICE includes the basic subcircuit, but does not provide a way to consistently name nodes. However, PrimeSim HSPICE provides a simple method for naming subcircuit nodes and elements: use the subcircuit call name as a prefix to the node or element name.

In PrimeSim HSPICE advanced analog analyses, you cannot replicate output commands within subcircuit (`.subckt`) definitions.

Figure 21 Subcircuit Representation



The following input creates an instance named X1 of the INV cell macro, which consists of two MOSFETs, named MN and MP:

```
X1 IN OUT VD_LOCAL VS_LOCAL inv W=20
.MACRO INV IN OUT VDD VSS W=10 L=1 DJUNC=0
MP OUT IN VDD VDD PCH W=W L=L DTEMP=DJUNC
MN OUT IN VSS VSS NCH W='W/2' L=L DTEMP=DJUNC
.EOM
```

Note:

To access the name of the MOSFET, inside of the `INV` subcircuit that `X1` calls, the names are `X1.MP` and `X1.MN`. So to print the current that flows through the MOSFETs, use `.PRINT I (X1.MP)`.

The following sections discuss these topics:

- [Hierarchical Parameters](#)
- [Undefined Subcircuit Search \(PrimeSim HSPICE\)](#)
- [Troubleshooting Subcircuit Node Issues](#)

Hierarchical Parameters

You can use two hierarchical parameters, the `M` (multiply) parameter and the `S` (scale) parameter.

The following sections discuss these topics:

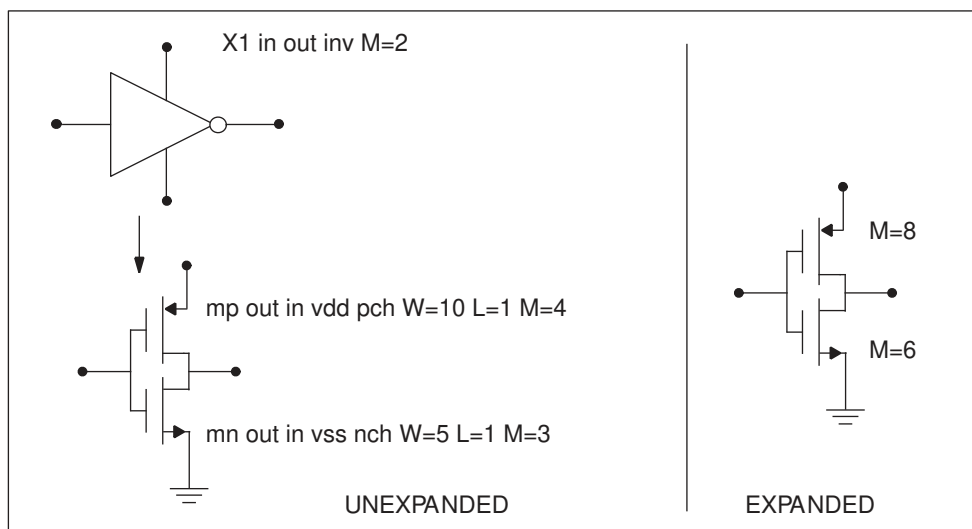
- [M \(Multiply\) Parameter](#)
- [S \(Scale\) Parameter](#)
- [Using Hierarchical Parameters to Simplify Simulations](#)

M (Multiply) Parameter

The most basic PrimeSim HSPICE subcircuit parameter is the `M` (multiply) parameter. This keyword is common to all elements, including subcircuits, except for voltage sources. The `M` parameter multiplies the internal component values. This, in effect, creates parallel copies of the element. For example, if you have an inverter and specify `M=2`, then PrimeSim HSPICE multiplies the internal component by 2. The `M` parameter multiplies the internal component values, which, in effect, creates parallel copies of the element. To simulate 32 output buffers switching simultaneously, you need to place only one subcircuit; for example,

```
X1 in out buffer M=32
```

Figure 22 How Hierarchical Multiply Works



Multiply works hierarchically. For a subcircuit within a subcircuit, PrimeSim HSPICE multiplies the product of both levels. Values of *M* must be positive. Do not assign a negative value or zero as the *M* value.

S (Scale) Parameter

To scale a subcircuit, use the *S* (local scale) parameter. This parameter behaves in much the same way as the *M* parameter in the preceding section.

```
.OPTION hier_scale=value
.OPTION scale=value
X1 node1 node2 subname S=valueM parameter
```

The `.OPTION HIER_SCALE` statement defines how PrimeSim HSPICE interprets the *S*-parameter, where *value* is either:

- 0 (the default), indicating a user-defined parameter, or
- 1, indicating a scale parameter.

The `.OPTION SCALE` statement defines the original (default) scale of the subcircuit. The specified *S* scale is relative to this default scale of the subcircuit.

The scale in the *subname* subcircuit is *value***scale*. Subcircuits can originate from multiple sources, so scaling is multiplicative (cumulative) throughout your design hierarchy.

```
x1 a y inv S=1u
subckt inv in out
x2 a b kk S=1m
.ends
```

In this example:

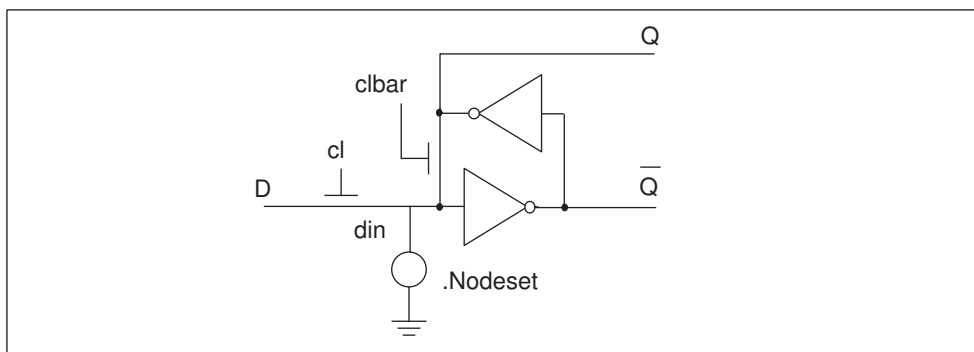
- PrimeSim HSPICE scales the `x1` subcircuit by the first `s` scaling value, `1u*scale`.
- Because scaling is cumulative, PrimeSim HSPICE then scales `x2` (a subcircuit of `x1`), in effect, by the `S` scaling values of both `X1` and `X2`: `1m*1u*scale`.

Using Hierarchical Parameters to Simplify Simulations

You can use the hierarchical parameter to simplify simulations. For example, see the following listing and [Figure 23 on page 178](#).

```
X1 D Q Qbar CL CLBAR dlatch flip=0
.macro dlatch
+ D Q Qbar CL CLBAR flip=vcc
.nodeset v(din)=flip
xinv1 din qbar inv
xinv2 Qbar Q inv
m1 q CLBAR din nch w=5 l=1
m2 D CL din nch w=5 l=1
.eom
```

Figure 23 D Latch with Nodeset



PrimeSim HSPICE does not limit the size or complexity of subcircuits; they can contain subcircuit references, and any model or element statement. However, in PrimeSim HSPICE advanced analog analyses, you cannot replicate output commands within subcircuit definitions.

To specify subcircuit nodes in `.PRINT` statements, specify the full subcircuit path and node name. Use a `.print/.probe/.measure` statement to print the hierarchical parameter. For example:

```
x1 1 0 sub1
.subckt sub1 n1 n2
.param p1=1
...
.ends
.tran ...
```

```
.print tran par('x1.p1')  
.measure tran m1 param='x1.p1'
```

You add function `valp( )` to refer to a parameter defined in subckt for output (including a forward slash). For example:

```
.meas tran asdf param='valp(x1/p1)'
```

Undefined Subcircuit Search (PrimeSim HSPICE)

If a subcircuit call is in a data file that does not describe the subcircuit, PrimeSim HSPICE automatically searches directories in the following order:

1. Current directory for the file.
2. Directories specified in `.OPTIONSEARCH = "directory_path_name"` statements.
3. Directory containing the Discrete Device Library.

PrimeSim HSPICE searches for the model reference name file that has an `.inc` suffix. For example, if the data file includes an undefined subcircuit, such as `X 1 1 2 INV`, PrimeSim HSPICE searches the system directories for the `inv.inc` file and when found, places that file in the calling data file.

Troubleshooting Subcircuit Node Issues

Use the following workarounds or solutions to subcircuit-related errors:

- [Subcircuit Name Missing Error](#)
- [Duplicated Subckt Definition Error](#)
- [Duplicate Node Message](#)
- [Duplicating Ports](#)
- [Using Assigned Circuit Numbers instead of Full Path Node Names](#)
- [Simple Reporting of Floating Nodes](#)
- [Rules for Defining Subcircuit Global and Local Nets](#)

Subcircuit Name Missing Error

Simulation of a subcircuit named `16_inv`, for example, results in an error issued:

```
**error** (/remote/user/hspice/inv.sp:18) subcircuit name missing
```

PrimeSim HSPICE does not support subcircuit names that begin with a number. Subcircuit names must begin with an alpha character and can contain up to 1023 additional alphanumeric and special characters.

Duplicated Subckt Definition Error

Use .OPTION REDEFSUB to define how multiple subcircuit definitions in the netlist are treated. The syntax for the REDEFSUB option is as follows:

```
.OPTION REDEFSUB =[0|1|2]
```

The default value is 0 which causes PrimeSim HSPICE to issue an error message if there are multiple subcircuit definitions.

Set REDEFSUB=1 for PrimeSim HSPICE to use the last subcircuit definition.

Set REDEFSUB=2 for PrimeSim HSPICE to use the first subcircuit definition.

Duplicate Node Message

In the .SUBCKT definition, if PrimeSim HSPICE finds two or more node names that are the same, it issues the following error:

```
**error** subcircuit definition duplicates node node_name **error**  
.ends card missing at readin
```

In the following example, for the .SUBCKT definition, the node “in” gets two definitions. The second definition of the node “in” is a duplicate of the first node, which is illegal in PrimeSim HSPICE.:

```
.subckt ABC in in out  
.  
.  
.ends
```

Duplicating Ports

To create duplicate ports in PrimeSim HSPICE you can define them in the instance definition of the subcircuit.

For example:

```
.subckt DUP A B C D  
.  
.  
.ends
```

If you want to make node “B” a duplicate of node “A” then the instance definition should look like:

```
XDUP A A C D DUP
```


Using Assigned Circuit Numbers instead of Full Path Node Names

In PrimeSim HSPICE, you can use circuit numbers as an alternative to the full subcircuit path names to reference nodes or elements in `.PRINT`, `.NODESET` or `.IC` statements. PrimeSim HSPICE assigns a circuit number to all subcircuits in the netlist. The circuit number abbreviates the path name, as follows: *subckt-num:name*

You can find the abbreviated path name information in the `*.pa0` file. Nodes that are not in a subcircuit have a 0 prefix which references the main or top level circuit.

Simple Reporting of Floating Nodes

PrimeSim HSPICE can report simple instances of floating nodes when there is no path to ground. The `*.lis` file prints: `**warning** only 1 connection at node`. You can increase the number of warning messages printed by setting `.option warnlimit=n` in the netlist.

The list file reports nodes that have no DC path to ground (such as a cap with one node to gnd and no connect on other end), but there is no way to check for tri-state nodes.

Rules for Defining Subcircuit Global and Local Nets

The following rules apply (from lowest precedence to highest):

1. An internal subckt net with the same name as a global net is global:

```
.subckt res net1
R1 net1 gnd 100 $ local gnd is connected to global net gnd
.ends
```

2. Interface subckt pins on a subckt are always local:

```
.subckt res2 net1 gnd
R1 net1 gnd 100 $ local gnd is connected to local net net2, not to
  global gnd
.end
X1 net1 net2 res2
```

3. An instance connected to a global net will make the subckt pin and its internal connections global:

```
.subckt res2 net1
R1 net1 gnd 100 $ local gnd is connected to global net gnd through
  interface pin gnd
.end
X1 net1 gnd res2
```

Subcircuit Call Statement Discrete Device Libraries

The Synopsys Discrete Device Library (DDL) is a collection of PrimeSim HSPICE device models of discrete components, which you can use with PrimeSim HSPICE. The `$installdir/parts` directory contains the various subdirectories that form the DDL. Synopsys used its own ATEM discrete device characterization system to derive the BJT, MESFET, JFET, MOSFET, and diode models from laboratory measurements. The behavior of op-amp, timer, comparator, SCR, and converter models closely resembles that described in manufacturers' data sheets.

Note:

The value of the `$INSTALLDIR` environment variable is the pathname to the directory where you installed PrimeSim HSPICE. This installation directory contains subdirectories, such as `/parts` and `/bin`. It also contains certain files, such as the `hspice.ini` file and the license files.

The following sections discuss these topics:

- [DDL Library Access](#)
- [Vendor Libraries](#)
- [Subcircuit Library Structure](#)

DDL Library Access

To include a DDL library component in a data file, use the `X` subcircuit call statement with the `DDL` element call. The `DDL` element statement includes the model name, which the actual DDL library file uses.

For example, the following element statement creates an instance of the 1N4004 diode model:

```
X1 2 1 D1N4004
```

Where `D1N4004` is the model name.

See [Element and Source Statements on page 158](#) and the *PrimeSim™ Continuum User Guide: Elements* for descriptions of element statements.

Optional parameter fields in the element statement can override the internal specification of the model. For example, for op-amp devices, you can override the offset voltage, and the gain and offset current. Because the DDL library devices are based on PrimeSim HSPICE circuit-level models, simulation automatically compensates for the effects of supply voltage, loading, and temperature.

PrimeSim HSPICE accesses DDL models in several ways:

- The installation script creates an hspice.ini initialization file.
- PrimeSim HSPICE writes the search path for the DDL and vendor libraries into a `.OPTION SEARCH='lib_path'` statement.

This provides immediate access to all libraries for all users. It also automatically includes the models in the input netlist. If the input netlist references a model or subcircuit, PrimeSim HSPICE searches the directory to which the `DDLPATH` environment variable points for a file with the same name as the reference name. This file is an include file so its filename suffix is `.inc`.

- Set `.OPTION SEARCH='lib_path'` in the input netlist.

Use this method to list the personal libraries to search. PrimeSim HSPICE first searches the default libraries referenced in the hspice.ini file, then searches libraries in the order listed in the input file.

- Directly include a specific model, using the `.INCLUDE` statement. For example, to use a model named T2N2211, store the model in a file named T2N2211.inc, and put the following statement in the input file:

```
.INCLUDE path/T2N2211.inc
```

This method requires you to store each model in its own `.inc` file, so it is not generally useful. However, you can use it to debug new models, when you test only a small number of models.

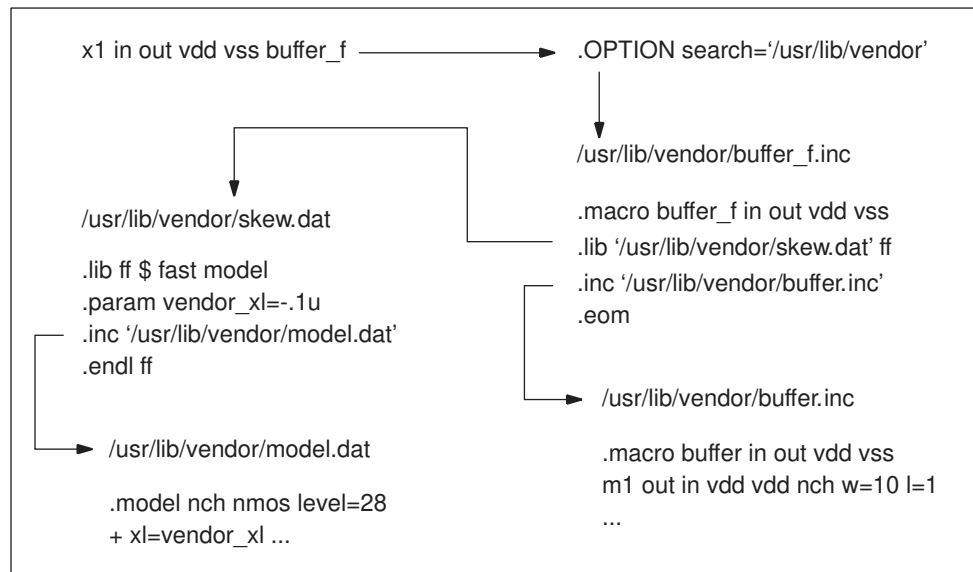
Vendor Libraries

The vendor library is the interface between commercial parts and circuit or system simulation.

- ASIC vendors provide comprehensive cells, corresponding to inverters, gates, latches, and output buffers.
- Memory and microprocessor vendors supply input and output buffers.
- Interface vendors supply complete cells for simple functions and output buffers, to use in generic family output.
- Analog vendors supply behavioral models.

To avoid name and parameter conflicts, models in vendor cell libraries should be within the subcircuit definitions.

Figure 24 Vendor Library Usage



Subcircuit Library Structure

The `.OPTION SEARCH` command is an implicit include command. The include file can contain `.lib` calls in addition to the subcircuit definition. (See Figure 24 for a graphic view.) Your library structure must adhere to the `.INCLUDE` statement specification in the implicit subcircuit. You can use this statement to specify the directory that contains the `subname.inc` subcircuit file, and then reference the *subname* in each subcircuit call.

The component naming conventions for each subcircuit is:

subname.inc

Store the subcircuit in a directory that is accessible by a `.OPTION SEARCH='lib_path'` statement.

Create subcircuit libraries in a hierarchy. Typically, the top-level subcircuit fully describes the input/output buffer; any hierarchy is buried inside. The buried hierarchy can include model statements, lower-level components, and parameter assignments. Your library cannot use `.LIB` or `.INCLUDE` statements anywhere in the hierarchy.

Part 2: Parameters, Functions, and Output

Describes how to use parameters within PrimeSim HSPICE netlists and use output format statements and variables to display the steady state, frequency, and time domain simulation results.

Part 2 presents the following chapters/topics:

- Parameters and Functions
- [Chapter 9, Simulation Output](#)
- Using Verilog-A

8

Parameters and Functions

This chapter has been moved to the PrimeSim Continuum Overview. For more information , see Parameters and Functions in the *PrimeSim™ Continuum Overview* guide.

9

Simulation Output

Describes how to use output format statements and variables to display the steady state, frequency, and time domain simulation results.

You can also use output variables in behavioral circuit analysis, modeling, and simulation techniques. To display electrical specifications such as rise time, slew rate, amplifier gain, and current density, use the output format features.

PrimeSim HSPICE ships hundreds of examples for your use; see Listing of Demonstration Input Files in the *PrimeSim HSPICE User Guide: Demonstration Netlists* for paths to the demo files.

For descriptions of individual PrimeSim HSPICE commands referenced in this chapter, see the *PrimeSim™ Continuum Reference Manual: Commands and Control Options*.

Note:

Parameter Storage Format (PSF) output supports all PrimeSim HSPICE analyses in the PrimeSim HSPICE integration to the Cadence® Virtuoso® Analog Design Environment.

Platform limitation: PC Windows does not support PSF format for PrimeSim HSPICE.

The following sections discuss these topics:

- [Overview of Output Statements](#)
- [Displaying Simulation Results](#)
- [Selecting Simulation Output Parameters](#)
- [Specifying User-Defined Analysis \(.MEASURE\)](#)
- [Expected State of Digital Output Signal \(.DOUT\)](#)
- [Reusing Simulation Output as Input Stimuli \(PrimeSim HSPICE Only\)](#)
- [Element Template Listings](#)
- [Vdmargin for Vdsat Characterization](#)

- [G-Function Based Vdsat, Gvdsat](#)
- [Output Listing \(*.lis\) File with .OPTION LIS_NEW Set](#)
- [For MOSFET Information Use .OPTION LIST](#)
-
- [Verilog-A Simulation Output](#)
- [Field Solver *.str File](#)
- [Redirecting the Simulation Output Results Files to a Different Directory](#)
- [Getting Data Out of PrimeSim HSPICE Plot Files](#)
- [Using the PrimeSim HSPICE Output Converter Utility](#)
- [Troubleshooting Issues](#)

Overview of Output Statements

The following sections discuss these topics:

- [Output Commands](#)
- [Output Variables](#)
- [Output Analysis Type](#)

Output Commands

The input netlist file contains output statements, including `.PRINT`, `.PROBE`, `.MEASURE`, `.DOUT`, and `.STIM`. Each statement specifies the output variables, and the type of simulation result, to display—such as `.DC`, `.AC`, or `.TRAN`. When you specify `.OPTION POST`, PrimeSim HSPICE puts all output variables referenced in `.PRINT`, `.PROBE`, `.MEASURE`, `.DOUT`, and `.STIM` statements into PrimeSim HSPICE output files.

Use the `.STIM` command if you want to reuse the simulation results in another simulation. For example:

```
.stim tran pwl filename=new v(2)
```

In the example, the `.STIM` command creates a file with an extension `.pwl0_tr#` and the results are in the form of a PWL source.

PrimeSim HSPICE advanced analog analyses supports only `.OPTION POST`, `.OPTION PROBE`, `.PRINT`, `.PROBE`, and `.MEASURE` statements. It does not support `.DOUT` or `.STIM` statements. Refer to the *PrimeSim HSPICE Reference Manual: Commands and Control*

Options for information on all listed statements. See [Table 23](#) for output table variable types.

Table 22 *Output Statements*

Output Statement	Description
.PRINT	Prints numeric analysis results in the output listing file (and post-processor data, if you specify .OPTION POST). See .PRINT.
.PROBE	Outputs data to post-processor output files, but not to the output listing (used with .OPTION PROBE, to limit output). See .PROBE.
.MEASURE	Prints the results of specific user-defined analyses (and post-processor data, if you specify .OPTION POST), to the output listing file. See .MEASURE.
.DOUT (PrimeSim HSPICE only)	Specifies the expected final state of an output signal. See .DOUT or Expected State of Digital Output Signal (.DOUT) .
.STIM (PrimeSim HSPICE only)	Specifies simulation results to transform to PWL, Data Card, or Digital Vector File format. See .STIM.

Output Variables

The output format statements require special output variables, to print or plot analysis results for nodal voltages and branch currents. PrimeSim HSPICE uses the following output variables:

- DC and transient analysis
- AC analysis
- element template (PrimeSim HSPICE only)
- .MEASURE statement
- parametric analysis

For PrimeSim HSPICE, DC and transient analysis displays:

- individual nodal voltages: $V(n1 [,n2])$
- branch currents: $I(Vxx)$
- element power dissipation: $In(element)$

AC *analysis* displays imaginary and real components of a nodal voltage or branch current, and the magnitude and phase of a nodal voltage or branch current. AC analysis results also print impedance parameters, and input and output noise.

Element template analysis displays element-specific nodal voltages, branch currents, element parameters, and the derivatives of the element's node voltage, current, or charge.

The `.MEASURE` statement variables define the electrical characteristics to measure in a `.MEASURE` statement analysis.

Parametric analysis variables are mathematical expressions, which operate on nodal voltages, branch currents, element template variables (PrimeSim HSPICE only), or other parameters that you specify. Use these variables when you run behavioral analysis of simulation results. See Using Algebraic Expressions.

Output Analysis Type

An output statement matches the last analysis command in the netlist before it. When there are no analysis statements before output statements with no analysis type, the PrimeSim HSPICE simulator automatically assigns the 'TRAN' analysis type to the output statements. It also adds a message in the output listing file to inform you that TRAN type was assigned to the `.meas/.probe/.print/.plot` statement without an analysis type, with no analysis statements before it.

Consider the following example:

```
.tran 20p 1.0n sweep sigma -3 3 0.5
.tran 20p 1.0n sweep monte=20
.meas mover max v(2,1)
```

In the preceding example, `.meas` matches the second `.tran` statement and generates only one measure output file.

Consider another example:

```
.meas max_m max v(2)
.print v(n3)
.dc vg 1 5 1
```

In the second example, there are no `.dc` or `.tran`, or `.ac` statements before the `.meas` and `.print` statements. Also, no analysis type is provided in the `.meas` and `.print` statements. Hence, the PrimeSim HSPICE simulator assigns the 'TRAN' analysis type to the `.meas` and `.print` statements. Also, there is only a `.dc` statement, so you cannot get any output from `.meas` and `.print` during DC analysis.

Displaying Simulation Results

The following sections describes the statements that you can use to display simulation results for your specific requirements:

- [.PRINT Statement](#)
- [.PROBE Statement](#)
- [Using Wildcards in PRINT and PROBE Statements](#)
- [Print Control Options](#)
- [Printing the Subcircuit Output](#)
- [Using .MODEL_INFO to Print Model Parameters](#)
- [Progress Message Time Estimates](#)

.PRINT Statement

The `.PRINT` statement specifies output variables for which PrimeSim HSPICE prints values.

To simplify parsing of the output listings, PrimeSim HSPICE prints a single `x` in the first column, to indicate the beginning of the `.PRINT` output data. A single `y` in the first column indicates the end of the `.PRINT` output data.

PrimeSim HSPICE advanced analog analyses prints the `.PRINT` output data to a separate file.

You can include wildcards in `.PRINT` statements.

You can also use the `iaall` keyword in a `.PRINT` statement, to print all branch currents of all diode, BJT, JFET, or MOSFET elements in your circuit design. PrimeSim HSPICE outputs complex data for `i(*)`, `i1(*)`, `i2(*)`, `i3(*)`, `i4(*)` of an AC analysis into `*.ac#` files when specifying the POST, PSF, CSDF or WDF formats for both the `.PRINT` and `.PROBE` commands.

Example

If your circuit contains four MOSFET elements (named `m1`, `m2`, `m3`, `m4`), then `.PRINT iaall (m*)` is equivalent to `.PRINT i(m1) i(m2) i(m3) i(m4)`. It prints the output currents of all four MOSFET elements. A resultant PSF file has only one signal, `i(c0)`, that contains two real and imaginary points (Re, Im) for each AC frequency point. In other words, if you open the PSF file in a waveform viewer, you see one signal name that contains both the real and Imaginary data, per frequency point.

Statement Order

PrimeSim HSPICE creates different `.sw0` and `.tr0` files, based on the order of the `.PRINT` and `.DC` statements. If you do not specify an analysis type for a `.PRINT` command, the type matches the last analysis command in the netlist, before the `.PRINT` statement.

.PROBE Statement

PrimeSim HSPICE usually saves all voltages, supply currents, and output variables. Set `.OPTION PROBE`, to save output variables only. Use the `.PROBE` statement to specify the quantities to print in the output listing.

If your interest is only in the output data file, and you do not want tabular or plot data in your listing file, set `.OPTION PROBE` and use `.PROBE` to select the values to save in the output listing.

You can include wildcards in `.PROBE` statements.

Using Wildcards in PRINT and PROBE Statements

You can include wildcards in `.PRINT` and `.PROBE` statements. Refer to this example netlist in the discussion that follows:

```
* test wildcard
.option post
v1 1 0 10
r1 1 n20 10
r20 n20 n21 10
r21 n21 0 10
.dc v1 1 10 1
***Wildcard equivalent for:
*.print i(r1) i(r20) i(r21) i(v1)
.print i(*)
***Wildcard equivalent for:
*.probe v(0) v(1)
.probe v(?)
***Wildcard equivalent for:
*.print v(n20) v(n21)
.print v(n2?)
***Wildcard equivalent for:
*.probe v(n20, 1) v(n21, 1)
.probe v(n2*, 1)
.end
```

The following sections discuss these topics:

- [Supported Wildcard Templates](#)
- [Using filter in .PRINT and .PROBE Statements](#)

- [Using level in .PRINT and .PROBE Statements](#)
- [Switching to .PROBE to Output Subcircuit Port Names](#)

Supported Wildcard Templates

```
v vm vr vi vp vdb vt
i im ir ii ip idb it
p pm pr pi pp pdb pt
lxn<n> lvn<n> (n is a number 0~9)
i1 im1 ir1 ii1 ip1 idb1 it1
i2 im2 ir2 ii2 ip2 idb2 it2
i3 im3 ir3 ii3 ip3 idb3 it3
i4 im4 ir4 ii4 ip4 idb4 it4
iall isub
```

For details on wildcard templates, see `.PRINT` statement (see [Selecting Simulation Output Parameters on page 198](#)).

When you use the wildcard `i(*)` in a `.print` or `.probe` statement, PrimeSim HSPICE outputs all branch currents.

For `.AC` analysis, to plot all currents for each valid AC output-variable type, you can also use the following in statements:

```
im(*) ir(*) ip(*) idb(*) it(*)
```

In the preceding test case (named `test wildcard`), if you use an `.AC` statement instead of a `.DC` statement, you can use any valid AC output-variable types with the wildcards `v(n2?)` and `v(n2*,1)`. For example:

```
vm(n2?) vr(n2?) vi(n2?) vp(n2?) vdb(n2?) vt(n2?)
vm(n2*,1) vr(n2*,1) vi(n2*,1) vp(n2*,1) vdb(n2*,1) vt(n2*,1)
```

To output the branch current at all terminals of a diode, BJT, JFET or MOSFET, use the output template `iall`. For example, `iall(m*)` is equivalent to:

```
i1(m*) i2(m*) i3(m*) i4(m*)
```

Using filter in .PRINT and .PROBE Statements

You can include the `filter` clause in `.PRINT` and `.PROBE` statements:

```
filter=pattern
```

PrimeSim HSPICE does not print nodes/elements that match the pattern specified in the `filter` clause when it prints node voltage(s) and/or element current(s) that you specify by wildcard patterns such as: `.print v(x1.x2.*)`. Each `filter` applies to all wildcard voltages/currents that the current `.print` or `.probe` statement prints.

For example:

```
.print v(x1.x2.*) i(x1.x2.*) filter='x1.x2.n*' filter='x1.x2.a*'
```

This syntax example prints the voltages of all nodes in subckt `x1.x2` that do not start with `n` or `a`, and the current of all elements in subckt `x1.x2` that do not start with either `n` or `a`.

Using level in .PRINT and .PROBE Statements

You can include the `level` clause in `.PRINT` and `.PROBE` statements:

```
level=val2
```

This setting is effective only when you specify the wildcard character in the output variable. The level value `val2` specifies the number of hierarchical depth levels when the wildcard node/element name matches.

- When you set `val2` to 1, the wildcard match applies to the same depth level where the `.print` statement is.
- When the `val2` is 2, it applies to the same level and to one level below the current level where `.print` is. When `val2` is -1, the wildcard match applies to all the depth levels below and including the current level of `.print` statement. The default value of `val2` is -1.

Switching to .PROBE to Output Subcircuit Port Names

In cases where you have a subcircuit whose port names are different than instance node names, you can replace the `.PRINT` command with `.PROBE` to print voltage values of a subcircuit.

For example, assume you have a netlist as follows:

```
x3 1 2 inv
.subckt inv in out
mn out in 0 0 nch w=1u l=90n
mp out in 1 1 pch w=1u l=90n
.ends
.print tran v(*)
```

By default PrimeSim HSPICE prints only the instance node names and not the subcircuit ports. If you include `.print v(*)` in the netlist, PrimeSim HSPICE outputs the top level instance nodes 1 and 2. However, if you want to print the subcircuit ports `in` and `out` then you need to use `.OPTION PROBE` along with an explicit `.PROBE` command. In this case: `.PROBE tran v(x1.in) v(x1.out)`

The following netlist outputs the subcircuit port waveforms:

```
.OPTION POST PROBE
x3 1 2 inv
.subckt inv in out
```

```
mn out in 0 0 nch w=1u l=90n
mp out in 1 1 pch w=1u l=90n
.ends
.PROBE tran v(x1.in) v(x1.out)
```

Print Control Options

The codes that you can use to specify the element templates for output in PrimeSim HSPICE are:

- `.OPTION INGOLD` for output in exponential form.
- `.OPTION POST` where an interactive waveform viewer displays plots.

PrimeSim HSPICE supports the following plot file formats: `*.tr#`, `*.ac#`, and `*.sw#`. If a plot fails to open, it is due to one of the following reasons:

- Does not support the waveform file format.
- Does not read the file format.
- Does not find the file.
- File exceeds the max size of (x).

Changing the File Descriptor Limit (PrimeSim HSPICE Only)

A simulation that uses a large number of `.ALTER` statements might fail because of the limit on the number of file descriptors. For example, for a workstation for which the default number of file descriptors is 64, a design with more than 50 `.ALTER` statements will probably fail with the following error message:

```
error could not open output spool file /tmp/tmp.nnn
a critical system resource is inaccessible or exhausted
```

To prevent this error on your PrimeSim HSPICE platform, ask your system administrator to help you increase the number of files that you can open concurrently.

Printing the Subcircuit Output

The following examples demonstrate how to print or plot voltages of nodes that are in subcircuit definitions, using `.PRINT`.

Note:

In the following example, you can substitute `.PROBE`, instead of `.PRINT`.

Example 1

```
.GLOBAL vdd vss
X1 1 2 3 nor2
X2 3 4 5 nor2
.SUBCKT nor2 A B Y
.PRINT v(B) v(N1) $ Print statement 1
M1 N1 A vdd vdd pch w=6u l=0.8u
M2 Y B N1 vdd pch w=6u l=0.8u
M3 Y A vss vss vss nch w=3u l=0.8u
M4 Y B vss vss nch w=3u l=0.8u
.ENDS
```

Print statement 1 prints out the voltage on the B input node, and on the N1 internal node for every instance of the nor2 subcircuit.

```
.PRINT v(1) v(X1.A) $ Print statement 2
```

The preceding .PRINT statement specifies two ways to print the voltage on the A input of the X1 instance.

```
.PRINT v(3) v(X1.Y) v(X2.A) $ Print statement 3
```

The preceding .PRINT statement specifies three different ways to print the voltage at the Y output of the X1 instance (or the A input of the X2 instance).

```
.PRINT v(X2.N1) $ Print statement 4
```

The preceding .PRINT statement prints the voltage on the N1 internal node of the X2 instance.

```
.PRINT i(X1.M1) $ Print statement 5
```

The preceding .PRINT statement prints out the drain-to-source current, through the M1 MOSFET in the X1 instance.

Example 2

```
X1 5 6 YYY
.SUBCKT YYY 15 16
X2 16 36 ZZZ
R1 15 25 1
R2 25 16 1
.ENDS
.SUBCKT ZZZ 16 36
C1 16 0 10P
R3 36 56 10K
C2 56 0 1P
.ENDS
.PRINT V(X1.25) V(X1.X2.56) V(6)
```


Value	Description
V(X1.25)	Local node to the YYY subcircuit definition, which the X1 subcircuit calls.
V(X1.X2.56)	Local node to the ZZZ subcircuit. The X2 subcircuit calls this node; X1 calls X2.
V(6)	Voltage of node 16, in the X1 instance of the YYY subcircuit.

This example prints voltage analysis results at node 56, within the X2 and X1 subcircuits. The full path, X1.X2.56, specifies that node 56 is within the X2 subcircuit, which in turn is within the X1 subcircuit.

Using .MODEL_INFO to Print Model Parameters

Use the command `.MODEL_INFO ALL |Instance1,...Instance2, ...` to generate a text output file with the suffix `*.model_info#`. (ALL overrides *Instance1,...Instance2*.) See `.MODEL_INFO` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

Different MOSFET instances may use the same model card, to avoid printing of duplicate model information and reduce the file size, so the unique model card prints only once.

The output file contains two parts, instance information and model information. The instance information section contains the instance name and its corresponding model name used. The model information section contains all model parameters of each unique model card. Each instance and `.model` statement is in one line. For example:

```
Output file: *.model_info#
<sweep param =...>
*** mosfets ***
Element Name          Model Name
Instance_name1        model_name
Instance_name2        model_name
...
*** models ***
.model      Model_name1    model_par1 =val1 model_par2=val2 ...
.model      Model_name2    model_par1 =val1 model_par2=val2 ...
```

Sample *.model_info# Output File

Input file: test.sp...

```
X1 c d test1
X2 a b test2

.model_info x1.m1 x2.m2
.subckt test1 d g
m1 d g dl pw nch w=0.2u l=0.03u
```

```
m2 d1 g s nw pch w=0.5u l=0.03u
.model pch.1 pmos level=54 version=4.6
+ binunit=1 paramchk=1 mobmod=0
+ ...
.model pch.2 pmos level=54 version=4.6
+ binunit=1 paramchk=1 mobmod=0
+ ...
.model pch.26 pmos level=54 version=4.6
```

Progress Message Time Estimates

During the simulation, simulation progress messages are displayed to the screen. In the simulation progress messages, `etc` is the estimated time to completion and `ett` is the estimated total time. See the sample below.

```
99.6% time = 199.200374 ns ( etc = 1:05, ett = 4:26:22 )
( wall = 4:25:16 cpu = 1:11:06:58 s=7.9424 )
99.7% time = 199.404835 ns ( etc = 49.4 sec, ett = 4:26:20 )
( wall = 4:25:30 cpu = 1:11:08:49 s=7.94239 )
99.8% time = 199.600000 ns ( etc = 32.9 sec, ett = 4:26:20 )
( wall = 4:25:47 cpu = 1:11:11:04 s=7.94241 )
99.9% time = 199.800806 ns ( etc = 16.5 sec, ett = 4:26:22 )
( wall = 4:26:05 cpu = 1:11:13:28 s=7.9424 )
100.0% time = 200.000000 ns ( etc = 0.0 sec, ett = 4:26:19 )
( wall = 4:26:19 cpu = 1:11:15:12 s=7.94241 ) 100.0% time = 200.000000 ns
( etc = 0.0 sec, ett = 4:26:19 )
( wall = 4:26:19 cpu = 1:11:15:12 s=7.94241 )
```

Selecting Simulation Output Parameters

Parameters provide the appropriate simulation output. To define simulation parameters, use the `.OPTION` and `.MEASURE` statements, and define specific variable elements.

The following sections discuss these topics:

- [DC and Transient Output Variables](#)
- [Power Output](#)
- [Operating Point Variables](#)
- [AC Analysis Output Variables](#)
- [Element Template Output \(PrimeSim HSPICE Only\)](#)

DC and Transient Output Variables

- Voltage differences between specified nodes (or between one specified node and ground).
- Current output for an independent voltage source.
- Current output for any element.
- Current output for a subcircuit pin.
- Element templates (PrimeSim HSPICE only). For each device type, the templates contain:
 - values of variables that you set
 - state variables
 - element charges
 - capacitance currents
 - capacitances
 - derivatives
 - user-specified element length
 - user-specified element width

[Print Control Options on page 195](#) summarizes the codes that you can use, to specify the element templates for output in PrimeSim HSPICE.

The following sections discuss these topics:

- [Nodal Capacitance Output](#)
- [Nodal Voltage](#)
- [Current: Independent Voltage Sources](#)
- [Terminal Voltage: MOS Instance](#)
- [Current: Element Branches](#)
- [Current: Subcircuit Pin](#)
- [Independent Source Power Output](#)
- [Wildcard Support](#)

- [User-defined Element Length and Width](#)
- [Power Output](#)

Nodal Capacitance Output

Syntax

`Cap (nxxx)`

For nodal capacitance output, PrimeSim HSPICE prints or plots the capacitance of the specified node `nxxxx`.

Example

```
.print dc Cap(5) Cap(6)
```

Nodal Voltage

Syntax

`V(n1[,n2])`

Parameter	Description
<code>n1, n2</code>	PrimeSim HSPICE prints or plots the voltage difference (<i>n1-n2</i>) between the specified nodes. If you omit <i>n2</i> , PrimeSim HSPICE prints or plots the voltage difference between <i>n1</i> and ground (node 0).

Current: Independent Voltage Sources

Syntax

`I (Vxxx)`

Parameter	Description
<code>Vxxx</code>	Voltage-source element name. If an independent power supply is within a subcircuit, then to access its current output, append a dot and the subcircuit name to the element name. For example, <code>I(X1.Vxxx)</code> .

Example

```
.PRINT TRAN I (VIN)
.PRINT DC I (X1.VSRC)
.PRINT DC I (XSUB.XSUBSUB.VY)
```

Terminal Voltage: MOS Instance

Syntax

`Vn(MOSFET_name)`

Parameter	Description
<i>n</i>	Node position number in the element statement. <i>n</i> is 1, 2, 3 or 4. For example: v1 is the first node (drain) voltage.
<i>MOSFET_name</i>	MOSFET instance name. To access current output for an element in a subcircuit, append a dot and the subcircuit name to the element name. For example, v3(X1.Wwww).

Example

The following example outputs the gate node voltage for the MOSFET.

```
.probe tran v2(XINST1.MN0)
```

Current: Element Branches

Note:

The direction of the current can follow the PrimeSim HSPICE-specific mode or the generic current polarity mode, based on use of the `.OPTION GEN_CUR_POL=ON|OFF`. The default is `OFF`. See `.OPTION GEN_CUR_POL` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

Syntax

`In(Wwww)`
`Iall(Wwww)`

Parameter	Description
<i>n</i>	Node position number, in the element statement. For example, if the element contains four nodes, I3 is the branch-current output for the third node. If you do not specify <i>n</i> , PrimeSim HSPICE assumes the first node.
<i>Wwww</i>	Element name. To access current output for an element in a subcircuit, append a dot and the subcircuit name to the element name. For example, I3(X1.Wwww).
<i>Iall(Wwww)</i>	An alias just for diode, BJT, JFET, and MOSFET devices. - If <i>Wwww</i> is a diode, it is equivalent to: I1(Wwww) I2(Wwww). - If <i>Wwww</i> is one of the other device types, it is equivalent to: I1(Wwww) I2(Wwww) I3(Wwww) I4(Wwww)

Example 1

I1 (R1)

This example specifies the current through the first R1 resistor node.

Example 2

I4 (X1.M1)

This example specifies the current, through the fourth node (the substrate node) of the M1 MOSFET, defined in the X1 subcircuit.

Example 3

I2 (Q1)

The last example specifies the current, through the second node (the base node) of the Q1 bipolar transistor.

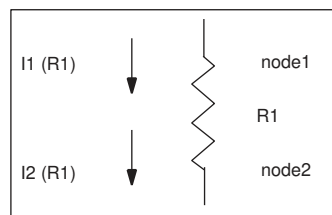
To define each branch circuit, use a single element statement. When PrimeSim HSPICE evaluates branch currents, by default, it inserts a zero-volt power supply, in series with branch elements to enable the PrimeSim HSPICE current direction rule to be more device-aware. The `.OPTION GEN_CUR_POL=ON|OFF` is available to allow use of Custom Compiler + PrimeSim HSPICE. When `.OPTION GEN_CUR_POL=ON`, the `i2( ) ... in( )` direction uses a generic direction rule, that is: the current *in* is positive, and the current *out* is negative (see [Figure 25 on page 202](#) through [Figure 30 on page 204](#)).

If PrimeSim HSPICE cannot interpret a `.PRINT` statement that contains a branch current, it generates a warning.

Branch current direction for the elements in s defined in terms of arrow-notation (current direction) and node-position number (terminal type). (See [Figure 25](#) though [Figure 30](#).)

Figure 25 Resistor (node1, node2)

.OPTION GEN_CUR_POL=OFF (default)



.OPTION GEN_CUR_POL=ON

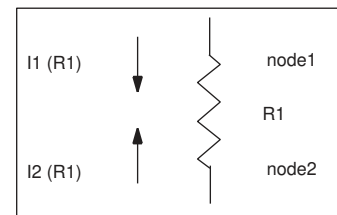
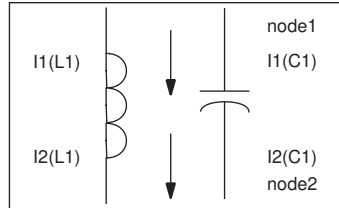


Figure 26 Inductor (node1, node2); capacitor (node 1, node2)

.OPTION GEN_CUR_POL=OFF (default)



.OPTION GEN_CUR_POL=ON

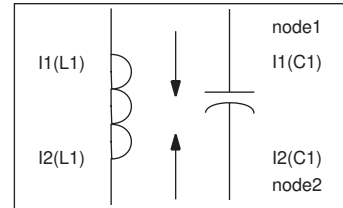
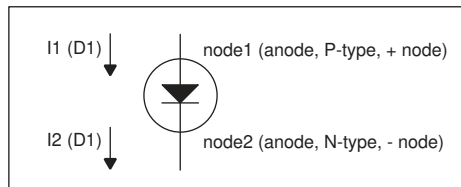


Figure 27 Diode (node1, node2)

.OPTION GEN_CUR_POL=OFF (default)



.OPTION GEN_CUR_POL=ON

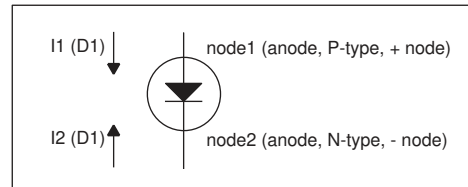
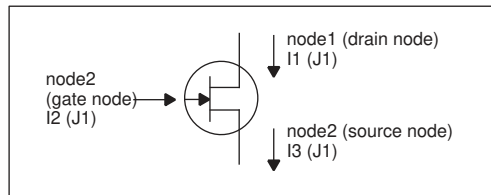


Figure 28 JFET (node1, node2, node3) - n-channel

.OPTION GEN_CUR_POL=OFF (default)



.OPTION GEN_CUR_POL=ON

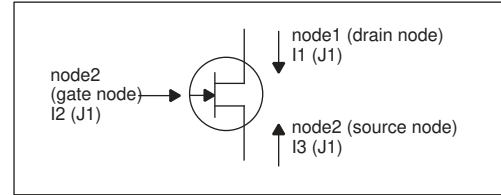
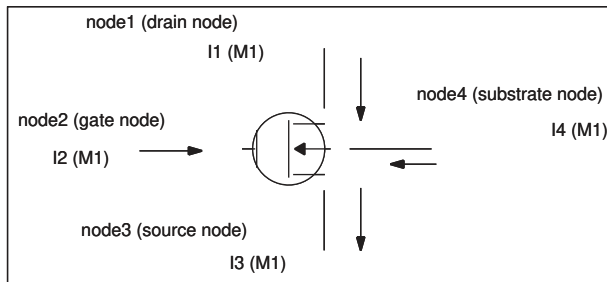


Figure 29 MOSFET (node1, node2, node3, node4) - n-channel

.OPTION GEN_CUR_POL=OFF (default)



.OPTION GEN_CUR_POL=ON

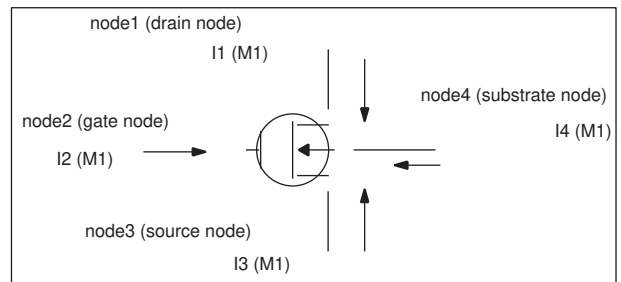
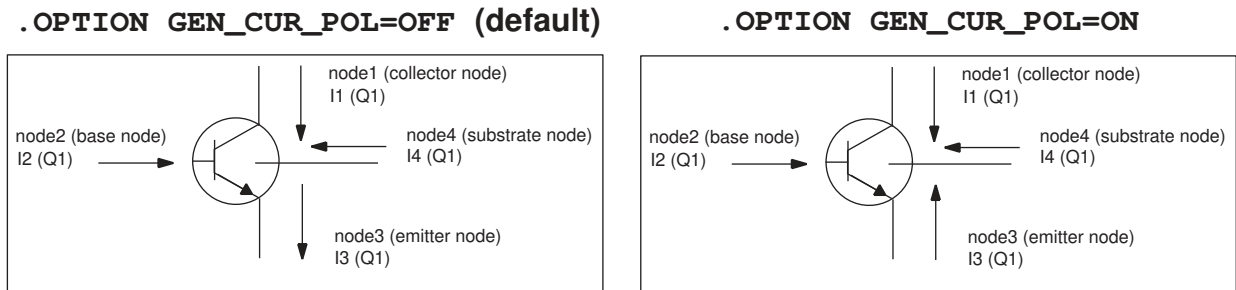


Figure 30 BJT (node1, node2, node3, node4) - npn



Current: Subcircuit Pin

Syntax

```
ISUB (X****.****)
```

Example

```
.PROBE ISUB (X1.PIN1)
```

Independent Source Power Output

Syntax

```
SRC_PWR
```

Example

```
.print [dc|tran] src_pwr
```

For power calculations, PrimeSim HSPICE computes dissipated or stored power in each passive element (R, L, C), and source (V, I, G, E, F, and H). To compute this power, PrimeSim HSPICE multiplies the voltage across an element, and its corresponding branch current.

However, for semiconductor devices, PrimeSim HSPICE calculates only the dissipated power. It excludes the power stored in the device junction or parasitic capacitances from the device power computation. The following sections show equations for calculating the power that different types of devices dissipate.

PrimeSim HSPICE also computes the total power of a circuit, which is the dissipated power + stored power. Total power is the negative value of sum of independent sources.

Note:

Since PrimeSim HSPICE compute only the dissipated power for semiconductor devices, total power is *not* the sum of power of elements that exclude independent sources.

Wildcard Support

Wildcard support is available for current subcircuit pins in single and multiple hierarchies using asterisk (*) and question mark (?) characters. (Exception: (?) is disallowed.) For example:

Single Hierarchy

```
.print isub(x1.*) isub(x1.a?)
```

Multi-level Hierarchy

```
.print isub(x1.x2.*) isub(x1.x?.a?)
```

User-defined Element Length and Width

```
.PRINT [DC | TRAN] ul(element_name)Length  
.PRINT [DC | TRAN] uw(element_name)Width
```

The user-defined length and width for MOSFETs, JFETs, BJTs, diodes, capacitors, and resistors can be printed. Supported analysis types are DC and transient.

Parameter	Description
ul	Element length specified by the user.
uw	Element width specified by the user.

Example 1: This example prints user-defined lengths.

```
.PRINT DC ul(m1)  
.PRINT TRAN ul(m1)
```

Example 2: This example prints user-defined widths.

```
.PRINT DC uw(m1)  
.PRINT TRAN uw(m1)
```

Power Output

The following sections discuss these topics

- [Total Dissipation Power](#)
- [Element Dissipated Power](#)
- [Element Total Power](#)

Total Dissipation Power

Syntax

`Power`

Examples

Output total dissipation power of DC or transient analysis.

```
.probe tran power
```

Element Dissipated Power

Syntax

`pdis(element_or_subcircuit_name) / p(element_or_subcircuit_name#`

Examples

Output resistor and sub-circuit dissipated power of DC or transient analysis.

```
.print dc pdis(r1) pdis(x1) p(rload)
```

Element Total Power

Syntax

`pdyn(element_or_subcircuit_name)`

Examples

Output mosfet and sub-circuit total power including both dissipated and dynamic power of DC or transient analysis.

```
.print tran pdyn(x1.x2.m1) pdyn(x1)
```

Operating Point Variables

When you set `PRINT/.PROBE OP`, only the OP information specified in the statement is output. These commands have a higher priority than the `.OP` command.

Nodal Voltage Syntax

`V(n1[,n2])`

Example 1: This example only outputs the operating point information of variables `I(VIN)` and `I(X1.VSRC)`.

```
.PRINT OP v(1) v(2)
```

Current: Independent Voltage Sources Syntax

I (Vxxx)

Example 2: This example only outputs the operating point information of the variables I (VIN) and I (X1.VSRC).

```
.PRINT OP I (VIN) I (X1.VSRC)
```

For detailed information on controlling the files to which operating point information is written see: .OP, .OPTION OPFILE and .OPTION SPLIT_DP.

AC Analysis Output Variables

Output variables for AC analysis include:

- Voltage differences between specified nodes (or between one specified node and ground).
- Current output for an independent voltage source.
- Current output for a subcircuit pin.
- Element branch current.
- Impedance (Z), admittance (Y), hybrid (H), and scattering (S) parameters.
- Input and output impedance, and admittance.

Table 23 lists AC output-variable types. In this table, the type symbol appends the variable symbol, to form the output variable name. For example, VI is the imaginary part of the voltage, or IM is the magnitude of the current.

Table 23 AC Output Variable Types

Type Symbol	Variable Type
DB	decibel
I	imaginary part
M	magnitude
P	phase
R	real part
T	group delay

Specify real or imaginary parts, magnitude, phase, decibels, and group delay for voltages and currents.

The following sections topics discuss these topics:

- [Algebraic Expressions](#)
- [Nodal Capacitance Output](#)
- [Nodal Voltage](#)
- [Current: Independent Voltage Sources](#)
- [Current: Element Branches](#)
- [Current: Subcircuit Pin](#)
- [Group Time Delay](#)
- [Network](#)
- [Noise and Distortion](#)

Algebraic Expressions

Syntax

```
PAR('algebraic expression')
```

Starting from PrimeSim HSPICE 2013.03-SP1, if the '*algebraic expression*' contains the node voltage or element branch current, the values of the expression displayed in *.ac# file is output in complex data including both real and imaginary components. The expression is evaluated based on the complex data type. If the '*algebraic expression*' does not contain any node voltage or element branch current, the values of expression are real type instead of complex type.

For a complex expression, the results of .PRINT in the .lis file are the magnitude of the complex data.

Example

Example 1:

```
.print ac Cxx=par('v(n1)-v(n2)')
```

In example 1, the results of Cxx are complex data in the *.ac# file and in the .lis file; they are magnitude values.

The real, imaginary and magnitude values are calculated as:

```
Cxx_real= real( v(n1) ) -real ( v(n2) )  
Cxx_imag=imag( v(n1) ) -imag (v(n2) )  
Cxx_mag=sqrt(Cxx_real*Cxx_real)+Cxx_imag*Cxx_imag)
```

Example 2:

```
.param p1=2 p2=3  
.print ac Rxx=par('p1-p2')
```

In example 2, the results of Rxx are real data in the *.ac# file.

Nodal Capacitance Output

Syntax

```
Cap (nxxx)
```

For nodal capacitance output, PrimeSim HSPICE prints the capacitance of the specified node nxxxx.

Example

```
.print ac Cap(5) Cap(6)
```

Nodal Voltage

Syntax

```
Vz (n1<, n2>)
```

Parameter	Description
z	Specifies the voltage output type (see Table 23)
n1, n2	Specifies node names. If you omit n2, PrimeSim HSPICE assumes ground (node 0).

Example

This example applies to PrimeSim HSPICE, but not to PrimeSim HSPICE advanced analog analyses. It prints the magnitude of the AC voltage of node 5, using the VM output variable. PrimeSim HSPICE uses the VDB output variable to print the voltage at node 5, and uses the VP output variable to print the phase of the nodal voltage at node 5.

```
.PRINT AC VM(5) VDB(5) VP(5)
```

Current: Independent Voltage Sources

Syntax

`IZ (Vxxx)`

Parameter	Description
<code>z</code>	Current output type (see Table 23).
<code>Vxxx</code>	Voltage-source element name. If an independent power supply is within a subcircuit, then to access its current output, append a dot and the subcircuit name to the element name. For example, <code>IM(X1.Vxxx)</code> .

Example

```
.PRINT AC IR(V1) IM(VN2B) IP(X1.X2.VSRC)
```

Current: Element Branches

Syntax

`IZn (Wwww)`

Parameter	Description
<code>z</code>	Current output type (see Table 23).
<code>n</code>	Node position number, in the element statement. For example, if the element contains four nodes, <code>IM3</code> denotes the magnitude of the branch current output for the third node.
<code>Wwww</code>	Element name. If the element is within a subcircuit, then to access its current output, append a dot and the subcircuit name to the element name. For example, <code>IM3(X1.Wwww)</code> .

```
.PRINT AC IP1(Q5) IM1(Q5) IDB4(X1.M1)
```

If you use the form `Im(Xxxx)` for AC analysis output, then PrimeSim HSPICE prints the magnitude value, `IMn(Xxxx)`.

Current: Subcircuit Pin

Syntax

`ISUB (X****,****)`

Example

```
.PROBE ISUB(X1.PIN1)
```

Group Time Delay

AC analysis associates the TD group time delay. TD is the negative derivative of the phase in radians, with respect to radian frequency. PrimeSim HSPICE uses the difference method to compute TD:

AC analysis associates T with the group time delay. The group time delay, T, is the negative derivative of the phase in radians, with respect to radian frequency. PrimeSim HSPICE uses the difference method to compute T:

$$T = -\frac{1}{360} \cdot \frac{(phase2 - phase1)}{(f2 - f1)}$$

phase1 and *phase2* are the phases (in degrees) of the specified signal, at the f1 and f2 frequencies (in hertz).

Syntax

```
.PRINT AC VT(10) VT(2,25) IT(RL)
.PRINT AC IT1(Q1) IT3(M15) IT(D1)
```

Note:

Because the phase has a discontinuity every 360°, T shows the same discontinuity, even though T is continuous.

Example

```
INTEG.SP ACTIVE INTEGRATOR
***** INPUT LISTING
*****
V1 1 0 .5 AC 1
R1 1 2 2K
C1 2 3 5NF
E3 3 0 2 0 -1000.0
.AC DEC 15 1K 100K
.PRINT AC VT(3) VP(3)
.END
```

Network

Syntax

$X_{ij}(z)$, $Z_{IN}(z)$, $Z_{OUT}(z)$, $Y_{IN}(z)$, $Y_{OUT}(z)$

Parameter	Description
X	Specifies Z (impedance), Y (admittance), H (hybrid), or S (scattering).
ij	i and j can be 1 or 2. They identify the matrix parameter to print.

Parameter	Description
z	Output type (see Table 23). If you omit z, PrimeSim HSPICE prints the magnitude of the output variable.
ZIN	Input impedance. For a one-port network, ZIN, Z11, and H11 are the same.
ZOUT	Output impedance.
YIN	Input admittance. For a one-port network, YIN and Y11 are the same.
YOUT	Output admittance.

Example

```
.PRINT AC Z11 (R) Z12 (R) Y21 (I) Y22 S11 S11 (DB)
.PRINT AC ZIN (R) ZIN (I) YOUT (M) YOUT (P) H11 (M)
.PRINT AC S22 (M) S22 (P) S21 (R) H21 (P) H12 (R)
```

Noise and Distortion

This section describes the variables used for noise and distortion analysis.

Syntax

```
ovar <(z)>
```

Parameter	Description
ovar	Noise and distortion analysis parameter. It can be ONOISE (output noise), INOISE (equivalent input noise), or any of the distortion analysis parameters (HD2, HD3, SIM2, DIM2, DIM3).
z	Output type (only for distortion). If you omit z, PrimeSim HSPICE outputs the magnitude of the output variable.

Example

```
.PRINT DISTO HD2 (M) HD2 (DB)
```

Prints the magnitude and decibel values of the second harmonic distortion component, through the load resistor that you specified in the `.DISTO` statement (not shown). You cannot use the `.DISTO` statement in PrimeSim HSPICE advanced analog analyses.

```
.PRINT NOISE INOISE ONOISE
```

Note:

You can specify the noise and distortion output variable, and other AC output variables, in the `.PRINT AC` statements.

Element Template Output (PrimeSim HSPICE Only)

The `.PRINT`, and `.PROBE` statements use element templates to output user-input parameters, state variables, stored charges, capacitor currents, capacitances, and derivatives of variables. See [Element Template Listings on page 228](#).

Syntax

The property syntax is:

Elname:Property

Parameter	Description
Elname	Name of the element.
Property	Property name of an element, such as a user-input parameter, state variable, stored charge, capacitance current, capacitance, or derivative of a variable.

Example:

```
.PRINT TRAN V(1,12) I(X2.VSIN) I2(Q3) DI01:GD  
.PRINT TRAN X2.M1:CGGBO M1:CGDBO X2.M1:CGSBO
```

The alias syntax is:

LVnn (Elname)
LXnn (Elname)

Parameter	Description
LV	Form to obtain output of user-input parameters, and state variables.
LX	Form to obtain output of stored charges, capacitor currents, capacitances, and derivatives of variables.
nn	Code number for the desired parameter (See Element Template Listings on page 228 and MOSFET Output Templates, Table 4, <i>Parameters in MOSFET Output Templates</i>).
Elname	Name of the element.

Example

```
.PRINT TRAN V(1,12) I(X2.VSIN) I2(Q3) DI01:GD  
.PRINT TRAN X2.M1:CGGBO M1:CGDBO X2.M1:CGSBO
```

Specifying User-Defined Analysis (.MEASURE)

Use the `.MEASURE` statement to modify information, and to define the results of successive PrimeSim HSPICE simulations. Computing the measurement results uses postprocessing output. If you use the `INTERP` option to reduce the size of the postprocessing output, then the measurement results can contain interpolation errors. For more information, see `.OPTION INTERP` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

This section describes the fundamental measurement modes in the following topics:

- [.MEASURE Statement Order](#)
- [.MEASURE Parameter Types](#)
- [FIND and WHEN Functions](#)
- [Continuous Measurement](#)
- [Equation Evaluation](#)
- [Average, RMS, MIN, MAX, INTEG, PP, and EM_AVG](#)
- [INTEGRAL Function](#)
- [DERIVATIVE Function](#)
- [ERROR Function](#)
- [Generating a Measure File \(*.mt0, *.ac0, etc.\) with All Values in a Single Row](#)
- [Outputting Pass/Fail Measure Data](#)
- [Measurements in MOSRA Analysis](#)

.MEASURE Statement Order

The `.MEASURE` statement matches the last analysis command in the netlist before the `.MEASURE` statement.

Example

```
.tran 20p 1.0n sweep sigma -3 3 0.5
.tran 20p 1.0n sweep monte=20
.meas mover max v(2,1)
```

In this example, `.meas` matches the second `.tran` statement and generates only one measure output file.

Users need to be aware that there are certain differences on how PrimeSim HSPICE handles `.measures` are handle in the case of complex measurements. Complex `.MEASURE` statements are measure statements dependent on other measure statements.

“Undefined variables” is a common reported issue in `.measure` cases for PrimeSim HSPICE that lead to failed measurements. You may not find the same case in the FASTSPICE tools due to the differences in how other tools process `.measures`.

Note:

PrimeSim HSPICE has a dependency requirement which may not exist in other formats. Fastspice simulators XA, Nanosim, and HSIM support various languages, including PrimeSim HSPICE and other major EDA simulators. These Fastspice simulators process `.measure` results in very flexible manner. These simulators read and reread a netlist for variable identification. Then, XA, NS, and HSIM perform simulation, write results, and then post-process the results to determine the `.measure` results.

If a `.MEASURE` statement does not execute, then PrimeSim HSPICE writes 0.0e0 in the `.mt#` file as the `.MEASURE` result, and writes FAILED in the output listing file.

Use `.OPTION MEASFAIL` to write results to the `.mt#`, `.ms#`, or `.ma#` files. For more information, see `.OPTION MEASFAIL` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

To control the output variables, listed in `.MEASURE` statements, use the `.PUTMEAS` option. For more information, see `.OPTION PUTMEAS` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

Note:

If a `.measure` statement uses the result of previous `.meas` statement, then the calculation starts when the previous calculation concludes. PrimeSim HSPICE outputs zero until the previous result concludes.

For information on measuring MOSFET parameters, see Measuring the Value of MOSFET Model Card Parameters.

.MEASURE Parameter Types

You cannot use measurement parameter results that the `.PARAM` statements in `.SUBCKT` blocks produce, outside of the subcircuit. That is, you cannot pass any measurement parameters defined in `.SUBCKT` statements, as bottom-up parameters in hierarchical designs.

Measurement parameter names must not conflict with standard parameter names. PrimeSim HSPICE issues an error message if it encounters a measurement parameter with the same name as a standard parameter definition.

To prevent .MEASURE statement parameters from overwriting parameter values in other statements, PrimeSim HSPICE keeps track of parameter types. If you use the same parameter name in both a .MEASURE statement and a .PARAM statement at the same hierarchical level, the simulation terminates and reports an error.

No error occurs if parameter assignments are at different hierarchical levels. .PRINT statements that occur at different levels do not print hierarchical information for parameter name headings.

Example

In PrimeSim HSPICE advanced analog analyses output, you cannot apply .MEASURE to waveforms generated from another .MEASURE statement in a parameter sweep.

The following example illustrates how PrimeSim HSPICE handles .MEASURE statement parameters.

```
...
.MEASURE tran length TRIG v(clk) VAL=1.4
+ TD=11ns RISE=1 TARGv(neq) VAL=1.4 TD=11ns
+ RISE=1
.SUBCKT path out in width=0.9u length=600u
+ rm1 in m1 m2mg w='width' l='length/6'
...
.ENDS
```

In the above listing, the *length* in the resistor statement:

```
rm1 in m1 m2mg w='width' l='length/6'
```

does not inherit its value from *length* in the .MEASURE statement:

```
.MEASURE tran length ...
```

because they are of different types.

The correct value of *l* in *rm1* should be:

```
l=length/6=100u
```

In transient analysis, you should not derive the value from a measured value.

FIND and WHEN Functions

The `FIND` and `WHEN` functions of the `.MEASURE` statement specify to measure:

- Any independent variables (time, frequency, parameter).
- Any dependent variables (voltage or current for example).
- Derivative of a dependent variable, if a specific event occurs.

You can use these measure statements in unity gain frequency or phase measurements. You can also use these statements to measure the time, frequency, or any parameter value:

- When two signals cross each other.
- When a signal crosses a constant value.

The measurement starts after a specified time delay, `TD`. To find a specific event, set `RISE`, `FALL`, or `CROSS` to a value (or parameter), or specify `LAST` for the last event.

`LAST` is a reserved word; you cannot use it as a parameter name in the above measure statements. For definitions of parameters of the measure statement, see [Displaying Simulation Results on page 191](#).

For a full demonstration file for `FIND` and `WHEN` functions follow the path to `ampgain.sp`, which sets unity gain frequency of a BJT differential pair, in Circuit Optimization Examples in the *PrimeSim HSPICE User Guide: Demonstration Netlists*.

Continuous Measurement

The continuous measurement feature allows you to specify the continuous measurement of a result derived from a DC, AC, or transient analysis. This feature only applies to `TRIG-TARG` and `Find-When` functions. For example:

```
.measure tran_cont vout1 find v(out1) when v(a1)=2.5 fall=1
```

The `.measure` statements continuously finds the voltage `out1` when the voltage value of node `a1` reaches to 2.5 starting from the first falling edge.

See `.MEASURE (Continuous Results)` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

Continuous Measure Output Files (Default)

PrimeSim HSPICE outputs the continuous measure output into a separate files of the following types:

```
output_prefix_measure_result.mt#  
output_prefix_measure_result.ms#  
output_prefix_measure_result.ma#
```

The file is in text format and readable directly. For example:

```
.measure tran_cont vout1_cont find v(out1) when v(a1)=2.5 fall=1
```

The additional output file name is t1_vout1_cont.mt, if the output file prefix is t1.

Note:

Continuous measurement output is *not* written to the *.lis file.

The following is an example of an output file for the measure statement:

```
.measure tran_cont crossing when v(1) = v(2) t1_crossing.mt  
1.      crossing, result=1.000000000000e-09  
2.      crossing, result=1.000000000000e-07  
3.      crossing, result=2.000000000000e-07  
4.      crossing, result=3.000000000000e-07  
5.      crossing, result=4.000000000000e-07  
6.      crossing, result=5.000000000000e-07  
7.      crossing, result=6.000000000000e-07  
8.      crossing, result=7.000000000000e-07  
9.      crossing, result=8.000000000000e-07  
10.     crossing, result=9.000000000000e-07  
11.     crossing, result=1.000000000000e-06
```

Continuous Measure Output Files (With .OPTION MEASFORM=2)

PrimeSim HSPICE outputs the continuous measure output into one file when .option measform=2 is set:

```
output_prefix_cont.mt#  
output_prefix_cont.ms#  
output_prefix_cont.ma#
```

For example:

```
.measure tran_cont crossing when v(t1) = v(t2)  
.measure tran_cont vt2 when v(t1) = 4 rise=1  
.measure tran_cont period trig v(t1) val=4 rise=1 targ v(t1) val=4 rise=2
```

The additional output file name is output_cont.mt#, if the output file prefix is output.

Note:

Continuous measurement output is *not* written to the *.lis file.

The following is an example of an output file for the measure statement:

```
crossing@1 =      0.
crossing@2 =  2.271034737e-07
crossing@3 =  6.814809009e-07
...
vt2@1 =  1.475989329e-07
vt2@2 =  1.147681340e-06
vt2@3 =  2.147720241e-06
...
period@1 =  1.000082408e-06      trig =  1.475989329e-07      targ
      =  1.147681340e-06
period@2 =  1.000038900e-06      trig =  1.147681340e-06      targ
      =  2.147720241e-06
period@3 =  9.999610996e-07      trig =  2.147720241e-06      targ
      =  3.147681340e-06
...
temp =  25.0000000000
alter# =  1.0000000000
```

Equation Evaluation

Use the Equation Evaluation form of the `.MEASURE` statement to evaluate an equation that is a function of the results of previous `.MEASURE` statements. The equation must not be a function of node voltages or branch currents.

The `expression` option is an arithmetic expression that uses results from other prior `.MEASURE` statements. If `equation` or `expression` includes node voltages or branch currents, Unexpected results may incur.

Average, RMS, MIN, MAX, INTEG, PP, and EM_AVG

Average (AVG), RMS, MIN, MAX, and peak-to-peak (PP) measurement modes report statistical functions of the output variable, rather than analysis values.

- AVG calculates the area under an output variable divided by the periods of interest.
- RMS divides the square root of the area under the output variable square by the period of interest.
- MIN reports the minimum value of the output function over the specified interval.
- MAX reports the maximum value of the output function over the specified interval.
- PP (peak-to-peak) reports the maximum value minus the minimum value over the specified interval.

Note:

AVG, RMS, and INTEG have no meaning in a DC data sweep, so if you use them, PrimeSim HSPICE issues a warning message.

- **EM_AVG** Calculates the average electromigration current. For a symmetric bipolar waveform, the current is: $I_{avg}(0, T/2) - R \cdot I_{avg}(T/2, T)$, where R is the recovery factor you specify by using `.option em_recovery`. This measurement also supports wildcards.

Measuring Recovered Electromigration

The `.MEAS` keyword, `EM_AVG`, enables you to calculate “recovered” average current due to electromigration. Recovered average current is specially meaningful for bipolar currents (such as output of the inverter), as the mathematical average for such a waveform is zero. The keyword uses the From-To measurement function to provide a range to measure. For example:

```
.measure tran em em_avg I(out) from=5n to=50n
```

where `out` is the node which specifies the measurement point.

The example does the following operations:

1. Measure the average of positive part of the waveform (`Ipos_avg`) from 5ns to 50ns.
2. Measure the average of negative part (absolute value) of the waveform (`Ineg_avg`) from 5ns to 50ns.
3. Does the operation “ $\max(Ipos_avg, Ineg_avg) - R \cdot \min(Ipos_avg, Ineg_avg)$ ”,

Where R is a user-provided coefficient using `.option em_recovery=value`. The default value of `em_recovery` is 1. See `.OPTION EM_RECOVERY` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

4. The polarity of `em_avg` current is same as the polarity of the $\max(Ipos_avg, Ineg_avg)$. Positive, if $\text{abs}(\text{pos})$ is more than $\text{abs}(\text{neg})$ and otherwise.

For this feature PrimeSim HSPICE also supports wildcards (*) during `em_avg` measurement. For example:

```
.meas tran em em_avg I(m*) from=10n to=100n
```

INTEGRAL Function

The `INTEGRAL` function reports the integral of an output variable, over a specified period.

DERIVATIVE Function

The `DERIVATIVE` function provides the derivative of:

- An output variable, at a specified time or frequency.
- Any sweep variable, depending on the type of analysis.
- A specified output variable, when some specific event occurs.

In the following PrimeSim HSPICE advanced analog analyses example, the `SLEW` measurement provides the slope of `V(OUT)` during the first time, when `V(1)` is 90% of `VDD`.

```
.MEAS TRAN SLEW DERIV V(OUT) WHEN V(1)='0.90*VDD'
```

ERROR Function

The relative error function reports the relative difference between two output variables. You can use this format in optimization and curve-fitting of measured data. The relative error format specifies the variable to measure and calculate from the `.PARAM` variable. To calculate the relative error between the two, PrimeSim HSPICE uses the `ERR`, `ERR1`, `ERR2`, or `ERR3` function. With this format, you can specify a group of parameters to vary, to match the calculated value and the measured data. The following discusses the `ERR` equations:

Error Equations

ERR

1. `ERR` sums the squares of $(M-C)/\max(M, \text{MINVAL})$ for each point.
2. It then divides by the number of points.
3. Finally, it calculates the square root of the result.
 - `M` (`meas_var`) is the measured value of the device or circuit response.
 - `C` (`calc_var`) is the calculated value of the device or circuit response.
 - `NPTS` is the number of data points.

$$ERR = \left[\frac{1}{NPTS} \cdot \sum_{i=1}^{NPTS} \left(\frac{M_i - C_i}{\max(\text{MINVAL}, M_i)} \right)^2 \right]^{\frac{1}{2}}$$

ERR1

ERR1 computes the relative error at each point. For NPTS points, PrimeSim HSPICE calculates NPTS ERR1 error functions. For device characterization, the ERR1 approach is more efficient than the other error functions (ERR, ERR2, ERR3).

$$ERR1_i = \frac{M_i C_i}{\max(MINVAL, M_i)}, i=1, NPTS$$

PrimeSim HSPICE does not print out each calculated ERR1 value. When you set the ERR1 option, PrimeSim HSPICE calculates an ERR value, as follows:

$$ERR = \left[\frac{1}{NPTS} \cdot \sum_{i=1}^{NPTS} ERR1_i^2 \right]^{\frac{1}{2}}$$

ERR2

This option computes the absolute relative error, at each point. For NPTS points, PrimeSim HSPICE calls NPTS error functions.

$$ERR2_i = \left| \frac{M_i C_i}{\max(MINVAL, M_i)} \right|, i=1, NPTS$$

The returned value printed for ERR2 is:

$$ERR = \frac{1}{NPTS} \cdot \sum_{i=1}^{NPTS} ERR2_i$$

ERR3

$$ERR3_i = \frac{\pm \log \left| \frac{M_i}{C_i} \right|}{\left| \log [\max(MINVAL, |M_i|)] \right|}, i=1, NPTS$$

The + and - signs correspond to a positive and negative M/C ratio.

Note:

If the M measured value is less than MINVAL, PrimeSim HSPICE uses MINVAL instead. If the absolute value of M is less than the IGNOR or YMIN value, or greater than the YMAX value, the error calculation does not consider this point.

Generating a Measure File (*.mt0, *.ac0, etc.) with All Values in a Single Row

The `MEASFORM` option allows you to change how the measure values are written to the measure file. If you set `MEASFORM=1`, then PrimeSim HSPICE writes the measure values in a single space-delimited row.

You can also use `MEASFORM=3` to output values in a single row and generate the file in CSV file format (*.csv) that can be opened in Excel by double-clicking on the file name.

By default, a transient measure file (*.mt0) looks like:

tdelay	vmax	vmin	trise
tfall	temper	alter#	
1.781e-09	5.0072	-4.504e-03	4.778e-09
3.386e-09	25.0000	1	

By using `.OPTION MEASFORM=3`, the measure file appears as:

tdelay	vmax	vmin	trise	tfall	temper
alter#					
1.781e-09	5.0072	-4.504e-03	4.778e-09	3.386e-09	25.0000 1

Outputting Pass/Fail Measure Data

You can use `.measure` to create a logic equation that describes the pass/fail condition. It outputs a 0/1 value for pass/fail in the `.mt0` file. For example:

```
.meas m1 find v(1) at=10n
.meas m2 find v(2) at=10n
.meas pass param="(m1 > 1) && (m2 < 2)"
```

Measurements in MOSRA Analysis

When working with measurements in a MOSFET model reliability analysis (MOSRA), you need to use a workaround for measurements beyond the first `mt0` file. For example, in the following netlist, four measures fail:

```
.param t_step=600u
.tran 0.1u '2*t_step'
vBGR BGR 0 pulse( 0 3 0ns 10us 10us 50u 100u)
.meas tran vbgr_out find V(BGR) at='t_step/20'
.meas tran trise1 when V(BGR)='0.9*vbgr_out'
rise=last
.mosra reltotaltime=3.15e+8 relstep=7.88e+7
.option post=1 probe
.end
```

The file returns the following:

```
result of mt0
vbgr_out  trisel    temper    alter#
3.0000    1.109e-03  25.0000    1.0000
results of mt1,2,3,4
vbgr_out  trisel    temper    alter#
3.0000    failed    25.0000    1.0000
```

The first `.measure` statement is independent of `.MOSRA` analysis, so the first measure file (`*.mt0`) succeeds. The second `.measure` statement depends on `.MOSRA` analysis, therefore next four measure files (`mt1`, `mt2`, `mt3`, `mt4`) fail.

Workaround

The workaround to this issue is:

1. Run the simulation without the `.measure` statement. This generates `*.tr0`, `*.tr1` ... files.
2. Copy the `.measure` statement to another file named (for example) `measure_file`.

```
.param t_step=600u
.meas tran vbgr_out find V(BGR) at='t_step/20'
.meas tran trisel when V(BGR)='0.9*vbgr_out' rise=last
.end
```

3. Run the post process measure utility as follows to successfully generate all measurement files:

```
hspice -meas measure_file -i *.tr1 -o tr1.lis
hspice -meas measure_file -i *.tr2 -o tr2.lis
hspice -meas measure_file -i *.tr3 -o tr3.lis
hspice -meas measure_file -i *.tr4 -o tr4.lis
```

Expected State of Digital Output Signal (.DOUT)

The digital output (`.DOUT`) statement specifies the expected final state of an output signal (PrimeSim HSPICE only). During simulation, PrimeSim HSPICE compares the simulated results with the expected output vector. An error results if states are different. The `.DOUT` statement uses either of two syntaxes. In both syntaxes, the time and state parameters define the expected output of the `nd` node.

- The first syntax specifies a single threshold voltage, `VTH`. Any voltage level above `VTH` is high; any level below `VTH` is low.

```
.DOUT nd VTH (time state time_state)
```

where:

`nd` is the node name

`VTH` is the single voltage threshold

`time` is an absolute time-point (max 60)

`state` is one of the following expected conditions of the `nd` node, at the specified time:

0: expect ZERO, LOW

1: expect ONE, HIGH

else: Do not care

- The second syntax defines a threshold for both a logic high (`VHI`) and low (`VLO`).

`.DOUT nd VLO VHI (time state time_state)`

where:

`nd` is the node name

`VLO` is the voltage of the logic low state

`VHI` is the voltage of the logic high state

`time` is an absolute time-point (max 60)

`state` is one of the following expected conditions of the `nd` node, at the specified time:

0: expect ZERO, LOW

1: expect ONE, HIGH

else: Do not care

Note:

If you specify both syntaxes (`VTH`, plus `VHI` and `VLO`), then PrimeSim HSPICE processes only `VTH`, and ignores `VHI` and `VLO`.

For both cases, the time, state pair describes the expected output. During simulation, PrimeSim HSPICE compares the simulated results against the expected output vector. If the states are different, PrimeSim HSPICE reports an error.

The legal values for `state` are:

- 0: Expect ZERO
- 1: Expect ONE
- X, x: Do not care.

- U, u: Do not care
- Z, z: Expect HIGH IMPEDANCE (do not care).

Example

The `.PARAM` statement in the following example sets the value of the `VTH` variable to 3. The `.DOUT` statement operates on the `node1` node and uses `VTH` as its threshold voltage.

```
.PARAM VTH = 3.0
.DOUT node1 VTH(0.0n 0 1.0n 1
+ 2.0n X 3.0n U 4.0n Z 5.0n 0)
```

When `node1` is above 3V, PrimeSim HSPICE considers it a logic **1**; otherwise, it is a logic **0**.

- At 0ns, the expected state of `node1` is logic-low
- At 1ns, the expected state is logic-high
- At 2ns, 3ns, and 4ns, the expected state is “do not care”
- At 5ns, the expected state is again logic-low

PrimeSim HSPICE supports multiple nodes in the `.DOUT` statement. This enables you to verify signals at the same time point in a single `.DOUT` statement.

For example: `.DOUT B C D (0n 1 1 0 5n 0 0 0)`

Reusing Simulation Output as Input Stimuli (PrimeSim HSPICE Only)

You can use the `.STIM` statement to reuse the results (output) of one simulation, as input stimuli in a new simulation.

Note:

`.STIM` is an abbreviation of `.STIMULI`. You can use either form to specify this statement in PrimeSim HSPICE.

The `.STIM` statement specifies:

- Expected stimulus (PWL source, data card, or VEC file).
- Signals to transform.
- Independent variables.

One `.STIM` statement produces one corresponding output file. To control the precision and data format, you can use the same options as you would in a normal simulation. For example:

```
.option numdgt=6      $ sets precision, range is 0 to 10, numdgt=4
                      is the default
.option ingold=0      $ sets format, 0=eng 1=combined 2=exponential
```

These settings affect how data prints out for your entire testcase. There is no way to only affect the `.STIM` command because the simulation data is the source of the output of the `.STIM` command.

For the syntax and description of the `.STIM` statement, see the `.STIM` command in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

Output Files

The `.STIM` statement generates the following output files:

Output File Type	Extension
PWL Source	*.pwl\$ _{tr} # The <code>.STIM</code> statement writes PWL source results to <code>output_file.pwl\$_{tr}#</code> . This output file results from a <code>.STIM [tran] pwl</code> statement in the input file.
Data Card	.dat\$ _{tr} #, .dat\$ _{ac} #, or .dat\$ _{sw} #The <code>.STIM</code> statement writes DATA Card results to <code>output_file.dat\$_{sw}#</code> , <code>output_file.dat\$_{ac}#</code> , or <code>output_file.dat\$_{tr}#</code> . This output file is the result of a <code>.stim [tran ac dc] data</code> statement in the input file.
Digital Vector File	.vec\$ _{tr} #The <code>.STIM</code> statement writes Digital Vector File results to <code>output_file.vec\$_{tr}#</code> . This output file is the result of a <code>.stim [tran] vec</code> statement in the input file.

Symbol	Description
tr ac sw	- tr=transient analysis. - ac=AC analysis. - sw=DC sweep analysis.
#	Either a sweep number, or a hard-copy file number. For a single sweep run, the default number is 0.
\$	Serial number of the current <code>.STIM</code> statement, within statements of the same stimulus type (pwl, data, or vec).\$=0 ~ n-1 (n is the number of the <code>.STIM</code> statement of that type). The initial \$ value is 0. For example, if you specify three <code>.STIM pwl</code> statements, PrimeSim HSPICE generates three PWL output files, with the suffix names <code>pwl0__{tr}#</code> , <code>pwl1__{tr}#</code> , and <code>pwl2__{tr}#</code> .

Element Template Listings

- [Resistor \(R-element\)](#)
- [Capacitor \(C-element\)](#)
- [Inductor \(L-element\)](#)
- [Mutual Inductor \(K-element\)](#)
- [Voltage-Controlled Current Source \(G-element\)](#)
- [Voltage-Controlled Voltage Source \(E-element\)](#)
- [Current-Controlled Current Source \(F-element\)](#)
- [Current-Controlled Voltage Source \(H-element\)](#)
- [Independent Voltage Source \(V-element\)](#)
- [Independent Current Source \(I-element\)](#)
- [Diode \(D-element\)](#)
- [BJT \(Q-element\)](#)
- [JFET \(J-element\)](#)
- [Saturable Core Element \(K-element\)](#)
- [Saturable Core Winding](#)
- [Vdmargin for Vdsat Characterization](#)
- [G-Function Based Vdsat, Gvdsat](#)

A full and extensive listing of MOSFET output templates, is in the *PrimeSim HSPICE Reference Manual: MOSFET Models*, MOSFET Output Templates.

Table 24 *Resistor (R-element)*

Name	Alias	Description
G	LV1	Conductance at analysis temperature.
R	LV2	Resistance at analysis temperature.
TC1	LV3	First temperature coefficient.
TC2	LV4	Second temperature coefficient.

Table 25 *Capacitor (C-element)*

Name	Alias	Description
CEFF	LV1	Computed effective capacitance.
IC	LV2	Initial condition.
Q	LX0	Charge, stored in capacitor.
CURR	LX1	Current, flowing through capacitor.
VOLT	LX2	Voltage, across capacitor.

Table 26 *Inductor (L-element)*

Name	Alias	Description
LEFF	LV1	Computed effective inductance.
IC	LV2	Initial condition.
FLUX	LX0	Flux, in the inductor.
VOLT	LX1	Voltage, across inductor.
CURR	LX2	Current, flowing through inductor.

Table 27 *Mutual Inductor (K-element)*

Name	Alias	Description
K	LV1	Mutual inductance.

Table 28 *Voltage-Controlled Current Source (G-element)*

Name	Alias	Description
CURR	LX0	Current, through the source, if VCCS.
R	LX0	Resistance value, if VCR.
C	LX0	Capacitance value, if VCCAP.
CV	LX1	Controlling voltage.
CQ	LX1	Capacitance charge, if VCCAP.
DI	LX2	Derivative of the source current, relative to the control voltage.

Table 28 *Voltage-Controlled Current Source (G-element) (Continued)*

Name	Alias	Description
ICAP	LX2	Capacitance current, if VCCAP.
VCAP	LX3	Voltage, across capacitance, if VCCAP.

Table 29 *Voltage-Controlled Voltage Source (E-element)*

Name	Alias	Description
VOLT	LX0	Source voltage.
CURR	LX1	Current, through source.
CV	LX2	Controlling voltage.
DV	LX3	Derivative of the source voltage, relative to the control current.

Table 30 *Current-Controlled Current Source (F-element)*

Name	Alias	Description
CURR	LX0	Current, through source.
CI	LX1	Controlling current.
DI	LX2	Derivative of the source current, relative to the control current.

Table 31 *Current-Controlled Voltage Source (H-element)*

Name	Alias	Description
VOLT	LX0	Source voltage.
CURR	LX1	Source current.
CI	LX2	Controlling current.
DV	LX3	Derivative of the source voltage, relative to the control current.

Table 32 *Independent Voltage Source (V-element)*

Name	Alias	Description
VOLT	LV1	DC/transient voltage.

Table 32 *Independent Voltage Source (V-element) (Continued)*

Name	Alias	Description
VOLTM	LV2	AC voltage magnitude.
VOLTP	LV3	AC voltage phase.

Table 33 *Independent Current Source (I-element)*

Name	Alias	Description
CURR	LV1	DC/transient current.
CURRM	LV2	AC current magnitude.
CURRP	LV3	AC current phase.

Table 34 *Diode (D-element)*

Name	Alias	Description
AREA	LV1	Diode area factor.
AREAX	LV23	Area, after scaling.
IC	LV2	Initial voltage, across diode.
PJ	LV3	Diode perimeter
VD	LX0	Voltage, across diode (VD), excluding RS (series resistance).
IDC	LX1	DC current, through diode (ID), excluding RS. Total diode current is the sum of IDC and ICAP.
GD	LX2	Equivalent conductance (GD).
QD	LX3	Charge of diode capacitor (QD).
ICAP	LX4	Current, through the diode capacitor. Total diode current is the sum of IDC and ICAP.
C	LX5	Total diode capacitance.
PID	LX7	Photo current, in diode.

Table 35 BJT (Q-element)

Name	Alias	Description
AREA	LV1	Area factor.
ICVBE	LV2	Initial condition for base-emitter voltage (VBE).
ICVCE	LV3	Initial condition for collector-emitter voltage (VCE).
MULT	LV4	Number of multiple BJTs.
FT	LV5	FT (Unity-gain bandwidth).
ISUB	LV6	Substrate current (DC only, not accounted for charge-induced current).
GSUB	LV7	Substrate conductance.
LOGIC	LV8	LOG 10 (IC).
LOGIB	LV9	LOG 10 (IB).
BETA	LV10	BETA.
LOGBETAI	LV11	LOG 10 (BETA) current.
ICTOL	LV12	Collector current tolerance.
IBTOL	LV13	Base current tolerance.
RB	LV14	Base resistance.
GRE	LV15	Emitter conductance, 1/RE.
GRC	LV16	Collector conductance, 1/RC.
PIBC	LV18	Photo current, base-collector.
PIBE	LV19	Photo current, base-emitter.
VBE	LX0	VBE.
VBC	LX1	Base-collector voltage (VBC).
CCO	LX2	Collector current (CCO) (DC only, not accounted for charge-induced current).
CBO	LX3	Base current (CBO) (DC only, not accounted for charge-induced current).
GPI	LX4	$g_{\pi} = I_{ib} / V_{be}$, constant vbc.
GU	LX5	$g_{\mu} = I_{ib} / V_{bc}$, constant vbe.
GM	LX6	$g_m = I_{ic} / V_{be} + I_{ic} / V_{be}$, constant vce.
G0	LX7	$g_0 = I_{ic} / V_{ce}$, constant vbe.

Table 35 *BJT (Q-element) (Continued)*

Name	Alias	Description
QBE	LX8	Base-emitter charge (QBE).
CQBE	LX9	Base-emitter charge current (CQBE).
QBC	LX10	Base-collector charge (QBC).
CQBC	LX11	Base-collector charge current (CQBC).
QCS	LX12	Current-substrate charge (QCS).
CQCS	LX13	Current-substrate charge current (CQCS).
QBX	LX14	External base-collector charge (QBX).
CQBX	LX15	External base-collector charge current (CQBX).
GXO	LX16	1/Rbeff Internal conductance (GXO).
CEXBC	LX17	Base-collector equivalent current (CEXBC).
CAP_BE	LX19	cbe capacitance (C_{π}).
CAP_IBC	LX20	cbc internal base-collector capacitance (C_{μ}).
CAP_SCB	LX21	csc substrate-collector capacitance for vertical transistors.csb substrate-base capacitance for lateral transistors.
CAP_XBC	LX22	cbcx external base-collector capacitance.
CMCMO	LX23	$1/(TF \cdot I_{BE}) / I_{VBC}$.
VSUB	LX24	Substrate voltage.

Table 36 *JFET (J-element)*

Name	Alias	Description
AREA	LV1	JFET area factor.
VDS	LV2	Initial condition for drain-source voltage.
VGS	LV3	Initial condition for gate-source voltage.
PIGD	LV16	Photo current, gate-drain in JFET.
PIGS	LV17	Photo current, gate-source in JFET.
VGS	LX0	VGS.
VGD	LX1	Gate-drain voltage (VGD).

Table 36 *JFET (J-element) (Continued)*

Name	Alias	Description
CGSO	LX2	Gate-to-source (CGSO).
CDO	LX3	Drain current (CDO).
CGDO	LX4	Gate-to-drain current (CGDO).
GMO	LX5	Transconductance (GMO).
GDSO	LX6	Drain-source transconductance (GDSO).
GGSO	LX7	Gate-source transconductance (GGSO).
GGDO	LX8	Gate-drain transconductance (GGDO).
QGS	LX9	Gate-source charge (QGS).
CQGS	LX10	Gate-source charge current (CQGS).
QGD	LX11	Gate-drain charge (QGD).
CQGD	LX12	Gate-drain charge current (CQGD).
CAP_GS	LX13	Gate-source capacitance.
CAP_GD	LX14	Gate-drain capacitance.
QDS	LX16	Drain-source charge (QDS).
CQDS	LX17	Drain-source charge current (CQDS).
GMBS	LX18	Drain-body (backgate) transconductance (GMBS).

Table 37 *Saturable Core Element (K-element)*

Name	Alias	Description
MU	LX0	Dynamic permeability (μ), Weber/(amp-turn-meter).
H	LX1	Magnetizing force (H), Ampere-turns/meter.
B	LX2	Magnetic flux density (B), Webers/meter ² .

Table 38 *Saturable Core Winding*

Name	Alias	Description
LEFF	LV1	Effective winding inductance (Henry).
IC	LV2	Initial condition.

Table 38 Saturable Core Winding (Continued)

Name	Alias	Description
FLUX	LX0	Flux, through winding (Weber-turn).
VOLT	LX1	Voltage, across winding (Volt).

Vdmargin for Vdsat Characterization

Output template `Vdmargin(m*)`, alias: `LX286(m*)`, is provided to probe the Vdmargin simulation result of either selected or all MOSFET elements. The template prints in the *.lis file and as .sw0/.tr0 files. For example:

```
.print dc Vdmargin(m1) Vdmargin(m2)
```

For operating point analysis, Vdmargin is reported in the OP output; for DC sweep, Vdmargin is calculated at every sweep point; for Tran analysis, only the points specified in the .OP command are calculated.

Here, PrimeSim HSPICE only calculates Vdmargin at the following points: 1n, 2.5n, 10n:

```
.op 1n 2.5n 10n
```

If Vdmargin is not found in the sweep range, PrimeSim HSPICE reports `Vdmargin=Vd0`, with the following warning message:

```
**Warning** Vdmargin of m1 is out of range
```

See .IVDMARGIN and .OPTION IVDMARGIN in the *PrimeSim HSPICE Reference Manual: Commands and Control Options* for more information.

G-Function Based Vdsat, Gvdsat

Output template `Gvdsat(m*)`, alias: `LX288(m*)`, is provided to probe the Gvdsat simulation result of either a selected or all MOSFET elements. The template prints in the *.lis file and the .sw0/.tr0 files.

Example:

```
.print dc Gvdsat(m1) Gvdsat(m2)
```

For operating point analysis, Gvdsat is reported in the OP output; for DC sweep, Gvdsat is calculated at every sweep point; for transient analysis, only the points specified in the .OP command are calculated.

In the following example, PrimeSim HSPICE only calculates Gvdsat at points 1n, 2.5n, and 10n:

```
.op 1n 2.5n 10n
```

For more information, see Saturation Voltage Correction — The G Function Method in the *PrimeSim HSPICE Reference Manual: MOSFET Models*.

Output Listing (*.lis) File with .OPTION LIS_NEW Set

The following is a sample of what PrimeSim HSPICE generates during a simulation in the output listing file *.lis when you set .OPTION LIS_NEW in the netlist. See [Output Listing File](#) for discussion of the contents of the file.

```
*****
<< header information>>
***** Loading Files *****
Loading '<path>/top.sp '
Loading '<path>/netlist.spc '
Loading '<path>/library.lib '
Loading '<path>/nets.spf
<< any warning messages >>

***** Options set in the netlist *****
ingold  = 1          lis_new = 1          measdg  = 7
numdgt  = 7          post    = binary    probe   = 1
runlvl  = 5          scale   = 900.0000000m

***** Summary of possible problems in netlist *****
Negative resistor :      0
Negative capacitor:      0
Negative inductor :      0
Unconnected node  :      0
Dangling terminal :      0
Source loop       :      0

***** Circuit Statistics *****
Resistors          :    218
Capacitors(total)  :    544
Coupling Capacitors :    304
Voltage Sources    :      2
MOSFETs           :     68
Total # of Nodes   :    417
Total # of Elements :    832

***** Analysis Details *****
dcop: begin dcop,  cpu clock=  5.00E-01          **info** dc convergence

successful at GMINDC ramping method
<< print first line of the netlist>>
project: ring oscillator
```


Chapter 9: Simulation Output

Output Listing (*.lis) File with .OPTION LIS_NEW Set

```

***** transient analysis tnom= 25.000 temp= 25.000 *****
dcop: end dcop,  cpu clock= 7.90E-01 peak memory=      191 mb
tot_iter=      625

Opening '<path>/top.tr0' for
output.
Waveform Format: POST=Binary, POST_VERSION=2013
Transient Beginning
10.14% Tran Time= 2.02706e-07s Estimated Time Left: 0:04:28
20.30% Tran Time= 4.05987e-07s Estimated Time Left: 0:03:53
30.11% Tran Time= 6.02234e-07s Estimated Time Left: 0:03:23
40.26% Tran Time= 8.05232e-07s Estimated Time Left: 0:02:53
50.08% Tran Time= 1.00161e-06s Estimated Time Left: 0:02:24
60.24% Tran Time= 1.20471e-06s Estimated Time Left: 0:01:55
70.33% Tran Time= 1.40665e-06s Estimated Time Left: 0:01:26
80.07% Tran Time= 1.60148e-06s Estimated Time Left: 0:00:57
90.12% Tran Time= 1.80236e-06s Estimated Time Left: 0:00:28
100.00% Tran Time= 2.00000e-06s Estimated Time Left: 0:00:00
Transient End
*****
project: ring oscillator
***** transient analysis tnom= 25.000 temp= 25.000 *****

***** Measured values for the netlist *****
tdelay= 1.3030378e-08 targ= 2.5672644e-08 trig= 1.2642266e-08
***** job concluded *****
project: ring oscillator
***** job statistics summary tnom= 25.000 temp= 25.000 *****
***** HSPICE Threads Information *****
Command Line Threads Count : 1
Available CPU Count : 16
Actual Threads Count : 1
***** Runtime Statistics (seconds) *****
analysis time # points tot. iter conv.iter
op point 0.29 1 625
transient 289.66 20001 1277658 459780 rev=
27837
readin 0.41
errchk 0.01
setup 0.01
output 0.00
peak memory used 196.11 megabytes
total cpu time 290.38 seconds
total elapsed time 291.15 seconds
job started at 10:28:30 06/29/2016
job ended at 10:33:21 06/29/2016
lic: Release hspice token(s)
lic: total license checkout elapse time: 0.17(s)
*****

```

In the `Runtime Summary`, note the following definitions:

- `# Points`: `TSTOP/TSTEP+1`
- `tot.iter`: Total number of iterations in transient analysis taken by the tool to find the solution.
- `conv.iter`: Count of only the iterations that converge. This number equals the number of time points which PrimeSim HSPICE evaluates to form the waveforms.
- `rev` (on the `transient` row): Number of times that the simulator had to reject timestep (reversals). This measures how difficult the design is to simulate; if `rev` is very high, it means the circuit is difficult to converge. Each rejected timestep equals eight iterations and no solution.
- When you use `.OPTION LIS_NEW=1`, any `.PRINT` statement in your netlist generates a text file containing the simulation results. For a transient analysis, the file has the extension, `.printtr#`.

For MOSFET Information Use .OPTION LIST

To print the effective width and length after scaling include `.OPTION LIST` in your netlist. Then search for `mosfets` in your listing file. For example:

```
**** mosfets
**** BSIM4 Model (Level 54)
element name      1:mn1      1:mn2      1:mn3      1:mn4
drain             1:104      1:158      1:164      1:160
gate              1:0       1:102      1:214      1:106
source            1:157      1:195      1:159      1:165
bulk              0:vss     0:vss     0:vss     0:vss
model             0:nch     0:nch     0:nch     0:nch
w eff             288.0000n  288.0000n  288.0000n  288.0000n
l eff             90.0000n  90.0000n  90.0000n  90.0000n
```

You can see `w eff` and `l eff` (scaled) for each MOS instance in the `circuit name directory` section. The "1" in `1:mn1` refers to the subckt instance number:

```
***** circuit name directory
circuit number to circuit name directory
  number circuitname      definition      multiplier
    0 main circuit
    1 xmn1.                nch_mac      1.00
```

Where, "1:mn1" refers to `xmn1.mn1`.

You can also use PrimeSim HSPICE MOSFET output templates. For details, see MOSFET Output Templates in the *PrimeSim HSPICE Reference Manual: MOSFET Models*. To illustrate here, the general syntax is:

```
.print tran | dc LV1(mn1)  $ effective length for all MOSFET
                           models except levels 54, 57, 69 and 70.
.print tran | dc LV2(mn1)  $ effective width for all MOSFET
                           models except levels 54, 57, 69 and 70.
.print tran | dc LX63(mn1) $ effective length for MOSFET model
                           levels 54, 57, 69 and 70.
.print tran | dc LX62(mn1) $ effective width for MOSFET model
                           levels 54, 57, 69 and 70.
```

For MOS models inside subckts, preface the MOS instance name with the subckt name:

```
.print tran LX62(xmn1.mn1) $ using the subckt name
.print tran LX62(1:mn1)    $ using the subckt number
```

You can also wildcard the subckt name and/or the MOS instance name:

```
.print tran LX63(*.mn1)    $ all MN1 lengths one level down
.print tran LX63(XMN1.*)   $ all MOS instance lengths below XMN1
.print tran LX63(*.*)      $ all MOS instance lengths two levels down
.print tran LX63(*)        $ all MOS instance lengths in the design
```

The following is a complete example, naming the outputs width and length:

```
.print tran width=par('lx62(1:mn1)') length=par('lx63(1:mn1)')
```

The output in your listing file then appears as follows:

time	width	length
0.	288.0000n	90.0000n
100.00000p	288.0000n	90.0000n
200.00000p	288.0000n	90.0000n
300.00000p	288.0000n	90.0000n

Verilog-A Simulation Output

The PrimeSim HSPICE standard output files consist of these basic files:

- The *.valog file, Verilog-A log file, which contains Verilog-A specific message from compiling and simulating phase. The contents of *.valog file also echoes to the *.lis file.
- Compiled Verilog-A code (*.pvalib file) (when you compile Verilog-A modules manually).

Verilog-A Output Directory

The Verilog-A output directory `-o.pvadir/` contains the following shared output (*.so) intermediate files on Linux/UNIX (and *.dll files on the Windows platform).

File Type	Contains
pvaRTL.log	Log file output from gcc compilation output
pvaRTL_linux.so	Runtime library input to PrimeSim HSPICE and PrimeSim HSPICE advanced analog analyses simulators
pvaEnv	Storage information for incrementally compiling user Verilog-A files

For example:

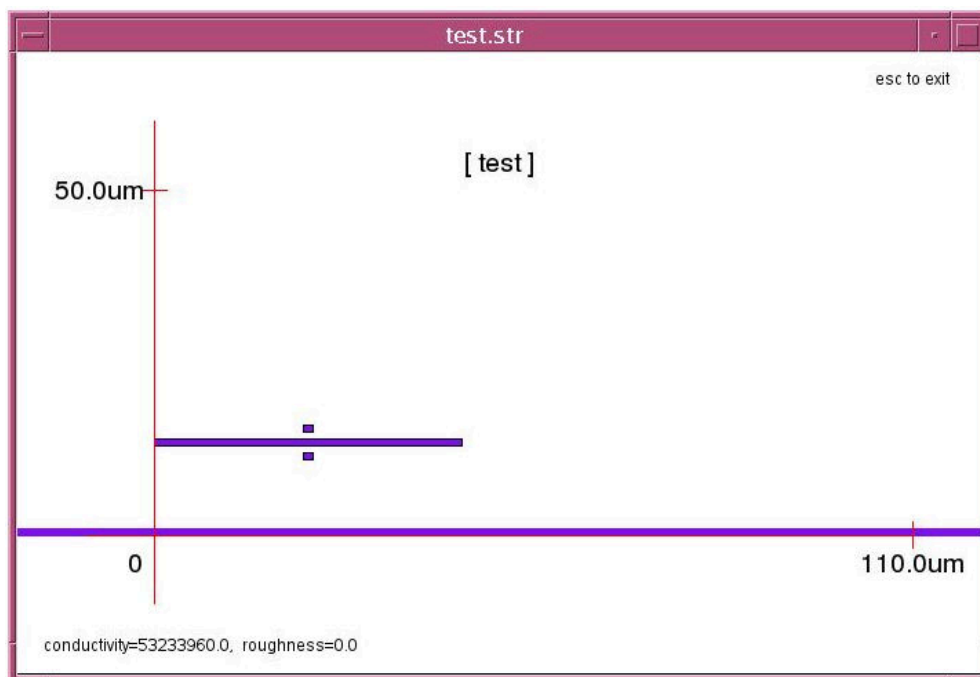
Command-line statement	Creates this directory:
<code>hspice test.sp -o output</code>	<code>output.pvadir</code>
<code>hspice test.sp</code>	<code>test.pvadir</code>
<code>hspice test.sp -o test</code>	<code>test.pvadir</code>
<code>hspice test.sp -o /remote/hspice/benchmark</code>	<code>/remote/hspice/ benchmark.pvadir</code>

For further information, see Using Verilog-A.

Field Solver *.str File

PrimeSim HSPICE creates an *.str file when a field solver model is created. The *.str file is a Tcl interpretation of the layerstack. View the layerstack by using the “wish” command on Linux machines. (No equivalent command exists on Windows machines.) Mouse over the dielectric and conductors to see the characteristics of the materials reported in the lower left corner of the window.

Figure 31 Field Solver *.str file



While you can use the “wish” command to view your layerstack, you cannot zoom, edit, or netlist out a new stack. You can edit the *.str file. However, any changes you make are not reflected in your field solver model.

For detailed information, see *Visualizing Cross-Sectional Geometric Information in the PrimeSim HSPICE User Guide: Signal Integrity Modeling and Analysis*.

Redirecting the Simulation Output Results Files to a Different Directory

If you need to redirect the simulation-output result files to a directory other than the current working directory, use either of the following two options. At a command line prompt, enter either:

```
% hspice -i test.sp -o /root/user/hspice/result/test.lis
```

PrimeSim HSPICE redirects the simulation results to the specified location /root/user/hspice/result. Alternatively, enter:

```
% hspice -i test.sp -o results/test.lis
```

Where, PrimeSim HSPICE redirects the simulation results to the specified folder with respect to the current working directory. However, you should create the destination folder

before you start the simulation. Otherwise, PrimeSim HSPICE returns an error message and aborts.

Directing .PRINT Output to a Separate File

Set `.OPTION LIS_NEW` in your netlist, to print the data from `.PRINT` statements to a separate file.

Refer to `.OPTION LIS_NEW` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options* for more details.

Getting Data Out of PrimeSim HSPICE Plot Files

The waveform file formats `*.tr0`, `*.ac0`, etc., are proprietary formats. PrimeSim HSPICE designed these formats for native waveform viewers and not for other programs or scripts. PrimeSim HSPICE does not publish proprietary formats, as these files can change between releases following enhancements.

However, there are some alternatives for extracting waveform data from PrimeSim HSPICE. The following example illustrates some common methods. This example performs a simple frequency sweep of a tank circuit and creates some AC analysis points. At about 160Hz (the resonant frequency of the circuit) the current drops to its lowest point then goes back up. The `.probe` command selects the signals desired in the table data. Here is the netlist:

```
Title: tank circuit frequency sweep
* reference: http://www.allaboutcircuits.com/vol_2/chpt_6/2.html
.option ingold=1 probe resmin=1e-15 csdf
v1 1 0 ac 1
c1 1 0 10u ic=1
r1 1 2 1e-12
l1 2 0 100m ic=.1
.ac lin 20 100 200
.print ac im(v1)
.plot ac im(v1)
.probe ac im(v1)
.stim data filename=foo my_current im(v1)
.end
```

Likely, the best option is to use **File > Export Waveform Data** in WaveView (or CTRL-E).

1. You name the output file, set a step size, and the result looks like this:

```
#format table ## [WaveView] 14:00:27 Mon Aug 24 2009
FREQ      im(v1)
1.000E+002 9.632E-003
1.010E+002 9.418E-003
1.020E+002 9.204E-003
```

```
1.030E+002  8.990E-003
1.040E+002  8.776E-003
1.050E+002  8.562E-003
...
```

2. You can use `.option csdf` (instead of `post`) to create a column-delimited format that is much easier to parse, especially with a limited number of signals. The `*.ac0` file still stores this data, in but is now in the CSDF format.

```
#N 'im(v1) '
#C 1.00000e+002  1  9.63231e-003 /0.0
#C 1.05263e+002  1  8.50584e-003 /0.0
#C 1.10526e+002  1  7.45516e-003 /0.0
#C 1.15789e+002  1  6.46993e-003 /0.0
...
```

Note:

If you accidentally leave `.option post` in your netlist and it occurs after `csdf`, the file reverts back to the traditional plot file format and PrimeSim HSPICE issues the message:

```
**warning** multiple output options specified, using post
```

3. Use the “stim” data card. In the example above, you create an ASCII file `foo.dat0_ac0`. However, the file does not preserve the frequency values, only the magnitudes. In the `.stim` card, “data” is a keyword that creates a file which PrimeSim HSPICE can read in with the `.data` statement but is also user-readable. The output file is takes the name “foo”, and the data set name is “my_current”.

```
.data my_current
imv1
 9.632e-03
 8.506e-03
 7.455e-03
 6.470e-03
 5.542e-03
 4.663e-03
...
```

4. When you use `.OPTION LIS_NEW=1`, any `.PRINT` statement in your netlist generates a text file containing the simulation results. For a AC analysis, the file has the extension, `.printac#`.

```
freq      i mag          v1
100.00000  9.632e-03
105.26316  8.506e-03
110.52632  7.455e-03
115.78947  6.470e-03
```

```
121.05263      5.542e-03
...
```

5. Use the PrimeSim HSPICE Output Converter Utility, documented in the following section, [Using the PrimeSim HSPICE Output Converter Utility](#). The converter utility converts *.tr#, *.ac#, and *.sw# files to PSF and PWL/DATA/VEC files.

A final important point is when dealing with AC voltages and currents: be sure to specify which of the complex parts you want in your output. The default is real + imaginary but you can select which components you get by adding a “modifier” after the `v` for voltage:

- `.print ac vm(v1)` — voltage magnitude
- `.print ac vr(v1)` — the real part
- `.print ac vi(v1)` — the imaginary part
- `.print ac vdb(v1)` — voltage in dB

For a discussion of how to control the time point intervals of data in a plot file, refer to [How TSTEP Affects a Transient Simulation](#).

Compressing Waveform Files

PrimeSim HSPICE can generate compressed waveform files. You need to add the `-gz` command to the `hspice` command line. For example:

```
hspice -i test.sp -o test -gz
```

PrimeSim HSPICE generates a compressed format waveform file named `test.tr0.gz`. For many cases, your use of compression saves disk space by 30% to 40%. Compression is supported for transient, AC, and DC analysis. You can use Custom Explorer/Waveview to view the compressed waveform files generated by PrimeSim HSPICE.

Using the PrimeSim HSPICE Output Converter Utility

This section describes how to convert output generated by PrimeSim HSPICE.

The converter utility is a post-process tool that converts the output files (*.tr#, *.ac#, and *.sw#) generated by PrimeSim HSPICE. Use the converter to get the Parameter Storage Format (PSF) output files directly from the .tr#, .ac#, or .sw# files generated by PrimeSim HSPICE with the `POST` output control option. Or, use the converter to get the PWL Source, DATA Card, and Digital Vector File (VEC) from the *.tr#, *.ac#, and *.sw# files generated by PrimeSim HSPICE with the `POST` or `CSDF` control options. You can reuse these stimuli in a new simulation.

Note:

PrimeSim HSPICE advanced analog analyses does not support the converter utility.

The following sections discuss these topics:

- [PSF Converter](#)
- [PWL/DATA/VEC Converter](#)

PSF Converter

Table 39 Supported Platforms

Linux RHEL	Linux SUSE	Windows
Yes	Yes	No

Syntax

```
converter -t PSF -i input_file [-o output_file] [-a |-b]
```

Table 40 PSF Converter Parameters

Argument	Description
-t	Specifies the file type (must be <code>psf</code>).]
-i	Specifies input file name. The input file must be the output file that PrimeSim HSPICE generates with the POST output control option.
-o	Specifies output file name. The converter assigns a <code>.psf</code> as the extension of the output file. If you do not specify the output file name, the converter appends <code>_psf</code> to the root name of the input file, and it remains the extension of the input file.
-a	Specifies the ASCII format for the output file.
-b	Specifies the binary format for the output file. By default, the output file is in binary format. The content included in angled brackets (< >) is optional.

Example

```
converter -t PSF -i testpost.tr0 -o testpsf
```

The input file is `testpost.tr0`, which PrimeSim HSPICE generates with the POST option. The output file name is `testpsf`. After running, PrimeSim HSPICE generates two new files: `testpsf.psf` and `logFile`. The `testpsf.psf` file is a PSF file that you can view

with the Analog Waveform Display (AWD). The `logFile` is necessary for the AWD to load the waveform.

PWL/DATA/VEC Converter

Linux SUSE	Linux RHEL	Windows
Yes	Yes	Yes

The PWL/DATA/VEC Converter is mainly for reusing previous simulation results directly from the `*.tr#`, `*.ac#`, and `*.sw#` files produced by PrimeSim HSPICE. The converter is in accordance with the `.STIM` statement in the PrimeSim HSPICE netlist.

Syntax

```
converter -t PWL/DATA/VEC -i input_file <-o output_file>
```

Table 41 PWL/DATA/VEC Converter Parameters

Argument	Description
-t	Specifies the type of the stimulus (PWL).
-i	Specifies the input file name. Input files are the output files which PrimeSim HSPICE generates with the <code>POST=x</code> or <code>CSDF=x</code> output control options.
-o	Specifies the output file name. If you do not specify the output file name, the converter automatically assigns the following file names: - input_filename.tr#_PWL# - input_filename.ac#_DAT# - input_filename.tr#_DAT# - input_filename.sw#_DAT# - input_filename.tr#_VEC# The content included by angled brackets (< >) is optional.

Note:

Transient analysis must generate the input file for PWL and VEC.

The following sections discuss these topics:

- [Prompt User Mode](#)
- [Input Line Dependencies](#)
- [Running the Converter Utility in Batch Mode](#)

Prompt User Mode

The PWL/DATA/VEC Converter is a prompt user mode. The converter displays corresponding prompts and asks you to input some data after you start it successfully.

Input the following at the command line and press the Enter key:

```
Converter -t PWL -i sample.tr0
```

The following input prompts appear one at a time and require your specified entries on the command line.

1. Enter the number of output variables(>0):

Specify the number of output variables from the waveform file to convert.

2. Enter output variables reused:

Specify the name of the node(s) in the design to convert. The node names must match a node name in the waveform file that you are converting.

3. Enter name of the source (optional):

If you do not specify a source, the source name is `vmnode_name`.

4. Enter positive node name (optional):

If you do not specify a positive node, the positive node name is be the same as node name(s) specified for the output variable(s).

5. Enter negative node name (optional):

If you do not specify a negative node, PrimeSim HSPICE specifies 0 (ground) as the negative node for each node name.

6. Enter independent variable type [1--from/to, 2--dispersed]:

This input line is optional. If you do in put anything and you press the Enter key, the input prompts end, the executable automatically runs and generates the `design_name.tr0_PWL0` stimuli file that contains all time points from the original waveform file.

If you specify an independent variable type, the utility displays the following prompts. For each prompt, you need to provide a value.

7. For 1-- from/to,

Enter the start point:

Starting point of the output file.

8. Enter the end point:

Ending point of the output file.

9. Enter the number of output points:

Number of output points.

10. For 2-- :dispersed

Enter the dispersed points:

Enter a list of time points you want to write to the file.

Once you enter the necessary information at the last prompt and press the Return key, the executable automatically runs and generates the `design_name.tr0_PWL0` stimuli files.

Input Line Dependencies

The input lines you use must adhere to the following conditions:

- Variables used in a PWL source must be voltage or current signals.
- Variables used in a VEC file can only be voltage signals.
- PWL Source Names must begin with V or I.
- Dispersed time points must be increasing in value when the stimulus type is PWL or VEC.
- For the optional items, you can enter the Return Key directly to adopt the default value.

Running the Converter Utility in Batch Mode

While the converter is interactive, prompting you with a series of questions, you can run in batch mode by redirecting input from an “answer” file.

The command to run the converter in batch mode has two parts and requires two files. The first file (see the following batch file), invokes the converter and tells it the waveform file to use. The second file is the “answer file” containing the answers to the conversion questions. The section titled [Prompt User Mode](#) lists the questions asked by the converter. You can create sample batch files using the following syntax:

```
converter -t pwl -i file1.tr0 < answers1.txt
converter -t pwl -i file2.tr0 < answers2.txt
```

where, `file1.tr0` and `file2.tr0` are PrimeSim HSPICE generated transient output files. The above creates `file1.tr0_PWL0` and `file2.tr0_PWL0`.

Examples of input files with answers:

```
// single PWL created :
1          // # of signals
v(nd)      // names of signals
vsig1      // name of PWL source
sig1       // + node of PWL
0          // - node of PWL
1          // choose 1 for from/to , 2 to define each point
0          // start time
600p       // end time
100        // # of points
// Answer file to create multiple PWL signals in one tr0_PWL0 file
// after first answer, an answer is needed for each // signal even if
they are the same
2
v(sig1) v(sig2)
vsig1 vsig2
n1 n2
0 0
1 1
0 0
100n 100n
100 100
```

Troubleshooting Issues

The following sections discuss these topics:

- [Resolving Inductor/Voltage Source Loop Errors](#)

Resolving Inductor/Voltage Source Loop Errors

PrimeSim HSPICE issues an inductor/voltage source loop error when:

- Two or more voltage sources connect to the same nodes.
- A voltage source with an inductor connects directly across its nodes.
- Two or more inductors connect in a loop and there is no limit to the current.

Avoid using these topologies.

However, if PrimeSim HSPICE reports this error, then follow these steps to correct the error:

1. Find out where the topology exists and correct it.
2. Combine multiple voltage sources into a single equivalent voltage source.
3. Limit the current by connecting a small series resistance (1n ohm or smaller) to the voltage source loop.

Using Verilog-A

This chapter has been moved to the PrimeSim Continuum Overview. For more information , see Using Verilog-A in the *PrimeSim™ Continuum Overview* guide.

Part 3: Analyses and Simulation

Describes the various analyses and simulation.

Part 3 presents the following chapters/topics:

- [Chapter 10, Initializing DC-Operating Point Analysis](#)
- [Chapter 11, AC Small-Signal and Noise Analysis](#)
- [Chapter 12, Transient Analysis](#)
- [Chapter 13, Spectrum Analysis](#)
- [Chapter 14, Pole-Zero Analysis](#)
- [Chapter 15, MOSFET Reliability Analysis \(MOSRA\)](#)
- [Chapter 16, Post-Layout Simulation: RC Network Reduction and Back-Annotation](#)
- [Chapter 18, Simulation of Stacked Devices](#)
- [Data-To-Design \(D2D\)](#)

The following analyses are described in the *PrimeSim HSPICE® User Guide: Signal Integrity Modeling and Analysis*.

- Linear Network Parameter Analysis
- Statistical Eye Analysis

10

Initializing DC-Operating Point Analysis

Describes DC initialization and operating point analysis.

PrimeSim HSPICE ships numerous examples for your use; see Listing of Demonstration Input Files in the *PrimeSim HSPICE User Guide: Demonstration Netlists* for paths to the demo files.

For descriptions of individual PrimeSim HSPICE commands referenced in this chapter, see the *PrimeSim™ Continuum Reference Manual: Commands and Control Options*. For discussion of use of the `.DC` command in a subckt block, see [Using Isomorphic Analyses in Subcircuit Blocks on page 89](#).

The following sections discuss these topics:

- [Simulation Flow—Initialization and Analysis](#)
- [DC Initialization and Operating Point Calculation](#)
- [.DC Statement—DC Sweeps](#)
- [Other DC Analysis Statements](#)
- [Accuracy and Convergence](#)
- [Reducing DC Errors](#)
- [Diagnosing Convergence Problems](#)

Simulation Flow—Initialization and Analysis

Before it performs `.OP`, `.DC` sweep, `.AC`, or `.TRAN` analyses, PrimeSim HSPICE first sets the DC operating point values for all nodes and sources. To do this, PrimeSim HSPICE does one of the following:

- Calculates all values
- Applies values specified in `.NODESET` and `.IC` statements
- Applies values stored in an initial conditions file.

The `.OPTION OFF` statement, and the `OFF` and `IC=val` element parameters, also control initialization.

Initialization is fundamental to simulation. PrimeSim HSPICE starts any analysis with known-nodal voltages (or initial estimates for unknown voltages) and some branch currents. It then iteratively finds the exact solution. Initial estimates that are close to the exact solution increase the likelihood of a convergent solution and a lower simulation time.

A transient analysis first calculates a DC operating point using the DC equivalent model of the circuit (unless you specify the `UIC` parameter in the `.TRAN` statement). PrimeSim HSPICE then uses the resulting DC operating point as an initial estimate to solve the next timepoint in the transient analysis.

The following describes the process:

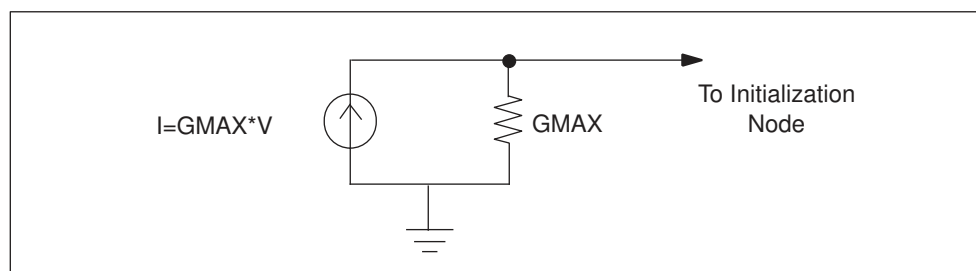
1. If you do not provide an initial guess or if you provide only partial information, PrimeSim HSPICE provides a default estimate for each node in the circuit.
2. PrimeSim HSPICE then uses this estimate to iteratively find the exact solution.

The `.NODESET` and `.IC` statements supply an initial guess for the exact DC solution of nodes within a circuit.

3. To set the seed value for the iterative dc algorithm for any circuit node to any value, use the `.NODESET` statement.
4. PrimeSim HSPICE then connects a voltage source equivalent, to each initialized node (a current source, with a `GMAX` parallel conductance, set with a `.OPTION` statement).
5. PrimeSim HSPICE next calculates a DC operating point, with the `.NODESET` voltage source equivalent connected.
6. PrimeSim HSPICE disconnects the equivalent voltage sources, which you set in the `.NODESET` statement, and recalculates the DC operating point.

This is the DC operating point solution.

Figure 32 Equivalent Voltage Source: `NODESET` and `.IC`



The `.IC` statement provides both an initial guess and a solution for selected nodes within the circuit. Nodes that you initialize with the `.IC` statement become part of the solution of the DC operating point.

Use `.IC` and `.NODESET` statements in a `.DC` analysis, in addition to `.TRAN` statements, unless you set `.OPTION DCIC=0`. You can also use the `OFF` option to initialize active devices. The `OFF` option works with `.IC` and `.NODESET` voltages as follows:

1. If the netlist includes any `.IC` or `.NODESET` statements, PrimeSim HSPICE sets node voltages, according to those statements.
2. If you set the `OFF` option, PrimeSim HSPICE sets values to zero for the terminal voltages of all active devices (BJTs, diodes, MOSFETs, JFETs, MESFETs) that you do not set in `.IC` or `.NODESET` statements, or by sources.
3. If element statements specify any IC parameters, PrimeSim HSPICE sets those initial conditions.
4. PrimeSim HSPICE uses the resulting voltage settings, as the initial guess at the operating point.

Use `OFF` to find an exact solution, during an operating point analysis, in a large circuit. The majority of device terminals are at zero volts for the operating point solution.

To initialize the terminal voltages to zero for selected active devices, set the `OFF` parameter in the element statements for those devices.

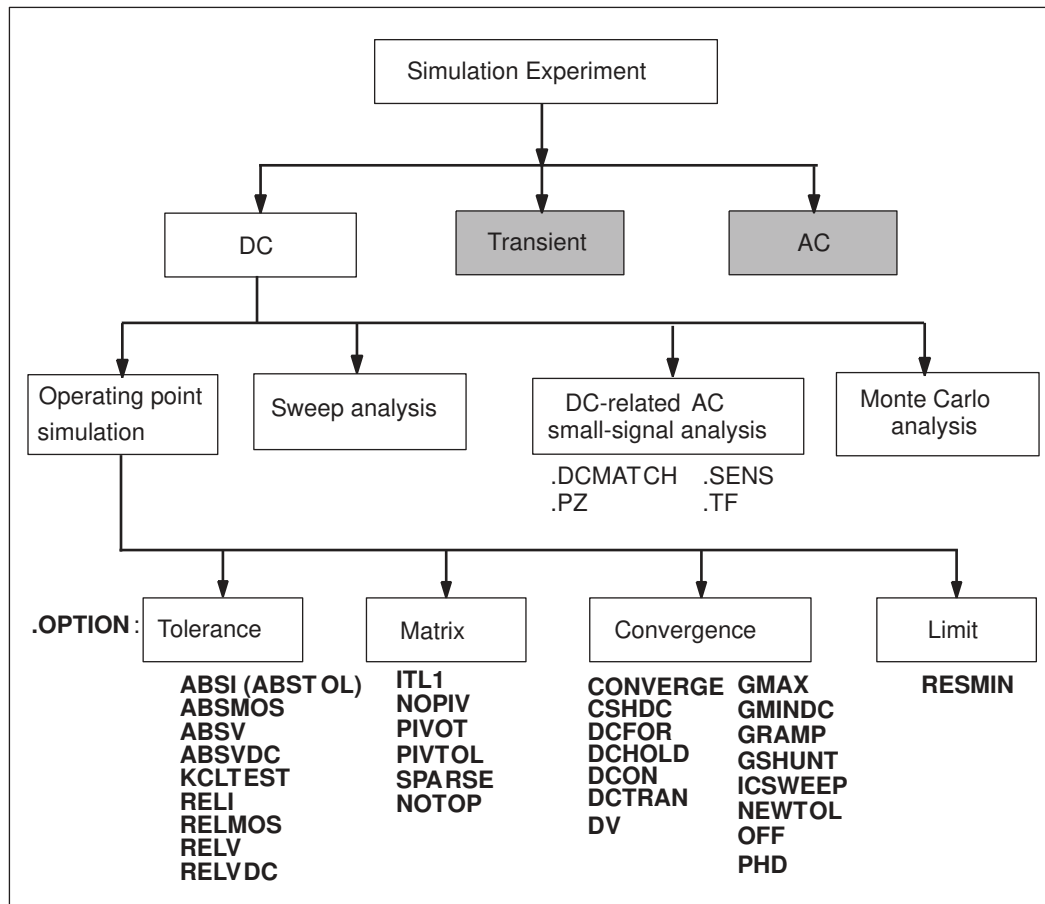
After PrimeSim HSPICE finds a DC operating point, use `.SAVE` to store operating-point node voltages in a `design.ic` file. Then use the `.LOAD` statement to restore operating-point values from the `*.ic` file for later analyses.

When you set initial conditions for Transient Analysis:

- If you include `UIC` in a `.TRAN` statement, PrimeSim HSPICE starts a transient analysis, using node voltages specified in an `.IC` statement.
- Use the `.OP` statement, to store an estimate of the DC operating point, during a transient analysis.
- An internal timestep too small error message indicates that the circuit failed to converge. The cause of the failure can be that PrimeSim HSPICE cannot use stated initial conditions to calculate the actual DC operating point.

Figure 33 shows the simulation flow for DC analysis in PrimeSim HSPICE.

Figure 33 DC Initialization and Operating Point Analysis Simulation Flow



DC Initialization and Operating Point Calculation

Use a `.OP` statement in PrimeSim HSPICE to:

- Calculate the DC operating point of a circuit
- Produce an operating point during a transient analysis

A simulation can only have one `.OP` statement.

The following sections discuss these topics:

- [.OP Statement — Operating Point](#)
- [Element Statement IC Parameter](#)

- [Initial Conditions and UIC Parameters](#)
- [.SAVE and .LOAD Statements \(PrimeSim HSPICE Only\)](#)

.OP Statement — Operating Point

When you include an `.OP` statement in an input file, PrimeSim HSPICE calculates the DC operating point of the circuit. You can also use the `.OP` statement to produce an operating point, during a transient analysis.

If an analysis requires calculating an operating point, you do not need to specify the `.OP` statement; PrimeSim HSPICE calculates an operating point. If you use a `.OP` statement, and if you include the UIC keyword in a `.TRAN` analysis statement, then simulation omits the time=0 operating point analysis, and issues a warning in the output listing.

Output

```
***** OPERATING POINT INFORMATION  TNOM=25.000  TEMP=25.000
***** OPERATING POINT STATUS IS ALL  SIMULATION TIME IS 0.
NODE      VOLTAGE  NODE      VOLTAGE  NODE      VOLTAGE
+ 0:2=0    0:3=437.3258M  0:4=455.1343M
+ 0:5=478.6763M  0:6=496.4858M  0:7=537.8452M
+ 0:8=555.6659M  0:10=5.0000  0:11=234.3306M

**** VOLTAGE SOURCES
SUBCKT
ELEMENT  0:VNCE  0:VN7  0:VPCE  0:VP7
VOLTS  0  5.000000  0  -5.000000
AMPS  -2.07407U  -405.41294P  2.07407U  405.41294P
POWER  0.  2.02706N  0.  2.02706N
TOTAL VOLTAGE SOURCE POWER DISSIPATION=4.0541 N WATTS
**** BIPOLAR JUNCTION TRANSISTORS
SUBCKT
ELEMENT  0:QN1  0:QN2  0:QN3  0:QN4
* Note: HSPICE advanced analog analyses does not
*       support qn(element)
* charge output.
MODEL  0:N1  0:N1  0:N1  0:N1
IB  999.99912N  2.00000U  5.00000U  10.00000U
IC  -987.65345N  -1.97530U  -4.93827U  -9.87654U
VBE  437.32588M  455.13437M  478.67632M  496.48580M
VCE  437.32588M  17.80849M  23.54195M  17.80948M
VBC  437.32588M  455.13437M  478.67632M  496.48580M
VS  0.  0.  0.  0.
POWER  5.39908N  875.09107N  2.27712U  4.78896U
BETAD -987.65432M  -987.65432M  -987.65432M  -987.65432M
GM  0.  0.  0.  0.
RPI  2.0810E+06  1.0405E+06  416.20796K  208.10396K
RX  250.00000M  250.00000M  250.00000M  250.00000M
RO  2.0810E+06  1.0405E+06  416.20796K  208.10396K
```

```
CPI 1.43092N 1.44033N 1.45279N 1.46225N
CMU 954.16927P 960.66843P 969.64689P 977.06866P
CCS 800.00000P 800.00000P 800.00000P 800.00000P
BETAAC 0. 0. 0. 0.
FT 0. 0. 0. 0.
```

Element Statement IC Parameter

Use the element statement parameter, `IC=<val>`, to set DC terminal voltages for selected active devices.

PrimeSim HSPICE uses the value, set in `IC=<val>`, as the DC operating point value.

Example

This example describes an H-element dependent-voltage source:

```
HXCC 13 20 VIN1 VIN2 IC=0.5, 1.3
```

The current, through VIN1, initializes to 0.5 mA. The current, through VIN2, initializes to 1.3 mA.

Initial Conditions and UIC Parameters

Use the `.IC` (or `.DCVOLT`), for the IC parameter on an element statement, and the `.NODESET` commands to set transient initial conditions in PrimeSim HSPICE. How it initializes depends on whether the `.TRAN` analysis statement includes the UIC parameter. If you do not specify the UIC parameter in the `.TRAN` statement, PrimeSim HSPICE computes the DC operating point solution before the transient analysis. The node voltages that you specify in the `.IC` statement determine the DC operating point.

PrimeSim HSPICE uses the node voltages that you specify in the `.NODESET` statement only in the first iteration to set an initial guess for the DC operating point analysis. Transient analysis releases the initialized nodes to calculate the second and later time points.

If you specify the UIC parameter in the `.TRAN` statement, PrimeSim HSPICE does not calculate the initial DC operating point, but directly enters transient analysis.

When you use `.TRAN` with UIC, PrimeSim HSPICE determines the `.TRAN` node values (at time zero) by searching for the first value found in this order: from `.IC` value, then `IC` parameter on an element statement, then `.NODESET` value; otherwise it uses a voltage of zero. Note that forcing a node value of the DC operating point may not satisfy KVL and KCL. In this event you likely see activity during the initial part of the simulation. This may happen if you use UIC and do not specify some node values, specify too many (conflicting) `.IC` values, or force node values and the topology changes. In this event, you likely see activity during the initial part of the simulation. Forcing a node voltage applies

a fixed equivalent voltage source during DC analysis and transient analysis removes the voltage sources to calculate the second and later time points.

Therefore, to correct DC convergence problems use `.NODESET` commands (without `.TRAN` with UIC) liberally (when you can provide a good guess) and use `.ICs` sparingly (when you know the exact node voltage).

.SAVE and .LOAD Statements (PrimeSim HSPICE Only)

PrimeSim HSPICE saves the operating point, unless you use the `.SAVE LEVEL=NONE` statement. PrimeSim HSPICE restores the saved operating-point file, only if the input file contains a `.LOAD` statement.

The `.SAVE` statement in PrimeSim HSPICE stores the operating point of a circuit, in a file that you specify. You can save the operating point data as either an `.IC` or a `.NODESET` statement. For quick DC convergence in subsequent simulations, use the `.LOAD` statement to input the contents of this file. PrimeSim HSPICE saves the operating point by default, even if the PrimeSim HSPICE input file does not contain a `.SAVE` statement. To not save the operating point, specify `.SAVE LEVEL=NONE`.

A parameter or temperature sweep saves only the first operating point.

If any node initialization commands, such as `.NODESET` and `.IC`, appear in the netlist after the `.LOAD` command, then they overwrite the `.LOAD` initialization. If you use this feature to set particular states for multistate circuits (such as flip-flops), you can still use the `.SAVE` command to speed up the DC convergence.

`.SAVE` and `.LOAD` work even on changed circuit topologies. Adding or deleting nodes results in a new circuit topology. PrimeSim HSPICE initializes the new nodes, as if you did not save an operating point. PrimeSim HSPICE ignores references to deleted nodes, but initializes coincidental nodes to the values that you saved from the previous run.

When you initialize nodes to voltages, PrimeSim HSPICE inserts Norton-equivalent circuits at each initialized node. The conductance value of a Norton-equivalent circuit is `GMAX=100`, which might be too large for some circuits.

If using `.SAVE` and `.LOAD` does not speed up simulation, or causes simulation problems, use `.OPTION GMAX=1e-12` to minimize the effect of Norton-equivalent circuits on matrix conductances.

PrimeSim HSPICE still uses the initialized node voltages to initialize devices. Do not use the `.LOAD` command for concatenated netlist files.

.DC Statement—DC Sweeps

You can use the `.DC` statement in DC analysis to:

- Sweep any parameter value.
- Sweep any source value.
- Sweep temperature range.
- Perform a DC Monte Carlo (random sweep) analysis.
- Perform a data-driven sweep.
- Perform a DC circuit optimization for a data-driven sweep.
- Perform a DC circuit optimization, using start and stop.
- Perform a DC model characterization.

The `.DC` statement format depends on the application that uses it. For syntax details, refer to the `.DC` command in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

The following sections discuss these topics:

- [Multi-dimensional DC Sweep](#)
- [Circuit and Model Temperature Simulation](#)

Multi-dimensional DC Sweep

To get the measure results of a multi-dimensional `.DC` sweep in a single measure file, you need to create a dummy inner sweep, and use the `.DATA` block to sweep the multi-dimensional data points in the outer sweep.

In the following example, the `.DATA` block sweeps `pvdd` for each temperature. The measure results of all the 6 data points is recorded in the same measure file.

```
*****2D sweep measure setup
v0 1 0 pvdd
r0 1 0 1k

.param dummy=1 $ define dummy outer sweep
.dc dummy 1 1 1 SWEEP DATA=vpoint $ vpoint contains 2D sweep params
.DATA vpoint
+ temp pvdd
+ 15 1
+ 15 2
+ 15 3
+ 40 1
```

```
+ 40 2
+ 40 3
.ENDDATA

* Measure current at each pvdd and temperature value
.measure dc cur_res find i(r0) when par('dummy')=1
.end
```

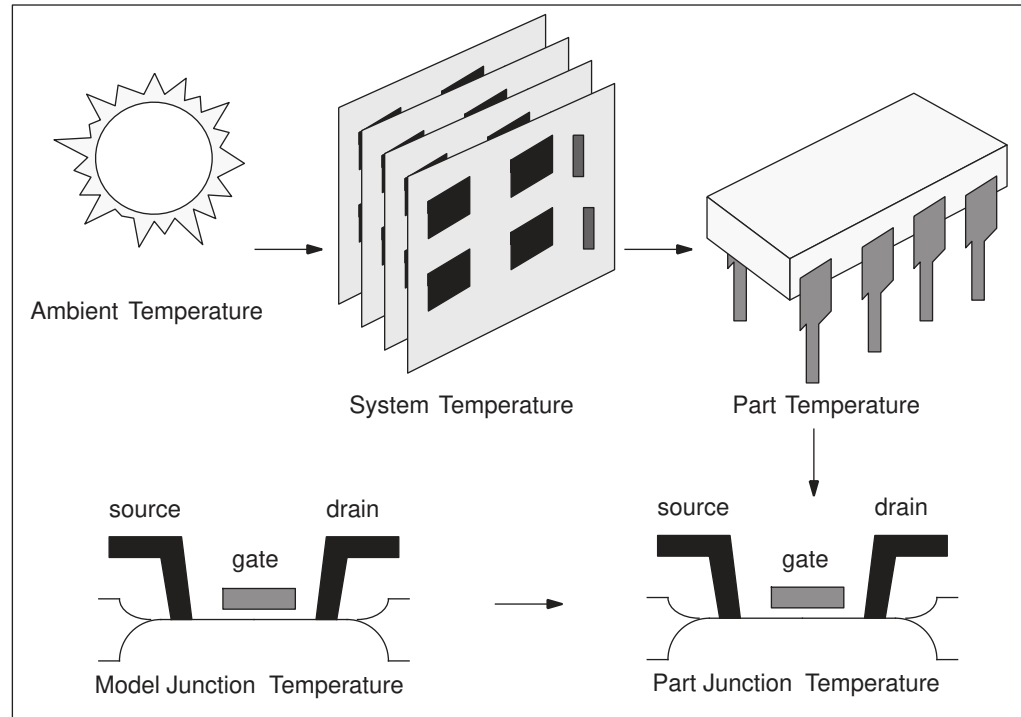
Circuit and Model Temperature Simulation

Temperature affects *all* electrical circuits. [Figure 34](#) shows the key temperature parameters associated with circuit simulation:

- Model reference temperature – You can model different models at different temperatures. Each model has a `TREF` (temperature reference) parameter.
- Element junction temperature – Each resistor, transistor, or other element generates heat so that an element is hotter than the ambient temperature.
- Part temperature – At the system level each part has its own temperature.
- System temperature – A collection of parts form a system, which has a local temperature.

- Ambient temperature – The ambient temperature is the air temperature of the system.

Figure 34 Part Junction Temperature Sets System Performance



PrimeSim HSPICE calculates temperatures as differences from the ambient temperature:

$$T_{ambient} + \Delta_{system} + \Delta_{part} + \Delta_{junction} = T_{junction}$$

$$I_{ds} = f(T_{junction}, T_{model})$$

Every element includes a DTEMP keyword, which defines the difference between junction and ambient temperature.

Example

The following example uses DTEMP in a MOSFET element statement:

```
M1 drain gate source bulk Model_name W=10u L=1u DTEMP=+20
```

Temperature Variation Analysis

Temperature variation analysis varies the circuit and component temperatures and compare the circuit responses. You can study the temperature-dependent effects of the circuit in detail.

You can specify three temperatures:

- Model reference temperature specified in a `.MODEL` statement. The temperature parameter is usually `TREF`, but can be `TEMP` or `TNOM` in some models. This parameter specifies the temperature, in °C, at which PrimeSim HSPICE measures and extracts the model parameters. Set the value of `TNOM` in an `.OPTION` statement. Its default value is 25°C.
- Circuit temperature that you specify using a `.TEMP` statement or the `TEMP` parameter. This is the temperature, in °C, at which PrimeSim HSPICE simulates all elements. To modify the temperature for a particular element, use the `DTEMP` parameter. The default circuit temperature is the value of `TNOM`.
- Individual element temperature, which is the circuit temperature, plus an optional amount that you specify in the `DTEMP` parameter.

To specify the temperature of a circuit in a simulation run, use either the `.TEMP` statement or the `TEMP` parameter in the `.DC`, `.AC`, or `.TRAN` statements. PrimeSim HSPICE compares the circuit simulation temperature that one of these statements sets against the reference temperature that the `TNOM` option sets. `TNOM` defaults to 25°C, unless you use the `SPICE` option, which defaults to 27°C. To calculate the derating of component values and model parameters, PrimeSim HSPICE uses the difference between the circuit simulation temperature and the `TNOM` reference temperature.

Elements and models within a circuit can operate at different temperatures. For example, a high-speed input/output buffer that switches at 50 MHz is much hotter than a low-drive NAND gate that switches at 1 MHz. To simulate this temperature difference, specify both an element temperature parameter (`DTEMP`), and a model reference parameter (`TREF`). If you specify `DTEMP` in an element statement, the element temperature for the simulation is:

```
element temperature=circuit temperature + DTEMP
```

Specify the `DTEMP` value in the element statement (resistor, capacitor, inductor, diode, BJT, JFET, or MOSFET statement) or in a subcircuit element. Assign a parameter to `DTEMP`; then use the `.DC` statement to sweep the parameter. The `DTEMP` value defaults to zero.

If you specify `TREF` in the model statement, the model reference temperature changes (`TREF` overrides `TNOM`). Derating the model parameters is based on the difference between circuit simulator temperature and `TREF` (instead of `TNOM`).

.TEMP Statement

To specify the temperature of a circuit for a simulation, use the `.TEMP` statement.

Other DC Analysis Statements

PrimeSim HSPICE also provides the following DC analysis statements. Each statement uses the DC-equivalent model of the circuit in its analysis. For `.PZ`, the equivalent circuit includes capacitors and inductors.

Statement	Description
<code>.DCMATCH</code>	A technique for computing the effects of local variations in device characteristics on the DC solution of a circuit.
<code>.PZ</code>	Performs pole/zero analysis.
<code>.SENS</code>	Obtains DC small-signal sensitivities of output variables for circuit parameters.
<code>.TF</code>	Calculates DC small-signal values for transfer functions (ratio of output variable, to input source).

PrimeSim HSPICE includes DC control options, and DC initialization statements, to model resistive parasitics and initialize nodes. These statements enhance convergence properties and accuracy of simulation. This section describes how to perform DC-related, small-signal analysis.

DC Initialization Control Options

Use control options to control DC convergence properties and simulation algorithms. Many of these options also affect transient analysis and AC analysis, because DC convergence is an integral part of transient convergence and AC analyses. Include the following options for both DC and transient convergence:

ABSTOL	DCFOR	DV	ICSWEEP	MAXAMP	PIVOT
CAPTAB	DCHOLD	GDCPATH	ITLPTRAN	NEWTOL	PIVTOL
CONVERGE	DCIC	GRAMP	ITL1	NOPIV	RESMIN
CSHDC	DCON	GSHDC	ITL2	OFF	SPARSE
DCCAP		GSHUNT	KCLTEST	PHD	SYMB

See `.OPTION PHD` for the PHD flow that can show performance improvement in simulations that require large DC OP convergence iterations.

For a description of transient analysis, see [Chapter 12, Transient Analysis](#).

For a description of AC analysis, see [Chapter 11, AC Small-Signal and Noise Analysis](#).

Accuracy and Convergence

Convergence is the ability to solve a set of circuit equations, within specified tolerances, and within a specified number of iterations. In numerical circuit simulation, a designer specifies a relative and absolute accuracy for the circuit solution. The simulator iteration algorithm then attempts to converge to a solution that is within these set tolerances. That is, if consecutive simulations achieve results within the specified accuracy tolerances, circuit simulation has converged. How quickly the simulator converges, is often a primary concern to a designer—especially for preliminary design trials. So designers willingly sacrifice some accuracy for simulations that converge quickly.

The following sections discuss these topics:

- [Accuracy Tolerances](#)
- [Autoconverge Process](#)

Accuracy Tolerances

PrimeSim HSPICE uses accuracy tolerances that you specify, to assure convergence. These tolerances determine when, and whether, to exit the convergence loop. For each iteration of the convergence loop PrimeSim HSPICE subtracts previously calculated values from the new solution and compares the result with the accuracy tolerances.

If the difference between two consecutive iterations is within the specified accuracy tolerances, the circuit simulation has converged.

$$| V_n^k - V_n^{k-1} | \leq \text{accuracy tolerance}$$

- V_n^k is the solution at the n timepoint for iteration k .
- V_n^{k-1} is the solution at the n timepoint for iteration $k - 1$.

As [Table 42](#) shows, PrimeSim HSPICE defaults to specific absolute and relative values. You can change these tolerances, so that simulation time is not excessive, and you do not compromise accuracy.

Table 42 *Absolute and Relative Accuracy Tolerances*

Type	.OPTION	Default
Nodal Voltage Tolerances	ABSVDC	50 μ v
	RELVDC	.001

Table 42 *Absolute and Relative Accuracy Tolerances
(Continued)*

Type	.OPTION	Default
Current Element Tolerances	ABSI	1 nA
	RELI	.01
	ABSMOS	1 uA
	RELMOS	.05

PrimeSim HSPICE compares nodal voltages and element currents to the values from the previous iteration.

- If the absolute value of the difference is less than `ABSVDC` or `ABSI`, then the node or element has converged.

`ABSV` and `ABSI` set the floor value, below which PrimeSim HSPICE ignores values. Values above the floor use `RELVDC` and `RELI` as relative tolerances. If the iteration-to-iteration absolute difference is less than these tolerances, then it is convergent.

Note:

`ABSMOS` and `RELMOS` are the tolerances for MOSFET drain currents.

Accuracy settings directly affect the number of iterations before convergence.

- If accuracy tolerances are tight, the circuit requires more time to converge.
- If the accuracy setting is too loose, the resulting solution can be inaccurate and unstable.

[Table 43](#) shows an example of the relationship between the `RELVDC` value, and the number of iterations.

Table 43 *RELV vs. Accuracy and Simulation Time for 2 Bit Adder*

RELVDC	Iteration	Delay (ns)	Period (ns)	Fall time (ns)
.001	540	31.746	14.336	1.2797
.005	434	31.202	14.366	1.2743
.01	426	31.202	14.366	1.2724
.02	413	31.202	14.365	1.3433
.05	386	31.203	14.365	1.3315
.1	365	31.203	14.363	1.3805

Table 43 RELV vs. Accuracy and Simulation Time for 2 Bit Adder (Continued)

RELVDC	Iteration	Delay (ns)	Period (ns)	Fall time (ns)
.2	354	31.203	14.363	1.3908
.3	354	31.203	14.363	1.3909
.4	341	31.202	14.363	1.3916
.4	344	31.202	14.362	1.3904

Autoconverge Process

If a circuit does not converge in the number of iterations that `ITL1` specifies, PrimeSim HSPICE initiates an autoconvergence process. This process manipulates `DCON`, `GRAMP`, and `GMINDC`, and even `CONVERGE` in some cases. [Figure 35 on page 268](#) shows the autoconverge process.

Note:

PrimeSim HSPICE uses autoconvergence in transient analysis, but it also uses autoconvergence in DC analysis if the Newton-Raphson (N-R) method fails.

In the process flow shown in [Figure 35 on page 268](#):

- Setting `.OPTION DCON=-1` disables Steps 2 and 3.
- Setting `.OPTION CONVERGE=-1` disables Steps 4, 5 and 6.
- Setting `.OPTION DCON=-1 CONVERGE=-1` disables Steps 2, 3, 4, 5 and 6.
- If you set the DV option to a value other than the default, Step 2 uses the value you set for DV, but Step 3 changes DV to 1e6.
- Setting `.OPTION GRAMP` has no effect on autoconverge. Autoconverge sets `GRAMP` independently.
- If you set `.OPTION GMINDC`, then `GMINDC` ramps to the value you set, instead of to 1e-12, in Steps 2 and 3.

Effects of GMINDC

The `GMINDC` option helps stabilize the circuit, during DC operating-point analysis. For MOSFETs, `GMINDC` helps stabilize the device in the vicinity of the threshold region. PrimeSim HSPICE inserts `GMINDC` between:

- Drain and bulk
- Source and bulk
- Drain and source

The drain-to-source `GMINDC` helps to:

- Linearize the transition from cutoff to weakly on
- Smooth-out model discontinuities
- Compensate for the effects of negative conductances.

The pn junction insertion of `GMINDC` in junction diodes linearizes the low conductance region. As a result, the device behaves like a resistor in the low-conductance region. This prevents the occurrence of zero conductance, and improves the convergence of the circuit. If a circuit does not converge, PrimeSim HSPICE automatically sets the `DCON` option. This option invokes `GMINDC` ramping, in steps 2 and 3 of [Figure 35 on page 268](#).

[Figure 36 on page 269](#) shows `GMINDC` for various elements.

Figure 35 Autoconvergence Process Flow Diagram

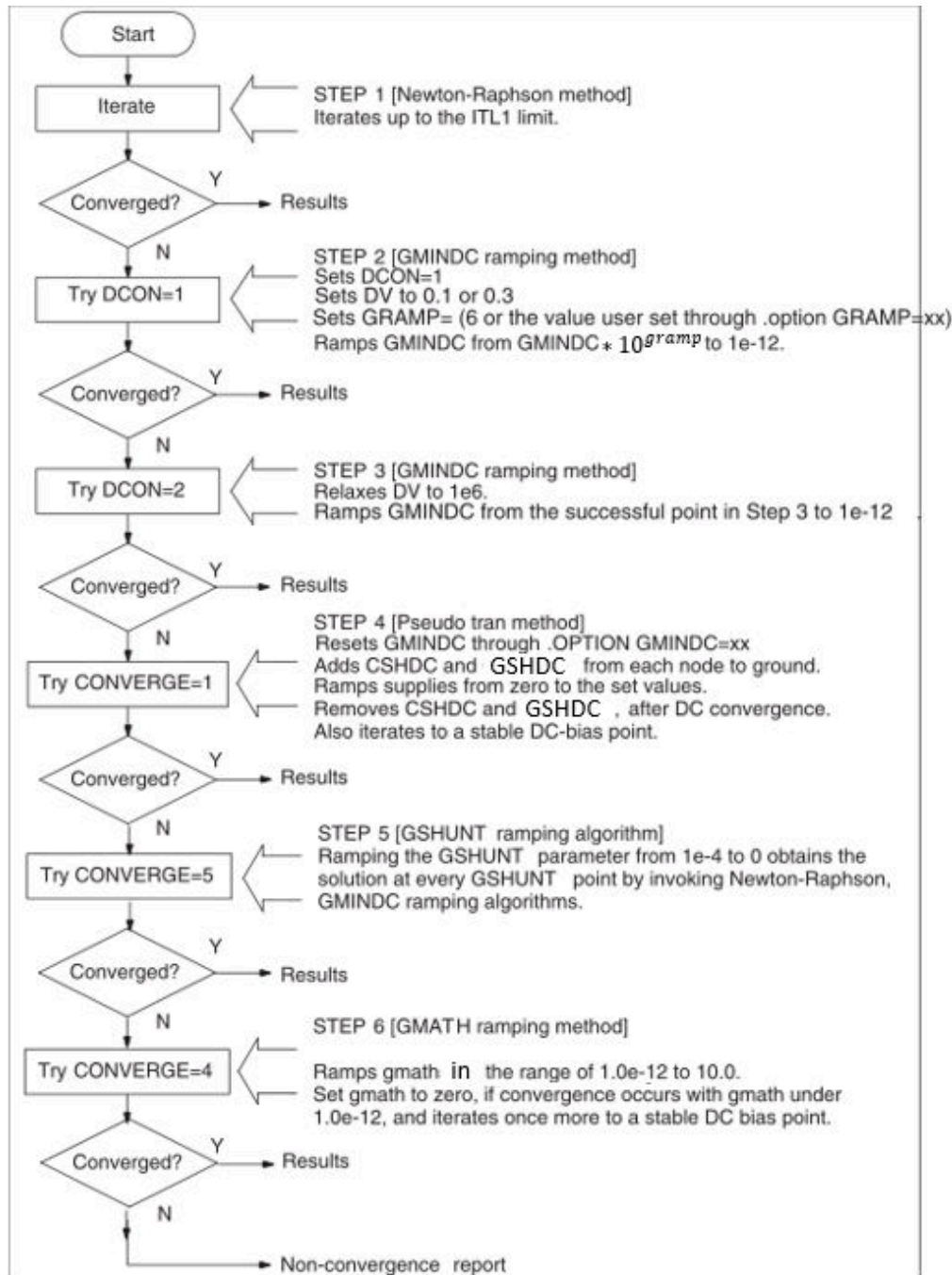
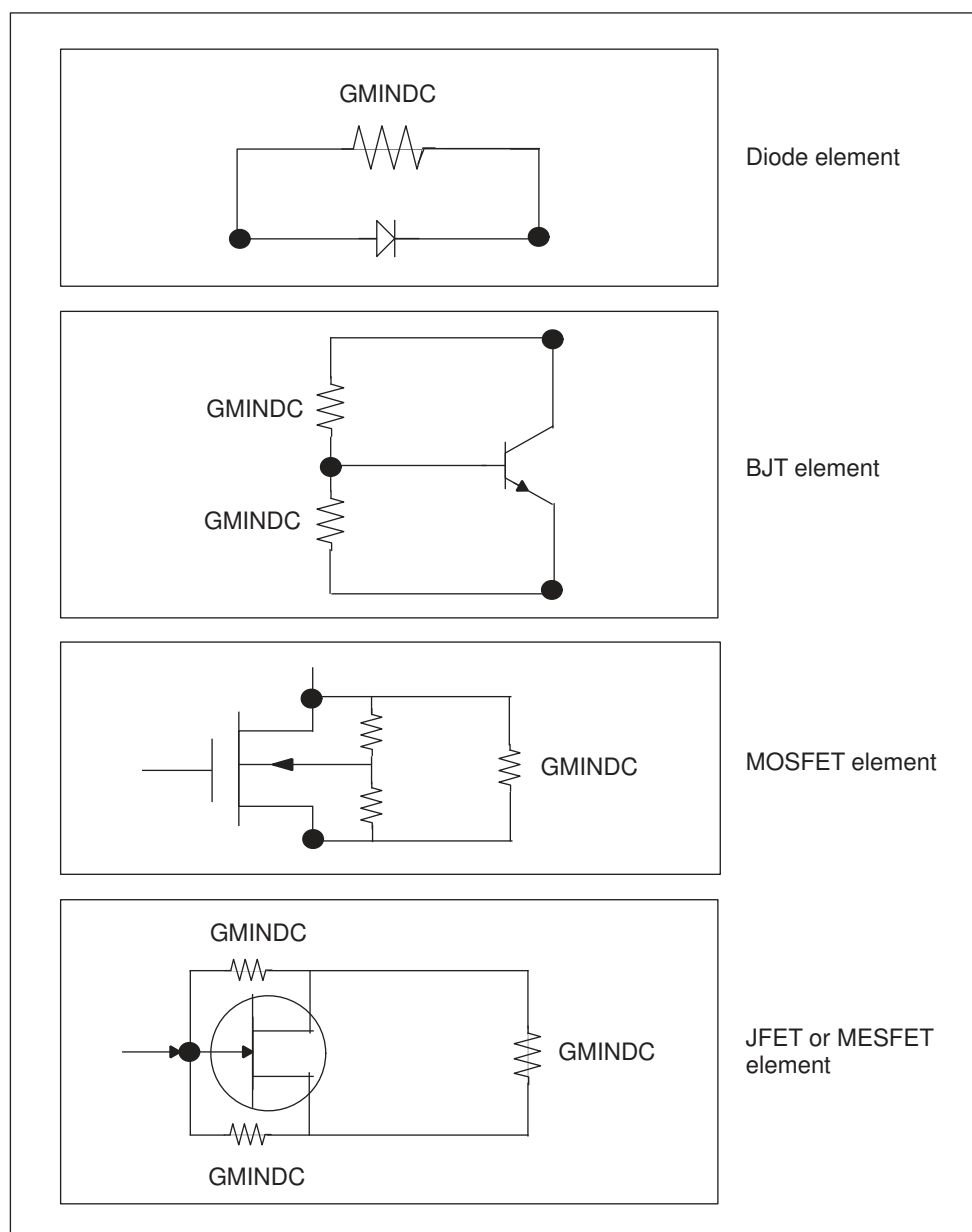


Figure 36 GMINDC Insertion



Reducing DC Errors

To reduce DC errors, perform the following steps:

1. To check topology, set `.OPTION NODE`, to list nodal cross-references.
 - Do all MOS p-channel substrates connect to either VCC or positive supplies?
 - Do all MOS n-channel substrates connect to either GND or negative supplies?
 - Do all vertical NPN substrates connect to either GND or negative supplies?
 - Do all lateral PNP substrates connect to negative supplies?
 - Do all latches have either an OFF transistor, a `.NODESET`, or an `.IC`, on one side?
2. Verify your `.MODEL` statements.
 - Check all model parameter units. Use model printouts to verify actual values and units because PrimeSim HSPICE multiplies some model parameters by scaling options.
 - Are subthreshold parameters of MOS models at reasonable values?
 - Are JS and JSW set in the MOS model for the DC portion of a diode model? A typical JS value is $1e-4A/M^2$.
 - Are CJ and CJSW set in MOS diode models?
 - Is weak-inversion NG and ND set in JFET/MESFET models?
 - Make sure that DIODE models have non-zero values for saturation current, junction capacitance, and series resistance.
 - Use MOS ACM=1, ACM=2, or ACM=3 source and drain diode calculations to automatically generate parasitics.
3. General remarks:
 - Ideal current sources require large values of `.OPTION GRAMP`, especially for BJT and MESFET circuits. Such circuits do not ramp up with the supply voltages, and can force reverse-bias conditions, leading to excessive nodal voltages.
 - Schmitt triggers are unpredictable for DC sweep analysis, and sometimes for operating points for the same reasons that oscillators and flip-flops are unpredictable. Use slow transient.
 - Large circuits tend to have more convergence problems because they have a higher probability of uncovering a modeling problem.
 - Circuits that converge individually, but fail when combined, usually have a modeling problem.

- Open-loop op-amps have high gain, which can lead to difficulties in converging. Start op-amps in unity-gain configuration, and open them up in transient analysis, by using a voltage-variable resistor, or a resistor with a large AC value (for AC analysis).

4. Check your options:

- Remove all convergence-related options, and try first with no special `.OPTION` settings.
- Check non-convergence diagnostic tables for non-convergent nodes. Look up non-convergent nodes in the circuit schematic. They are usually latches, Schmitt triggers, or oscillating nodes.
- For stubborn convergence failures, bypass DC all together, and use `.TRAN` with UIC set. Continue transient analysis until transients settle out, then specify the `.OP` time, to obtain an operating point during the transient analysis. To specify an AC analysis during the transient analysis, add an `.AC` statement to the `.OP` time statement.
- `SCALE` and `SCALM` scaling options have a significant effect on parameter values in both elements and models. Be careful with units.

The following sections discuss these topics:

- [Shorted Element Nodes](#)
- [Floating-Point Overflow](#)

Shorted Element Nodes

PrimeSim HSPICE disregards any capacitor, resistor, inductor, diode, BJT, or MOSFET if all of its leads connect together. The simulation ignores it in its component tally, and issues a warning:

```
**warning**  
all nodes of element x:<name> are connected together
```

Floating-Point Overflow

If MOS conductance is negative or zero, PrimeSim HSPICE might have difficulty converging. An indication of this type of problem is a floating-point overflow, during matrix solutions. PrimeSim HSPICE detects floating-point overflow, and invokes the Damped Pseudo Transient algorithm (`CONVERGE=1`), to try to achieve DC convergence without requiring you to intervene. If `GMINDC` is `1.0e-12` or less when a floating-point overflows, PrimeSim HSPICE sets it to `1.0e-11`.

Diagnosing Convergence Problems

Before simulation, PrimeSim HSPICE diagnoses potential convergence problems in the input circuit, and provides an early warning, to help you in debugging your circuit. If PrimeSim HSPICE detects a circuit condition that might cause convergence problems, it prints the following message into the output file:

```
Warning: Zero diagonal value detected at node ( ) in equation solver,
which might cause convergence problems. If your simulation fails, try
adding a large resistor between node ( ) and ground.
```

The following sections discuss these topics:

- [Non-Convergence Diagnostic Table](#)
- [Traceback of Non-Convergence Source](#)
- [Solutions for Non-Convergent Circuits](#)

Non-Convergence Diagnostic Table

If a circuit cannot converge, PrimeSim HSPICE automatically generates two printouts, called the diagnostic tables:

- Nodal voltage printout: Prints the names of all no-convergent node voltages, and the associated voltage error tolerances (tol).
- Element printout: Lists all non-convergent elements, and their associated element currents, element voltages, model parameters, and current error tolerances (tol).

The element-printout diagnostic tables associate the error tolerances: `tolds`, `tolbd`, and `tolbs`. These indicate how close the element currents (drain to source, bulk to drain, and bulk to source) are to a convergent solution.

For the `tolxx` variables, a value close to or below 1.0 is a convergent solution.

The equation for `tol` is:

$$tol = \frac{abs(In - (In - In))}{(RELMOS \cdot \max(abs(In), (abs(In - 1) + ABSMOS)))}$$

where `RELMOS`, `ABSMOS` are PrimeSim HSPICE control options.

`In` --> current value at `n`th iteration

`In-1` --> current value at `(n-1)`th iteration

This equation calculates the values for `tolds`, `tolbs`, and `tolbd` by substituting corresponding currents values to the equation.

To locate the branch current or nodal voltage that causes non-convergence, use the following steps:

1. Analyze the diagnostic tables. Look for unusually large values of branch currents, nodal voltages or tolerances.
2. After you locate the cause, use the `.NODESET` or `.IC` statements, to initialize the node or branch.

If circuit simulation does not converge, PrimeSim HSPICE automatically generates a non-convergence diagnostic table, indicating:

- The quantity of recorded voltage failures.
- The quantity of recorded branch element failures.

Any node in a circuit can create voltage failures, including *hidden* nodes (such as extra nodes that parasitic resistors create).

3. Check the element printout for the subcircuit, model, and element name for all parts of the circuit where node voltages or currents do not converge.

For example, [Table 44 on page 273](#) identifies the xinv21, xinv22, xinv23, and xinv24 inverters, as problem subcircuits in a ring oscillator. It also indicates that the p-channel transistors, in the xinv21, xinv22, xinv24 subcircuits, are nonconvergent elements. The n-channel transistor of xinv23 is also a nonconvergent element.

The table lists voltages and currents for the transistors, so you can check whether they have reasonable values. The `tol`s, `tolbd`, and `tolbs` error tolerances indicate how close the element currents (drain to source, bulk to drain, and bulk to source) are, to a convergent solution. For `tol` variables, a value close to or below 1.0 is a convergent solution. In [Table 44](#), the `tol` values that are around 100, indicate that the currents were far from convergence. The table displays the element current and voltage values (`id`, `ibs`, `ibd`, `vgs`, `vds`, and `vbs`). Examine whether these values are realistic, and determine the transistor regions of operation.

Table 44 Subcircuit Voltage, Current, and Tolerance

subckt element model	xinv21 21:mphc1 0:p1	xinv22 22:mphc1 0:p1	xinv23 23:mphc1 0:p1	xinv23 23:mnch1 0:n1	xinv24 24: mphc1 0:p1
id	27.5809f	140.5646u	1.8123p	1.7017m	5.5132u
ibs	205.9804f	3.1881f	31.2989f	0.	200.0000f
ibd	0.	0.	0.	-168.7011f	0.
vgs	4.9994	-4.9992	69.9223	4.9998	-67.8955
vds	4.9994	206.6633u	69.9225	-64.9225	2.0269

Table 44 Subcircuit Voltage, Current, and Tolerance (Continued)

subckt element model	xinv21 21:mphc1 0:p1	xinv22 22:mphc1 0:p1	xinv23 23:mphc1 0:p1	xinv23 23:mnch1 0:n1	xinv24 24: mphc1 0:p1
vbs	4.9994	206.6633u	69.9225	0.	2.0269
vth	-653.8030m	-745.5860m	-732.8632m	549.4114m	-656.5097m
tolds	114.8609	82.5624	155.9508	104.5004	5.3653
tolbd	0.	0.	0.	0.	0.
tolbs	3.534e-19	107.1528m	0.	0.	0.

Traceback of Non-Convergence Source

To locate a non-convergence source, trace the circuit path for error tolerance. For example, in an inverter chain, the last inverter can have a very high error tolerance. If this is the case, examine the error tolerance of the elements that drive the inverter. If the driving tolerance is high, the driving element could be the source of non-convergence. However, if the tolerance is low, check the driven element as the source of non-convergence.

Examine the voltages and current levels of a non-convergent MOSFET to discover the operating region of the MOSFET. This information can flow to the location of the discontinuity in the model—for example, subthreshold-to-linear, or linear-to-saturation.

When considering error tolerances, check the current and nodal voltage values. If you set these values extremely low, you divide an relatively large number by a very small number. This produces a large calculation result, which can cause the non-convergence errors. To solve this, increase the value of the absolute-accuracy options.

Use the diagnostic table, with the DC iteration limit (`ITL1` option), to find the sources of non-convergence. When you increase or decrease `ITL1`, PrimeSim HSPICE prints output for the problem nodes and elements for a new iteration—that is, the last iteration of the analysis that you set in `ITL1`.

Solutions for Non-Convergent Circuits

Non-convergent circuits generally result from:

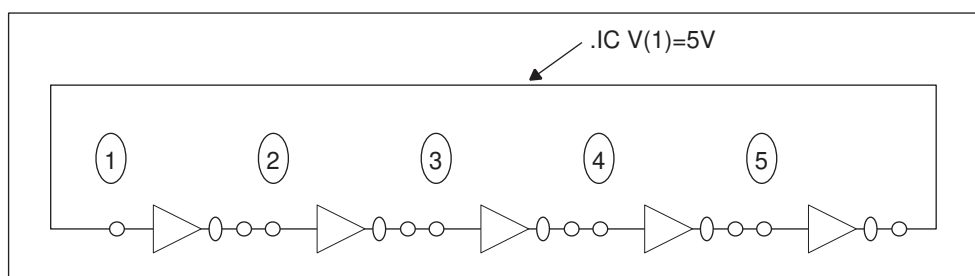
- [Poor Initial Conditions](#)
- [Inappropriate Model Parameters](#)
- [PN Junctions \(Diodes, MOSFETs, BJTs\)](#)

- [Troubleshooting DC Bias Point and DC Sweep Non-Convergence](#)
- [Convergence Failure: Too Many Current Probes in Netlist](#)
- [Troubleshooting: Nodes set to initial conditions with .IC may not always begin at those voltage values](#)

Poor Initial Conditions

Multi-stable circuits need state information, to guide the DC solution. You must initialize ring oscillators and flip-flops. These multi-stable circuits can either produce an intermediate forbidden state, or cause a DC convergence problem. To initialize a circuit, use the `.IC` statement, which forces a node to the requested voltage. Ring oscillators usually require you to set only one stage.

Figure 37 Ring Oscillator



The best way to set up the flip-flop is to use an `.IC` statement in the subcircuit definition.

Example

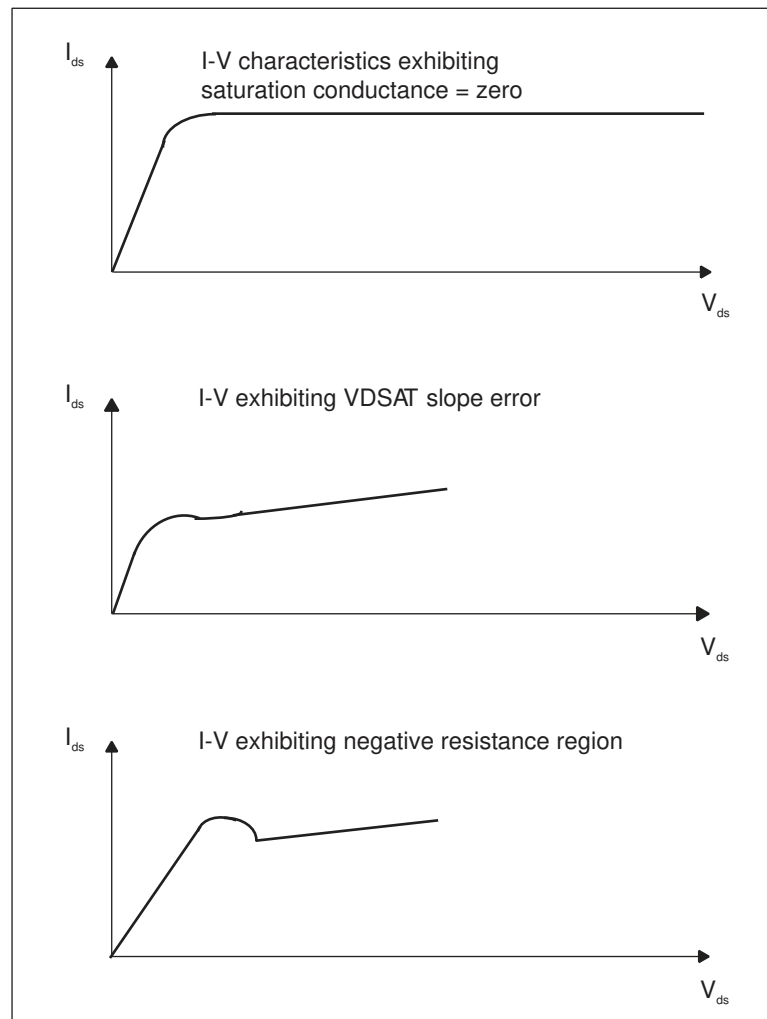
The following example sets the local `Qset` parameter to 0, and uses this value for the `.IC` statement, to initialize the Q latch output node. As a result, all latches have a default state of Q low. Set `Qset` to `vdd` to call a latch, which overrides this state.

```
.subckt latch in Q Q/ d Qset=0
.ic Q=Qset
...
.ends
Xff data_in[1] out[1] out[1]/ strobe LATCH Qset=vdd
```

Inappropriate Model Parameters

If you impose non-physical model parameters, you might create a discontinuous `IDS` or capacitance model. This can cause an *internal timestep too small* error, during the transient simulation. The `mosivcv.sp` demonstration file shows `IDS`, `VGS`, `GM`, `GDS`, `GMB`, and `CV` plots for MOS devices. A sweep near threshold from $V_{th}-0.5\text{ V}$ to $V_{th}+0.5\text{ V}$ (using a delta of 0.01 V), sometimes discloses a possible discontinuity in the curves.

Figure 38 Discontinuous I-V Characteristics



If simulation does not converge when you add a component or change a component value, then the model parameters are not appropriate or do not correspond to physical values they represent.

To locate the problem, follow these steps:

1. Check the input netlist file for non-convergent elements.
Devices with a *TOL* value greater than 1, are non-convergent.
2. Find the devices at the beginning of the combined-logic string of gates that seem to start the non-convergent string.

3. Check the operating point of these devices very closely, to see what region they operate in.

Model parameters associated with this region are probably inappropriate.

Circuit simulation uses single-transistor characterization, to simulate a large collection of devices. If a circuit fails to converge, the cause can be a single transistor, anywhere in the circuit.

PN Junctions (Diodes, MOSFETs, BJTs)

PN junctions found in diode, BJT, and MOSFET models, might exhibit non-convergent behavior, in both DC and transient analysis.

Example

PN junctions often have a high *off* resistance, and result in an ill-conditioned matrix. To overcome this, use `.OPTION GMINDC` and `.OPTION GMIN` to automatically parallel every PN junction in a design, with a conductance.

Non-convergence can occur if you overdrive the PN junction. This happens if you omit a current-limiting resistor, or if the resistor has a very small value. In transient analysis, protection diodes are often temporarily forward-biased (due to the inductive switching effect). This overdrives the diode, and can result in non-convergence, if you omit a current-limiting resistor.

Troubleshooting DC Bias Point and DC Sweep Non-Convergence

The following procedures trade runtime performance and loosen certain tolerance bounds to overcome DC non-convergence. PrimeSim HSPICE steps from one DC convergence algorithm to another other to find a solution. You can assist this process as follows (in the same order).

1. Remove or comment out all simulation control options from your PrimeSim HSPICE testbench/netlists to allow the default auto-convergence procedure to work.
2. For circuits with feedback or multiple bias states (FF and latches), it is important to provide PrimeSim HSPICE with an initial guess that is close to the final solution. Use the `.NODESET` command to set initial voltage guesses. In particular, focus on those nodes the output `*.lis` file lists as nonconvergent.

```
.nodeset v(in)=0 v(out)=3.3
```

3. Use the symbolic (`.OPTION SYMB`) operating point algorithm which finds initial guesses before calculating the operating point.

```
.option SYMB=1
```

Explanation

When you use the `SYMB` option, PrimeSim HSPICE assumes the circuit is digital and assigns a low/high state to all nodes as a reasonable initial voltage guess. This option improves DC convergence for oscillators, logic, and mixed-signal circuits.

4. Increase ITL1 from default value of 200 up to 500 in steps of 100. To further help DC sweep analysis, you may increase ITL2 in the same manor which increases the number of iterations PrimeSim HSPICE takes at each DC sweep point.

```
.option ITL1=300 ITL2=300
```

Explanation

If increasing ITL2 does not help DC sweep analysis, the problem likely lies in model discontinuities. In other words, if you set ITL2=400 and do not solve the convergence problem, it is unlikely that any further increase of the value of ITL2 can help convergence. As a workaround, try to increase and offset the sweep size in an attempt to miss model problems.

Original: `.dc vin 0v 3.3v .1v`

Increase: `.dc vin 0v 3.3v .2v`

Offset: `.dc vin .01v 3.31v .1v`

5. PrimeSim HSPICE tries various convergence algorithms to achieve DC convergence. Read the `.lis` file to see where PrimeSim HSPICE was when the job aborted. PrimeSim HSPICE first tries `DCON=1,2`, then `converge=1`. If this is not enough, try the other two converge choices along with larger `gmindc` values (`CONVERGE=3` is the source stepping method listed in “Inside SPICE”). However, do not set `gmindc` larger than `1e-9`.

```
.option converge=2 gmindc=1e-11  
.option converge=3 gmindc=1e-11
```

6. If certain active element nodes seem to be non-convergent, you may have PrimeSim HSPICE perform 2 DC bias point calculations. PrimeSim HSPICE performs the first calculation with the active elements turned off. Then, this solution becomes the first guess for the DC solution with the elements turned on. You may choose one or more elements to turn off, declared on the element line.

```
Diode n1 n2 diode_model off  
Qbjt n1 n2 n3 bjt_model off  
Mosfet n1 n2 n3 n4 mos_model off
```

Convergence Failure: Too Many Current Probes in Netlist

PrimeSim HSPICE accomplishes probing current by the insertion of a zero-volt source. When you explicitly add large numbers of current probes or by use of wildcard syntax such as `.probe i(*)`, the size of the solution matrix increases significantly which can

lead to convergence failures. These failures generate the message: `**diagnostic** rebuilding matrix with pivot option for special current probe process`

An error message follows: `**error** no convergence in operating point.`

Workarounds:

- Reduce the number of current probes by only probing specific nodes of interest, or adding a qualification to the wildcard.
- Create a saved operating point and tell PrimeSim HSPICE to use those initial conditions in the transient analysis.

The basic steps are:

- Run PrimeSim HSPICE without all the current probes, but include a `.OP` statement to create an initial conditions (`.ic0`) file.
- Include that file in your netlist. Example: `.include my_design.ic0`
- Add “uic” for Use Initial Conditions on the `.TRAN` line. Example: `.tran 1n 100n uic`

Then, it is possible that the design runs to completion even with the large number of current probes.

For more information on non-convergence, refer to [Autoconverge Process](#) and [Reducing DC Errors](#).

Troubleshooting: Nodes set to initial conditions with `.IC` may not always begin at those voltage values

The value set by `.IC` is not a voltage source, but a voltage source equivalent in the form of a current source with a parallel conductance. By default, that conductance is 100 mho (siemens) for an effective resistance of .01 ohms. The `GMAX` parameter sets that default conductance.

For example, if a Norton equivalent circuit created by that source is comparable with the conductance of other parts of the circuit, the DC node voltages deviate from those specified in `.IC` statement. Adjusting the `GMAX` parameter can reduce this effect. For instance, setting `GMAX` to 1e6 creates a very low internal resistance. Here is a simple case that illustrates the effect of changing `GMAX` from its default value.

```
*** initial condition test ***
.option ingold=1
v01 n1 0 1v
r01 n1 n2 1m
r02 n2 n3 1m
r03 n3 0 1m
*.option gmax=1e6
.ic v(n2)=.25v v(n3)=.25v
.print tran v(n2) v(n3)
```

```
.tran 1n 5n
.end
*****
```

Results with GMAX defaulting to 100 siemens...

time	voltage n2	voltage n3
0.	0.6386	0.3160
1.0000e-09	0.6667	0.3333
2.0000e-09	0.6667	0.3333
3.0000e-09	0.6667	0.3333
4.0000e-09	0.6667	0.3333
5.0000e-09	0.6667	0.3333

Results with GMAX set to 1e6 siemens...

time	voltage n2	voltage n3
0.	0.2507	0.2498
1.0000e-09	0.6667	0.3333
2.0000e-09	0.6667	0.3333
3.0000e-09	0.6667	0.3333
4.0000e-09	0.6667	0.3333
5.0000e-09	0.6667	0.3333

Note that the initial conditions are much closer to the desired values.

11

AC Small-Signal and Noise Analysis

Describes how to perform AC small signal and noise analyses in PrimeSim HSPICE.

This chapter covers AC small signal analysis, AC analysis of an RC network, noise analysis, and other AC analysis statements. For information on output variables, see [AC Analysis Output Variables on page 207](#).

PrimeSim HSPICE ships numerous examples for your use; see Listing of Demonstration Input Files in the *PrimeSim HSPICE User Guide: Demonstration Netlists* for paths to the demo files.

For descriptions of individual PrimeSim HSPICE commands referenced in this chapter, see PrimeSim HSPICE Netlist Commands in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

For discussion of use of the `.AC` command in subcircuit blocks, see [Using Isomorphic Analyses in Subcircuit Blocks on page 89](#) in this manual.

For discussion of Transient Noise Analysis and Simulation of Random Noise, see the *PrimeSim HSPICE User Guide: Advanced Analog Simulation and Analysis*.

The following sections discuss these topics:

- [Using the .AC Statement](#)
- [AC Small Signal Analysis](#)
- [Using .NOISE for Small-Signal Noise Analysis](#)
- [Using .AC/.NOISE Analyses with .TRAN](#)
- [Measuring Total Noise](#)
- [Using .LSTB for Loop Stability Analysis](#)
- [Other AC Analysis Statements - .DISTO, .NET, and .SAMPLE](#)

Using the .AC Statement

You can use the .AC statement for the following applications:

- Single/double sweeps
- Sweeps using parameters
- .AC analysis optimization
- Random/Monte Carlo analyses

For .AC command syntax, see the .AC command in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

The following sections discuss these topics:

- [.AC Control Options](#)
- [.AC Command Examples](#)

.AC Control Options

You can use the following .AC control options when performing an AC analysis:

MAXAMP	UNWRAP
--------	--------

For syntax descriptions for these options, see PrimeSim HSPICE Netlist Simulation Control Options in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

.AC Command Examples

Example 1

```
.AC DEC 10 1K 100MEG
```

This example performs a frequency sweep by 10 points per decade from 1kHz to 100MHz.

Example 2

```
.AC LIN 100 1 100HZ
```

This example runs a 100-point frequency sweep from 1- to 100-Hz.

Example 3

```
.AC DEC 10 1 10K SWEEP cload LIN 20 1pf 10pf
```

This example performs an AC analysis for each value of `cload`. This results from a linear sweep of `cload` between 1- and 10-pF (20 points), sweeping the frequency by 10 points per decade from 1- to 10-kHz.

Example 4

```
.AC DEC 10 1 10K SWEEP rx POI 2 5k 15k
```

This example performs an AC analysis for each value of `rx`, 5k and 15k, sweeping the frequency by 10 points per decade from 1- to 10-kHz.

Example 5

```
.AC DEC 10 1 10K SWEEP DATA=datanm
```

This example uses the `.DATA` statement to perform a series of AC analyses, modifying more than one parameter. The `datanm` file contains the parameters.

Example 6

```
.AC DEC 10 1 10K SWEEP MONTE=30
```

This example illustrates a frequency sweep and a Monte Carlo analysis with 30 trials.

Example 7

```
AC DEC 10 1 10K SWEEP MONTE=10 firstrun=15
```

This example illustrates a frequency sweep and a Monte Carlo analysis from the 15th to the 24th trials.

Example 8

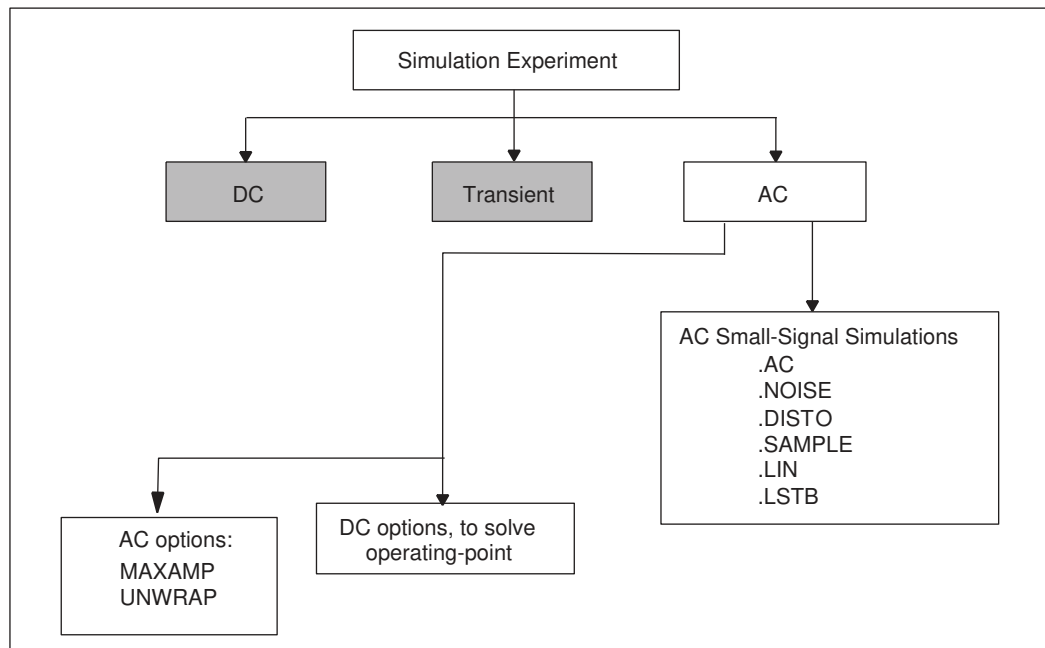
```
.AC DEC 10 1 10K SWEEP MONTE=list(10 20:30 35:40 50)
```

This example illustrates a frequency sweep and a Monte Carlo analysis at 10th trial and then from the 20th to 30th trial, followed by the 35th to 40th trial and finally at 50th trial.

AC Small Signal Analysis

AC small signal analysis in PrimeSim HSPICE computes AC output variables as a function of frequency (see [Figure 39 on page 284](#)). PrimeSim HSPICE first solves for the DC operating point conditions. It then uses these conditions to develop linear, small-signal models for all non-linear devices in the circuit.

Figure 39 AC Small Signal Analysis Flow



In PrimeSim HSPICE, the output of AC Analysis includes voltages and currents.

PrimeSim HSPICE converts capacitor and inductor values to their corresponding admittances:

$$y_C = j\omega C \quad \text{for capacitors}$$

$$y_L = \frac{1}{j\omega L} \quad \text{for inductors}$$

Resistors can have different DC and AC values. If you specify `AC=<value>` in a resistor statement, PrimeSim HSPICE uses the DC value of resistance to calculate the operating point, but uses the AC resistance value in the AC analysis. When you analyze operational amplifiers, PrimeSim HSPICE uses a low value for the feedback resistance to compute the operating point for the unity gain configuration. You can then use a very large value for the AC resistance in AC analysis of the open loop configuration.

AC analysis of bipolar transistors is based on the small-signal equivalent circuit, as described in the *PrimeSim HSPICE Elements and Device Models Manual*. MOSFET AC-equivalent circuit models are described in the *PrimeSim HSPICE Elements and Device Models Manual*.

The AC analysis statement can sweep values for:

- Frequency.
- Element.
- Temperature.
- Model parameter.
- Randomized (Monte Carlo) distribution.
- Optimization and AC analysis.

Additionally, as part of the small-signal analysis tools, PrimeSim HSPICE provides:

- Noise analysis.
- Distortion analysis.
- Network analysis.
- Sampling noise.

You can use the `.AC` statement in several different formats, depending on the application. You can also use the `.AC` statement to perform data-driven analysis in PrimeSim HSPICE.

Using .NOISE for Small-Signal Noise Analysis

A circuit noise analysis can be performed associated with a small-signal `.AC` analysis. The `.NOISE` command will activate a noise analysis that calculates the output noise generated based on the contributions from all noise sources within the circuit. This noise may be from passive elements, such as thermal (Johnson) noise in resistors, or from sources such as shot, channel, and flicker noise present within transistors. Most transistors will have several noise sources. For descriptions of noise models for each device type, see the *PrimeSim™ Continuum Reference Manual: Device Models*. In most cases, the individual noise sources in PrimeSim HSPICE lack statistical correlation, and this allows their contributions to output noise to be computed independently. The total output noise voltage is the RMS sum of the individual noise contributions:

Equation 1

$$onoise = \sqrt{\sum_{k=0}^N |Z_k|^2 |i_{nk}|^2}$$

Where,

onoise is the output noise spectral density ($V / \sqrt{Hz}$) at the AC analysis frequency.

$\overline{i_{nk}^2}$ is the mean-squared noise spectral density (A^2 / Hz) for each noise current source due to thermal, shot, flicker, or other noise.

Z_k is the equivalent transimpedance between each noise current source and the output.

N is the number of noise sources associated with all circuit elements.

This analysis will be performed for every frequency specified with the .AC command. The output for noise analysis is specified in the .NOISE syntax:

Basic Syntax

```
.NOISE outvar1 [outvar2 outvar3...] [src] [interval] [portnoise[=1|0]]
```

For the complete .NOISE syntax, see .NOISE in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

The output noise (onoise) voltage or current is computed at the out node specified by outvar. If output noise voltage is needed, outvar has the form v(out[, ref]). If the (optional) ref node is given, the output is taken as the differential output noise voltage v(out, ref). If the output noise current is needed, outvar has the form i(device).

There must be at least one outvar to specify the output port. There can be arbitrary multiple outvars for the .noise command.

The independent input source (src) is optional. It is used to specify the input port. When the input source is specified, PrimeSim HSPICE allows the calculation of the equivalent input noise given by

Equation 2

$$inoise = (onoise) / |G|$$

Where, inoise is the equivalent input noise spectral density at the input source, G is the gain between the input source (src) and the output.

From the PrimeSim HSPICE 2017.03 version, the input source impedance noise contribution is also added to output noise spectral density (onoise), by default, with the temperature reference at 290K, according to IEEE standard. Before this version, the source impedance noise contribution was not included in the output noise spectral density.

You can use the option portnoise=0 to keep backward compatibility.

When the input source is a port element (P), or a power voltage source (Vsrc with power=1|2), noise figure is calculated in the following manner:

When the input source is a port element, the source impedance is the Z_o value of P. When the input source is a power voltage source, the source impedance is 50 ohms. The noise figure value is listed in the `.lis` file.

If the input source is not specified, the equivalent input noise spectral density at the input source, gain, and noise figure are not calculated.

The `.NOISE` analysis can also generate a summary on how each noise generator within the circuit contributes to the output noise at each output port. Specify an integer value for `interval` to include a device noise summary for every `interval` frequency points in the PrimeSim HSPICE output listing. No summary will be included unless an `interval` is specified.

The `.NOISE` analysis will also compute the total integrated noise over the AC frequency range, which is also included in the output listing. The output summary will include values for the device noise sources given in [Table 45 on page 288](#) — [Table 47 on page 289](#).

To request `.NOISE` analysis results (magnitude and decibel) with `.print/.probe` use:

```
.probe noise onoise onoise(m) onoise(db) onoise('outvar')  
.probe noise inoise inoise(m) inoise(db) inoise('outvar')
```

When there are more than one output port, and no `outvar` name is specified to inoise/onoise in `.probe/.print/.meas` command, PrimeSim HSPICE will output the inoise/onoise corresponding to the first output port.

The results will be included in the `*.ac0` file. Output noise voltage or current units are V/sqrt(Hz) and A/sqrt(Hz) respectively. Device-level noise source contributions will also be included in the `*.ac0` file (unless you have set `.option probe=1`).

The naming convention and units for device level noise contributions are shown in the tables in the subsequent sections. When there are more than one output port, the noise source name in the `*.ac0` file will have the output port name `outvar` attached to specify to which output port the noise source is contributing.

To ensure that device noise models will be included in the analysis, verify that noise parameters are being set in your transistor models. Include values for AF and KF, for example, if you wish to activate flicker noise models for your devices.

See also, Using Noise Analysis Results as Input Noise Sources in the *PrimeSim HSPICE User Guide: Advanced Analog Simulation and Analysis*.

For a complete description of the `.NOISE` command syntax and examples, see the `.NOISE` command in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*. Note that the `.NOISE` analysis requires an `.AC` statement, and that if more than one `.NOISE` statement is included, PrimeSim HSPICE will run only the last statement.

- [.NOISE Measurements for MOSFETs](#)
- [.NOISE Measurements for BJTs](#)
- [.NOISE Measurements for Diodes](#)

.NOISE Measurements for MOSFETs

Table 45 .NOISE Measurements Available for MOSFETs

.ac	.lis	Unit	Description
nd	rd	$\frac{V^2}{Hz}$	Output thermal noise due to drain resistor
ns	rs	$\frac{V^2}{Hz}$	Output thermal noise due to source resistor
ni	id	$\frac{V^2}{Hz}$	Output channel thermal noise
nf	fn	$\frac{v^2}{hz}$	Output flicker noise
ntg	total	$\frac{V^2}{Hz}$	Total output noise: TOT=RD + RS + ID + FN

.NOISE Measurements for BJTs

Table 46 .NOISE Measurements Available for BJTs

.ac	.lis	Unit	Description
rb	rb	$\frac{V^2}{Hz}$	Output thermal noise due to base resistor
rc	rc	$\frac{V^2}{Hz}$	Output thermal noise due to collector resistor
re	re	$\frac{V^2}{Hz}$	Output thermal noise due to emitter resistor
nb	ib	$\frac{V^2}{Hz}$	Output noise due to base shot noise source
nc	ic	$\frac{V^2}{Hz}$	Output thermal noise due to collector shot noise source
nf	fn	$\frac{V^2}{Hz}$	Output noise due to flicker noise source

Table 46 .NOISE Measurements Available for BJTs (Continued)

.ac	.lis	Unit	Description
nt	total	$\frac{V^2}{Hz}$	Total output noise: TOT=RB + RC + RE + IB + IC + FN

.NOISE Measurements for Diodes

Table 47 .NOISE Measurements Available for Diodes

.ac	.lis	Unit	Description
nr	rs	$\frac{V^2}{Hz}$	Output noise due to diode series resistance
ni	id	$\frac{V^2}{Hz}$	Output noise due to shot noise
nf	fn	$\frac{V^2}{Hz}$	Output noise due to flicker noise
nt	total	$\frac{V^2}{Hz}$	Total output noise: TOT=RS + ID + FN

Using .AC/.NOISE Analyses with .TRAN

In some situations, a .TRAN analysis may be needed to establish the operating point used for .AC or .NOISE analysis. To do this, use the combination of commands as shown in the example below:

```
.TRAN 1n 5u $ Transient analysis
.OP 1u 2 u 3u $ Request operating point analysis
.AC DEC 100 1 20e9 $ AC analysis
.NOISE V(out) V1 $ NOISE analysis
```

PrimeSim HSPICE performs separate .AC analyses for all time values specified as well as one .AC run at time zero. This happens during the .TRAN analysis as it does the .OP and .AC evaluations, and results in separate *.AC0 files with unique labels for each time value specified. An *.ac0@tranop.grp file is also created when you run .AC/.NOISE with .TRAN analysis.

In addition, the .OP used at the time values specified is fully dynamic, meaning it uses all sources and non-linearities involved at that time value during the .TRAN. The charges and currents of the .TRAN are used and preserved for the .AC. It does this by using the derivatives ($C=dQ/dv$, $G=dl/dv$) at that point in the .TRAN for computing the .AC small-signal analysis.

The `.AC` and `.NOISE` analysis is then performed at this operating point.

For additional information, see `.OP` analysis in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

Measuring Total Noise

You can measure total noise using the `.MEASURE` command with `ac`. After performing a small signal noise analysis, PrimeSim HSPICE prints the total output noise voltage to the `*.lis` file. For example:

```
**** total output noise voltage    =    51.8809m    volts
```

You can get the total noise output as reported in the `*.lis` or the `*.noise#` file by using the following two `.MEASURE` commands:

```
.meas ac int_noise_sq integ par('onoise(m)*onoise(m)')  
+ from='fstart' to='fstop'  
  
.meas ac total_noise_v param='sqrt(int_noise_sq)'
```

Using .LSTB for Loop Stability Analysis

Stability analysis can be applied on feedback circuits to analyze loop gain and phase characteristics in the frequency domain. Examples of circuits with feedback are amplifiers, bandgaps, oscillators, and voltage regulators. The `.LSTB` command aids in studying the analog circuit stability margin both for oscillators and the circuits which are not supposed to be oscillating. Stability analysis is performed without breaking the feedback loop of the circuit on AC analysis while maintaining the DC operating point and considering AC loading.

A 0V DC voltage source is required to place in series in the loop of interest. Two voltage sources are required for differential or common-mode loop analysis. The `.LSTB` command measures the loop gain by successive injection (Middlebrook Technique, see <http://authors.library.caltech.edu/4537/1/MIDieeemm06.pdf>).

A zero voltage source is placed in series in the loop: one pin of the voltage loop must be connected to the loop input, the other pin to the loop output. The orientation of inserted voltage sources in differential/common-mode testing is significant. It is required that the positive terminal of both voltage sources go to the input of amplifier or go to the output of amplifier.

The `.LSTB` command can be used for these modes: single-ended, differential, and common. This analysis supports `.PRINT` / `.PROBE` / `.MEASURE` statements and can be used with `.ALTER` to generate multiple loop analyses. The functionality is available on all platforms.

For command syntax and additional examples, refer to .LSTB and .MEASURE LSTB in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

Note:

When measuring loop gain for the circuit by using AC analysis, if you open the feedback loop and use AC analysis, you run the risk of changing the DC operating points on either side of the loop and changing the impedances on both sides of the loop from their closed loop impedance. This will give you errors in your results.

However, there is a two-step AC analysis technique that uses ideal voltage and current sources to measure the voltage and current gains. After you get the voltage and current gains, you need to post-process the results to get the final loop gain. This method works without introducing errors into the results.

The following sections discuss these topics:

- [Output Formats for Loop Stability \(.LSTB\) Analysis](#)
- [Loop Stability Analysis Usage](#)
- [Multiple Loop Stability Analysis](#)
- [Single-Ended Mode Example: Ideal Inverter](#)
- [Differential Mode Example: Bandgap](#)
- [Controlling LSTB Warning Messages](#)

Output Formats for Loop Stability (.LSTB) Analysis

The outputs for loop stability analysis are as follows:

- The gain margin (GM), phase margin (PM), unity gain frequency (FU) and gain at minimum frequency (ADC) are reported in the *.lis file.
- The Loop Gain is reported to the *.cx# file, which is always produced for .LSTB analysis. The *.cx# file is a general file for all the complex outputs. It contains the data for waveforms as complex vectors.
- If you specify `.probe ac lstb(db[:lstbname]) lstb(m[:lstbname])
lstb(r[:lstbname]) lstb(i[:lstbname]) lstb(p[:lstbname])`, the specific format of loop gain goes to the *.ac# file for viewing.
- If an *.ac# file is produced with `.probe ac lstb[(lstbname)]`, then both *.ac# and *.cx# file could be used to view magnitude, phase, real, and imaginary versus frequency as complex vectors.

Loop Stability Analysis Usage

```
.LSTB [lstbname] mode=[single|diff|comm]  
+ vsource=[vlstb|vlstbp, vlstbn]
```

Examples

Single-mode loop analysis on loop indicated by *vx* voltage source:

```
.LSTB mode=single vsource=vx
```

Differential-mode loop analysis on loops indicated by *vp* and *vn* voltage sources:

```
.LSTB mode=diff vsource=vp,vn
```

Common-mode loop analysis on loops indicated by *vp* and *vn* voltage sources:

```
.LSTB mode=comm vsource=vp,vn
```

Multiple Loop Stability Analysis

You can perform multiple loop stability analyses using `.alter` blocks. This is useful in cases where the circuit has multiple feedback loops. The top-level netlist should contain the `.AC` analysis command and the first LSTB analysis command. The `.alter` blocks to define different loop conditions can contain different LSTB analysis commands and any altered parameters. You should not redefine the `.AC` analysis command in the `.alter` blocks because this will cause PrimeSim HSPICE to produce unexpected results.

An example:

```
* top level LSTB analysis  
.include 'netlist.cir'  
.ac DEC 10 1e3 1e9 $ AC analysis for all loops  
.lstb mode=single vsource=vx1  
.probe ac lstb  
* alter 1  
.alter loop1  
.lstb mode=single vsource=vx2  
* alter 2  
.alter loop2  
.lstb mode=single vsource=vx3  
.end
```

You can also perform multiple LSTB analysis by specifying the LSTB analysis with names.

An example:

```
* top level LSTB analysis  
.include 'netlist.cir'  
.ac DEC 10 1e3 1e9 $ AC analysis for all loops  
.lstb name1 mode=single vsource=vx1
```

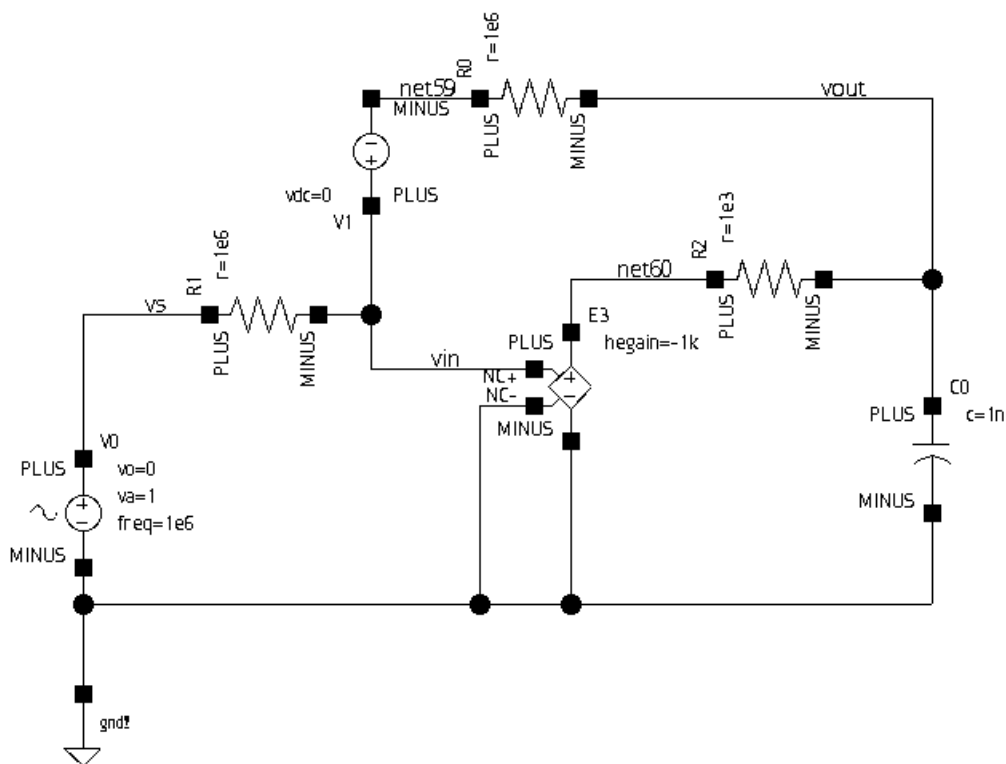


```
.lstb name2 mode=single vsource=vx2
.lstb name3 mode=single vsource=vx3
.probe ac lstb(name1) lstb(name2) lstb(name3)
.end
```

Single-Ended Mode Example: Ideal Inverter

The following is an example available in the demo directory that ships with PrimeSim HSPICE showing use of a single-ended mode input netlist. (See `$installdir/demo/hspice/lstb/single.sp.`)

Figure 40 Ideal inverter amplifier with single pole



This example netlist simulates single mode loop stability for a single-pole ideal inverter amplifier.

```
.GLOBAL gnd!

v1 vin net59 dc=0
v0 vs gnd! dc=0 ac=1 sin ( 0 1 1e6 0 0 0 )
e3 net60 gnd! vcvs vin gnd! -1000 max=1 min=-1 abs=0
r0 net59 vout r=1e6
r1 vs vin r=1e6
```

```
r2 net60 vout r=1e3
c0 vout gnd! c='1n'

.ac DEC '100' '100' '100e9'
.lstb mode=single vsource=v1
.option post
.probe ac lstb(m) lstb(db) lstb(p) lstb(r) lstb(i)
.end
```

The sequence of commands, controls, and parameters is as follows:

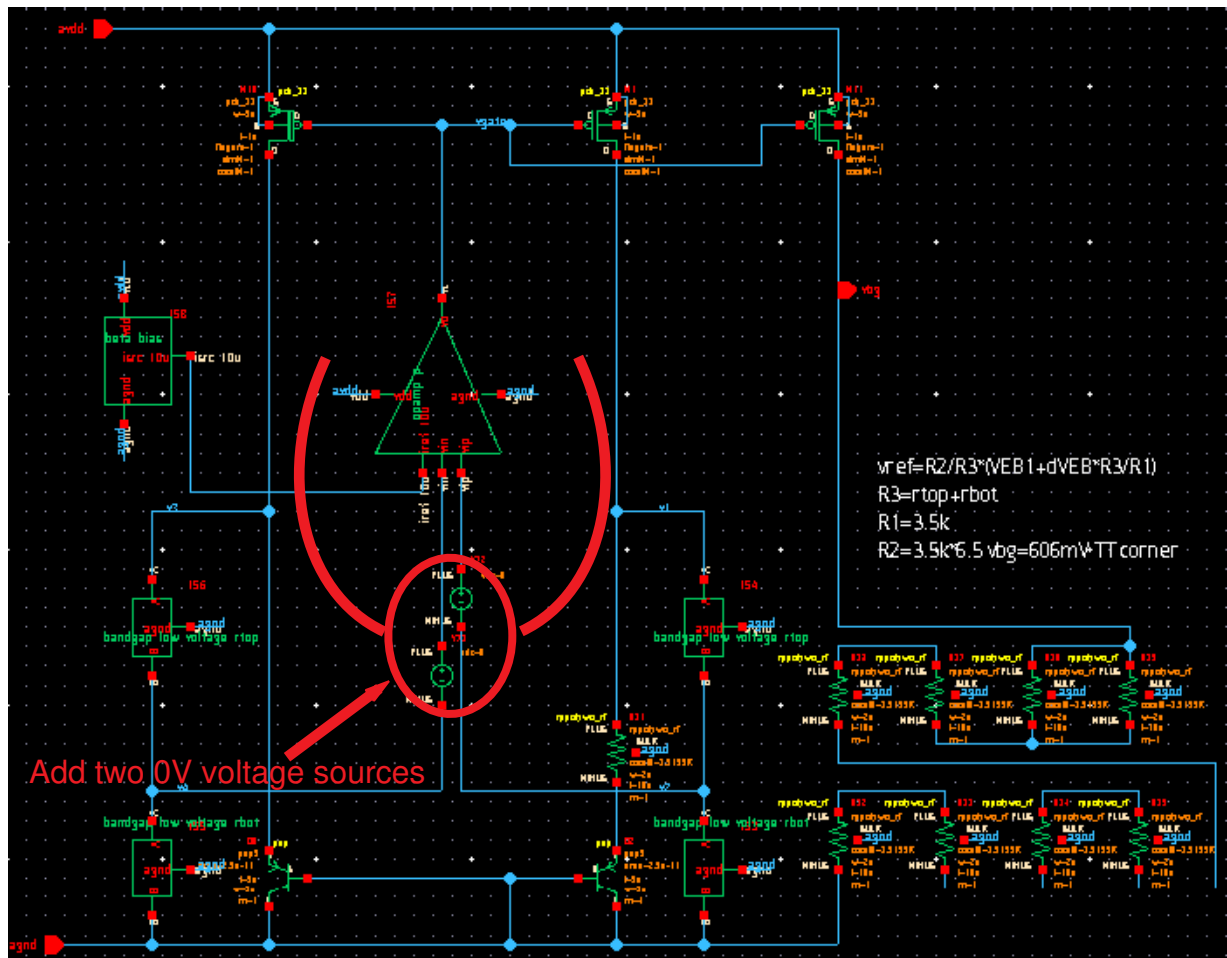
1. Insert a 0V voltage source in the feedback path
2. Specify an input source
3. Specify an ideal amplifier
4. Specify a feedback resistor
5. Specify a source resistor
6. Specify an output pole
7. Perform an .AC analysis (required)
8. Perform a single-mode .LSTB analysis
9. Add the .PROBE command for signals to be plotted

Differential Mode Example: Bandgap

The following is partial netlist showing the addition of two 0V DC sources for stability:

```
.subckt bandgap_low_voltage agnd avdd vbg
xi57 agnd net317 avdd v4_i v2_i vgate opamp_p
vlstbn v4_i v4 0          $0V DC source for stability loop analysis
vlstbp v2_i v2 0          $0V DC source for stability loop analysis
*
*
.ends bandgap_low_voltage
.AC DEC 100 100 10e9      $ AC analysis is required
.LSTB mode=diff vsource=xi4.vlstbp,xi4.vlstbn $ LSTB analysis
.option post probe
.probe AC lstb(m) lstb(p) lstb(db) $ Probe mag, phase, dB
.measure LSTB pm phase_margin    $ Measure phase margin
.end
```

Figure 41 Adding two 0V voltage sources to bandgap subcircuit



Differential mode output can be viewed in both X-Y and Polar plots:

Figure 42 WaveView plots enable you to access differential output



The *.lis output for this differential mode analysis is as follows:

```
*** LSTB analysis ***
gain_margin(dB) = 13.23320
phase_margin(deg) = 51.28824
unity_gain_freq(Hz) = 9.6913777E+07
loop_gain_mini_freq(dB) = 50.47896
```

Controlling LSTB Warning Messages

To avoid re-running simulations if there is a syntax error in the .LSTB statement you can use .OPTION STRICT_CHECK. Use of this option is especially useful when running long AC and LSTB analyses on a batch machine and the AC analysis succeeds.

You can use `.OPTION STRICT_CHECK` to control the following messages:

1. The second v-source for diff/comm mode testing',' is missing in LSTB statement.',' Analysis is omitted.'
2. 'The v-source name in LSTB',' statement is invalid. Analysis is omitted.
3. 'The mode type is missing in',' LSTB statement. Analysis is omitted.'
4. 'The mode type in LSTB statement',' is invalid. Analysis is omitted.'
5. 'Expected a keyword in LSTB statement.',' Analysis is omitted.'
6. 'The v-sources are missing in',' LSTB statement. Analysis is omitted.'
7. 'The v-source for single mode is',' missing in LSTB statement.',' Analysis is omitted.'
8. 'The first v-source for diff/comm mode testing',' is missing in LSTB statement.',' Analysis is omitted.'
9. 'The second v-source for diff/comm',' mode in LSTB statement is missing.',' Analysis is omitted.'
10. 'Found an invalid keyword "',1a7,'" for LSTB analysis.'
11. 'The second v-source for diff/comm mode testing',' is missing in LSTB statement.',' Analysis is omitted.'

For details on usage, see `.OPTION STRICT_CHECK` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

The LSTB analysis name in FIND...WHEN should be consistent.

With the following example:

```
.lstb name1 mode=single vsource=v1
.lstb name2 mode=single vsource=v2
.measure ac phase_margin find lstb(p:name1) when lstb(db:name2)=0
```

You will receive the warning message:

```
WHEN lstb name does not match its FIND lstb name
```

Other AC Analysis Statements - .DISTO, .NET, and .SAMPLE

The following sections describe the commands you can use to perform other types of AC analyses:

- [Using .DISTO for Small-Signal Distortion Analysis](#)
- [.NET Parameter Analysis](#)
- [Using .SAMPLE for Noise Folding Analysis](#)

Using .DISTO for Small-Signal Distortion Analysis

The `.DISTO` statement computes the distortion characteristics of the circuit in an AC small-signal, sinusoidal, steady-state analysis. PrimeSim HSPICE computes and reports five distortion measures at the specified load resistor. The analysis is performed assuming that one or two signal frequencies are imposed at the input. The first frequency, F1 (used to calculate harmonic distortion), is the nominal analysis frequency set by the `.AC` statement frequency sweep. The optional second input frequency, F2 (used to calculate intermodulation distortion), is set implicitly by specifying the `skw2` parameter, which is the ratio F2/F1.

For command syntax and examples, see the `.DISTO` command in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

.NET Parameter Analysis

PrimeSim HSPICE uses the AC analysis results to perform network analysis. The `.NET` analysis can only extract linear transfer parameters for one- and two-port networks. Because of this limitation, `.LIN` analysis is the preferred linear network analysis method, see Linear Network Parameter Analysis in *PrimeSim HSPICE® User Guide: Signal Integrity Modeling and Analysis*.

The `.NET` statement defines Z, Y, H, and S-parameters to calculate. The following list shows various combinations of the `.NET` statement for network matrices that PrimeSim HSPICE calculates:

```
.NET Vout Isrc    V = [Z]      [I]
.NET Iout Vsrc    I = [Y]      [V]
.NET Iout Isrc    [V1 I2]T = [H]      [I1 V2]T
.NET Vout Vsrc    [I1 V2]T = [S]      [V1 I2]T
([M]T represents the transpose of the M matrix).
```

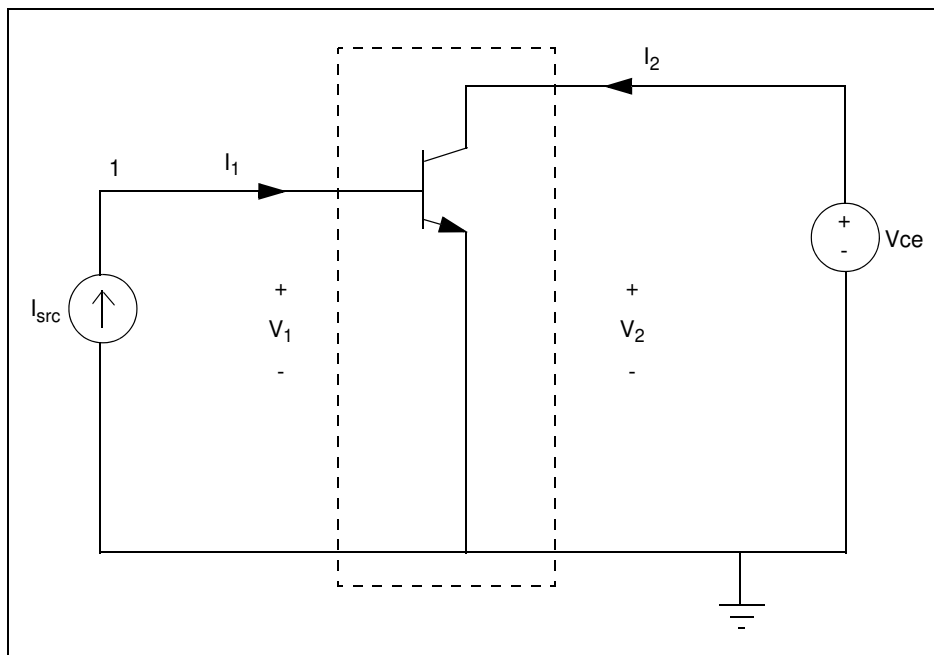
Note:

The preceding list does not mean that you must use combination (1) to calculate Z parameters. However, if you specify `.NET Vout Isrc`, PrimeSim

HSPICE initially evaluates the Z matrix parameters. It then uses standard conversion equations to determine S-parameters or any other requested parameters.

Figure 43 shows the importance of variables in the .NET statement. Here, I_{src} and V_{ce} are the DC biases, applied to the BJT.

Figure 43 Parameters with .NET V(2) Isrc



This .NET statement provides an incorrect result for the Z parameter calculation:

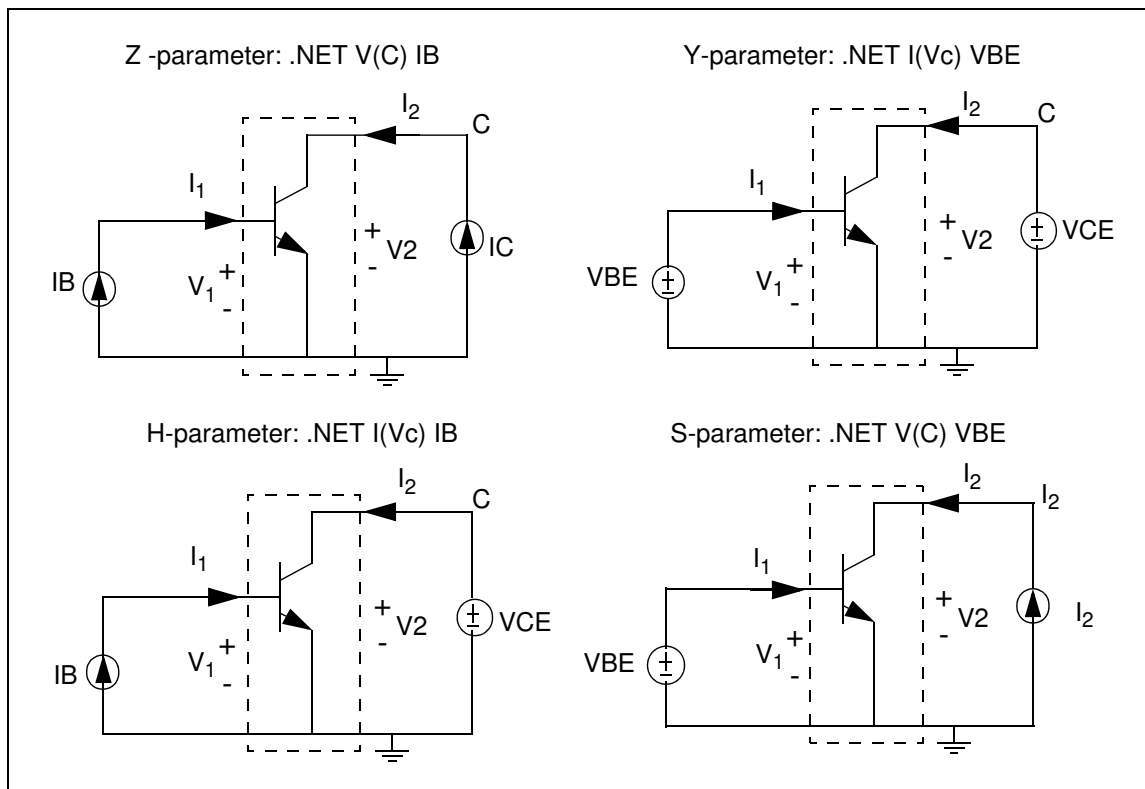
```
.NET V(2) Isrc
```

When PrimeSim HSPICE runs AC analysis, it shorts all DC voltage sources; all DC current sources are open-circuited. As a result, V(2) shorts to ground and its value is zero in AC analysis. This affects the results of the network analysis.

In this example, PrimeSim HSPICE attempts to calculate the Z parameters (Z_{11} and Z_{21}), defined as $Z_{11}=V1/I1$ and $Z_{21}=V2/I1$ with $I2=0$. The above example does not satisfy the requirement that $I2$ must be zero. Instead, $V2$ is zero, which results in incorrect values for Z_{11} and Z_{21} .

Figure 44 shows the correct biasing configurations for performing network analysis for the Z, Y, H, and S-parameters.

Figure 44 Network Parameter Configurations



Example

To calculate the H parameters, PrimeSim HSPICE uses the .NET statement.

```
.NET I(Vc) IB
```

VC denotes the voltage at the C node, which is the collector of the BJT. With this statement, PrimeSim HSPICE uses the following equations to calculate H parameters immediately after AC analysis:

$$V_1 = H_{11} \cdot I_1 + H_{12} \cdot V_2$$

$$I_2 = H_{21} \cdot I_1 + H_{22} \cdot V_2$$

To calculate Hybrid parameters (H_{11} and H_{21}), the DC voltage source (V_{CE}) sets V_2 to zero, and the DC current source (I_B) sets I_1 to zero. Setting I_1 and V_2 to zero, precisely meets the conditions of the circuit under examination: the input current source is open-circuited, and the output voltage source shorts to ground.

A data file containing measured results can drive external DC biases applied to a BJT. Not all DC currents and voltages (at input and output ports) might be available. When you

run a network analysis, examine the circuit and select suitable input and output variables. This helps you to obtain correctly calculated results. The following example demonstrates PrimeSim HSPICE network analysis of a BJT.

Using .SAMPLE for Noise Folding Analysis

For data acquisition of analog signals, data sampling noise often needs to be analyzed. This is accomplished with the `.SAMPLE` statement used in conjunction with the `.NOISE` and `.AC` statements. The `SAMPLE` analysis performs a simple noise folding analysis at the output node.

For the syntax and description of the `.SAMPLE` statement, see the `.SAMPLE` command in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

12

Transient Analysis

Describes how to use transient analysis to compute the circuit solution.

Transient analysis computes the circuit solution, as a function of time, over a time range specified in the `.TRAN` statement.

For descriptions of individual PrimeSim HSPICE commands referenced in this chapter, see the *PrimeSim™ Continuum Reference Manual: Commands and Control Options*.

For discussion of use of the `.TRAN` command in subcircuit blocks, see [Using Isomorphic Analyses in Subcircuit Blocks on page 89](#).

For full description of analyzing time-variant noise, see Transient Noise Analysis in the *PrimeSim HSPICE User Guide: Advanced Analog Simulation and Analysis*.

PrimeSim HSPICE ships hundreds of examples for your use; see Listing of Demonstration Input Files in the *PrimeSim HSPICE User Guide: Demonstration Netlists* for paths to the demo files.

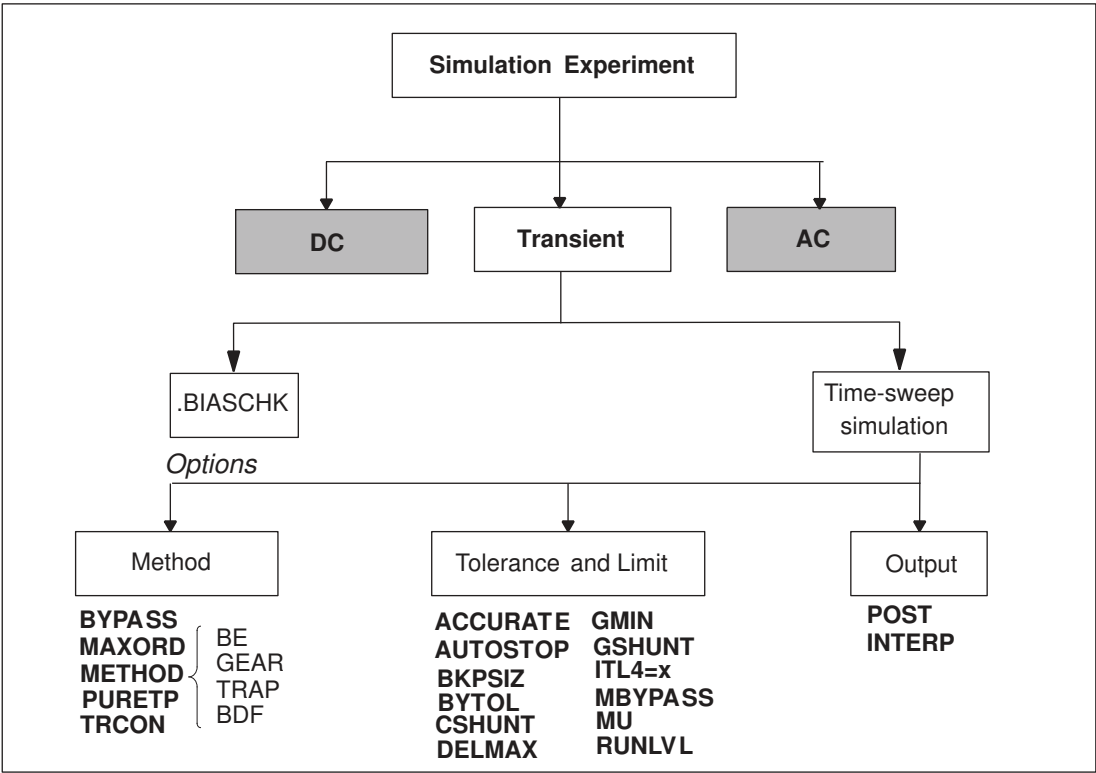
The following sections present these topics:

- [Simulation Flow](#)
- [Overview of Transient Analysis](#)
- [Transient Control Options](#)
- [Simulation Speed and Accuracy Using the RUNLVL Option](#)
- [Numerical Integration Algorithm Controls](#)
- [Dynamic Check Using the .BIASCHK Statement](#)
- [Checking and Detecting Dynamic Switching, Power, IR drops, and Surge Currents](#)
- [Storing and Restoring Checkpoint Files](#)
- [Specifying Monte Carlo or Temperature Sweeps](#)
- [Golden Reference for Control Options](#)
- [Troubleshooting: Internal Time Step, Measurement Errors](#)

Simulation Flow

Figure 45 on page 303 illustrates the simulation flow for transient analysis in PrimeSim HSPICE.

Figure 45 Transient Analysis Simulation Flow



Overview of Transient Analysis

Transient analysis simulates a circuit at a specific time. *Some* of its algorithms, control options, convergence-related issues, and initialization parameters are different than those used in DC analysis. However, a transient analysis first performs a DC operating point analysis, unless you specify the UIC option in the `.TRAN` statement.

Unless you set the initial circuit operating conditions, some circuits (such as oscillators, or circuits with feedback) do not have stable operating point solutions. For these circuits, either:

- Break the feedback loop, to calculate a stable DC operating point, or
- Specify the initial conditions in the simulation input.

For setting initial conditions, see [Initial Conditions and UIC Parameters](#).

Example

In the following example, the UIC parameter (in the `.TRAN` statement) bypasses the initial DC operating point analysis. The `.OP` statement calculates the transient operating point (at $t=20$ ns), during the transient analysis.

```
.TRAN 1ns 100ns UIC
.OP 20ns
```

In a transient analysis, the internal "timestep too small" error message indicates that the circuit failed to converge. The cause of this convergence failure might be that stated initial conditions are not close enough to the actual DC operating point values. Use the commands in this chapter to help achieve convergence in a transient analysis. See also: [Troubleshooting: Internal Time Step, Measurement Errors](#).

The following sections discuss these topics:

- [Data-Driven vs. Outer Parameter Sweeps](#)
- [Transient Analysis Output](#)

Data-Driven vs. Outer Parameter Sweeps

The following are the differences between a data-driven sweep and an outer parameter sweep:

Data-Driven Sweep

The use of a data set allows the sweeping of both nonuniform values and multiple parameters. You need to specify each value to vary in the simulation. This method generates one output file for the entire simulation. When viewing signals, the traces correspond to each parameter sweep.

Example: Data-Driven Sweep

```
.tran 1n 100n sweep data=mydata
.data mydata param1 param2 ...
val1 vala ...
val2 valb ...
....
.enddata
```

Parameter Sweep

When you express the values of a parameter by using decade, octave, linear, or point-of-interest variation, you can use the `sweep` keyword to control the parameter. This method does not allow for the sweeping of multiple parameters. Similar to the data-driven sweep,

PrimeSim HSPICE creates only one output file with the signals having multiple traces. Be sure to sequence the *var* (param) before the *type* (DEC, LIN).

Examples

In this example, *param* varies 10 times for each decade from 1 to 10 u and a transient analysis runs for each value.

```
.tran 1n 100n sweep param DEC 10 1u 10u
```

In this example, *param* varies 5 equal times from 1u to 10u with a transient analysis for each value.

```
.tran 1n 100n sweep param LIN 5 1u 10u
```

Sweeping Multiple Parameters

Although PrimeSim HSPICE does not directly provide the facility to sweep multiple parameters, it does offer the *.DATA* table structure. A perl script is available to allow you to specify lists of parameters and values at <https://solvnetplus.synopsys.com/s/article/Sweeping-multiple-parameters-1576070024270>.

Note:

You must be logged in to SolvNetPlus for the link to connect directly to the article. If you are prompted to log in to SolvNetPlus upon clicking the link to the article, log in, then click the link again to reach the article.

This script creates a *.DATA* table with all permutations of the listed values. It also allows you to create the *.ALTER* command instead of a *.DATA* table, if preferred. For usage details, run `hspice_param_sweeper -h`.

The script's output goes to STDOUT, so redirect it to a file, for example: `hspice_param_sweeper > param_sweep.sp`), and then *.INCLUDE* the file into your PrimeSim HSPICE netlist. If you choose to create the *.ALTER* command, make sure you *.INCLUDE* them at the very end of your netlist.

If you create a *.DATA* table, you can invoke it as follows:

```
.TRAN 10p 100n SWEEP DATA=sweeper_params
```

Here is a sample input to the script.

```
vddr: 1.1, 1.0, 0.9  
vssr: 0.0  
temp: 0, 55, 100
```

Note that *temp* is a special parameter that sweeps the simulation temperature. This example produces a *.DATA* table with 9 rows (3*1*3) containing all combinations of the listed parameter values, or 1 base sim + 8 *.ALTER* if you use the *-alter* option.

Here is the output produced by the sample input.

```
.DATA sweeper_params temp vddr vssr
 0 1.1 0.0
55 1.1 0.0
100 1.1 0.0
 0 1.0 0.0
55 1.0 0.0
100 1.0 0.0
 0 0.9 0.0
55 0.9 0.0
100 0.9 0.0
.ENDDATA
```

After you download the script named `hspice_param_sweeper.gz` (right-click and select “Save Target As...”), be sure to modify the first line of the script to point to your local installation of perl. The default path should work on most systems.

Specifying Data Driven Timesteps

Instead of using a constant time step in a `.TRAN` statement, you can specify the timesteps using an inline data statement for the transient simulation.

The data defined in the `.DATA` statement should define the time point and current value for a PWL current source. In the following example, the `.DATA` statement *tstep_val* defines the time step, *step_val* and the current value, *ival*. PrimeSim HSPICE uses the timesteps defined in the `.DATA` statement during the transient simulation.

```
Ipwl nd1 0 PWL (step_val ival)
.tran DATA = tstep_val
.DATA tstep_val step_val ival
+ 10p 1m
+ 30p 10m
+ 70P 10m
+ 100p 100m
.ENDDATA
```

The time step value specified in the data table (`DATA=tstep_val`) controls the print intervals.

Transient Analysis Output

```
.print tran ov1 [ov2 ... ovN]
.probe tran ov1 [ov2 ... ovN]
.measure tran measspec
```

The *ov1*, ... *ovN* output variables can include the following:

- *V(n)*: voltage at node *n*.
- *V(n1<,n2>)*: voltage between the *n1* and *n2* nodes.

- $V_n(d1)$: voltage at n^{th} terminal of the $d1$ device.
- $I_n(d1)$: current into n^{th} terminal of the $d1$ device.
- 'expression': expression, for the plot variables above

You can use wildcards to specify multiple output variables in a single command. Output content depends on `.OPTION POST` or `.OPTION PROBE`.

Parameter	Description
*.print	Writes the output from the <code>.PRINT</code> statement to a *.print file. PrimeSim HSPICE does not generate a *.print# file. • The header line contains column labels. • The first column is time. • The remaining columns represent the output variables specified with <code>.PRINT</code>. • Rows that follow the header contain the data values for simulated time points.
*.tr#	Writes output from the <code>.PROBE</code> , <code>.PRINT</code> , or <code>.MEASURE</code> statement to a *.tr# file.

Transient Control Options

Method, tolerance, and limit options in this section modify the behavior of transient analysis integration routines. Delta is the internal time step. `TSTEP` and `TSTOP` are the step and stop values in the `.TRAN` statement.

Table 48 lists the options for `RUNLVL`.

Table 48 Transient Control Options with `RUNLVL` Turned On, by Category

Method	Tolerance and Limit		Output
BYPASS MAXORD METHOD= Backward-Euler (BE)	ACCURATE	GMIN GSHUNT ITL4	POSTINTERP
GEAR	AUTOSTOP	MBYPASS MAXAMP	
TRAP	BKPSIZ BYTOL	MU RUNLVL	
BDF	CSHUNT DELMAX		
PURETP TRCON			

For discussion of `METHOD` options, see [Numerical Integration Algorithm Controls on page 310](#).

Simulation Speed and Accuracy Using the RUNLVL Option

The following sections discuss these topics:

- [The RUNLVL Algorithm](#)
- [RUNLVL Features](#)
- [Guidelines for RUNLVL Setting](#)
- [Interactions Between .OPTION RUNLVL and Other Options](#)

The RUNLVL Algorithm

The RUNLVL algorithm, which is on by default in PrimeSim HSPICE, focuses on a balance between speed and accuracy. The RUNLVL algorithm:

1. Uses an enhanced Local Truncation Error (LTE) method based on nodal voltage for time step control. This is advantageous because voltage is the target result users want from a simulation, and there is a clear mathematical relation between error tolerance and time step.
2. Adopts a new Newton-Raphson (NR) iteration method for transient analysis. It not only improves the convergence but also makes the convergence faster.
3. Improves the BYPASS algorithm, as well.

RUNLVL Features

The RUNLVL algorithm provides the following characteristics:

- Simplifies accuracy control by setting RUNLVL values between 1 and 6 with 6 discrete settings (1=fastest, 6=most accurate).
- Avoids interpolation error in .MEASURE statements by using the interpolating polynomial used by the time integration method.
- Dynamically checks for correct handling of input signals and controlled sources between computed time steps to avoid setting small time steps before transient simulation start.
- Allows PrimeSim HSPICE to take time steps no larger than $(T_{stop}-T_{step})/20$. DELMAX automatically sets $(T_{stop}-T_{step})/20$ if there is no specific setting of DELMAX. The effect is that, for example, PrimeSim HSPICE can take larger time steps for flat regions.

The RUNLVL algorithm scales all simulation tolerances simultaneously and affects time step control, convergence, and model bypass all at once.

This algorithm activates only by use of `.OPTION RUNLVL=value`. Higher values of RUNLVL result in smaller time steps (more Newton-Raphson iterations) to meet stricter error tolerances, and higher simulation accuracy.

Guidelines for RUNLVL Setting

A valid value for `.OPTION RUNLVL` is an integer from 1 to 6. Values outside of this range cause an error. The default value for RUNLVL is 3. This is the recommended starting setting. For simulations that require high accuracy use higher values. Use lower values for simulating pure digital or mostly digital circuits. PrimeSim HSPICE uses the last setting if you specify multiple settings of RUNLVL options.

The `.OPTION RUNLVL` invokes the advanced simulation algorithm, with the default value of RUNLVL=3. This is the recommended starting setting. However, you can set it to a higher value if the circuit type is pure analog and/or the simulation needs high accuracy.

Table 49 *Guidelines for RUNLVL Settings*

Circuit Type	RUNLVL Setting
Digital	RUNLVL=1-3
Analog or mixed signal accuracy	RUNLVL=3-5
Cell characterization	RUNLVL=5-6

A `*.lis` file reports the RUNLVL flag and its effective value. PrimeSim HSPICE sets the RUNLVL option automatically in the `$install_dir/hspice/hspice.ini` file, when it generates the file during the installation process.

All PrimeSim HSPICE simulations first try to find ONE implicit `hspice.ini` file and take it as the first include file; the search order for `hspice.ini` is:

1. Current working directory
2. User's home directory
3. `$install_dir/hspice` directory

Interactions Between .OPTION RUNLVL and Other Options

Since the latest algorithm invoked by RUNLVL sets the time step and error tolerance internally, many transient error tolerance and time step control options are no longer valid. Furthermore, to ensure the most efficiency of the new RUNLVL algorithm, you should let the new engine manage everything itself. Also, you should not tune the options listed in the following table.

Note:

Turning off the `RUNLVL` option by setting it to 0 is not supported. If you set `RUNLVL` to 0, then the PrimeSim HSPICE simulator resets `RUNLVL` to 5.

Option	Default value with <code>RUNLVL=3</code>
<code>ACCURATE</code> ⁴	0
<code>BYPASS</code> ⁵	2 for <code>RUNLVL=1-6</code>
<code>METHOD</code> ⁶	TRAP

If `.OPTION ACCURATE` is specified in the netlist together with `RUNLVL`, the value of `RUNLVL` is limited to 5 or 6; that is, specifying a `RUNLVL` value of 1, 2, 3, or 4 defaults to 5.

There is no dependency with `GEAR` and `ACCURATE` options.

For detailed information on `.option runlvl`, see `.OPTION RUNLVL` in *PrimeSim HSPICE® Reference Manual: Commands and Control Options*.

Numerical Integration Algorithm Controls

In PrimeSim HSPICE transient analysis, you can select one of several options solve the circuit differential algebraic equations:

- Backward-Euler
- Gear
- Trapezoidal
- BDF (Backward Differentiation Formulae)

Table 50 *Integration Method*

Integration Algorithm	Option Settings	Comments
Backward-Euler (BE)	<code>METHOD=GEAR MAXORD=1</code> or <code>METHOD=GEAR MU=0</code>	Backward-Euler only

4. If `.option ACCURATE` is set, then the `RUNLVL` value is limited to 5 or 6. Specifying a `RUNLVL` less than 5 results in a simulation at `RUNLVL=5`. When both `ACCURATE` and `RUNLVL` are set, the `RUNLVL` algorithm will be used.
5. When `RUNLVL` is set, `BYPASS` is set to 2. Users can redefine the `BYPASS` value by setting `.option BYPASS=value`; this behavior is independent of the order of `RUNLVL` and `BYPASS`.
6. `RUNLVL` can work with `METHOD=GEAR`; in cases where `GEAR` only determines the numeric integration method during transient analysis, the other options that were previously set by `GEAR` (when there is no `RUNLVL`) now are determined by the `RUNLVL` mode. This behavior is independent of the order of `RUNLVL` and `METHOD`.

Table 50 Integration Method (Continued)

Integration Algorithm	Option Settings	Comments
GEAR	METHOD=GEAR METHOD=GEAR MAXORD=2 3	Combines GEAR and BE 2nd/3rd order increases accuracy
TRAP	METHOD=TRAP METHOD=TRAP PURETP	Combines Trapezoidal and BE Trapezoidal only
BDF	METHOD=BDF METHOD=GEAR	Higher order integration (Backward Differentiation Formulae) PrimeSim HSPICE automatically selects BDF based on circuit type when METHOD=GEAR; To prohibit GEAR from automatically selecting BDF, use .OPTION MAXORD.

The advanced multicore algorithm (`-mn`) which you can use in multithread simulation (`-mt`) supports 2nd order GEAR, TRAP, and BDF methods. TRAP is the default method. Each integration algorithm has advantages and disadvantages.

The following sections discuss these topics:

- [TRAP](#)
- [GEAR and Backward-Euler](#)
- [BDF](#)

TRAP

The trapezoidal is often the preferred algorithm because of its high accuracy level and low simulation time. Recommendation: use the pure trapezoidal (PURETP) for oscillators to avoid numerical damping which can cause oscillator simulations to die out when you view output in a waveform plot. Unlike GEAR, using the TRAP method (`.OPTION METHOD = TRAP`) is not subject to automatic selection after the circuit fails to converge using GEAR method.

GEAR and Backward-Euler

The GEAR method is an appropriate algorithm for convergence. 2nd-order GEAR is more accurate than Backward-Euler. Recommendation: use the GEAR method for those circuit simulations that require high accuracy on current such as leakage current measurement.

If the circuit fails to converge using the Trapezoidal integration method, PrimeSim HSPICE uses the autoconvergence process where it changes to the GEAR method to run the transient analysis again from time=0.

When appropriate, to take advantage of high accuracy on medium to large analog and mixed signal circuits and also improve performance by using fewer iterations per time point, PrimeSim HSPICE automatically switches from method GEAR to BDF. However, circuits with some design constructs such as oscillators may not simulate correctly using BDF. In these cases, you can override the automatic selection of BDF by also using `.OPTION MAXORD = [1|2|3]`.

BDF

The BDF method is a high-order integration method that uses the backward differentiation formulae. Since BDF is for high accuracy applications, PrimeSim HSPICE recommends a RUNLVL setting of 3 or above. Use the BDF method with multithreading.

Two tolerance options are available to the user for the BDF method: `.OPTIONS BDFRTOL` (relative) and `BDFATOL` (absolute); each has a default of 1e-3. BDF can provide a speed enhancement to mixed-signal circuit simulation, especially for circuits with a large number of devices. The BDF method provides no performance advantage for use with small circuits in standard cell characterization. BDF could be an alternative to GEAR when TRAP fails to converge.

The `BDFATOL` and `BDFRTOL` options operate independent of `.OPTIONS RUNLVL` and `ACCURATE` settings with the following exception:

If either `.OPTION RUNLVL` or `ACCURATE` follows an `.OPTION BDFATOL` or `BDFRTOL` value, the `RUNLVL` or `ACCURATE` setting overrides the tolerance of the BDF algorithm. If you set `ACCURATE` with or without `RUNLVL`, the default for `BDFATOL` or `BDFRTOL` always defaults to 1e-5.

RUNLVL	BDFATOL and BDFRTOL
1	1e-2
2	1e-2
3	1e-3
4	1e-4
5	1e-4
6	1e-5

Dynamic Check Using the .BIASCHK Statement

The `.BIASCHK` statement can monitor the voltage bias, current, device-size, expression and region during transient analysis, and reports:

- Element name
- Time
- Terminals
- Bias that exceeds the limit
- Number of times the bias exceeds the limit for an element

For the syntax and description of this statement, see the `.BIASCHK` command in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

PrimeSim HSPICE saves the information as both a warning and a BIASCHK summary in the `*.lis` file. You can use this command only for active elements, resistors, capacitors, and subcircuits.

You can also use `.OPTION BIASFILE` and `.OPTION BIAWARN` with a `.BIASCHK` statement.

The following limitations apply to the `.BIASCHK` statement:

- Only these models support `.BIASCHK`: diode, jfet, nmos, pmos, bjt, and r/c models, as well as subcircuits.
- Only W and L MOSFET models support device-size checks.
- There is support for wildcards in element and model names and `except` definitions but *not* in expressions.

Four methods are available to check the data with the `.BIASCHK` command:

- [Limit and Noise Method](#)
- [Maximum Method](#)
- [Minimum Method](#)
- [Region Method](#)

Note:

Only MOSFET models support the region method of data checking.

See also .

Limit and Noise Method

For a transient simulation that uses the limit and noise method to check the data, use the following syntax:

For `local_max`

```
v(tn-1) > limit_value
```

The bias corresponds to any one of the following two conditions:

- `v(tn-1) > v(tn) && v(tn-1) >= v(tn-2)`
- `v(tn-1) >= v(tn) && v(tn-1) > v(tn-2)`

`local_min`: The minimum bias after the time last local max occurs.

A transient analysis records the `local_max` if it is greater than the limit. In the summary PrimeSim HSPICE reports after transient analysis, the `local_max(next)` replaces the `local_max(current)` when the following comparison is true:

```
local_max(current) - local_min < noise && local_max(next) - local_min < noise &&  
local_max(current) < local_max(next)
```

At the end of the simulation, the listing file reports all `local_max` values as BIASCHK warnings. During other analyses, PrimeSim HSPICE issues warnings when the value you want to check is greater than the `limit_value` you specify.

Maximum Method

Use this syntax for a transient simulation that specifies the maximum method:

For `local_max`:

```
v(tn-1) > max_value
```

The bias corresponds any one of the following two conditions:

- `v(tn-1) > v(tn) && v(tn-1) >= v(tn-2)`
- `v(tn-1) >= v(tn) && v(tn-1) > v(tn-2)`

PrimeSim HSPICE lists all `local_max` values as BIASCHK warnings during a transient analysis. During other analyses, PrimeSim HSPICE issues warnings when the value you want to check is greater than `max_value` you specify.

Minimum Method

For a transient simulation that specifies the minimum method to check the data, use the following syntax:

For `local_min`:

```
v(tn) < min_value
```

The bias corresponds any one of the following two conditions:

- `v(tn-1) < v(tn) && v(tn-1) <= v(tn-2)`
- `v(tn-1) <= v(tn) && v(tn-1) < v(tn-2)`

During a transient analysis, all `local_min` values are listed as BIASCHK warnings. During other analyses, warnings are issued when the value you want to check is smaller than `min_value` you specify.

Region Method

This method is only for MOSFET models. Three regions exist:

- `cutoff`
- `linear`
- `saturation`

When the specified transistor enters and exits during transient analysis, the specified region is reported.

The `biaschk.sp` demo example is a netlist that uses the `.BIASCHK` command for a transient simulation. You can find the sample netlist for this example in:

```
$installdir/demo/hspice/apps/biaschk.sp.
```

The `soa.sp` demo example illustrates the use of `.BIASCHK` expression and condition statement to enforce Safe Operating Area (SOA). You can find the sample netlist for this example in: `$installdir/demo/hspice/apps/soa.sp.`

Using BIASCHK with HPP

The `.BIASCHK` command is available for use with PrimeSim HSPICE Precision Parallel (HPP) in the following applications: as an expression monitor and as an element and model monitor.

HPP plus `.BIASCHK` netlist syntax for the expression monitoring function is:

```
.BIASCHK 'expression' [limit=lim] [noise=ns] + [max=max] [min=min]
+ [simulation=op|dc|tr|all] [monitor=v|i|w|l]
+ [tstart=time1] [tstop=time2] [autostop]
+ [interval=time]
```

HPP plus `.BIASCHK` netlist syntax for an element and model monitoring function is:

```
.BIASCHK type terminal1=t1 [terminal2=t2]
+ [limit=lim] [noise=ns] [max=max] [min=min]
+ [simulation=op|dc|tr|all] [monitor=v|i]
+ [name=name1,name2,...]
+ [mname=modname_1,modname_2,...]
+ [tstart=time1] [tstop=time2] [autostop]
+ [except=name_1,name_2,...]
+ [interval=time] [sname=subckt_name1,subckt_name2,...]
```

See the `.BIASCHK` command in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*. See also [PrimeSim HSPICE Precision Parallel](#).

Checking and Detecting Dynamic Switching, Power, IR drops, and Surge Currents

This section discusses the following topics:

- [Using CHECK Statements](#)
- [POWER DC Analysis](#)
- [Detecting and Reporting Surge Currents](#)

Using CHECK Statements

The CHECK statements in PrimeSim HSPICE offer the following instrumentation:

- [Setting Global Hi/Lo Levels](#)
- [Slew, Rise, and Fall Conditions](#)
- [Edge Timing Verification](#)
- [Setup and Hold Verification](#)
- [IR Drop Detection](#)

The results of these statements appear in a file with an `.err` extension. To prevent creating unwieldy files, PrimeSim HSPICE advanced analog analyses reports only the first 10 violations for a particular check in the `.err` file.

Setting Global Hi/Lo Levels

You use the `.CHECK GLOBAL_LEVEL` statement to globally set the desired high and low definitions for all CHECK statements. For example,

```
.CHECK GLOBAL_LEVEL (hi lo hi_th lo_th)
```

This statement defines values for hi, lo, and the thresholds.

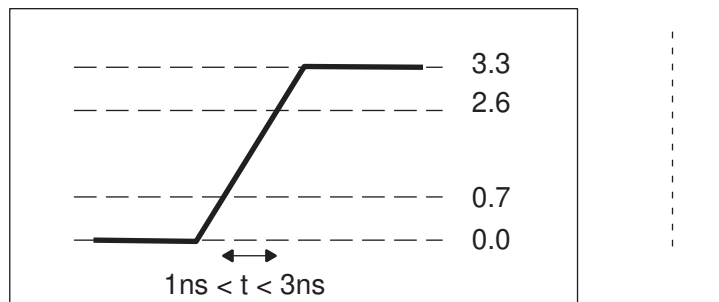
For syntax and description of this statement, see `.CHECK GLOBAL_LEVEL` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

Slew, Rise, and Fall Conditions

You use the `.CHECK SLEW` statement to verify that a slew rate occurs within the specified window of time. For example,

```
.CHECK SLEW (min max) node1 [node2 ...] [(hi lo hi_th lo_th)]
```

Figure 46 SLEW Example

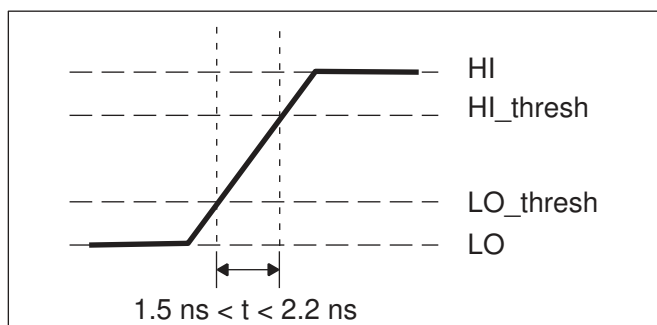


For syntax and description of this statement, see `.CHECK SLEW` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

You use the `.CHECK RISE` statement to verify that a rise time occurs within the specified window of time. For example,

```
.CHECK RISE (min max) node1 [node2 ...] [(hi lo hi_th lo_th)]
```

Figure 47 RISE Time Example



For syntax and description of this statement, see `.CHECK RISE` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

You use the `.CHECK FALL` statement to verify that a fall time occurs within the specified window of time. For example,

```
.CHECK FALL (min max) node1 [node2 ...>] [(hi lo hi_th lo_th)]
```

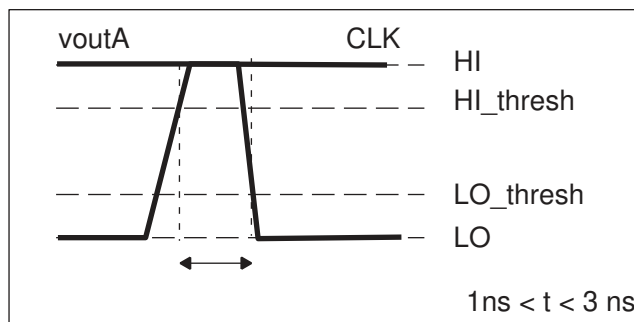
For syntax and description of this statement, see `.CHECK FALL` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

Edge Timing Verification

The edge condition verifies that a triggering event provokes an appropriate RISE or FALL action, within the specified time window. You use the `.CHECK EDGE` statement to verify this condition. For example,

```
.CHECK EDGE (ref RISE|FALL min max RISE|FALL)
+ node1 [node2 . . . ] [(hi lo hi_th low_th)]
```

Figure 48 EDGE Example



For syntax and description of this statement, see `.CHECK EDGE` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

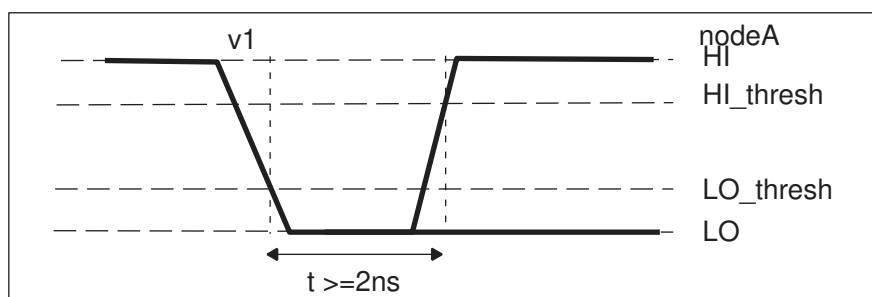
Setup and Hold Verification

You use the `.CHECK SETUP` and `.CHECK HOLD` statements to ensure that specified signals do not switch for a specified period of time. For example,

```
.CHECK SETUP (ref RISE|FALL duration RISE|FALL) node1
+[node2 . . . ] [(hi lo hi_th low_th)]
.CHECK HOLD (ref RISE|FALL duration RISE|FALL) node1
+[node2 . . . ] [(hi lo hi_th low_th)]
```

- For a SETUP condition, this is the minimum time before the triggering event, during which the specified nodes cannot rise or fall.

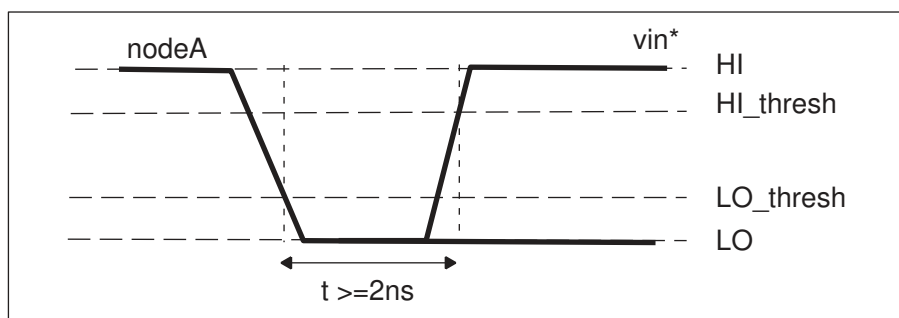
Figure 49 SETUP Example



For syntax and description of this statement, see `.CHECK SETUP` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

- For a HOLD condition, specify the minimum time after the triggering event, before the specified nodes can rise or fall.

Figure 50 HOLD Example



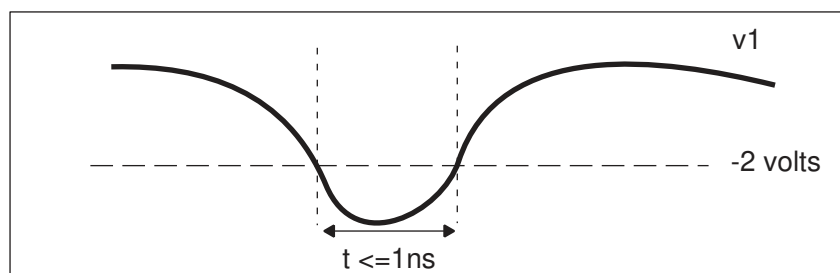
For syntax and description of this statement, see `.CHECK HOLD` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

IR Drop Detection

You use the `.CHECK IRDROP` statement to verify that the IR drop does not exceed, or does not fall below, a specified value for a specified duration. For example,

```
.CHECK IRDROP ( volt_val time ) node1 [node2 . . . ]
+ [( hi lo hi_th low_th )]
```

Figure 51 IR Drop Example



For syntax and description of this statement, see `.CHECK IRDROP` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

POWER DC Analysis

You use the `.POWERDC` (standby current) statement to calculate the DC leakage current of a design hierarchy. For example,

```
.POWERDC keyword subckt_name1...
+ [ith=threshold_current]
```

This statement creates a table that lists the measurements of the AVG, MAX, and MIN values for the current of every instance in the subcircuit. This table also lists the sum of the power of each port in the subcircuit.

You use the `SIM_POWERDC_HSPICE` option to increase the accuracy of operating point (OP) calculations.

Or for even higher accuracy in operating point calculations, you use the `SIM_POWERDC_ACCURACY` option.

For syntax and description of this statement and options, see `.POWERDC`, `.OPTION SIM_POWERDC_ACCURACY`, or `.OPTION SIM_POWERDC_PrimeSim HSPICE` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

This section covers the following topics:

- [Power DC Analysis Output Format](#)
- [POWER Analysis](#)

Power DC Analysis Output Format

```
*** Leakage Current Result ***
Subckt Name=XXX
Instance Name      Port  Max (A)   Min (A)   Avg (A)
Total Power        Max (W)   Min (W)   Avg (W)
NOTE:      Power=Sum{Ii * Vi}
Subckt Name=XXX
Instance Name      Port  Max (A)   Min (A)   Avg (A)
Total Power        Max (W)   Min (W)   Avg (W)
```

Example:

```
.global vdd vss
.powerdc all
x1 in1 mid1 inv
x2 mid1 out1 inv
.subckt inv in out
mn out in vss vss nch
mp vdd in out vdd pch
.ends
.end
```

Output:

```
*** Leakage Current Result ***
Subckt Name=Top Level
Instance Name      Port  Max (A)   Min (A)   Avg (A)
  x1   in          .....
  x1   out          .....
  x2   in          .....
  x2   out          .....
Total Power        .....
Subckt Name=inv
Instance Name      Port  Max (A)   Min (A)   Avg (A)
  mn   d           .....
  mn   g           .....
  mn   s           .....
  mn   b           .....
  mp   d           .....
  mp   g           .....
  mp   s           .....
  mp   b           .....
Total Power        .....
```

POWER Analysis

The `.POWER` statement in PrimeSim HSPICE creates a table, which by default contains the measurements for AVG, RMS, MAX, and MIN for every signal specified. For example,

```
.POWER signals [REF=vname FROM=start_time TO=end_time]
```

By default, the scope of these measurements are from 0 to the maximum time point specified in the `.TRAN` statement.

For syntax and description of `.POWER` statement, see `.POWER` in the *PrimeSim HSPICE Reference Manual: Command and Control Options*.

In the following example, no simulation start and stop time exists for the `x1.in` signal, so the simulation scope for this signal runs from the start (0ps) to the last `.tran` time (100ps).

```
.power x1.in
.tran 4ps 100ps
```

In the following example, you can use the `FROM` and `TO` times to specify a separate measurement start and stop time for each signal. In this example:

- The scope for simulating the `x2.in` signal is from 20ps to 80ps.
- The scope for simulating the `x0.in` signal is from 30ps to 70ps.

```
.param myendtime=80ps
.power x2.in REF=a123 from=20ps to=80ps
.power x0.in REF=abc from=30ps to='myendtime - 10ps'
```

Setting Default Start and Stop Times

In addition to using `FROM` and `TO` times in a `.POWER` statement, you can also use the `SIM_POWERSTART` and `SIM_POWERSTOP` options with `.POWER` statements to specify default start and stop times for measuring signals during simulation. These times apply to all signals that do not have their own defined `FROM` and `TO` measurement times. For example,

```
.OPTION SIM_POWERSTART=time
.OPTION SIM_POWERSTOP=time
```

These options control the power measurement scope; the default is for the entire run.

For syntax and description of these options, see `.OPTION SIM_POWERSTART` or `.OPTION SIM_POWERSTOP` in the *PrimeSim HSPICE Reference Manual: Command and Control Options*.

Controlling Power Analysis Waveform Dumps

Use the `SIM_POWERPOST` option to dump the control-power analysis waveform. For example,

```
.OPTION SIM_POWERPOST=ON|OFF
```

When you consider the potentially enormous number of signals, there is no waveform dump by default for the signals in the `.POWER` statement. Setting `SIM_POWERPOST=ON` turns on power analysis waveform dumping.

Detecting and Reporting Surge Currents

The `.SURGE` statement in PrimeSim HSPICE automatically detects and reports a current surge that exceeds the specified surge tolerance. For example,

```
.SURGE surge_threshold surge_width node1 [node2 .... noden]
```

This statement reports any current surge that is greater than `surge_threshold` for a duration of more than `surge_width`.

For additional information, see `.SURGE` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

Storing and Restoring Checkpoint Files

Store/restore is a feature that creates checkpoint files describing a running process during transient analysis; the operating system can later reconstruct the process from the contents of this file. This feature is not supported PrimeSim HSPICE advanced analog analyses.

Table 51 Supported Platforms

Linux RHEL	Linux SUSE	Windows
Yes	Yes	Only support IC/OP Not support SNAPSHOT

Note:

The `-restore` operation should be submitted on a machine that has the same kernel version as the machine used to store, otherwise, a failure may occur.

The following sections discuss these topics:

- [Store by Using the .STORE Command](#)
- [Store by Interrupting the Simulation Process](#)
- [Restore Operation](#)
- [Storing and Restoring Using HSPUI](#)
- [Usage Requirements for type=IC](#)

Store by Using the .STORE Command

You can trigger the store function either by issuing a `.option primesim_auto_restart="modepercent"` command in the netlist or by interrupting the running simulation process.

Use the following syntax in a netlist:

```
.STORE [TYPE=IC/MEMDUMP]
+      [FILE=save_file_prefix]
+      [TIME=time1] [TIME=time2] ... [TIME=timeN]
+      [PERECNT=value]
+      [REPEAT=period]
+      [TRANTIME=0/1]
+      [SAVE_ON_KILL=0/1]
```

Argument	Description
TYPE=IC MEMDUMP	Stores checkpoint data to either an IC type or a memory dump file. If unspecified, the default checkpoint file is of the TYPE=IC and the name prefix is same as the PrimeSim HSPICE output file.
FILE=save_file_prefix	Changes the prefix of the output file names.
TIME=time1,time2,...timeN	Collects checkpoint data beginning at <i>time1</i> after the start of transient analysis. It then updates the checkpoint data every 21,600 wall-clock seconds if no checkpoint period is specified.
REPEAT=period	If you specify a nonzero period, new checkpoint data is collected at every period, starting at transient time=0 and overwriting previous interval checkpoint data. If a nonzero time1 is specified, checkpoint data is collected at time1 + period * n, where n is an integer. Period is always calculated based on time1. If repeat=0, the store operation is disabled. If you set both time=0 and repeat=0, checkpoint data is saved at transient time=0 only.
TRANTIME=0 1	• If set to 0, time1 and period are taken as wall-clock time. • If set to 1, time1 and period are transient times or times is smaller than TSTOP. Note: If TYPE=MEMDUMP, TRANTIME is ignored.

Argument	Description
<code>PERECNT=value</code>	Specifies how often the snapshots are saved as a percentage of <code>tstop</code> when <code>type=SNAPSHOT</code> .
<code>SAVE_ON_KILL=0 1</code>	If set to 1, the checkpoint data is saved on kill and halts the simulation.

For more information of using the `.STORE` command, see *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

Store by Interrupting the Simulation Process

The following system command can interrupt PrimeSim HSPICE transient analysis simulation and create checkpoint files: `kill -USR2 process_id`. This command allows users to trigger a checkpoint without exiting; the simulation continues after the checkpoint is done.

After the checkpoint files are generated, use `kill -9 process_id` to stop the simulation, if needed.

Example

If system command `kill -USR2 process_id` is submitted, PrimeSim HSPICE generates checkpoint files, then continues the current simulation. If the simulation is terminated later on, the checkpoint files can be used to continue the simulation when you enter the command-line `restore` command.

Store by Terminating the Simulation Process

The following system command creates checkpoint files in PrimeSim HSPICE transient analysis simulation before exiting, if `SAVE_ON_KILL` to 1 for `TYPE= SNAPSHOT` is set.

The following commands allow to trigger a checkpoint before exiting; the simulation restores based on the checkpoint files.

```
kill -INT process_id
kill -TERM process_id
kill -QUIT process_id
```

Note:

The checkpoint files does not get generated when you use the `kill -9 process_id` command to stop the simulation.

Restore Operation

The restore operation takes place at the command line:

```
hspice -restore checkpoint_file
```

```
hspice -auto_restore
```

```
hspice -auto_restart
```

The `checkpoint_file` specifies from which simulation the checkpoint data is to be restored.

Any output files generated by the previous simulation should not be removed. After the restore simulation is done, the output files will be updated.

Example

For `TYPE=IC`:

In this example, PrimeSim HSPICE output file will be tagged with “1e-7”, like `test.1e-7.tr0`:

```
HSPICE -i test.sp -restore test.1e-7.ic0 -o test
```

In this example, PrimeSim HSPICE output file will have “save” tagged, like `test.save.tr0`:

```
HSPICE -i test.sp -restore test.save.ic0 -o test
```

For `TYPE=SNAPSHOT`:

In this example:

PrimeSim HSPICE output file is tagged with “201ns”, such as `test_tmp_201ns-@Linux:3.10.0.el7.x86_64.isnapshot`:

```
HSPICE -i test.sp -o test -restore test_tmp_201ns-@Linux:3.10.0.el7.x86_64.isnapshot.
```

In this example, PrimeSim HSPICE continues simulation automatically as listed below:

```
HSPICE -i test.sp -o test -auto_restart
```

```
HSPICE -i test.sp -o test -auto_restore
```

Storing and Restoring Using HSPUI

Use the following broad steps to store and restore a PrimeSim HSPICE simulation on a Windows platform:

1. To configure HSPUI to store and later restore a simulation, click **Configuration > Options > Store/Restore > Trigger the Store Function**.
2. Perform a simulation. While running the simulation, the **Simulation** button changes to **Store**. Click the button to suspend the simulation.
3. Once suspended, the button changes to **Restore**. Select the checkpoint file `*.time.ic0` and then click **Restore** to restore the simulation.

Usage Requirements for type=IC

- Store and restore can be performed on 64-bit Linux and Windows operating systems.
- Restore must be performed on the same platform that the store operation used.
- The restore-machine must have at least as much (available) RAM as the process was using when the checkpoint data was saved.
- Store/restore is not available if you are using Cadence[®] PSF output.

Usage Requirements for type=SNAPSHOT

This feature has the following limitations:

- The PrimeSim HSPICE binary must be the same for snapshot generation and restart.
- The Linux Kernel and System Architecture must be the same for snapshot generation and restart.
- Restart may not work if the CPU model is different in the restart run even with the same kernel.
- The simulation condition, such as changing the number of CPUs, cannot be changed.
- Does not work with DATA sweeps including Monte Carlo, DP distribution simulation.
- Any changes in the netlist or options are ignored with `primesim_auto_restart` modes 2 or 3.

Note:

This feature only support `type= IC` on Windows platform.

Specifying Monte Carlo or Temperature Sweeps

Using the `.TRAN` statement you can either specify a Monte Carlo sweep or a temperature sweep. A `.temp` statement with more than one value can be used to sweep the temperatures of interest.

In the following example, PrimeSim HSPICE performs a transient Monte Carlo analysis at each temperature value specified in the `.temp` statement— -30C, 40C, and 125C, respectively:

```
.temp -30 40 125
.tran 1n 20n sweep monte=10
```

Example netlist: Simulating the netlist below results in three waveform files, `netlist.tr0`, `netlist.tr1`, and `netlist.tr2` with each displaying the results of the Monte Carlo sweep at each temperature value.

```
.options post
.temp -30 40 125
.tran 1n 5n sweep monte=10
.param resval=aunif(1000,400)
vsrc_one 1 0 5v
r_one 1 2 resval tc1=0.02 tc2=0.01
c_one 2 0 1u
.end
```

Golden Reference for Control Options

When trying to determine the acceptable trade-off between PrimeSim HSPICE accuracy and transient analysis simulation performance, it is important to first establish a reference value for the measurements you are using to evaluate the performance (speed and accuracy) of a given PrimeSim HSPICE configuration. There are multiple ways to configure PrimeSim HSPICE for higher accuracy. The following is a good starting point that you might want to modify for your specific application:

```
.OPTION RUNLVL=6 KCLTEST DELMAX=a_small_value
```

The options are described as follows:

Options	Description
<code>.OPTION RUNLVL</code>	Invokes the RUNLVL algorithm and sets tolerances to their tightest values.
<code>.OPTION KCLTEST</code>	Activates Kirchhoff's Current Law testing for every circuit node.

Options	Description
.OPTION DELMAX	Sets the largest time step that PrimeSim HSPICE is allowed to take. It should be set to the smallest value (1ps, for example) that still allows the simulation to finish in a reasonable amount of time. Typically, it should be set approximately $1 / (20 * \text{highest-frequency-activity-in-the-circuit})$. Warning: This option can create very large <i>tr0</i> files. Be careful to only probe the needed nodes (use .OPTION PROBE combined with .PROBE).

Troubleshooting: Internal Time Step, Measurement Errors

- [Troubleshooting 'Time step too small' Errors](#)
- [How TSTEP Affects a Transient Simulation](#)
- [Troubleshooting .MEASUREMENT Issues](#)

Troubleshooting 'Time step too small' Errors

These are the usual steps to follow when you get an “internal time step too small” in transient analysis errors. The best approach is to incrementally change the values of these options, one at a time. Note the time immediately following the time step error in the list file. If your simulation gets further into the run, then the option is beneficial and you may wish to try higher or lower values as appropriate.

1. Be sure you are using the latest version of PrimeSim HSPICE if you can. Improvements are continuously made to convergence algorithms.
2. Comment out all time step and convergence options you already have and try increasing the value of .OPTION RUNLVL as a first step.

The RUNLVL option is turned on by default to level=3. It implements improved convergence techniques. If a higher RUNLVL such as 5 or 6 is set, try a lower RUNLVL to get convergence.

Note:

Remove any other convergence options when you use RUNLVL.

3. Use GSHUNT and CSHUNT to add small amounts of conductance and or capacitance from each node to ground. Together or alone, these options can help solve time step too small problems caused by either high-frequency oscillations or numerical noise.

```
.option gshunt=1e-13 cshunt=1e-17  
.option gshunt=1e-12 cshunt=1e-16  
.option gshunt=1e-11 cshunt=5e-15
```

```
.option gshunt=1e-10 cshunt=1e-15
```

```
.option gshunt=1e-9 cshunt=1e-14
```

4. Increase the time step value, to step over possible model discontinuities.

From original time step settings, change the `.TRAN` statement to incrementally increase TSTEP:

```
.tran (2)*tstep tstop
```

```
.tran (2.5)*tstep tstop
```

```
.tran (3)*tstep tstop
```

5. Using `.OPTION METHOD=GEAR` may help certain high gain analog (such as op-amps) and/or oscillatory circuits (such as a ring oscillators) during transient analysis by changing integration methods. `.option method=gear`
6. Investigate the device models used. Be sure the version of the models was developed for or qualified with the version of PrimeSim HSPICE you are using. For CMOS devices, make sure you have finite terminal capacitances and resistances. For level 49, be sure you have the model parameters as specified in the following example: (these are samples, not defaults)

```
.model mname nmos level=49 version=3.2  
+ cj=5e-4 cjsw=1e-10 cgd0=1e-10 cgs0=1e-10 rs=1e-9 rd=1e-9
```

In the case of BJT device, be sure to have the following model parameters set (again, examples, not defaults).

```
.model mname npn rb=50 r c=.4 re=1e-3
```

How TSTEP Affects a Transient Simulation

With the introduction of the RUNLVL algorithm, the impact of TSTEP has been greatly reduced to the point that it can be ignored except for niche applications like INTERP that causes transient data points to be saved at the interval defined by the TSTEP parameter.

TSTEP is specified in the transient analysis netlist command.

```
.TRAN TSTEP1 TSTOP1 TSTEP2 TSTOP2 ... TSTEPN TSTOPN
```

The most common usage is a single TSTEP/TSTOP pair. For example:

```
.TRAN 0.1ps 100ns
```

TSTEP has a variety of effects on the operation of PrimeSim HSPICE.

1. The minimum time step is determined by the RUNLVL algorithm:
 - RUNLVL=3 is the default.
 - The RMIN and RMAX options are ignored.
 - The maximum time step has a “soft” limit of $(T_{stop} - T_{step}) / 20$. The RUNLVL algorithm allows the time step to exceed $RMAX * TSTEP$.
 - The DELMAX option can be used to override the RUNLVL algorithm's choice of maximum time step. When DELMAX is used, TSTEP has no impact on simulation accuracy (except when used in conjunction with .OPTION INTERP (see # 2 below).
2. When OPTION INTERP is used (off by default):
 - INTERP forces the probed expressions to be printed to the output file every multiple of TSTEP. Probed expression will *only* be printed to the output at multiples of TSTEP.
 - INTERP has no impact on the simulator's internal timesteps.
 - The probed values printed to the output are linearly interpolated between the simulator solved timesteps preceding and following the $N * TSTEP$ time to print at.

Note:

Only use INTERP when specifically needed. For example, FFT post-processing.

Troubleshooting .MEASUREMENT Issues

If .MEASURE results are incorrect compared to the waveforms, the differences you see may be due to one or more of the following issues.

- You are not comparing the same point. Make sure that the proper nodes and sweep points are being used for each comparison.
- If .option INTERP is in your netlist, remove it. PrimeSim HSPICE only saves data points at the interval defined by the tstep parameter in the .TRAN statement. For example, for the .TRAN statement:

```
.tran 1n 100n
```

PrimeSim HSPICE saves 100 points at 1ns intervals to the .tr0 file. The lack of precision can cause issues with your measurements.

- In your `.TRAN` statement, use of the `START` keyword to delay output generation should be removed as it interferes with `.measure` calculations.
- If a `.measure` statement uses the result of previous `.meas` statement, then the calculation starts when the previous result is found. Until the previous result is found, it outputs zero.

13

Spectrum Analysis

Describes PrimeSim HSPICE implementation of spectrum analysis based on the Fourier transforms.

Spectrum analysis represents a time-domain signal within the frequency domain. It most commonly uses the Fourier transform. A Discrete Fourier Transform (DFT) uses sequences of time values to determine the frequency content of analog signals in circuit simulation.

PrimeSim HSPICE ships numerous examples for your use; see Listing of Demonstration Input Files in the *PrimeSim HSPICE User Guide: Demonstration Netlists* for paths to the demo files. See also Fourier Analysis Examples in the *PrimeSim HSPICE User Guide: Demonstration Netlists* for examples of the analyses discussed in this chapter.

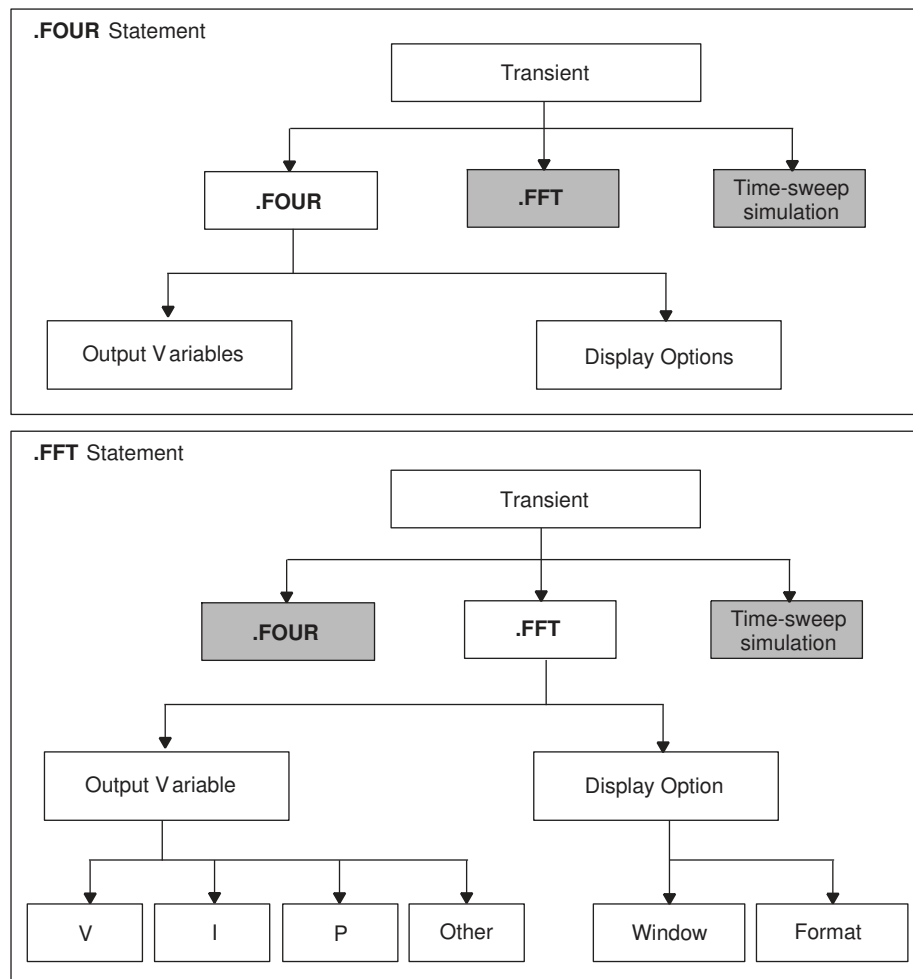
The following sections discuss these topics:

- [Spectrum Analysis \(Fourier Transform\)](#)
- [.FFT Analysis](#)
- [Examining the FFT Output](#)
- [AM Modulation](#)
- [Balanced Modulator and Demodulator](#)
- [Signal Detection Test Circuit](#)

Spectrum Analysis (Fourier Transform)

This section describes the Fourier and FFT Analysis flow for PrimeSim HSPICE.

Figure 52 Fourier and FFT Analysis



PrimeSim HSPICE provides two different Fourier analyses.

- **.FOUR** is the same as is available in SPICE 2G6: a standard, fixed-window analysis tool. The **.FOUR** statement performs a Fourier analysis, as part of the transient analysis.
- **.FFT** is a much more flexible Fourier analysis tool. Use it for analysis tasks that require more detail and precision.

The following sections discuss these topics:

- [Using the Fourier-Related Statements and Options](#)
- [Fourier Accuracy](#)
- [Fourier Equation](#)

Using the Fourier-Related Statements and Options

For syntax and examples, see the following commands and control options in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*:

- .FFT
- .FOUR
- .MEASURE FFT
- .OPTION FFT_ACCURATE
- .OPTION ACCURATE
- .OPTION FFTOUT

Fourier Accuracy

Fourier Accuracy is dependent on transient simulation accuracy. For best accuracy, set small values for .OPTION RMAX or .OPTION DELMAX. For maximum accuracy, set .OPTION DELMAX to $1 / (500 * \text{frequency})$. For circuits with very high resonance factors (high-Q circuits, such as crystal oscillators, tank circuits, and active filters), set DELMAX to less than

$1 / (500 * \text{frequency})$

where *frequency* refers to the fundamental frequency.

Fourier Equation

The total harmonic distortion is the square root of the sum of the squares, of the second through ninth normalized harmonic, times 100, expressed as a percent:

Equation 3

$$THD = \frac{1}{R1} \cdot \left(\sum_{m=2}^9 R_m^2 \right)^{\frac{1}{2}} \cdot 100\%$$

The following equation calculates the Fourier coefficients:

Equation 4

$$g(t) = \sum_{m=0}^9 C_m \cdot \cos(mt) + \sum_{m=0}^9 D_m \cdot \sin(mt)$$

The following equations calculate values for the preceding equation:

Equation 5

$$C_m = \frac{1}{\pi} \cdot \int_{-\pi}^{\pi} g(t) \cdot \cos(mt) \cdot dt$$

Equation 6

$$D_m = \frac{1}{\pi} \cdot \int_{-\pi}^{\pi} g(t) \cdot \sin(mt) \cdot dt$$

Equation 7

$$g(t) = \sum_{m=0}^9 C_m \cdot \cos(m \cdot t) + \sum_{m=0}^9 D_m \cdot \sin(m \cdot t)$$

The following equations approximate the C and D values:

Equation 8

$$C_m = \sum_{n=0}^{500} g(n \cdot \Delta t) \cdot \cos\left(\frac{2 \cdot \pi \cdot m \cdot n}{501}\right)$$

Equation 9

$$D_m = \sum_{n=0}^{500} g(n \cdot \Delta t) \cdot \sin\left(\frac{2 \cdot \pi \cdot m \cdot n}{501}\right)$$

The following equations calculate the magnitude and phase:

Equation 10

$$R_m = (C_m^2 + D_m^2)^{\frac{1}{2}}$$

Equation 11

$$\Phi_m = \arctan\left(\frac{C_m}{D_m}\right)$$

Example 1

The following is input-content for an .OP, .TRAN, or .FOUR analysis. This example uses demonstration netlist `four.sp`, which is available in the directory `$installdir/demo/hspice/apps`.

```
CMOS INVERTER
*
M1 2 1 0 0 NMOS W=20U L=5U
M2 2 1 3 3 PMOS W=40U L=5U
```

```
VDD 3 0 5
VIN 1 0 SIN 2.5 2.5 20MEG
*
.MODEL NMOS NMOS LEVEL=3 CGDO=0.2N CGSO=0.2N CGB0=2N
.MODEL PMOS PMOS LEVEL=3 CGDO=0.2N CGSO=0.2N CGB0=2N
.OP
.TRAN 1N 500N
.FOUR 20MEG V(2)
.PRINT TRAN V(2) V(1)
.END
```

Example 2

```
*****
cmos inverter
**** fourier analysis tnom = 25.000 temp = 25.000 ****
fourier components of transient response v(2)
dc component=2.430D+00
harmonic    frequency    fourier    normalized    phase    normalized
no          (hz)         component  component    (deg)    phase (deg)
1           20.0000x      3.0462    1.0000      176.5386    0.
2           40.0000x    115.7006m 37.9817m -106.2672 -282.8057
3           60.0000x   753.0446m 247.2061m 170.7288  -5.8098
4           80.0000x   77.8910m 25.5697m -125.9511 -302.4897
5          100.0000x  296.5549m 97.3517m 164.5430 -11.9956
6          120.0000x   50.0994m 16.4464m -148.1115 -324.6501
7          140.0000x  125.2127m 41.1043m 157.7399 -18.7987
8          160.0000x   25.6916m 8.4339m 172.9579  -3.5807
9          180.0000x   47.7347m 15.6701m 154.1858 -22.3528
total harmonic distortion= 27.3791 percent
```

.FFT Analysis

The `.FFT` statement uses the internal time point values. By default, `.FFT` uses a second-order interpolation to obtain waveform samples, based on the number of points that you specify.

You can use windowing functions to reduce the effects of waveform truncation on the spectral content. You can also use the `.FFT` command to specify:

- Output format
- Output frequency range
- Start and stop time point
- Fundamental frequency
- Window type
- Number of sampling time points

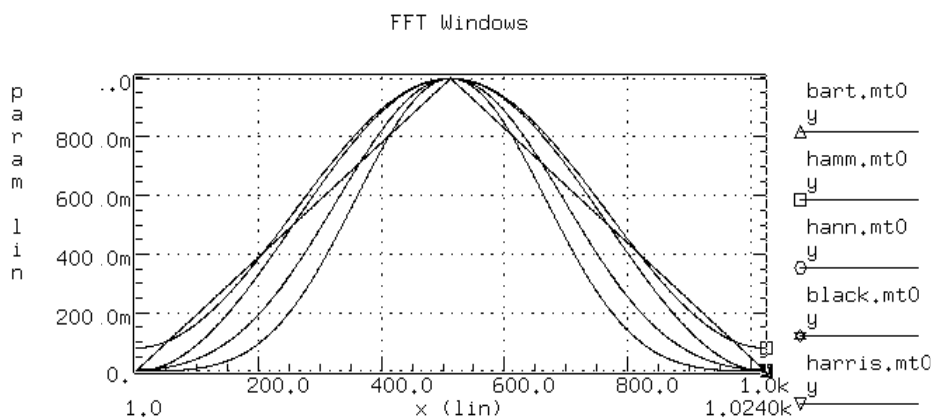
Using Windows in FFT Analysis

One problem with spectrum analysis in circuit simulators is that the duration of the signals is finite, although adjustable. Applying the FFT method to finite-duration sequences can produce inadequate results. This occurs because DFT assumes periodic extensions and causes spectral leakage.

The effect occurs when the finite duration of the signal does not result in a sequence that contains a whole number of periods. This is especially true when you use FFT to detect or estimate signals – that is, to detect weak signals in the presence of strong signals, or to resolve a cluster of equal-strength frequencies.

In FFT analysis, windows are frequency-weighting functions that PrimeSim HSPICE applies to the time-domain data, to reduce the spectral leakage associated with finite-duration time signals. Windows are smoothing functions, which peak in the middle frequencies, and decrease to zero at the edges. Windows reduce the effects of discontinuities, as a result of finite duration. [Figure 53](#) shows the windows available in PrimeSim HSPICE. [Table 52 on page 339](#) lists the common performance parameters, for FFT windows.

Figure 53 *FFT Windows*



The most important parameters in [Table 52](#) are:

- Highest side-lobe level (to reduce bias, the lower the better).
- Worst-case processing loss (to increase detectability, the lower the better).

Table 52 Window Weighting Characteristics in FFT Analysis

Window	Equation	Highest Side-Lobe (dB)	Side-Lobe Roll-Off (dB/octave)	3.0-dB Bandwidth (1.0/T)	Worst-Case Process Loss (dB)
Rectangular	$W(n)=1, 0 \leq n < NP^7$	-13	-6	0.89	3.92
Bartlett	$W(n)=2n/(NP-1), 0 \leq n \leq (NP/2)-1$ $W(n)=2-2n/(NP-1), NP/2 \leq n < NP$	-27	-12	1.28	3.07
Hanning	$W(n)=0.5-0.5[\cos(2\pi n/(NP-1))], 0 \leq n < NP$	-32	-18	1.44	3.18
Hamming	$W(n)=0.54-0.46[\cos(2\pi n/(NP-1))], 0 \leq n < NP$	-43	-6	1.30	3.10
Blackman	$W(n)=0.42323 - 0.49755[\cos(2\pi n/(NP-1))]$ $+ 0.07922\cos[4\pi n/(NP-1)], 0 \leq n < NP$	-58	-18	1.68	3.47
Blackman- Harris	$W(n)=0.35875 - 0.48829[\cos(2\pi n/(NP-1))]$ $+ 0.14128[\cos(4\pi n/(NP-1))]$ $- 0.01168[\cos(6\pi n/(NP-1))], 0 \leq n < NP$	-92	-6	1.90	3.85
Gaussian	$W(n)=\exp[-0.5a^2(NP/2-1-n)^2/(NP)^2],$ $0 \leq n \leq (NP/2)-1$	-42	-6	1.33	3.14
a=2.5		-55	-6	1.55	3.40
a=3.0	$W(n)=\exp[-0.5a^2(n-NP/2)^2/(NP)^2],$ $NP/2 \leq n < NP$	-69	-6	1.79	3.73
a=3.5					
Kaiser- Bessel	$W(n)=I_0(x_2)/I_0(x_1)$	-46	-6	1.43	3.20
a=2.0	$x_1=pa$	-57	-6	1.57	3.38
a=2.5	$x_2=x_1*\sqrt{1-(2(NP/2-1-n)/NP)^2},$	-69	-6	1.71	3.56
a=3.0	$0 \leq n \leq (NP/2)-1$	-82	-6	0.89	3.74
a=3.5	$x_2=x_1*\sqrt{1-(2(n-NP/2)/NP)^2},$ $NP/2 \leq n < NP$ I_0 is the zero-order modified Bessel function				

Some compromise usually is necessary, to find a suitable window-filtering for each application. As a rule, window performance improves with functions of higher complexity (those listed lower in the table).

- The Kaiser window has an `ALFA` parameter, which adjusts the compromise between different figures of merit for the window.
- The simple rectangular window produces a simple bandpass truncation, but results in the classical Gibbs phenomenon.

7. *NP* is the number of points used for the FFT analysis.

- The Bartlett or triangular window has good processing loss, and good side-lobe roll-off, but lacks sufficient bias reduction.
- The Hanning, Hamming, Blackman, and Blackman-Harris windows use progressively more complicated cosine functions. These functions provide smooth truncation, and a wide range of side-lobe level and processing loss.
- The last two windows in the table, Kaiser-Bessel, are parameterized-windows. Use these windows to adjust the side-lobe level, the 3 dB bandwidth, and the processing loss.

Figure 54 and Figure 55 show the characteristics of two typical windows.

Figure 54 *Bartlett Window Characteristics*

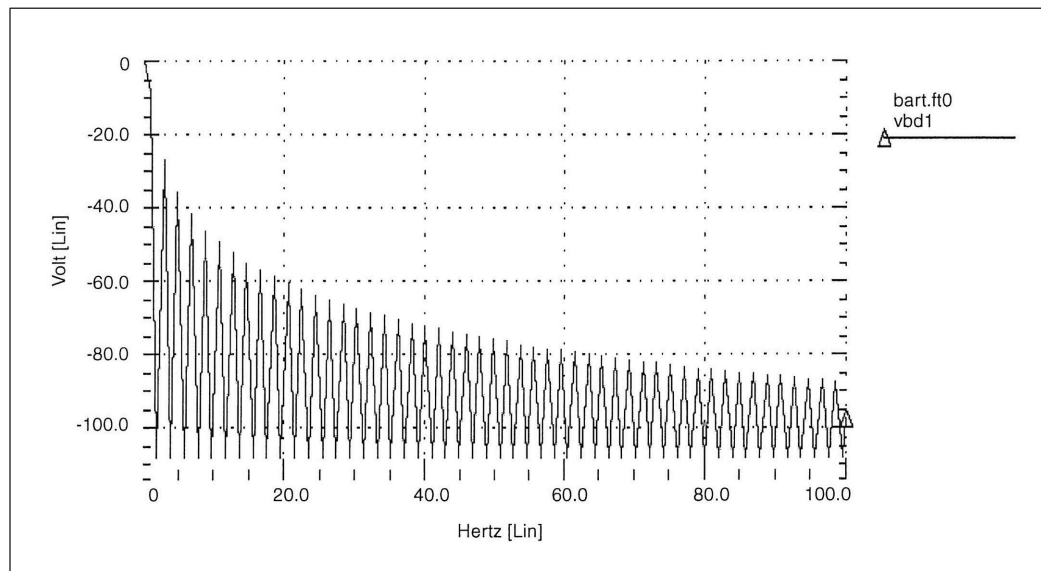
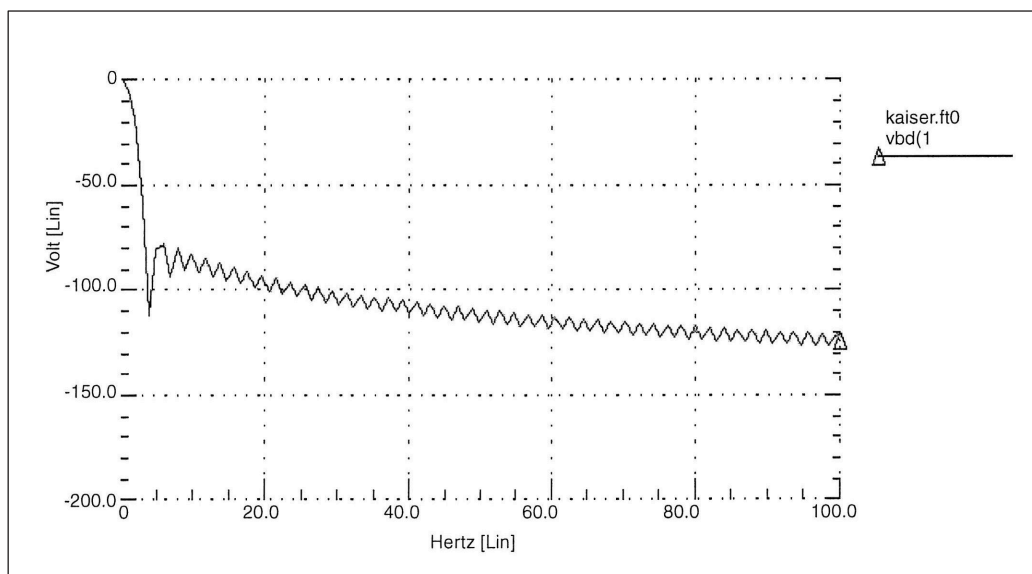


Figure 55 Kaiser-Bessel Window Characteristics, ALFA=3.0



Examining the FFT Output

PrimeSim HSPICE prints FFT analysis results in tabular format, in the `.lis` file. These results use parameters in the `.FFT` statement. PrimeSim HSPICE prints normalized magnitude values, unless you specify `FORMAT=UNORM`, in which case it prints unnormalized magnitude values. The number of printed frequencies is half the number of points (NP) specified in the `.FFT` statement.

- If you use `FMIN` to specify a minimum frequency, or `FMAX` to specify a maximum frequency, PrimeSim HSPICE prints only the specified frequency range.
- If you use `FREQ` to specify a frequency, PrimeSim HSPICE outputs harmonics of this frequency.

PrimeSim HSPICE generates a `.ft#` file and the listing file for each FFT output variable that contains data to display in FFT analysis waveforms (such as in WaveView). You can view the magnitude in dB, and the phase in degrees.

In the following sample FFT analysis `.lis` file output, the header defines parameters in the FFT analysis.

```
***** Sample FFT output extracted from the .lis file
fft test ... sine
***** fft analysis   tnom= 25.000   temp= 25.000
***** fft components of transient response v(1)
Window:   Rectangular
First Harmonic:   1.0000k
```

```
Start Freq: 1.0000k
Stop Freq: 10.0000k
dc component: mag(db)= -1.132D+02 mag= 2.191D-06 phase= 1.800D+02
```

frequency index	frequency (hz)	fft_mag (db)	fft_mag (deg)	fft_phase
2	1.0000k	0.	1.0000	-3.8093m
4	2.0000k	-125.5914	525.3264n	-5.2406
6	3.0000k	-106.3740	4.8007u	-98.5448
8	4.0000k	-113.5753	2.0952u	-5.5966
10	5.0000k	-112.6689	2.3257u	-103.4041
12	6.0000k	-118.3365	1.2111u	167.2651
14	7.0000k	-109.8888	3.2030u	-100.7151
16	8.0000k	-117.4413	1.3426u	161.1255
18	9.0000k	-97.5293	13.2903u	70.0515
20	10.0000k	-114.3693	1.9122u	-12.5492

The preceding example specifies a frequency of 1 kHz, and a THD up to 10 kHz, which corresponds to the first ten harmonics.

The highest FFT output frequency might not match the specified FMAX, due to adjustments that PrimeSim HSPICE makes.

Table 53 describes the output of the FFT analysis.

Table 53 .FFT Output Description

Column Heading	Description
Frequency Index	Runs from 1 to NP/2, or the corresponding index for FMIN and FMAX. The DC component, corresponding to the 0 index, displays independently.
Frequency	The actual frequency, associated with the index.
fft_mag (dB), fft_mag	The first FFT magnitude column is in dB. The second FFT magnitude column is in units of the output variable. PrimeSim HSPICE normalizes the magnitude, unless you specify UNORM format.
fft_phase	Associated phase, in degrees.

Notes:

- Use the following formula as a guideline to specify a frequency range for FFT output:

$$\text{frequency increment} = 1.0/(\text{STOP} - \text{START})$$

Each frequency index is a multiple of this increment. To obtain a finer frequency resolution, maximize the duration of the time window or specify more time points (larger NP).

- FMIN and FMAX have no effect on the .ft0, .ft1, ..., .ftn files.

- If you specify `FFTOUT` in an `.OPTION` statement, PrimeSim HSPICE can print results of `THD`, `SNR`, `SFDR`, `SNDR`, and `ENOB` and then sort the harmonics of fundamental by magnitude size.

Assume that $\text{freq}=f_0$ is the fundamental frequency.

- `THD` is total harmonic distortion, which Equation 12 computes as:

Equation 12

$$THD = \frac{\sqrt{\text{sum}(\text{mag}(n \cdot f) \cdot \text{mag}(n \cdot f))}}{\text{mag}(f_0)}$$

- `SNR` is the ratio of signal to noise, which following formula computes as:

$$SNR = 10 \log \left[\frac{\text{mag}(f_0) \cdot \text{mag}(f_0)}{\text{sum}(\text{mag}(f) \cdot \text{mag}(f))} \right]$$

The `f` loop over the whole spectrum except f_0 and its harmonics. If you specify `FMIN/FMAX`, `f` loops over the spectrum between `FMIN` and `FMAX` except f_0 and its harmonics, as well as all the frequency components above `FMAX`.

- `SFDR` is the spurious-free dynamic range and is the distance from the fundamental input signal to the highest spur (in dB). `SFDR` involves both magnitude distance and frequency separation.
- `SNDR` is the signal to noise and distortion ratio, which is the level of the fundamental divided by the square root of the sum of squares of all frequency components other than the fundamental frequency. The following formula computes `SNDR`:

Equation 13

$$SNDR = 10 \log \left[\frac{\text{mag}(f_0) \cdot \text{mag}(f_0)}{\text{sum}(\text{mag}(f) \cdot \text{mag}(f))} \right]$$

- `ENOB` is the effective number of bits; the following formula computes `ENOB`:

$$ENOB = \frac{(SNDR - 1.76 \text{ dB})}{6.02}$$

The `f` loops over the whole spectrum except f_0 . If you specify `FMIN/FMAX`, `f` loops over the spectrum between `FMIN` and `FMAX` except f_0 ; `f` also loops all the frequency components above `FMAX`.

Measuring FFT Output Information

You can measure all of the FFT output information in the previous section by using the following syntax:

Measuring frequency component at certain frequency point:

```
.MEASURE FFT result  
+ Find [vm|vp|vr|vi|vdb|im|ip|ir|ii|idb] (signal) AT=freq
```

Measuring THD of a signal spectrum up to a certain harmonic:

```
.MEASURE FFT result THD signal_name [NBHARM=num]
```

Default: NBHARM=maximum harmonic in FFT result.

Measuring SNR/SNDR/ENOB of a signal up to a certain frequency point:

```
.MEASURE FFT result [SNR|SNDR|ENOB] signal_name  
+ [NBHARM=num|MAXFREQ=val] [BINSIZ=num]
```

Default: NBHARM=maximum harmonic in FFT result. PrimeSim HSPICE considers all the frequency components above NBHARM to be noise.

MAXFREQ=maximum frequency in FFT result. PrimeSim HSPICE considers all the frequency components above MAXFREQ to be noise. BINSIZ=num filters out noise components. PrimeSim HSPICE calculates the noise component above index “fundamental_freq_idx+BINSIZ+1”. PrimeSim HSPICE filters out the noise components within the bin. The default value of BINSIZ is 0 in PrimeSim HSPICE. When you set the window function in FFT analysis, you can use BINSIZ to filter out the noise component caused by window function.

Measuring SFDR of a signal from minfreq to maxfreq:

```
.MEASURE FFT result SFDR signal_name [MAXFREQ=val] [MINFREQ=val]
```

Default: MINFREQ=0.

MAXFREQ=Maximum frequency in FFT result.

Syntax to perform FFT measurements from previous simulation results:

```
hspice -i *.tr0 -meas measure_file
```

For more information, see .MEASURE FFT in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

AM Modulation

This example uses demonstration netlist `exam1.sp`, which is available in directory `$install_dir/demo/hspice/fft`. The example shows a 1 kHz carrier (FC), which a 100 Hz signal (FM) modulates.

AM Modulation

```
.OPTION post
.PARAM sa=10 offset=1 fm=100 fc=1k td=1m
VX 1 0 AM(sa offset fm fc td)
Rx 1 0 1

.TRAN 2u 50m
.FFT V(1) START=10m STOP=40m FMIN=833 FMAX=1.16K
.END
```

The following equation describes the voltage at node 1, which is an AM signal:

Equation 14

$$v(1) = sa \cdot (offset + \sin(\omega_m(Time-td))) \cdot \sin(\omega_c(Time-td))$$

You can expand the preceding equation, as follows.

Equation 15

$$v(1) = (sa \cdot offset \cdot \sin(\omega_c(Time-td)) + 0.5 \cdot sa \cdot \cos((\omega_c - \omega_m)(Time-td)))$$

Equation 16

$$-0.5 \cdot sa \cdot \cos((\omega_c + \omega_m)(Time-td))$$

where

Equation 17

$$\omega_c = 2\pi f_c$$

$$\omega_f = 2\pi f_m$$

The preceding equations indicate that $v(1)$ is a summation of three signals, with the following frequency:

$$f_c, (f_c - f_m), \text{ and } (f_c + f_m)$$

This is the carrier frequency and the two sidebands.

See also Behavioral Amplitude Modulator in the *PrimeSim HSPICE User Guide: Advanced Analog Simulation and Analysis*.

Graphical Output

Figure 56 and Figure 57 on page 347 display the results.

- Figure 56 shows the time-domain curve for node 1.
- Figure 57 shows the frequency-domain components, for the magnitude of node 1.

The carrier frequency is 1 kHz, with two sideband frequencies 100 Hz apart.

The third, fifth, and seventh harmonics are more than 100 dB below the fundamental, indicating excellent numerical accuracy. The time-domain data contains an integer multiple of the period, so you do not need to use windowing.

Figure 56 AM Modulation

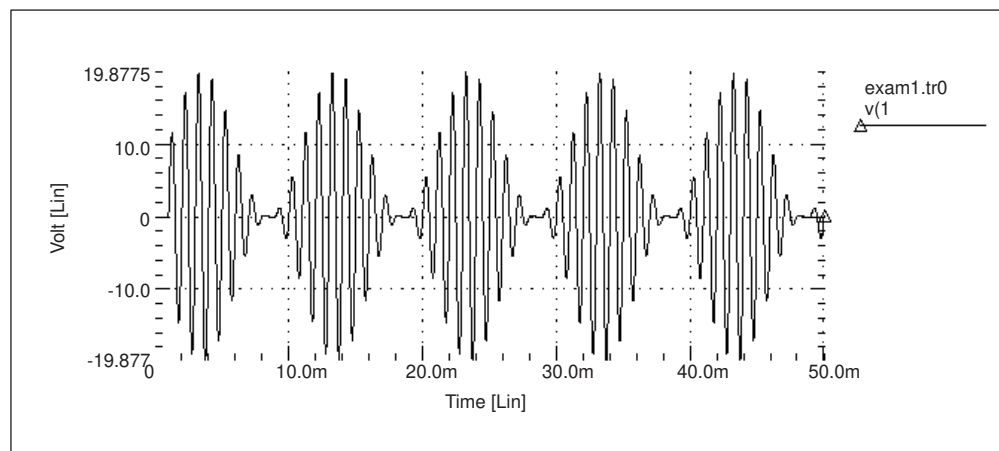
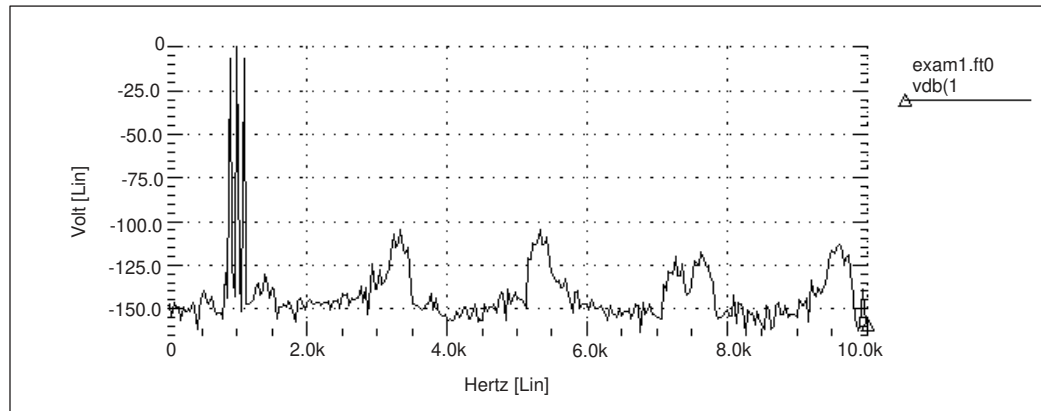


Figure 57 AM Modulation Spectrum



Balanced Modulator and Demodulator

Demodulation, or detection, recovers a modulating signal from the modulated output voltage. The netlist, in the Input and Output Listing section that follows, shows this process. This example uses PrimeSim HSPICE behavioral models, and FFT analysis, to confirm the validity of the process, in the frequency domain.

The low-pass filter uses the Laplace element. This filter introduces delay in the output signal, which causes spectral leakage if you do not use FFT windowing. However, if you use window-weighting to perform FFT, you eliminate most of the spectral leakage. The THD of the two outputs, shown in “Input and Output Listing,” verifies this. PrimeSim HSPICE expects a 1 kHz output signal, so specify a 1 kHz frequency in the `.FFT` command. Also specify `FMAX`, to provide the first few harmonics in the output listing, for THD calculations.

Input and Output Listing

The sample input and output listing files are in the following directory:

```
$installdir/demo/hspice/fft/balance.sp
```

Figure 58 through Figure 66 show the signals, and their spectral content. The modulated signal contains only the sum, and the difference of the carrier frequency and the modulating signal (1 kHz and 10 kHz). At the receiver end, this example recovers the carrier frequency, in the demodulated signal. This example also shows a 10 kHz frequency shift, in the above signals (to 19 kHz and 21 kHz).

A low-pass filter uses a second-order Butterworth filter, to extract the carrier frequency. A Harris window significantly improves the noise floor in the filtered output spectrum, and reduces `THD` in the output listing (from 9.23 percent to 0.047 percent). However, this

example needs a filter with a steeper transition region, and better delay characteristics, to suppress modulating frequencies below -60 dB. [Figure 61](#) is a normalized filtered output signal waveform.

Figure 58 *Modulating and Modulated Signals*

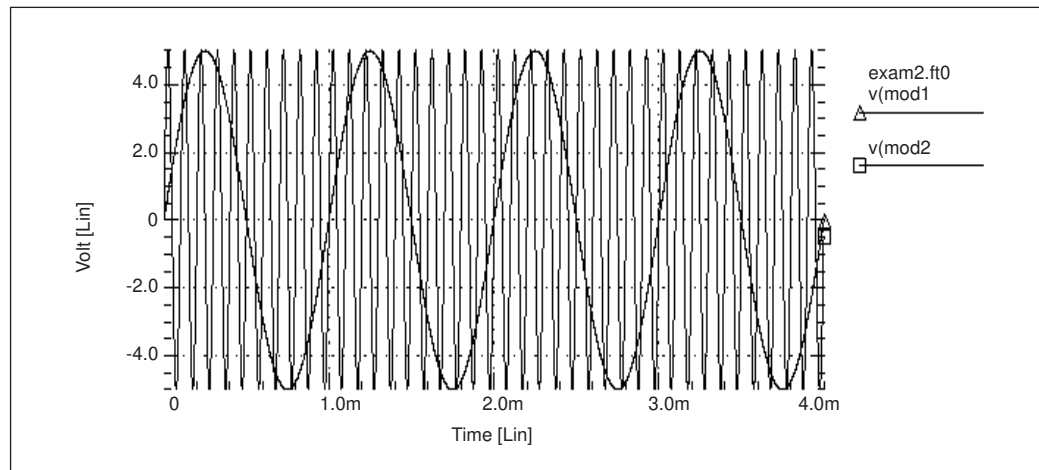


Figure 59 *Modulated Signal*

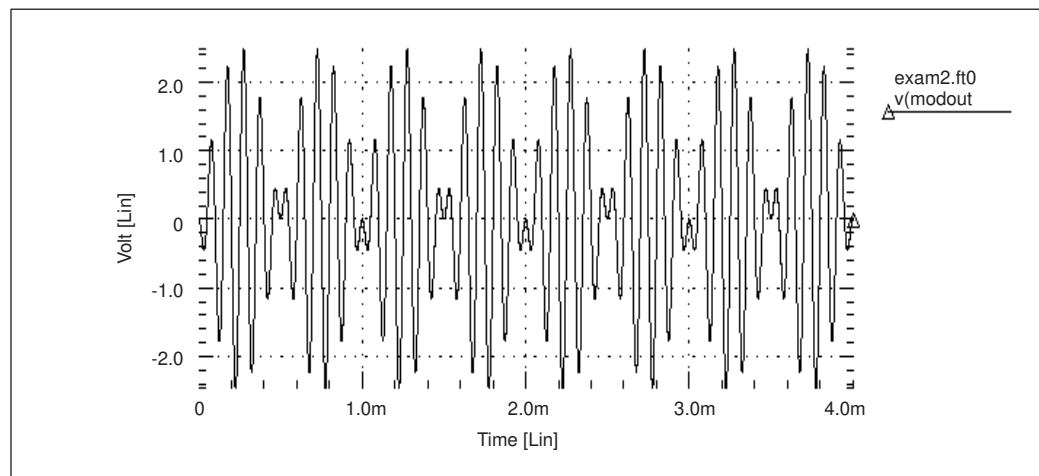


Figure 60 Demodulated Signal

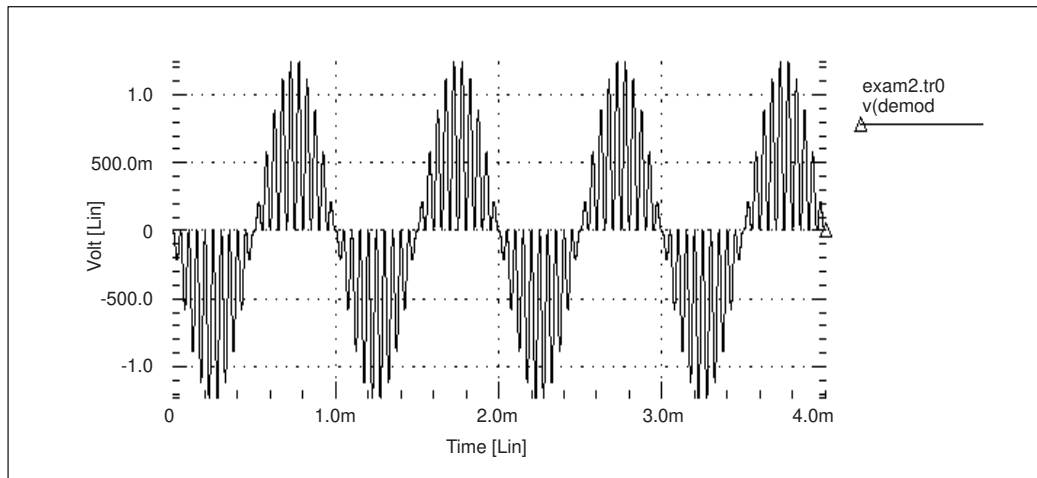


Figure 61 Filtered Output Signal

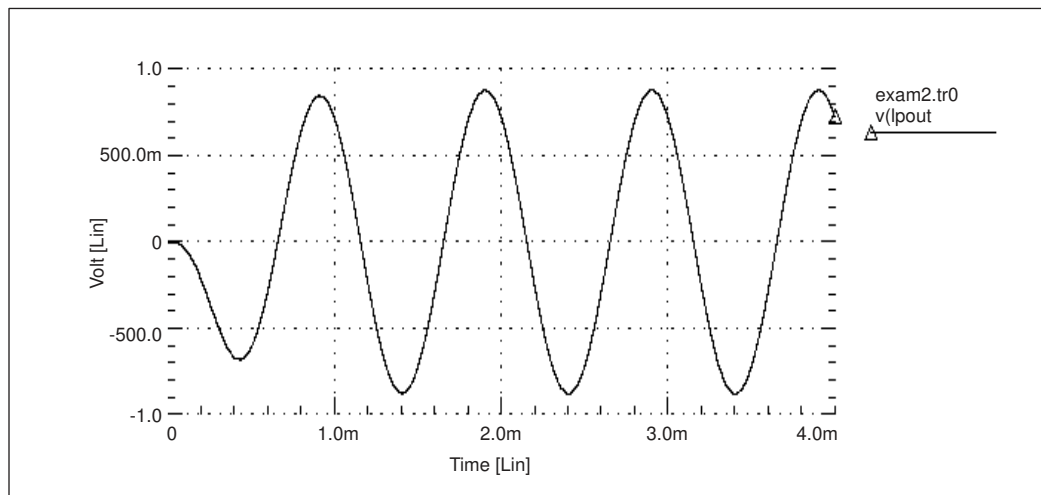


Figure 62 Modulating and Modulated Signal Spectrum

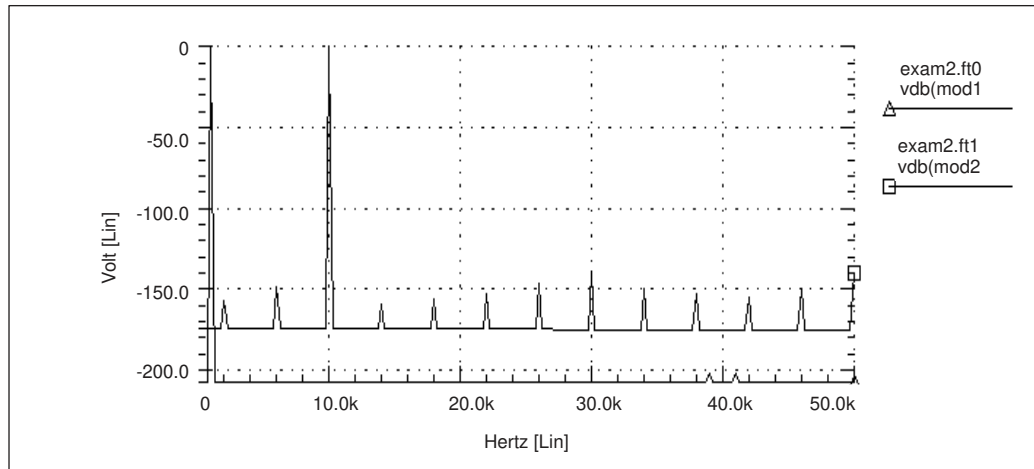


Figure 63 Modulated Signal Spectrum

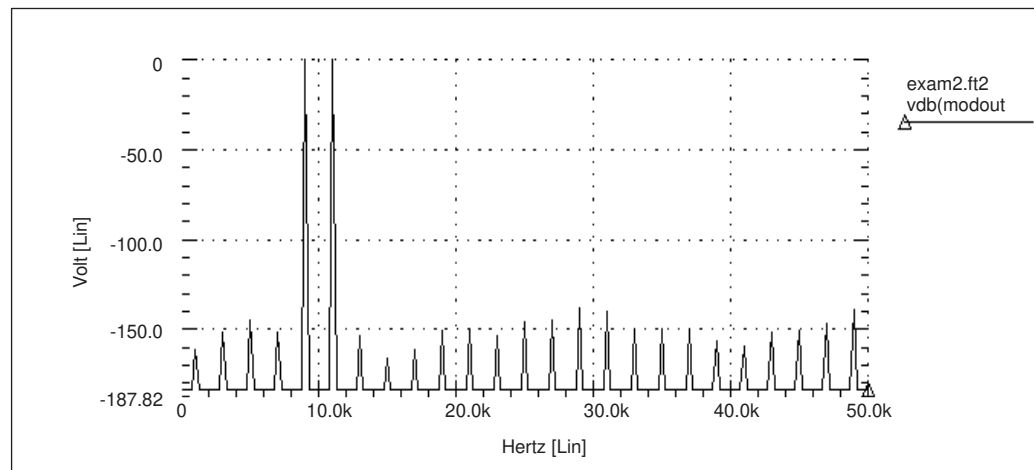


Figure 64 Demodulated Signal Spectrum

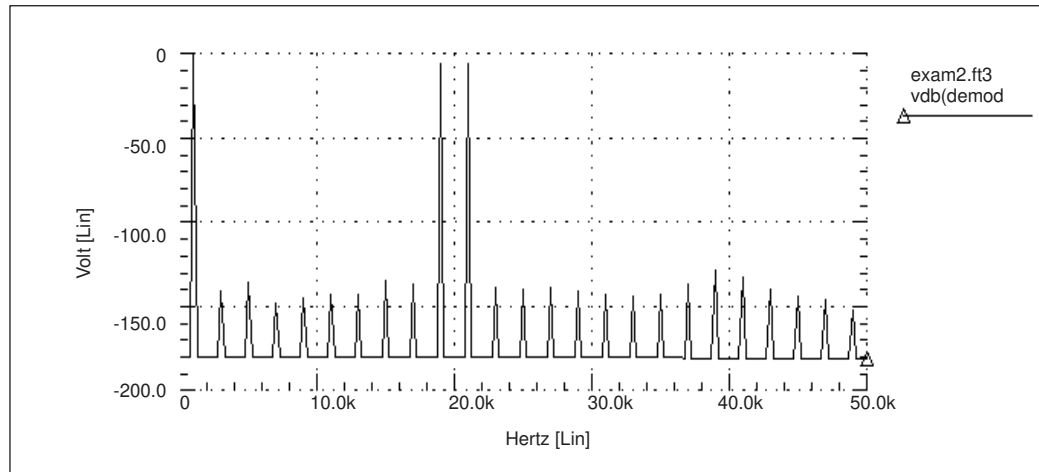


Figure 65 Filtered Output Signal (no window)

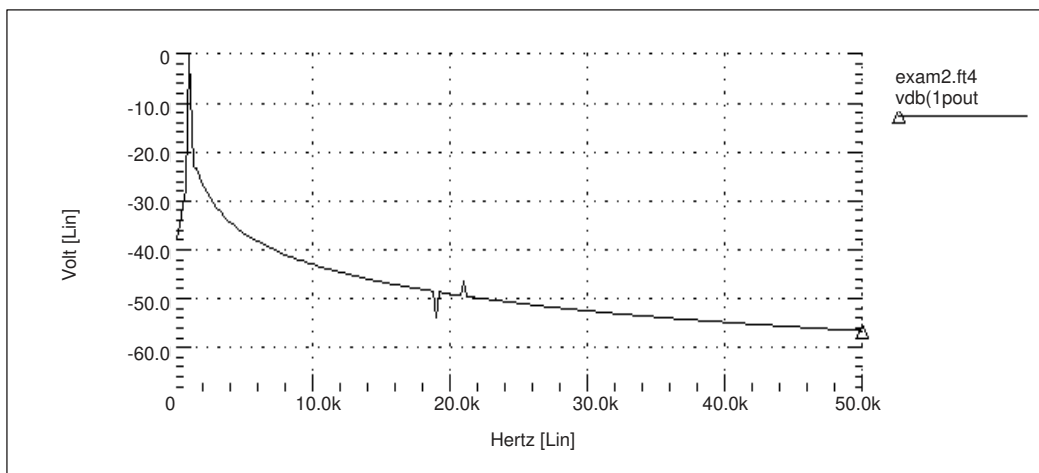
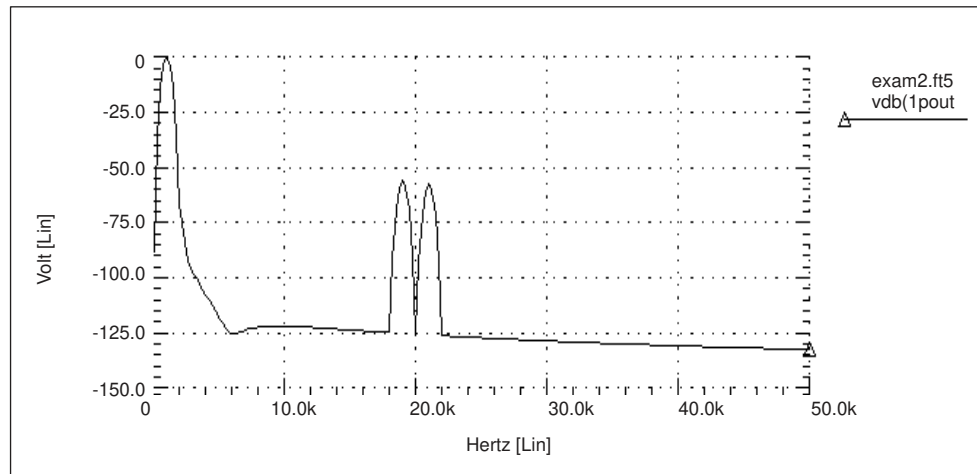


Figure 66 Filtered Output Signal (Blackman-Harris window)



Signal Detection Test Circuit

This example is a high-frequency mixer test circuit. It illustrates the effect of using a window to detect a weak signal, in the presence of a strong signal that is at a nearby frequency. This example adds two high-frequency signals, with a 40 dB separation (amplitudes are 1.0 and 0.01).

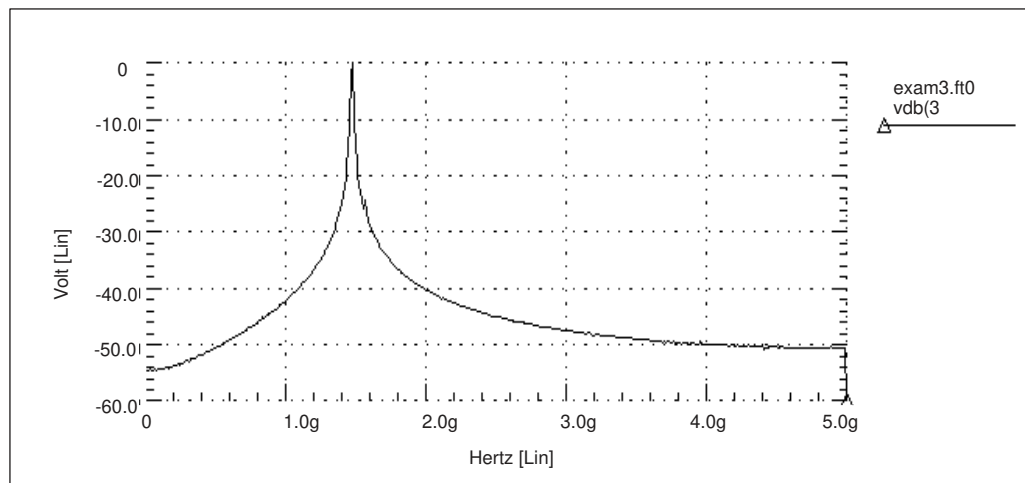
Input Listing

The sample input listing file is in the following directory: `$installdir/demo/hspice/fft/exam3.sp`

Output

Figure 67 shows the rectangular window. Compare this with the spectra of the output for all FFT window types, as shown in Figure 68 through Figure 74. Without windowing, PrimeSim HSPICE does not detect the weak signal because of spectral leakage.

Figure 67 Mixer Output Spectrum, Rectangular Window



- In the Bartlett window (Figure 68), the noise floor increases dramatically, compared to the rectangular window (from -55, to more than -90 dB).
- The cosine windows (Hanning, Hamming, Blackman, and Blackman-Harris) all produce better results than the Bartlett window. However, the Blackman-Harris window provides the highest degree of separation for the two tones, and the lowest noise floor.
- The final two windows (Figure 73 and Figure 74) use the `ALFA=3.0` parameter, which is the default value in PrimeSim HSPICE. These two windows also produce acceptable results, especially the Kaiser-Bessel window, which sharply separates the two tones, and has a noise floor of almost -100-dB.

PrimeSim HSPICE processes high frequencies, and this example demonstrates the numerical stability and accuracy of the FFT spectrum analysis algorithms.

Figure 68 Mixer Output Spectrum, Bartlett Window

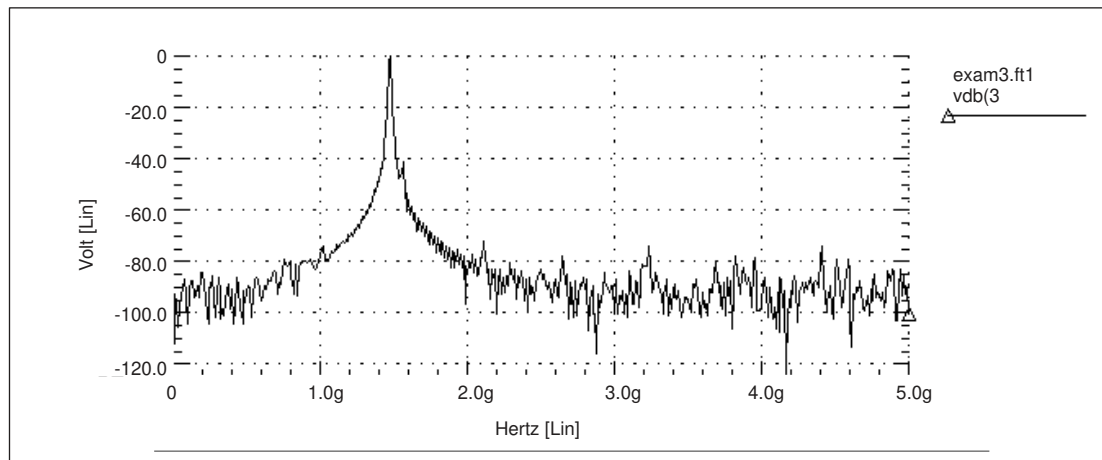


Figure 69 Mixer Output Spectrum, Hanning Window

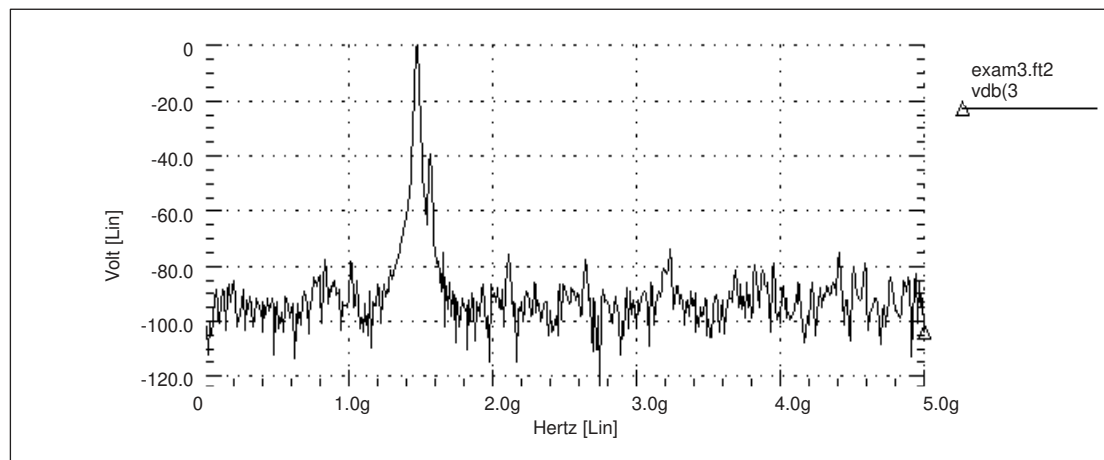


Figure 70 Mixer Output Spectrum, Hamming Window

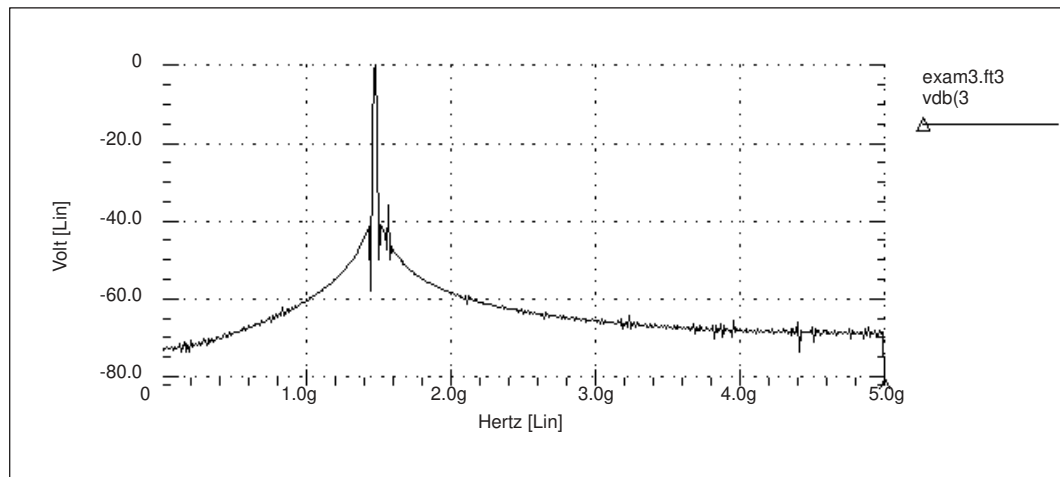


Figure 71 Mixer Output Spectrum, Blackman Window

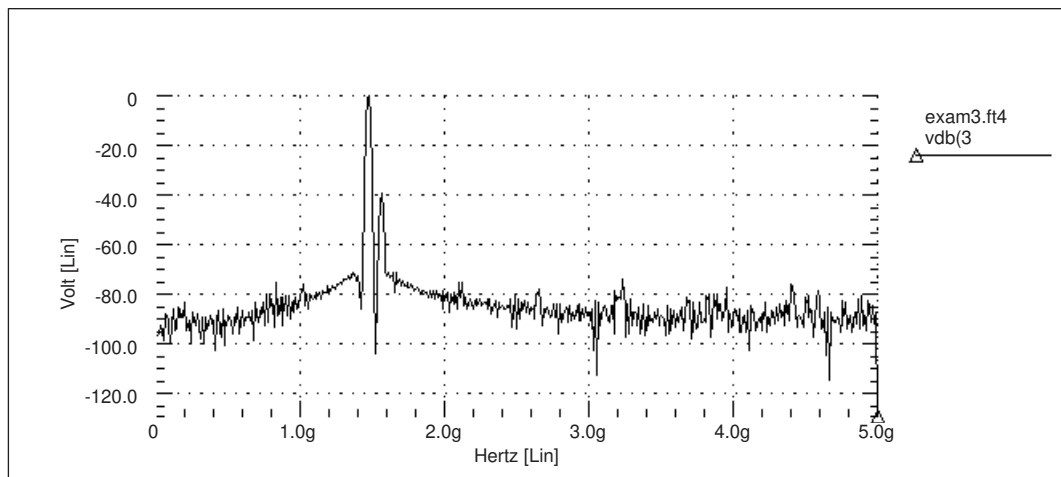


Figure 72 Mixer Output Spectrum, Blackman-Harris Window

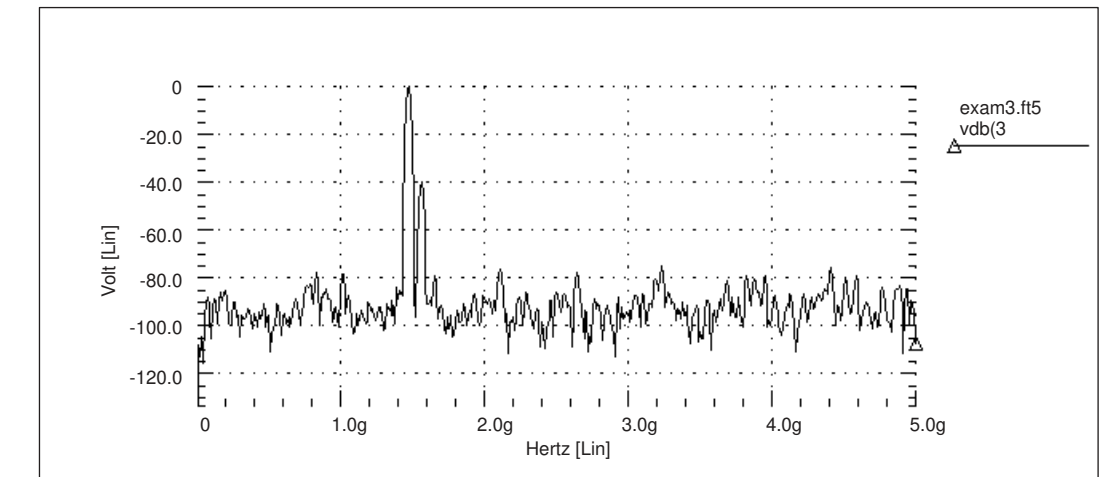


Figure 73 Mixer Output Spectrum, Gaussian Window

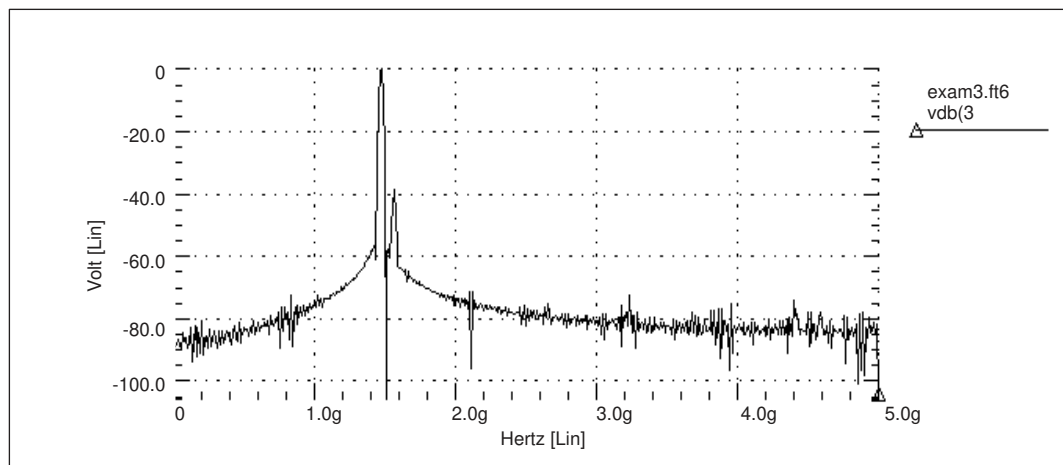
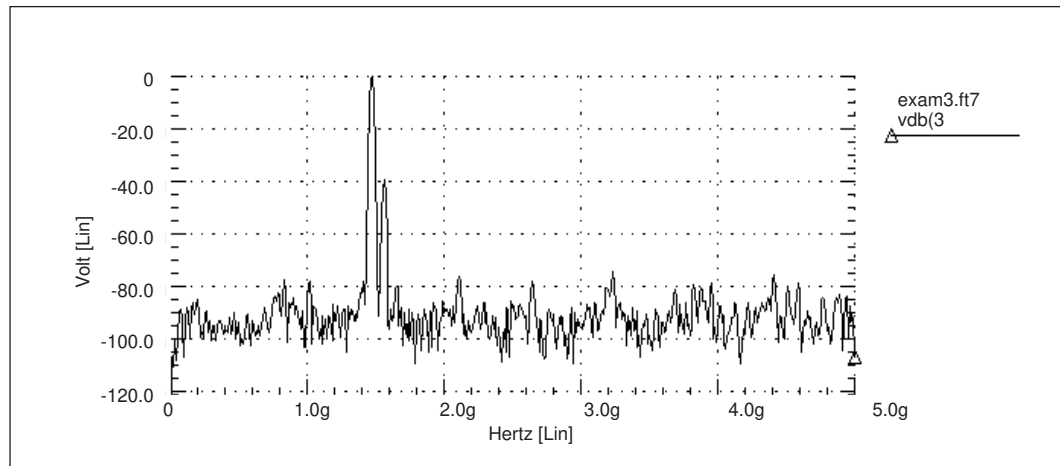


Figure 74 Mixer Output Spectrum, Kaiser-Bessel Window



14

Pole-Zero Analysis

Describes how to use pole/zero analysis in PrimeSim HSPICE.

PrimeSim HSPICE ships numerous examples for your use; see Listing of Demonstration Input Files in the *PrimeSim HSPICE User Guide: Demonstration Netlists* for paths to the demo files. See also Filters Examples in the *PrimeSim HSPICE User Guide: Demonstration Netlists* for examples of the analyses discussed in this chapter.

You can use pole/zero analysis in PrimeSim HSPICE to study the behavior of linear, time-invariant networks. You can apply the results to the design of analog circuits, such as amplifiers and filters. Use pole/zero analysis to determine the stability of a design, or to calculate the poles and zeroes to specify in a `POLE` function (see [Using Pole-Zero Analysis on page 359](#)).

Pole/zero analysis uses the `.PZ` (Pole/Zero) statement, instead of pole/zero (`POLE` function) and Laplace (`LAPLACE` function) transfer function modeling. See [Using Pole-Zero Analysis on page 359](#) for discussion of this technology.

The following sections present these topics:

- [Overview of Pole-Zero Analysis](#)
- [Using Pole-Zero Analysis](#)
- [Pole/Zero Analysis Examples](#)
- [References](#)

Overview of Pole-Zero Analysis

In pole/zero analysis, a network transfer function describes a network. For any linear time-invariant network, you can use this general form to write the function:

Equation 18

$$H(s) = \frac{N(s)}{D(s)} = \frac{a_0 s^m + a_1 \cdot s^{(m-1)} + \dots + a_m}{b_0 s^n + b_1 \cdot s^{(n-1)} + \dots + b_n}$$

In the factorized form, the general function is:

Equation 19

$$H(s) = \frac{a_0}{b_0} \cdot \frac{(s+z_1)(s+z_2)\dots(s+z_i)\dots(s+z_m)}{(s+p_1)(s+p_2)\dots(s+p_j)\dots(s+p_m)}$$

- The roots of the numerator $N(s)$ (that is, z_i) are the zeros of the network function.
- The roots of the denominator $D(s)$ (that is, p_j) are the poles of the network function.
- s is a complex frequency.

The dynamic behavior of the network depends on the location of the poles and zeros, on the network function curve (complex plane). The (real) poles are the natural frequencies of the network. You can graphically deduce the magnitude and phase curve of most network functions from the location of its poles and zeros (reference 2).

[References on page 373](#) lists a variety of source materials that address:

- Transfer functions of physical systems.
- Design of systems and physical modeling.
- Interconnect transfer function modeling.

Using Pole-Zero Analysis

PrimeSim HSPICE advanced analog analyses uses the exact matrix approach and the Muller method, while PrimeSim HSPICE uses only the Muller method to calculate the roots of the $N(s)$ and $D(s)$ polynomials.

The following sections discuss these topics:

- [Matrix Approach](#)
- [Muller Method](#)
- [How PrimeSim HSPICE Calculates Poles and Zeros](#)

Matrix Approach

The matrix approach uses the singular-value matrix decomposition algorithm. It applies primarily to a network that has no frequency-dependent elements. In this case, PrimeSim HSPICE advanced analog analyses writes the $D(s)$ function as the determinant of the network matrices, $D(s) = \det(G + sC)$, where G is the frequency-independent conductance matrix and C is the capacitance matrix. The poles can be the *eigen* values of the matrix equation $(G + sC)X = 0$, where X is the *eigen* vector.

Similarly, following Cramer's rule, the roots of the $N(s)$ function can also be the *eigen* values of a matrix.

Muller Method

You can apply the Muller method if the network contains frequency-dependent elements (such as S- or W-elements).

The Muller method approximates the polynomial, using a quadratic equation that fits through three points in the vicinity of a root. To obtain successive iterations toward a particular root, PrimeSim HSPICE finds the nearer root of a quadratic, whose curve passes through the last three points.

Selection of the three initial points affects both the convergence of the process, and the accuracy of the roots obtained:

1. If the poles or zeros occupy a wide frequency range, then choose (X0R, X0I) close to the origin, to find poles or zeros at the zero frequency first.
2. Find the remaining poles or zeros, in increasing order.

The (X1R, X1I) and (X2R, X2I) values can be orders of magnitude larger than (X0R, X0I). If any poles or zeros occur at high frequencies, adjust X1I and X2I accordingly.

Pole/zero analysis results use the circuit's DC operating point, so the operating point solution must be accurate. Use the `.NODESET` statement (not `.IC`) for initialization, to avoid DC convergence problems.

For the syntax, see `.PZ` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

How PrimeSim HSPICE Calculates Poles and Zeros

PrimeSim HSPICE calculates poles and zeros independently from the denominator and numerator polynomials of the transfer function respectively. It is possible that there exist common multipliers, such as the pole and zero appearing at exactly the same location. In this case, the pole and zero factor out. PrimeSim HSPICE only presents the roots of the two polynomials without doing the factorization.

For example, if you use the netlist `fkerwin.sp` from the demo folder, from the transfer function there should be two poles and zeros:

```
*      = (s**2 + 2) / (s**2 + 0.1*s + 1)
*      poles --- (-0.05004,+0.9987) , (-0.05004,-0.9987)
*      zeros --- ( 0.0      ,+1.4142) , ( 0.0      ,-1.4142)
```

But PrimeSim HSPICE reports three poles and zeros:

```
poles (rad/sec)                poles (hertz)
*****
real      imag      real      imagzeros (rad/sec)
      zeros (hertz)
-50.0394m    998.7214m    -7.9640m    158.9515m
-50.0394m   -998.7214m    -7.9640m   -158.9515m
*****
real      imag      real      imag
0.        -1.4142    0.        -225.0812m
0.         1.4142    0.         225.0812m
-1.4142     0.      -225.0812m    0.
-1.4142     0.      -225.0812m    0.
```

The transfer function cancels the common pole and zero at -1.4142 0. Since PrimeSim HSPICE solves the denominator and numerator separately, the results reports these separately.

Pole/Zero Analysis Examples

The following are examples of different types of pole/zero analysis.

- [Example 1 – Low-Pass Filter](#)
- [Example 2 – Kerwin's Circuit](#)
- [Example 3 – High-Pass Butterworth Filter](#)
- [Example 4 – CMOS Differential Amplifier](#)
- [Example 5 – Simple Amplifier](#)
- [Example 6 – Active Low-Pass Filter](#)

Example 1 – Low-Pass Filter

This example uses PrimeSim HSPICE demonstration netlist flp5th.sp, which is available in directory \$installdir/demo/hspice/filters:

```
file:flp5th.sp lowpass 5th order filter
*****
* reference: gabor c. temes and sanjit k. mitra, "modern filter theory
* and design", j. wiley, 1973, page 74.
* t = v(3) / iin
* =0.113*(s**2 + 1.6543)*(s**2 + 0.2632) /
* (s**5 + 0.9206*s**4 + 1.26123*s**3 + 0.74556*s**2
* + 0.2705*s + 0.09836)
*****
* pole zero, ac(.001hz-10hz) analysis
```

```
*
.options post
.pz v(3) iin
.ac dec 100 .001hz 10hz
.probe ac vdb(3) vp(3)
*
iin 1 0 1.00 ac 1
r1 1 0 1.0
c3 1 0 1.52
c4 2 0 1.50
c5 3 0 0.83
c1 1 2 0.93
l1 1 2 0.65
c2 2 3 3.80
l2 2 3 1.00
r2 3 0 1.0
.end
```

The following is an equivalent example in PrimeSim HSPICE advanced analog analyses:

```
* HSPICE advanced analog analysis example:
5TH-ORDER LOW_PASS FILTER
*
.OPTION POST
.PZ I(R2) IN
.AC DEC 100 .001HZ 10HZ
IN 0 1 1.00 AC 1
R1 1 0 1.0
C3 1 0 1.52
C4 2 0 1.50
C5 3 0 0.83
C1 1 2 0.93
L1 1 2 0.65
C2 2 3 3.80
L2 2 3 1.00
R2 3 0 1.00
.END
```

Figure 75 Low-Pass Prototype Filter

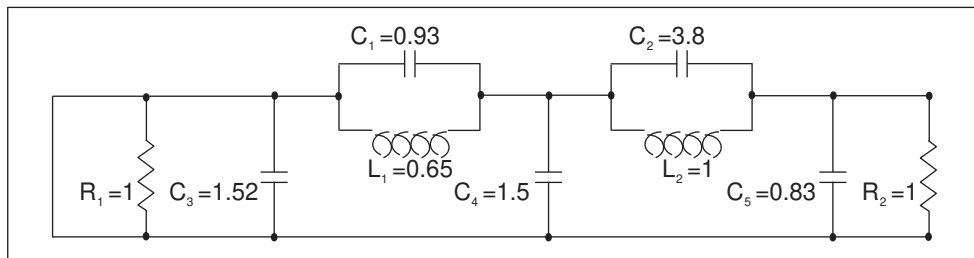
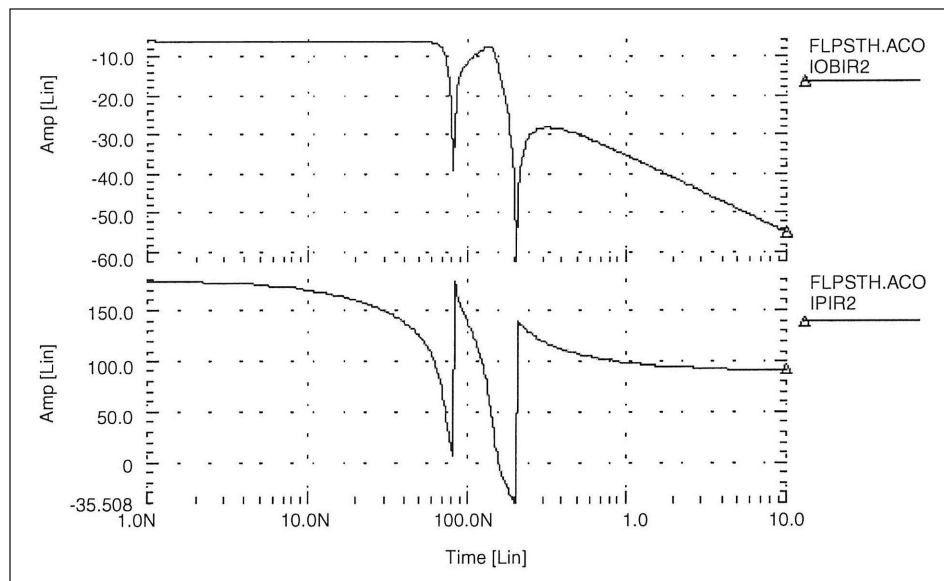


Table 54 shows the magnitude and phase variation of the current output, resulting from AC analysis. These results are consistent with pole/zero analysis. The pole/zero unit is radians per second, or hertz. The X-axis unit, in the plot, is in hertz.

Table 54 Pole/Zero Analysis Results for Low-Pass Filter

Poles (rad/sec)		Poles (Hertz)	
Real	Imaginary	Real	Imaginary
-6.948473e-02	-4.671778e-01	-1.105884e-02	-7.435365e-02
-6.948473e-02	4.671778e-01	-1.105884e-02	7.435365e-02
-1.182742e-01	-8.914907e-01	-1.882392e-02	-1.418852e-01
-1.182742e-01	8.914907e-01	-1.882392e-02	1.418852e-01
-5.450890e-01	0.000000e+00	-8.675361e-02	0.000000e+00
0.000000e+00	-1.286180e+00	0.000000e+00	-2.047019e-01
0.000000e+00	-5.129892e-01	0.000000e+00	-8.164476e-02
0.000000e+00	5.129892e-01	0.000000e+00	8.164476e-02
0.000000e+00	1.286180e+00	0.000000e+00	2.047019e-01
Constant Factor = 1.129524e-01			

Figure 76 Fifth-Order Low-Pass Filter Response



Example 2 – Kerwin’s Circuit

This example is a PrimeSim HSPICE input file for pole/zero analysis of Kerwin’s circuit, which is in the following directory: `$installdir/demo/hspice/filters/fkerwin.sp`.

Figure 77 Design Example for Kerwin’s Circuit

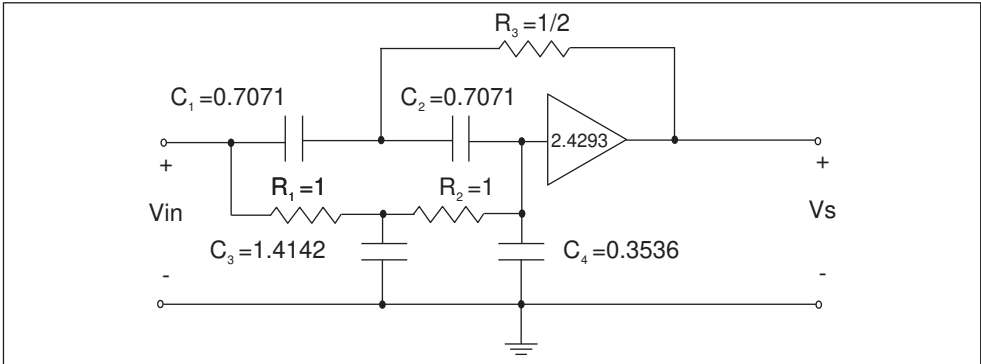


Table 55 on page 364 lists analysis results.

Table 55 Pole/Zero Analysis Results for Kerwin’s Circuit

Poles (rad/sec)		Poles (Hz)	
Real	Imaginary	Real	Imaginary
-5.003939e-02	9.987214e-01	-7.964016e-03	1.589515e-01
-5.003939e-02	-9.987214e-01	-7.964016e-03	-1.589515e-01
-1.414227e+00	0.000000e+00	-2.250812e-01	0.000000e+00
0.000000e+00	-1.414227e+00	0.000000e+00	-2.250812e-01
0.000000e+00	1.414227e+00	0.000000e+00	2.250812e-01
-1.414227e+00	0.000000e+00	-2.250812e-01	0.000000e+00
Constant Factor = 1.214564e+00			

Example 3 – High-Pass Butterworth Filter

This example is a PrimeSim HSPICE input file, for pole/zero analysis of a fourth-order high-pass Butterworth filter. This file can be found at `$installdir/demo/hspice/filters/fhp4th.sp`. Table 56 on page 365 shows the analysis results.

Figure 78 Fourth-Order High-Pass Butterworth Filter

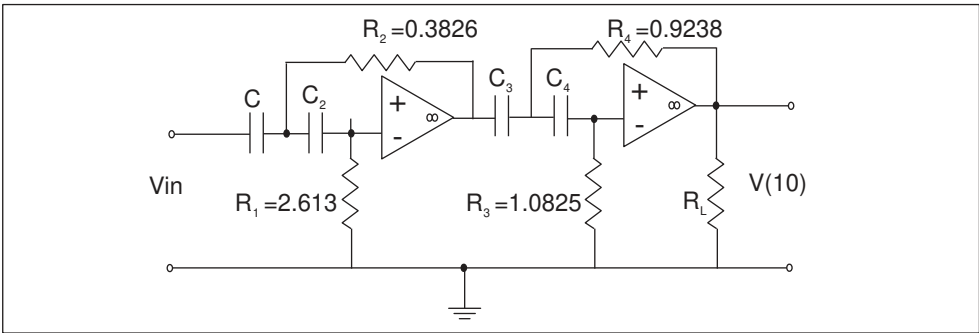


Table 56 Pole/Zero Analysis Results for High-Pass Butterworth Filter

Poles (rad/sec)		Poles (Hz)	
Real	Imaginary	Real	Imaginary
-3.827019e-01	-9.240160e-01	-6.090889e-02	1.470617e-01
-3.827019e-01	9.240160e-01	-6.090890e-02	-1.470617e-01
-9.237875e-01	3.828878e-01	-1.470254e-01	6.093849e-02
-9.237875e-01	-3.828878e-01	-1.470254e-01	-6.093849e-02
0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00
0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00
0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00
0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00
Constant Factor = 1.000000e+00			

Example 4 – CMOS Differential Amplifier

This example uses PrimeSim HSPICE demonstration netlist `mcdiff.sp`, which is available in directory `$installdir/demo/hspice/apps`. [Table 57 on page 366](#) shows the analysis results.

```
*file: mcdiff.sp cmos differential amplifier
* analysis : ac(20khz-500mhz), pole-zero
* mos level=5
*
.options scale=1e-6 scaln=1e-6 wl opts post
.pz v(5) vin
```

```
vin 7 0 0 ac 1
.ac dec 10 20k 500meg
.probe ac vdb(5) vp(5)
m1 4 0 6 6 mn 100 10 2 2
m2 5 7 6 6 mn 100 10 2 2
m3 4 4 1 1 mp 60 10 1.5 1.5
m4 5 4 1 1 mp 60 10 1.5 1.5
m5 6 3 2 2 mn 50 10 1.0 1.0
vdd 1 0 5
vss 2 0 -5
vgg 3 0 -3
rin 7 0 1
.model mn nmos level=5 vt=1 ub=700 frc=0.05 tox=800 dnb=1.6e16
+ xj=1.2 latd=0.7 cj=0.13 phi=1.2 tcv=0.003
.model mp pmos level=5 vt=-1 ub=245 frc=0.25 tox=800 dnb=1.3e15
+ xj=1.2 latd=0.9 cj=0.09 phi=0.5 tcv=0.002
.end
```

The following is an equivalent example in PrimeSim HSPICE advanced analog analyses:

```
* HSPICE advanced analog analysis example
CMOS DIFFERENTIAL AMPLIFIER
.OPTION PIVOT SCALE=1.e-6 SCALM=1.e-6 HQR
.PZ V(5) VIN
VIN 7 0 0 AC 1
.AC DEC 10 20K 500MEG
M1 4 0 6 6 MN 100 10 2 2
M2 5 7 6 6 MN 100 10 2 2
M3 4 4 1 1 MP 60 10 1.5 1.5
M4 5 4 1 1 MP 60 10 1.5 1.5
M5 6 3 2 2 MN 50 10 1.0 1.0
VDD 1 0 5
VSS 2 0 -5
VGG 3 0 -3
RIN 7 0 1
.MODEL MN NMOS LEVEL=5 VT=1 UB=700 FRC=0.05 DNB=1.6E16
+ XJ=1.2 LATD=0.7 CJ=0.13 PHI=1.2 TCV=0.003 TOX=800
.MODEL MP PMOS LEVEL=5 VT=-1 UB=245 FRC=0.25 TOX=800
+ DNB=1.3E15 XJ=1.2 LATD=0.9 CJ=0.09 PHI=0.5 TCV=0.002
.END
```

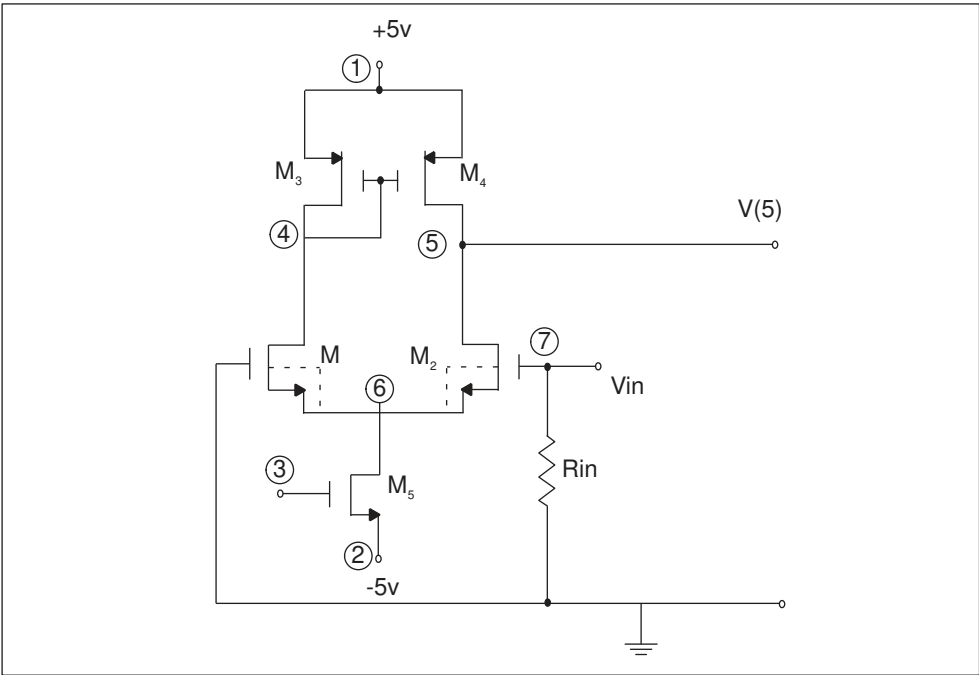
Table 57 Pole/Zero Analysis Results for CMOS Differential Amplifier

Poles (rad/sec)		Poles (Hz)	
Real	Imaginary	Real	Imaginary
-1.798766e+06	0.000000e+00	-2.862825e+05	0.000000e+00
-1.126313e+08	-6.822910e+07	-1.792583e+07	-1.085900e+07
-1.126313e+08	6.822910e+07	-1.792583e+07	1.085900e+07

Table 57 Pole/Zero Analysis Results for CMOS Differential Amplifier
(Continued)

Poles (rad/sec)		Poles (Hz)	
Real	Imaginary	Real	Imaginary
-1.315386e+08	7.679633e+07	-2.093502e+07	1.222251e+07
-1.315386e+08	-7.679633e+07	-2.093502e+07	-1.222251e+07
7.999613e+08	0.000000e+00	1.273178e+08	0.000000e+00
Constant Factor = 3.103553e-01			

Figure 79 CMOS Differential Amplifier



Example 5 – Simple Amplifier

This example is a PrimeSim HSPICE input file for pole/zero analysis of an equivalent circuit for a simple amplifier with:

- $RS_{1/4}=RPI=RL=1000$ ohms
- $gm=0.04$ mho

- CMU=1.0e-11 farad
- CPI¼=1.0e-9 farad

The file is in `$installdir/demo/hspice/apps/ampg.sp`. Table 58 shows the analysis results.

Figure 80 Simple Amplifier

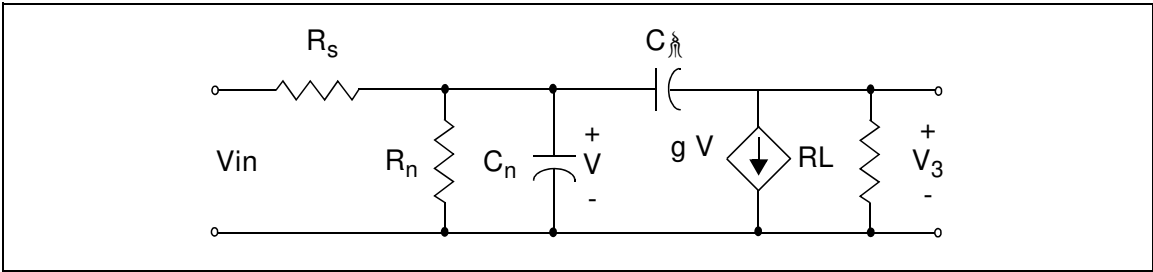


Table 58 Pole/Zero Analysis Results for Amplifier

Poles (rad/sec)		Poles (Hz)	
Real	Imaginary	Real	Imaginary
-1.412555e+06	0.000000e+00	-2.248151e+05	0.000000e+00
-1.415874e+08	0.000000e+00	-2.253434e+07	0.000000e+00
4.000000e+09	0.000000e+00	6.366198e+08	0.000000e+00
Constant Factor = 1.000000e+06			

Example 6 – Active Low-Pass Filter

This example is uses demonstration netlist `flp9th.sp`, which is available in directory `$installdir/demo/hspice/filters`. It is for a pole/zero analysis of an active ninth-order low-pass filter by using the ideal operational amplifier element. This example performs an AC analysis. Table 59 on page 371 shows the analysis results.

```
* file flp9th.sp----9th order low-pass filter
*
* reference: jiri vlach and kishore singhal, 'computer
* methods for circuit analysis and design',
* van nostrand reinhold co., 1983, pages 142
* and 494 to 496.
*
* pole/zero analysis and using vcvs as an ideal op-amp.
* for just pole/zero analysis .ac statement is not required.
vin in 0 ac 1
.ac dec 100 1 100k
```

Chapter 14: Pole-Zero Analysis

Pole/Zero Analysis Examples

```
.print vm(out) vm(in) vp(out)
.probe ac vdb(out,in) par('db(vm(out)/vm(in))')
.pz v(out) vin
.option post
+ x0r=-1.23456e+3 x1r=-1.23456e+2 x2r=1.23456e+3
+ fscale=1e-6 gscale=1e3 cscale=1e9 lscale=1e3
.subckt fdnr 1 r1=2k c1=12n r4=4.5k
r1 1 2 r1
c1 2 3 c1
r2 3 4 3.3k
r3 4 5 3.3k
r4 5 6 r4
c2 6 0 10n
eop1 5 0 2 4 level=1
eop2 3 0 6 4 level=1
.ends
*
rs in 1 5.4779k
r12 1 2 4.44k
r23 2 3 3.2201k
r34 3 4 3.63678k
r45 4 out 1.2201k
c5 out 0 10n
x1 1 fdnr r1=2.0076k c1=12n r4=4.5898k
x2 2 fdnr r1=5.9999k c1=6.8n r4=4.25725k
x3 3 fdnr r1=5.88327k c1=4.7n r4=5.62599k
x4 4 fdnr r1=1.0301k c1=6.8n r4=5.808498k
.end
```

The following is an equivalent example in PrimeSim HSPICE advanced analog analyses:

```
* HSPICE advanced analog analysis example
VIN IN 0 AC 1

.PZ V(OUT) VIN
.AC DEC 50 .1K 100K
.OPTION PLST XOR=-1.23456E+3 X1R=-1.23456E+2
+ X2R=1.23456E+3 FSCALE=1E-6 GSCALE=1E3 CSCALE=1E9 LSCALE=1E3
.PRINT AC VDB(OUT)

.SUBCKT OPAMP IN+ IN- OUT GM1=1 RI=1K CI=26.6U GM2=1.33333 RL=75
RII IN+ IN- 2MEG
RI1 IN+ 0 500MEG
RI2 IN- 0 500MEG
G1 1 0 IN+ IN- GM1
C1 1 0 CI
R1 1 0 RI
G2 OUT 0 1 0 GM2
RLD OUT 0 RL
.ENDS
.SUBCKT FDNR 1 R1=2K C1=12N R4=4.5K RLX=75
R1 1 2 R1
C1 2 3 C1
```

```

R2 3 4 3.3K
R3 4 5 3.3K
R4 5 6 R4
C2 6 0 10N
XOP1 2 4 5 OPAMP
XOP2 6 4 3 OPAMP
.ENDS
$
$
RS IN 1 5.4779K
R12 1 2 4.44K
R23 2 3 3.2201K
R34 3 4 3.63678K
R45 4 OUT 1.2201K
C5 OUT 0 10N

X1 1 FDNR R1=2.0076K C1=12N R4=4.5898K
X2 2 FDNR R1=5.9999K C1=6.8N R4=4.25725K
X3 3 FDNR R1=5.88327K C1=4.7N R4=5.62599K
X4 4 FDNR R1=1.0301K C1=6.8N R4=5.808498K
.END

```

Figure 81 Linear Model of the 741C Operational Amplifier

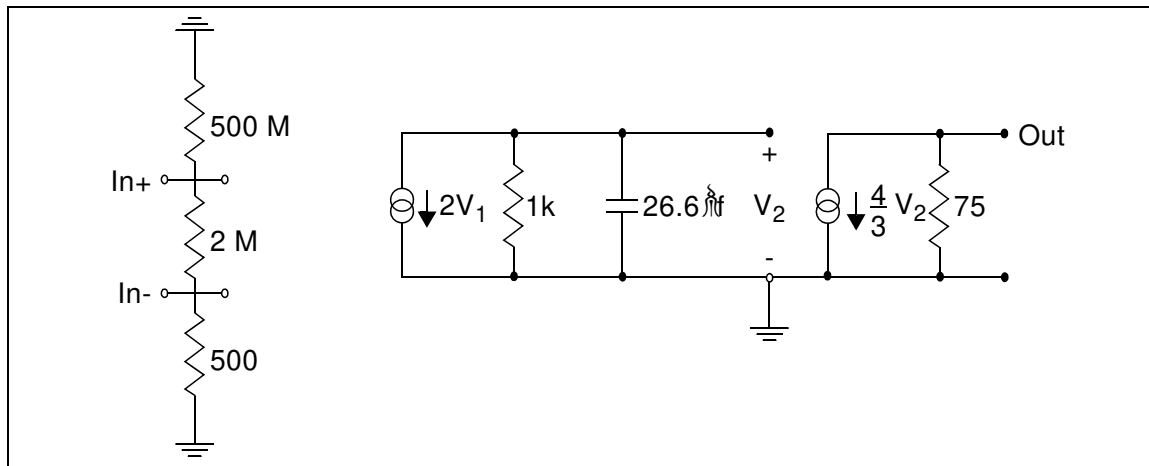


Figure 82 FDNR Subcircuit

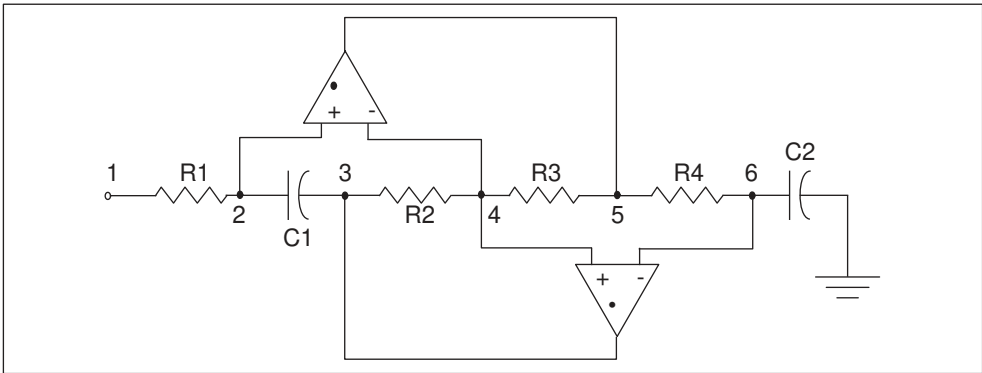


Figure 83 Active Realization of the Low-Pass Filter

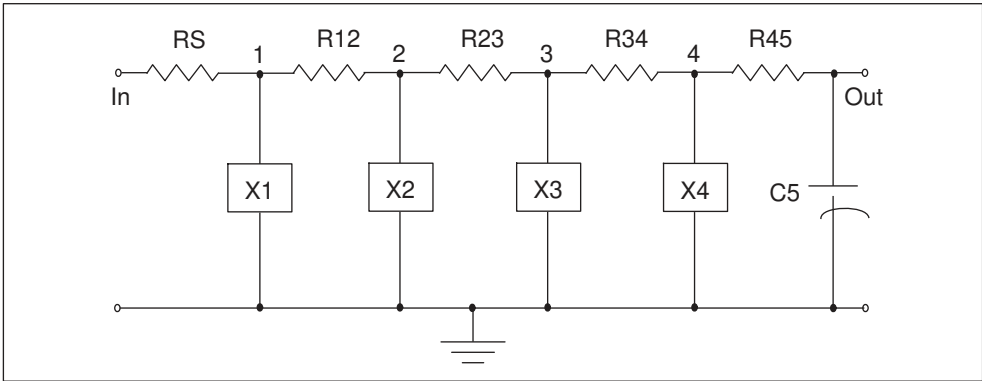


Table 59 Pole/Zero Analysis Results for the Active Low-Pass Filter

Poles (rad/sec)		Poles (Hz)	
Real	Imaginary	Real	Imaginary
-4.505616e+02	-2.210451e+04	-7.170911e+01	-3.518042e+03
-4.505616e+02	2.210451e+04	-7.170911e+01	3.518042e+03
-1.835284e+03	2.148369e+04	-2.920944e+02	3.419236e+03
-1.835284e+03	-2.148369e+04	-2.920944e+02	-3.419236e+03
-4.580172e+03	1.944579e+04	-7.289571e+02	3.094894e+03
-4.580172e+03	-1.944579e+04	-7.289571e+02	-3.094894e+03
-9.701962e+03	1.304893e+04	-1.544115e+03	2.076802e+03

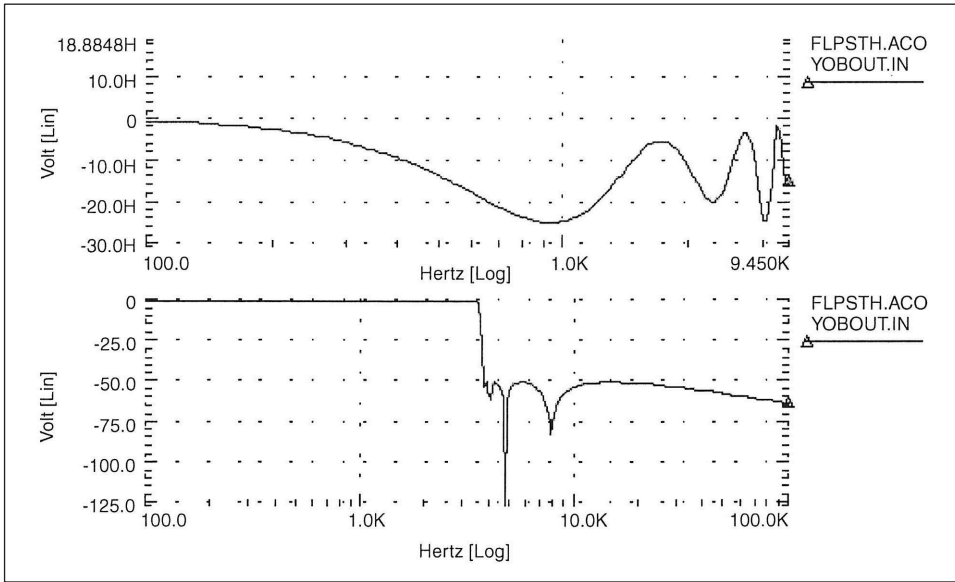
Table 59 Pole/Zero Analysis Results for the Active Low-Pass Filter
(Continued)

Poles (rad/sec)		Poles (Hz)	
Real	Imaginary	Real	Imaginary
-9.701962e+03	-1.304893e+04	-1.544115e+03	-2.076802e+03
-1.353908e+04	0.000000e+00	-2.154811e+03	0.000000e+00
-3.668995e+06	-3.669793e+06	-5.839386e+05	-5.840657e+05
-3.668995e+06	3.669793e+06	-5.839386e+05	5.840657e+05
-3.676439e+06	-3.676184e+06	-5.851234e+05	-5.850828e+05
-3.676439e+06	3.676184e+06	-5.851234e+05	5.850828e+05
-3.687870e+06	3.687391e+06	-5.869428e+05	5.868665e+05
-3.687870e+06	-3.687391e+06	-5.869428e+05	-5.868665e+05
-3.695817e+06	-3.695434e+06	-5.882075e+05	-5.881466e+05
-3.695817e+06	+3.695434e+06	-5.882075e+05	5.881466e+05
-3.220467e-02	-2.516970e+04	-5.125532e-03	-4.005882e+03
-3.220467e-02	2.516970e+04	-5.125533e-03	4.005882e+03
2.524420e-01	-2.383956e+04	4.017739e-02	-3.794184e+03
2.524420e-01	2.383956e+04	4.017739e-02	3.794184e+03
1.637164e+00	2.981593e+04	2.605627e-01	4.745353e+03
1.637164e+00	-2.981593e+04	2.605627e-01	-4.745353e+03
4.888484e+00	4.852376e+04	7.780265e-01	7.722796e+03
4.888484e+00	-4.852376e+04	7.780265e-01	-7.722796e+03
-3.641366e+06	-3.642634e+06	-5.795413e+05	-5.797432e+05
-3.641366e+06	3.642634e+06	-5.795413e+05	5.797432e+05
-3.649508e+06	-3.649610e+06	-5.808372e+05	-5.808535e+05
-3.649508e+06	3.649610e+06	-5.808372e+05	5.808535e+05
-3.683700e+06	3.683412e+06	-5.862790e+05	5.862333e+05
-3.683700e+06	-3.683412e+06	-5.862790e+05	-5.862333e+05
-3.693882e+06	3.693739e+06	5.878995e+05	5.878768e+05

Table 59 Pole/Zero Analysis Results for the Active Low-Pass Filter
(Continued)

Poles (rad/sec)		Poles (Hz)	
Real	Imaginary	Real	Imaginary
-3.693882e+06	-3.693739e+06	-5.878995e+05	-5.878768e+05
Constant Factor = 4.451586e+02			

Figure 84 9th Order Low-Pass Filter Response



The graph in [Figure 84](#) shows overall response of the low-pass filter.

References

- [1] Desoer, Charles A. and Kuh, Ernest S. *Basic Circuit Theory*. New York: McGraw-Hill.1969. Chapter 15.
- [2] Van Valkenburg, M. E. *Network Analysis*. Englewood Cliffs, New Jersey: Prentice Hall, Inc., 1974, Chapters 10 & 13.
- [3] R.H. Canon, Jr. *Dynamics of Physical Systems*. New York: McGraw-Hill, 1967. This text describes electrical, mechanical, pneumatic, hydraulic, and mixed systems.
- [4]

- B.C. Kuo. *Automatic Control Systems*. Englewood Cliffs, New Jersey: Prentice-Hall, 1975. This source discusses control system design, and provides background material about physical modeling.
- [5] L.T. Pillage, and R.A. Rohrer. "Asymptotic Waveform Evaluation for Timing Analysis", *IEEE Trans CAD*. Apr. 1990, pp. 352 - 366. This paper is a good references about interconnect transfer function modeling, and discusses extracting transfer functions for timing analysis.
- [6] S. Lin, and E.S. Kuh. "Transient Simulation of Lossy Interconnects Based on the Recursive Convolution Formulation", *IEEE Trans CAS*. Nov. 1992, pp. 879 - 892. This paper is another source for how to model interconnect transfer functions.
- [7] Muller, D. E., *A Method for Solving Algebraic Equations Using a Computer, Mathematical Tables, and Other Aids to Computation (MTAC)*. 1956, Vol. 10,. pp. 208-215.
- [8] Temes, Gabor C. and Mitra, Sanjit K. *Modern Filter Theory And Design*. J. Wiley, 1973, page 74.
- [9] Temes, Gabor C. and Lapatra, Jack W. *Circuit Synthesis And Design*. McGraw-Hill. 1977, page 301, example 7-6.
- [10] Temes, Gabor C. and Mitra, Sanjit K., *Modern Filter Theory And Design*. J. Wiley, 1973, page 348, example 8-3.
- [11] Desoer, Charles A. and Kuh, Ernest S. *Basic Circuit Theory*. McGraw-Hill, 1969, page 613, example 3.
- [12] Vlach, Jiri and Singhal, Kishore. *Computer Methods For Circuit Analysis and Design*. Van Nostrand Reinhold Co., 1983, pages 142, 494-496.

15

MOSFET Reliability Analysis (MOSRA)

Describes the procedures for PrimeSim HSPICE MOSFET reliability analysis (MOSRA).

The following sections cover these topics:

- [MOSRA Overview](#)
- [MOSRA Model Overview](#)
- [Level 1 MOSRA BTI and HCI Model](#)
- [Level 3 MOSRA BTI and HCI Model](#)

MOSRA Overview

As the industry scales down CMOS technology, reliability requirements to maintain the long-term device become both more challenging and more important. Two of the most critical reliability issues, the hot carrier injection (HCI) and the bias temperature instability (BTI) can change the characteristics of MOS devices. PrimeSim HSPICE reliability analysis allows circuit designers to predict the reliability of their design to allow enough margin for their circuits to function correctly over their lifetime.

PrimeSim HSPICE MOSRA analysis currently supports Level 49, Level 53, Level 54, Level 57, Level 62, Level 66, Level 68, Level 69, Level 70, Level 71, Level 72, Level 73, Level 76, Level 77, Level 78, and external CMI MOSFET models.

PrimeSim HSPICE MOSRA analysis also supports CMC MOS Varactor Model (level 7) for fresh simulations.

For more information on using .MOSRA commands and control options, see *PrimeSim™ Continuum Reference Manual: Commands and Control Options*.

The following sections discuss these topics:

- [Reliability Analysis Use Model](#)
- [Example Setup](#)

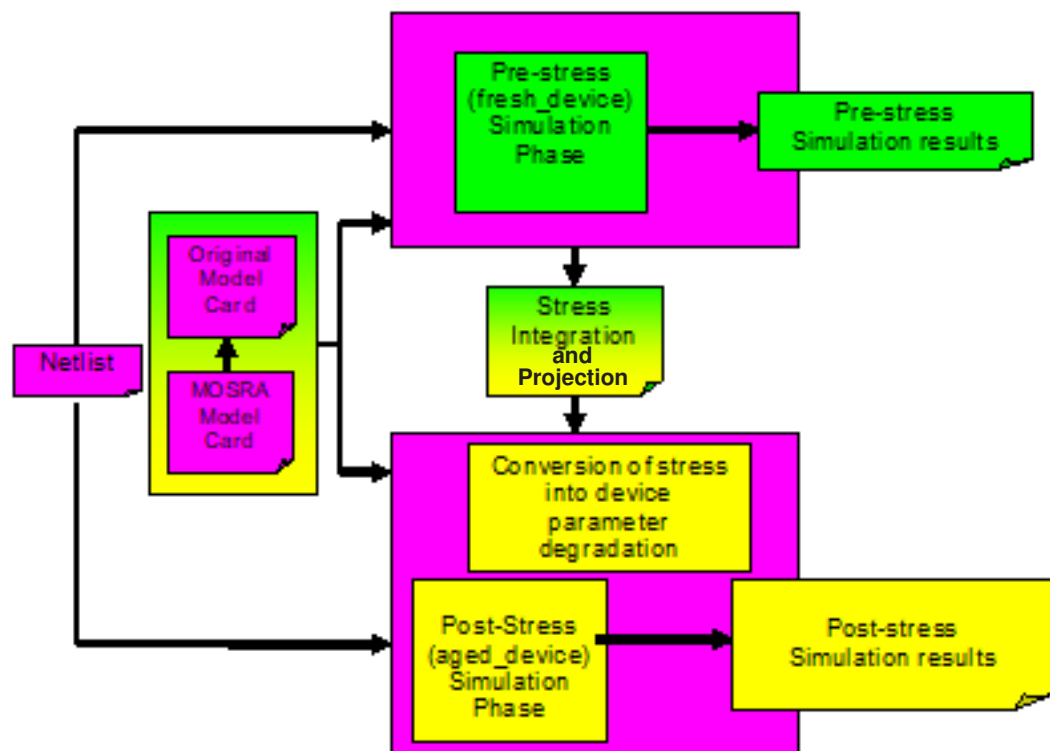
Reliability Analysis Use Model

PrimeSim HSPICE reliability analysis (or HCI and BTI analysis), is a two-phase simulation: the fresh simulation phase and the post-stress simulation phase. The two-phase simulation can run separately or together.

- Fresh simulation phase: PrimeSim HSPICE computes the electron age/stress of selected MOS transistors in the circuit based on circuit behavior and on the PrimeSim HSPICE built-in stress model including HCI and/or BTI effect.
- Post-stress simulation phase: PrimeSim HSPICE simulates the degradation effect on circuit performance, based on the stress information that the fresh simulation phase produces.

Figure 85 presents the PrimeSim HSPICE reliability analysis process.

Figure 85 PrimeSim HSPICE Reliability Simulation Flow



Example Setup

The following example file demonstrates how to set up a PrimeSim HSPICE reliability analysis.

```
* MOSRA TEST
vdd 1 0 2
mp1 3 2 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn1 3 2 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
mp2 4 3 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn2 4 3 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
mp3 2 4 1 1 p1 l=0.1u w=10u ad=5p pd=6u as=5p ps=6u
mn3 2 4 0 0 n1 l=0.1u w=5u ad=5p pd=6u as=5p ps=6u
c1 2 0 .1p

.model p1 pmos level=54 version=4.5
.model n1 nmos level=54 version=4.5

.model pl_ra mosra level=1
+tit0 = 5e-8 titfd = 7.5e-10 tittd = 1.45e-20
+tn = 0.25

.appendmodel pl_ra mosra p1 pmos
.mosra reltotaltime=1e8

.ic v(2)=2
.tran .1ps 5ns
.options post
.end
```

MOSRA Model Overview

PrimeSim HSPICE MOSRA supports three modeling approaches to describe MOSFET aging effects. They include MOSRA Level 1 model, MOSRA Level 3 model, and MOSRA custom reliability-modeling API. By choosing the right approach, users can achieve the desired balance between model extraction/implementation effort and design requirements.

MOSRA Level 1 BTI and HCI model is physics-based. Its capability is best utilized when sweep measurement of an entire family of I-V curves of an aged MOS transistor is available for parameter extraction. For detailed information on this model, see [Level 1 MOSRA BTI and HCI Model on page 378](#). The mathematical formulation of this model and its detailed model equations are disclosed only with a non-disclosure agreement (NDA). Contact your Synopsys technical support team for more information.

MOSRA Level 3 BTI and HCI model provides a concise and efficient modeling approach that has proven to be sufficient for many designs. It can adequately fit a device's electrical parameter degradation (for example, I_{dlin} and v_{th} of an aged MOS transistor) under custom-defined bias conditions. Parameters of MOSRA Level 3 BTI and HCI model

and the full model equations are described in [Level 3 MOSRA BTI and HCI Model on page 384](#).

MOSRA custom reliability-modeling API provides the flexible platform for formulating MOSFET aging equations and stress computation algorithms. It allows users to develop and implement fully custom MOSFET aging models as well as custom stress integration/prediction algorithms. Access to documentation of MOSRA custom reliability-modeling API is provided under a non-disclosure agreement (NDA). Contact your Synopsys technical support team for more information.

Level 1 MOSRA BTI and HCI Model

The following tables list parameters and their descriptions for HCI and BTI:

- [Table 60](#)
- [Table 61](#)
- [Table 62](#)
- [Table 63](#)

Synopsys Level 1 MOSRA Model

Table 60 Control Flags

Name	Default	Description	Notes
EXT_VOLT_MODE	0	- Flag to control terminal bias usage: 0: Use internal terminal bias. 1: Use external terminal bias.	-

Synopsys LEVEL1 MOSRA model, BTI Vth degradation

Table 61 Vth Degradation BTI Parameters

Name	Default	L term available	Description	Notes
TIT0	0	Yes	First parameter for interface-trap-inducing threshold voltage degradation	
TITCE	0	Yes	Inversion charge exponent for interface-trap-inducing threshold voltage degradation	
TITGA	0	Yes	Vgs dependence offset	

Table 61 *Vth Degradation BTI Parameters (Continued)*

Name	Default	L term available	Description	Notes
TITFD	0	Yes	Oxide electric field dependence for interface trap inducing threshold voltage degradation	
TITTD	0		Temperature dependent component of interface-trap-inducing threshold voltage degradation	
TITWC	0		Channel width coefficient for interface-trap-inducing threshold voltage degradation	
TITWE	0		Channel width exponent for interface trap inducing threshold voltage degradation	
TITLC	0		Channel length coefficient for oxide trap inducing threshold voltage degradation	
TITLE	0		Channel length exponent for oxide-trap-inducing threshold voltage degradation	
TN	0.25		Stress time exponent for interface-trap-inducing threshold voltage degradation	
TOT0	0		First parameter for oxide-trap-inducing threshold voltage	
TOTFD	0		Oxide electric field dependent component for oxide-trap-inducing threshold voltage degradation	
TOTTD	0		Temperature dependent component for oxide-trap-inducing threshold voltage degradation	
TOTDD	0	Yes	Drain voltage dependent coefficient for oxide electric field in threshold voltage degradation	
TOTWC	0		Channel width coefficient of oxide-trap-inducing threshold voltage degradation	
TOTWE	0		Channel width exponent of oxide-trap-inducing threshold voltage degradation	
TOTLC	0		Channel length coefficient of oxide-trap-inducing threshold voltage degradation	
TOTLE	0		Channel length component of oxide-trap-inducing threshold voltage degradation	
TK	0.5		Stress time exponent for oxide-trap-inducing threshold voltage degradation	
TTD0	1		First parameter for transient degradation of threshold voltage	TTD0=0 disables the Vth recovery effect

Table 61 *Vth Degradation BTI Parameters (Continued)*

Name	Default	L term available	Description	Notes
TDCD	0		Duty cycle dependent exponent for transient degradation of threshold voltage	TDCD is expected to be 0 or negative
TFC	0		Frequency dependence coefficient for BTI recovery of threshold voltage	
TFE	0		Frequency dependence exponent for BTI recovery of threshold voltage.	
TOTDE	1	Yes	Drain voltage exponent for oxide electric field in threshold voltage degradation	
EOXMOD	0 (disabled)		Enables separate electric field equations for mobility, DIBL, and VSAT degradations	
DLBTI	0		Length dependence offset for BTI model	
DWBTI	0		Width dependence offset for BTI model	
LNOMRA	0		Nominal length for NBTI length dependence limiting	

Mobility Degradation BTI Parameters

Table 62 *Mobility Degradation BTI Parameters*

Name	Default	L term available	Description	Notes
UIT0	0	Yes	First parameter for interface-trap-inducing mobility degradation	
UITCE	0	Yes	Inversion charge exponent for interface-trap-inducing mobility degradation	
UITGA	0	Yes	Vgs dependence offset	
UITFD	0	Yes	Oxide electric field dependence for interface-trap-inducing mobility degradation	
UITTD	0		Temperature dependent coefficient of interface-trap-inducing mobility degradation	
UITWC	0		Channel width dependent coefficient for interface-trap-inducing mobility degradation	
UITWE	0		Channel width exponent for interface-trap-inducing mobility degradation	

Table 62 Mobility Degradation BTI Parameters (Continued)

Name	Default	L term available	Description	Notes
UITLC	0		Channel length dependent coefficient for interface-trap-inducing mobility degradation	
UITLE	0		Channel length exponent for interface-trap-inducing mobility degradation	
UN	0.25		Stress time exponent for interface-trap-inducing mobility degradation	
UOT0	0		First parameter for oxide-trap-inducing mobility degradation	
UOTFD	0		Oxide electric field dependence for oxide-trap-inducing mobility degradation	
UOTTD	0		Temperature dependence for oxide-trap-inducing mobility degradation	
UOTWC	0		Channel width coefficient for oxide-trap-inducing mobility degradation	
UOTWE	0		Channel width exponent for oxide-trap-inducing mobility degradation	
UOTLC	0		Channel length coefficient for oxide-trap-inducing mobility degradation	
UOTLE	0		Channel length exponent for oxide-trap-inducing mobility degradation	
UK	0.5		Stress time exponent for oxide-trap-inducing mobility degradation	
UTD0	1		First parameter transient mobility degradation	UTD0=0 disables the mobility recovery effect
UDCD	0		Duty cycle dependent coefficient for transient mobility degradation	UDCD is expected to be 0 or negative
UFC	0		Frequency dependence coefficient for BTI recovery of mobility.	
UFE	0		Frequency dependence exponent for BTI recovery of mobility.	
UOTDD	TOTDD	Yes	Drain voltage dependent coefficient for oxide electric field in mobility degradation	Used if EOXMOD=1
UOTDE	TOTDE	Yes	Drain voltage exponent for oxide electric field in mobility degradation	Used if EOXMOD=1

Synopsys LEVEL1 MOSRA model, HCI for NMOS and PMOS

Table 63 Threshold Voltage and Mobility Degradation HCI Parameters

Name	Default	L term available	Description	Notes
THCI0	0	Yes	First parameter for threshold voltage degradation induced by HCI	
THCI1	0		Coefficient for I_{sub} -dependent term of mobility HCI degradation	
UHCI0	0	Yes	First parameter for mobility degradation induced by HCI	
UHCI1	0		Coefficient for I_{sub} -dependent term of mobility HCI degradation	
TDII	0	Yes	Impact ionization current exponent for threshold voltage degradation induced by HCI	
TDLE	1		Channel length exponent for threshold voltage degradation induced by HCI	
TDCE	0	Yes	Channel current exponent for threshold voltage degradation induced by HCI	
TEA	0		Equivalent activation energy of V_{th} HCI degradation	
TDLT	0		Channel length dependence term of V_{th} HCI degradation	
THCVD	0	Yes	Coefficient for V_{ds} -dependent term of V_{th} HCI degradation	
TDVD	0	Yes	V_{ds} term exponent of V_{th} HCI degradation	
TDID	1	Yes	Channel current exponent for V_{ds} -dependent term of V_{th} HCI degradation	
THCVB	0		Coefficient for V_{bs} dependence of V_{th} HCI degradation	
TDVB	1		Exponent for V_{bs} dependence of V_{th} HCI degradation	
HN	0.5	Yes	Time exponent for threshold voltage degradation induced by HCI	
UDCE	0	Yes	Channel current exponent for mobility degradation induced by HCI	
UDII	0	Yes	Impact ionization current exponent for mobility degradation induced by HCI	

Table 63 *Threshold Voltage and Mobility Degradation HCI Parameters (Continued)*

Name	Default	L term available	Description	Notes
UDLE	1		Channel length exponent for mobility degradation induced by HCI	
UEA	0		Equivalent activation energy of mobility HCI degradation	
UDLT	0		Channel length dependence term of mobility HCI degradation	
UHCVD	0	Yes	Coefficient for Vds-dependent term of mobility HCI degradation	
UDVD	0	Yes	Vds term exponent of mobility HCI degradation	
UDID	1	Yes		Channel current exponent for Vds-dependent term of mobility HCI degradation
UHCVB	0		Coefficient for Vbs dependence of mobility HCI degradation	
UDVB	1		Exponent for Vbs dependence of mobility HCI degradation	
0.5	HK	Yes	Time exponent for mobility degradation induced by HCI	
HIIL	0		First parameter for impact ionization	
HIIVD0	0		First vds dependent parameter of the impact ionization current	
HIIVD1	0		Second vds dependent parameter of the impact ionization current	
HIIVD2		0	Third vds dependent parameter of the impact ionization current	
VDSAT0	0		Nominal drain saturation voltage of the impact ionization current	
HIIT	0		Temperature dependent parameter for the impact ionization current	
HIIL	0		Channel length dependent parameter for the impact ionization current	
HIIE	0		Saturation channel electric field for the impact ionization current	

Table 63 Threshold Voltage and Mobility Degradation HCI Parameters (Continued)

Name	Default	L term available	Description	Notes
HIIVG0	0		First vgs dependent parameter for the impact ionization current	
HIIVG1	0		Second vgs dependent parameter for the impact ionization current	
HIIVG2	0		Third vgs dependent parameter for the impact ionization current	
HIIVGD	0		vds dependent parameter for the impact ionization current	
HIIVOFF	-1000 disabled		Offset voltage for Vgstep limiting	
ISUBMODE	1		lii vs. lsub model flag	
DLHCI	0		Length dependence offset for HCI model	
DWHCI	0		Width dependence offset for HCI model	
VBSMAX	1e6		Positive VBS limiting value for HCI model	

Level 3 MOSRA BTI and HCI Model

This section describes the level 3 model formulations, model parameters, and their descriptions for BTI and HCI, in the following sections:

- [BTI Model](#)
- [HCI Model](#)

The parameter listing tables are:

- [Table 1](#)
- [Table 2](#)

BTI Model

BTI Degradation Equation (unit of mV)

$$Vth_{shift} = A_p e^{G \times type \times V_{gs}} e^{(-Ea)/(KT)} t_{stress}^n e^{-D \times abs(V_{ds})} e^{mL}$$

Where:

- T is the channel temperature in Kelvin,
- K is the Boltzmann's constant (8.617×10^{-5} eV/K),
- L is the polysilicon length of the device in μm ,
- V_{ds} is the drain to source voltage in Volts,
- V_{gs} is the gate to source voltage in Volts,
- and t is the total stress time in seconds.

Synopsys LEVEL3 MOSRA Model, BTI Vth Degradation

Table 64 Vth Degradation BTI Parameters

Name	Default	Description	Notes
BTIFLAG	1	BTI model selector	Disable/enable the BTI model
BTIAP	0.0	Pre-factor	Used as Ap in equation
BTIM	3.0	L dependency	Used as m in equation
BTIEA	0.143	Temp Accel	Used as Ea in equation
BTID	0.0	Vds voltage dependency	Used as D in equation
BTIG	5.0	Vgs voltage dependency	Used as G in equation
BTIN	0.175	Time dependency	Used as n in equation
BTIR	0.5	Recovery parameter	See 1 below
$V_{GS\text{MAX}}$	1.5	$V_{GS\text{MAX}}$	See 1 below
BTILFLAG	0	Flag of L selection	See 2 below

1. Partial recovery effect model:

$$\Delta V_{thAC} = \Delta V_{thDC} \cdot \exp(-Rg)$$

Where R is the recovery parameter and g is the duty cycle defined as:

$$g = 1 - \left(\int_0^t (V_{gs} \times type) dt \right) / t \cdot V_{gsmax}$$

2. BTI equation uses the BTILFLAG model parameter in Leff/L selection. BTILFLAG = 0: default, uses Leff. BTILFLAG = 1: uses L.

Note:

V_{GSMAX} should be positive. If V_{GSMAX} is less than or equal to 0, set $g=0$; For V_{GSMAX} usage, if V_{GSMAX} is given in the model card, uses the given value; otherwise, uses the maximum value of ($V_{gs} \times type$).

HCI Model

HCI Degradation Equation

$$Ids_{shift}(percentage) = A_p e^{G \times type \times V_{gs}} e^{-(E_a)/(kT)} t_{stress}^n e^{D \times abs(V_{ds})} e^{-mL} Switch$$

Where:

- Switch means that if ($V_{gs} < V_{gsmin}$, then 0; otherwise, 1),
- T is the channel temperature in Kelvin,
- K is the Boltzmann's constant (8.617×10^{-5} eV/K),
- L is the polysilicon length of the device in μm ,
- V_{ds} is the drain to source voltage in Volts,
- V_{gs} is the gate to source voltage in Volts,
- and t is the total stress time in seconds.

Synopsys LEVEL3 MOSRA Model, HCI Ids Degradation

Table 65 Ids Degradation HCI Parameters

Name	Default	Description	Notes
HCIFLAG	1	HCI model selector	Disable/enable the HCI model
HCIAP	0.0	Pre-factor	Used as A_p in equation
HCIM	2.0	L dependency	Used as m in equation

Table 65 *Ids Degradation HCI Parameters (Continued)*

Name	Default	Description	Notes
HCIEA	0.05	Temp Accel	Used as Ea in equation
HCID	9.0	Vds voltage dependency	Used as D in equation
HCIG	0.0	Vgs voltage dependency	Used as G in equation
HCIN	0.5	Time dependency	Used as n in equation
V _{GSMIN}	0.2	V _{GSMIN}	See 1 below
HCILFLAG	0	Flag of L selection	See 2 below

1. For V_{GSMIN} usage, if V_{GSMIN} is given in the model card, uses the given value; otherwise, uses Vt (the threshold voltage).
2. HCI equation uses the HCILFLAG model parameter in Leff/L selection. HCILFLAG = 0: default, uses Leff. HCILFLAG = 1: uses L.

Clamping

If HCI degradation > 90%, it is clamped to 90%.

16

Post-Layout Simulation: RC Network Reduction and Back-Annotation

Describes post layout simulation including RC network reduction, simulations that contain a large number of parasitic elements, and back-annotation in PrimeSim HSPICE.

PrimeSim HSPICE back annotation supports Full Back-Annotation and Selective Net Back-Annotation for the DSPF and SPEF formats.

In PrimeSim HSPICE, the post-layout simulation is similar to pre-layout simulation. You can do the post-layout simulation with DSPF only if it is a fully extracted netlist with instances. You can include the DSPF file in the pre-layout netlist. You need to replace the ideal `.SUBCKT` blocks from your original netlist with `.SUBCKT` blocks containing the extracted parasitics. Remember to verify that the port order in the extracted `.SUBCKT` blocks match the port order in the ideal netlist. RC reduction ignores nodes with initial conditions set. If your circuit contains many initial condition statements, it is possible to see little or no change in resistor and/or capacitor counts after reduction.

If your extracted netlist is not too large (approximately 100,000 elements or fewer), then PrimeSim HSPICE can give you very good results. Otherwise, you can also employ an RC reduction.

PrimeSim HSPICE ships numerous of examples for your use; see Listing of Demonstration Input Files in the *PrimeSim HSPICE User Guide: Demonstration Netlists* for paths to the demo files. See Back-Annotation Demo Cases for examples of the techniques discussed in this chapter.

The following sections cover these topics:

- [Linear Acceleration](#)
- [Supporting Parasitic L- and K-elements](#)
- [Pruning Parasitics from a Post-Layout Flat Netlist](#)
- [Post-Layout Back-Annotation](#)

Linear Acceleration

The following sections discuss these topics:

- [Using the SIM_LA Option](#)
- [Control Options for Linear Acceleration](#)

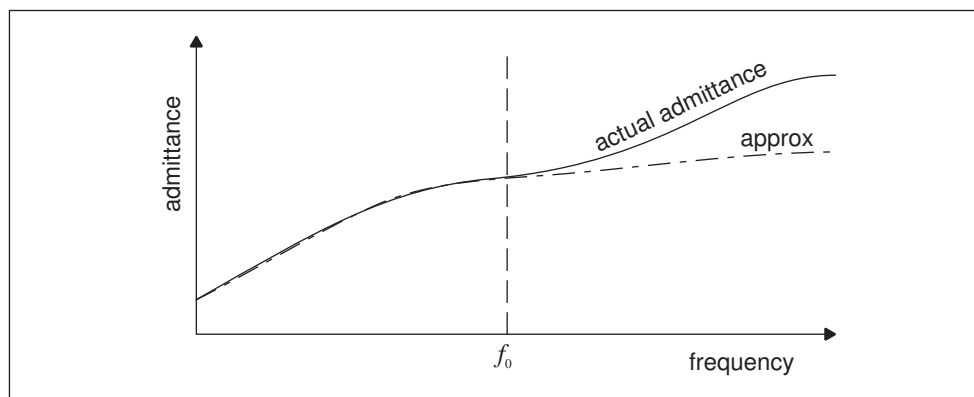
Using the SIM_LA Option

By use of the `SIM_LA` option, you can accelerate the simulation of circuits that include large linear RC networks. To achieve this acceleration, PrimeSim HSPICE linearly reduces all matrices that represent RC networks. The result is a smaller matrix that maintains the original port behavior, yet achieves significant savings in memory and computation. Thus, the `SIM_LA` option is ideal for circuits with large numbers of resistors and capacitors, such as clock trees, power lines, or substrate networks.

In general, the RC elements separate into their own network. The shared nodes of both main circuit elements (including `.PRINT`, `.PROBE`, and `.MEASURE` statements) and RC elements are the port nodes of the RC network. All other RC nodes are internal nodes. The currents that flow into the port nodes are a frequency-dependent function of the voltages at those nodes. The multi-port admittance of a network represents this relationship.

- The `SIM_LA` option formulates matrices to represent multi-port admittance.
- Then, to eliminate as many internal nodes as possible, it reduces the size of these matrices, while preserving the admittance, otherwise known as port node behavior.
- The amount of reduction depends on the f_0 upper frequency, the threshold frequency where `SIM_LA` preserves the admittance ([Figure 86](#)).

Figure 86 *Multiport Admittance vs. Frequency*



The `SIM_LA` option is very effective for post-layout simulation because of the volume of parasitics. For frequencies below f_0 , the *approx* signal matches that of the original admittance. Above f_0 , the two waveforms diverge, but the higher frequencies are not of interest. The lower the f_0 frequency, the greater the amount of reduction.

For the syntax and description of this control option, see `.OPTION SIM_LA` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

You can also choose one of following algorithms, explained in the following sections:

- [PACT Algorithm](#)
- [LNE Algorithm](#)

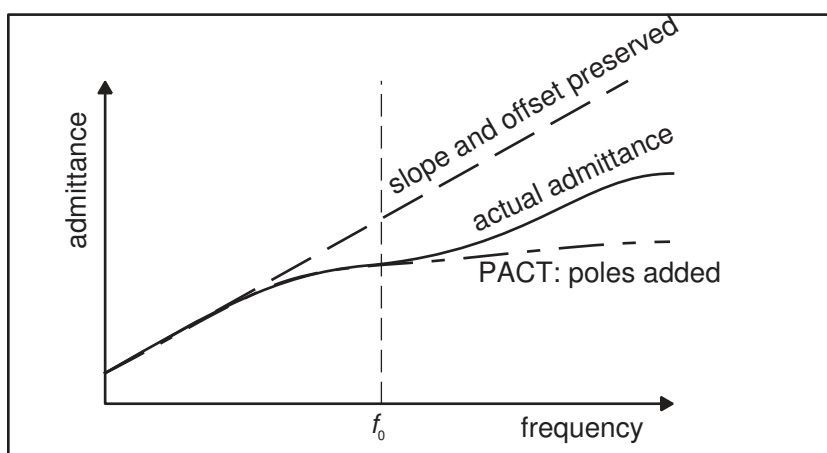
PACT Algorithm

The `PACT` (Pole Analysis via Congruence Transforms) algorithm reduces the RC networks in a well-conditioned manner, while preserving network stability.

- The transform preserves the first two moments of admittance at DC (slope and offset), so that DC behavior is correct (see [Figure 87](#)).
- The algorithm preserves enough low-frequency poles from the original network to maintain the circuit behavior up to a specified maximum frequency f_0 , within the specified tolerance.

This approach is more accurate between these two algorithms, and is the default.

Figure 87 PACT Algorithm



LNE Algorithm

The Linear Node Elimination (LNE) algorithm first checks the order of the nodes for low-to-high R-degree. If the R-degree is less than `MAXDEG` and the time constant ($=\text{sum}(C)/$

sum(G)) of this node is small enough, the removal of the node takes place. Currently, the MAXDEG value is 7.

Control Options for Linear Acceleration

The following table provides a summary of these control options. See the *PrimeSim HSPICE Reference Manual: Commands and Control Options* for details on each of these options.

.OPTION SIM_LA	Activates linear matrix reduction and selects the reduction method.
.OPTION LA_FREQ	Upper frequency where you need accuracy preserved.
.OPTION LA_MAXR	Maximum resistance for linear matrix reduction.
.OPTION LA_MINC	Minimum capacitance for linear matrix reduction.
.OPTION LA_TIME	Minimum time for preservation of accuracy.
.OPTION LA_TOL	Error tolerance for the PACT and LNE algorithms.

Example

In this example, the circuit has a typical rise time of 1ns. Set the maximum frequency to 1 GHz, or set the minimum switching time to 1ns.

```
.OPTION LA_FREQ = 1GHz
-or-
.OPTION LA_TIME = 1ns
```

However, if spikes occur in 0.1ns, PrimeSim HSPICE does not accurately simulate them. To capture the behavior of the spikes, use:

```
.OPTION LA_FREQ = 10GHz
-or-
.OPTION LA_TIME = 0.1ns
```

Note:

Higher frequencies (smaller times) increase accuracy, but only up to the minimum time step used in PrimeSim HSPICE.

Supporting Parasitic L- and K-elements

PrimeSim HSPICE supports simulation with parasitic L- and K-elements. You need to set the minimum value of mutual inductance by using the KLIM option. The default value of KLIM is 0.01. PrimeSim HSPICE does not calculate the second-order mutual inductance for values less than KLIM, but parasitic mutual inductance values can be many orders smaller than the default value.

Also, note that RC reduction is not very effective with respect to L- and K-elements. If you increase the simulation speed of your netlist having a huge number of parasitic elements, you need to properly understand the accuracy versus speed trade-off. For more information about the KLIM option, see .OPTION KLIM.

Pruning Parasitics from a Post-Layout Flat Netlist

The following sections discuss these topics:

- [Using the .PRUNE Command](#)
- [Example](#)
- [Limitation of .PRUNE](#)

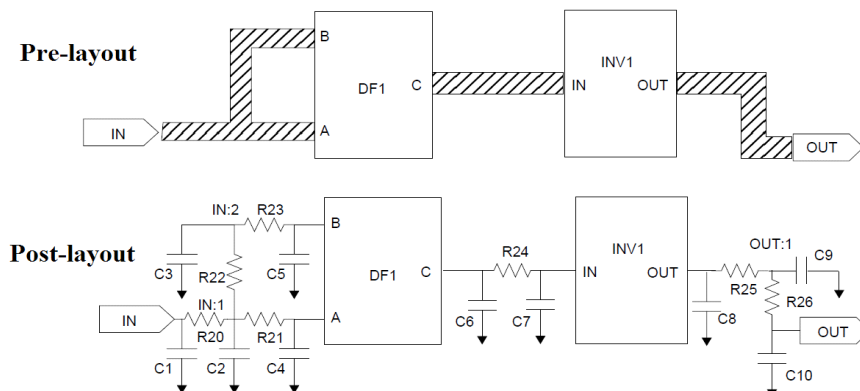
Using the .PRUNE Command

To prune parasitics from a post-layout flat netlist, you can use the .PRUNE command. This command takes two files: one post-layout netlist file and one active-net file. The active-net file can be obtained from the PrimeSim HSPICE .BA_ACHECK command (see .BA_ACHECK in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*) or other simulators. With this command, PrimeSim HSPICE prunes parasitic RCs on the inactive nets to speed up the simulation with minimum loss of accuracy. It is best used on large post-layout circuits with few active nets.

Use the following syntax to specify the post-layout netlist file and the active-net file:

```
.PRUNE "post-layout_flat_netlist_file" "active-net_file"
```

Figure 88 Simplifying cell characterization by pruning R/C parasitics



The process is as follows:

1. Obtain an active net file from a pre-layout netlist.
2. Use this active net file to create a pruned netlist on the flat post-layout netlist.
3. Re-use this pruned netlist in subsequent characterization runs.

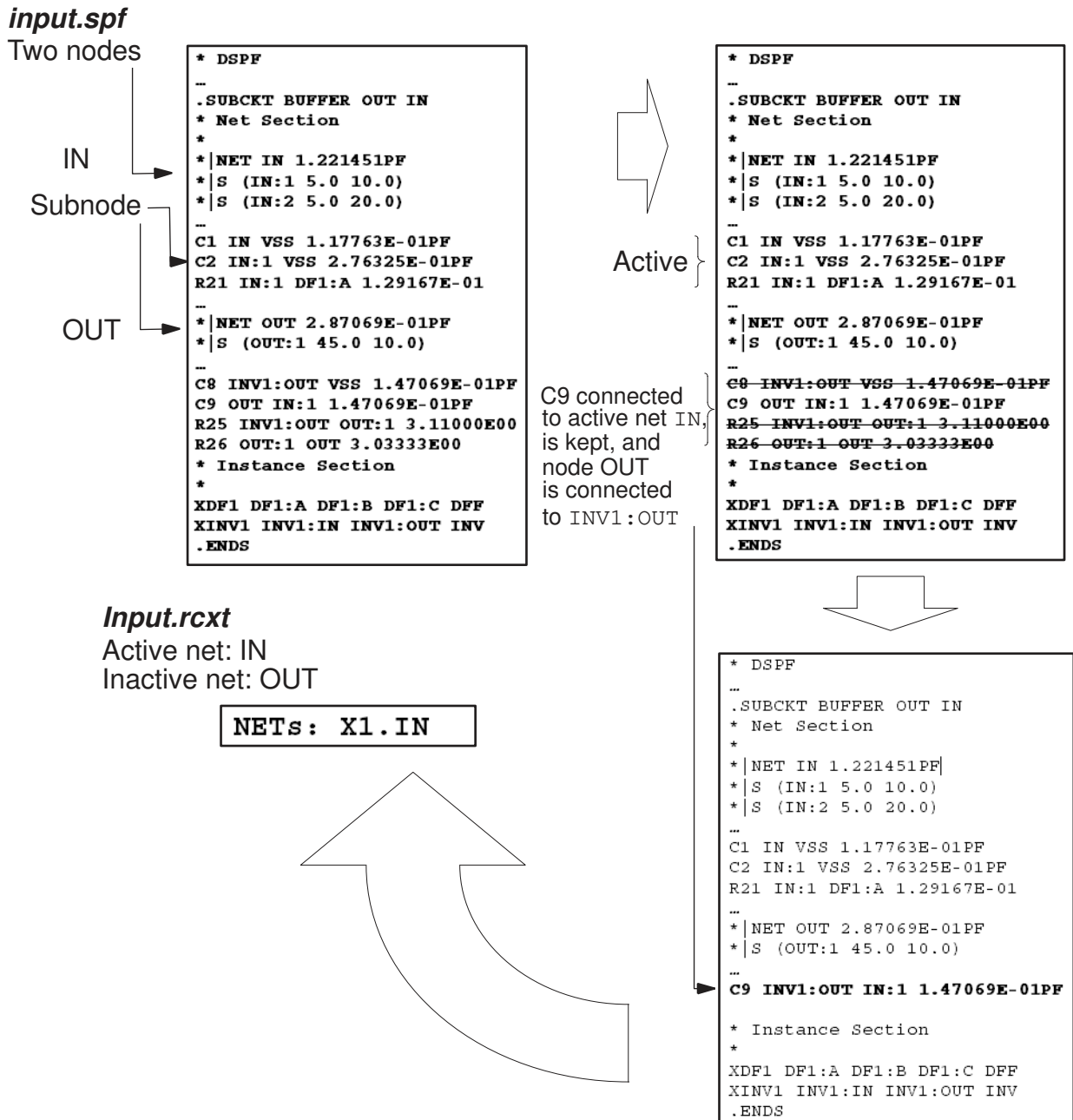
Note:

This flow is similar to the PrimeSim HSPICE selective net Back-Annotation flow. However, PrimeSim HSPICE performs the R/C de-coupling on the flat netlist instead of during back-annotation. See [Selective Net Back-Annotation on page 397](#).

Example

The following illustrates pruning capacitor and resistor parasitics from a flat post-processed *.spf netlist to create an active net input file in *.rcxt format.

Figure 89 Converting a flat post-layout *.spf file to an active-net *.rcxt file



Limitation of .PRUNE

The active net name cannot contain a wildcard (*) character. For example, 'net*33' is illegal.

Post-Layout Back-Annotation

The traditional and straightforward way for post-layout analysis by PrimeSim HSPICE is to include the parasitic netlist containing parasitic RCs and devices as an ordinary netlist for simulation.

The problem with this approach is that the parasitic netlist is flat; hence you do not gain many advantages of hierarchical netlist, such as answers to these questions:

- How can different options apply to different blocks for better trade-off between accuracy and performance?
- How do you perform power analysis on a flat netlist to check the power consumption?

The traditional flow flattens all nodes after extraction so it is more difficult to compare the delay before and after extraction. The traditional flow can also stress the limits of an extraction tool so reliability also becomes an issue.

To address these problems, PrimeSim HSPICE

- Needs a hierarchical (or flat) Layout Versus Schematic (LVS) ideal netlist for use by tools such as Star-RC or Star-RCXT, and a flat parasitic netlist in DSPF or SPEF format.
- Runs these two netlists to with back-annotation analysis. PrimeSim HSPICE can annotate the parasitic RCs (and devices in the instance section of DSPF file) to the ideal netlist and use the annotated ideal netlist for post-layout-like simulation.

This hybrid flat-hierarchical approach provides full control and advantages of simulating a hierarchical netlist. For example:

- You can back-annotate a part of a design for better accuracy and performance trade-off.
- You can perform power analysis based on the hierarchical ideal netlist to determine the power consumption of each block. Besides, you can reuse all post-processing statements for the pre-layout simulation for post-layout back-annotation simulation.

PrimeSim HSPICE enables parsing and annotating of two types of parasitic netlist formats:

- Standard Parasitic Format (SPF)
- Standard Parasitic Exchange Format (SPEF)

The parasitic netlist describes the interconnect delay and load due to parasitic resistance and capacitance. You can represent parasitics on a net-by-net basis from a simple lumped capacitance to a fully distributed resistance capacitance tree.

The following sections cover these topics:

- [Full Back-Annotation](#)
- [Selective Net Back-Annotation](#)
- [Warnings/Error Messages](#)
- [Listing of Back-Annotation Commands and Options](#)
- [Back-Annotation Cases](#)
- [DSPF and SPEF File Structures](#)

Full Back-Annotation

The Full Back-Annotation flow annotates all nets from the parasitic netlist, and thus can produce the most accurate simulation result. For a large case with enormous number of parasitic RCs the full back-annotation flow could take much time and memory for the simulation, in which case [Selective Net Back-Annotation](#) might be a better choice. To invoke full back-annotation, you must supply two types of input files: an ideal netlist and a parasitic netlist in the format of DSPF/SPEF.

The following sections discuss these topics:

- [Flow for Full Back-Annotation](#)
- [Example](#)

Flow for Full Back-Annotation

The option for invoking full back-annotation flow is `.OPTION BA_FILE`. For several examples of usage, see the demo cases and files shipped with PrimeSim HSPICE. Go to Back-Annotation Demo Cases in the *PrimeSim HSPICE User Guide: Demonstration Netlists* and follow the path to `$installdir/demo/hspice/back_annotation/.....`

The following cases illustrate the flow for full back-annotation:

<code>./option_ba_file/dspf/multiba</code>	Demonstrates use of the <code>.OPTION BA_FILE</code> command to launch multiple DSPF files for parasitic back-annotation.
<code>./option_ba_file/dspf</code>	Demonstrates use of the <code>.OPTION BA_FILE</code> command for single DSPF file.
<code>./option_ba_file/spef</code>	Demonstrates use of the <code>.OPTION BA_FILE</code> command for single SPEF file.

You need not specify which format (DSPF/SPEF) the parasitic netlist uses. PrimeSim HSPICE determines it automatically by analyzing the header of the parasitic netlist, so the file header must be clean to avoid adding confusing comments.

You can specify multiple parasitic netlists, each delimited by a semicolon. These parasitic netlists must be independent and *cannot* cross-reference each other.

Note:

PrimeSim HSPICE back annotation supports encryption of .DSPF and .SPF files.

For descriptions and usage examples, see .OPTION BA_FILE in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

Example

Sample case for Full Back-Annotation Flow

```
*$ Full Back Annotation example for Inverter Circuit for DSPF post layout
netlist $*
Vsupply Vdd33 0 3.3
Vground Vss33 0 0.0
.temp 25
*****HSPICE BA implementation*****
.option ba_file='rc.spi' $$ DSPF post layout netlist
*****
.inc 'sch.spi' $$ schematic netlist
vin in 0 pulse (0 3.3 0 100p 100p 2n 4n)
.lib 'model.1' TT 3V
.option nomod post
.tran 1p 300n
.probe tran v(out)
**measurement to check period of the clock
.measure tran t_PERIOD TRIG v(out) val=0.5 RISE=15 TARG v(out)
+ val=0.5 RISE=16
.end
```

Selective Net Back-Annotation

Enable selective net back-annotation to improve the performance in post-layout simulations that contain a large number of parasitic RCs and focus on the sensitive blocks. A selective net back-annotation only annotates the nets that are active during an initial simulation run. This reduces both back-annotation and simulation time and improves the overall performance with the cost of some loss of accuracy.

Selective BA supports the HSIMBA format.

You need three types of input files to invoke selective net back-annotation.

- An ideal netlist
- A parasitic netlist as used by full back-annotation
- An active net file, which can be in either of two formats: StarRC or HSPICE.

The following sections discuss these topics:

- [Flow for Selective Net Back-Annotation](#)
- [Example](#)
- [Active Net Generation](#)

Flow for Selective Net Back-Annotation

To invoke selective net back-annotation, specify both `.OPTION BA_FILE` and `BA_ACTIVE`.

The option for invoking the selective net back-annotation flow is `.OPTION BA_ACTIVE`. For examples of usage, see the demo cases and files shipped with PrimeSim HSPICE. Go to Back-Annotation Demo Cases and follow the path to `$installdir/demo/hspice/back_annotation/...` The following case illustrates the flow for selective net back-annotation:

`selective_ba/option_ba_active`

Demonstrates use of `.OPTION BA_ACTIVE` to specify the active net file name(s) for selective net back-annotation.

You do not need to specify the format of the active net file. PrimeSim HSPICE determines it automatically by analyzing its header. You can specify multiple active net files, each delimited by a semicolon.

Note:

For net names such as `module.xi1/xi2/net_name`, by default, PrimeSim HSPICE truncates this name from the last period and identifies the net name as `xi1/xi2/net_name`. To use the full net name, i.e., `module.xi1/xi2/net_name`, use the PrimeSim HSPICE control option `.OPTION BA_ACTIVEHIER`.

Example

The following case illustrates a Selective Net Back-Annotation Flow.

```
*$ Selective Net Back Annotation example for Inverter Circuit for DSPF
post layout netlist $*
Vsupply Vdd33 0 3.3
Vground Vss33 0 0.0
.temp 25
```

```
*****HSPICE BA implementation*****
.option ba_file='rc.spi' $$ DSPF post layout netlist
.option BA_ACTIVE='selective.rcxt' $$active net file
*****
.inc 'sch.spi' $$ schematic netlist
.lib 'model.1' TT_3V
cload out 0 10f
.option nomod converge=100
.tran lp 300n
.probe tran v(out)
**measurement to check period of the clock
.measure tran t_PERIOD TRIG v(out) val=0.5 RISE=15
+ TARG v(out) val=0.5 RISE=16
.end
```

Active Net Generation

An active net file is needed to run Selective Net Back-Annotation. This active net file used to be generated by other tools, such as Synopsys HSPICE. Now, PrimeSim HSPICE can generate active net files.

The following sections discuss these topics:

- [Examples of Active Net Files](#)
- [Flow for Active Net Generation](#)
- [Example](#)

Examples of Active Net Files

The content of active net file is case-insensitive for PrimeSim HSPICE.

Example 6 Active Net File sample in format of StarRC

```
NETS: A0
NETS: B0
NETS: A1
NETS: B1
NETS: X1/N6
```

Example 7 Active Net File sample in HSPICE format

```
NETS = {
  A0
  B0
  A1
  B1
  X1/N6
}
```

Flow for Active Net Generation

Use the `.BA_ACHECK` command to invoke Active Net Generation. The demo case below shows how to use this command. Go to Back-Annotation Demo Cases in the *PrimeSim HSPICE User Guide: Demonstration Netlists* and follow the path `$installdir/demo/hspice/back_annotation/`

The following case illustrates the flow for active net back-annotation:

<code>selective_ba/ba_acheck/</code>	Demonstrates the use of the <code>.BA_ACHECK</code> command to specify the rule for detecting node activity.
--------------------------------------	--

The default format for the active net file is HSPICE format; the file name is `design.hsimba0`. Another available format available is the StarRC format; the file name is `design.rcxt0`. To generate the StarRC format, use the PrimeSim HSPICE control option `.OPTION BA_NETFMT=1`.

For descriptions and usage examples, see `.BA_ACHECK` and `.OPTION BA_NETFMT` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

Example

The following case illustrates an Active Net Generation Flow:

```
*$ Selective Back Annotation example for TG BASED XOR Circuit for DSPF
post layout netlist $*

Vsupply vdd 0 1.2
Vground vss 0 0.0
.option runlvl
.temp 25
*****HSPICE BA Active Net implimentation*****
.ba_acheck dv=1 tstart=1u tstop=20u exclude='x*'
.option ba_netfmt=1 $ 0: HSPICE 1: StarRC
*****
.inc 'sch.spi' $$ schematic netlist
*xadder sum vss carry vdd in2 in1 adder
vin1 a 0 pulse (0 1.2 100p 100p 100p 2u 4u)
vin2 b 0 pulse (0 1.2 100p 100p 100p 3.5u 7u)
vclk clk 0 pw1 (0 1.2 23u 1.2 23.1u 0)
vclkb clkb 0 pw1 (0 0 23u 0 23.1u 1.2)
.option ba_activehier=1
.lib 'model.1' TT
.option nomod post
.tran 1p 30u
.probe tran v(out) v(a) v(b) v(clk) v(clkb)
.end
```

Warnings/Error Messages

PrimeSim HSPICE may issue warnings when doing back-annotation. The following are workarounds and solutions for common warning messages:

- Warnings for cutoff coupling capacitors—Coupling capacitors across two nets are very common in parasitic netlists. For example, assume one coupling capacitor (CC) with terminals connected to two nodes on nets A and B, respectively. When PrimeSim HSPICE launches selective net back-annotation and net A is active while net B is inactive, it cuts off CC from the node under net B and the terminal becomes a dangling node. PrimeSim HSPICE issues a low-level warning message as such as “`undefined node; might be defined in inactive net.`” By default, PrimeSim HSPICE gives such a warning and processes the cutoff terminal of CC as an ordinary dangling node. Additional modes are available in PrimeSim HSPICE to change the default behavior (see `.OPTION BA_COUPLING`). When you apply `.OPTION BA_COUPLING`, the warning changes to `undefined node; reset to GROUND_NET` or `undefined node; reset to pre_layout node: N1` according to the option setting.
- Warnings for terminal name mismatch—Cases where the terminal name used in the parasitic netlist is not consistent with the one used in the ideal netlist might generate a warning. For example: The terminal name for MOSFET in parasitic netlist is `M1:UDRN`, while the default terminal name for MOSFET recognized by PrimeSim HSPICE is `D[R][A][I][N]`. Another case is the terminal name in parasitic netlist `XM1:D`, while in the ideal netlist the corresponding subckt definition has a node list of `N1, N2, N3, N4`. Generally, PrimeSim HSPICE is able to correct this kind of mismatch automatically. If PrimeSim HSPICE cannot achieve this auto-correction, the tool gives a warning message such as: `cannot find the node pin`. To eliminate such warning messages, specify `.OPTION BA_TERMINAL` to explicitly map these terminals from the parasitic netlist to the ideal netlist.
- Warning for undefined PIN node—You must declare all pin nodes in a parasitic netlist. If you violate this rule, PrimeSim HSPICE warns `undefined node`, and all cuts off all connected element terminals to that node.
- Warning for missing instance—When the element instance referenced in the parasitic netlist cannot be found in the ideal netlist, PrimeSim HSPICE gives the warning message: `cannot find the instance`.
- Warning for invalid connectivity—If the node connectivity in the parasitic netlist conflicts with the connectivity in the ideal netlist, PrimeSim HSPICE corrects this error automatically and then gives the warning message `invalid connectivity; corrected now`.

Listing of Back-Annotation Commands and Options

The following is an alphabetical listing of all the back annotation command and options available in PrimeSim HSPICE. See the *PrimeSim HSPICE Reference Manual: Commands and Control Options* for details on each of these modes. For sample usage of these commands, see [Back-Annotation Cases](#).

- `.BA_ACHECK`: Specifies the rule for detecting node activity.
- `.OPTION BA_ACTIVE`: Specifies the active net file name(s) for selective net back-annotation.
- `.OPTION BA_ACTIVEHIER`: Annotates full hierarchical net names for `BA_ACTIVE` files.
- `.OPTION BA_ADDPARAM`: Specifies extra parameters for scale by `.OPTIONS BA_SCALE/BA_GEOSHRINK`.
- `.OPTION BA_COUPLING`: Controls how to treat cutoff coupling capacitors when invoking selective net back-annotation.
- `.OPTION BA_DPFPFX`: Prepends an extra prefix when searching the ideal netlist for referenced instances in the parasitic file (DSPF/SPEF/DPF).
- `.OPTION BA_DPF_ELEM_ENABLE`: Enables or disables back-annotation of instance section in all `ba_files`.
- `.OPTION BA_DPF_ELEM_TYPE`: Controls the element types you want to annotate during back-annotation.
- `.OPTION BA_ERROR`: Handles errors on nets.
- `.OPTION BA_EXTEND`: Option to back-annotate extra instances in the postlayout netlist.
- `.OPTION BA_FILE`: Launches parasitic back-annotation.
- `.OPTION BA_FINGERDELIM`: Explicitly specifies the delimiter character used for finger devices and subcircuit instances.
- `.OPTION BA_GEOSHRINK`: Acts as element scaling factor for `.OPTION BA_SCALE`.
- `.OPTION BA_HIERDELIM`: Specifies the hierarchical separator in the DPF file.
- `.OPTION BA_IDEALPFX`: Specifies the extra prefix to prepend when searching the ideal netlist for referenced instances in the parasitic file (DSPF/SPEF/DPF). For an example, see `.OPTION BA_IDEALPFX` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.
- `.OPTION BA_INST`: Specifies the scope of a back-annotation file to a particular subckt or instance.

- `.OPTION BA_MERGEPORT`: Controls whether to merge net ports into one node.
- `.OPTION BA_NETFMT`: Specifies the format of the active net file.
- `.OPTION BA_PRINT`: Controls whether to output nodes and resistors/capacitors introduced by back-annotation.
- `.OPTION BA_SCALE`: Sets the element scaling factor for instances in the DPF file separately.
- `.OPTION BA_TERMINAL`: Specifies the terminal name mapping between the parasitic netlist and the terminal names that the simulator recognizes.
- `.OPTION BA_WARNLIMIT`: Limits how many times BA warnings appear in the output listing.

Back-Annotation Cases

PrimeSim HSPICE provides for both full back-annotation and selective back annotation.

The following sections present these topics:

- [Full Back-Annotation](#)
- [Selective Back-Annotation](#)
- [Recommended Settings for STAR-RC-XT Command File](#)

Full Back-Annotation

For back-annotation in PrimeSim HSPICE, you need the following files:

- Schematic netlist
- Post-layout netlist

To perform back-annotation in PrimeSim HSPICE:

1. Specify the post-layout netlist file with:

```
.option ba_file="full_path_of_post_layout_netlist_file"
```

2. Include the schematic netlist as follows:

```
.include "schematic_netlist"
```

Result: PrimeSim HSPICE returns a back-annotation report in the *.lis file. Sample report:

In the following report, the number within the parentheses is the number parsed and the number outside the parentheses is the actual annotated figure.

```
***** begin BA *****
BA File="postlayout_netlist.spi"
*** BA Statistics ***
BA Nets back annotated (parsed): 6 (6)
BA Resistors back annotated (parsed): 204 (204)
BA Capacitors back annotated (parsed, coupled): 251 (251, 222)
BA Instances back annotated (parsed): 6 (6)
BA Time = 0.01 sec
***** end BA *****
```

The following sections present the syntaxes to handle these situations:

- [Multi Post-Layout Back-Annotation](#)
- [Back-Annotation for Stem Ports](#)
- [Output Control for Back-Annotation](#)
- [Scaling During Back-Annotation](#)
- [Terminal Mapping in Back-Annotation](#)
- [Prefix Handling for Devices of Post-layout Netlist](#)
- [Prefix Handling for Devices of a Pre-layout Netlist](#)
- [Delimiter Handling for Fingered Devices](#)
- [Handling the Hierarchy Separator](#)

Multi Post-Layout Back-Annotation

PrimeSim HSPICE back-annotation supports multi post-layout netlists at all levels of the hierarchy. You can specify multiple file by using `.option ba_file`, separated by a semicolon (;). This syntax is:

```
.option ba_file = "file1 [; file2; file3; ...]"
```

PrimeSim HSPICE modifies the reporting structure to issue a back-annotation report for an individual post-layout netlist, and then issues a separate back-annotation report.

```
***** begin BA *****
BA File="rcl.spi"

*** BA Statistics ***
BA Nets back annotated (parsed): 6 (6)
```



```
BA Resistors back annotated (parsed): 204 (204)
BA Capacitors back annotated (parsed, coupled): 251 (251, 222)
BA Instances back annotated (parsed): 6 (6)
BA File="rc2.spi"

*** BA Statistics ***
BA Nets back annotated (parsed): 4 (4)
BA Resistors back annotated (parsed): 78 (78)
BA Capacitors back annotated (parsed, coupled): 30 (30, 0)
BA Instances back annotated (parsed): 2 (2)

*** Multi-BA Summary ***
BA File="rc1.spi"
BA File="rc2.spi"
BA Nets back annotated (parsed): 10 (10)
BA Resistors back annotated (parsed): 282 (282)
BA Capacitors back annotated (parsed, coupled): 281 (281, 222)
BA Instances back annotated (parsed): 8 (8)
BA Time = 0.01 sec
***** end BA *****
```

Back-Annotation for Stem Ports

In a schematic, a pin or net which appears as a single connection between two points can actually be multiple parallel connections of metals in the layout.

This extraction is controlled by the `SHORT_PINS` command of STAR-RCXT. When you set this option to `NO`, the stem connects to a pin in the form of a bus.

Users employ this extraction feature for measuring stem currents. PrimeSim HSPICE supports the bus type pin and provides it with an option:

```
.option ba_mergeport= [1|0]
```

By default, the value is 1 which shorts the ports. To incorporate stems, set `.option ba_mergeport=0`.

Output Control for Back-Annotation

During back-annotation, PrimeSim HSPICE creates its own node names for parasitics. Because of the large number of parasitics, your search for a particular node or operating point in the waveform may become tedious. For ease of use, PrimeSim HSPICE supports storing only pre-layout nodes names by using:

```
.option ba_print = ideal
```

The default value for this option is `all` which includes parasitic nodes.

Scaling During Back-Annotation

PrimeSim HSPICE provides scaling for back-annotation using two options for back-annotated post layout netlists:

- `.option ba_scale`
- `.option ba_geoshrink`

In back-annotation (for a post layout netlist):

$$\text{final_dimension} = \text{original_dimension} \cdot \text{ba_scale} \cdot \text{ba_geoshrink}$$

These options, by default, are equal to `.option scale` and `.option geoshrink`, respectively.

There is a standard practice in PDKs for parameters that scale. However there is possibility of different parameter names, for example, instead of 'W' use of 'Wr', etc. To account for such situations, PrimeSim HSPICE provides:

```
.option ba_scaleparam="LINEAR: [param11 param12]...;  
+ QUAD: [param21 param22] ..."
```

For example:

The following commands PrimeSim HSPICE to scale WR, LR parameters linearly and ASR, AREAR parameters quadratic- ally.

```
.OPTION BA_SCALEPARAM = "LINEAR: WR LR; QUAD: ASR AREAR"
```

Terminal Mapping in Back-Annotation

The terminal name is a set of strings found in device parasitic information for the RC netlist, following the netlist delimiter. For example:

In this example Here DRN, GATE, SRC, and BULK are terminal names for the MOSFET.

```
Mmn1 mn1#DRN mn1#GATE mn1#SRC mn1#BULK nch3 ad=0.2016p  
+ as=0.2016p l=0.35u nrd=0 nrs=0 pd=1.8u ps=1.8u sa=0.48u  
+ sb=0.48u sca=8.27147 scb=0.004406 scc=3.6e-05 w=0.42u  
...
```

PrimeSim HSPICE provides terminal names for other elements, as well.

Note:

The letters in the square brackets are optional.

Device	Terminal 1	Terminal 2	Terminal 3	Terminal 4
M(MOSFET)	D [R] [A] [I] [N]	G [A] [T] [E]	S [O] [U] [R] [C] [E]	B [U] [L] [K]

Device	Terminal 1	Terminal 2	Terminal 3	Terminal 4
Q(BJT)	C [O] [L] [L] [E] [C] [T] B [A] [S] [E] [O] [R]		E [M] [I] [T] [T] [E] [R]	S [U] [B] [S] [T] [R] [A] [T] [E]
R(Resistor)	A [N] [O] [D] [E]	C [A] [T] [H] [O] [D] [E]	S [U] [B] [S] [T] [R] [A] [T] [E]	N/A
C(Capacitor)	P [L] [U] [S]	M [I] [N] [U] [S]	S [U] [B] [S] [T] [R] [A] [T] [E]	N/A
D(Diode)	P [O] [S] [I] [T] [I] [V] [E]	N [E] [G] [A] [T] [I] [V] [E]	S [U] [B] [S] [T] [R] [A] [T] [E]	N/A

When a terminal name may be different from those listed above for elements, PrimeSim HSPICE provides an option to map different user terminal names to what the simulator recognizes.

```
.option ba_terminal = "terminal_index1 alias1
+ [; terminal_index2 alias2; terminal_index3 alias3; ...]"
```

The following example maps user-defined terminals (UDRN, UGATE) in the parasitic netlist to default terminal characters (D, G).

```
.OPTION BA_TERMINAL="D UDRN ;G UGATE"
```

Prefix Handling for Devices of Post-layout Netlist

Sometimes, a post-layout netlist has device names with prefixes. For example, for a pre-layout device name such as M1, PrimeSim HSPICE may change it to M_mM1 in post-layout netlist. The PrimeSim HSPICE back-annotation engine tries to match such devices. However, you should explicitly include the prefix by using:

```
.option ba_dpfpfx = "prefix_string"
```

For the example above, you need to specify:

```
ba_dpfpfx = "M_"
```

Prefix Handling for Devices of a Pre-layout Netlist

Sometimes, a pre-layout netlist has device names with prefixes. For example, a pre-layout device name such as xmM1 may change to mM1 in a post-layout netlist.

The PrimeSim HSPICE back annotation engine tries to match such devices. However, you should explicitly include the prefix using:

```
.option ba_idealpfx = "prefix_string"
```

For the example above, you must specify:

```
.option ba_idealpfx = "xm"
```

Delimiter Handling for Fingered Devices

In a post-layout netlist, multi-fingered device names have delimiters followed by a number. For a pre-layout netlist device listed as:

```
MP1 d g s b w=1u l=1u M=3
```

In the post-layout, it may appear as:

```
MMP1 d g s b w=1u l=1u ...  
MMP1#1 d g s b w=1u l=1u ...  
MMP1#2 d g s b w=1u l=1u ...
```

where: '#' is the finger delimiter.

The PrimeSim HSPICE back annotation engine, by default, recognizes '@'. However, for other delimiters, you must explicitly include the same using:

```
.option ba_fingerdelim = "delimiter_string"
```

So, for above example, you have to specify:

```
.option ba_fingerdelim = "#"
```

Delimiter Handling for Fingered Subcircuit Instances

In a pre-layout netlist, when an instance is instantiated multiple times with M:

```
XSUB1 ... M=3
```

And the corresponding SPF file has the following entries:

```
XSUB1 ...  
XSUB1@1 ...  
XSUB1@2 ...
```

Then, PrimeSim HSPICE can recognize the fingered subcircuit instances and map to the pre-layout instance correctly.

Handling the Hierarchy Separator

Usually, a STARC DSPF file displays Hierarchy divider information as:

```
*|DIVIDER :
```

This notation plays a role in understanding hierarchy. However, in cases when a divider is not present, explicitly define the separator by using:

```
.option ba_hierdelim = "delimiter_string"
```

For example, for the colon (:) hierarchy separator (DIVIDER), specify:

```
.option ba_hierdelim = ":"
```

Selective Back-Annotation

Selective back-annotation allows you to selectively back-annotate the post-layout netlist per *active net* criteria. PrimeSim HSPICE uses active net, a set of nets / pins specified in *.rcxt or *.rc format for back-annotating an RC netlist.

For selective back-annotation in PrimeSim HSPICE, you need the following files:

- Schematic netlist
- Post-layout netlist
- Active net file (in .rcxt or .rc format)

To perform selective back-annotation in PrimeSim HSPICE:

1. Specify the post-layout netlist file with

```
option ba_file="full_path_of_post_layout_netlist_file"
```

2. Include the schematic netlist by using:

```
.include "schematic_netlist"
```

3. Specify the path of an active net file by using:

```
.option ba_active="active_net_file"
```

PrimeSim HSPICE writes the back-annotation report to the *.lis file as in the following sample report:

```
***** begin BA *****
Active Nets File="selective.rcxt"
BA File="rc.spi"
*** BA Statistics ***
BA Nets back annotated (parsed): 11 (11)
BA Resistors back annotated (parsed): 255 (255)
BA Capacitors back annotated (parsed, coupled): 100 (100, 0)
BA Instances back annotated (parsed): 22 (22)
BA Time = 0.01 sec
***** end BA *****
```

When you define tolerance change in voltage value of nets, you create the active net files by using the following syntax:

```
.BA_ACHECK [include=node_pattern] [exclude=node_pattern]
+ [dv=val1] [level=val2] [tstart=start_time] [tstop=stop_time]
```

where:

- `include` and `exclude` are useful options for specifying nets with wildcards.
- `dv` specifies the threshold of 'change of voltage'. So, if you specify `dv=0.1`, PrimeSim HSPICE considers all nets which vary more than 0.1 V as active nets.
- `level` causes the simulator to pick nets from the TOP level to the hierarchy level specified. For `level=2`, PrimeSim HSPICE considers active nets from the TOP level and immediate hierarchy level. By default, `level=0`, which means that HSPICE considers the full hierarchy.
- `tstart` and `tstop` define start and stop times, for which you want to perform an active net check. By default, these are the same as transient start and stop times.

PrimeSim HSPICE displays the report on created active net file as:

```
***** Active Net Information *****
Active Net file:hsp.rcxt
Active net threshold:1.00E+00v
#Active nets:68(7.83% of total nets)
*****
```

Active net files include embedded subckt instance names. By default, PrimeSim HSPICE creates an active net file in `.hsimba#` format, which is the HSPICE active net format.

To write the active net file in `.rcxt#` format, use:

```
.option ba_netfmt = 1
```

By default, PrimeSim HSPICE does not recognize module names and results in incorrect back-annotation. To correct such a situation, use:

```
.option ba_activehier = [0|1]
```

The default is 0; to correct problems due to use of a module instead of a subckt instance name, set:

```
ba_activehier=1
```

Coupling Capacitance Handling during Selective Back-Annotation

During selective back-annotation a coupling capacitance between active and inactive net may occur. By default, PrimeSim HSPICE does not connect the coupling cap of an active node to an inactive node and so leaves the node dangling.

PrimeSim HSPICE provides flexibility for the user to choose the connection by using:

```
.option ba_coupling = [0|1|2]
```

where:

- `.option ba_coupling = 0` (default) - Leaves the active net terminal of coupling capacitor under process as an ordinary dangling node.
- `.option ba_coupling = 1` - Connects the active net terminal coupling capacitance to the node which STAR-RC defines by `*|GROUND_NET` in the command file.
- `.option ba_coupling = 2` - Connects the active net terminal coupling capacitance to the unexpanded inactive node.

To achieve the most accurate results, set:

```
.option ba_coupling=2
```

Recommended Settings for STAR-RC-XT Command File

Besides the simulator, back-annotation depends on a STAR-RC-XT command file. Recommended settings for this file are as follows:

- `EXTRACTION: RC`
- `REDUCTION: NO`
- `COUPLE_TO_GROUND: YES`
- `XREF: YES`
- `POWER_EXTRACT: YES`
- `NETLIST_FORMAT: SPF`
- `NETLIST_FILE: output_filename`
- `NETLIST_INSTANCE_SECTION: YES`
- `NETLIST_CONNECT_SECTION: YES`
- `SHORT_PINS: NO`
- `NETLIST_GROUND_NODE_NAME: 0`
- `NETLIST_SUBCKT: YES`

DSPF and SPEF File Structures

The following sections discuss these topics:

- [DSPF File Structure](#)
- [SPEF File Structure](#)

DSPF File Structure

Open Verilog International (OVI) publishes the DSPF standard.

```
DSPF_file ::=
*|DSPF{version}
{*|DESIGN design_name}
{*|DATE date}
{*|VENDOR vendor}
{*|PROGRAM program_name}
{*|VERSION program_version}
{*|DIVIDER divider}
{*|DELIMITER delimiter}
.SUBCKT
*|GROUND_NET
{path divider} net_name
*|NET {path divider} net_name ||
{path divider} instance_name ||
pin_name
net_capacitance
*|P (pin_name pin_type
pinCap
{resistance {unit} {O}
capacitance {unit} {F}}
{x_coordinate y_coordinate})
||
*|I {path divider} instance_name
delimiter pin_name
{path divider} instance_name
pin_name pin_type
pinCap
{resistance {unit} {O}
capacitance {unit} {F}}
{x_coordinate y_coordinate}
*|S ({path divider} net_name ||
{path divider} instance_name
delimiter pin_name ||
pin_name
instance_number
{x_coordinate y_coordinate})
capacitor_statements
resistor_statements
subcircuit_call_statements
.ENDS
{.END}
```


SPEF File Structure

The IEEE-1481 specification requires the following file structure in a SPEF file. PrimeSim HSPICE only annotates the typical set (triple value SPEF file). Parameters in [brackets] are optional:

```
SPEF_file ::=
*SPEF version
*DESIGN design_name
*DATE date
*VENDOR vendor
*PROGRAM program_name
*VERSION program_version
*DESIGN_FLOW flow_type {flow_type}
*DIVIDER divider
*DELIMITER delimiter
*BUS_DELIMITER bus_prefix bus_suffix
*T_UNIT time_unit NS|PS
*C_UNIT capacitance_unit FF|PF
*R_UNIT resistance_unit OHM|KOHM
*L_UNIT inductance_unit HENRY|MH|UH
[*NAME_MAP name_index
name_id|bit|path|name|physical_ref]
[*POWER_NETS logical_power_net physical_power_net ...]
[*GROUND_NETS ground_net ...]
[*PORTS logical_port I|B|O
*C coordinate ...
*L par_value
*S rising_slew falling_slew [low_threshold high_threshold]
*D cell_type]
[*PHYSICAL_PORTS [physical_instance delimiter]
physical_port I|B|O
*C coordinate ...
*L par_value
*S rising_slew falling_slew [low_threshold high_threshold]
*D cell_type]
[*DEFINE logical_instance design_name |
*PDEFINE physical_instance design_name]
*D_NET net_path total_capacitance
[*V routing_confidence]
[*CONN
*P [logical_instance delimiter]
logical_port|physical_port
I|B|O
*C coordinate ...
*L par_value
*S rising_slew falling_slew
[low_threshold high_threshold]
*D cell_type
|
*I [physical_instance delimiter]
logical_pin|physical_node
```

```
I|B|O
*C coordinate ...
*L par_value
*S rising_slew falling_slew
[low_threshold high_threshold]
*D cell_type
*N net_name delimiter net_number coordinate
[*CAP cap_id node1 [node2] capacitance]
[*RES res_id node1 node2 resistance]
[*INDUC induc_id node1 node2 inductance]
*END
```

17

Multi-Technology Simulation of 3D Integrated Circuit

This chapter has been moved to the PrimeSim Continuum Overview. For more information, see Multi-Technology Simulation of 3D Integrated Circuit in the *PrimeSim™ Continuum Overview* guide.

18

Simulation of Stacked Devices

Describes the PrimeSim HSPICE stacked devices structure and its usage.

With ever changing demands and added complexity from the emerging advanced technology nodes, a new method to create stacked devices is introduced. This method can be used to meet designers' needs for ease of use and increased productivity when massive stacked devices are introduced in analog/RF designs from the emerging advanced technology nodes.

This chapter discusses the PrimeSim HSPICE stacked devices structure, its syntax, usage, and the related options.

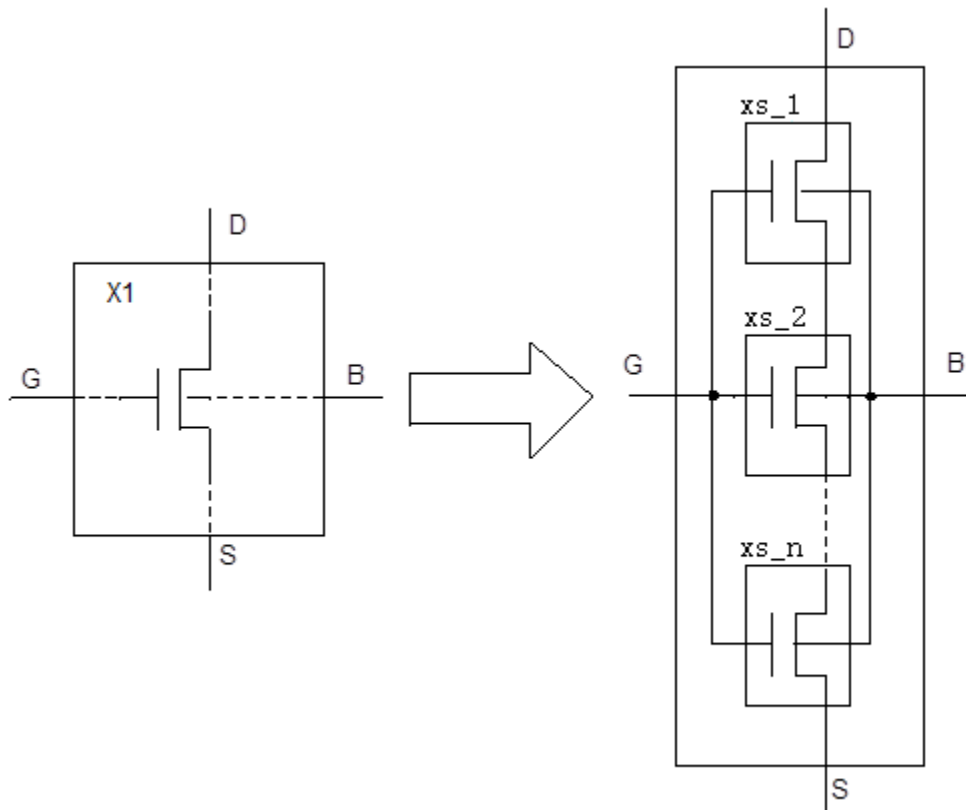
The following sections discuss these topics:

- [Stacked Devices Structure](#)
- [Syntax](#)
- [Supported Output Templates](#)
- [Calculation of IVTH](#)
- [Limitations](#)
- [Error Handling and Recovery](#)

Stacked Devices Structure

A stacked device is a marco subcircuit consisting of two or more series-connected MOSFETs or subcircuit instances. [Figure 90 on page 417](#) illustrates how a subcircuit instance is replicated and connected in series and becomes a new macro subcircuit instance.

Figure 90 Structure of a Stacked Device — An Example



Syntax

PrimeSim HSPICE provides the following commands and control options to support MOSFET (NMOS/PMOS) stacked devices:

- [MOSFET Stacked Devices](#)
- [.OPTION DEV_STACK](#)
- [.OPTION OUT_DEV_STACK](#)
- [Options SSDP and SSSP](#)
- [Instance Parameters SSDP and SSSP of MOSFET Stacked Device](#)
- [Parameter DLSX for IVTH Calculation](#)

MOSFET Stacked Devices

A new parameter NSX defines the number of devices to be stacked. It takes in an integer value or an expression of integer value greater than or equal to 1. It can be appended to any 4-terminal MOSFET or 4-terminal subcircuit instance. For 4-terminal subcircuit instances, it must consist of at least one MOSFET device.

```
Xxxx nd ng ns nb subckt_name [other_options...] [NSX=val]
Mxxx nd ng ns nb mos_name [other_options...] [NSX=val]
```

Sample Netlist

```
.option DEV_STACK = 1
.subckt sub1 d g s b ...
...
Msl nd ng ns nb mos_name ...
...
.ends
x1 d g s b sub1 L=L0 W=W0 NF=NF0 NSX=3
m1 d g s b nmos1 L=L0 W=W0 NF=NF0 NSX=3
```

Argument Descriptions

Argument	Description
<i>subckt_name</i>	Represents a 4-terminal subcircuit and has at least one MOSFET (NMOS/PMOS) device. In Sample Netlist , subcircuit "sub1" has 4 terminals and at least one MOSFET device 'Ms1'.
<i>mos_name</i>	Represents a 4-terminal MOSFET (NMOS/PMOS) device. In Sample Netlist , MOSFET device "nmos1" has 4 terminals, d, g, s, and b.
<i>NSX=val</i>	Number of devices to be stacked; is enabled when <code>.option DEV_STACK</code> is set (to 1). <i>val</i> can be constant integer or expression of integer value; NSX should be ≥ 1 .

When `.option DEV_STACK` is set (to 1), PrimeSim HSPICE elaborates devices `Xxxx` and `Mxxx` according to the given number of devices to be stacked `NSX`.

In [Sample Netlist](#), the number of devices to be stacked is 3.

Note:

- For FinFET technologies (BSIM-CMG) with this feature, the value of the instance parameter NF can only be 1. If NF>1, PrimeSim HSPICE reports an error message and aborts the simulation.
- For FinFET technologies (BSIM-CMG) with this feature, the value of the instance parameter M (multiplier) is $M = \text{multi} \times \text{NF}$. M is handled on the top of the stacked macro device.

.OPTION DEV_STACK

Enables MOSFET (NMOS/PMOS) stacked devices.

```
.OPTION DEV_STACK [= 0|1]
```

Default: Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

- **0:** Default value. Disables the number of devices to be stacked `NSX` keyword. `NSX` on MOSFET device is treated as common parameter and additional options described for stacked devices become ineffective.
- **1:** Enables the number of devices to be stacked `NSX` keyword. `NSX` on MOSFET device is treated as the number of devices to be stacked. The MOSFET device with the `NSX` keyword is elaborated according to the numerical number assigned to `NSX`.
- **No value:** A value of 1 is assigned to `.OPTION DEV_STACK` to enable the number of devices to be stacked `NSX`.

For detailed information on this option, see `.OPTION DEV_STACK` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

.OPTION OUT_DEV_STACK

Prints the information of stacked devices when `.option dev_stack` is also set to 1.

```
.OPTION OUT_DEV_STACK [=0|1]
```

Default: Value if option is not specified in the netlist: 0; Value if option name is specified without a corresponding value: 1

- **0:** Default value. No information of stacked devices is printed.
- **1:** When the option `OUT_DEV_STACK` is set to 1 (or specified without a value), an expanded structure of the stacked device is printed in a file with extension `.stack#`, where # corresponds to the n^{th} `.ALTER` block. The operating points of the internal nodes of the stacked device are also printed to either the `*.lis`, `*.dp#`, or the `*.op#`

file and saved to waveform output files depending on output file type and analysis performed.

- No value: A value of 1 is assigned to `.OPTION OUT_DEV_STACK`.

For detailed information on this option, see `.OPTION OUT_DEV_STACK` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

Example of a *.stack0 File

```
*** Summary of Stacked Device ***
Original:  x1 d g s b sub1 L=L0 W=W0 NF=NF0 NSX=3
Result:    x1.xs_1 d g x1.s1 b sub1 L=L0 W=W0 NF=NF0
           x1.xs_2 x1.s1 g x1.s2 b sub1 L=L0 W=W0 NF=NF0
           x1.xs_3 x1.s2 g s b sub1 L=L0 W=W0 NF=NF0
```

Options SSDP and SSSP

```
.OPTION SSDP="ndparam1=ndvalue1 [... ndparamN=ndvalueN] "
.OPTION SSSP="nsparam1=nsvalue1 [... nsparamN=nsvalueN] "
```

You can use these options to pass device-specific parameters and values to devices inside the stacked devices. These options are global options and all the stacked devices will share the same parameter-value pairs as defined by these options.

Argument	Description
<i>ndparamn</i>	Specifies the parameters for node D of all MOSFET stacked devices in the netlist.
<i>ndvaluen</i>	Specifies the value of parameter <i>ndparamn</i> ; can be a constant, a parameter, or an expression.
<i>nsparamn</i>	Specifies the parameters for node S of all MOSFET stacked devices in the netlist.
<i>nsvaluen</i>	Specifies the value of parameter <i>nsparamn</i> ; can be a constant, a parameter, or an expression.

Sample Netlist:

```
.option SSSP="sp1=sval1 sp2=sval2"  SSDP="dp1=dval1 dp2=dval2" $where
value* can be parameters
x1 d g s b n1 L=L0 W=W0 NF=NF0 NSX=n... $NSX: number of stacked
```

PrimeSim HSPICE Parser Result:

```
x1.xs_1 d g x1.s1 b n1 L=L0 W=W0 NF=NF0 sp1=sval1 sp2=sval2
x1.xs_2 x1.s1 g x1.s2 b n1 L=L0 W=W0 NF=NF0 sp1=sval1 sp2=sval2 dp1=dval1
dp2=dval2
```



```
...
X1.xs_n-1 x1.sn-2 g x1.sn-1 b n1 L=L0 W=W0 NF=NF0 sp1=sval1 sp2=sval2
  dp1=dval1 dp2=dval2
X1.xs_n x1.sn-1 g s b n1 L=L0 W=W0 NF=NF0 dp1=dval1 dp2=dval2
```

For detailed information on these options, see .OPTION SSDP and .OPTION SSSP in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

Instance Parameters SSDP and SSSP of MOSFET Stacked Device

You can also use the SSDP and the SSSP arguments to define instance parameters (locally) of a specified MOSFET stacked device. Local instance parameters override the SSDP and SSSP global options.

```
Xxxx nd ng ns nb subckt_name [other_options...] [NSX=val]
+ [SSDP="ndparam1=ndvalue1 [... ndparamN=ndvalueN]"]
+ [SSSP="nsparam1=nsvalue1 [... nsparamN=nsvalueN]"]

Mxxx nd ng ns nb mos_name [other_options...] [NSX=val]
+ [SSDP="ndparam1=ndvalue1 [... ndparamN=ndvalueN]"]
+ [SSSP="nsparam1=nsvalue1 [... nsparamN=nsvalueN]"]
```

Sample Netlist:

```
.option DEV_STACK = 1
.subckt sub1 d g s b ...
...
Msl nd ng ns nb mos_name ...
...
.ends
x1 d g s b sub1 L=L0 W=W0 NF=NF0 NSX=n SSSP="sp1=sval1 sp2=sval2"
  SSDP="dp1=dval1 dp2=dval2"
$where value* can be parameters, NSX: number of stacked
x2 d g s b sub1 L=L0 W=W0 NF=NF0 NSX=3
```

PrimeSim HSPICE Parser Result:

```
x1.xs_1 d g x1.s1 b sub1 L=L0 W=W0 NF=NF0 sp1=sval1 sp2=sval2
x1.xs_2 x1.s1 g x1.s2 b sub1 L=L0 W=W0 NF=NF0 sp1=sval1 sp2=sval2
  dp1=dval1 dp2=dval2
...
x1.xs_n-1 x1.sn-2 g x1.sn-1 b sub1 L=L0 W=W0 NF=NF0 sp1=sval1 sp2=sval2
  dp1=dval1 dp2=dval2
x1.xs_n x1.sn-1 g s b sub1 L=L0 W=W0 NF=NF0 dp1=dval1 dp2=dval2

x2.xs_1 d g x2.s1 b sub1 L=L0 W=W0 NF=NF0
x2.xs_2 x2.s1 g x2.s2 b sub1 L=L0 W=W0 NF=NF0
x2.xs_3 x2.s2 g s b sub1 L=L0 W=W0 NF=NF0
```

Parameter DLSX for IVTH Calculation

You can define the `DLSX` parameter in both `.IVTH` and in a subcircuit instance definition.

```
.IVTH model_name Ivth0=val DW=val DL=val VDSMIN=val
+ [DLSX=value]
X1 d g s b n1 L=L0 W=W0 NF=NF0 NSX=n [DLSX=value]
```

- The `DLSX` parameter can be defined in both `.IVTH` and in a subcircuit instance definition. When it is defined in the subcircuit instance, it is valid only when there is `NSX` defined on the same subcircuit instance.
- The value given in the subcircuit instance overrides the value in `.IVTH` for each stacked device.
- When not defined in the subcircuit instance, the value from `.IVTH` is used. The default value of `DLSX` is 0.0 in `.IVTH`.
- The value of `DLSX` can be negative or positive.
- If the denominator of the IVTH equation (see [Calculation of IVTH](#)) goes to 0 or becomes negative, PrimeSim HSPICE displays an error message.
- If the absolute value of the denominator of the IVTH equation is too small (for example, $<1e-9$), PrimeSim HSPICE issues a warning message.

Supported Output Templates

Same output template syntax as MOSFET wrapper is supported.

For example:

```
LX0 (X*) / VBD (X*)           LX0 (M*) / VBD (M*)
```

Calculation of IVTH

The constant current for the stacked device is given in the following equation:

$$Ivth = Ivth0 * ((W * SX_factor + DW) / (NSX * (L * SX_factor + DL) + DLSX))$$

Internal devices in the stacked device macro have the same length and width (NFIN). W is the Weff from any one of them. $L=L0$ and `.OPTION SX_factor=x` (default 1.0) is the scale for W/L for IVTH definition.

Limitations

- Only 4-terminal MOSFET devices are supported.
- Only 4-terminal subcircuit devices are supported with following limitations:
 - There must be *at least one MOSFET* within the subcircuit.
 - The model cards of the MOSFET devices within the subcircuit can be different but their type must be same (NMOS/PMOS).
 - For IVTH calculation (see .IVTH in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*), all MOSFETs within the subcircuit must have the same model card.
 - The subcircuit cannot include U, W, T, S, B, P, and VEC elements.
 - Not supported in the Variation Block (see .VARIATION in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*).

Error Handling and Recovery

- Invalid to use `NSX` as the number of devices to be stacked on an X element when `.option DEV_STACK` is not set; `NSX` will be treated as a common parameter.
- Invalid to use `NSX` as the number of devices to be stacked on an M element when `.option DEV_STACK` is not set; `NSX` will be treated as a common parameter.
- Invalid to use `NSX` on non-4-terminal MOSFET devices.
- Invalid to use `NSX` on non-4-terminal subcircuit instances.
- For IVTH equation, PrimeSim HSPICE issues an error message if the denominator goes to 0 or negative.
- For IVTH equation, PrimeSim HSPICE issues a warning message if the absolute value of the denominator is too small (for example, <1e-9)

19

Random Telegraph Noise Model and Simulation

The following sections cover these topics:

- [Random Telegraph Noise Overview](#)
- [Example](#)

Random Telegraph Noise Overview

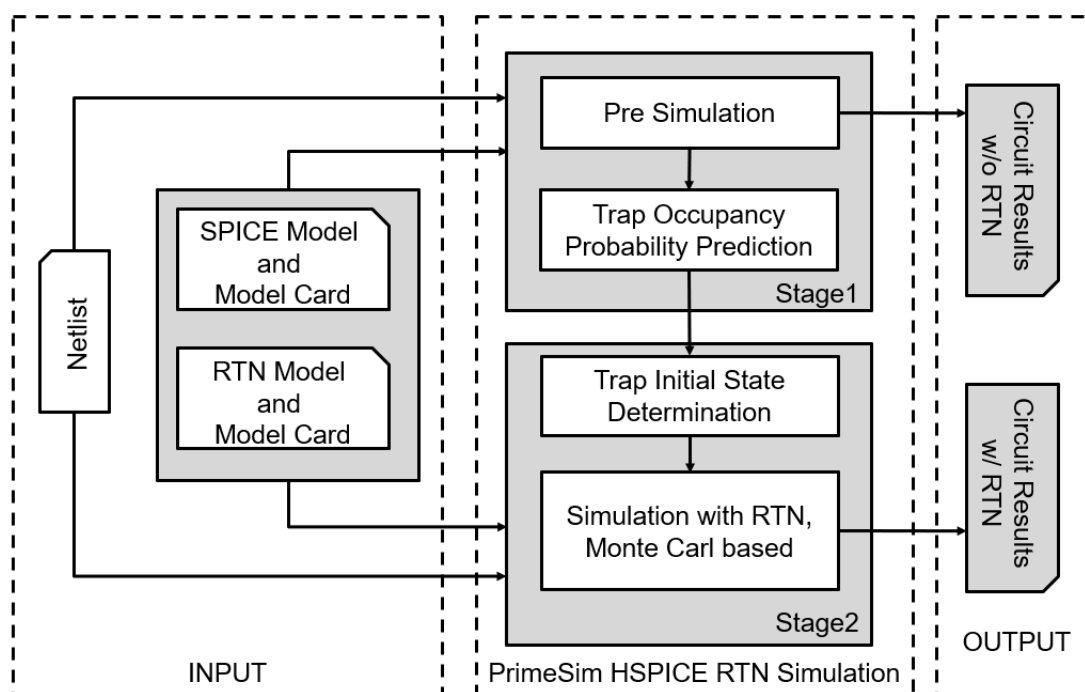
Random telegraph noise (RTN) in a MOSFET originates from the stochastic trapping/de-trapping of the channel carrier into/from the switching oxide traps in the gate dielectrics, which is a Markov process between two states (trap empty or occupied). RTN manifest its impact to MOSFET characteristic as $\Delta I_D/I_D$ or equivalent ΔV_{TH} changes. The amplitude of MOSFET characteristic change due to RTN increases with device size scaling down, therefore the design reliability concern by RTN increases in advanced technology nodes where smaller headroom ($V_{dd}-V_{th}$) and stringent design margin are present.

The compact model of RTN needs to consider the statistical distribution of trap energy and position, as well as the bias and frequency dependencies. The simulation of RTN needs to account for the stochastic process of trap emission and capture. Simulation of RTN effect in large scale IC design is challenging because RTN traps can have wide range of time constants (nano seconds to days). The transient simulation time step needs to be sufficiently small for the RTN with shortest time constant; alternatively, the total simulation time need to be long enough to include traps with longer time constants.

The methodology of RTN modeling and simulation in PrimeSim HSPICE is developed based on two papers 1) *Too Noisy at the Bottom? Random Telegraph Noise (RTN) in Advanced Logic Devices and Circuits, IEDM 2018*; and 2) *OMI/TMI-based Modeling and Fast Simulation of Random Telegraph Noise (RTN) in Advanced Logic Devices and Circuits, ASICON 2019*.

The RTN model is implemented utilizing CMC OMI API and interoperate with BSIM-CMG model in PrimeSim HSPICE. It can be extended the any native MOSFET model in PrimeSim. Two-step simulation scheme, as shown in [Figure 91](#), together with the trap occupancy probability prediction model (OPP) and embedded Monte Carlo method, is adopted to enable the efficient and accurate simulation of RTN impact on circuit functionality and timing power characteristic.

Figure 91 RTN simulation framework in PrimeSim HSPICE



Example

A simple example is provided in this section. For further details about the RTN model and simulation setup, contact Synopsys Technical Support Center.

The example is based on the method described in paper *OMI/TMI-based Modeling and Fast Simulation of Random Telegraph Noise (RTN) in Advanced Logic Devices and Circuits*. The RTN model is implemented using CMC OMI API and OMI-age two-step simulation.

The random variables represent process variation that impact the trap number, trap energy level, trap amplitude and trap time constant in each MOSFET. Uniform distribution is used in RTN model card, the RTN model in OMI convert the uniform distribution to the required distribution form for each trap property as described in the papers.

Example RTN model card

```

Model lib
.LIB RTN
.param
+ rtn_par0=aunif(0.5,0.5,1)   rtn_par1=aunif(0.5,0.5,1)
+ rtn_par2=aunif(0.5,0.5,1)   rtn_par3=aunif(0.5,0.5,1)
+ rtn_par4=aunif(0.5,0.5,1)   rtn_par5=aunif(0.5,0.5,1)
  
```

```
+ rtn_par6=aunif(0.5,0.5,1)    rtn_par7=aunif(0.5,0.5,1)
+ rtn_par8=aunif(0.5,0.5,1)    rtn_par9=aunif(0.5,0.5,1)
+ rtn_par10=aunif(0.5,0.5,1)   rtn_par11=aunif(0.5,0.5,1)
+ rtn_par12=aunif(0.5,0.5,1)   rtn_par13=aunif(0.5,0.5,1)
+ rtn_par14=aunif(0.5,0.5,1)   rtn_par15=aunif(0.5,0.5,1)
+ rtn_par16=aunif(0.5,0.5,1)   rtn_par17=aunif(0.5,0.5,1)
+ rtn_par18=aunif(0.5,0.5,1)   rtn_par19=aunif(0.5,0.5,1)
+ rtn_par20=aunif(0.5,0.5,1)   rtn_par21=aunif(0.5,0.5,1)
+ rtn_par22=aunif(0.5,0.5,1)   rtn_par23=aunif(0.5,0.5,1)
+ rtn_par24=aunif(0.5,0.5,1)   rtn_par25=aunif(0.5,0.5,1)
+ rtn_par26=aunif(0.5,0.5,1)   rtn_par27=aunif(0.5,0.5,1)
+ rtn_par28=aunif(0.5,0.5,1)   rtn_par29=aunif(0.5,0.5,1)
+ rtn_par30=aunif(0.5,0.5,1)   rtn_par31=aunif(0.5,0.5,1)
+ rtn_par32=aunif(0.5,0.5,1)   rtn_par33=aunif(0.5,0.5,1)
+ rtn_par34=aunif(0.5,0.5,1)   rtn_par35=aunif(0.5,0.5,1)
+ rtn_par36=aunif(0.5,0.5,1)   rtn_par37=aunif(0.5,0.5,1)
+ rtn_par38=aunif(0.5,0.5,1)   rtn_par39=aunif(0.5,0.5,1)
.ENDL RTN

.LIB MOS
.option omiflag=1 modmonte=1 omipath='../code/lib' omiinput='omi.cfg'
.param omirtnflag_mos=0 dagetime=0
.model nch nmos level=72 version=110
+ omimodel=1 omimod=1
+ par0='rtn_par0' par1='rtn_par1' ...
+ omirtnflag='omirtnflag_mos' nrand=30 dagetime='dagetime'
...
```

Two netlists with same circuit configuration and SPICE models are required. In step 1 simulation, RTN model computes trap occupancy probability prediction (OPP) model based on the bias waveforms each device experiences. The OPP model is stored in the *.omiage output generated by the step 1 simulation. Step 2 simulation read backs the OPP model, compute each trap's initial state at the user specified operation time of the circuit, and invokes Monte Carlo simulation to simulate the stochastic circuit responds considering RTN impact starting from the operation time. Related options and settings for RTN simulation are as follows:

- `.option RTNMOD`: Flag to invoke RTN simulation. 1: RTN on. 0 (default): RTN off
- `.option seed`: Seed for random number generation for RTN trap modeling (default seed=1). Same seed need to be set for step 1 and 2 simulations.
- `.option omiInput`: OMI option to load OPP model for step 2 simulation
- `.param dagetime`: Parameter to specify operation time in step 2 simulation (default dagetime=0)

- Model parameter `omiRtnFlag`: Turn on/off RTN effect for a MOS model. 1: RTN on. 0 (default): RTN off
- Instance parameter `omirtnflag`. Turn on/off RTN effect for MOS instance. 1: RTN on; 0: RTN off. Default is model parameter `omiRtnFlag`.

The example section describes example of step 1 and 2 RTN simulation netlist, respectively.

Step1 RTN Netlist

```
* Two stage RTN Simulation: stage 1
.option RTNMOD=1
.option seed=2
.lib './modelcard.1' MOS
.lib './modelcard.1' RTN
.option post

m1 d g 0 0 nch l=30n nfin=10 tfin=15n nrd=1 nrs=1

vgs g 0 0.72
vds d 0 0.05
.tran 1e-6 600e-6
.print i(vds)
.end
```

Step2 RTN Netlist

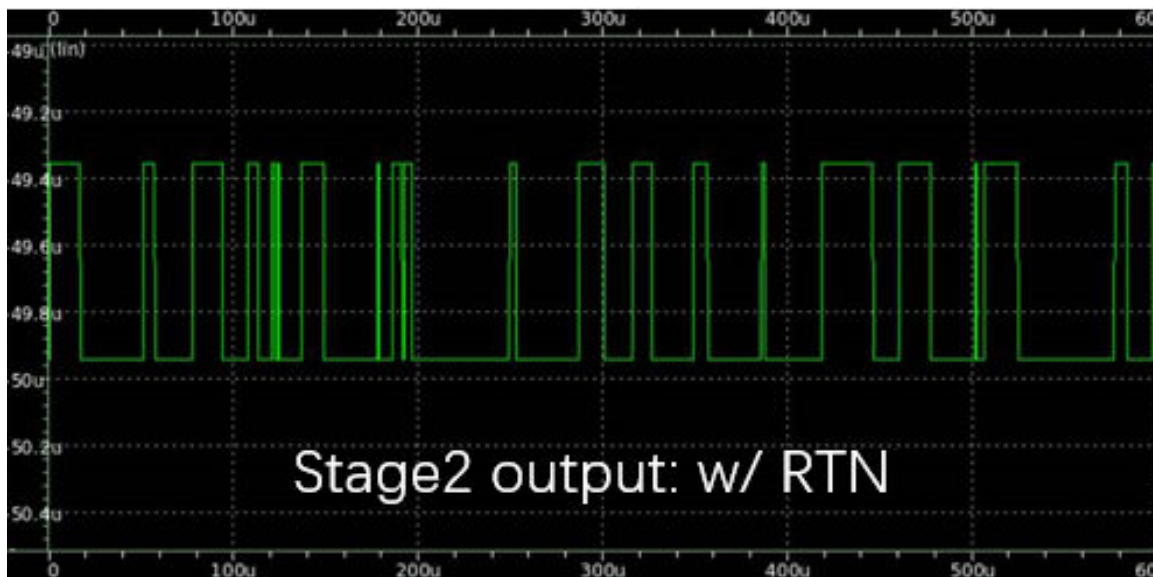
```
* Two stage RTN Simulation: stage 2
.option RTNMOD=1
.option seed=2
.lib './modelcard.1' MOS
.lib './modelcard.1' RTN
.option post
.option omiinput='rtn1.omiage0'
.param dagetime=1 omirtnflag_mos=1
m1 d g 0 0 nch l=30n nfin=10 tfin=15n nrd=1 nrs=1 omirtnflag=1
vgs g 0 0.72
vds d 0 0.05
.tran 1e-6 600e-6
.print i(vds)
.end
```

[Figure 92](#) and [Figure 93](#) is an example waveform of single MOSFET for w/ and w/o RTN.

Figure 92 Example Waveforms of Without RTN for Single MOSFET



Figure 93 Example Waveforms of With RTN for Single MOSFET



20

Data-To-Design (D2D)

PrimeSim HSPICE Data-to-Design feature enables the circuit simulation in technology pathfinding, for special device(s) that lack compact model. For further details, contact Synopsys Support .

21

Simulation of Thermal Effect in Macro Models

This chapter describes how to enable the thermal effect in macro models.

This chapter covers the following topics:

- [Macro Model Extension to Enable Thermal Effect](#)
- [Syntax](#)
- [Example](#)

Macro Model Extension to Enable Thermal Effect

To enable the thermal effect, you need to provide an additional thermal node and the related R/C thermal network in subcircuit definition. Simulator calculates the subcircuit temperature change based on the total power consumed by the subcircuit. Temperature change is played back to device(s) which have temperature dependencies.

Syntax

PrimeSim HSPICE provides the following commands and control options to support the enabling of thermal effect in subcircuit macro model:

- [.SET_NODE_TYPE](#)
- [.OPTION ENABLE_SUBCKT_SHE](#)
- [SUB_SHE_TEMP\(\)](#)

.SET_NODE_TYPE

Sets the device node type. This command is used only in subcircuit to declare thermal node for subcircuit thermal effect. Components connected to thermal node are taken as thermal network.

```
.SET_NODE_TYPE node = 'node_name' type = 'node_type'
```

Argument Descriptions

Argument	Description
<i>node</i>	Specifies the node name.
<i>type</i>	Specifies the node type, only 'thermal' is supported.

Example

Node 'tn' is set as a thermal node.

```
.set_node_type node='tn' type='thermal'
```

For detailed information on this option, see .SET_NODE_TYPE in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

.OPTION ENABLE_SUBCKT_SHE

Controls the subcircuit thermal effect simulation.

```
.OPTION ENABLE_SUBCKT_SHE [MODE=0|1|2 SHE_TEMP_TOL=value SUBCKT
='SUBCKT_NAME1 SUBCKT_NAME2 ...' SUBCKT_EXCEPT ='SUBCKT_NAME1 SUBCKT_NAME2
...']
```

Argument Descriptions

Argument	Description
<i>MODE</i>	0: Deactivate thermal effect for all subcircuits. 1 (default): Faster mode. Keep device status unchanged, if the subcircuit temperature change is less than the tolerance set by <i>SHE_TEMP_TOL</i>. 2: Normal mode. Always update the device temperature at each time step.
<i>SHE_TEMP_TOL</i>	Specifies the temperature change tolerance. Valid only when <i>mode</i> =1. Default = 1 (°C).
<i>SUBCKT</i>	Selects the subcircuit (s) to enable the thermal effect. Default: all subcircuits contain thermal node and thermal network. Wildcard supported. Valid only when <i>mode</i>=1 2.
<i>SUBCKT_EXCEPT</i>	Selects the subcircuit (s) to disable the thermal effect. Default is NULL. Wildcard supported. Valid only when <i>mode</i>=1 2.

Example

```
.option enable_subckt_she mode=1 she_temp_tol=2 subckt='sub*'
subckt_except='sub1'
```

For detailed information on this option, see .OPTION ENABLE_SUBCKT_SHE in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

SUB_SHE_TEMP()

A reserved function to get the subcircuit temperature change due to thermal effect. Only 0 is supported as function input parameter.

Example

```
.subckt pd_dev d g s b tn she_temp=sub_she_temp(0)
.set_node_type node='tn' type='thermal'
m1 d g s b dtemp=she_temp
g1 d s cur=f(she_temp,...)
rth tn 0 r_value
cth tn 0 c_value
.ends
```

- 'tn' is defined as a thermal node. You/modeler need to modify the subcircuit declaration to explicitly define the thermal node, by using the command `.set_node_type`.
- she_temp is defined as a subcircuit instance parameter to apply the thermal effect to the component (s) inside the subcircuit. You/modeler can define the parameter name.
- rth and cth are connected to the thermal node 'tn' to form the thermal network. User/modeler needs to add this auxiliary circuitry into the subcircuit and extract the corresponding thermal time constant value.
- Temperature change due to thermal effect is retrieved though reserved function `sub_she_temp()`.

Note:

Only one thermal node is permitted in a single subcircuit.

Part 4: Variation, Optimization, and Statistical Analysis

Describes the variation, optimization and statistical analysis.

Part 4 presents the following chapters/topics:

- [Performing Digital Cell Characterization](#)
- [Timing Analysis Using Bisection](#)
- [Variation Analysis](#)
- [Statistical Representation Using the Variation Block](#)
- [Monte Carlo Analysis—Flow and Outputs](#)
- [Mismatch Analyses](#)
- [Exploration Block](#)
- [Optimization](#)
- [Model Parameter Optimization](#)

22

Performing Digital Cell Characterization

Describes how to characterize cells in data-driven analysis and shows some typical data sheet parameters.

PrimeSim HSPICE ships numerous examples for your use; see Cell Characterization Examples in the *PrimeSim HSPICE User Guide: Demonstration Netlists* for paths to the demo files discussed in this chapter.

Most ASIC vendors use the basic capabilities of the `.MEASURE` statement in Synopsys PrimeSim HSPICE to characterize standard cell libraries, and to prepare data sheets.

PrimeSim HSPICE stores input sweep parameters and measure output parameter, in measure output data files (*design.mt0*, *design.sw0*, and *design.ac0*). These files store multiple sweep data. You can use WaveView to plot this data; for example, to generate fanout plots of delay versus load. You can also use the slope and intercept of the loading curves to calibrate VHDL, Verilog, Lsim, and Synopsys models.

This chapter shows you some typical data sheet parameters. A series of typical data sheet examples, demonstrates the flexibility of the `.MEASURE` statement.

This chapter also shows you how to characterize cells in data-driven analysis. Data-driven analysis automates cell characterization, including calculating the delay coefficient for the timing-simulator polynomial. You can simultaneously vary an unlimited number of parameters, or the number of analyses to perform. Cell characterization uses an ASCII file format for automated parameter input to PrimeSim HSPICE.

The following sections discuss these topics:

- [Performing Basic Cell Measurements](#)
- [Performing Advanced Cell Characterization](#)
- [Monte Carlo Plus Data Sweep in Transient Simulations](#)
- [Cell Characterization Examples](#)

Performing Basic Cell Measurements

The following sections describe how to perform basic cell measurements:

- [Rise, Fall, and Delay Calculations](#)
- [Delay versus Fanout](#)
- [Pin Capacitance Measurement](#)
- [Op-amp Characterization of LM124](#)

Rise, Fall, and Delay Calculations

The following example does the following:

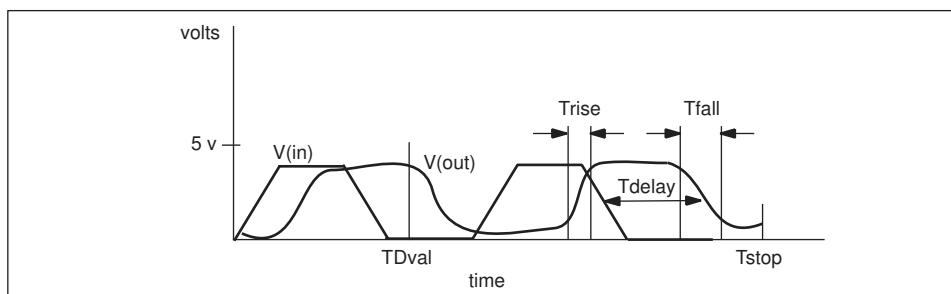
- Uses the `MAX` function to calculate `vmax`, over the time region of interest.
- Uses the `MIN` function to calculate `vmin`.
- Uses the measured parameters in subsequent calculations, for accurate 10 percent and 90 percent points, when determining the rise and fall time.

`RISE=1` is relative to the time window that the `TDval` delay forms.

- Uses a fixed value for the measure threshold, to calculate the `Tdelay` delay.

```
.MEAS TRAN vmax MAX V(out) FROM=TDval TO=Tstop
.MEAS TRAN vmin MIN V(out) FROM=TDval TO=Tstop
.MEAS TRAN Trise TRIG V(out) val='vmin+0.1*vmax'
+ TD=TDval RISE=1 TARG V(out) val='0.9*vmax' RISE=1
.MEAS TRAN Tfall TRIG V(out) val='0.9*vmax' TD=TDval
+ FALL=2 TARG V(out) val='vmin+0.1*vmax' FALL=2
.MEAS TRAN Tdelay TRIG V(in) val=2.5 TD=TDval FALL=1
+ TARG V(out) val=2.5 FALL=2
```

Figure 94 Rise, Fall, and Delay Time Demonstration



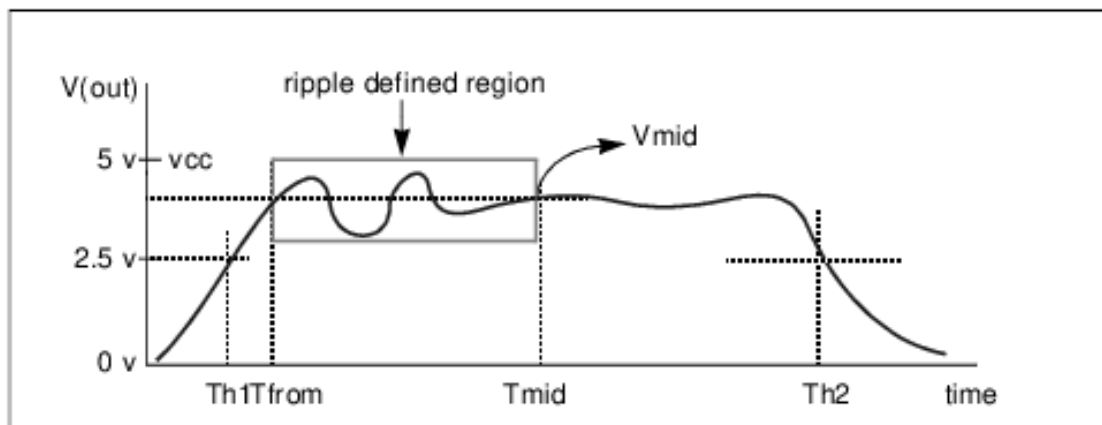
Ripple calculation performs the following:

- Delimits the wave at the 50 percent of VCC points
- Finds the T_{mid} midpoint
- Defines a bounded region by finding the pedestal voltage (V_{mid}) and then finding the first time that the signal crossed this value, T_{from}
- Measures the ripple in the defined region using the peak-to-peak (PP) measure function from T_{from} to T_{mid}

The following is an example:

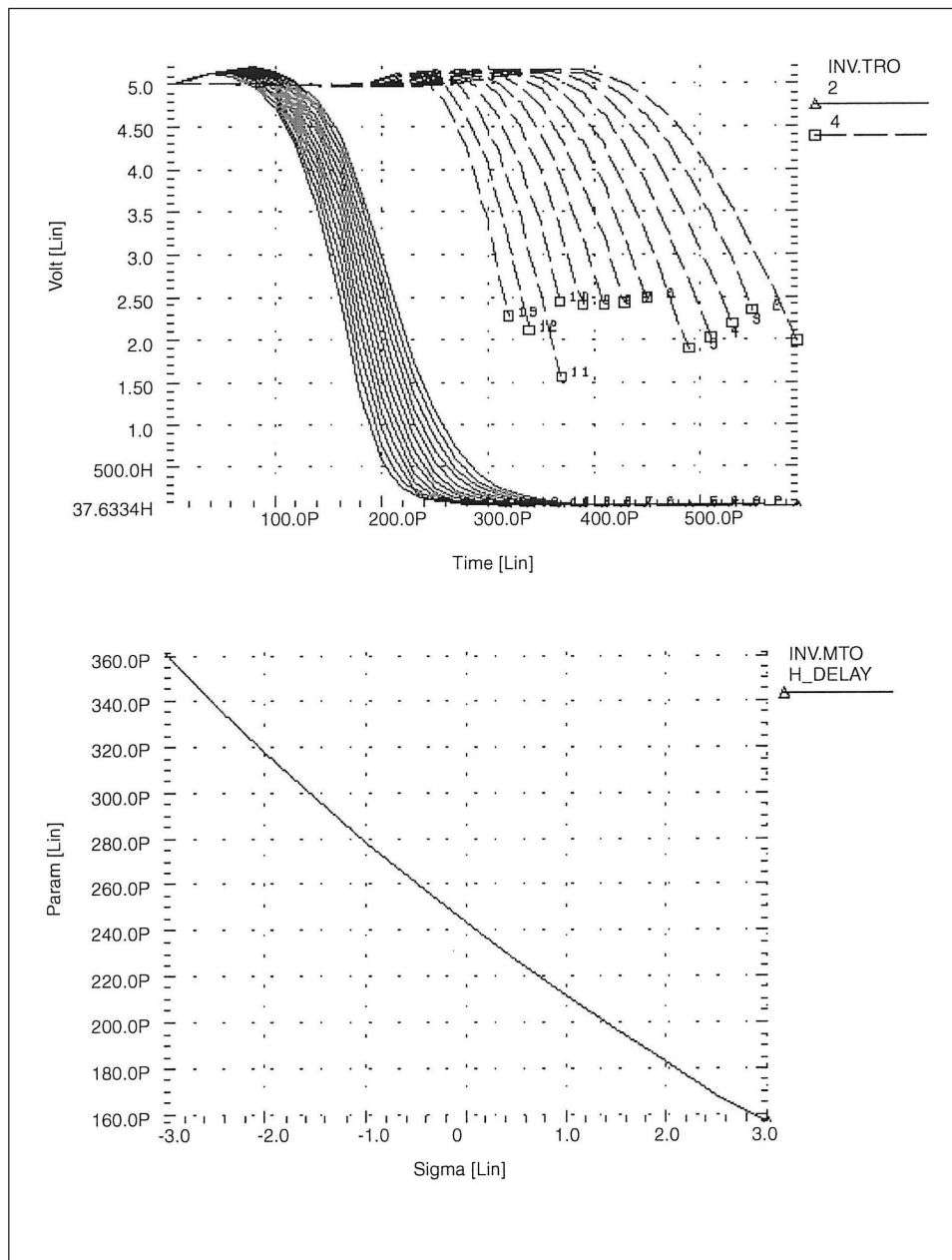
```
.MEAS TRAN Th1 WHEN V(out)='0.5*vcc' CROSS=1
.MEAS TRAN Th2 WHEN V(out)='0.5*vcc' CROSS=2
.MEAS TRAN Tmid PARAM='(Th1+Th2)/2'
.MEAS TRAN Vmid FIND V(out) AT='Tmid'
.MEAS TRAN Tfrom WHEN V(out)='Vmid' RISE=1
.MEAS TRAN Ripple PP V(out) FROM='Tfrom' TO='Tmid'
```

Figure 95 Waveform to Demonstrate Ripple Calculation



This file sweeps the sigma of the model parameter distribution, while it examines the delay. It shows you the delay derating curve, for the worst cases in the model. This example uses demonstration netlist `sigma.sp`, which is available in directory `$installdir/demo/hspice/cchar`.

Figure 96 Inverter Pair Transfer Curves and Sigma Sweep vs. Delay



Delay versus Fanout

The example sweeps the subcircuit multiplier to quickly generate five load curves. To obtain more accurate results, buffer the input source with one stage.

For each second-sweep variable (`m_delay` and `rms_power`), the example calculates:

- mean
- variance
- sigma
- average deviance

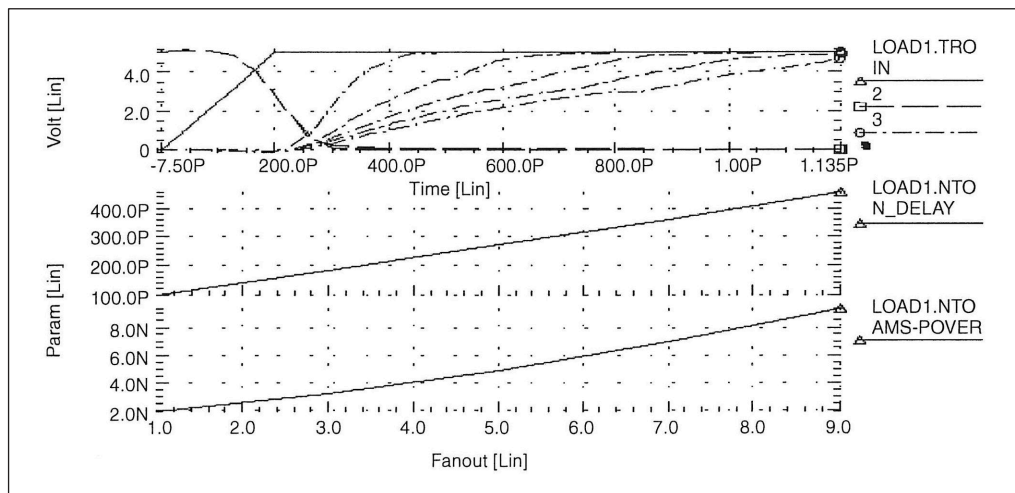
This example uses the demonstration netlist `load1.sp`, which is available in directory `$installdir/demo/hspice/cchar`.

This example outputs the following results:

```
meas_variable = m_delay
mean = 273.8560p varian = 1.968e-20
sigma = 140.2711p avgdev = 106.5685p
```

```
meas_variable = rms_power
mean = 5.2544m varian = 8.7044u
sigma = 2.9503m avgdev = 2.2945m
```

Figure 97 Inverter Delay and Power, versus Fanout



Pin Capacitance Measurement

This example does the following:

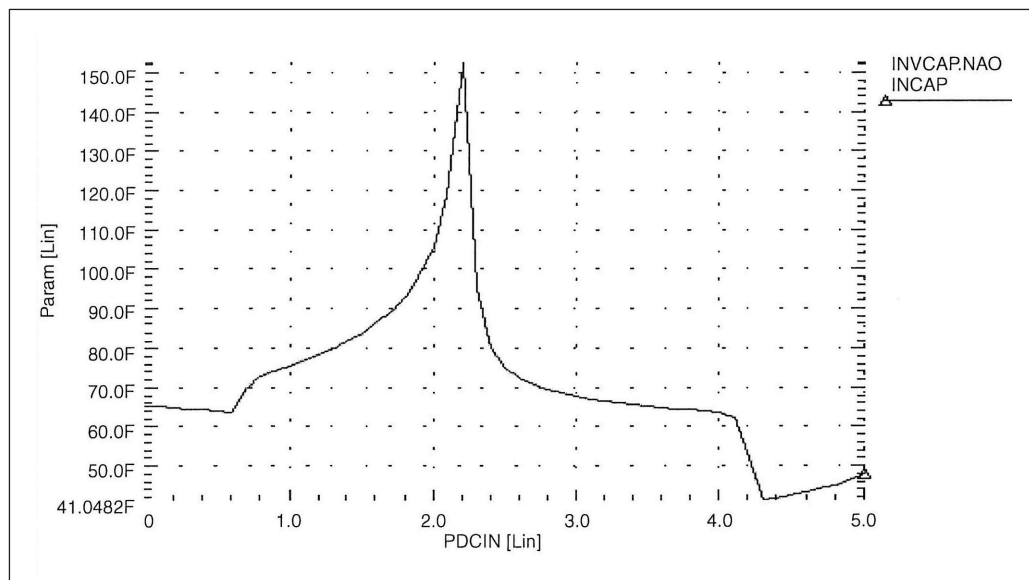
- Shows the effect of dynamic capacitance, at the switch point.
- Sweeps the DC input voltage (`pdcin`) to the inverter.

- Performs an AC analysis, at each 0.1 V increment.
- Calculates the `incap` measure parameter from the imaginary current through the voltage source at 10 kHz in the AC curve (not shown).

The peak capacitance (at the switch point) occurs when the voltage at the output side changes, in the direction opposite the input side of the Miller capacitor. This adds the Miller capacitance, times the inverter gain, to the effective capacitance.

```
mp out in 1 1 mp w=10u l=3u
mn out in 0 0 mn w=5u l=3u
vin in 0 DC= pdcin AC 1 0
.ac lin 2 10k 100k sweep pdcin 0 5 .1
.measure ac incap find par( '-1 * ii(vin)/
+ (hertz*twopi)' ) AT=10000hertz
```

Figure 98 Graph of Pin Capacitance versus Inverter Input Voltage



Op-amp Characterization of LM124

This example analyzes op-amps. This example uses:

- `.MEASURE` statements to present a very complete data sheet.
- Four `.MEASURE` statements, to reference the `out0` output node of an op-amp circuit. These statements use output variable operators for parameters:
 - decibels `vdb(out0)`
 - voltage magnitude `vm(out0)`
 - phase `vp(out0)`

This example uses the demonstration file `alm124.sp`, in `$installdir/demo/hspice/apps`.

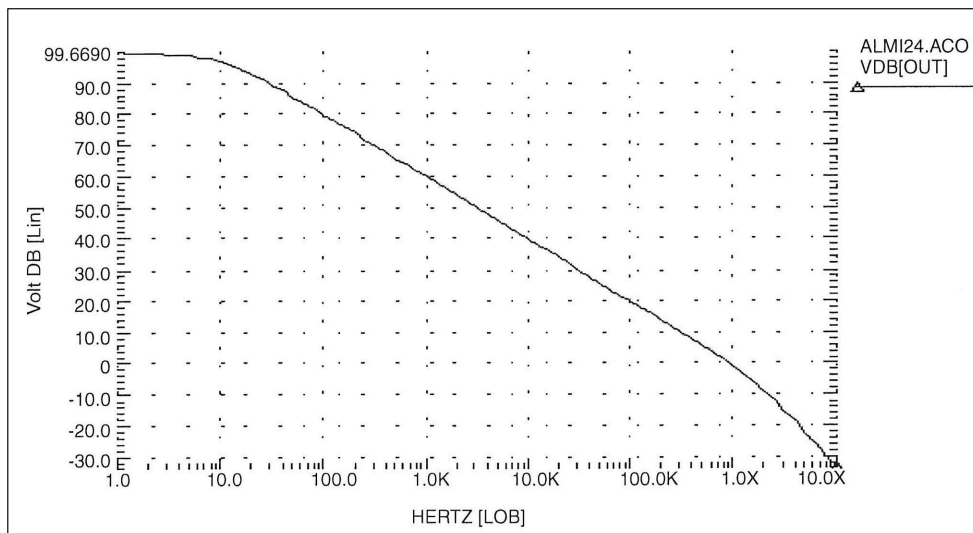
This example outputs the following results:

```
unitfreq = 9.0786E+05 targ= 9.0786E+05 trig= 1.0000E+00  
phasemargin = 6.6403E+01
```

```
gain(db) = 9.9663E+01 at= 1.0000E+00 from= 1.0000E+00  
+ to= 1.0000E+07
```

```
gain(mag)= 9.6192E+04 at= 1.0000E+00 from= 1.0000E+00  
+ to= 1.0000E+07
```

Figure 99 Magnitude Plot of Op-Amp Gain



Performing Advanced Cell Characterization

This section provides example input files, which characterize cells for an inverter, based on 3-micron MOSFET technology. PrimeSim HSPICE finds the best, worst, and typical cases for different fanouts. Use this library data for digital-based simulators, such as those used to simulate gate arrays and standard cells.

The example uses the demonstration file `cellchar.sp`, in `$installdir/demo/hspice/apps`. It demonstrates how to use the following to characterize a CMOS inverter:

- `.MEASURE` statement
- `.DATA` statement
- `AUTOSTOP` option
- `SUBCKT` definition
- `SUBCKT` call
- Models

Figure 100 Plotting the Simulation Outputs

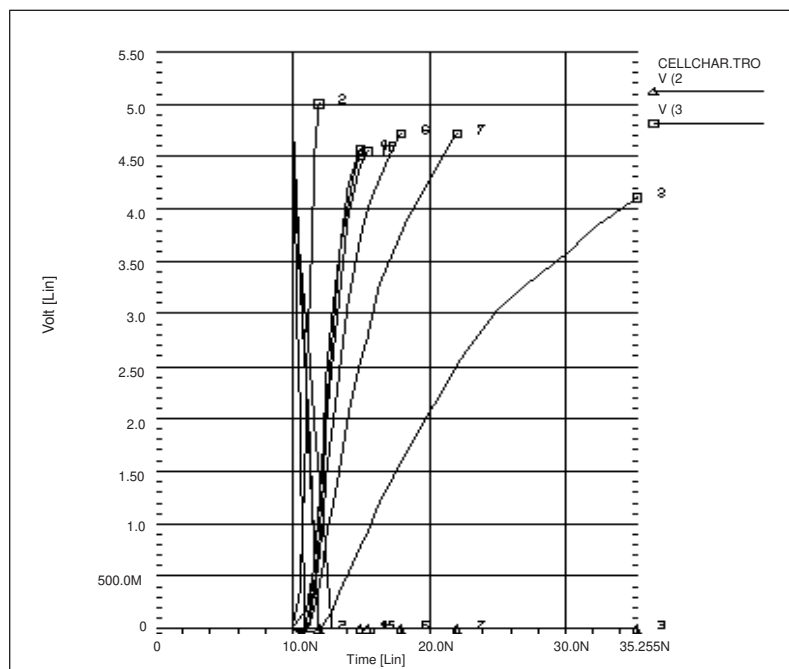
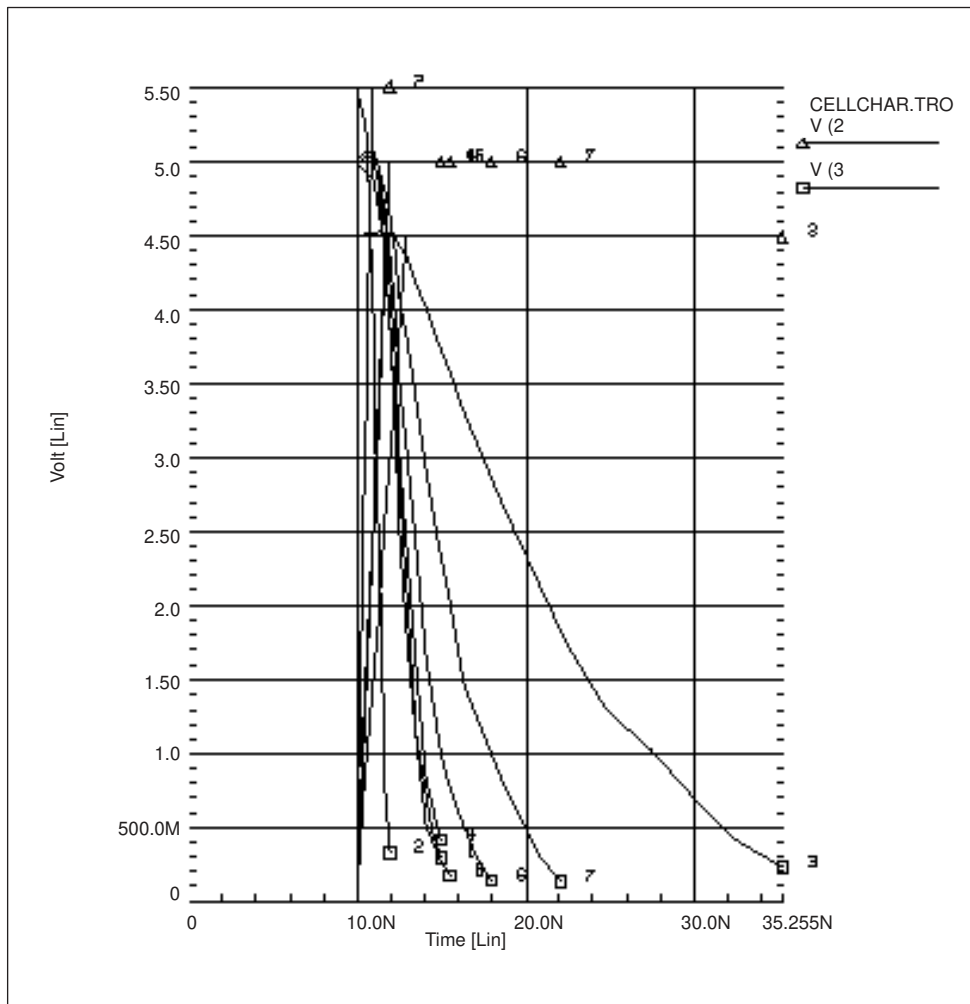


Figure 101 Verifying the Measure Statement Results by the Plots



Monte Carlo Plus Data Sweep in Transient Simulations

PrimeSim HSPICE now allows you to implement multi-dimensional sweeps such as a Monte Carlo (MC) plus data sweep in transient simulation. Such a sweep can help improve the performance of cell characterization cases in LVF analysis. There is also the ease of use advantage (no need for the `.ALTER` commands). PrimeSim HSPICE currently allows two types of multi-dimensional sweeps: one is Monte Carlo + data sweep with normal transient simulation and the other is Monte Carlo + data sweep + bisection.

For detailed information on Monte Carlo analysis in PrimeSim HSPICE, see [Chapter 26, Monte Carlo Analysis—Flow and Outputs](#).

MC+Data Sweep

LVF characterizations require transient MC computations for a sequence of parameter changes such as input signal slope and output capacitance load.

Currently, PrimeSim HSPICE users use multiple `.ALTER` commands for this purpose. The syntax for this approach looks like:

```
...  
.tran ..... SWEEP Monte = N_Samples  
...  
.Param Load_Cap = value  
.Param In_Slope = value  
.Alter  
.Param Load_Cap = value  
.Param In_Slope = value  
.Alter  
...  
.End
```

This approach lacks ease of use and can lead to loss of performance. As arbitrary parameter or netlist changes are possible with `.ALTER`, the special properties specific to cell characterization are not utilized. Also, the *readin* and *errchk* time can be large. In a typical library, the cost of these two steps can be over 30% of the total simulation time.

In the new MC+data sweep approach, you will use the following syntax:

```
...  
.tran ... SWEEP DATA=StimData Monte = N_Samples  
.Data StimData  
Index Load_Cap In_Slope  
...  
.EndData
```

In the second approach, the parameter changes in the `.ALTERS` are replaced by a data sweep. The Monte Carlo is executed for each row in the DATA block and the output is written to a single file (see [Output of Multi-Dimensional Sweeps on page 444](#)).

MC+Data Sweep+Bisection

In the existing approach to sweep for bisection (design optimization), the netlist structure currently used is similar to:

```
.tran ...  
+ sweep  
+ OPTIMIZE = OPTIMIZATION_model  
+ RESULT = opt_meas1,opt_meas2,opt_meas3,...,opt_measn  
+ MODEL = optmod  
+ monte=50  
.Alter
```

```
.Param Load_Cap = value  
.Param In_Slope = value  
.Alter  
.Param Load_Cap = value  
.Param In_Slope = value  
.Alter  
...  
.End
```

In the new approach, this syntax is simplified to:

```
.tran ...  
+ sweep  
+ OPTIMIZE = OPTIMIZATION_model  
+ RESULT = opt_meas1,opt_meas2,opt_meas3,...,opt_measn  
+ MODEL = optmod  
+ DATA=StimData  
+ monte=50  
.Data StimData  
Index Load_Cap In_Slope  
...  
.EndData
```

Output of Multi-Dimensional Sweeps

PrimeSim HSPICE writes all the sweep results into a single `.mt0` file. To help you distinguish between the DATA sweep index and Monte Carlo sweep index, PrimeSim HSPICE will write two columns in the measure file with the labels `MC_Index` and `DataBlockName_Index`. This also permits additional levels of sweeps for voltage and temperature that will be required for multi-VT characterization.

In the `.mt0` file, the index values are printed as integers to save storage.

Limitations of Multi-Dimensional Sweeps

The following limitations apply to the MC+data sweep and the MC+data+bisection sweeps:

- Only transient sweeps are supported.
- The sweeps work only if the option `Cell_Char` is set to Yes. If `Cell_Char = No`, PrimeSim HSPICE will execute the normal Monte Carlo transient simulation and the data sweep part will be ignored with a warning message.

It is recommended that the you use the new syntax with:

```
.Option Cell_Char = Yes, MeasFile=1, MCBrief=6
```

- In the syntax for the sweeps, the order of `Data = ...` and `Monte = ...` is fixed and cannot be interchanged. Otherwise, PrimeSim HSPICE will provide an error message.

- Only signal delay, signal slopes, and load capacitance are assumed to change within the data sweep. You or the characterization tool should ensure this.

The PrimeSim HSPICE computational steps are organized to take advantage of this restriction. For example, the operating point computation is done only once for the data sweep.

- PrimeSim HSPICE computes a new OP at each MC sweep point as the transistor parameters change. The initial value (nodeset, including internal nodes) will correspond to the solution at MC sweep index 1, that is, the nominal solution. OP computation is skipped during the DATA block sweep as changes in the voltage source slope and load capacitance do not change the operating point.
- As the option `Cell_Char = Yes`, PrimeSim HSPICE generates only the measurement files, but not the waveform files or data mining reports.
- Distributed processing is not supported for these sweeps.

Cell Characterization Examples

Figure 102 and Figure 103 are identical, except that their input signals are complementary.

- The circuit in Figure 102 calculates the rise time and the low-to-high propagation delay time.
- The circuit in Figure 103 calculates the fall time and the high-to-low propagation delay time.

If you use only one circuit, CPU time increases because analysis time increases when PrimeSim HSPICE calculates both rise and fall times.

The XOUTL or XOUTH subcircuit represents the fanout of the cell (inverter). To modify fanout, specify different multipliers (m) in the subcircuit calls.

You can also specify local and global temperatures. This example characterizes the cell at a global temperature of 27, but the temperature of the M1 and M2 devices is $(27 + DTEMP)$. The `.DATA` statement specifies the DTEMP value.

The example uses a transient parameterized sweep, with `.DATA` and `.MEASURE` statements, to determine the inverter timing, for best, typical, and worst cases.

Figure 102 Cell Characterization Circuit 1

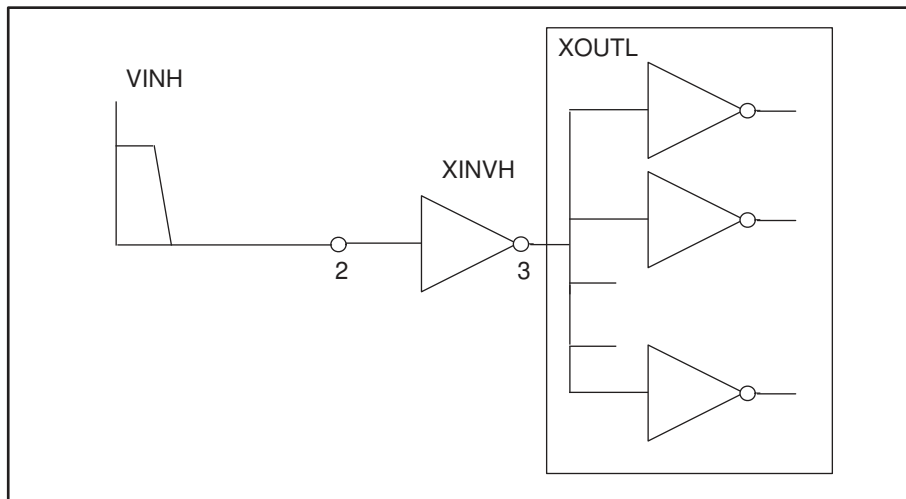
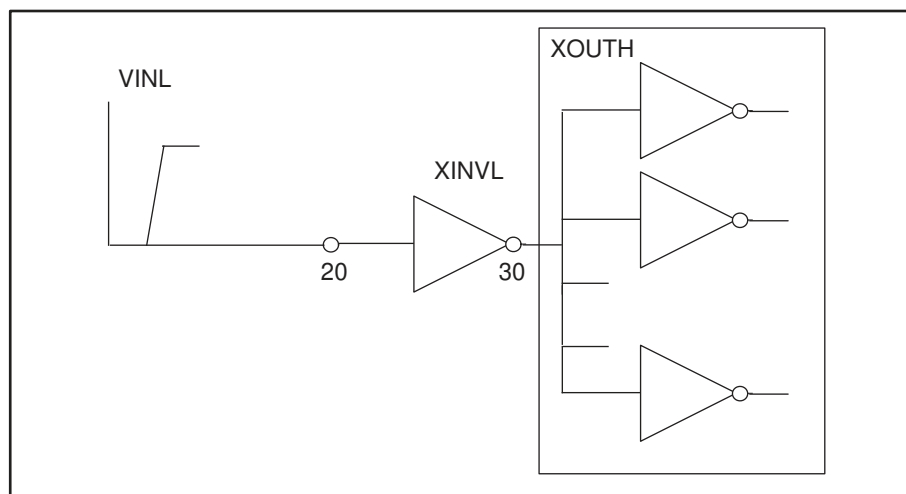


Figure 103 Cell Characterization Circuit 2



This example varies the following parameters:

- power supply
- input rise and fall time
- fanout
- MOSFET temperature

- n-channel and p-channel threshold
- drawn width and length of the MOSFET

Use the `.MEASURE` statement to specify a parameter to measure.

Use the `AUTOSTOP` option, to speed simulation time. The `AUTOSTOP` option terminates the transient sweep, although it has not completely swept the specified transient sweep range.

The `.MEASURE` statement uses quoted string parameter variables to measure the rise time, fall time, and propagation delays.

Note:

Do not use character strings as parameter values in PrimeSim HSPICE advanced analog analyses.

Rise time starts when the voltage at node 3 (the output of the inverter) is equal to $0.1 \cdot VDD$ (that is, $V(3) = 0.1VDD$). Rise time ends when the voltage at node 3 is equal to $0.9 \cdot VDD$ (that is, $V(3) = 0.9VDD$).

For more accurate results, start the `.MEASURE` calculation after either:

- A time delay, or
- A simulation cycle, specifying delay time in the `.MEASURE` statement, or
- An input pulse statement.

The following example features:

- `AUTOSTOP` option and `.MEASURE` statements.
- Mean, variance, sigma, and avgdev calculations.
- Circuit and element temperature.
- Algebraic equation handling.
- `PAR( )` as an output variable, in the `.MEASURE` statement.
- Subcircuit parameter-passing, and subcircuit multiplier.
- `.DATA` statement.

23

Timing Analysis Using Bisection

Describes how to use the bisection function in timing optimization.

To analyze circuit timing violations, a typical methodology is to generate a set of operational parameters that produce a failure in the required behavior of the circuit. When a circuit timing failure occurs, you can identify a timing constraint, which can lead to a design guideline. You must perform an iterative analysis to define the violation specification.

Typical types of timing constraint violations include:

- Data setup time, before the clock
- Data hold time, after the clock
- Minimum pulse width required for a signal to propagate to the output
- Maximum toggle frequency of the component(s)

PrimeSim HSPICE ships numerous examples for your use; see Bisection-Timing Analysis Examples in the *PrimeSim HSPICE User Guide: Demonstration Netlists* for paths to the demo files discussed in this chapter.

For more information about optimization, see [Chapter 29, Optimization](#) in this user guide.

For information on Monte Carlo, see [Chapter 24, Variation Analysis](#) and [Chapter 26, Monte Carlo Analysis—Flow and Outputs](#).

This chapter discusses the following topics:

- [Overview of Bisection](#)
- [Bisection Methodology](#)
- [Using Bisection](#)
- [Setup Time Analysis](#)
- [Minimum Pulse Width Analysis](#)
- [Pushout Bisection Methodology](#)

- [Using RELOUT and RELIN to Affect PrimeSim HSPICE Bisection Optimization](#)
- [Using Bisection with Monte Carlo Analysis](#)
- [Bisection Optimization Application Example](#)

Overview of Bisection

Before bisection methods were developed, engineers built external drivers to submit multiple parameterized simulations to SPICE-type simulators. Each simulation explored a region of the operating envelope for the circuit. To provide part of the analysis, the driver also post-processed the simulation results, to deduce the limiting conditions.

If you characterize small circuits this way analysis times are relatively small, compared with the overall job time. This method is inefficient, due to overhead of submitting the job, reading and checking the netlist, and setting up the matrix. The newer bisection methods increase efficiency when you analyze timing violations, to find the causes of timing failure. Bisection optimization is an efficient cell-characterization method, in Synopsys PrimeSim HSPICE.

For a full demo example of finding early, optimal, and late setup times of a DFF, follow the path to `fig26_4.sp` in Bisection-Timing Analysis Examples in the *PrimeSim HSPICE User Guide: Demonstration Netlists*.

The bisection methodology saves time in three ways:

- Reduces multiple jobs to a single characterization job.
- Removes post-processing requirements.
- Uses accuracy-driven iterations.

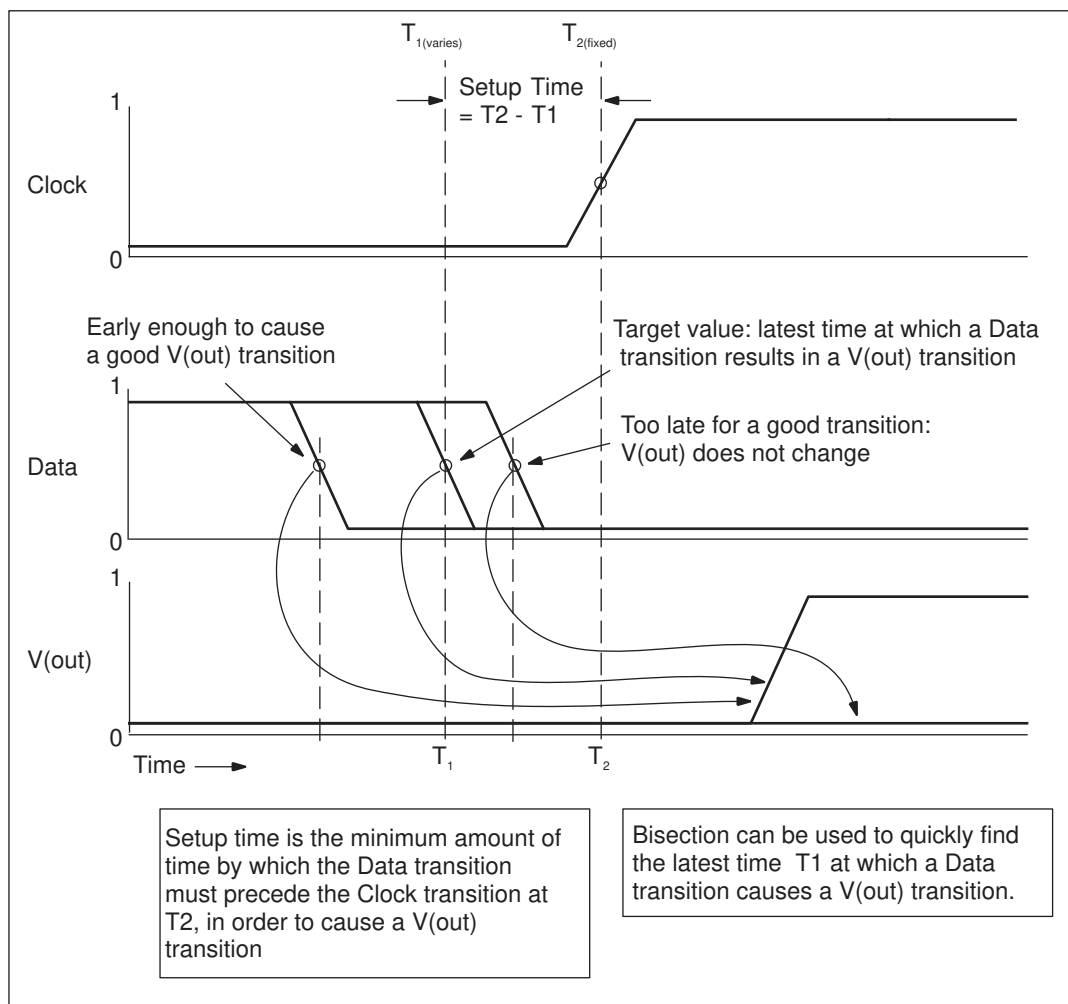
[Figure 104 on page 450](#) shows a typical analysis of setup-time constraints. Clock and data input waveforms drive a cell. Two input transitions (rise and fall) occur at times T_1 and T_2 . The result is an output transition, when $V(\text{out})$ changes from low to high. The following relationship between the $T_1(\text{data})$ and $T_2(\text{clock})$ times must be true for the $V(\text{out})$ transition to occur: $T_2 > (T_1 + \text{setup time})$.

Characterization or violation analysis determines the setup time. To do this, PrimeSim HSPICE keeps T_2 fixed and repeats the simulation with different T_1 values. It then observes which T_1 values produce an output transition and which do not.

Before bisection, users had to run tight sweeps of the delay between the data setup and clock edge, and look for the value at which no transition occurs. To do this, you swept a value that specifies how far the data edge precedes a fixed clock edge. This method is time consuming, and is accurate only if the sweep step is very small. Linear search methods cannot accurately determine the setup time value, unless you use extremely small steps from T_1 to T_2 to simulate the circuit at each point, and monitor the outcome.

For example, even if you know that the desired transition occurs during a particular 5 ns period, you might need to run 50 simulations to search for the setup time to within 0.1 ns over that 5 ns period. But the error in the result can be as large as 0.05 ns.

Figure 104 Determining Setup Time with Bisection Violation Analysis



The bisection feature greatly reduces the amount of work and computational time required to find an accurate solution for this type of problem. The following sections show examples of using this feature to identify timing violations for the setup, hold, and minimum clock pulse width.

Bisection Methodology

Bisection is an optimization method that uses a binary search method, to find the value of an input variable (target value). This variable is associated with a goal value of an output variable.

The type of the input and output variables can be voltage, current, delay time, or gain, related by some transfer function. In general, use a binary search to locate the goal value of the output variable within a search range of the input variable. Then, iteratively halve that range to rapidly converge on the target value. At each iteration, PrimeSim HSPICE compares the measured value of the output variable with the goal value. Both the PASSFAIL method and the bisection method use bisection (see [Using Bisection](#)).

The bisection procedure consists of two measurement and optimization steps, when solving the timing violation problem:

- Detecting whether the output transition occurred.
- Automatically varying the input parameter (T_1 in [Figure 104](#)) to find the value for which the transition barely occurs.

Measurement

Use the `MAX` measurement function to detect the success or failure of an output transition. For a low-to-high output transition, a `MAX` measurement produces zero on failure, or approximately the V_{dd} supply voltage on success. This measurement, using a goal of V_{dd} (minus a suitable small value to ensure a solution), is sufficient to drive the optimization.

Optimization

The bisection method is straightforward if you specify a single measurement with a goal, and known upper and lower boundary values for the input parameter. The characterization engineer must specify acceptable upper and lower boundary values.

Using Bisection

Before you can use bisection, you must specify the following:

- A pair of values, for the upper and lower boundaries of the input variables. To find a solution, one of these values must result in an output variable `>|goal value|` and the other must result in `<|goal value|`.
- A goal value. If there is no goal keyword in the statement, the goal value will not be defaulted to zero, and PrimeSim HSPICE considers the measure result as a relative error expression.

- Error tolerance value. The bisection process stops when the difference between successive test values is $\leq$ error tolerance. If the other criteria are also met, see the following steps.
- Related variables. Use a monotonic transfer function to relate variables where a steadily progressing time (increase or decrease) results in a single occurrence of the goal value at the target input variable value.

PrimeSim HSPICE includes the error tolerance in a relation, used as a process-termination criterion.

The following sections discuss these topics:

- [Examples](#)
- [Examining the Command Syntax](#)
- [Performing Transient Analyses with Bisections](#)

Examples

[Figure 105](#) shows an example of the binary search process that the bisection algorithm uses. This example is the pass/fail type, and is appropriate for a setup-time analysis that tests for the presence of an output transition. In the example depicted in [Figure 104](#) note that:

1. A long setup time $TS (= T2 - T1)$ results in a `VOUT` transition (a pass).
2. A too-short setup time (where the latch has not stabilized the input data, before the clock transition) results in a fail.

For example, you might define a pass time value as any setup time, TS , that produces a `VOUT` output minimum high logic output level of 2.7V, which is the goal value.

3. The target value is a setup time that just produces the `VOUT` value of 2.7V. Finding the exact value is impractical, if not impossible, so you need to specify an error tolerance to calculate a solution arbitrarily close to the target value.
4. The bisection algorithm performs tests for each specified boundary value to determine the direction in which to pursue the target value, after the first bisection. In this example shown in [Figure 104](#), the upper boundary has a pass value and the lower boundary has a fail value.
5. To start the binary search you specify the lower and upper boundaries. The program tests the point midway between the lower and upper boundaries (see [Figure 105](#)).

- If the initial value passes the test, the target value must be less than the tested value (in this example). The bisection algorithm moves the upper search limit to the value that it just tested.
 - If the test fails, the target value must be greater than the tested value. Bisection moves the lower limit to the value that it just tested.
6. The algorithm tests a value midway between the new limits.
 7. The search continues in this manner, moving one limit or the other to the last midpoint, and testing the value midway between the new limits.
 8. The process stops when the difference between the latest test values is less than or equal to the error tolerance that you specified. To normalize this value, multiply by the initial boundary range.

For the path to a full demo file example of a DFF bisection search for setup time, `dff_top.sp`, see Bisection-Timing Analysis Examples in the *PrimeSim HSPICE User Guide: Demonstration Netlists*.

Examining the Command Syntax

The following syntax is used for bisection:

```
.MODEL OptModelName OPT METHOD=BISECTION ...
```

```
.MODEL OptModelName OPT METHOD=PASSFAIL .....
```

`OptModelName` is the model to be used. Refer to the [Optimization Examples](#) for name information on specifying optimization models in PrimeSim HSPICE. The `METHOD` keyword specifies which optimization method to use. The `OPT` keyword indicates that optimization is to be performed.

For bisection, the method can be one of the following:

- `BISECTION`

When the difference between the two latest test input values is within the error tolerance and the latest measured value exceeds the goal, bisection has succeeded and then ends. This process reports the optimized parameter that corresponded to the test value that satisfies this error tolerance and this goal (passes). See [Bisection Optimization Application Example on page 470](#).

- `PASSFAIL`

When the difference between the last two optimization parameter test values is $<$ the error tolerance and the associated goal measurement fails for one of the values and passes for the other, bisection has succeeded and then ends. The process reports

the optimization parameter test value associated with the last passing measurement. “Pass” is defined as a condition in which the associated goal measurement can produce a valid result. “Fail” is defined as a condition in which the associated goal measurement is unable to produce a valid result. For example, if the measurement is of TRIG/TARG form, and the TARG event is not found, then this optimization parameter test value is deemed a failure. When using PUSHOUT bisection, the definition of a failure is modified to also include any goal measurement result that is valid and > the push-out specification.

You can also monitor multiple measurement results and find the parameter value at which all measurements begin to succeed. For example:

```
.tran lp 100n sweep optimize=opt1 result=delq,delqn model=optmod
```

The parameters are passed in a normal optimization specification:

```
.PARAM ParamName=OptParFun (Initial, Lower, Upper)
```

In the **BISECTION** method, the measure results for *Lower* and *Upper* limits of *ParamName* must be on opposite sides of the goal value in the **.MEASURE** statement. In the **PASSFAIL** method, the measure must pass for one limit and fail for the other limit. The process ignores the value of the *Initial* field. The error tolerance is a parameter in the model which is being optimized. Using the **BISECTION** method, a bisectional search is applied to multiple parameters. The logical relationship of these parameters is based on 'AND'.

When the **OPTLST** option is set (**.OPTION OPTLST=1**), the process outputs the following information for the **BISECTION** method:

```
bisec-opt iter = num_iterations xlo = low_val xhi = high_val  
x = result_low_val xnew = result_high_val err = error_tolerance
```

The *x* is the old parameter value and *xnew* is the new parameter value.

When **.OPTION OPTLST=1**, the process outputs the following information for the **PASSFAIL** method:

```
bisec-opt iter = num_iterations xlo = low_val xhi = high_val x  
= result_low_val xnew = result_high_val measfail = 1
```

In this syntax, *measfail=0* for a test failure for the *x* value.

For more information about using the **.MODEL** statement for bisection, see **.MODEL** in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

Performing Transient Analyses with Bisections

When performing transient analysis bisection with the `.TRAN` statement, use the following syntax:

```
.TRAN TranStep TranTime SWEEP OPTIMIZE=OptParFun  
+ RESULTS=meas1,...,measN MODEL=OptModelName
```

When performing a transient analysis bisection with the `.MEASURE` statement, use the following syntax:

```
.MEASURE TRAN MeasureName MeasureClause  
+ [[GOAL=val] | GOALMAX | GOALMIN]
```

Setup Time Analysis

This example uses a bisectional search to find the minimum setup time for a D flip-flop. The circuit for this example is `dff_top.sp`, which is located in directory `$installdir/demo/hspice/bisect`.

[Figure 105 on page 457](#) and [Figure 106 on page 458](#) show the results of this demo. PrimeSim HSPICE does not directly optimize the setup time, but extracts it from its relationship with the `DelayTime` parameter (the time before the data signal), which is the parameter to optimize.

The following sections discuss these topics:

- [Input Listing](#)
- [Results](#)

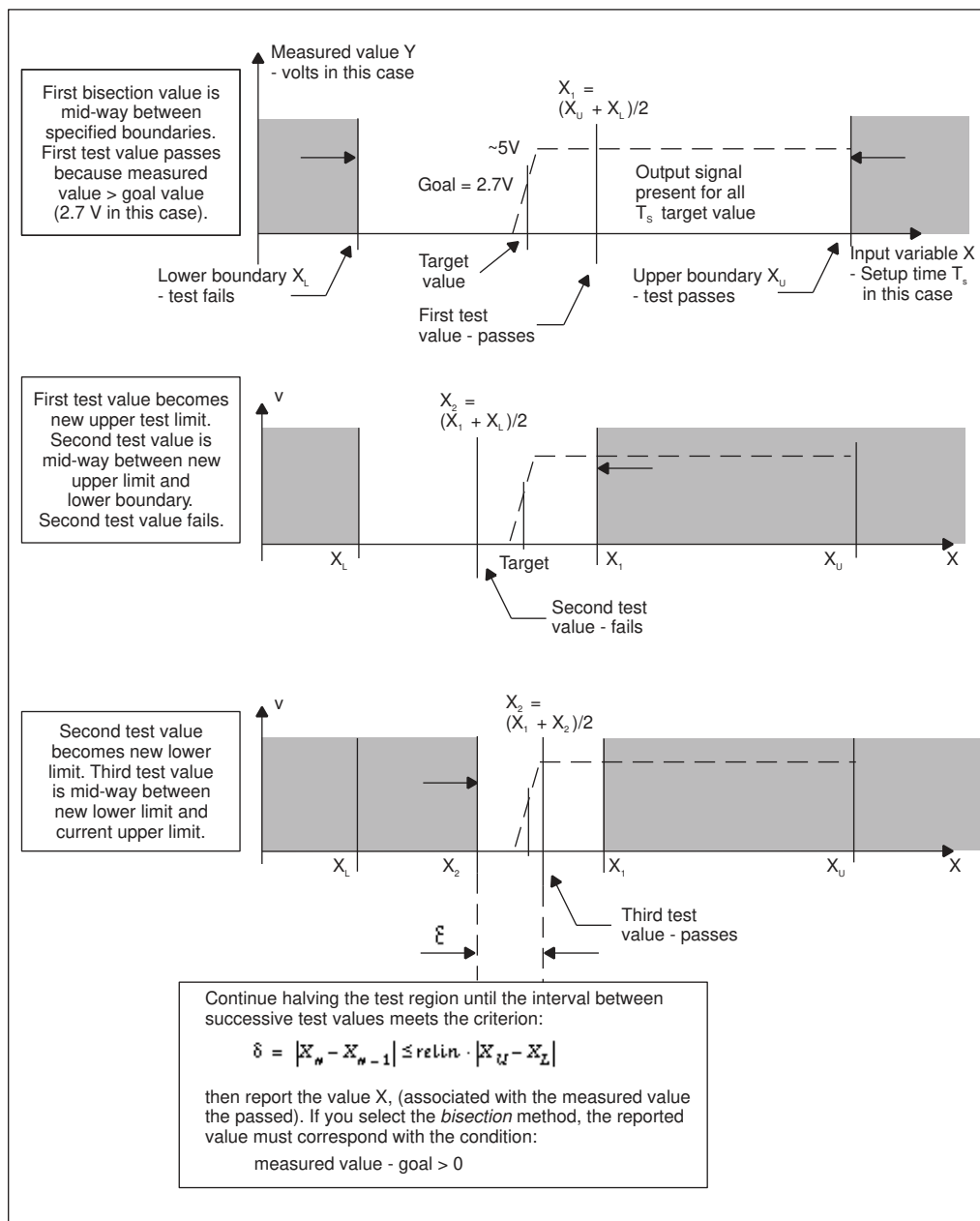
Input Listing

The following portion of the input listing shows how `.TRAN` analysis, the `DelayTime` parameter, and `.MEASURE` statements are used in bisection:

```
* DFF_top Bisection Search for Setup Time  
* PWL Stimulus  
v28 data gnd PWL  
+ 0s 5v  
+ 1n 5v  
+ 2n 0v  
+ Td = "DelayTime" $ Offsets Data from time by DelayTime  
v27 clock gnd PWL  
+ 0s 0v  
+ 3n 0v  
+ 4n 5v  
* Specify DelayTime as the search parameter and provide
```

```
* the lower and upper limits.
.PARAM DelayTime= Opt1 ( 0.0n, 0.0n, 5.0n )
* Transient simulation with Bisection Optimization
.TRAN 0.1n 8n Sweep Optimize = Opt1
+ Result = MaxVout$ Look at measure
+ Model = OptMod
* This measure finds the transition if it exists
.MEASURE Tran MaxVout Max v(D_Output) Goal = 'v(Vdd)'
* This measure calculates the setup time value
.MEASURE Tran SetupTimeTrig v(Data)Val = 'v(Vdd)/2'
+ Fall = 1
+ Targ v(Clock)Val = 'v(Vdd)/2'
+ Rise = 1
* Optimization Model
.MODEL OptMod Opt
+ Method = Bisection
.OPTION Post NoMod
```

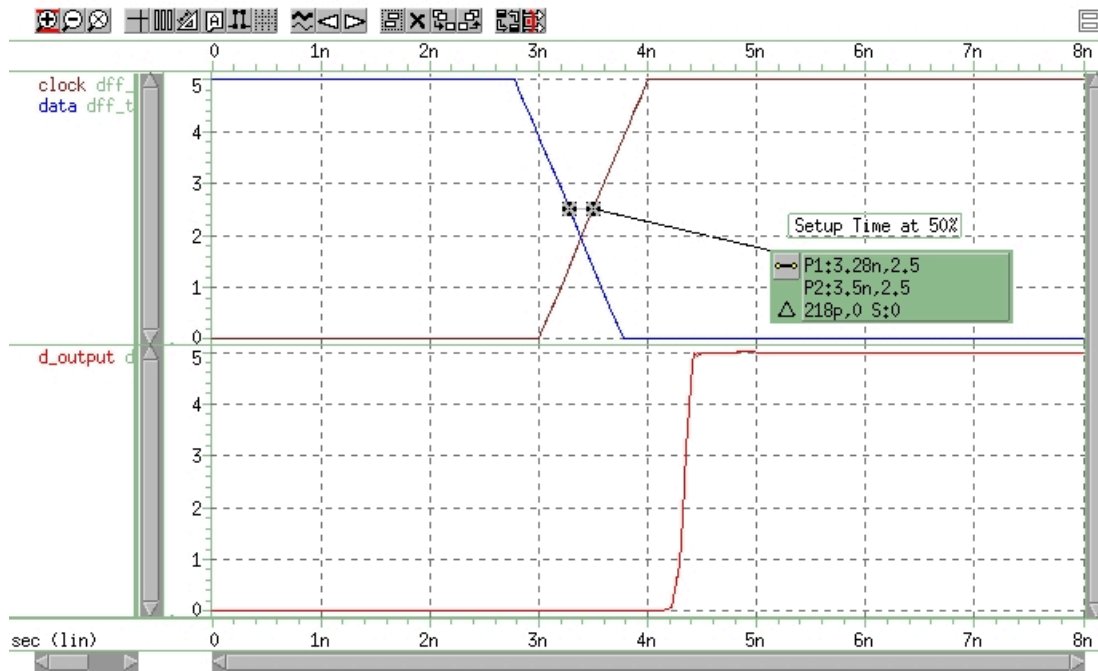
Figure 105 Bisection Example for Three Iterations



Results

The upper plot in [Figure 106 on page 458](#) shows the relationship between the clock and the data pulses that determine the setup time. The bottom plot is the output transition.

Figure 106 Transition at Minimum Setup Time



Find the actual value for the setup time in the “Optimization Results” section of the output listing file:

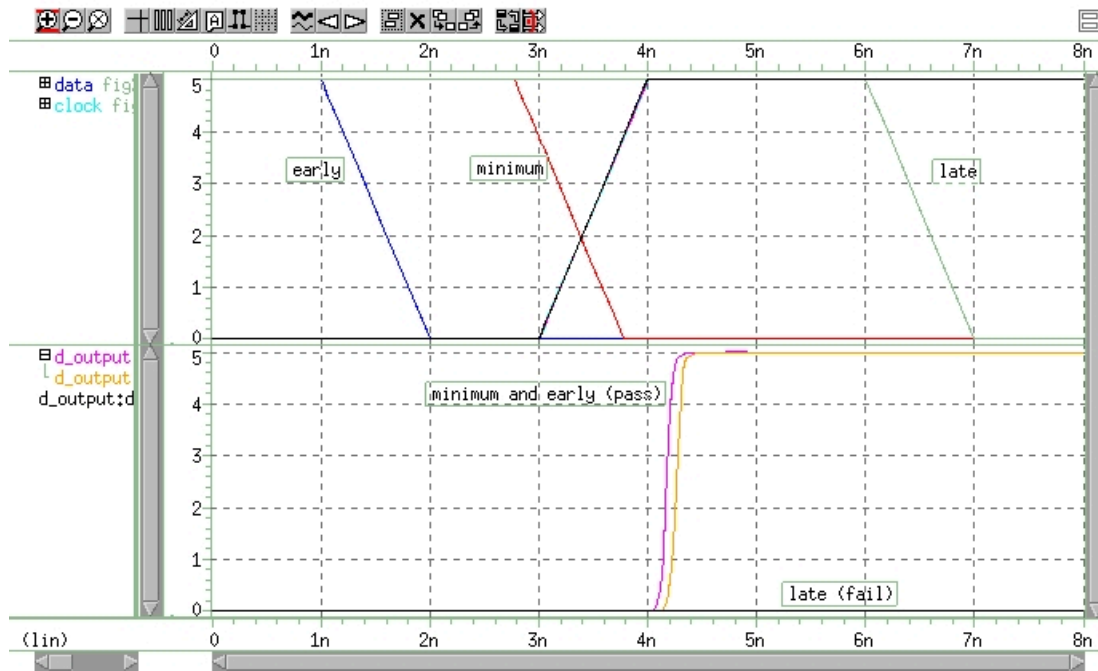
```
optimization completed, the condition
relin = 1.0000E-03 is satisfied
**** optimized parameters opt1
.PARAM DelayTime = 1.7822n
...
maxvout = 5.0001E+00 at= 4.8984E-09
from    = .0000E+00    to= 8.0000E-09
setuptime= 2.1777E-10 targ= 3.5000E-09 trig= 3.2822E-09
```

This listing file excerpt shows that the optimal value for the setup time is 0.21777 ns.

The upper plot in [Figure 107](#) shows examples of early and late data transitions, and the transition at the minimum setup time. The bottom plot shows how the timing of the data transition affects the output transition. The following analysis statement produces these results:

```
* Sweep 3 values for DelayTime Early Optim Late
.TRAN 0.1n 8n Sweep DelayTime Poi 3 0.0n 1.7822 5.0n
```

Figure 107 Early, Minimum, and Late Setup and Hold Times



This analysis produces the following results:

```
*** parameter DelayTime = .000E+00 *** $ Early
setuptime= 2.0000E-09 targ= 3.5000E-09 trig= 1.5000E-09
*** parameter DelayTime = 1.7822E-09 *** $ Optimal
setuptime= 2.1780E-10 targ= 3.5000E-09 trig= 3.2822E-09
*** parameter DelayTime = 5.000E-09 *** $ Late
setuptime= -3.0000E-09 targ= 3.5000E-09 trig= 6.5000E-09
```

Minimum Pulse Width Analysis

This example uses a pass/fail bisectional search to find a minimum pulse width required so the input pulse can propagate to the output of an inverter. It is based on demonstration netlist `inv_a.sp`, which is available in directory `$installdir/demo/hspice/bisect`. [Figure 108](#) shows the results of this demo.

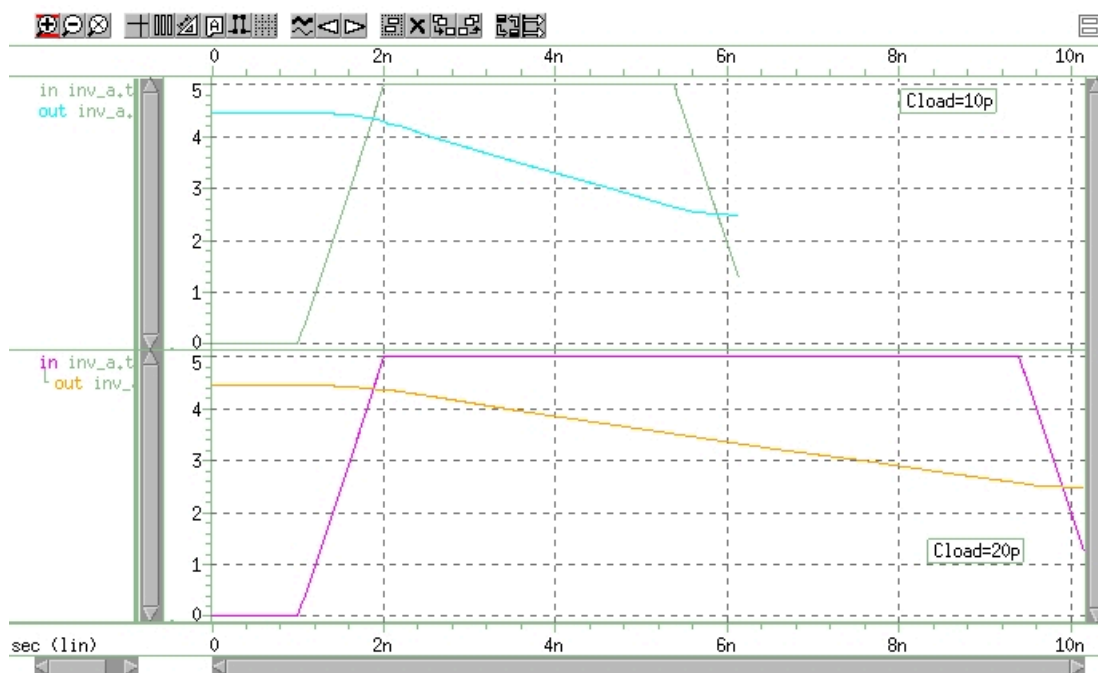
Input Listing Directory

This input listing file is located in: `$installdir/demo/hspice/bisect/inv_a.sp`.

Results

Figure 108 shows results of pass/fail search, for two different capacitive loads.

Figure 108 Results of Bisectional Pass/Fail Search



Pushout Bisection Methodology

For setup- or hold-time optimization analysis, a normal bisection method varies the input timing to find the point just before failure. At this point, delaying the input longer results in failure, and the output does not transition.

In pushout analysis, if `.option bisec_pushout_init` is not set, instead of finding the last point just before failure, the first successful output transition is used as the golden target. If `.option bisec_pushout_init` is set, measure result on initial value is used as the golden target. (See `.OPTION BISEC_PUSHOUT_INIT` in *PrimeSim HSPICE Reference Manual: Commands and Control Options*). You can then apply a maximum allowed pushout time to decide if the subsequent results are classified as passes or failures. Finding the optimized pushout result is similar to a normal bisection because both use a binary search to approach the desired solution. The main difference is the goal or the optimization criteria.

You can add measure options to support selective pushout, using these keywords:

- **POSITIVE:** pushout constraints only take effect when the measuring results are larger than the golden measure. For example:

```
.MEASURE TRAN result MeasureClause pushout=time  
+ pushout_perpercentage POSITIVE
```

- **NEGATIVE:** pushout constraints only take effect when the measuring results are smaller than the golden measure. For example:

```
.MEASURE TRAN result MeasureClause pushout=time  
+ pushout_perpercentage NEGATIVE
```

To limit the range you can add both absolute and relative pushout together. (Note the comma-separated parameters.)

For example:

```
.Measure Tran pushout When v(D_Output)='vih/2'  
+ rise=1 pushout=20p,50p pushout_per=0.1
```

The final measure result for the preceding example should be in the range of:

```
| measresult-goldmeas | < Min (pushout_max, pushout_per*goldmeas)
```

Or the final measure result should satisfy,

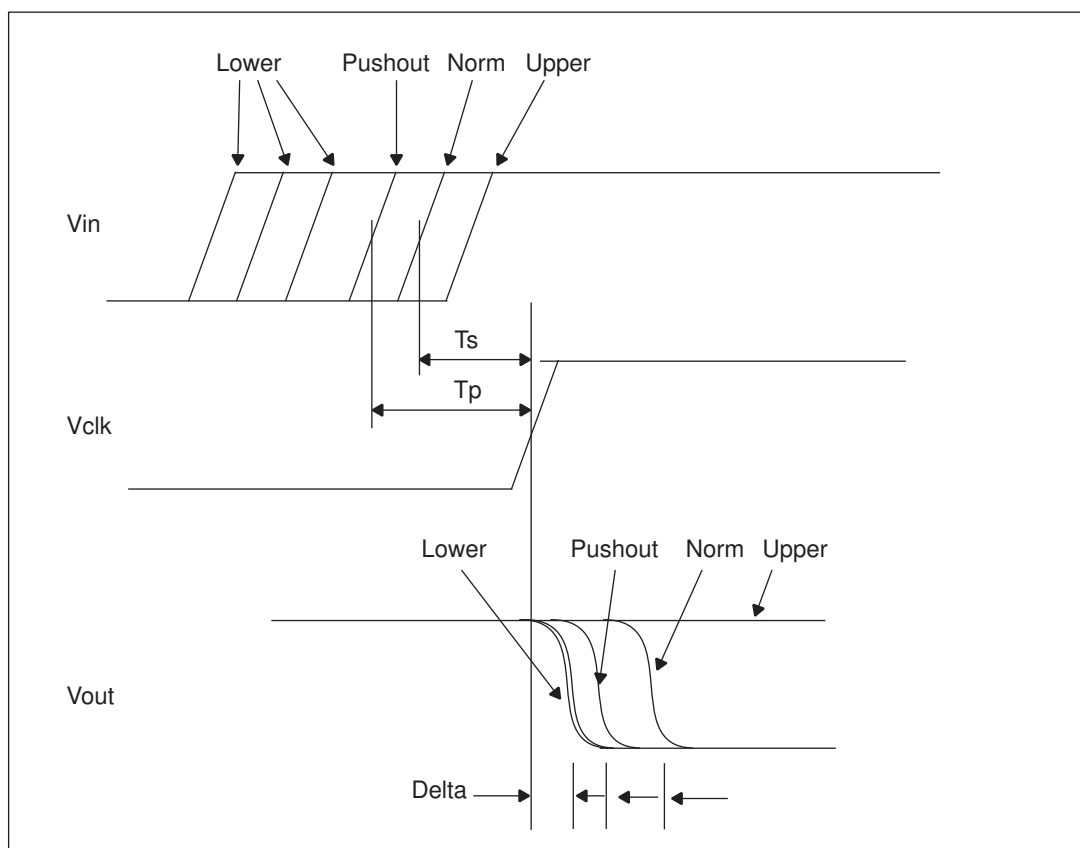
```
Max(pushout_per*goldmeas, pushout_min)
```

See **.MEASURE (Pushout Bisection)** for more information. If neither of the keywords above is set, the pushout flow follows the standard described above and shown in the next example.

Example

The following example ([Figure 109 on page 462](#)) shows a transition of Vin with a varying delay during a Vclk transition. When the “lower” input transitions, it indicates that the device being tested is functioning. The “upper” input does not transition, which indicates that the device is not functioning.

Figure 109 Pushout Bisection Example



Consider the pushout bisection example located in the following directory: `$installdir/demo/hspice/bisect/dff_push.sp`. (See the path to this example, which is a DFF pushout bisection search for setup time, in Bisection-Timing Analysis Examples in the *PrimeSim HSPICE User Guide: Demonstration Netlists*.)

The transition of V_{in} is delayed by varying amounts with respect to a V_{clk} transition. For the Lower input transition, the output transitions and indicates that the device under test is functioning. For the Upper input transition, the output does not transition and indicates that the device is not functioning.

Normal bisection varies the input timing to find the point just before failure (called Norm here). At this point, a failure occurs when the device is delayed longer and the output does not transition. The circuit works at points between Lower and Norm, but the output transition is delayed from the lower conditions by setting Delta. This is called the Pushout. The pushout can also lie between Norm and Upper, which depends on your use of the lower or upper option.

In you use normal bisection in this example, the resulting gain is delaytime=7.374e-10 pushout=-1.672e-09. Instead, when setting pushout=0.01, the result is delaytime=3.918e-11 pushout=3.970e-09.

Using RELOUT and RELIN to Affect PrimeSim HSPICE Bisection Optimization

You can use the `.MODEL` command parameters `RELOUT` and `RELIN` to influence the PrimeSim HSPICE Bisection optimization convergence process. The optimization statement has many parameters. The default values of these parameters are suitable for most applications. However, if the optimization does not converge, you might need to adjust the error tolerance of the `RELOUT` and `RELIN` command-line parameters on the `.MODEL` card.

Note:

If you use `.OPTION RELIN` its value overrides any value you enter on the `.MODEL` statement for the parameter `RELIN`. (There is *no* `.OPTION RELOUT` in PrimeSim HSPICE).

For more information about using `RELOUT` and `RELIN` in the `.MODEL` command for bisection, see `.MODEL` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

The following sections discuss these topics:

- [.MODEL Optimization Syntax](#)
- [RELOUT Parameter](#)
- [RELIN Parameter](#)
- [Example](#)

.MODEL Optimization Syntax

```
.MODEL mname OPT [METHOD=BISECTION|PASSFAIL] [close=num]
+ [max] [cut=val] [difsiz=val] [grad=val] [parmin=val]
+ [relin=val] [relout=val] [absout=val]
+ [itrop=val] [absin=val]
+ [DYNACC=0|1] [cendif=num]
```

RELOUT Parameter

Use the `RELOUT` parameter to set the output error tolerance. For example, if you specified the `GOAL` value as `5v` and `RELOUT=0.1`, when the bisection output is smaller than $0.5v(5v * 0.1)$, `RELOUT` is satisfied.

RELIN Parameter

Use the `RELIN` parameter to set the input error tolerance. For example, if the bisection initial input window is `5ns`, and `RELIN` is set to `1E-3` (default), when the difference between the latest bisection input and the previous one is smaller than $5ps(5ns * 1E-3)$, `RELIN` is satisfied.

Example

The following lines are from the `*.lis` file of a successful optimization run that appear only if you use the `OPTLST=3` option in the netlist.

```
*START OF *.lis FILE
.
.
.
parm names init guess, lower, upper bounds

delaytime          0.          0.          5.0000E-09          0.
bisec-opt.  iter =   1 xlo =          0.          xhi =   5.00000E-09
                x =          0.          xnew =   5.00000E-09
                err =  -1.91761E-02

.
.
.
bisec-opt.  iter =   7 xlo =   1.87500E-09  xhi =   2.03125E-09
                x =   2.03125E-09  xnew =   1.95313E-09
                err =   0.99949
bisec-opt.  iter =   8 xlo =   1.95313E-09  xhi =   2.03125E-09
                x =   1.95313E-09  xnew =   1.99219E-09
                err =  -0.11111

.
.
.
bisec-opt.  iter =  10 xlo =   1.21094E-09  xhi =   1.23047E-09
                x =   1.23047E-09  xnew =   1.22070E-09
                err =   0.46334
bisec-opt.  iter =  11 xlo =   1.22070E-09  xhi =   1.23047E-09
                x =   1.22070E-09  xnew =   1.22559E-09
                err =  -1.89848E-02
                optimization completed, the condition
```

```
relin = 1.0000E-03 is satisfied
optimization completed, the condition
relout = 0.4000 is satisfied
* END OF *.lis FILE
```

In each iteration, `err` is calculated as $(\text{GOAL} - \text{RESULT}) / \text{GOAL}$

where: `GOAL` is the target value and `RESULT` is the value calculated for each iteration.

`RELOUT` is satisfied when $|\text{err}| < \text{RELOUT}$

`RELIN` is satisfied when $|X(\text{new}) - X(\text{new-1})| < \text{RELIN} * |X(\text{upper}) - X(\text{lower})|$

where: `X(new)` is the `xnew` value of the `n`th iteration and `X(new-1)` is the `xnew` value of the `(n-1)`th iteration.

Both `RELIN` and `RELOUT` must be satisfied before the optimization can converge.

Note that in `iter = 8`, the condition for `RELOUT` is satisfied but the condition for `RELIN` is not. (Substitute `5n` for `X(upper)` and `0n` for `X(lower)` in the formula for `RELIN`). In `iter = 11`, check that both `RELIN` and `RELOUT` are satisfied.

Using Bisection with Monte Carlo Analysis

Bisection method integrated with Monte Carlo analysis enable both analyses to run in a single simulation. Bisection with Monte Carlo analysis can be run either with traditional PrimeSim HSPICE Monte Carlo (see [Chapter 24, Variation Analysis](#)) or Monte Carlo using the Variation Block flow (see [Chapter 26, Monte Carlo Analysis—Flow and Outputs](#)).

Monte Carlo analysis is the generic tool for simulating the effects of process variation on circuit performance. Bisection uses a binary search to find the value of an input variable (target value), which satisfies a goal value of an output variable. It is used extensively in analyzing circuit timing violations, performing timing optimization, and in sequential cell characterization.

More than one sweep loop can run in transient simulations. Bisection is enabled as the inner loop, working with a Monte Carlo sweep as the outer loop, during transient analysis. This 2-sweep loop support is exclusive to a bisection with Monte Carlo sweep.

If you add additional independent random variables to a Monte Carlo run you might see that none of these variables has any impact on the simulation run. You might see differences in statistical results between simulations with and without these additional independent random variables. The difference is due the way random values are assigned to them, not to varying the number independent random variables. In the two Monte Carlo modes discussed in the following section, the only difference is the number of independent random variables.

The following sections discuss these topics:

- [Setting Up Monte Carlo Analysis with Bisection](#)
- [Performing Bisection with Monte Carlo Sweep](#)
- [Output Files — Monte Carlo and Bisection with Monte Carlo](#)

Setting Up Monte Carlo Analysis with Bisection

Running a bisection/Monte Carlo simulation combines both existing procedures and uses enhanced syntax.

Bisection with a Monte Carlo sweep retains the setup requirements for bisection and Monte Carlo. To set up a Monte Carlo analysis, you use one of the two following PrimeSim HSPICE representation styles:

- **.Variation** statement—Specify distributions on model parameters in a Variation Block see [Options for Monte Carlo Analysis](#)
- **.PARAM** statements—Set a model or element parameter to a Gaussian, Uniform, or Limit function distribution; this statistical process description is used with analysis commands that typically include **.MEASURE** statements to calculate the response mean, variance, sigma, and standard deviation. **.DC**, **.AC**, or **.TRAN** analysis statements enable **MONTE**.

To set up a bisection run, you use the following statements.

- **.PARAM** *ParamName= OptParFun (Initial, Lower, Upper*—Defines a normal optimization specification of the parameter to be optimized.
- **.MEASURE** *Tran GOAL=GoalValue*—Sets a **GOAL** value
- **.MODEL** *optmodel OPT*—Specifies bisection method and relative input/output tolerance.
- **.TRAN** ... **SWEEP OPTIMIZE**—Performs transient analysis bisection.

Performing Bisection with Monte Carlo Sweep

When performing bisection with a Monte Carlo sweep, use one or more **.Variation** blocks or parameter statements and one or more **.MEASURE** statements for Monte Carlo and another set of **.MEASURE**, **.PARAM** and **.MODEL** *Opt* statements for bisection setup. However, only a single common **.TRAN** analysis statement is used for both analyses, plus an enhanced syntax.

.TRAN Syntax for Bisection with Monte Carlo Sweep

```
.TRAN tstep tstop [START=val]
+ SWEEP
+ OPTIMIZE=opt_par_fun|OPTxxx
+ RESULTS=measnames MODEL=optmod
+ MONTE=MCcommand
```

All arguments have the same meanings as those of the current bisection and Monte Carlo `.TRANSyntax`. For argument names and their descriptions, refer to `.TRAN` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

Note:

Monte Carlo random data generation procedure is independent of the bisection analysis results.

Example

In the following example, PrimeSim HSPICE runs 30 bisection analyses, using the 10th to 39th Monte Carlo iterations.

```
.Tran 1n 8n
+ SWEEP                      $$ keyword to enable both functions
+ Optimize = Opt1            $$ Bisection setup
+ Result = MaxVout           $$ Bisection setup
+ Model = OptMod             $$ Bisection setup
+ MONTE = 30 firstrun=10    $$ Monte Carlo setup
```

Output Files — Monte Carlo and Bisection with Monte Carlo

The following sections discuss these topics:

- [Monte Carlo Output Files](#)
- [Output List File for Bisection with Monte Carlo Sweep](#)
- [Output Measurement File](#)
- [Output Waveform File](#)

Monte Carlo Output Files

Ordinary Monte Carlo output depends on how you set the output commands. Transient Monte Carlo results are saved in measurement report files (`*.mct#`, `*.mt#`), the output list `*.lis` file, and/or the waveform output file (`*.tr#`) which superimposes all iterations as a single plot.

Note:

Output List File for Bisection with Monte Carlo Sweep

See the following sample *.lis file, where the types of information are defined on the right side as:

- Monte Carlo output/information
- Bisection output/information
- Common output/information

List File Content	Info type
<pre>Combination of output *.lis report for bisection with monte carlo.sweep Opening plot unit= 15 file=./results/inv.pa0 ***** HSPICE C-2009.03 32-BIT 11:21:02 01/16/2009 linux ***** operating point information tnom= 25.000 temp=25.000 ***** operating point status is voltage simulation time is 0. node =voltage node =voltage node =voltage +0:1 = 5.0000 0:2 = 5.0000 0:3 = 3.6493n +0:4 = 3.6493n 0:in = 0. 0:out = 5.0000 1 ***** HSPICE C-2009.03 32-BIT 11:21:02 01/16/2009 linux ***** ***** transient analysis tnom= 25.000 temp= 25.000 *** monte carlo index = 1 *** MONTE CARLO PARAMETER DEFINITIONS par1 = 2.7518E-08 par2 = -3.1778E-07 par3 = -4.1848E-08</pre>	Common output/informat ion Monte Carlo output/informat ion

List File Content	Info type
entering lmoptparm names init guess, lower, upper boundsdelaytime 0.000 0.000 5.0000E-09 0.000bisec-opt. iter = 1 xlo = 0.0000 xhi = 5.00000E-09 x = 0.0000 xnew = 5.00000E-09 err = -4.97943E-05bisec-opt. iter = 2 xlo = 0.0000 xhi = 5.00000E-09 x = 5.00000E-09 xnew = 2.50000E-09 err = 0.99997bisec-opt. iter = 11 xlo = 1.80664E-09 xhi = 1.81641E-09 x = 1.80664E-09 xnew = 1.81152E-09 err = -1.91099E-05 optimization completed, the condition relin = 1.0000E-03 is satisfied optimization completed, the conditionrelout = 1.0000E-03 is satisfied**** optimized parameters opt1.param delaytime = 1.8066n	Bisection output/informat ion
s_delay= 2.5204E-10 targ= 4.3714E-10 trig= 1.8510E-10s_power= 6.6497E-03 from= 0.0000E+00 to= 1.0000E-09m_delay= 2.5204E-10 targ= 4.3714E-10 trig= 1.8510E-10	Common output/informat ion
**** job concluded*** monte carlo index = 2 ***par1 = 2.3518E-08par2 = -3.3778E-07par3 = -4.3848E-08	Monte Carlo output/informat ion
entering lmoptparm names init guess, lower, upper boundsdelaytime 0.000 0.000 5.0000E-09 0.000bisec-opt. iter = 1 xlo = 0.0000 xhi = 5.00000E-09x = 0.0000 xnew = 5.00000E-09err = -4.97943E-05bisec-opt. iter = 2 xlo = 0.0000 xhi = 5.00000E-09 x = 5.00000E-09 xnew = 2.50000E-09 err = 0.99997....bisec-opt. iter = 11 xlo = 1.80664E-09 xhi = 1.81641E-09 x = 1.80664E-09 xnew = 1.81152E-09 err = -1.91099E-05 optimization completed, the condition relin = 1.0000E-03 is satisfied**** optimized parameters opt1.param delaytime = 1.8066n	Bisection output/informat ion
s_delay= 2.5204E-10 targ= 4.3714E-10 trig= 1.8510E-10s_power= 6.6497E-03 from= 0.0000E+00 to= 1.0000E-09m_delay= 2.5204E-10 targ= 4.3714E-10 trig= 1.8510E-10	Common output/informat ion
**** job concluded*** monte carlo index = 3 ***par1 = 2.3518E-08par2 = -3.3778E-07par3 = -4.3848E-08...meas_variable = s_delaymean = 251.0688p varian = 4.397e-22sigma = 20.9695p avgdev = 14.2926pmax = 271.5352p min = 229.6299p1-sigma = 20.9695p median = 271.5352pmeas_variable = s_powermean = 6.6620m varian = 971.5124psigma = 31.1691u avgdev = 23.6264umax = 6.6974m min = 6.6388m1-sigma = 31.1691u median = 6.6974mmeas_variable = m_delaymean = 251.0688p varian = 4.397e-22sigma = 20.9695p avgdev = 14.2926pmax = 271.5352p min = 229.6299p1-sigma = 20.9695p median = 271.5352p....	Monte Carlo output/informat ion
**** job concluded***** HSPICE C-2009.03 32-BIT (Jan 16 2009) 11:21:02 01/16/2009 linux***** job statistics summary tnom= 25.000 temp= 25.000 ***** total memory used 167 kbytes# nodes = 23 # elements= 10# diodes= 0 # bjts = 0 # jfets = 0 # mosfets = 8# va device = 0analysis time # points tot. iter conv.iterop point 0.02 1 24transient 0.00 153 219 76 rev=1readin 0.00errchk 0.01setup 0.00output 0.00 total cpu time 0.02 seconds job started at 11:21:02 01/16/2009 job ended at 11:21:04 01/16/2009	Common output/informat ion

Output Measurement File

The output measurement file has the same information recorded based on the .MEAS statements for either bisection analysis or the Monte Carlo sweep. The measure file is one

set of results per Monte Carlo sample; and the results are for the optimized value from the bisection analysis. The bisection measure results include the optimized parameter and the measure results.

In the following example, `delaytime` is the parameter optimized, `setuptime` is the measurement result with this `delaytime`.

```
.TITLE '*****'  
index rbl@rblx delaytime setuptime  
temper alter#  
1.0000 2.100e+04 5.982e-10 4.018e-10  
25.0000 1.0000  
2.0000 1.900e+04 3.448e-10 6.552e-10  
25.0000 1.0000  
...
```

Note:

If a particular Monte Carlo run fails, either due to Bisection failure or any other reason, PrimeSim HSPICE outputs a 'failed' flag in the *.mt# file, and continues with the next Monte Carlo run. The failed point is not included in the summary statistical computations.

If a Monte Carlo simulation terminates when one sample is failed, you can use `.option MONTECON=1` (the default) to force the simulation to continue under this condition.

Output Waveform File

The output waveform file superimposes all iterations of the Monte Carlo sweep as a single plot, while each iteration simulation of Monte Carlo uses the optimized parameter from Bisection.

Bisection Optimization Application Example

```
.model optMod1 OPT METHOD=BISECTION  
.param VDx = optFunc1(1.0,1.0,1.4)  
.option OPTLST = 1  
.measure optMeasure TRIG v(clk) VAL='0.5*VDx' RISE=2 TARG v(q)  
+ VAL='0.5*VDx' RISE=1 GOAL=60p  
.tran 0.01n 10n SWEEP OPTIMIZE=optFunc1 RESULTS=optMeasure  
+ MODEL=optMod1
```

Caution:

- A simple `.param VDx = 1.1` statement defined after `.param VDx = optFunc1(...)` will overwrite the optimization function `optFunc1`.
- A voltage source with the name `VDx` must not coexist with the parameter `VDx`. Otherwise, the DC value of the voltage source `VDx` will be varied while the parameter `VDx` is left unchanged.

24

Variation Analysis

Describes the statistical analysis features supported by PrimeSim HSPICE.

The advanced sampling capabilities are described in this chapter. The features described in this chapter differ from, and are generally a subset of, the enhanced statistical analysis features described in [Chapter 25, Statistical Representation Using the Variation Block](#), and [Chapter 26, Monte Carlo Analysis—Flow and Outputs](#).

PrimeSim HSPICE ships hundreds of examples for your use. For paths to the demo files, see Listing of Demonstration Input Files in the *PrimeSim HSPICE User Guide: Demonstration Netlists*. To find traditional Monte Carlo demonstration files, see [Variability Examples](#).

For information on bisection in conjunction with Monte Carlo, see [Chapter 23, Timing Analysis Using Bisection](#).

This chapter discusses the following topics:

- [Overview of Variation on Silicon](#)
- [Application of Statistical Analysis](#)
- [Analytical Model Types](#)
- [Traditional Monte Carlo Simulations](#)
- [Worst-case Analysis](#)
- [Traditional Monte Carlo Analysis Characteristics](#)
- [Traditional and Variation Block Monte Carlo](#)
- [Monte Carlo Examples](#)
- [Monte Carlo Results](#)
- [Troubleshooting Monte Carlo Issues](#)

Overview of Variation on Silicon

As semiconductor technologies migrate to ever-smaller geometries, larger relative variations in device characteristics are being observed. These fluctuations in device characteristics have been analyzed and classified for the purpose of dealing with the variations in manufacturing during the design phase. The following types of variations can be identified at the wafer level:

- Global variations from foundry, lot, or due to wafer processing.
- Across-wafer variations due to materials, gas flow, thermal, optical, and spin processes.
- Linear variations across the area of a chip.
- Local variations are observed between closely spaced identically designed devices as a result of microscopic random processes. Microscopic variations include line edge roughness, finite number of dopant atoms in the channel, and atomic level oxide thickness changes.

In analog design, certain circuit characteristics can be made insensitive to global variations and across-chip variations by applying the concept of matched devices; however, these characteristics are still affected by the local variations. In digital designs for nanometer technologies, large local variations can cause unacceptable variations in path delays and signal slopes.

Large circuits suffer from spatial or position-dependent variations, which create problems with clock skew for devices that are far apart. Finally, device characteristics are affected by features in proximity (metal coverage, fill patterns, mechanical strain, shape variation due to lithography, and so on) and orientation. Most of these variations are systematic and can be reduced through layout restrictions or accounted for in post-layout verification.

Historically, only the effects of variation on device characteristics (transistors, resistors, and capacitors) have been considered. In nanometer technologies, variations in the interconnect should also be taken into account because the relative variation in the resistance and capacitance has increased due to smaller wire width and inter-conductor spacing.

These variations combined, summarized as *parametric variability*, dominate yield loss in nanometer technologies. The circuits function in terms of connectivity, but do not meet specifications on metrics such as speed, leakage, or offset. For example, while the threshold of MOS devices gets smaller, approaching 200 mV, the variation in threshold gets larger, with standard deviation up to 30 mV for short devices. Due to the low supply voltages, in combination with requirements for high speed, the circuits stop working with these large spreads in device characteristics.

Therefore, simulating (or predicting) the effects of these variations on circuit response is increasingly important, in particular when considering the high mask costs and time-to-market constraints for the majority of today's products.

To simulate the effects of the variations in device characteristics due to materials and manufacturing, they need to be described in a way that the simulator can handle in an efficient manner. Traditionally, global variations were specified through process corner files, and the other types of variations mentioned above were either guessed or ignored. In recent years, statistics blocks were added to the model files. They describe variations in terms of distributions on device model parameters. An even newer approach for defining variations is the Variation Block, described later in this chapter. The following analyses are available in PrimeSim HSPICE to simulate the effects of variations on circuit response:

- Monte Carlo analysis is the traditional method for finding the variation in circuit response resulting from parameter variations.
- DC and AC mismatch analyses are efficient methods for simulating the effects of variations on a circuit's DC or AC response.

To get satisfactory answers from these analyses, the variation definitions must have been generated for the target technology of the design, similar to device models.

Application of Statistical Analysis

When you design an electrical circuit, it must meet tolerances for the specific manufacturing process. The electrical yield is the number of parts that meet the electrical test specifications. Overall process efficiency requires maximum yield. To analyze and optimize the yield, PrimeSim HSPICE supports statistical techniques and observes the effects of variations in element and model parameters.

The basic functionality of Monte Carlo analysis is to simulate the effects of variations on circuit performance. When measurements are executed, the results from all the samples form a distribution. This distribution has characteristics that can be described in statistical terms: mean, standard deviation, and so on. The statistical terms are calculated by the simulator and reported at the end of the run listing.

The main reason to run Monte Carlo analysis is to find out whether the circuit will still have acceptable yield if it is subject to variations. Comparing the statistical results reported by the simulator can answer this question. However, in many cases the shape of the distribution and how it relates to the specification are of interest. For visual inspection, it is useful to create a histogram. The histogram helps to determine whether the variations in the performance look *normal* by considering, not only the bell shape of the Normal distribution, but also unexpected behavior like outliers and gaps. The next question typically is whether the distribution is well centered with respect to the specification, and what is the predicted yield. Another question in this context is whether the circuit is over-designed—meaning that the margins are too big for the particular characteristic.

This might be at the expense of secondary properties, like power and area, which affect cost. These are the more traditional ways of looking at the results from Monte Carlo analysis. However, more information can be accessed with more sophisticated tools. (See [Chapter 25, Statistical Representation Using the Variation Block](#) and [Chapter 26, Monte Carlo Analysis—Flow and Outputs](#).)

Analytical Model Types

For statistical variation analysis of circuit behavior, foundries can model variation in data using one of the following methods:

- Investigate the performance of a circuit with change in circuit parameters.

For defining variations on parameters, use `.PARAM` statements. For example:

```
.param var=AGAUSS(20,1.2,3)
```

See `.PARAM` in *PrimeSim HSPICE® Reference Manual: Commands and Control Options* for details on this command.

- To specify global, local, and spatial variations on model parameters, resulting from variations in materials and manufacturing, define a Variation block using the `.VARIATION` command.

For example:

```
.Variation  
    Define global and local variation definitions  
.End_Variation
```

For complete list and details of the `.Variation` block control options, see Enhanced Monte Carlo Control Options in *PrimeSim HSPICE® Reference Manual: Commands and Control Options*.

Use Monte Carlo analysis when you know the statistical standard deviations of component values to center a design. This provides maximum process yield and determines component tolerances. For Monte Carlo specific `.Variation` block control options, see Enhanced Monte Carlo Control Options in *PrimeSim HSPICE® Reference Manual: Commands and Control Options*.

- Defining variations on models using `lot` and `dev` parameters in the model file. For example:

```
vth0=0.6 lot/0.1 dev/0.02
```

To analyze the statistical block results, use `.MEASURE` statements to save results for delay times, power, or any other characteristic extracted in a `.MEASURE` statement. PrimeSim HSPICE generates a table of results in an `.mt#` file in ASCII format. You can analyze the numbers directly or read this file into WaveView to view the distributions.

If you use `.MEASURE` statements in a Monte Carlo or data-driven analysis, then the PrimeSim HSPICE output file includes the following outputs in the listing:

- Mean $\frac{x_1 + x_2 + \dots + x_n}{N}$
- Variance $\frac{(x_1 - \text{Mean})^2 + \dots + (x_n - \text{Mean})^2}{N-1}$
- Sigma $\sqrt{\text{Variance}}$
- Average Deviation $\frac{|x_1 - \text{Mean}| + \dots + |x_n - \text{Mean}|}{N-1}$

Other types of variation analysis (non-statistical analysis) include circuit and temperature analysis, and worst-case analysis.

Traditional Monte Carlo Simulations

This section provides a high-level overview of PrimeSim HSPICE traditional Monte Carlo analysis. Later sections provide in-depth information.

The basic premise of a Monte Carlo analysis is that you are going to parameterize one or more circuit variables, vary those values by a randomized amount from the norm, and run PrimeSim HSPICE a predetermined number of times. Each run is called a *sweep* and will generate tabular or plot data as specified by the user. Measurements are also typically used to look at circuit operating conditions from run to run.

You can randomize anything that can be set with a parameter or variable. Examples include things as diverse as a simple resistor value, a model parameter for a MOSFET, or the length of a transmission line.

Values can be varied using three basic statistical variations: uniform, limit, and Gaussian. Using those methods, you choose the nominal value and the absolute or relative variation. You can optionally supply the standard deviation and a multiplier.

Note that PrimeSim HSPICE does not run a nominal simulation with traditional Monte Carlo. For traditional Monte Carlo, the first sample is index 1; it is not the nominal value. For example: `monte=10`, you get 10 samples (1-10).

However, you can run Monte Carlo using the Variation Block features (where the first run is the nominal case) with AGAUSS (traditional) style definitions. See [Chapter 26, Monte Carlo Analysis—Flow and Outputs](#).

The following sections discuss these topics:

- [Basic Syntax](#)
- [Local and Global Parameter Variation](#)
- [Exception for Model Parameters](#)
- [Starting Values and Seeds](#)
- [Other Monte Carlo Control Options](#)
-

Basic Syntax

The basic syntax of a Monte Carlo analysis includes three elements:

1. Defining a parameter with one of the distribution keywords
2. Using the parameter in your netlist as the value for an element or model parameter
3. Including the SWEEP and MONTE keywords in the analysis statement

Consider the following example. In this simple RC charging circuit, the value of `r_one` has a nominal value of 1K and is varied by 400 ohms for 10 iterations.

RC charging circuit:

```
.option post probe
*define a parameter called "resval" with an absolute, uniform
distribution
.param resval=aunif(1000,400)
vsrc_one 1 0 5v
r_one 1 2 resval
c_one 2 0 1u
.ic 2=0
*specify 10 Monte Carlo iterations
.tran 1e-5 5e-3 sweep monte=10
*measure to find when 1 time constant (.632*vdd) occurs
.meas tran tc when v(2)='.632*5'
*create plots of the charging curve and resistor values
.probe v(2) par(resval)
.end
```

The resulting waveforms are called *multi-member*. Plotting one signal displays the curves from all the runs.

Local and Global Parameter Variation

A common source of confusion is local and global parameter variation. The key is that each time you use a parameter, it is assigned a new random value. In this case, the resistors are said to be under local variation. The following examples clarify this:

```
.param resval=aunif(1000,400)
r_one 1 2 resval
r_two 2 3 resval
r_three 3 4 resval
```

In this case, all three resistors are given unique, random values. (However, as discussed in the next section, there is an exception to this rule.) If you want to set a group of components to the same random value, first assign an intermediate parameter:

```
.param resval=aunif(1000,400)
.param my_resval=resval
r_one 1 2 my_resval
r_two 2 3 my_resval
r_three 3 4 my_resval
```

In the second example, the assignment of a random value is only done once, then used three times. In this case, the resistors are said to be under global variation.

Exception for Model Parameters

Because a model definition is only done once, the behavior described in the previous section would assign the same parameter value to all devices referencing that model. To overcome this, `.OPTION MODMONTE` lets you decide if all instances of a device should get the same or unique model parameters.

Starting Values and Seeds

Another source of confusion is the starting value. If you run the same Monte Carlo simulation twice, the results will be identical because PrimeSim HSPICE always uses the same “seed” value for the first run. If it randomized the seed by default, it would be difficult to determine whether changes you made to the circuit and topology were the result of your changes or the new random values. If desired, you can specify a seed or have PrimeSim HSPICE select a random seed with `.OPTION SEED`.

Other Monte Carlo Control Options

- `.OPTION MONTECON` – Some random parameter assignments can cause PrimeSim HSPICE not to converge. This parameter is used to decide whether to terminate a simulation or move to the next run if convergence fails.
- `.OPTION RANDGEN` – Use this option to specify the type of random number generator used.
- `.OPTION MCBRIEF` – This option controls how PrimeSim HSPICE outputs Monte Carlo parameters and generates or suppresses output files.

See PrimeSim HSPICE Netlist Simulation Control Options in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

Monte Carlo Analysis in HPP

From the O-2018.09 release, HPP no longer supports Monte Carlo analysis. The PrimeSim HSPICE simulator switches to the traditional transient engine when Monte Carlo sweep is defined in the netlist.

Worst-case Analysis

Circuit designers often use worst-case analysis when designing and analyzing MOS and BJT IC circuits. To simulate the worst case, set all variables to their 2- or 3-sigma worst-case values. Because several independent variables rarely attain their worst-case values simultaneously, this technique can lead to over-designing the circuit. However, this analysis is useful as a fast check.

Worst-case corner analysis is used when you know the component value limit to automate quality assurance for:

- Basic circuit function
- Process extremes
- Quick estimation of speed and power tradeoffs
- Best-case and worst-case model selection
- Parameter corners
- Library files

The following sections discuss these topics:

- [Model Skew Parameters](#)
- [Using Skew Parameters](#)
- [Skew File Interface to Device Models](#)

Model Skew Parameters

The PrimeSim HSPICE device models include physically measurable model parameters. The circuit simulator uses parameter variations to predict how an actual circuit responds to extremes in the manufacturing process. Physically measurable model parameters are called *skew parameters* because they skew from a statistical mean to obtain predicted performance variations.

Examples of skew parameters are the difference between the drawn and physical dimension of metal, postillion, or active layers on an integrated circuit.

Generally, you specify skew parameters independently of each other, so you can use combinations of skew parameters to represent worst cases. Typical skew parameters for CMOS technology include:

- XL – Polysilicon CD (critical dimension of the poly layer, representing the difference between drawn and actual size).
- XW_n, XW_p – Active CD (critical dimension of the active layer, representing the difference between drawn and actual size).
- TOX – Thickness of the gate oxide.
- RSH_n, RSH_p – Resistivity of the active layer.
- $DELVTO_n, DELVTO_p$ – Variation in threshold voltage.

You can use these parameters in any level of MOS model within the PrimeSim HSPICE device models. The $DELVTO$ parameter shifts the threshold value. PrimeSim HSPICE adds this value to VTO for the Level 3 model and adds or subtracts it from $VFB0$ for the BSIM model. [Table 66](#) shows whether PrimeSim HSPICE adds or subtracts deviations from the average.

Table 66 Sigma Deviations

Type	Parameter	Slow	Fast
NMOS	XL	+	-
	RSH	+	-
	DELVTO	+	-

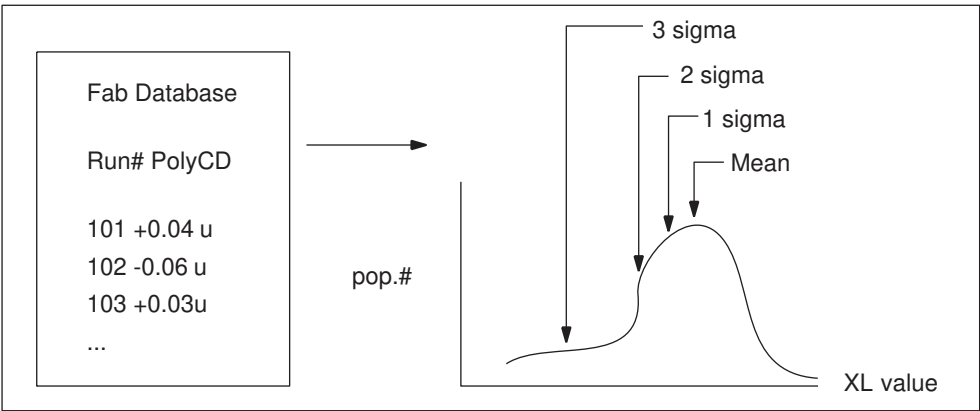
Table 66 *Sigma Deviations (Continued)*

Type	Parameter	Slow	Fast
PMOS	TOX	+	-
	XW	-	+
	XL	+	-
	RSH	+	-
	DELVTO	-	+
	TOX	+	-
	XW	-	+

PrimeSim HSPICE selects skew parameters based on the available historical data that it collects either during fabrication or electrical test. For example, PrimeSim HSPICE collects the *XL skew* parameter for poly CD during fabrication. This parameter is usually the most important skew parameter for a MOS process.

[Figure 110 on page 481](#) is an example of data that historical records produce.

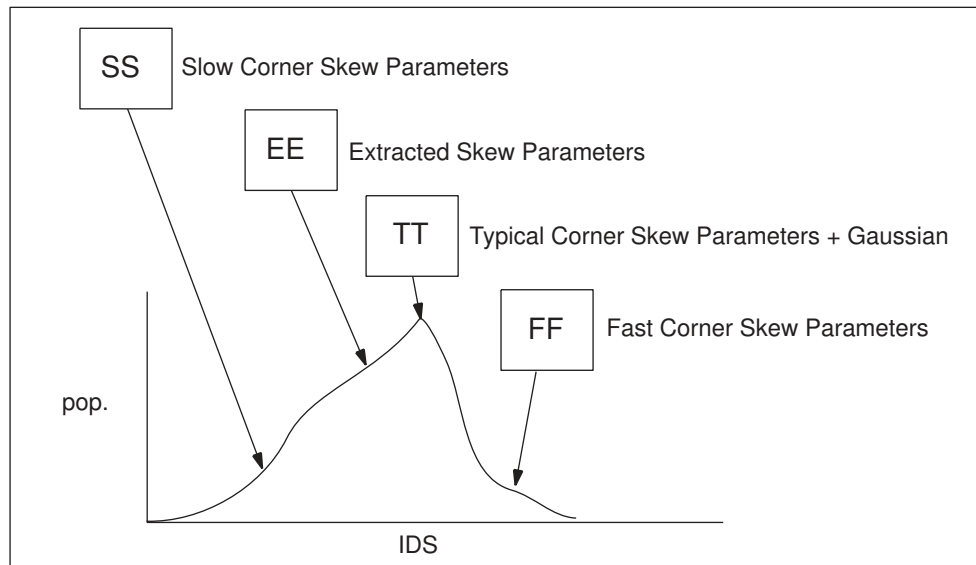
Figure 110 *Historical Records for Skew Parameters in a MOS Process*



Using Skew Parameters

[Figure 111](#) shows how to create a worst-case corners library file for a CMOS process model. Specify the physically measured parameter variations so that their proper minimum and maximum values are consistent with measured current (*IDS*) variations. For example, PrimeSim HSPICE can generate a 3-sigma variation in *IDS* from a 2-sigma variation in physically measured parameters.

Figure 111 Worst-Case Corners Library File for a CMOS Process Model



The `.LIB` (library) statement and the `.INCLUDE` or `.INC` (include file) statement access the models and skew. The library contains parameters that modify `.MODEL` statements. The following example of `.LIB` features both worst-case and statistical-distribution data by using model skew parameters. In statistical distribution, the median value is the default for all non-Monte Carlo analysis.

Example

```
.LIB TT
$TYPICAL P-CHANNEL AND N-CHANNEL CMOS LIBRARY DATE:3/4/91
$ PROCESS: 1.0U CMOS, FAB22, STATISTICS COLLECTED 3/90-2/91
$ following distributions are 3 sigma ABSOLUTE GAUSSIAN

.PARAM
$ polysilicon Critical Dimensions
+ polycd=AGAUSS(0,0.06u,1) xl='polycd-sigma*0.06u'
$ Active layer Critical Dimensions
+ nactcd=AGAUSS(0,0.3u,1) xwn='nactcd+sigma*0.3u'
+ pactcd=AGAUSS(0,0.3u,1) xwp='pactcd+sigma*0.3u'
$ Gate Oxide Critical Dimensions (200 angstrom +/- 10a at 1
$ sigma)
+ toxcd=AGAUSS(200,10,1) tox='toxcd-sigma*10'

$ Threshold voltage variation
+ vtoncd=AGAUSS(0,0.05v,1) delvton='vtoncd-sigma*0.05'
+ vtopcd=AGAUSS(0,0.05v,1) delvtop='vtopcd+sigma*0.05'

.INC '/usr/meta/lib/cmos1_mod.dat' $ model include file
.ENDL TT
.LIB FF
```

```
$HIGH GAIN P-CH AND N-CH CMOS LIBRARY 3SIGMA VALUES

.PARAM TOX=230 XL=-0.18u DELVTON=-.15V DELVTO= 0.15V
.INC '/usr/meta/lib/cmos1_mod.dat' $ model include file

.ENDL FF
```

The `/usr/meta/lib/cmos1_mod.dat` include file contains the model.

```
.MODEL NCH NMOS LEVEL=2 XL=XL TOX=TOX DELVTO=DELVTON . .
.MODEL PCH PMOS LEVEL=2 XL=XL TOX=TOX DELVTO=DELVTO . .
```

Note:

The model keyname (left) equals the skew parameter (right). Model keys and skew parameters can use the same names.

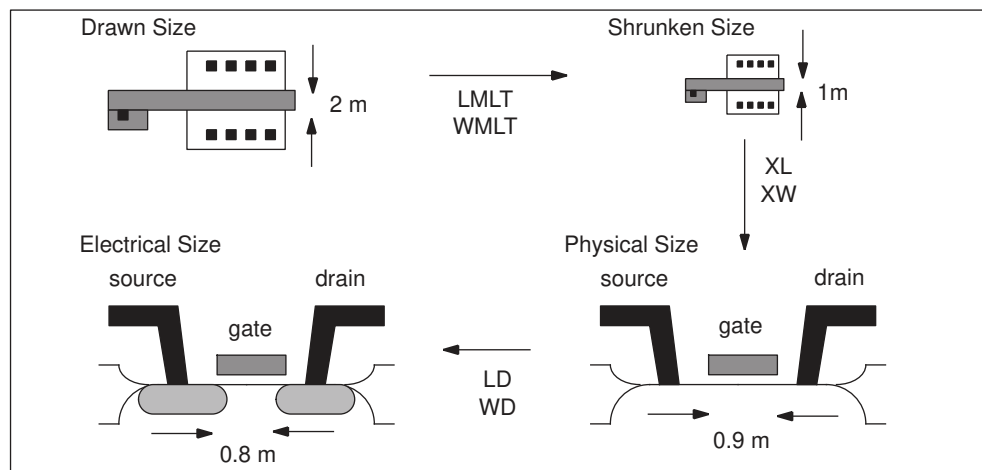
Skew File Interface to Device Models

Skew parameters are model parameters for transistor models or passive components. A typical device model set includes:

- MOSFET models for all device sizes by using an automatic model selector.
- RC wire models for polysilicon, metal1, and metal2 layers in the drawn dimension. Models include temperature coefficients and fringe capacitance.
- Single-diode and distributed-diode models for N+, P+, and well (includes temperature, leakage, and capacitance based on the drawn dimension).
- BJT models for parasitic bipolar transistors. You also can use these for any special BJTs, such as a BiCMOS for ECL BJT process (includes current and capacitance as a function of temperature).
- Metal1 and metal2 transmission line models for long metal lines.
- Models must accept elements. Sizes are based on a drawn dimension. If you draw a cell at 2- μ dimension and shrink it to 1 μ , the physical size is 0.9 μ . The effective electrical size is 0.8 μ . Model scale factors account for the four dimension levels:
 - Drawn size
 - Shrunk size
 - Physical size
 - Electrical size

Most simulator models scale directly from *drawn* to *electrical* size. PrimeSim HSPICE MOS models support all four size levels ([Figure 112](#)).

Figure 112 Device Model from Drawn to Electrical Size



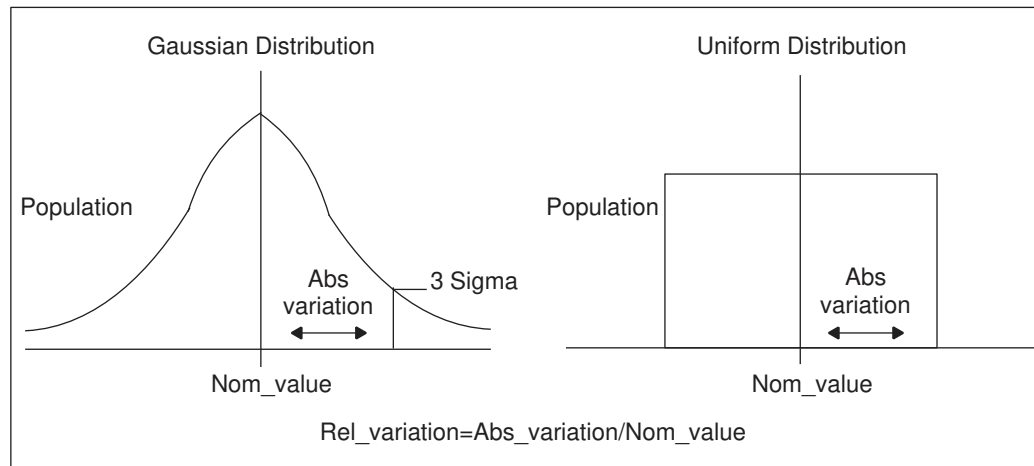
Traditional Monte Carlo Analysis Characteristics

Monte Carlo analysis uses a random number generator to create the following types of functions:

- Gaussian parameter distribution
 - Relative variation – Variation is a ratio of the average.
 - Absolute variation – Adds variation to the average.
 - Bimodal – Multiplies distribution to statistically reduce nominal parameters.
- Uniform parameter distribution
 - Relative variation – Variation is a ratio of the average.
 - Absolute variation – Adds variation to the average.
 - Bimodal – Multiplies distribution to statistically reduce nominal parameters.
- Random limit parameter distribution
 - Absolute variation – Adds variation to the average.
 - Monte Carlo analysis randomly selects the *min* or *max* variation.

The value of the `MONTE` analysis keyword determines how many times to perform operating point, DC sweep, AC sweep, or transient analysis.

Figure 113 Monte Carlo Distribution



The following sections discuss these topics:

- [Monte Carlo Setup](#)
- [Monte Carlo Output](#)
- [.PARAM Distribution Function](#)
- [Monte Carlo Parameter Distribution](#)
- [Global and Local Variations with Monte Carlo](#)
- [Non-Gaussian Probability Distribution Functions](#)

Monte Carlo Setup

To set up a Monte Carlo analysis, use the following PrimeSim HSPICE statements:

- **.PARAM** statement – Sets a model or element parameter to a Gaussian, Uniform, or Limit function distribution.
- **.DC**, **.AC**, or **.TRAN** analysis – Enables MONTE.
- **.MEASURE** statement – Calculates the output mean, variance, sigma, and standard deviation.
- **.MODEL** statement – Sets model parameters to a Gaussian, Uniform, or Limit function distribution.

Select the type of analysis to run, such as operating point, DC sweep, AC sweep, or TRAN sweep.

- Operating Point

```
.PARAM dummy=1  
.DC dummy 1 1 1 MONTE=[num] firstrun=[num]
```

or:

```
.DC MONTE=list[()] [num1:num2] [num3] [num5:num6] [num7] [()]
```

- DC Sweep

```
.DC vin 1 5 0.25 sweep MONTE=val [firstrun=num1]
```

or:

```
.DC vin 1 5 0.25 sweep MONTE=list[()] [num1:num2] [num3]  
+ [num5:num6] [num7] [()]
```

- AC Sweep

```
.AC dec 10 100 1meg sweep MONTE=val [firstrun=num1]
```

or:

```
.AC vin 1 5 0.25 sweep MONTE=list[()] [num1:num2] [num3]  
+ [num5:num6] [num7] [()]
```

- TRAN Sweep

```
.TRAN 1n 10n sweep MONTE=val [firstrun=num1]
```

or:

```
.TRAN vin 1 5 0.25 sweep MONTE=list[()] [num1:num2] [num3]  
+ [num5:num6] [num7] [()]
```

The *val* value specifies the number of Monte Carlo iterations to perform. A reasonable number is 30. The statistical significance of 30 iterations is high. If the circuit operates correctly for all 30 iterations, there is a 99 percent probability that over 80 percent of all possible component values operate correctly. The relative error of a quantity, determined through Monte Carlo analysis, is proportional to $val^{-1/2}$.

The *firstrun* values specify the desired number of iterations. PrimeSim HSPICE runs from *num1* to *num1+val-1*. The number after *firstrun* can be a parameter. You can write only one number after *list*. The colon represents "from ... to ...". Specifying only one number makes PrimeSim HSPICE run only at the one specified point.

Examples

Example 1: PrimeSim HSPICE runs from the 90th to 99th Monte Carlo iteration:

```
.tran ln 10 sweep monte=10 firstrun=90
```

You can write more than one number after `list`. The colon represents “from ... to ...”. Specifying only one number makes PrimeSim HSPICE run only at that single point.

Example 2: PrimeSim HSPICE begins running at the 10th iteration, then continues from the 20th to the 30th, then jumps to the 40th, and finally, runs from the 46th to 72nd Monte Carlo iteration.

```
.tran ln 10n sweep monte=list(10 20:30 40 46:72)
```

Example 3: `Firstrun` option, where the number of samples is 1000.

```
.tran ... sweep monte =1000, firstrun = 10
```

Example 4: `List` option, where the number of samples is 100.

```
.tran ... sweep monte = list(10:100)
```

Monte Carlo Output

The following probing commands provide these benefits:

- `.MEASURE` statements are the most convenient way to summarize the results.
- `.PRINT` statements generate tabular results and print the values of all Monte Carlo parameters.
- `.OPTION MCBRIEF` determines the output types of the random parameters during Monte Carlo analysis to improve output performance; select from several values for different results. See `.OPTION MCBRIEF` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.
- If one iteration is out of specification, you can obtain the component values from the tabular listing. A detailed re-simulation of that iteration might help identify the problem.
- WaveView superimposes all iterations as a single plot so you can analyze each iteration individually.

.PARAM Distribution Function

This section describes how to assign a `.PARAM` parameter in Monte Carlo analysis. For a general description of the `.PARAM` statement, see the `.PARAM` command in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

You can assign a `.PARAM` parameter to the keywords of elements and models and assign a distribution function to each `.PARAM` parameter. PrimeSim HSPICE recalculates the distribution function each time an element or model keyword uses a parameter. When you use this feature, Monte Carlo analysis can use a parameterized schematic netlist without additional modifications.

The following sections discuss these topics:

- [Syntax](#)
- [Examples](#)

Syntax

```
.PARAM xx=UNIF(nominal_val, rel_variation)
.PARAM xx=AUNIF(nominal_val, abs_variation)
.PARAM xx=GAUSS(nominal_val, rel_variation, [num_sigmas])
.PARAM xx=AGAUSS(nominal_val, abs_variation, [num_sigmas])
.PARAM xx=LIMIT(nominal_val, abs_variation)
.PARAM xx=LOGNORMAL(nominal_val, abs_variation, [num_sigmas])
.PARAM xx=CDF(x1, y1, ..., xn, yn)
```

Argument	Description
xx	Distribution function calculates the value of this parameter.
UNIF	Uniform distribution function by using relative variation.
AUNIF	Uniform distribution function by using absolute variation.
GAUSS	Gaussian distribution function by using relative variation.
AGAUSS	Gaussian distribution function by using absolute variation
LIMIT	Random-limit distribution function by using absolute variation. Adds +/- <i>abs_variation</i> to <i>nominal_val</i> based on whether the random outcome of a -1 to 1 distribution is greater than or less than 0.
nominal_val	The nominal (mean, average, or center) value of the distribution function. Non-Monte Carlo analyses use this value as the default.
abs_variation	Specifies the absolute variation about the nominal value for the distribution function. • AUNIF distributions will vary about nominal_val by +/- abs_variation. • AGAUSS distributions will vary about the nominal_val according to the number of standard deviations (num_sigmas) this absolute variation represents.
rel_variation	Specifies the relative variation about the nominal value for the UNIF and GAUSS distribution functions. • UNIF distributions will vary about the nominal value by +/- (nominal_val*rel_variation). • GAUSS distributions will vary about the nominal value with nominal_val*rel_variation being equal to the number of standard deviations given by num_sigmas.

Argument	Description
num_sigmas	An optional argument. Describes Gaussian distributions by a mean value (nominal_val) and a standard deviation (sigma) value. Therefore the specified abs_variation or rel_variation must be converted into a standard deviation value. The num_sigmas parameter allows you to specify how many standard deviations (sigmas) are represented by the abs_variation or rel_variation value. With num_sigmas=1, the variation specified corresponds to one standard deviation (one sigma). - For GAUSS, the standard deviation of the Gaussian distribution function is given by: $\text{Sigma} = \text{nominal_val} * \text{rel_variation} / \text{num_sigmas}$ - For AGAUSS, the standard deviation of the Gaussian distribution function is given by: $\text{Sigma} = \text{abs_variation} / \text{num_sigmas}$ Set num_sigmas=1 if you wish to specify variations in terms of root mean square (rms) values. Set num_sigmas to larger values to specify variations in terms of peak or peak-to-peak values with a certain error tolerance. For example, with num_sigmas=6.180, your input variation value will represent a peak-to-peak variation with 99.9 percent probability. With num_sigmas=9.507, your input variation value will represent a peak-to-peak variation with 99.9999 percent probability. If not specified, num_sigmas defaults to a value of 3.
LOGNORMAL	Lognormal distribution function by using absolute variation. Its logarithm is normally distributed and equal to AGAUSS(nominal_val, abs_variation, [num_sigmas]).
CDF	User-defined cumulative distribution function. A cumulative distribution function can be approximated by a piecewise linear function, which can be described as a sequence of pairs of points $[x_i, y_i]$. Here, x_i is monotonically increasing, and y_i is monotonically non-decreasing with $y_1=0$ and $y_n=1$.

Examples

Example 1

In this example, each resistor has a unique variation:

```
.param mc_var=AGAUSS(0,1,3)    $ +/-1 absolute swing or
                                $ +/-100% relative swing
.param val='1000*(1+mc_var)'
v_vin vin 0 dc=1 ac=.1
r1 vin 0 '1000*(1+mc_var)'
r2 vin 0 '1000*(1+mc_var)'
```

Example 2

In this example, each resistor has an identical variation:

```
.param mc_var=AGAUSS(0,1,3)    $ +/- 20% swing
.param val='1+mc_var'
v_vin vin 0 dc=1 ac=.1
r1 vin 0 '1000*val'
r2 vin 0 '1000*val'
```

Example 3

In this example, local variations are applied to an instance parameter by assigning randomly generated variations directly to each instance parameter. Each resistor r1 through r3 receives randomly different resistance values during each Monte Carlo run.

```
.param r_local=AGAUSS(...)
r1 1 2 r=r_local
r2 3 4 r=r_local
r3 5 6 r=r_local
```

Example 4

In this example, global variations are applied to an instance parameter by assigning the variation to an intermediate parameter before assigning it to each instance parameter. Each resistor r1 through r3 receives the same random resistance value during each Monte Carlo run.

```
.param r_random=AGAUSS(...)
.param r_global=r_random
r1 1 2 r=r_global
r2 3 4 r=r_global
r3 5 6 r=r_global
```

Monte Carlo Parameter Distribution

Each time you use a parameter, Monte Carlo analysis calculates a new random variable.

- If you do not specify a Monte Carlo distribution, PrimeSim HSPICE assumes the nominal value.
- If you specify a Monte Carlo distribution for only one analysis, PrimeSim HSPICE uses the nominal value for all other analyses.

You can assign a Monte Carlo distribution to all elements that share a common model. The actual element value varies according to the element distribution. If you assign a Monte Carlo distribution to a model keyword, then all elements that share the model use the same keyword value. You can use this feature to create double element and model distributions.

For example, the MOSFET channel length varies from transistor to transistor by a small amount that corresponds to the die distribution. The die distribution is responsible for offset voltages in operational amplifiers and for the tendency of flip-flops to settle into random states. However, all transistors on a die site vary according to the wafer or fabrication run distribution. This value is much larger than the die distribution, but affects all transistors the same way. You can specify the wafer distribution in the MOSFET model to set the speed and power dissipation characteristics.

Global and Local Variations with Monte Carlo

Monte Carlo analysis is dependent on a method to describe variability. Four different approaches are available in PrimeSim HSPICE:

1. Specify distributions on parameters and apply these to instance parameters.
2. Specify distributions on parameters and apply these to model parameters.
3. Specify distributions on model parameters using `DEV/LOT` construct.
4. Specify distributions on model parameters in a Variation Block (VB).

Note:

The `.MALIAS` command is supported for diode, BJT, JFET, and MOSFET models in `.Global_Variation` and `.Local_Variation` blocks.

In the following sections, the first three methods are described. The description relies on test cases, which can be found in the tar file `monte_test.tar` in directory `$installdir/demo/hspice/variability`.

The VB is described in [Chapter 25, Statistical Representation Using the Variation Block](#), and Monte Carlo analysis controlled by the VB is described in [Chapter 26, Monte Carlo Analysis—Flow and Outputs](#).

The following sections discuss these topics:

- [Key to Demonstration Examples for Monte Carlo](#)
- [Variation on Model Parameters as a Function of Device Geometry](#)

Key to Demonstration Examples for Monte Carlo

The following sections discuss sample files delivered with PrimeSim HSPICE. See Variability Examples in the *PrimeSim HSPICE User Guide: Demonstration Netlists*.

`monte_test.tar` is a suite of DC test files named `test1.sp` through `test20.sp` to test combinations of resistors, subckts, model/instance parameters, and so on.

The following sections discuss these demonstration files:

- [Variations Specified on Geometrical Instance Parameters](#)
- [Variations Specified in the Context of Subcircuits](#)
- [Variations on a Model Parameter Using a Local Model in Subcircuit](#)
- [Indirect Variations on a Model Parameter](#)
- [Variations Specified on Model Parameters](#)

- [Local Variations for Transistor Fingers](#)
- [Variations Specified Using DEV and LOT](#)
- [Combinations of Variation Specifications](#)

Variations Specified on Geometrical Instance Parameters

This method consists of defining parameters with variation using the distribution functions `UNIF`, `AUINF`, `GAUSS`, `AGAUSS`, and `LIMIT`. These parameters are then used to generate dependent parameters or to replace instance parameters. In a Monte Carlo simulation, at the beginning of each sample, new random values are calculated for these parameters. For each reference, a new random value is generated; however, no new value is generated for a derived parameter. Therefore, it is possible to apply independent variations to parameters of different devices, as well as the same variation to parameters of a group of devices. Parameters that describe distributions can be used in expressions, thus it is possible to create combinations of variations (correlations).

These concepts are best explained with circuit examples. In the following three examples, variation is defined on the width of a physical resistor, which has a model. If this device was a polysilicon resistor for example, then the variations describe essentially the effects of photoresist exposure and etching on the width of the poly layer.

- `test1.sp` has a distribution parameter defined called `globw`. A parameter called `globwidth` is assigned the value of `globw`. The parameter `globwidth` is assigned a different random value for each Monte Carlo sample. The parameter `globwidth` is used to define the width of the physical resistors `r1`, `r2`, `r3`, and `r4`, with model "resistor." Because parameter `globwidth` does not have its own distribution defined, but rather gets its value from the parameter `globw`, the value for `globwidth` is the same wherever it is used. The resistors have the same width for each Monte Carlo sample, and, therefore, the same resistance. When plotting the simulation results `v1`, `v2`, `v3`, and `v4` from the `.meas` file, the waveforms overlay perfectly. This type of setup is typically used to model global variations that affect all devices the same way.
- `test2.sp` has a distribution parameter defined called `locwidth`. This parameter is used to define the width of the physical resistors `r1`, `r2`, `r3`, and `r4`, with model "resistor." Because the parameter has its own distribution defined, its value is different for each reference, and for each Monte Carlo sample. Therefore, the resistors always have different values, and the voltages are different. This type of setup is typically used to model local variations, which means variations that affect devices in a different way.
- `test3.sp` has two kinds of distributions defined: `globw/globwidth` as in the first example, and `locwidth` as in the second example. The sum of the two is used to define the width of the resistors. Therefore, the resistors will always have different widths: a common variation due to `globwidth` and a separate variation due to `locwidth`. In the example, the distribution for `locwidth` was chosen as narrower than for `globwidth`. When overlaying the measurement results, the large common variation can easily be seen; however, all voltages are different.

In summary, each reference to a parameter with a specified distribution causes a new random variable to be generated for each Monte Carlo sample. When referencing the parameter on an instance, the effect of a local variation is created. When referencing the parameter on an expression for a second parameter and using the second parameter on an instance, the effect of a global variation is created.

Variations Specified in the Context of Subcircuits

The concept explained in the previous section applies to subcircuits as instances, and instances within subcircuits. Here again the example of a physical resistor is discussed, with variation of its width.

- In `test6.sp`, the resistor width is assigned inside the subcircuit. The variations are chosen from the top level. Because each subcircuit is a separate entity, the parameter `w` is treated as a separate reference. Thus each resistor will have its own value, partly defined through the common value of `globwidth` and partly through the separate value of `locwidth`.
- `test7.sp` has two resistors in the subcircuit. Each device in each subcircuit has a separate reference to the variation. Therefore each device gets its own value.
- In `test8.sp`, the variation definition for `locwidth` has been moved from the top level into the subcircuit. Each resistor has a common global variation and its own local variation.
- `test9.sp` assigns the top level variation to a local parameter, which in turn is applied to the width definition of the resistor. Because this happens independently within each subcircuit, the values are the same for the resistor pair in each subcircuit, but different values for the different pairs. This technique can be applied to long resistors when a middle terminal is required for connecting capacitance to the substrate. The resulting two resistor pieces will have the same resistance, but it will be different from other resistor pairs.

In summary, each subcircuit has its own parameter space. Therefore, it is possible to put groups of identical components into a subcircuit. Furthermore, within each group all devices have the same parameter values, but between the groups, parameters are different. When specifying variations on these parameters, the effects of local variations between the groups are created.

Variations on a Model Parameter Using a Local Model in Subcircuit

If a model is specified within a subcircuit, then the specified parameter values apply only to the devices in the same subcircuit. Therefore, it is possible to calculate the value of a model parameter within the subcircuit; for example, as a function of geometry information.

When specifying variations on these parameters, the effects of local variations between subcircuits are created. If this method is used at the extreme with one device per subcircuit, then each device has its own model. This approach is not recommended because it leads to a substantial overhead in the simulator.

Indirect Variations on a Model Parameter

Variations on geometrical parameters are presented in [Variations Specified on Geometrical Instance Parameters](#) and [Variations Specified in the Context of Subcircuits](#). To specify variations on a model parameter, for example, the threshold of a MOS device, you can use the approach explained in the previous section with one model per device in a subcircuit. However, this is impractical because the netlist needs to be created to call each device as a subcircuit, and because of the overhead. Since variations are of interest only on a few model parameters, an indirect method of varying model parameters can be used. Some special instance parameters are available for this purpose. For example, for MOS devices, the parameter `delvt0` defines a shift in threshold.

Referencing a parameter with a distribution as value for `delvt0` creates the effect of local threshold variations. A significant number of parameters of this type are available in PrimeSim HSPICE for BSIM3 and BSIM4 models. The variations can be tailored for each device depending, for example, on its size. A disadvantage of this method is that the netlist needs to be parameterized properly to get the correct variations. The process of preparing a basic netlist for Monte Carlo simulations with this approach is tedious and error-prone; therefore, it is best handled with scripts.

For a listing of supported BSIM3 and BSIM4 instance parameters, see the *PrimeSim HSPICE Reference Manual: MOSFET Models*, Supported Instance Parameters, BSIM3, BSIM4, BSIM3SOI and BSIM4SOI.

Variations Specified on Model Parameters

This section discusses the method of specifying distributions on parameters and using these parameters to define values of model parameters. With this approach, the netlist does not have to be parameterized. The `modmonte` option can be used to distinguish between global variations (all devices of a particular model have the same parameter set) or local variations (every device has a unique random value for the specified parameters).

- `test10.sp` shows a simple case where the model parameter for sheet resistivity is assigned a distribution defined on the parameter `rsheet`. The results show that all resistors have the same value for each Monte Carlo sample, but a different one for different samples. This setup is useful for studying global variations.
- `test11.sp` has `.option modmonte=1` added. Now every resistor has a different value.

Note that `.option modmonte` has no effect on any other approach presented here.

In summary, assigning parameters with specified distributions to model parameters allows for investigating the effects of global or local variations, but not both. The possibility of selecting one or the other with a simple option is misleading in the sense that the underlying definitions for global and local variations are not the same for a realistic semiconductor technology.

Local Variations for Transistor Fingers

You can run an MC simulation where each of the transistors' fingers of the gate are simulated to have a different process variation. Set option `MODMONTE=1` and get local variations for an instance. View MOSFET parameter variations while printing the alias LX, LV, and so on for different MOSFET parameters. For example:

```
*MOS L53 DC sweep test for monte carlo analysis
*
* MODMONTE option can be set to 1 or 0,
* MODMONTE=1: measured ids will be different for the 3 element
* MODMONTE=0: measured ids will be identical for the 3 element
*.options OPTS LIST NOPAGE INGOLD=2 ALT999 PROBE POST=1 TNOM=25
.model nch nmos LEVEL=53 TOX=4E-9 wint=wint_nch VERSION=3.2
.param wint_nch = AGAUSS ( 3e-7 , 1e-7 , 3.0 )
m11 2 11 0 0 nch W=1E-6 L=0.15E-6
m12 2 11 0 0 nch W=1E-6 L=0.15E-6
m13 2 11 0 0 nch W=1E-6 L=0.15E-6
v01 2 0 1.5
v02 11 0 0.0
.dc v02 0 2.0 0.1 sweep monte=3
.meas dc ids_11 find par('i(m11)*1E3') when v(11)=1.5
.meas dc ids_12 find par('i(m12)*1E3') when v(11)=1.5
.meas dc ids_13 find par('i(m13)*1E3') when v(11)=1.5
.end
```

Variations Specified Using DEV and LOT

The two limitations of the approach described in [Variations Specified on Model Parameters](#) are resolved in this method by specifying global and local variations directly on a model parameter with the syntax:

```
parameterName=parameterValue LOT/distribution LotDist
+ DEV/distribution DevDist
```

where:

- LOT is the keyword for global distribution
- DEV is the keyword for local distribution
- distribution is as explained in [Variations Specified on Geometrical Instance Parameters](#)
- LotDist and DevDist are the characteristic numbers for the distribution: 3-sigma is the fixed value for Gaussian distributions.

test12.sp has large global and small local variation, similar to the setup in the file test3.sp. The result shows four different curves with a large common part and small separate parts. The amount of variation defined in the two files is the same. The curves

look different from the `test3.sp` results because different random sequences are used. However, the statistical results (sigma) converge for a large number of samples.

There is no option available to select only local or only global variations. This can be an obstacle if the file is read-only or encrypted.

Combinations of Variation Specifications

Specifying distributions on parameters and applying them to model parameters can be used on some models. The `DEV/LOT` approach can be used on others in the same simulation.

- `test13.sp` has `DEV/LOT` specified for model `res1`, and the parameter `width` for model `res2`. The values for the resistors with model `res1` are different, and the values for resistors with model `res2` are the same.
- `test14.sp` is similar to `test7.sp` and has `modmonte=1` specified. All four resistors have different values. However, note that in reality, the sigma for `width` would be different when simulating local or global variations.
- `test15.sp` has instance parameter variations specified on two resistors and `DEV/LOT` on two others. From the waveforms, `v3` and `v4` form a first pair, and `v1` and `v2` form a second pair.

It also is possible to mix variations on instance parameters and model parameters in the same setup.

- `test16.sp` has small instance parameter variations specified on `width` and relatively large model parameter variations on the sheet resistivity, `rsh`. The results show four different waveforms with a common behavior.
- `test17.sp` shows instance and model parameter variations as in the previous test case, but `.option modmonte` is set to 1. Thus, the model variations affect every device in a different way. The results show completely independent behavior of all four resistors.

If an instance parameter or instance parameter variations and model parameter variations are specified on the same parameter, then the instance parameter always overrides the model parameter. Because only few parameters can be used in both domains, this case is rather seldom; however, it must be considered to avoid unexpected results.

- `test18.sp` has model variation specified on `width` with a parameter. Two resistors have `width` also defined on instance. The resistors with instance parameter do not vary at all. The other two resistors vary independently as expected because `.option modmonte` is set to 1.

- `test19.sp` is similar to `test18.sp` with `.option modmonte` set to 0. The two resistors that do not have width defined on the instance line vary together.
- `test20.sp` has `DEV/LOT` specified. Instance parameters override variations on selected resistors.

Variation on Model Parameters as a Function of Device Geometry

For local variations (see [Chapter 27, Mismatch Analyses](#)), it is a common requirement to specify variation on a model parameter as a function of device geometry. For example, the MOS device threshold was observed to vary with the total device area.

The approach explained in [Indirect Variations on a Model Parameter](#) can be used. While this allows for specifying local variations on each device, it does not include the capability of using expressions based on element parameters. Thus, variation cannot be described with an expression that includes the device's geometry. Conceptually, a netlist processor could be written that inserts the appropriate values for the parameters as a function of device size. (Synopsys does not provide such a tool).

The `DEV/LOT` approach has no mechanism to describe variation as a function of an element parameter.

Non-Gaussian Probability Distribution Functions

In traditional Monte Carlo analysis, there are only five different distributions:

- Uniform distribution, absolute
- Uniform distribution, relative
- Gaussian distribution, absolute
- Gaussian distribution, relative
- Limit distribution

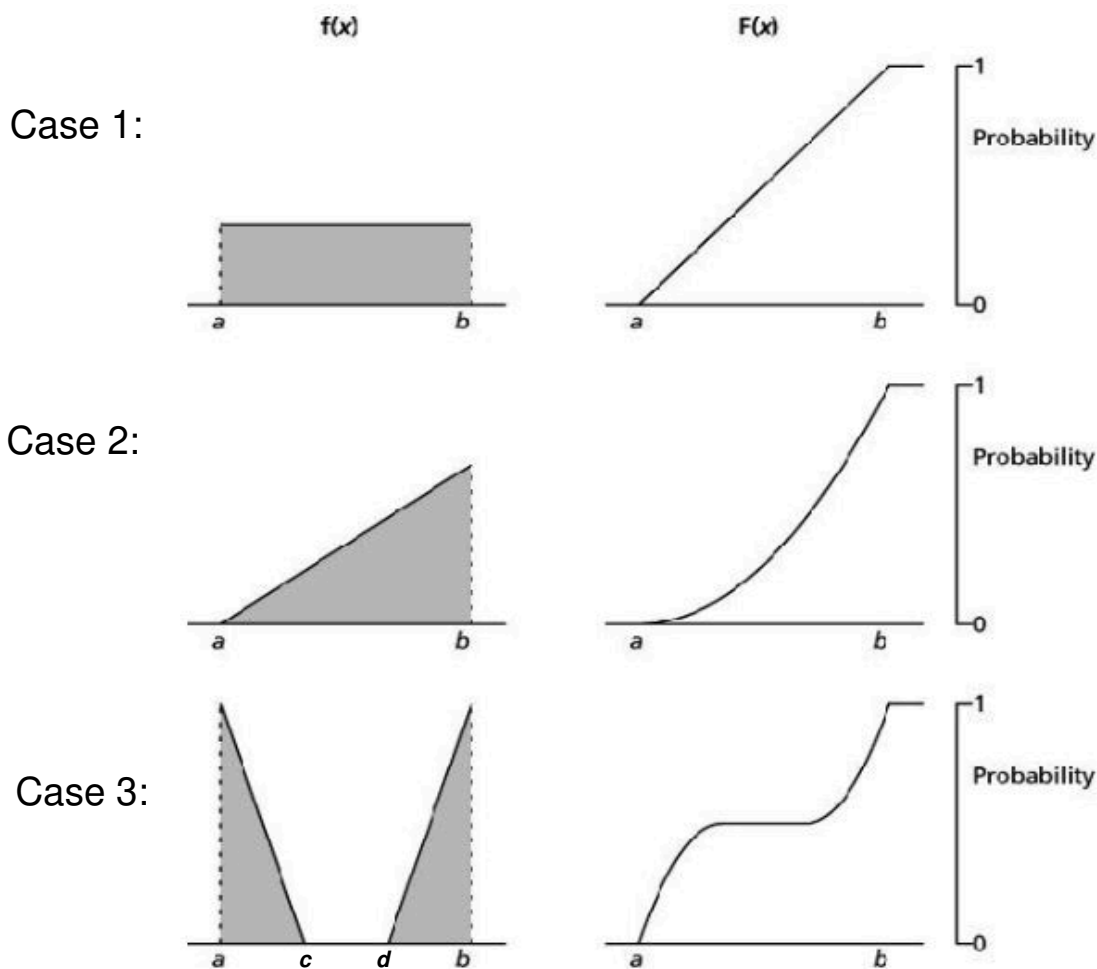
While no user-defined distributions are available, you can describe functions such as those shown in [Figure 114 on page 498](#) using the methodologies described in this section.

In PrimeSim HSPICE, samples from the distributions given in [Figure 114](#) can be created exactly for the Case 1 (top) function using the `CDF( )`—Cumulative Distribution Function construct in Variation Block. The other two cases can be approximated using a PWL function for $F(x)$, which are the illustrations on the right-hand side.

The distributions also can be sampled exactly in both Variation Block (VB) and in the traditional Monte Carlo format by using the “probability inverse” and defining the relationship in an expression. Let u be a sample from the uniform random number generator over the range $[0, 1]$. This is the default behavior with the traditional Monte Carlo

style. You would add 0.5 with the VB uniform generator, $U()$, because the variables are sampled in $-0.5, 0.5]$ to have zero mean.

Figure 114 Non-Gaussian functions



Map u to the y-axis on the right side of the figures in [Figure 114](#). Project the value onto the curve horizontally, and then down. This gives a value, for example x . Then x is a random sample from the desired distribution. Mathematically, it works as $x = F^{-1}(u)$. For instance:

Case 1:

$$x = a + (b-a) \cdot u$$

Case 2:

$$x = a + (b-a) \cdot \sqrt{u}$$

Case 3:

$$x = a + (b-a) \cdot \sqrt{u} \text{ if } u \leq 0.5$$

$$x = d + (b-d) \cdot \sqrt{u} \text{ if } u \geq 0.5$$

Here, c and d are the locations of the two additional coordinates between a and b that define the gap in the distribution.

Traditional and Variation Block Monte Carlo

This section discusses the following topics:

- [Simple Random Sampling and Other Sampling Methods](#)
- [Traditional SRS Versus VB SRS](#)
- [Invoking Advanced Sampling Methods](#)

Simple Random Sampling and Other Sampling Methods

PrimeSim HSPICE developed a new type of variation definition format called VB to address issues that the traditional style or the AGAUSS style was neither intuitive nor flexible to code. For more information, see [Chapter 25, Statistical Representation Using the Variation Block](#). The VB format enables different types of sampling methods, such as LHS, Sobol, Niederreiter, OFAT, Factorial, and External, along with Simple Random Sampling (SRS). The output of Monte Carlo analysis with VB-style definitions is more comprehensive than with the AGAUSS-style Monte Carlo. Later, it was decided to support the features associated with the VB-style within the AGAUSS style. As a result, the AGAUSS style is supported both in the traditional Monte Carlo flow and the VB flow. You must invoke the latter by entering the following: `option sampling_method=SRS`. The other sampling algorithms also are supported with the AGAUSS style.

PrimeSim HSPICE supports statistical modeling of variability parameters for Gaussian, Uniform, and Limit distribution. The keywords GAUSS, AGAUSS, UNIF, AUNIF, and LIMIT can be used in a SPICE netlist to represent these distributions. By using these keywords and following certain rules, you can code Global and Local variations, which together constitute parametric variations. This method is commonly known as AGAUSS-style variation definition. Until recently, PrimeSim HSPICE supported only the Simple Random Sampling (SRS) algorithm for Monte Carlo simulation with this type of variation definition.

Traditional SRS Versus VB SRS

Starting from the 2016.06 version, the option `sampling_method=SRS` is the default. This invokes the enhanced SRS flow. If you want to execute the traditional or the old SRS run,

you need to set the option `random_generator=3LC`. The differences between traditional SRS and enhanced SRS (also called the VB SRS as it was developed along with the Variation Block format) are explained in the following sections:

- [Random Number Generators](#)
- [Pseudo-Random Number Generators](#)
- [Data Mining](#)

Random Number Generators

The Random Number Generators (RNGs) used in the case of traditional SRS are different from those used for enhanced SRS. Therefore, a one-to-one match is not possible unless you set `.option RANDGEN=1` in the traditional SRS flow to match the RNGs.

In the traditional AGAUSS flow, there is no nominal simulation (that is, simulation without any variation on model parameters). The Monte Carlo flow with the `option sampling_method` executes nominal simulation for the first Monte index. Thus, when you compare the traditional SRS values for each sample with that of enhanced SRS, start the comparison from the first index of the traditional SRS to the second index of enhanced SRS, and so on.

The results may vary slightly because of normal limit, controlled by the option `Normal_Limit`, when using enhanced SRS. (See [Chapter 26, Monte Carlo Analysis—Flow and Outputs](#).) In some cases the order of value assignment to independent random variables may differ because of slight differences in traditional and enhanced front-end flows.

Pseudo-Random Number Generators

Pseudo-Random Number Generators (PRNGs) are used in two kinds of Monte Carlo analyses in PrimeSim HSPICE: Traditional AGAUSS-style Monte Carlo and VB-based Monte Carlo.

Traditional AGAUSS-style Monte Carlo does not support advanced sampling methods. You set the PRNG using the PrimeSim HSPICE global option `RANDGEN`.

The usage is:

```
.option RANDGEN=[3LC|MOA|UVS|MCG|WH]
```

Here, the default is `MCG`.

Note:

From the 2016.06 release, the default random number generator is changed to `MCG` from `3LC`. `3LC` was found to be unsuitable for use with large samples. For backward compatibility, you need to set `.option RANDGEN=3LC`.

All PRNGs can accept a seed, which is used to initialize the generator state. Using the PrimeSim HSPICE global option `seed`, you can set the value of the seed between 1 and 259200. In addition, you can set `.option SEED=random` to set a random number of seed between 1 and 259200. When the seed is set to `random`, PrimeSim HSPICE uses a different seed value when rerunning the same Monte Carlo case.

In the VB-based Monte Carlo or enhanced AGAUSS-style Monte Carlo, you can use a global option or a VB-based option to designate a sampling method.

Set the PRNG:

```
.variation  
Option Random_Generator=[MOA|MSG|UVS|MCG|WH]  
.end_variation
```

Or,

```
.option Random_Generator=[MOA|MSG|UVS|MCG|WH]
```

Here, the default is `MCG`.

Note:

From the 2016.06 release, the default random number generator is changed to `MCG` from `MOA`. `MOA` was found to be unsuitable for use with large samples. For backward compatibility, you need to set `.option Random_Generator=MOA`.

Data Mining

The traditional SRS flow generates a limited statistical summary (mean, median, sigma, and so on) in the output listing file.

The enhanced SRS flow generates the following:

- Enhanced statistical summary and variable screening report in the `*.mpp0` file.
- Independent random variable values for each sample in the `*.mc0` file.
- Element variation contribution information to back-annotate in Custom Compiler, in the `*.annotate` file.
- Partial `*.mpp0` file for traditional SRS (without variable screening report).

Invoking Advanced Sampling Methods

To invoke sampling methods such as Latin Hypercube, Factorial, OFAT, or low-discrepancy sequences in a Monte Carlo netlist, enter:

```
.OPTION SAMPLING_METHOD=name_of_method
```

The syntax is as follows:

```
.OPTION SAMPLING_METHOD=
SRS|LHS|Factorial|OFAT|Sobol|Niederreiter|External
```

The methods and their brief descriptions are listed below. For detailed discussion of these methods, see [Sampling Options](#) and [Comparison of Sampling Methods](#) in [Chapter 26, Monte Carlo Analysis—Flow and Outputs](#).

Method	Brief Description
SRS	(Default) Simple random sampling performed in traditional PrimeSim HSPICE Monte Carlo method. Note: Starting from the 2016.06 version, the option <code>sampling_method=SRS</code> is the default. To invoke the traditional or the old SRS flow, you need to set the option <code>random_generator=3LC</code>.
LHS	Latin Hypercube sampling; efficient for large number of variable parameters (used with <code>.OPTION REPLICATES</code>).
Factorial	Factorial sampling: - Evaluates the circuit response at the extremes of variable ranges to get an idea of the worst- and best-case behavior. - Creates polynomial response surface approximations.
OFAT	One-Factor-at-a-Time sampling; is useful for sensitivity studies and for constructing low-order response surface approximations.
Sobol	Sobol sampling uses low-discrepancy sequences (LDS). LDS sample points are more uniformly distributed compared to LHS and the sampling error is lower. Sobol is used with a sampling dimension of 21200 or less. If you use Sobol sampling when the number of independent random variables (dimension) is larger than 21200, then PrimeSim HSPICE uses Sobol sampling for the first 21200 variables and simple random sampling for the remaining dimensions.
Niederreiter	LDS sampling sequence is useful as a method for cases of a sampling dimension up to 318. If that number is exceeded, PrimeSim HSPICE switches to the Sobol sampling method.
External	Executes a data set of externally created perturbations. External sampling allows design and process exploration tools to run statistical experiments with the variables for each sample under their full control.

Use `.OPTION REPLICATES` after selecting `.OPTION SAMPLING_METHOD=LHS`. The `REPLICATES` option runs replicates of the Latin Hypercube samples. For more information, see [Latin Hypercube Sampling \(LHS\) on page 568](#).

Monte Carlo Examples

This section discusses the following examples:

- [Delay and Power Consumption Analysis of Inverters](#)
- [Gaussian, Uniform, and Limit Functions](#)
- [Major and Minor Distribution](#)
- [RC Time Constant](#)
- [Switched-Capacitor Filter Design](#)

Delay and Power Consumption Analysis of Inverters

The following example measures the delay and the power consumption of two inverters. Additional inverters buffer the input and load the output.

- [Netlist](#)
- [Transient Sigma Sweep Results](#)
- [Operating-Point Results in Transient Analysis](#)

Netlist

This netlist contains commands for two sets of transient analysis: parameter sweep from -3 to +3-sigma, and a Monte Carlo analysis. This netlist creates one set of output files (.mt0 and .tr0) for the sigma sweep, and one set (mt1 and tr1) for Monte Carlo.

```
$ inv.sp sweep mosfet -3 sigma to +3 sigma, use measure output
.param vref=2.5 sigma=0
.global 1
vcc 1 0 5.0
vin in 0 pwl 0,0 0.2n,5
x1 in 2 inv
x2 2 3 inv
x3 3 out inv
x4 out 4 inv
.macro inv in out
    mn out in 0 0 nch w=10u l=1u
    mp out in 1 1 pch w=10u l=1u
.eom
.param mult1=1
+ polycd=AGAUSS(0,0.06u,1)    x1='polycd-sigma*0.06u'
+ nactcd=AGAUSS(0,0.3u,1)    xwn='nactcd+sigma*0.3u'
+ pactcd=AGAUSS(0,0.3u,1)    xwp='pactcd+sigma*0.3u'
+ toxcd=AGAUSS(200,10,1)    tox='toxcd-sigma*10'
+ vtoncd=AGAUSS(0,0.05v,1)    delvton='vtoncd-sigma*0.05'
```

```
+ vtopcd=AGAUSS(0,0.05v,1) delvtop='vtoncd+sigma*0.05'
+ rshncd=AGAUSS(50,8,1) rshn='rshncd-sigma*8'
+ rshpcd=AGAUSS(150,20,1) rshp='rshpcd-sigma*20'
* level=28 example model
.model nch nmos
+ level=28 lmlt=mult1 wmlt=mult1 wref=22u lref=4.4u
+ xl=xl xw=xwn tox=tox delvto=delvton rsh=rshn
...
.model pch pmos
+ level=28 lmlt=mult1 wmlt=mult1 wref=22u lref=4.4u
+ xl=xl xw=xwp tox=tox delvto=delvtop rsh=rshp
+ ld=0.08u wd=0.2u acm=2 ldif=0 hdif=2.5u
+ rs=0 rd=0 rdc=0 rsc=0 rsh=rshp js=3e-04 jsw=9e-10
...
* transient with sweep
.tran 20p 1.0n sweep sigma -3 3 .5
.meas s_delay trig v(2) val=vref fall=1
+ targ v(out) val=vref fall=1
.meas s_power rms power
* transient with Monte Carlo
.tran 20p 1.0n sweep monte=100
.meas m_delay trig v(2) val=vref fall=1
+ targ v(out) val=vref fall=1
.meas m_power rms power
.probe tran v(in) v(1) v(2) v(3) v(4)
.end
```

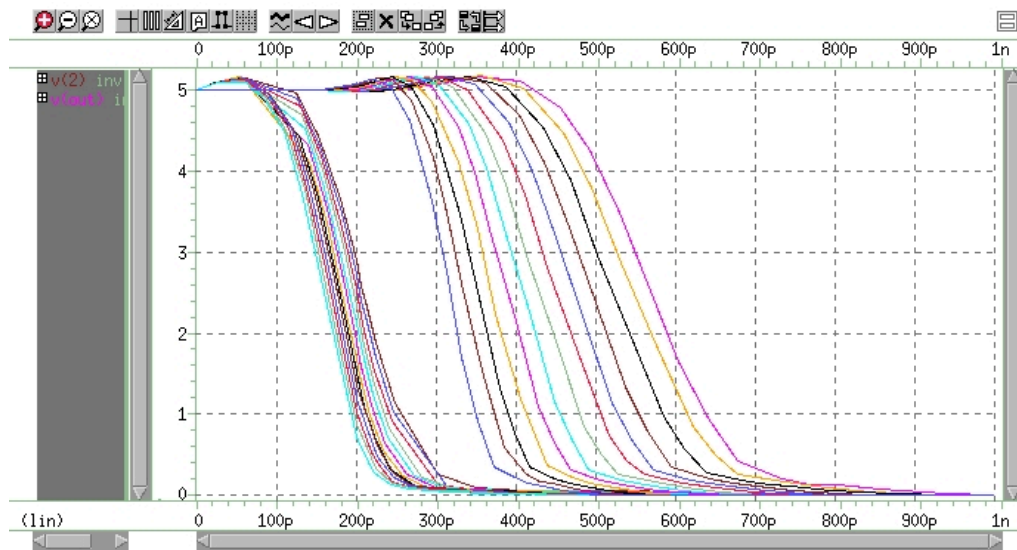
Transient Sigma Sweep Results

The plot in [Figure 115](#) shows the family of transient analysis curves for the transient sweep of the sigma parameter from -3 to +3 from the file inv.tr0. In the sweep, PrimeSim HSPICE uses the values of sigma to update the skew parameters, which in turn modify the actual NMOS and PMOS models.

Operating-Point Results in Transient Analysis

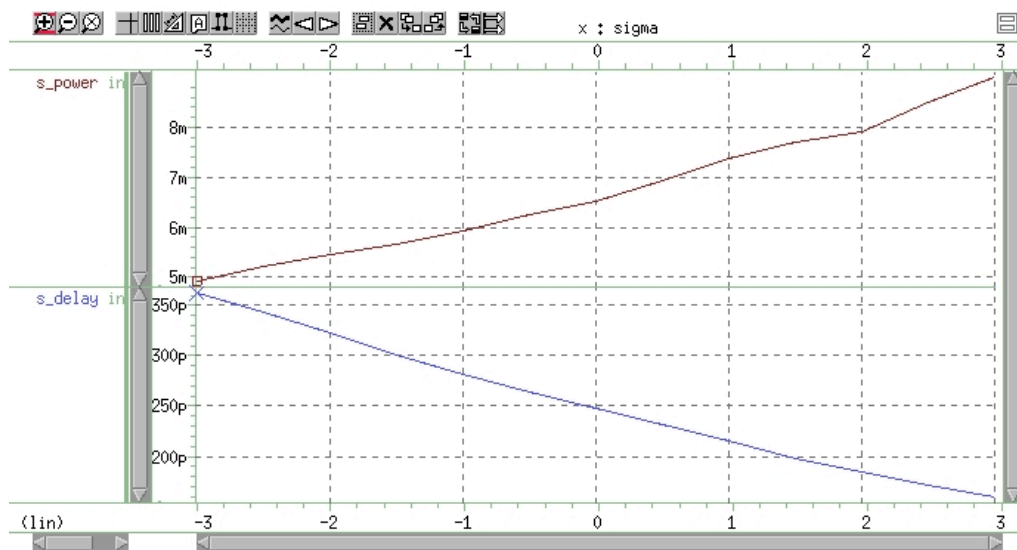
If you want to get OP results after every Monte Carlo simulation in transient analysis, you can add the option `opfile` to the netlist. OP results output to the file `*.dp0`.

Figure 115 Sweep of Skew Parameters from -3 Sigma to $+3$ Sigma



To view the measured results, plot the `inv.mt0` output file. The plot in [Figure 116 on page 505](#) shows the measured pair delay and the total dissipative power as a function of the parameter sigma. To get the specific operating point information of each Monte Carlo run, use `opfile=1`.

Figure 116 Sweep MOS Inverter, Pair Delay, and Power: -3 Sigma to 3 Sigma



Gaussian, Uniform, and Limit Functions

You can find the sample netlist for this example in the following directory:

`$installdir/demo/hspice/apps/mont1.sp.`

Figure 117 Uniform Functions

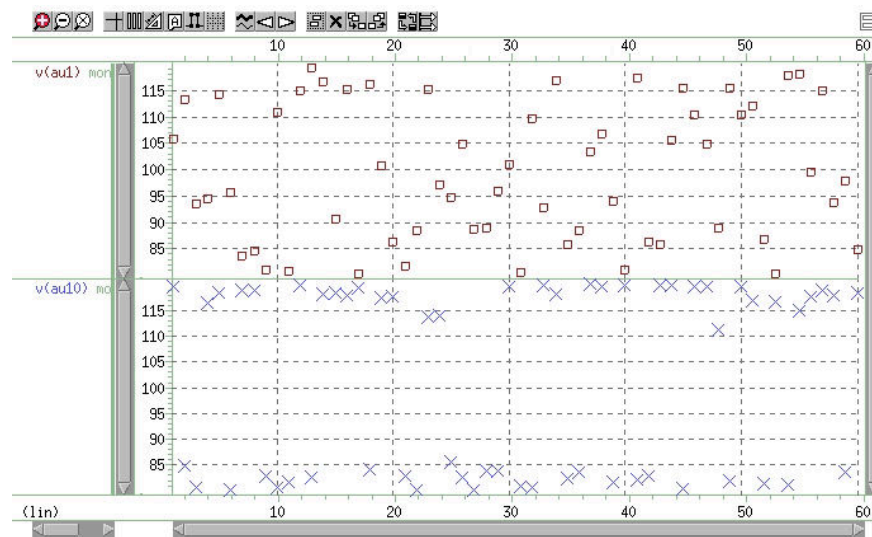


Figure 118 Gaussian Functions

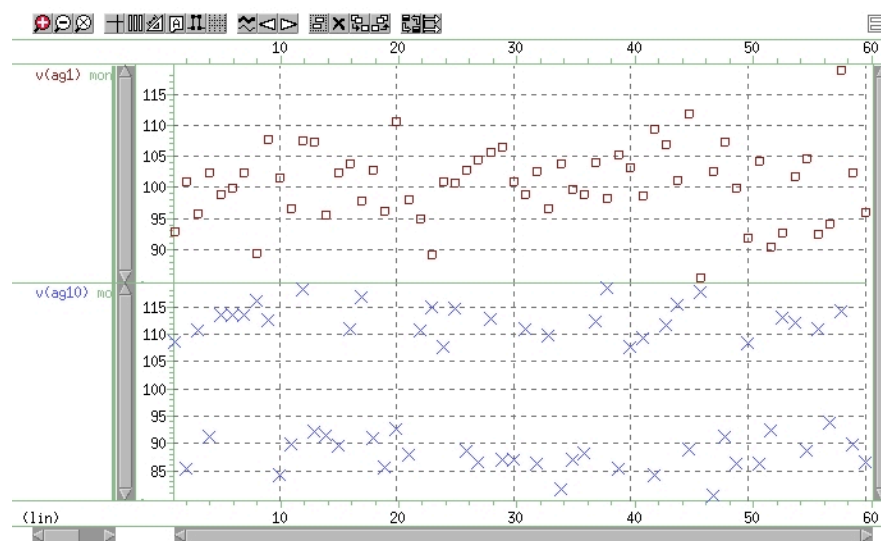


Figure 119 Limit Functions

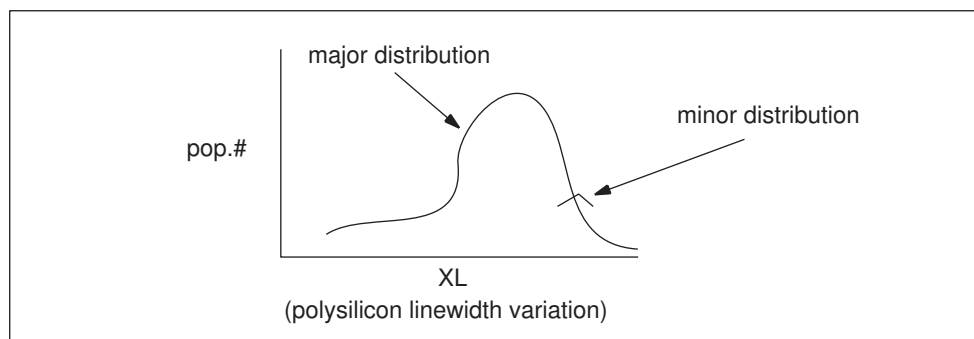


Major and Minor Distribution

In MOS IC processes, manufacturing tolerance parameters have both a major and a minor statistical distribution.

- The major distribution is the wafer-to-wafer and run-to-run variation that determines electrical yield.
- The minor distribution is the transistor-to-transistor process variation that is responsible for critical second-order effects, such as amplifier offset voltage and flip-flop preference.

Figure 120 Major and Minor Distribution of Manufacturing Variations



The following example is a Monte Carlo analysis of a DC sweep in PrimeSim HSPICE. Monte Carlo sweeps the VDD supply voltage from 4.5 to 5.5 volts.

You can find the sample netlist for this example in the following directory:

```
$installdir/demo/hspice/apps/mondc_a.sp
```

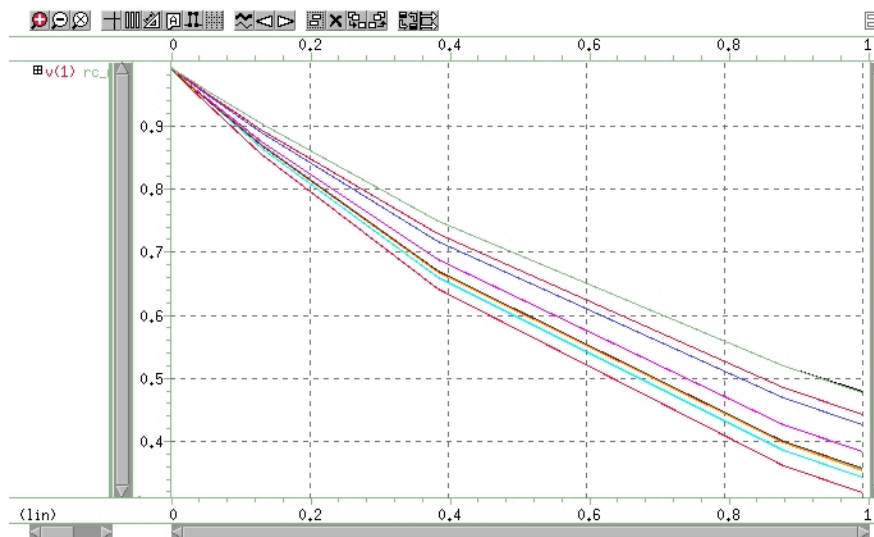
- The M1 through M4 transistors form two inverters.
- The nominal value of the `LENGTH` parameter sets the channel lengths for the MOSFETs, which are set to 1μ in this example.
- All transistors are on the same integrated circuit die. The `LEFF` parameter specifies the distribution—for example, a ± 5 -percent distribution in channel length variation at the ± 3 -sigma level.
- Each MOSFET has an independent random Gaussian value.

The `PHOTO` parameter controls the difference between the physical gate length and the drawn gate length. Because both n-channel and p-channel transistors use the same layer for the gates, Monte Carlo analysis sets `XPHOTO` distribution to the `PHOTO` local parameter. `XPHOTO` controls photolithography for both NMOS and PMOS devices, consistent with manufacturing physics.

RC Time Constant

This simple example shows uniform distribution for resistance and capacitance. It also shows the resulting transient waveforms for ten different random values.

Figure 121 Monte Carlo Analysis of RC Time Constant



You can find the sample netlist for this example in the following directory:

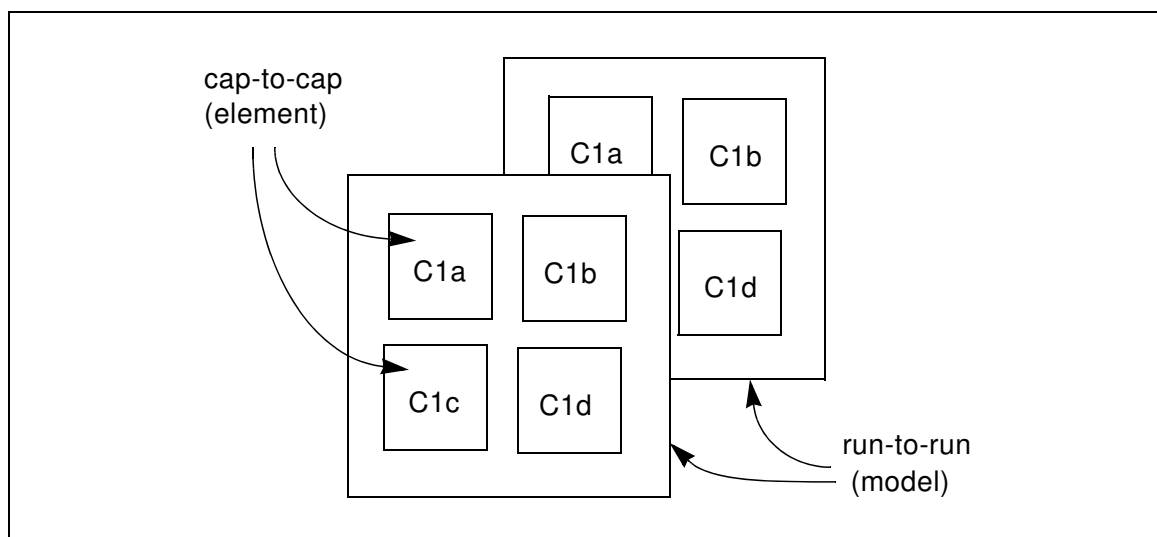
```
$installdir/demo/hspice/apps/rc_monte.sp
```


Switched-Capacitor Filter Design

Capacitors used in switched-capacitor filters consist of parallel connections of a basic cell. Use Monte Carlo techniques in PrimeSim HSPICE to estimate the variation in total capacitance. The capacitance calculation uses two distributions:

- Minor (element) distribution of cell capacitance from cell-to-cell on a single die.
- Major (model) distribution of the capacitance from wafer-to-wafer or from manufacturing run-to-run.

Figure 122 Monte Carlo Distribution



You can approach this problem from physical or electrical levels.

- The physical level relies on physical distributions, such as oxide thickness and polysilicon line width control. See [Physical Approach](#).
- The electrical level relies on actual capacitor measurements. See [Electrical Approach](#).

Physical Approach

Use the following steps for the physical approach:

1. Use a local variation in polysilicon to control the variation in capacitance for adjacent cells because oxide thickness control is excellent for small areas on a single wafer.
2. Define a local polysilicon line-width variation and a global (model-level) poly line-width variation. In this example:
 - The local polysilicon line width control for a line 10-m wide, manufactured with process A, is ± 0.02 m for a 1-sigma distribution.
 - The global (model-level) polysilicon line-width control is much wider; use 0.1 m for this example.
 - The global oxide thickness is 200 angstroms with a ± 5 -angstrom variation at 1 sigma.
 - The cap element is square with local poly variation in both directions.
 - The cap model has two distributions:
 - Poly line-width distribution
 - Oxide thickness distribution
 - The model poly distribution is half the physical per-side values.

The effective length is:

```

Leff=Ldrawn - 2
C1a 1 0 CMOD W=ELPOLY L=ELPOLY
C1b 1 0 CMOD W=ELPOLY L=ELPOLY
C1c 1 0 CMOD W=ELPOLY L=ELPOLY
C1d 1 0 CMOD W=ELPOLY L=ELPOLY
$ 10U POLYWIDTH,0.05U=1SIGMA
$ CAP MODEL USES 2*MODPOLY .05u= 1 sigma
$ 5angstrom oxide thickness AT 1SIGMA
.PARAM ELPOLY=AGAUSS(10U,0.02U,1)
+ MODPOLY=AGAUSS(0,.05U,1)
+ POLYCAP=AGAUSS(200e-10,5e-10,1)
.MODEL CMOD C THICK=POLYCAP DEL=MODPOLY
  
```

Electrical Approach

The electrical approach assumes no physical interpretation, but requires a local (element) distribution and a global (model) distribution. In this example:

- You can match the capacitors to ± 1 percent for the 2-sigma population.
- The process can maintain a ± 10 -percent variation from run to run for a 2-sigma distribution.

```
C1a 1 0 CMOD SCALE=ELCAP
C1b 1 0 CMOD SCALE=ELCAP
C1c 1 0 CMOD SCALE=ELCAP
C1d 1 0 CMOD SCALE=ELCAP
.PARAM ELCAP=Gauss(1,.01,2) $ 1% at 2 sigma
+ MODCAP=Gauss(.25p,.1,2) $10% at 2 sigma
.MODEL CMOD C CAP=MODCAP
```

Monte Carlo Results

This section describes the output of the Monte Carlo analysis in PrimeSim HSPICE. The plot in [Figure 123 on page 512](#) shows that the relationship between `TOX` against `XL` (polysilicon width=transistor length) is completely random, as set up in the input file.

To generate this plot, for example:

1. Read in the file `inv.mt1`.
2. Open the **Calculator**, select **TOX** (left mouse button), transfer to calculator (middle mouse button), and then select and transfer `XL`.
3. On the **WAVE** pulldown in the calculator, select `f(x)`, and then click the plot icon.
4. Using the right mouse button on the plotted waveform, select **Attributes** to change from the line plot to symbols.

Figure 123 Scatter Plot of XL vs. TOX

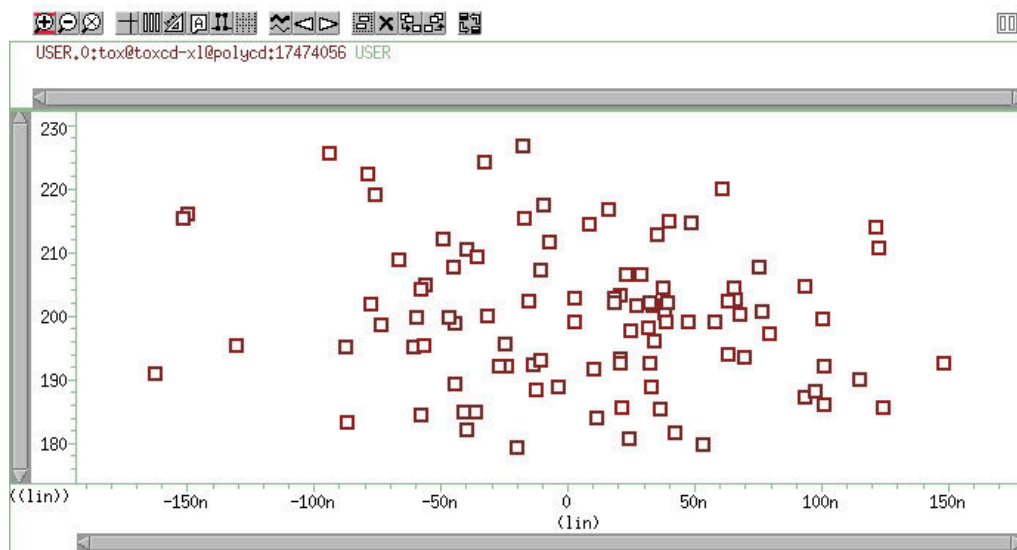
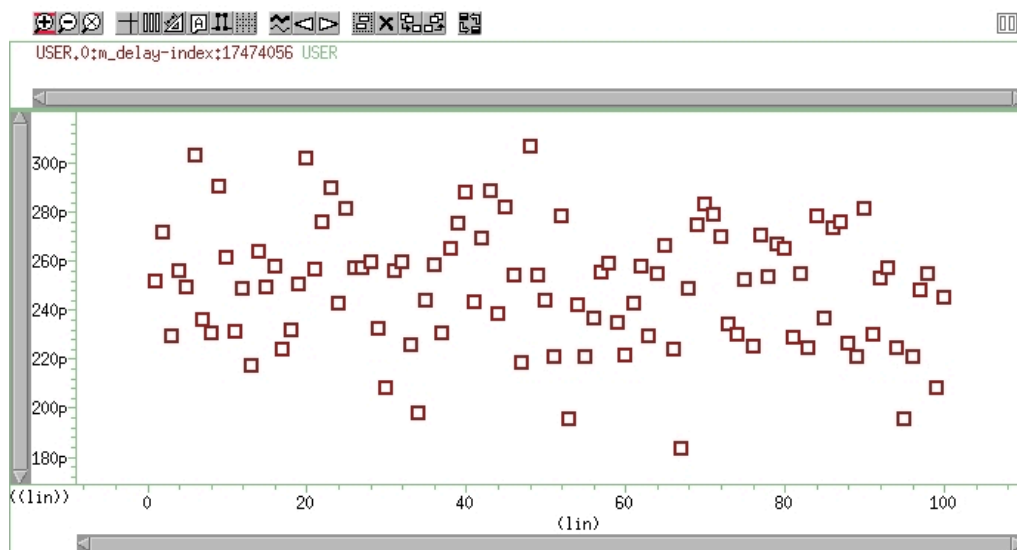


Figure 124 is a standard scatter plot showing the measured delay for the inverter pair against the Monte Carlo index number.

Figure 124 Scatter Plot of Inverter Pair Delay



If a particular result looks interesting; for example, if the simulation 68 (monte carlo index=68) produces the smallest delay, then you can obtain the Monte Carlo parameters for that simulation.

```
*** monte carlo index = 68 ***
MONTE CARLO PARAMETER DEFINITIONS
```

```

polycd  xl              = -1.6245E-07
nactcd  xwn             =  3.4997E-08
pactcd  xwp             =  3.6255E-08
toxcd   tox             =  191.0
vtoncd  delvton         = -2.2821E-02
         delvtop         =  4.1776E-02

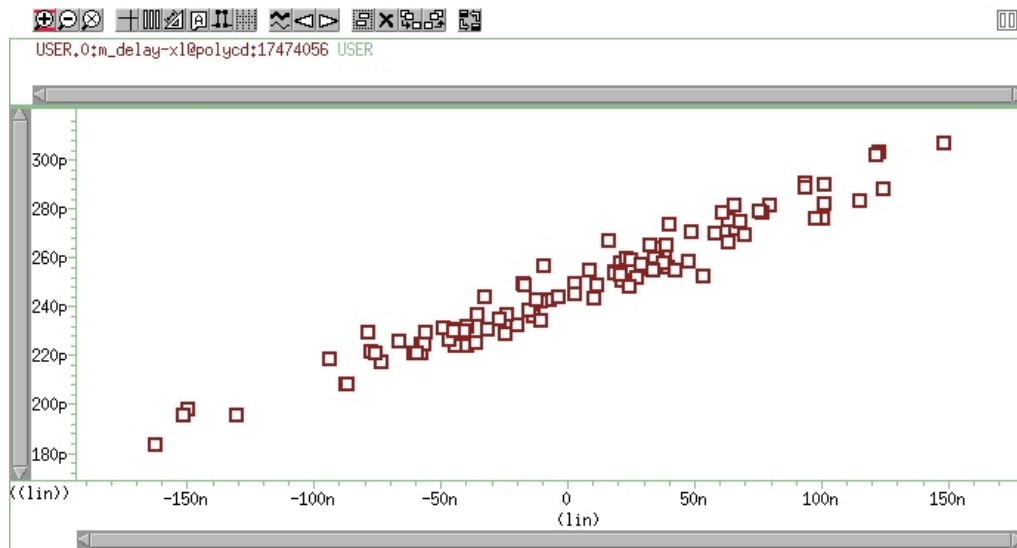
vtopcd
rshncd  rshn            =  45.16
rshpcd  rshp            =  166.2
m_delay= 1.7929E-10  targ= 3.4539E-10  trig= 1.6610E-10
m_power= 6.6384E-03  from= 0.0000E+00  to= 1.0000E-09

```

In the preceding listing, the `m_delay` value of $1.79\text{e-}10$ seconds is the fastest pair delay. You also can examine the Monte Carlo parameters that produced this result.

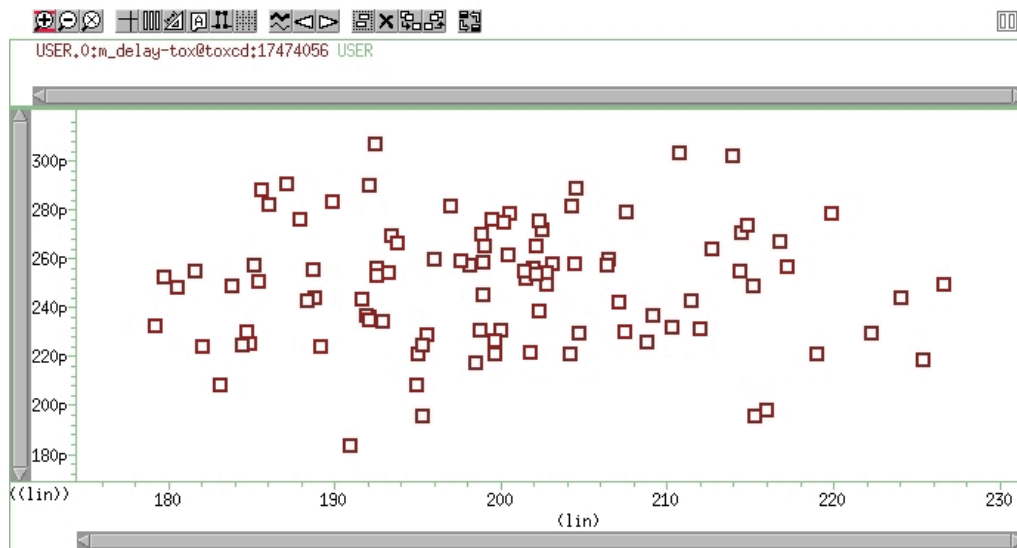
The information on shortest delay and so forth is also available from the statistics section at the end of the output listing. While this information is useful to determine whether the circuit meets specification, it is often desirable to understand the relationship of the parameters to circuit performance. Plotting the results against the Monte Carlo index number does not help this purpose. You need to generate plots that display a Monte Carlo result as a function of a parameter. For example, [Figure 125 on page 513](#) shows the inverter pair delay to channel as a function of poly width, which relates directly to device length.

Figure 125 Delay as a function of Poly width (XL)



[Figure 126](#) shows the pair delay against the `TOX` parameter. The scatter plot shows no obvious dependence, which means that the effect of `TOX` is much smaller than `XL`. To explore this in more detail, set the `XL` skew parameter to a constant and run a simulation.

Figure 126 Sensitivity of Delay with TOX



The plot in [Figure 127](#) overlays the skew result with the ones from Monte Carlo. The skew simulation traverses the design space with all parameters changing in parallel and then produces a relationship between power and delay, which appears as a single line. Monte Carlo exercises a variety of independent parameter combinations and shows that there is no simple relationship between the two results. Because the distributions were defined as Gaussian in the netlist, parameter values close to the nominal are more often exercised than the ones far away. With the relatively small number of samples, the chance of hitting a combination at the extremes is very small. In other words, designing for 3-sigma extreme for every parameter is probably not a good solution economically.

Figure 127 Superimposing Sigma Sweep Over Monte Carlo

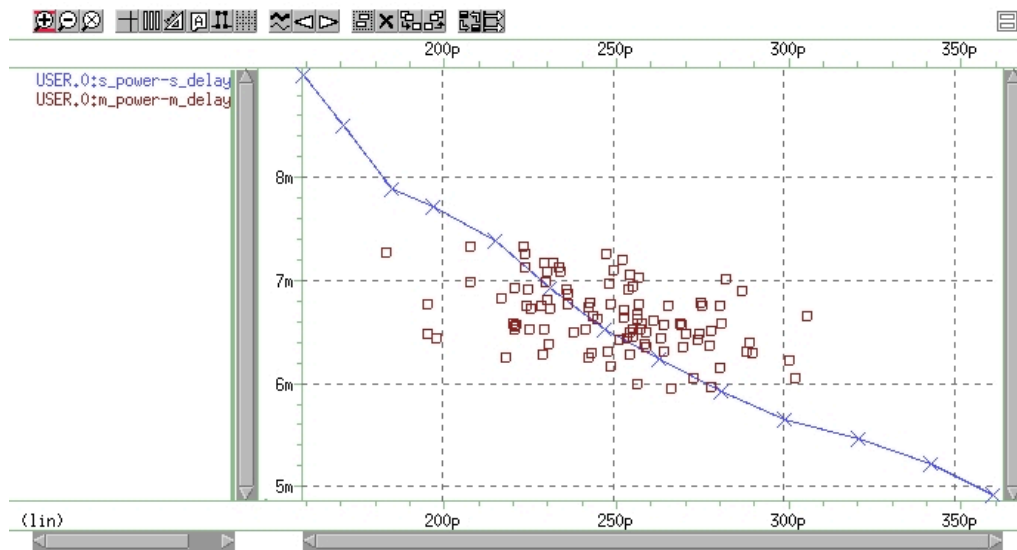
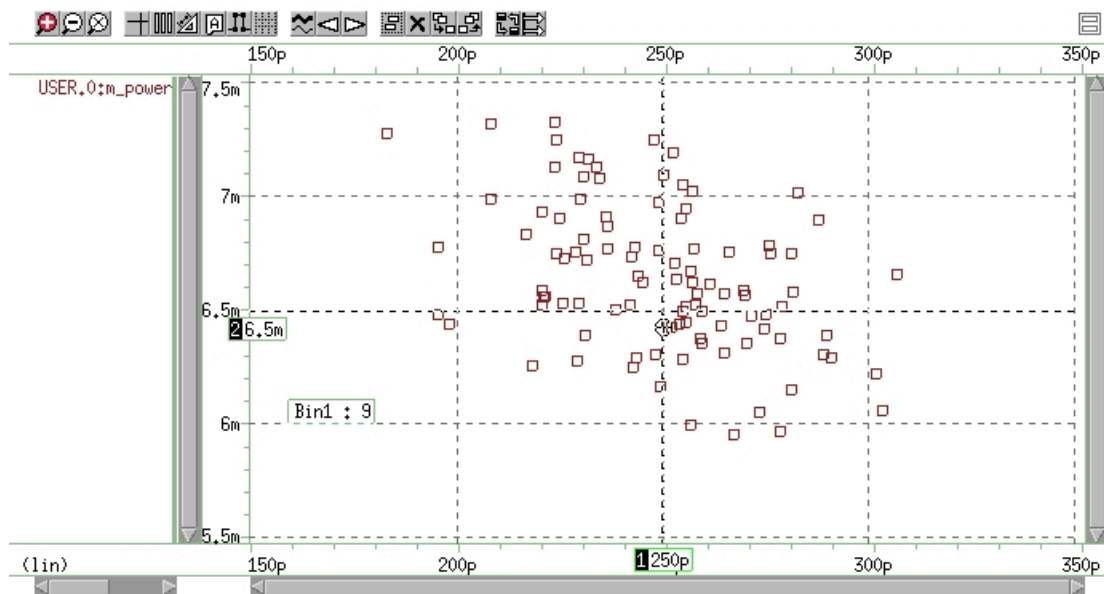


Figure 128 on page 515 superimposes the required part grades for product sales onto the Monte Carlo plot. This example uses a 250-ps delay and 6.5-mW power dissipation to determine the four binning grades.

Figure 128 Speed/Power Yield Estimation



Sorting the results from `inv.mtl` yields:

- Bin1 – 9 percent
- Bin2 – 38 percent
- Bin3 – 29 percent
- Bin4 – 24 percent

If this circuit is representative of the entire chip, then the present yield should be 9 percent for the premium Bin 1 parts, assuming variations in process parameters as specified in the netlist. This example only shows the principle on how to analyze the Monte Carlo results.

Troubleshooting Monte Carlo Issues

Perturbation Information Missing from Output Listing in Monte Carlo and Subcircuit Local Variables

A limitation in traditional Monte Carlo is that no perturbation information is printed in the output listing file in the case where a subcircuit has local variation parameters defined.

This limitation in the traditional Monte Carlo can be seen in cases similar to the following:

```
.param
+ my_global_lt_x=AGAUSS(0,1.1n,1)
+ temp_global_lt_x=my_global_lt_x
+ subc_global_lt_n=temp_global_lt_x
+ subc_global_lt_p=temp_global_lt_x

+ local_lt_n=AGAUSS(0,0.88n,1)
+ local_lt_p=AGAUSS(0,0.88n,1)
+ local_wt_n=AGAUSS(0,5.3n,1)
+ local_wt_p=AGAUSS(0,5.3n,1)
x1 in 2 inv
.subckt inv in out subc_global_lt_n=0
subc_global_lt_p=0 local_lt_n=0
+ local_lt_p=0 local_wt_n=0 local_wt_p=0
mn out in 0 0 nch W='3e-07+subc_global_lt_n+local_wt_n'
+
L='4e-08+subc_global_lt_n+local_lt_n'
mp out in 1 1 pch W='3e-07+subc_global_lt_p+local_wt_p'
+
L='4e-08+subc_global_lt_p+local_lt_p'
.ends
-----
*.lis file output
my_global_lt_x
temp_global_lt_x= -7.1777E-10

local_lt_n
```



```

1:mn          = 3.9274E-08  2:mn          =
3.9461E-08
3:mn          = 3.8590E-08  4:mn          = 3.9071E-08
-----

```

In the above case, PrimeSim HSPICE prints out variation in global variation parameter `temp_global_lt_x` directly. However, the local variation parameter `local_lt_n` is actually the expression value: `L='4e-08+subc_global_lt_n+local_lt_n'`. Therefore, you have to find the `local_lt_n` value from `[3.9574E-08 - (4e-8 + subc_global_lt_n)]`.

This can be overcome by a small modification in the subckt definition by reassigning the local parameters inside the subcircuit as shown next:

```

.subckt inv in out subc_global_lt_n=0 subc_global_lt_p=0
local_lt_n=0
+ local_lt_p=0 local_wt_n=0 local_wt_p=0
*Assign the local parameters inside the subckt again
.param subc_local_lt_n=local_lt_n
subc_local_lt_p=local_lt_p
+ subc_local_wt_n=local_wt_n
+ subc_local_wt_p=local_wt_p
mn out in 0 0 nch W='3e-07+subc_global_lt_n+subc_local_wt_n'
+
L='4e-08+subc_global_lt_n+subc_local_lt_n'
mp out in 1 1 pch W='3e-07+subc_global_lt_p+subc_local_wt_p'
+
L='4e-08+subc_global_lt_p+subc_local_lt_p'
.ends

```

This directly gives the `subc_local_lt_n` value as follows:

```

my_global_lt_x      temp_global_lt_x= 1.7577E-10
local_lt_n
1:subc_local_lt_n= 9.2081E-10  2:subc_local_lt_n= 1.4300E-10
3:subc_local_lt_n= -2.6043E-10  4:subc_local_lt_n= -2.6994E-09

```

25

Statistical Representation Using the Variation Block

Introduces variability, describes how it can be defined in PrimeSim HSPICE, and introduces the Variation Block.

PrimeSim HSPICE ships numerous examples for your use. See Variability Examples in the *PrimeSim HSPICE User Guide: Demonstration Netlists* for path to the demo files discussed in this chapter.

This chapter discusses the following topics:

- [Overview: Monte Carlo Using the Variation Block Flow](#)
- [Variability in PrimeSim HSPICE](#)
- [Overview of the Variation Block](#)
- [Variation Block Structure](#)
- [Variation Block Examples](#)
- [Group Operator {...} and Subexpressions](#)
- [Control Options and Syntax](#)
- [References](#)

Overview: Monte Carlo Using the Variation Block Flow

Monte Carlo analysis is the generic tool for simulating the effects of variations in device characteristics on circuit performance. PrimeSim HSPICE expresses the variations in device characteristics as distributions on the underlying model parameters. For each sample of the Monte Carlo analysis, PrimeSim HSPICE assigns random values to these parameters and executes a complete simulation, to produce one or more measurement results. The series of results from a particular measurement represents a distribution in statistical terms; for example, mean value and standard deviation (σ). With an increasing number of samples, the shape of the distribution gets a better definition with the effect that the two quantities converge to their final values.

You can analyze the results by arranging them in bins. Each bin represents how many results fall into a certain range (slice) of the overall distribution. A plot of these bins is a histogram that shows the shape of the distribution as the number of results versus slice. As the number of samples increases, the shape of the histogram gets smoother.

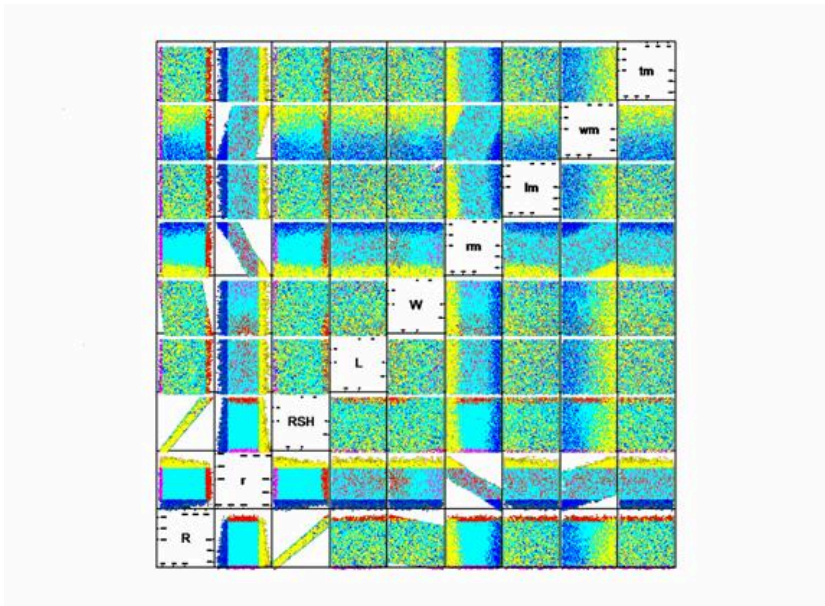
The ultimate interest of Monte Carlo simulation is to find out how the distribution in circuit response relates to the specification. Such a simulation considers these aspects of yield here:

- What is the percentage of devices which meet the specification?
- Is the design centered with respect to the specification?

The aspect of over-design is closely related. This is when the circuit characteristics are within specification with a wide margin, which could be at the expense of area or power and ultimately cost.

A typical design process is iterative, first for finding a solution which meets the nominal specification, and then moving on to a solution that meets yield and economic constraints, including the effects of variations in device characteristics. In this optimization process, it helps to understand the relationship of the design parameters to the circuit response, and the relationships of the different types of circuit response. This information is available after running Monte Carlo analysis and you can best present this data by Pairs Plots. This is a matrix of two-dimensional plots for investigating pair-wise relationships and exploring the data interactively. PrimeSim HSPICE does not produce such plots, but makes the necessary data available from Monte Carlo simulation. [Figure 129](#) shows an example of a Pairs Plot from a simple resistive divider.

Figure 129 Pairs Plot example



An application note, “Pairs Plots from PrimeSim HSPICE Monte Carlo,” describes the basic ideas and includes a MATLAB script to create such a plot. Contact the Synopsys Support Center for a copy of the application note.

Monte Carlo analysis is computationally expensive, so other types of analyses may produce certain results more efficiently. For cases where only the effects of variations on the DC or AC response of a circuit is of interest, you can use DCMatch/ACMatch analyses (see [Chapter 27, Mismatch Analyses](#)).

Variability in PrimeSim HSPICE

Among the three approaches that are available for defining variability in PrimeSim HSPICE, the Variation Block approach replaces the older methods of defining variations on parameters and models because it best fulfills the requirements for simulating nanometer technology devices.

The advantages of the Variation Block over previous solutions are:

- The Variation Block consolidates variation definitions in single records.
- A clear distinction exists between Global, Local, and Spatial Variations.
- A subset of variation types can be selected in a dependent simulation.
- The syntax allows for defining Local and Global Variation as a function of device geometry, and Spatial Variation as a function of device location.

- Monte Carlo results derived from the Variation Block are consistent with those from DCMatch or ACMatch analyses.
- Additional files, suitable for data mining, are generated.

Figure 130 Variation Block Example

```
.Variation
  .Global_Variation
    NMOS SNPS20N Vth0=0.07  U0=10 %
    PMOS SNPS20P Vth0=0.08  U0= 8 %
  .End_Global_Variation
  .Local_Variation
    NMOS SNPS20N
+   Vth0='2.5e-8/sqrt(get_E(W)*get_E(L)*get_E(M))'
+   U0  ='6.1e-5/sqrt(get_E(W)*get_E(L)*get_E(M))' %
    PMOS SNPS20P
+   Vth0='2.5e-8/sqrt(get_E(W)*get_E(L)*get_E(M))'
+   U0  ='6.1e-5/sqrt(get_E(W)*get_E(L)*get_E(M))' %
  .Element_Variation
    R r=10 %
  .End_Element_Variation
  .End_Local_Variation
.End_Variation
```

$\sigma_g(\Delta V_{th0}) = 70 \text{ mV}$
 $\sigma_g\left(\frac{\Delta U_0}{U_0}\right) = 0.1$
 $\sigma_l(\Delta V_{th0}) = \frac{2.5 * 10^{-8}}{\sqrt{Area}} \text{ V}$
 $\sigma_l\left(\frac{\Delta U_0}{U_0}\right) = \frac{6.1 * 10^{-5}}{\sqrt{Area}}$

In this example, the following *global* variations are defined:

- On NMOS devices with model snps20n
 - Absolute variation on threshold vth0, Normal distribution with sigma=70 mV
 - Relative variation on mobility u0, Normal distribution with sigma=10%
- On PMOS devices with model snps20p: the global variations are defined similarly

The following *local* variations are defined:

- On NMOS devices with model snps20n and PMOS devices with model snps20p, respectively
 - Absolute variation on threshold vth0, Normal distribution with sigma=2.5e-8/sqrt(total_device_area)
 - Relative variation on mobility u0, Normal distribution with sigma=6.1e-5/sqrt(total_device_area)
- On resistors which do not have a model: relative variation of 10 percent on the implicit value parameter

The following sections present these topics:

- [Overview of the Variation Block](#)
- [Variation Block Structure](#)
- [Variation Block Examples](#)
- [Control Options and Syntax](#)

Overview of the Variation Block

The characteristics of circuits produced in semiconductor processing are subject to variability, as is the case for any manufactured product. For a given target technology, the nominal device characteristics are described with a set of parameters, which applies to a certain device model (for example, BSIM4). In PrimeSim HSPICE, the variability of the model parameters is described through a *Variation Block*. A Variation Block is a container for specifying variations introduced by the effects in manufacturing on geometry and model parameters.

Variations in PrimeSim HSPICE are modeled as Global, Local, and Spatial variations.

- Global Variations are variations in device characteristics from lot to lot, wafer to wafer, and chip to chip; they are caused by variations in starting material and differences between equipment and manufacturing procedures. Global Variations affect all devices with the same model name in the same way.
- Local Variations are defined as variations between devices in proximity, or with common centroid layout on the same chip; they are caused by microscopic variations in materials and geometry, and affect different devices differently.
- Spatial Variations are defined as variations due to the physical arrangement of the devices on the layout; they are caused by gradients from material properties, imperfections of lenses, and spin processes. The dependence on distance means that large designs are more affected by Spatial Variations.

All three classes can be described in the Variation Block in a flexible way by user-defined expressions. Because there are currently no industry-wide standards for specifying process variability, this feature allows each company to implement their own proprietary models for variability. The Variation Block is generally provided by a modeling group, very similar to device models (for example, BSIM) because it must be created specifically for each technology from test circuits.

Like a model, the Variation Block can be part of a library which is encrypted; therefore, the content is not accessible to the designers. They can introduce additional Variation Blocks in their netlist to define options and variations on generic elements. See [Control Options and Syntax](#) and [Variations of Element Parameters](#).

The structure of the Variation Block allows for building expressions to model interdependence and hierarchy of the variations. For example, one random variable can control the variation in oxide thickness of both PMOS and NMOS devices, as it is generally the same for both types of devices.

For controlling the behavior of Variation Blocks, see [Control Options and Syntax](#). The Variation Block is currently used for Monte Carlo, and DC/AC mismatch analyses; for a description of these analyses, see [Chapter 26, Monte Carlo Analysis—Flow and Outputs](#) and [Chapter 27, Mismatch Analyses](#), respectively.

For the functions available to build expressions as presented in the next sections, see [Using Algebraic Expressions](#).

Variation Block Structure

A Variation Block is divided into four sections:

- General section
- Subblock for Global Variations
- Subblock for Local Variations
- Subblock for Spatial Variations

This section presents the syntax of a Variation Block, followed by a discussion of the contents of the four sections.

```
.Variation
  Define options
  Define common parameters that apply to all subblocks
  .Global_Variation
    Define the univariate independent random variables
    Define additional random variables through transformations
    Define variations of model parameters
  .Element_Variation
    Define variations of element parameters
  .End_Element_Variation
  .End_Global_Variation
  .Local_Variation
    Define the univariate independent random variables
    Define additional random variables through transformations
    Define variations of model parameters
  .Element_Variation
    Define variations of element parameters
  .End_Element_Variation
  .End_Local_Variation
  .Spatial_Variation
    Define the univariate independent random variables
    Define additional random variables through transformations
```

```
        Define variation of model parameters
    .End_Spatial_Variation
.End_Variation
```

General Section

In the general section, options can be defined that control the variability analyses that use the content of the Variation Block. Options can be specified, one per logical record.

Note:

“.OPTION” (with a leading period) does not work for options specified in the Variation Block.

The correct Variation Block syntax is:

```
Option OptionName = value
```

See [Control Options and Syntax](#), for a listing and description of Variation Block control options.

Parameters, also, can be defined that apply to all subblocks in which variations are specified; however, these cannot contain any distribution-related functions. Parameters defined within a Variation Block have local name scope and are independent of parameters defined outside it.

For example: `Parameter PI=3.1416`

Subblocks for Global, Local, and Spatial Variations

Within the variation subblocks, univariate independent random variables can be defined. These are random variables with specific distributions over a certain sample space. Additional random variables can be generated through transformations. These random variables form the basis for correlations and complicated distributions.

A basic rule of the Variation Block approach is to place the model definition on the top level, instead of inside a subcircuit, as necessary in the traditional approach. In all three subblocks, variations on model parameters can be defined. This is where Global or Local Variations on the parameters of semiconductor devices are specified.

A special section within the subblock for Local Variations allows for defining Local Variations on elements. Use this section either for specifying local temperature variations or variations on generic elements that do not have a model, as used early in the pre-layout design phase, or for off-chip components; for example, resistors and capacitors.

The following sections discuss these topics:

- [Independent Random Variables](#)
- [Dependent Random Variables](#)
- [Absolute Versus Relative Variation](#)
- [Variations on Model Parameters](#)
- [Variations on Subcircuit Parameters](#)
- [Variations on Top-Level Parameters](#)
- [Variations on Temperature](#)
- [Access Functions, Get_E\(\), Get_P\(\), Get_M\(\), Get_O\(\)](#)
- [Variations of Element Parameters](#)
- [Spatial Variation](#)

Independent Random Variables

When describing variations, a standard normal (Gaussian) distribution is assumed, unless otherwise specified explicitly. This default behavior is explained in later sections. Other types of distributions or correlations are modeled by applying transformations to the independent random variables. These independent random variables are derived from three basic distributions:

- Uniform distribution: defined over the range from -0.5 to 0.5: `U( )`
- Normal distribution: with mean=0 and variance=1, default range +/-4: `N( )`
- Use the `LIMIT` distribution to create discrete “corners” for a variable. Specify it as:

```
Parameter xx=LIMIT(nominal_val, abs_variation)
```

The `LIMIT` keyword provides a random-limit distribution function by using absolute variation. The syntax adds +/- `abs_variation` to `nominal_val` based on whether the random outcome of an underlying uniform distribution is over the range of -1 to 2.

For example, in the Variation Block `aa=Limit(-2,2)` means that the `nominal_value` is -2, and the `abs_variation` is 2.

In a Monte Carlo sampling, +/- `abs_variation` is added to `nominal_val`. So the sampling values are either -4 or 0.

- User-defined cumulative distribution function: CDF (xyPairs)

If $f(x)$ is the probability density of a random variable x , then the cumulative distribution function is the integral of $f(x)$. A cumulative distribution function can be approximated

by a piecewise linear function, which can be described as a sequence of pairs of points $[x_i, y_i]$. The following rules apply:

- At least two pairs are required
- White space or a comma is required between each number
- The CDF starts at zero: $y_1=0$
- The CDF ends at one: $y_n=1$
- x_i values must be monotonically increasing $x_{i+1} > x_i$
- y_i values must be monotonically non-decreasing $y_{i+1} \geq y_i$
- $\int_{-\infty}^{\infty} x \cdot f(x) \cdot dx = 0$

where, the probability density function, $f(x)$, is the derivative of the cumulative density.

The distributions $N()$ and $U()$ do not accept any arguments.

The syntax for defining independent random variables is:

Parameter $a=U()$ $b=N()$ $c=CDF(x_1, y_1, \dots, x_n, y_n)$

These distributions cannot be defined within expressions; variables must be assigned and then the variables can be used within expressions. See examples of this operation in [Non-Gaussian Probability Distribution Functions on page 497](#).

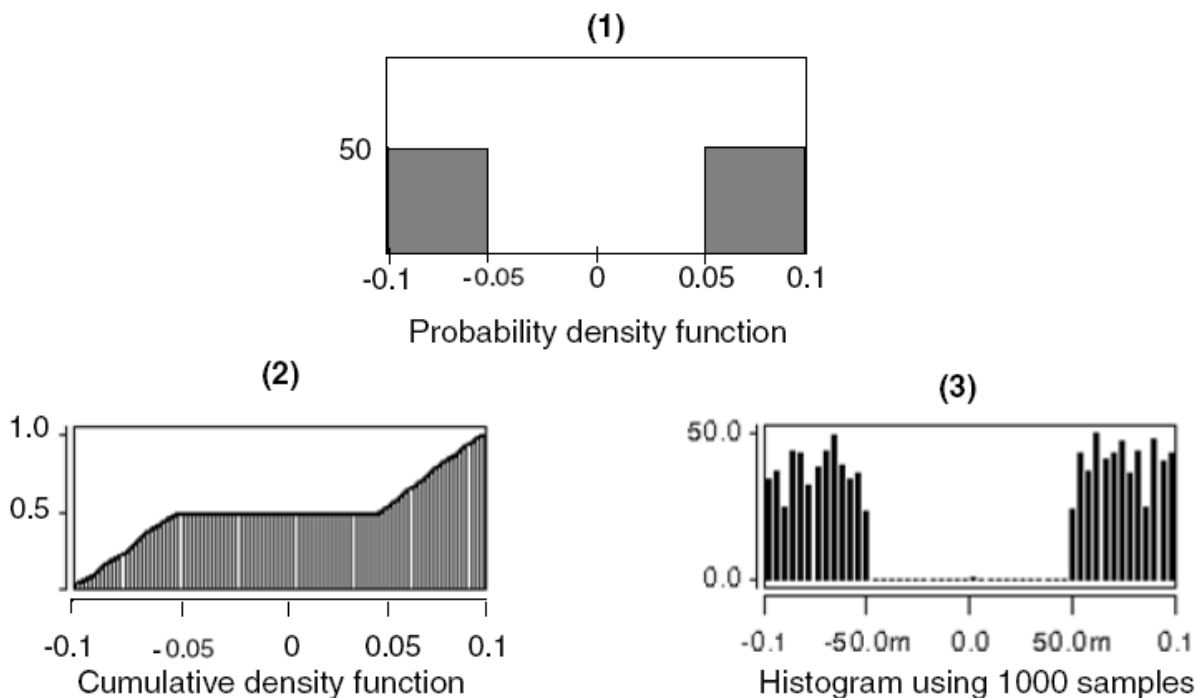
Example

The probability density function is shown in [Figure 131 \(1\)](#). [Figure 131 \(2\)](#) gives the corresponding cumulative distribution function. This is coded in the Variation Block as:

Parameter $var=CDF(-0.1 \ 0 \ -0.05 \ 0.5 \ 0.05 \ 0.5 \ 0.1 \ 1.0)$

The histogram generated by taking 1000 samples is shown in [Figure 131 \(3\)](#).

Figure 131 Probability, cumulative density and 1000-sample histogram



Dependent Random Variables

To model distributions which are more complicated than the ones which are available through the predefined independent random variables, transformations can be applied by using expressions on independent random variables. A dependent variable can also be created as a function of more than one independent random variable to express correlation.

Examples

Example 1

This example creates a random variable Y with normal distribution, with mean μ and standard deviation σ .

```
Parameter var=N( ) Y='μ + σ * var'
```

Example 2

This example creates a random variable with a uniform distribution from D to E, where D and E are arbitrary constants.

```
Parameter var=U( ) Y='0.5*(D+E) + (E-D) * var'
```

Example 3

A variable x has a log-normal distribution if $\log(x)$ is normally distributed. The probability density function for the log-normal distribution is: $f(x) = \frac{1}{\sigma x \sqrt{2\pi}} \cdot \exp\left(-\frac{(\ln(x)-\mu)^2}{2\sigma^2}\right)$

where μ is the mean and σ the standard deviation of associated normal distribution. Samples from such a distribution can be generated as

```
Parameter    var=N( )
Parameter    nor='mu+sigma*var'
Parameter    lognor='exp(nor)'
```

Example 4

If the components of the random vector $\underline{x} = (x_1, x_2, \dots, x_n)$ are all independently distributed as standard normal and the vector

$\underline{y} = (y_1, y_2, \dots, y_n)$ is defined as $\underline{y} = \underline{a} + \underline{B}\underline{x}$ then $\underline{y}$ is distributed as multivariate normal with mean $\underline{a}$ and covariance matrix $\underline{B}\underline{B}'$ where the prime indicates transpose.

Now consider the inverse problem of generating samples from a multivariate normal distribution with given mean vector $\underline{\mu}$ and covariance matrix $\underline{\Sigma}$.

The covariance matrix has the following properties:

- The matrix is symmetric.
- The diagonal elements are non-negative.
- The matrix is positive semi-definite where all eigenvalues are real and non-negative.

Now consider the Cholesky decomposition

$\underline{L}$ (a lower triangular matrix) of $\underline{\Sigma} = \underline{L}\underline{L}'$. Then samples from the distribution of $\underline{y}$ are generated as $\underline{y} = \underline{\mu} + \underline{L} \cdot \underline{x}$ with $\underline{x}$ being samples from the standard normal distribution.

$$\underline{\Sigma} = \begin{bmatrix} 1 & 0.5 & 0.5 \\ 0.5 & 2 & 0.3 \\ 0.5 & 0.3 & 1.5 \end{bmatrix}$$

Consider a numerical example with ($\underline{\mu} = \underline{0}$) and

$$\underline{L} = \begin{bmatrix} 1 & 0 & 0 \\ .5 & 1.323 & 0 \\ .5 & 0.0378 & 1.1174 \end{bmatrix}$$

An observed covariance matrix from a million samples gives

$$\begin{bmatrix} 1.0003 & 0.4990 & 0.5005 \\ 0.4990 & 1.9977 & 0.2994 \\ 0.5005 & 0.2994 & 1.4990 \end{bmatrix}$$

The correlation matrix is closely related to Σ through diagonal scaling and has these properties:

- The matrix is symmetric.
- All diagonal elements are unity.
- All off-diagonal elements are bounded by unity in magnitude.
- The matrix is positive semi-definite.

Note that the first three properties are not sufficient when describing multivariate normal distributions. For example, in the following matrix:

$$\begin{bmatrix} 1 & 0.7 & 0.0 \\ 0.7 & 1 & 0.8 \\ 0.0 & 0.8 & 1.0 \end{bmatrix}$$

All pair-wise correlations seem correct, but the eigenvalues of the matrix are (2.06, 1, -0.06) and the matrix is indefinite.

This example shows the danger of specifying the correlation matrix in a pair-wise fashion. PrimeSim HSPICE does not permit such variation definition and the correlation should be specified through transformations involving the Cholesky factors.

Absolute Versus Relative Variation

By default, the specified variation is absolute, which means additive to the original model or element parameter; however, sometimes it is more appropriate to specify relative variations. These are defined by appending a space and a “%” sign to the expression. The simulator divides the result of the expression by 100, and multiplies the result by the original parameter value and the random number from the appropriate generator to calculate the change.

Example

In the following example, the variation on the threshold parameter `vth0` is specified as normal with absolute sigma of 80 or 70 mV, and the variation on the mobility `u0` is specified as relative 15 or 13 percent.

```
.Global_Variation
  Nmos snps20N vth0=0.08 u0=15 %
  Pmos snps20P vth0=0.07 u0=13 %
.End_Global_Variation
```

Variations on Model Parameters

Variations on model parameters can be defined in subblocks for Global, Local, and Spatial Variation. In the course of the simulation, these variations are then applied to the specified device model parameters. Model parameter variations are described with the following syntax:

```
Model_Type Model_Name Model_Parameter=Expression for Sigma
```

If the expression references only constants and parameters that evaluate to constants, then a Gaussian variation with zero mean and a sigma equal to the expression is automatically implied. To describe variation as a function of previously-defined independent random variables, use the construct 'Perturb()', with the following syntax:

```
Model_Type Model_Name Model_Parameter=Perturb('Expression')
```

The expression should be enclosed in quotes, see the general PrimeSim HSPICE rules for Using Algebraic Expressions.

The following lines define a global Variation, with implicit normal distribution, with zero mean and sigma of 10, on the parameter `rsh` of resistors with model `Rpoly`.

```
.Global_Variation
  R Rpoly rsh=10
.End_Global_Variation
```

In the next example, the independent variable `Toxvar` is used to model Global Variations on oxide thickness. `Toxvar` is an independent random variable with a normal distribution, with mean=0 and sigma=1. In the device models `nch` and `pch`, `Toxvar` is applied to the parameters `tox` with a different multiplier. The oxide thicknesses in the two models are correlated.

```
.Global_Variation
  Parameter Toxvar=N( )
  Nmos nch tox=Perturb('7e-10*Toxvar')
  Pmos pch tox=Perturb('8e-10*Toxvar')
.End_Global_Variation
```

PrimeSim HSPICE supports the following model types: NMOS, PMOS, R, PNP, NPN, D, L, and C.

Variations can only be defined on parameters that are explicitly specified in the associated device model.

For binned models, variations can be defined separately by specifying the model name with the bin extension; for example, devices from bins 1 and 2 receive different variation on the parameter `lint`, which models length variation:

```
Nmos snps20N.1 lint=10n
Nmos snps20N.2 lint=12n
```

Variations on Subcircuit Parameters

The Variation Block allows for defining variation on parameters, which are specified in the subcircuit definition record with a default value. Default values can be overwritten by specifying them at subcircuit instantiation.

The syntax is:

```
Subckt SubcktName Parameter='expression for sigma'
```

The following rules apply for these types of definitions:

- Only parameters that are defined as formal numeric arguments on the subcircuit definition record can be subject to variation. (This is the line which starts with `.SUBCKT` and possibly has continuation lines.)
- The subcircuit must not be defined within another subcircuit.
- If the subcircuit contains a model, then variations on the model parameters, as described in section, [Variations on Model Parameters](#), are not supported. Instead, variations need to be defined on a subcircuit parameter and the parameter used inside the model.

The subckt parameters variation feature addresses the following needs:

- A component is defined with an expression, not available in a model:

```
r1 1 0 'Rsh*1/w*(1+b1*(tanh(b2*abs(v(1,0)/l))))'
```

This expression models a voltage-dependent resistor, with non-linear dependence not available in a traditional model. If this resistor is called within a subcircuit, and parameters are specified on the subcircuit definition record, then variation can be modeled, for example, on `Rsh`, `l`, and `w`.

```
i1 0 1 1m
x1 1 0 rtanh
.subckt rtanh a b rsh=1k w=1u l=1u
.param b1 = -0.4    b2 = 8u
r1 a b 'Rsh*1/w*(1+b1*(tanh(b2*abs(v(a,b)/l))))'
.ends
.Variation
.Global_Variation
    subckt rtanh rsh=10 %
.End_Global_Variation
.Local_Variation
    subckt rtanh rsh=3 %
.End_Local_Variation
.End_Variation
```

- A component is represented by a network and the subcomponents must have the same value when local variations are specified:

```
r1 end1 middle rmodel w='w+dw' l='l/2+dl'
r2 end2 middle rmodel w='w+dw' l='l/2+dl'
c1 middle sub cmodel w=w l=l
```

In this example, the resistor has a center tap, where a capacitor is connected. If these three components are defined within a subcircuit and parameters *dw*, *dl*, and *rsh* are defined on the subcircuit definition record, then the two resistors of the same network always have the same value; local variations only cause different instantiations of the subcircuit to have different equivalent resistance between the network terminals.

Variations on Top-Level Parameters

Variations on top-level parameters can be defined for Global Variation. The keyword `Top` is available to specify top-level parameter variation.

The `Perturb( )` construct is required when there is a univariate variable.

```
Top top_level_parameter='expression'
Top top_level_parameter=Perturb('expression')
```

Consider the following example. In this example, the parameter `rr1` is identified as an independent global variable with normal distribution, with an absolute variation specified by the given expression.

```
.param vddnum=0.95
.variation
  .global_variation
    Parameter a=2
    Top rr1 = '2*a*get_p(vddnum)'
  .end_global_variation
.end_variation
```

The following example defines the +/-10% global variation on parameter `VDD` with a uniform distribution.

```
.param vdd=2.5
.Variation
  .Global_Variation
    Parameter uniV=U( )
    Top vdd=Perturb('20*uniV') %
  .End_Global_Variation
.End_Variation
```

Note:

The top-level parameter variation can only be specified in Global variation.

Variations on Temperature

You can define Variation on temperature for global variations to support the whole space of Process-Voltage-Temperature. The temperature variations affect all devices in the netlist.

The keyword `Temp` (or `Temper`) specifies temperature variation.

```
Temp temp='expression'  
Temp temp=Perturb('expression')
```

The following example defines the global variation on temperature.

```
.Variation  
.Global_Variation  
    Temp temp='10'  
.End_Global_Variation  
.End_Variation
```

Note:

You can only specify temperature variation in global variation.

Access Functions, `Get_E( )`, `Get_P( )`, `Get_M( )`, `Get_O( )`

Caution:

An error results if any of the arguments to these functions is invalid or undefined. This strong action is to ensure that the technology library is bug free.

The following sections discuss these topics:

- [Get_E\(\)](#)
- [Get_E\(\) Function for Obtaining Channel L/W for MOSFETs](#)
- [Get_P\(\)](#)
- [Get_M\(\)](#)
- [Get_O\(\) to Return the Value of PrimeSim HSPICE Options](#)

Get_E()

Certain variations depend on element geometry, as defined with parameters at instantiation. The `Get_E( )` access function allows accessing these parameters in expressions by using the following syntax:

```
Get_E(Element_Parameter)
```

where *Element_Parameter* is the name of an element parameter, which you must define on the instantiation line (except for the `DTEMP` parameter and the multiplier `M` which have implicit values). Use this access function for specifying variations as a function of device geometry. The `Get_E( )` access function reports the effective device geometry, after

resolving parameters, scales and adjustments by process parameters, such as, xw, xl, wint, lint. Refer to the *PrimeSim™ Continuum Reference Manual: MOSFET Models* for equations which depend on the model LEVEL and Geometric Scaling for Diode Models in the *PrimeSim HSPICE® Reference Manual: Device Models* for scaling equations.

For example, you often specify that the local variation on the threshold is inversely proportional to the square root of the total area of the device, as calculated from the product of the element parameters W, L, and M.

```
Nmos nch vth0='1.234e-9/sqrt(Get_E(W)*Get_E(L)*Get_E(M))'
```

In addition, Variation Block can handle cases of calling NF and M for elemental variation with the Get_E() function. For example:

```
.Variation
.Global_Variation
    parameter var = N( )
.Element_Variation
    M(model_name ~= 'NCH') W = Perturb('var*100n*Get_E(NF)')
.End_Element_Variation
.End_Global_Variation
.End_Variation
```

Get_E() Function for Obtaining Channel L/W for MOSFETs

You can also use the get_E() function to obtain channel length and width with MOSFET models as follows:

- Get_E(L) | Get_E(W) — Returns effective channel length/width ((length/width taking into account scale and NF) for all MOSFET models, equivalent to LV1/LV2.
- Get_E(LDRAWN) | Get_E(WDRAWN) — Returns user-defined L/W for MOSFET models. (Does not take into account scale and NF.)
- Get_E(Leff) | Get_E(Weff) — For the following models: level=54, 57, 66, 69, and 70, returns effective channel length/width equivalent to LX62/LX63, which may be different from Get_E(L) | Get_E(W). For other models, this is equivalent to Get_E(L) | Get_E(W).

Get_P()

Another function allows for accessing the values of global parameters by using the following syntax:

```
Get_P(Global_Parameter)
```

The circuit context provides the parameter value, for example, from the subcircuit, if defined inside, otherwise from the top level. In the following example, sweep parameter “tol” determines the resistor variation:

```
.param tol=1
ral 1 0 1k
```

```
i1 0 1 1m
.Variation
  .Local_Variation
    .Element_Variation
      R R='Get_P(tol)' %
    .End_Element_Variation
  .End_Local_Variation
.End_Variation
.dc tol 1 5 1 monte=100
```

Get_M()

The Variation Block format also allows the access of model parameters. PrimeSim HSPICE provides the function `Get_M( )` for this purpose.

`Get_M(Model_Parameter_Name)`

The `Get_M( )` function normally has a single argument.

```
.Variation
  .Global_Variation
    nmos snps20N u0 ='2.345e-6*GET_M(u0)'
  .End_Global_Variation
.End_Variation
```

For bin models, `Get_M()` function returns the parameter value of each bin, so that each instance can get its own perturbation, although all instances share the same random value for the model parameter.

Get_O() to Return the Value of PrimeSim HSPICE Options

If a variation must be expressed as a function of a simulator option (specified as `.option=optionval` outside the Variation Block), the access function `Get_O( )` is available, using the construct `Get_O(option_name)`. For example, if you use the element parameter `nf` (number of fingers) with some advanced models, the device width reported by the `Get_E` function depends on the value of `.OPTION WNFLAG`. For variation as a function of total device area, the following definition produces the expected results, independent of the settings of `WNFLAG`:

```
vth0 = `6.0621e-9/sqrt(Get_E(W)*Get_E(L)*Get_E(M)*\\
(1-Get_O(WNFLAG)+Get_O(WNFLAG)*Get_E(NF)))`
```

Variations of Element Parameters

Variations affect devices not only in the underlying model parameters, but also through variations of properties specified at instantiation of an element, or variations on implied properties, such as local temperature. Also, early in the design phase, passive devices sometimes have only a nominal value, but no model as yet because designers have made no decision on a specific implementation. For these elements, you can specify variations on the implicit value parameter; for example: `R1 1 0 1k`.

You define variations on element parameters for Local Variations in a section within the Local Variation subblock.

You describe element parameter variations with the following syntax:

```
Element_Type Element_Parameter = 'Expression for Sigma'
```

```
Element_Type Element_Parameter = Perturb('Expression')
```

The following lines define a normal distribution with sigma of 10 on the resistors without a model:

```
.Element_Variation
  R R=10
.End_Element_Variation
```

Because different classes of devices might be affected differently, use of a condition clause provides a selection mechanism based on element name and model name:

```
Element_Type(condition_clause) Element_Parameter= 'Expression for Sigma'
```

The condition clause allows for specifying variations on selected elements, according to their name or associated model. You can indicate wildcard substitutions as “?” for single character and “*” for multiple characters.

Examples for condition clause syntax are:

```
Element_Type(model_name~='modelNameA')
Element_Type(element_name~='elNameB')
Element_Type(model_name~='modelNameC' OPERATOR
  element_name~='elNameD') par='expression'
Element_Type(model_name~='modelNameE' OPERATOR
  model_name~='modelNameF') par='expression'
Element_Type(element_name~='elNameG' OPERATOR element_name~='elNameH')
  par='expression'
```

where OPERATOR can be && (AND), || (OR). The operator “~=” stands for “matches”.

All pattern matching operations are case-insensitive. PrimeSim HSPICE ignores a leading subcircuit prefix when matching the element name.

Examples

In the following example, only resistor `ra1` varies.

```
ra1 1 0 1k
rb1 2 0 1k
.Variation
  .Local_Variation
  .Element_Variation
    R(element_name~='ra*') R=20
  .End_Element_Variation
```

```
.End_Local_Variation
.End_Variation
```

In the following example, the specified temperature variation affects only resistor `ra2` with a uniform distribution from 0 to 10 degrees (the resistor resides next to a power device).

```
ra1 1 0 1k
rb1 2 0 1k
ra2 3 0 rpoly l=10u w=1u
rb2 4 0 rpoly l=10u w=1u
.model rpoly r rsh=100 tc1=0.01
.Variation
  .Local_Variation
    .Element_Variation
      Parameter tempvar=U( )
      R(Element_Name~='ra*' && Model_Name~='rpoly')
      + dtemp=Perturb('10*tempvar+5')
    .End_Element_Variation
  .End_Local_Variation
.End_Variation
```

Supported element types and their parameters are:

Table 67 Supported elements and parameters

Element	Parameters
M	DTEMP, L, W, AD, AS, PD, PS, NRD, NRS, RDC, RSC, VDS, VGS, VBS, DELVTO
R	Rval* DTEMP, L, W, TC1, TC2, C, AC, SCALE
C	Cval* DTEMP, L, W
Q	AREA DTEMP, AREA, AREAB, AREAC, TEMP
J	L, W, DTEMP
D	DTEMP, L, W
L	Lval* DTEMP, TC1, TC2
I	DCval* mag phase
V	DCval* mag phase

The asterisk “*” denotes implicit value parameter. The DTEMP parameter is implicit; it needs specification on the element instantiation line.

Example for Voltage Source

Netlist element: V1 1 0 0.1 AC

Variation definition:

- V DC=5% (5% of 0.1)
- V MAG=5% (5% of 1)
- V PHASE=5 (5 degrees)

You define variations on element parameters for Global Variations in a section within the Global Variation subblock with almost the same syntax as within the Local Variation. But there is a limitation to when you can use a condition clause: If there is a condition clause in the variation definition line of a Global Element Variation, then use the Variation Block keyword `Parameter` to designate at least one independent random variable. The `Perturb` function uses an independent random variable element parameter perturbation. Other forms of perturbation are illegal. For example:

```
Parameter var1=N( ) var2=U( )
Element_Type (condition_clause) Element_Parameter = Perturb('Expression')
```

Example

In this example, `ra1` and `rb1` varies with global variation. So in each Monte Carlo trial, `ra1` and `rb1` are of the same resistor value.

```
ra1 1 0 1k
rb1 2 0 1k
.Variation
.Global_Variation
.Element_Variation
.Parameter a=N( )
.R(element_name~='r*') R=Perturb('20*a')
.End_Element_Variation
.End_Global_Variation
.End_Variation
```

Note:

If there are only two resistors of `ra1` and `rb1` in netlist, then an equivalent definition of Global Element Variation is:

```
.Variation
.Global_Variation
.Element_Variation
.R R=20
.End_Element_Variation
.End_Global_Variation
.End_Variation
```

But the next definition is *illegal* because the condition clause matches all resistors in the netlist:

```
.Variation
.Global_Variation
.Element_Variation
```

```
R(element_name~='r*') R=20
.End_Element_Variation
.End_Global_Variation
.End_Variation
```

Spatial Variation

To make the Spatial Variation useful, PrimeSim HSPICE needs the coordinate of a particular device. The element instantiation must extend to include placement information. For example, for a MOS device:

```
Mid Dn Gn Sn Bn ModelName w=width l=length x=xcoor y=ycoor
```

In the Spatial Variation definition, you access the element coordinates by using the `Get_E( )` function.

PrimeSim HSPICE supports only netlists with a single subcircuit, with devices on the top level or in the subcircuit. All devices of the model that has Spatial Variation defined, must have coordinates. These coordinates are numbers specified by you (no parameters allowed).

Special Rules Regarding Variation Block Usage

Generally, a foundry creates the contents of the Variation Block. To safeguard against unintentional overwriting of these variation definitions:

- The name-space of the Variation Block is separate from the netlist contents.
- Once you specify a variation on a parameter, you cannot redefine it later, even in `.ALTER` statements. For example, if you want to change the corners defined in a model library file with a `.ALTER` statement, then you must specify the Variation Block in a separate `*.lib` section.

Variation Block Examples

The following sections discuss these examples:

- [Simple VB Example with Global and Local Variations](#)
- [Principal Component-based Global Variation Modeling](#)
- [Local Variation Example for Submicron Technology](#)
- [Spatial Variation Example](#)

Simple VB Example with Global and Local Variations

You can review the following simple Variation Block in the example netlists `opampdcm.sp` and `opampmc.sp`. These netlists are available in the PrimeSim HSPICE demo directory: `$install_dir/demo/hspice/variability`.

The example defines the following variations:

- Global Variations on `vth0` (absolute)
- Global Variations on `u0` (relative)
- Local Variations on `vth0` (absolute), as a function of device area
- Local Variations on `u0` (relative), as a function of device area
- Local Variation on the implicit value of resistors (relative)

```
.Variation
.Global_Variation
  Nmos snps20N vth0=0.07 u0=10 %
  Pmos snps20P vth0=0.08 u0=8 %
.End_Global_Variation
.Local_Variation
  Nmos snps20N
vth0='1.234e-9/          sqrt (Get_E (W) *Get_E (L) *Get_E (M) ) '
+ u0='2.345e-6/sqrt (Get_E (W) *Get_E (L) *Get_E (M) ) ' %
  Pmos snps20P
vth0='1.234e-9/          sqrt (Get_E (W) *Get_E (L) *Get_E (M) ) '
+ u0='2.345e-6/sqrt (Get_E (W) *Get_E (L) *Get_E (M) ) ' %
  .Element_Variation
    R r=10 %
  .End_Element_Variation
.End_Local_Variation
.End_Variation
```

Principal Component-based Global Variation Modeling

In this example, the independent random variables `A1`, `A2`, and `A3` are the principal components on which all variations (`nmos` and `pmos`) are modeled. See [\[1\]](#) for details.

```
.Global_Variation
  Parameter A1=N( ) A2=N( ) A3=N( )
  Nmos nch
+   tox =Perturb(' -6.2E-12*A1-8.1E-12*A2-2.7E-12*A3')
+   vth0=Perturb(' -3.6E-03*A1+8.9E-03*A2-1.5E-03*A3')
+   cjn =Perturb(' -3.2E-06*A1+6.7E-06*A2-4.3E-06*A3')
+   u0  =Perturb(' 5.6E-04*A1-9.7E-04*A2+7.6E-04*A3')
+   ....
  Pmos pch
+   tox =Perturb(' -7.5E-12*A1-6.9E-12*A2-8.8E-12*A3')
```



```
+ vth0=Perturb('-7.4E-03*A1+3.3E-03*A2-7.2E-03*A3')
+ cjn =Perturb('-5.0E-06*A1+8.9E-06*A2-3.2E-06*A3')
+ u0 =Perturb(' 7.6E-04*A1-4.3E-04*A2+4.8E-04*A3')
+ ....
.End_Global_Variation
```

Local Variation Example for Submicron Technology

This Local Variation data was created using the methodology outlined in [2]. Note the different dependencies on w and l for the different parameters.

```
.Local_Variation
  Nmos nch
+ tox ='3.1e-07/sqrt(Get_E(L)*Get_E(W)*Get_E(M))' %
+ wint ='6.2e-12/sqrt(Get_E(L)*Get_E(M))'
+ lint ='2.0e-12/sqrt(Get_E(W)*Get_E(M))'
+ nch ='1.9e-06/sqrt(Get_E(L)*Get_E(W)*Get_E(M))' %
.End_Local_Variation
```

Spatial Variation Example

```
.Variation
  .Spatial_Variation
    Parameter a = N( )
    Parameter b = U( )
    Parameter Pi = 3.14159265
    Parameter Angle = 'Pi*2*b'
    NMOS snps20n
+ vth0 = Perturb('20*a*sqrt(Get_E(x)*Get_E(x)+Get_E(y)*Get_E(y)) \\  
    *cos(Angle-atan(Get_E(y)/Get_E(x))-Get_E(x)<0?Pi:0)')
  .End_Spatial_Variation
.End_Variation
```

The Spatial Variation is specified as a plane with a slope sigma of 20 mV/mm, and arbitrary rotation on the chip surface.

Group Operator {...} and Subexpressions

To improve readability of complex variation specifications, a group operator {...} and subexpressions are available. Used within a defined group, subexpressions can reference element and global parameters.

The following sections discuss these topics:

- [Syntax](#)
- [Syntax Extension with Bins](#)

- [Example](#)
- [Rules for Using the Group Operator](#)

Syntax

```
ModelType ModelName {Parameter ...ModelParameter= ...}
```

The group operator {...} separates variation definitions group by group. Each group uses one model, which means all parameters defined inside a group operator are specific to this model. A group definition starts after the Model Name, and must end after the last model parameter specification of the same Model Name. `Parameter` definitions support expressions with `Get_E( )`, `Get_P( )`, and `Get_M( )`.

`ModelParameter` definitions have no leading `Parameter` key.

Syntax Extension with Bins

```
ModelType ModelName {
  Parameter ...
  ModelParameter= ...
  ModelName.1 ModelParameter= ...
  .....
  ModelName.m ModelParameter= ...
}
```

Model parameter definitions within a group before the first bin name (`ModelName.1` in the example) apply to all bins; whereas the following definition is bin specific: `ModelName.1 ModelParameter= ...`

Example

In this example, note that the expressions before `NMOS` apply to all bins, whereas those for `mn.12` are bin specific.

```
.Global_Variation
  Parameter PG1=N( ) PG2=N( ) PG3=N( )
+ dx1=' 4.3e-9 * PG1 '
+ dvth0='0.02 * (-0.29 * PG1 + 0.95 * PG2) '
+ dtoxe='1.3e-10 * (0.39 * PG1 -0.87 * PG2 + 0.28 * PG3) '
+ F1='1.0/(2*SQRT(dvth0*dvth0)) '
+ F_FF='(-dvth0+SQRT(dvth0*dvth0))*F1 '
+ F_SS='(dvth0+SQRT(dvth0*dvth0))*F1 '
NMOS mn.12 {
  Parameter u0varg='-dvth0*(F_FF*2.1+F_SS*0.6) '
  x1=Perturb(dx1)
  vth0=Perturb('dvth0')
```

```
lvth0=Perturb('dvth0*(F_FF*0.097+F_SS*0.054)')
u0=Perturb('u0varg') %
wu0=Perturb('u0varg')%
lu0=Perturb('u0varg')%
pu0=Perturb('u0varg')%
toxe=Perturb('dtoxe') toxp=Perturb('dtoxe')
}
.End_Global_Variation
.Local_Variation
NMOS mn.12 {
  Parameter sqrtarea='SQRT(Get_E(W)*Get_E(L)*Get_E(M) '
  vth0='1.2e-9/sqrtarea'
  u0='2.3e-6/sqrtarea'
}
.End_Local_Variation
```

Rules for Using the Group Operator

The following rules apply when using the group operator:

- You cannot define independent random variables inside a group.
- Group operators do not support condition clauses inside a group.
- Any specifications that appear at the same line and after the opening '{' are ignored; a parameter definition should begin at a new line after the bracket.
- Group operators only support model parameter, *not* subcircuit parameter definitions.
- You can define the same `ModelType` and `ModelName` only once in a group; PrimeSim HSPICE aborts the simulation if it finds duplicate group definitions.

Parameter Scope

Parameters defined inside and outside a group have the following scope: parameters defined inside a group cannot conflict with those defined outside it. However, the same parameter can be redefined inside another group, and these are invisible to each other.

A few examples of parameter scoping follow.

<code>.Param a=2</code>	Defined outside a Variation Block; can be referenced using <code>get_P()</code> syntax
<code>.Variation</code>	Global parameter within Variation Block; can be used in all subblocks
<code>Parameter a=3</code>	<code>b = 6</code> ; valid in global variation subblock
<code>.Global_Variation</code>	<code>c = 2.4</code> ; only visible in this group;
<code>Parameter b='a*Get_P(a)'</code>	Can be redefined in other groups
<code>NMOS nch {</code>	<code>c = -1.8</code> ; <code>c</code> is only visible in this group;
<code>Parameter c=0.4*b</code>	Can be redefined in other groups
<code>....</code>	<code>b=12</code> ; valid in local variation subblock
<code>}</code>	Relative sigma of <code>r</code> is $0.1*2*2*3*0.01=0.012$
<code>PMOS pch {</code>	
<code>Parameter c='-0.3*b'</code>	
<code>.... }</code>	
<code>.End_Global_Variation</code>	
<code>.Local_Variation</code>	
<code>Parameter b='2*a*Get_P(a)'</code>	
<code>.Element_Variation</code>	
<code>R r='0.1*b' %</code>	
<code>.End_Element_Variation</code>	
<code>.End_Local_Variation</code>	
<code>.End_Variation</code>	

Syntax and Examples

Model Card in the Header Section

The purpose of the model card in the header section is to communicate to PrimeSim HSPICE the model name used in the parasitic section for the resistors as well as the reference temperature. The reference temperature is equal to the GLOBAL_TEMPERATURE in ITF with units in degree Celsius.

Syntax

```
.model model_name R Tref=global_temperature
```

Example

```
.model resStar R Tref=25
```

Parasitic Section

The resistance and capacitance records take the form:

```
Cxxx node1 node2 val SENS [param_id, param_id, ...]=  
[sens_coeff, sens_coeff, ...]
```

```
Rxxx node1 node2 model_name R=val TC1=val TC2=val
SENS [param_id, param_id, ...] = [sens_coeff, sens_coeff, ...] .....
```

Examples

A C record in NETNAME format is as follows:

```
C1 G2[21]:F12 Y2:897 0.699 Sens [0,1,5,6] = [0.009,0.001,0.006,0.010]
C2 X3:962 RX[12]:F74 0.324 Sens [0,1,5,8] = [0.010,0.006,0.017,-0.003]
```

An R record in NETNAME format is as follows:

```
R1 G2[21]:F12 G2[21]:8 resStar R=0.699 TC1=0.0023 TC2=4e-7
Sens [5,6,7] = [0.51,0.64,0.86]
```

The `Sens` keyword defines the start of sensitivity information and the two vectors are the sparse sensitivity indices and the corresponding values. The second vector of real numbers is interpreted as the sensitivities. The lengths of the two vectors must match. There must be one blank space between the `Sens` keyword and the sensitivity indices.

Control Options and Syntax

You can specify options, one per logical record in a Variation Block. Several of the listed options are useful if a Variation Block is part of a model file that a designer cannot edit. However, you can add a Variation Block with options to control how the contents of all Variation Blocks are used in the analysis. For Monte Carlo-specific options: see [Options for Monte Carlo Analysis on page 553](#).

Few of these options are also supported in the post processing application of PrimeSim AVA. For more information, see Postprocessing section in the *PrimeSim AVA User Guide*.

Note:

No period is required before the word `Option` in the Variation Block, and is, in fact, illegal.

The following options can be specified within the Variation Block:

- `Option Normal_Limit=Value` Theoretically, the normal distribution range extends from minus infinity to plus infinity. This can, however, lead to fundamental violations in model definitions like negative channel length or width. For this reason, the normal distribution is truncated at `Normal_Limit` by PrimeSim HSPICE.
- `Option Ignore_Variation=Yes` Ignores the Variation Block and executes earlier style variations (traditional Monte Carlo analysis). By default, the contents of the Variation Block are executed and other definitions (`AGAUSS`, `GAUSS`, `AUNIF`, `UNIF`, `LOT`, and `DEV`) are ignored. Previous methods of specifying variations on parameters and models

are not compatible with the Variation Block. Thus no changes are required in existing netlists other than adding the Variation Block.

- Option Ignore_Local_Variation=Yes Excludes effects of local variations in simulation. Default is No.
- Option Ignore_Global_Variation=Yes Excludes effects of global variations in simulation. Default is No.

PrimeSim HSPICE supports Ignore_Global_Variation and Ignore_Local_Variation with the AGAUSS or traditional-style variation definitions.

In the following example, global variations are not simulated, and the normal distributions are exercised to ± 6 . For information regarding Local and Global Variations, see [Subblocks for Global, Local, and Spatial Variations](#).

```
.Variation
    Option Ignore_Global_Variation=Yes
    Option Normal_Limit=6
    .Global_Variation
        Definitions for global variations
    .End_Global_Variation
    .Local_Variation
        Definitions for local variations
    .End_Local_Variation
.End_Variation
```

- Option Ignore_Spatial_Variation=Yes Excludes effects of spatial variations in simulation. Default is No.
- Option Output_Sigma_Value [=Value]
- Option Vary_Only Subckts=SubcktList - Use either this option or the following option, *but not both*, to limit variation to the specified subcircuits. Actual subcircuit names are specified here (not the hierarchical names).
- Option Do_Not_Vary Subckts=SubcktList - Excludes variation on the specified subcircuits. Use either this option to limit variation to the specified subcircuits or the one above, *but not both*. Actual subcircuit names are specified here (not the hierarchical names).

Caution:

Global variations would still be applied to the subcircuits specified using this option.

- Option Vary_Only instances=instance1[, instance2...]

Limits the variations to the specified elements only.

- Option Do_Not_Vary instances=instance1[, instance2...]

Excludes the specified elements varying during the Monte Carlo Simulation.

Both AGAUSS-type variation and variation block local and element variation are supported. All sampling methods, except external data block, are supported. Wildcards are supported in the instance list.

Caution:

- .DCMATCH and .ACMATCH analyses are not supported.
- Applying variation selectively is possible only for local variations and element variations (within either the `global_variation` or the `local_variation` section). Disabling/enabling global variation to a few devices would take out the very essence of it. So, global variations would still be applied to the elements specified using this option.

- Option `MC_File_Only=yes|no`

Use this option to generate a random number sample file (*.mc0) without invoking any analysis (applicable to AGAUSS style also). The feature is useful for an external block sampling simulation when you want to modify the samples before running the Monte Carlo simulation. If the netlist has a Monte Carlo command, then the MC command provides the number of samples; if the netlist has no MC command, then the number of samples is zero.

- Option `External_File=filename`

Use this command to enable read-in of an external block line-by-line during the simulation stage. This command distributes memory consumption and avoids overtaxing the front end with a data block containing large samples. This option is also available for DP + DC Monte Carlo.

- `.OPTION SET_MISSING_VALUES = Random | Zero`

Sub-option to `SAMPLING_METHOD=External` option to limit reporting of missing independent random variables.

You can use this option to control missing random values in a `.data` block for external sampling:

- `Set_Missing_Values=Random`: PrimeSim HSPICE generates its own random values for the missing random variables in a `.data` block.
 - `Set_Missing_Values=Zero`: PrimeSim HSPICE generates zero values for those missing random variables in `.data` block in external sampling table.
- Option `Other_Percentile=data_block_name`

Or Option `Other_Percentile=list(val1, val2,...,valn)`, where the value range for `val1` to `valn` is between 0.0 and 1.0.

Or `Option Other_Percentile=list_sigma(val1, val2,...,valn)`, where the value range for `val1` to `valn` is between -10.0 and 10.0.

Use this option to specify quantiles lower than 1 percent when data mining (`.mpp0/.ava.report` file) in conjunction with `.DATA=percentile` in a netlist to get results for user-provided percentiles. This option allows you to help to see how much impact there is from trailing data points, or to count samples near the absolute minimum for a sample set.

Use the first or the second syntax to specify the information in terms of a cumulative distribution. Or use the third syntax to specify the information in terms of sigma values of a normal distribution.

Examples:

```
Option other_Percentile = list(0.0013, 0.0228, 0.1587, 0.8413, 0.9772,
0.9987)
Option other_Percentile = list_sigma(-3.0, -2.0, -1.0, 1.0, 2.0, 3.0)
```

The `list_sigma` keyword indicates that the additional user-specified percentiles are provided in terms of the standard normal distribution.

PrimeSim HSPICE ignores any user-specified percentile that are the same as those from it.

PrimeSim HSPICE now generates the `*.mpp0/.ava.report` file as the statistical summary, if running MC simulation. The heading row of the percentile section of the `*.mpp0/.ava.report` file will show both percentile values (without any change) and sigma values.

See also [Using the Other_Percentile Option on page 606](#).

Note:

For options which can take Yes or No value, such as `Ignore_Variation`, `Ignore_Local_Variation`, and `Use_Agauss_Format` for example, 1 or 0 input (instead of Yes or No) is not supported.

For detailed descriptions and examples of these options, see Enhanced Monte Carlo Control Options in *PrimeSim HSPICE® Reference Manual: Commands and Control Options*.

References

- [1] K. Singhal and V. Visvanathan: Statistical device models from worst-case files and electrical test data. IEEE Trans. Semiconductor Manufacturing, November 1999. (Global variation modeling by principal components)
- [2]

P.G. Drennan and C.C. McAndrew: Understanding MOSFET mismatch for analog design. IEEE J. Solid-State Circuits, March 2003. (Modeling mismatch in nanometer technologies)

26

Monte Carlo Analysis—Flow and Outputs

Describes the enhanced Monte Carlo analysis in PrimeSim HSPICE using the Variation Block.

For information on bisection in conjunction with Monte Carlo, see [Chapter 23, Timing Analysis Using Bisection](#).

PrimeSim HSPICE ships numerous examples for your use. See Variability Examples in the *PrimeSim HSPICE User Guide: Demonstration Netlists* for paths to the demo files discussed in this chapter.

The following sections discuss these topics:

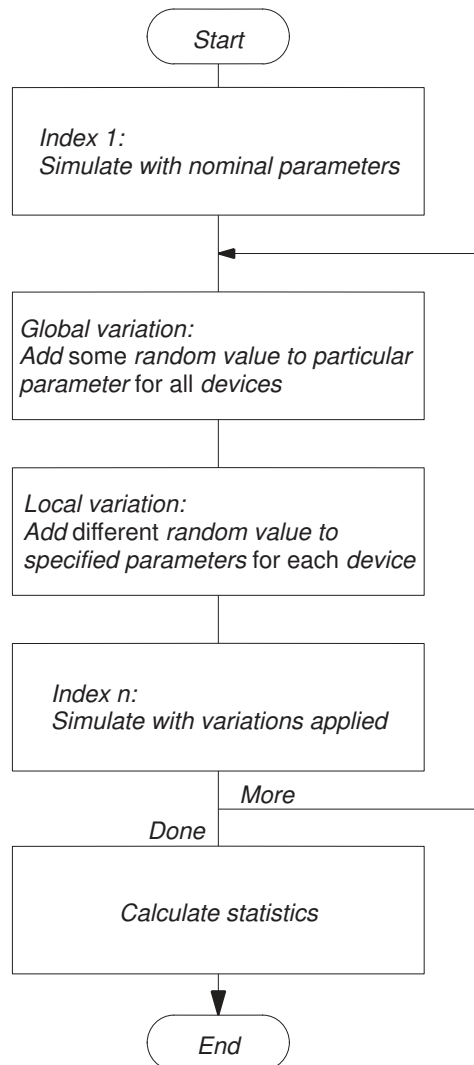
- [Monte Carlo Analysis in PrimeSim HSPICE](#)
- [Sampling Options](#)
- [Comparison of Sampling Methods](#)
- [Application Considerations](#)
- [Output Files for Monte Carlo Simulations](#)
- [Post-Processing of Monte Carlo Results](#)
- [Standalone Data Mining in PrimeSim HSPICE](#)
- [Large Scale Monte Carlo Simulation – Traditional and Variation Block](#)
- [Automatic Incremental DP Monte Carlo](#)
- [Modifying the Built-in Distribution for Monte Carlo Analysis](#)
- [Yield Computation in Monte Carlo Analysis](#)
- [Sigma Amplification MC and Outputs](#)
- [Troubleshooting Monte Carlo-VB Issues](#)
- [References](#)

Monte Carlo Analysis in PrimeSim HSPICE

[Figure 132 on page 552](#) shows the Monte Carlo simulation flow when you specify global and local variations.

- Sample number 1 of a Monte Carlo analysis always executes with nominal values and no variation.
- For subsequent samples, PrimeSim HSPICE updates the parameters specified for variation in the Variation Block with random values.
- For global variations, the same random value for all elements that share a common model changes a specified parameter.
- For local variation, a different random value for each element changes the specified parameter.
- The changes due to global and local variations are additive and saved in a file for post-processing.
- After an update of the elements, PrimeSim HSPICE executes the simulation and saves the measurement results. When PrimeSim HSPICE simulates all the requested samples, it calculates the statistics of the measurement results and includes them in the run listing.

Figure 132 Monte Carlo analysis flow in PrimeSim HSPICE



See the following sections for these topics:

- [Input Syntax](#)
- [Options for Monte Carlo Analysis](#)
- [Output for Variation Block Monte Carlo](#)

Input Syntax

You always execute Monte Carlo analysis in conjunction with another analysis (see [Traditional Monte Carlo Analysis Characteristics](#) for full discussion):

```
.DC sweepVar start stop step [SWEEP MONTE=MCCCommand]
.AC type step start stop [SWEEP MONTE=MCCCommand]
.TRAN step start stop [SWEEP MONTE=MCCCommand]
```

Syntax for *MCCCommand*:

```
MONTE=val | list (num) | val Firstrun=num |
+ list (num1:num2 [num3] [num4:num5])
```

Parameter	Description
val	Specifies the number of random samples to produce.
Firstrun=num	Specifies the sample number on which the simulation starts.
list (num)	Specifies the sample number to execute.
list (num1:num2 num3 num4:num5)	Executes samples from num1 to num2, sample num3, and samples from num4 to num5.

The parameter values and results are always the same for a particular sample, whether generated in one pass or using `Firstrun` or the `list` syntax (see [Monte Carlo Setup on page 485](#)). Therefore, you can split Monte Carlo analyses or distribute them across multiple machines. See [Running Distributed Processing on a Network Grid in Chapter 4, Distributed Processing and Multithreading](#).

DC Sweep Examples

These examples apply a DC sweep to a parameter *k*. The first case produces 10 samples. The second case produces five samples, starting with sample number 6. The last two examples simulate samples 5, 6, 7, and 10.

```
.dc k start=2 stop=4 step=0.5 monte=10
.dc k start=2 stop=4 step=0.5 monte=5 firstrun=6
.dc k start=2 stop=4 step=0.5 monte=list (5:7 10)
```

Options for Monte Carlo Analysis

When you execute simulations by using the Variation Block, PrimeSim HSPICE ignores control options for the traditional Monte Carlo style (see [Control Options and](#)

Syntax). However, you can specify the following Monte Carlo-specific options in the first section of the Variation Block:

Starting from S-2021.09 version, the Monte Carlo options can also be specified using PrimeSim AVA Monte Carlo. For more information, see Monte Carlo Analysis section in the *PrimeSim AVA User Guide*. When an option is mentioned in both netlist/variation block and PrimeSim AVA, the one in PrimeSim AVA Monte Carlo takes precedence.

- Option `Use_AGAUSS_Format=Yes|No`

Enables you to combine traditional Monte Carlo Gaussian trials (with the exclusion Lot/Dev) with Variation Block advanced sampling methods.

- Option `Random_Generator|RANDGEN [= 3LC|0|MOA|1|UVS|MCG|WH]`

Use this option to specify the random number generator used in PrimeSim HSPICE traditional Monte Carlo analysis.

RANDGEN Option	Description
3lc 0	Uses a traditional random number generator.
moa 1	Uses a multiply-with-carry type random number generator with longer cycle.
uvs	Uses a 64-bit universal random number generator with longer cycle.
mcg	(default) Uses a multiplicative congruential generator with longer cycle.
wh	Uses another longer cycle.

For the generators of `mcg` and `wh`, there is almost no time cost to skip random number, no matter how large the number is following the keyword `firststrun`.

- Option `Stream [= x | Random | Default]`

Specifies an integer stream number for random number generator (only for Variation Block). The minimum value of `x` is 1, the maximum value of `x` is 20; If `Stream=Random`, PrimeSim HSPICE creates a random stream number between 1 and 20 according to the system clock, and prints it in the `*.lis` file for later use. `Stream=Default` is equivalent to `Stream=1`.

- Option `Seed=x | Random` Where `x` is a positive integer from 1 to 259200. Setting `Random` allows PrimeSim HSPICE to select an integer from the range. This option also works for AGAUSS-style Monte Carlo when you use advanced sampling methods.

Note:

Option `Seed` is only valid for the random number generator of `MOA` and overrides the setting of Option `Stream`. Use `Stream` only when `Seed` is not set.

- `Option Normal_Limit [= Value]`

Limits the range for the numbers generated by the random number generator for standard normal distributions. The default value is 20 for sampling methods SRS, LHS, SOBOL and NIED. The default value is 4 for the OFAT and FACTORIAL sampling methods. For example, numbers in the range $-/+4$ are created. The allowed range for the option is 0.1 to 20. Negative values are automatically reset to the default.

For the supported sampling methods, the **Normal Limit** option can also be set from PrimeSim AVA Monte Carlo application. For more information, see Postprocessing in *PrimeSim AVA User Guide*.

- `Option Output_Sigma_Value [=Value]`

This option helps in reporting results in terms of sigma values which are typically 1-, 3-, or 6-sigma based on the standards used in different companies. Specifies the sigma value of the results of Monte Carlo, DCMATCH, and ACMATCH analyses. Default is 1; range is 1 to 10. This option only changes the output listings and that the input sigma is not affected.

- `Option Vary_Only instances=instance1[, instance2...]`

Limits the variations to the specified elements only.

- `Option Do_Not_Vary instances=instance1[, instance2...]`

Excludes the specified elements varying during the Monte Carlo Simulation.

Both AGAUSS-type variation and variation block local and element variation are supported. All sampling methods, except external data block, are supported. Wildcards are supported in the instance list.

Starting from S-2021.09 version, these options can be applied in the PrimeSim AVA, Monte Carlo application. This is enabled using the **IRV Selector** option in **Advanced Settings**. For more information, see Configuring Advanced Settings section in *PrimeSim AVA User Guide*.

Caution:

- .DCMATCH and .ACMATCH analyses are not supported.
- Applying variation selectively is possible only for local variations and element variations (within either the `global_variation` or the `local_variation` section). Disabling/enabling global variation to a few devices would take out the very essence of it. So, global variations would still be applied to the elements specified using this option.
- `Option Print_Only Subckts=SubcktList` Use this option to limit output in the *.mc# file to the specified subcircuits. Use the next option to exclude output from the specified

subcircuits to the *.mc# file. *Do not use both options.* Specify actual subcircuit names here (not the hierarchical names). See [Parameter File](#).

- Option Do_Not_Print Subckts=SubcktList

Use this option to exclude output from the specified subcircuits to the *.mc# file. Use the previous option to limit output in the *.mc# file to the specified subcircuits. *Do not use both options.* Specify actual subcircuit names here (not the hierarchical names). See [Parameter File](#).

- Option External_File=filename

Where this command enables read-in of external block line-by line-during the simulation stage. This command distributes memory consumption and avoids overtaxing front-end with block containing large samples. This option is also available for DP+ DC Monte Carlo.

- Option MC_File_Only=yes|no

Use this option to generate a random number sample file (*.mc0) without invoking any analysis (applicable to AGAUSS style also). The feature is useful for an external block sampling simulation when you want to modify the samples before running the Monte Carlo simulation. If the netlist has a Monte Carlo command, then the MC command provides the number of samples; if the netlist has no MC command, then the number of samples is zero.

- Option Add_Variation=yes Use this option to amplify local variation of the model parameters, especially when variation is provided by a foundry. Usually, the base variation is set by the foundry, but with this option you can add variation on model parameters based on a multiplier you supply when using a combined Variation Block and AGAUSS-style simulation. For example:

In this example, the first four lines are variations provided by the foundry. Option Add_Variation=yes and .Option Sampling_Method are user-supplied required options and nmos nch_mac.nch tox= 10% is the VB global variation where nch_mac is the subckt name and nch is the binned model name.

```
.lib 'mismatch_totalflag_b.1' stat
.lib 'mismatch_totalflag_b.1' global
.lib 'mismatch_totalflag_b.1' total
.lib 'mismatch_totalflag_b.1' tt
.Variation
  Option Add_Variation=yes
  .Global_Variation
    nmos nch_mac.nch= 10%
  .End_Global_Variation
  .Local_Variation
    nmos nch_mac.nch tox= 10%
  .End_Local_Variation
.End_Variation
```


- `Option Other_Percentile=data_block_name`

Or `Option Other_Percentile=list(val1, val2,...,valn)`, where the value range for *val1* to *valn* is between 0.0 and 1.0.

Or `Option Other_Percentile=list_sigma(val1, val2,...,valn)`, where the value range for *val1* to *valn* is between -10.0 and 10.0.

Use this option to specify quantiles lower than 1 percent. This option allows you to help to see how much impact there is from trailing data points, or to count samples near the absolute minimum for a sample set.

Use the first or the second syntax to specify the information in terms of a cumulative distribution. Or use the third syntax to specify the information in terms of sigma values of a normal distribution.

Examples:

```
Option other_Percentile = list(0.0013, 0.0228, 0.1587, 0.8413, 0.9772,
0.9987)
Option other_Percentile = list_sigma(-3.0, -2.0, -1.0, 1.0, 2.0, 3.0)
```

The `list_sigma` keyword indicates that the additional user-specified percentiles are provided in terms of the standard normal distribution.

PrimeSim HSPICE ignores any user-specified percentile that are the same as those from it.

PrimeSim HSPICE now generates the `*.mpp0/.ava.report` file as the statistical summary, if running MC simulation. The heading row of the percentile section of the `*.mpp0` file will show both percentile values (without any change) and sigma values.

The `Other_percentile` arguments can also be populated using the PrimeSim AVA Monte Carlo application. When using the `.ava.report` file, these quantiles are listed in Quantiles section. For more information, see the `*.ava.report` File section in *PrimeSim AVA User Guide*.

For more information on this option, see [Using the Other_Percentile Option](#).

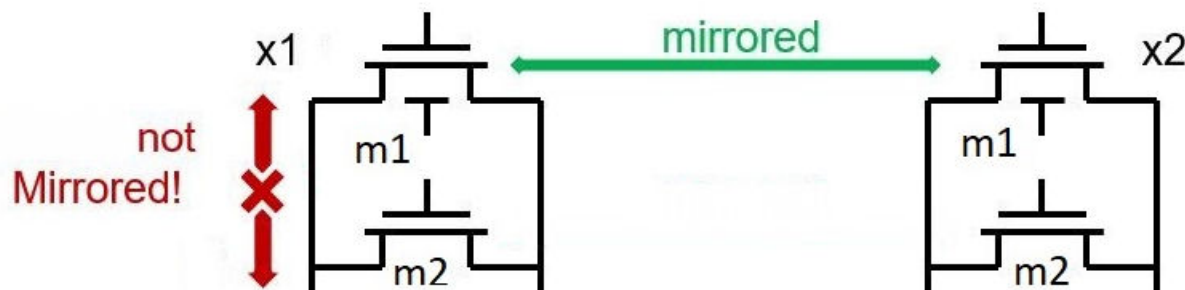
- `Option Mirror_Components = instanceList` Use this option to specify the list of instances. The instance list uses the same set of random values in Monte Carlo simulation. This option does not support external sampling, the sampling values in external data block always has higher priority. This option supports SRS, LHS, Factorial, OFAT, Sobel, Niederreiter sampling methods.

Mirror_Components Example

Consider the following statement:

```
option Mirror_Components = x1,x2
```

Here, subcircuits or instances in x1 are mirrored to those in x2. However, notice (see the following figure) that the subcircuits or instances inside x1 (or x2) are not expected to be mirrored.



Note:

This option is supported in:

- VB local/element and AGAUSS local type variation only.
- User needs to set `.OPTION SAMPLING_METHOD=SRS` (or other supported sampling methods) in the netlist with traditional or AGAUSS type variation definitions.

Example:

```
.variation
Option Mirror_Components="r1,r2","r3,r4"
.end_variation
```

or

```
.Option Mirror_Components='"r1,r2","r3,r4"' $as global option setting.
```

Caution:

This option is not supported for models specified within `.PROT` and `.UNPROT`.

- Option `large_scale_mc=no|yes`

(default) *no*, if sample size < 1M; *yes*, if sample size ≥ 1M. When sample size is larger than 1M, PrimeSim HSPICE will evoke streaming algorithm for Monte Carlo automatically and the following warning message is issued:

```
**warning** The Monte Carlo sample size >=, one million and HSPICE
is switching to the large sample mode, (option Large_Scale_MC = Yes)
for efficient computations and lower disk space requirements. If you
do not want this feature, please set Large_Scale_MC = No in variation
block.
```

- Option `measure_file=no|yes`

Essential for validation of the results and for initial deployment. You can get all the simulations done in one pass with `large_scale_mc=Yes` and `measure_file=Yes`. Default is `no`.

- Option `tail_samples [= x]`

Controls the tail samples to be retained. Default is `100`.

- Option `histogram_bins [= x]`

Controls the resolution of histograms. The value range is `[4, 10000]`. The default value is `800`.

- Option `sensitivity_analysis=yes|no`

Controls output of the sensitivity report section in the `.mpp/.ava.report` file. The default value for this option is `yes`.

Note:

In a Monte Carlo simulation, if the total number of independent random variables exceeds 50000, PrimeSim HSPICE does not generate the sensitivity report section in the `.mpp` file and `.mc0` file. Similarly, in DCMATCH or ACMATCH simulation, if the total number of independent random variables exceeds 5000, then the tool does not generate the sensitivity report section in the `.dm0` file. If you require this section, you should explicitly set `Option sensitivity_analysis=yes` in the netlist.

To generate the `.mc0` file, you should set the following options:

- Option `sensitivity_analysis=yes`
- Option `MC_File_Only=yes`

You can set the **Sensitivity Analysis** from **General Setting** using the PrimeSim AVA Monte Carlo application. It provides diverse options to generate more detailed report, incorporate composite device information to reduce clutter and customize it to show in CSV format or hide parameter level report.

The detailed sensitivity report can also be generated while performing the post processing analysis on existing `mc0` and `measure` files. For more information, see the Postprocessing section in the *PrimeSim AVA User Guide*.

- Option `qqnorm_file [= yes|no]`

Controls the generation of the Q-Q table file. This file has the extension `.qqt0.csv/.ava.qqt0.csv`. You can plot the data in the file in Excel. The default value for this option is `yes`.

- `OPTION SAMPLING_METHOD=SRS|LHS|Factorial|OFAT|Sobol|Niederreiter|External`

Enables use of advanced sampling methods with traditional Gaussian Monte Carlo trials. Starting from the 2016.06 version, the option `sampling_method=SRS` is the default. To invoke the traditional or the old SRS flow, you need to set the option `random_generator=3LC`.

SRS and Sobol are supported in S-2021.09 version, while using Monte Carlo application.

- `Option Replicates=Number`

When the advanced sampling method Latin Hypercube is used with traditional Monte Carlo simulation, you can add this option following `.OPTION SAMPLING_METHOD=LHS`. This option runs replicates of the Latin Hypercube samples. The sample with nominal conditions is simulated once. PrimeSim HSPICE repeats the LHS run the number of times specified by *Number*. For example, if in a regular run, you have 10+1 (including nominal value) iterations, if you set `.OPTION REPLICATES=2`, you generate 21 (or $2 \times 10 + 1$) Latin Hypercube samples.

- `Option Intervals`

Note:

- All VB options that support Gaussian-type variations are now enhanced as global options. Also, if PrimeSim HSPICE detects these options, it automatically sets the option `use_agauss_format = yes`, irrespective of whether these options are set inside the VB or set as global options. For such circumstances, the list file prints a warning message to indicate that PrimeSim HSPICE is using the advanced sampling method for Gaussian-type variations.
- For options which can take *Yes* or *No* value, such as `Ignore_Variation`, `Ignore_Local_Variation`, and `Use_Agauss_Format` for example, 1 or 0 input (instead of *Yes* or *No*) is not supported.

For detailed descriptions and examples of these options, see Enhanced Monte Carlo Control Options in *PrimeSim HSPICE® Reference Manual: Commands and Control Options*.

For examples of Monte Carlo analysis using these options, see SolvNetPlus article: [Useful Options in PrimeSim HSPICE Statistical Analysis](#).

Note:

You must be logged in to SolvNetPlus for the link to connect directly to the article. If you are prompted to log in to SolvNetPlus upon clicking the link to the article, log in, then click the link again to reach the article.

Output for Variation Block Monte Carlo

The following sections cover these topics:

- [Simulation Listing](#)
- [Measurement Output File](#)
- [Parameter File](#)

Simulation Listing

The output listing file contains a summary of the names of all input parameters that are subject to global or local variations. The measured results print for each sample. Then PrimeSim HSPICE reports the statistics for the measured data.

Partial printout of an output listing:

```
MONTE CARLO DEFINITIONS
Random number generator is default, and stream = 1
Global variations:      model      parameter
                        snps20n     vth0
                        snps20n     u0

Local variations:      model      parameter
                        snps20n     vth0
                        snps20n     u0

Element variations:    element     parameter
                        r1          r

*** monte carlo  index =      1 ***
    systoffset1=  1.3997E-03
*** monte carlo  index =      2 ***
    systoffset1= -9.2694E-04

MONTE CARLO STATISTICS
    meas_variable = systoffset
    mean   =  1.4398m      varian =  1.2391u
    sigma  =  1.1132m      avgdev = 893.3815u
    max    =  5.3035m      min    = -1.4532m
    1-sigma = 1.1132m      median =  1.4184m
```

More detailed statistics are given in the *.mpp or .ava.report file (see [Summary Statistics on page 597](#) and [Variable Screening on page 608](#)).

Measurement Output File

Measure commands save simulation results for each sample, along with its index number. Depending on the analysis type, the name of the result file has an extension of .ms#,

.ma#, or .mt#, where # denotes the regular sequence number for PrimeSim HSPICE output files.

Parameter File

A file with an extension of *.mc# saves the changes in all parameter values subject to variation. The structure of this file is similar to regular measure files. The header section presents the names of the parameters and independent variables as follows:

- For independent variables: Variable_Name:@:ID
- For global variation on model parameter: Model_Name:@:Parameter_Name:@:ID
- For local variation on element parameter: Element_Name:@:Parameter_Name:@:ID
- For local variation on element/model parameter:
Element_Name:@:Model_Name:@:Parameter_Name:@:ID
- For local variation + AGAUSS function on a subcircuit parameter:
Element_Name.parameter:@:Subckt_Name:@:Parameter_Name:@:ID
- For top-level parameters (see [Variations on Top-Level Parameters on page 532](#)):
top:@:parameter_name:@:GGR or top:@:parameter_name:@:GGA for absolute and relative variation, respectively
- For temperature variation (see [Variations on Temperature on page 533](#)):
temp:@:temp:@:IGN for implicit independent variable with normal distribution or
temp:@:temp:@:GGA for dependent variable absolute variation

The following sections discuss these topics:

- [Parameter and IRV Types and Keywords in the *.mc# File](#)
- [Generating the *.mc Output File](#)
- [Example of a *.mc# File](#)

Parameter and IRV Types and Keywords in the *.mc# File

[Table 68](#) lists the independent parameter types, respectively.

Table 68 Independent Parameter Type Identifier

First character		Second character		Third character	
I	Independent variable	G	Global	N	Normal distribution
		L	Local	U	Uniform distribution
		S	Spatial	C	Custom distribution
		G or L	Limit	L	Limit distribution

Table 68 *Independent Parameter Type Identifier (Continued)*

First character	Second character	Third character
	I	Interconnect

Table 69 lists the Independent Random Variable Types and Keywords for Variation and AGAUSS in *.mc# files. “N”, “U,” and “C” are normal, unified, and custom distributions, respectively.

Table 69 *IRV types/keywords for VB and AGAUSS format sampling in *.mc# files*

IRV Type	Keyword
Variation Block (Three-Letter Keyword String)	
N(), Global	IGN
U(), Global	IGU
C(), Global	IGC
L() Global (limit distribution)	IGL
N(), Local	ILN
U(), Local	ILU
C(), Local	ILC
L(), Local (limit distribution)	ILL
N(), Spatial	ISN
U(), Spatial	ISU
C(), Spatial	ISC
N(), Interconnect	ITN
U(), Interconnect	ITU
C(), Interconnect	ITC
AGAUSS Format (Four-Letter Keyword String)	
GAUSS()/AGAUSS()	IGNC
UNIF()/AUNIF()	IGUC
GAUSS()/AGAUSS(), Global (Limit Distribution)	IGLC

Table 69 *IRV types/keywords for VB and AGAUSS format sampling in *.mc# files (Continued)*

IRV Type	Keyword
GAUSS()/AGAUSS(), Local	ILNC
GAUSS()/AGAUSS(), Local	ILUC
GAUSS()/AGAUSS(), Local (Limit Distribution)	ILLC

[Table 70](#) lists the dependent parameter type identifiers.

Table 70 *Dependent Parameter Type Identifier*

First character		Second character		Third character	
M	Model	G	Global	R	Relative
E	Element	L	Local	A	Absolute
S	Subcircuit Variables	S	Spatial		
T	Interconnect				

The independent variables include explicitly specified random variables (for example: $A=N()$), and the internally generated random variables for implicit definitions in the expressions for sigma (for example: $Nmos\ snps20\ vth0=0.07$). PrimeSim HSPICE reports values for parameters that have absolute variation specified in the Variation Block as absolute deviation from the nominal value. The output reports values for parameters that have relative variation specified as a relative deviation in percent. If the netlist, or the model, or both are encrypted, PrimeSim HSPICE prints hash codes in the appropriate places, which are meaningful to PrimeSim HSPICE for [External Sampling](#).

Generating the *.mc Output File

The Monte Carlo feature generates the random value sample file, *.mc# output in an efficient way, by printing only one file in the error check stage. The *.mc file eliminates duplication of information that appears in other files, for example, “alters” and “status” columns, which appear in measurement file outputs. Independent random variable data (IRV) are only printed to the *.mc file.

For variations defined with Variation Block style, related variations such as $xxx:@:xxx:@: MGA$, $xxx:@:xxx:@: EGA$, and so forth, are omitted because such information is not required for data mining and unnecessarily increases the file size.

The values in the AGAUSS style *.mc file are the values from the standard normal or uniform distribution and not the derived values after accounting for mean and sigma. The

*.mc file incorporates data *that had been displayed in* *.mcs#, *.mca#, and *.mct# files, which PrimeSim HSPICE no longer generates.

If there are multiple Monte Carlo commands in the netlist, only one *.mc file is generated in each .ALTER simulation. In an *.mc# file, the sampling number is selected from the largest of all simulations. If Monte Carlo commands are specified with complex options, such as,

```
.dc step start stop sweep Monte = list(xx, xxx, xxx)
```

PrimeSim HSPICE only exports the numbers defined in the list() option.

The PrimeSim HSPICE simulator only exports the numbers defined in the list() option. When running the PrimeSim HSPICE simulator using PrimeSim AVA Monte Carlo application, the mc0 file is not generated by default. Use the **Generate MC0** option in **General** settings to enable the mc0 file. The .mc0 file is necessary if you intend to create a sensitivity report. For more information, see Configuring Advanced Settings section in the *PrimeSim AVA User Guide*.

Character separators in *.mc files are indicated by the :@: (colon-"at"-sign-colon) combination. This combination avoids confusion with hierarchy separators which use a single @ character.

Example of a *.mc# File

```
index      snps20n:@:vth0@IGN      snps20n:@:u0:@:IGN
           snps20n:@:vth0:@:MGA      snps20n:@:u0:@:MGR
           xi82.mn6:@:snps20n:@:vth0:@:MLA
xi82.mn6:@:snps20n:@:u0:@:MLR
           xi82.mn1:@:vth0@ILN      xi82.mn1:@:u0@ILN
           xi82.mn1:@:snps20n:@:vth0:@:MLA      xi82.mn1:@:snps20n:@:u0@MLR
           xi82.mn2:@:vth0:@:ILN      xi82.mn2:@:u0:@:ILN
           xi82.mn2:@:snps20n:@:vth0:@:MLA
xi82.mn2:@:snps20n:@:u0:@:MLR
           xi82.rcomp:@:r:@:ILN      xi82.rcomp:@:r:@:ELR
           status      alter#
1.0000      0.      0.      0.
           0.      0.      0.
           0.      0.      0.
           0.      0.      0.
           0.      0.
           1.0      1.0000
2.0000      0.6141      0.6284      4.299e-02
           6.284e-02      2.1837      0.2184
           1.7554      0.1755      1.6017
           0.1602      0.4769      4.769e-02
           -1.0088      0.5350
```

In this example, PrimeSim HSPICE first reports the changes due to the global variations on parameters vth0 (absolute) and u0 (relative). Then PrimeSim HSPICE reports the changes on each device due to local variations on the same parameters. Finally, PrimeSim

HSPICE reports the local variation on the parameter r of the element r_{comp} . Note that the parameter value applied to the device for a particular sample is the nominal value, plus the reported change due to global variations, plus the reported change due to local variations, and so on.

The contents of this parameter file are useful for data mining. You can investigate the relationship of circuit response variation to parameter variation in combination with the measured data in the regular output file, by using, for example, a Pairs Plot as shown in [Figure 129](#).

Note:

The contents of this file are subject to change.

Sampling Options

PrimeSim HSPICE provides simple random sampling (SRS) as well as advanced sampling schemes for related applications. OFAT and Factorial sampling use statistical design-of-experiment techniques while LHS and LDS reduce the sampling error in the results. External sampling provides a generic interface that permits users to overload the internal random number generators and pass PrimeSim HSPICE sample values generated from other statistical tools. See also [Comparison of Sampling Methods](#). The sampling options are as follows:

- [Simple Random Sampling \(SRS\)](#)
- [One-Factor-at-a-Time \(OFAT\) Sampling](#)
- [Factorial Sampling](#)
- [Latin Hypercube Sampling \(LHS\)](#)
- [Sobol and Niederreiter Sampling \(LDS\)](#)
- [External Sampling](#)

Simple Random Sampling (SRS)

`Option Sampling_Method = SRS`

Traditional Monte Carlo selects the samples in a random manner from the specified distributions. This is the default sampling method. You can select this through the option `Option Sampling_Method = SRS`.

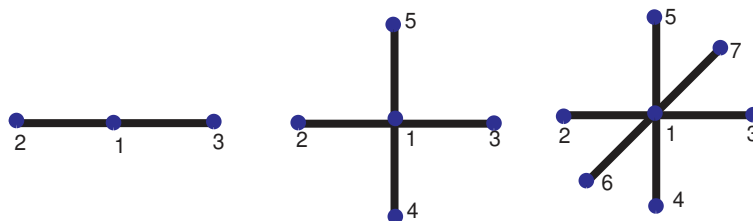
In SRS (or other) samplings, on the first point (which is also a nominal point), the limit distribution values use the nominal (no perturbation) values.

One-Factor-at-a-Time (OFAT) Sampling

Option `Sampling_Method=OFAT`

This sampling method varies One-Factor-at-a-Time, a Design of Experiments feature [1]. It is useful for sensitivity studies and for constructing low-order response surface approximations. The number of samples is $2m+1$ with m independent variables. OFAT ignores the number that the Monte Carlo command specifies, and m must be less than or equal to 15000. Sampling starts with no perturbation (nominal), then negative and positive perturbation only on the first parameter, negative and positive perturbation only on the second parameter, and so forth. In OFAT sampling, unsampled limit distributions return their nominal values, similar to SRS. The amounts of perturbation are the extreme values for a uniform distribution, and the `Normal_Limit` values for a normal distribution. Figure 133 illustrates OFAT examples.

Figure 133 One-Factor-at-a-Time sampling with one, two, and three independent variables



A sub-option, `intervals=n`, generates $2n+1$ equally spaced samples along the range of each independent variable. The total number of sample points increases to $2mn + 1$. The full syntax is: `Option Sampling_Method=OFAT Intervals=2`

Figure 134 Suboption intervals



An alternative sub-option is to declare `Intervals=+0.05|-0.05` to perform one-sided OFAT sampling in instances such as cell characterization. The keys `0.5` specify the one-sided samples with the samples being at 0 and $+ \text{Normal_Limits}$ and key `-0.5` for one-sided samples with the points being at nominal and $-\text{Normal_Limit}$.

Factorial Sampling

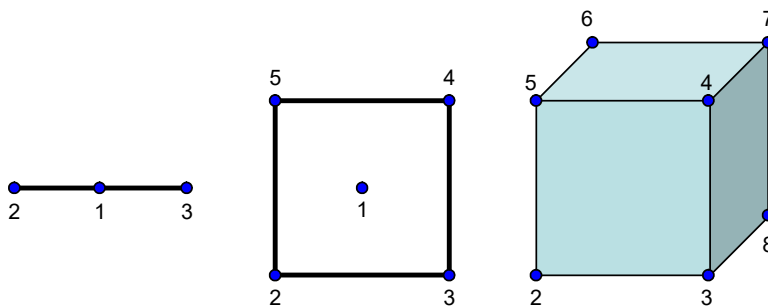
`Option Sampling_Method=Factorial`

Use this option to:

- Evaluate the circuit response at the extremes of variable ranges to get an idea of the worst and best case behavior.
- Create polynomial response surface approximations.

Factorial sampling evaluates the circuit at the center of the hypercube (nominal) and at all its corners (see [Figure 135](#)). There are $1+2^m$ samples for a circuit with m independent variables; this method ignores the number of samples that the Monte Carlo command specifies. To prevent large runaway jobs, PrimeSim HSPICE restricts the problem dimension to $m \leq 12$, which results in ~4K simulations. If you violate the size constraint, PrimeSim HSPICE ignores the command and generates an error message.

Figure 135 Factorial Hypercube Evaluation at Center and Corners for 1, 2, and 3 independent random variables



Latin Hypercube Sampling (LHS)

`Option Sampling_Method=LHS`

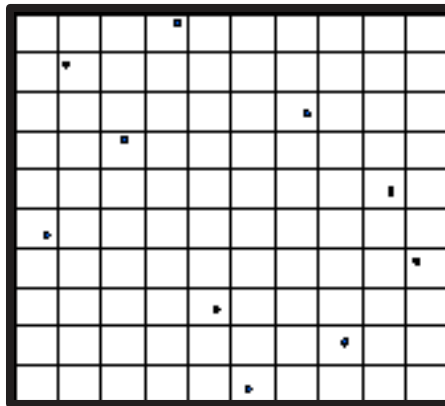
Latin Hypercube Sampling is an efficient sampling technique for Monte Carlo analysis of systems. These systems are modeled by computers and have large number of variable parameters [\[2\]](#)[\[3\]](#). Advantages of LHS are:

- The estimation error is smaller on most real world problems and you can use a smaller sample size to get the same precision in the results.
- The sample points are evenly spread over the entire range of variation of each parameter.
- The circuit is exercised over a wide range of parameter values. The circuit often detects weak spots in the design.

- You can replicate the sampling using `Option Replicates=Value`

This option runs replicates of the Latin Hypercube samples. The sample with nominal conditions is simulated once. PrimeSim HSPICE repeats the LHS run the number of times specified by `Value`. For example, if, in a regular run, you have 10+1 (including nominal value) iterations, if you set `Replicates=2`, you generate 21 (or $2 * \text{Value} + 1$) Latin Hypercube samples.

Figure 136 Example of the distribution of 10 sampling points in two dimensions



Monte Carlo with firstrun and list Options

LHS is a special sampling method. Unlike SRS, its sample values are decided by the sample size. LHS sampling with `firstrun` or `list` keyword in Monte Carlo analysis requires a new keyword `lhs_sample_size` to be set, which indicates the sample size of LHS.

Syntax:

```
.TRAN/DC/AC... [sweep] monte=list(num1:num2,...lastnum2, lastnum1)
lhs_sample_size=num
```

Or

```
.TRAN/DC/AC... [sweep] monte=num1 firstrun=num2 lhs_sample_size=num
```

Where the last monte sweep index in `list` and `firstrun` syntax are `lastnum1` and `num2+num1-1`, respectively.

The keyword `lhs_sample_size` should be set such that the last monte index must be equal to or less than `replicates*lhs_sample_size+1`.

Note:

With `firstrun`, the LHS sample size is set using `lhs_sample_size` and not with `monte` keyword. The `monte` number should be adjusted for the number specified in the `lhs_sample_size`.

If this condition of `lhs_sample_size` is not satisfied, then the following error message is displayed:

```
**error** (file:line) monte sweep index exceeds LHS_SAMPLE_SIZE limit
```

If the `LHS_SAMPLE_SIZE` is multi-defined, the maximum one will be used with the following warning message:

```
**warning** multiple definitions of LHS_SAMPLE_SIZE, the maximum one will be used.
```

Examples

For example, assuming `replicates=1`, if you set `monte=10`, `firstrun=8`, and `lhs_sample_size=15` then the last monte index is $10+8-1=17$, which is more than 16 ($\text{replicates} \times \text{lhs_sample_size} + 1$). Therefore, PrimeSim HSPICE displays the following error message:

```
**error** (file:line) monte sweep index exceeds LHS_SAMPLE_SIZE limit.
```

Sobol and Niederreiter Sampling (LDS)

```
Option Sampling_Method=NIEDERREITER
```

```
Option Sampling_Method=SOBOL
```

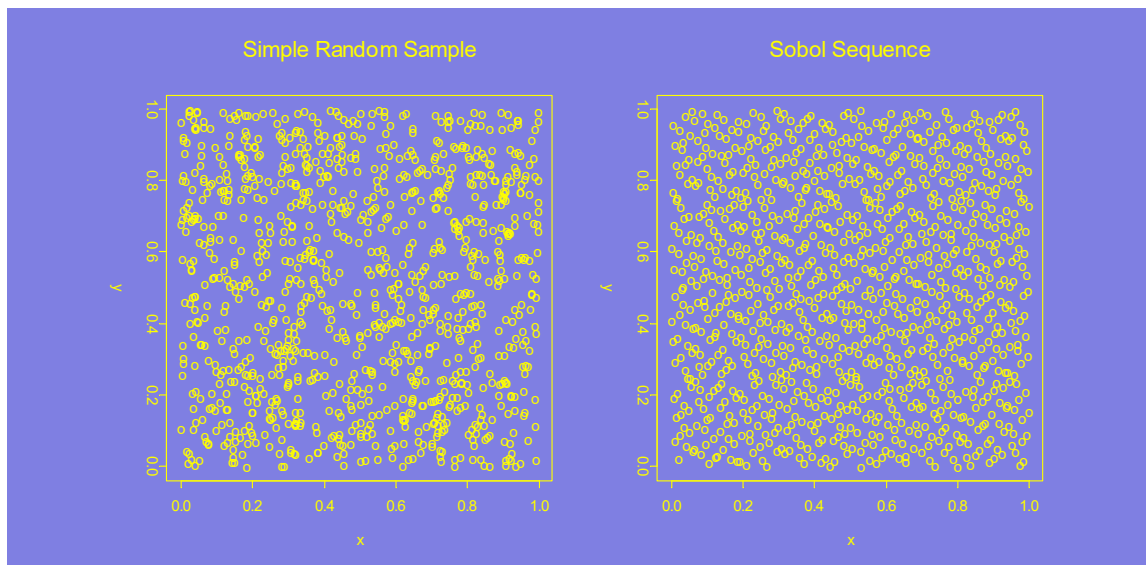
Two Low-Discrepancy Sequence (LDS) quasi-random number generators—Sobol and Niederreiter—support Variation Block-based Monte Carlo analysis [4] [5] [6]. LDS sample points are even more evenly distributed compared to LHS and the sampling error is lower. Dimension limitations exist for both Sobol and Niederreiter sampling. The current maximum dimensions are 318 for Niederreiter and 21200 for Sobol sampling.

Each of these sampling options has an auto-switch flow:

- If you use Niederreiter sampling with more than 318 independent random variables (dimension), then PrimeSim HSPICE switches to Sobol sampling.
- If you use Sobol sampling when the number of independent random variables (dimension) is larger than 21200, then PrimeSim HSPICE uses Sobol sampling for the first 21200 variables and simple random sampling for the remaining dimensions.

Figure 137 shows the locations of 1024 samples in two dimensions for simple random sampling and the Sobol Sequence. The Sobol points are better spaced while those from SRS are “lumpy.” This is the general property of LDS.

Figure 137 Comparing low-discrepancy Sequence: Simple Random versus Sobol



External Sampling

You can also execute a data set of externally created perturbations instead of relying on one of the built-in sampling methods. External sampling allows design and process exploration tools to run statistical experiments with the variables for each sample under their full control. In this way, for example, you can exclude certain transistors from the analysis (that is, have the parameters remain fixed instead of being randomly varied).

Note:

- External sampling does not support encrypted or protected model libraries.
- Starting from the 2017.03-SP2 release, external sampling supports the Limit distribution in addition to the Normal distribution independent random variables.

The following sections provide these details:

- [Usage Model for External Sampling](#)
- [Syntax](#)
- [Controlling the Read-in of an External File](#)
- [Using the MC_File_Only Option](#)
- [Allowing Zero Variations in External Sampling for Missing IRVs](#)

Usage Model for External Sampling

Use the following procedure to add the custom-generated samples to PrimeSim HSPICE Monte Carlo flow:

1. Execute PrimeSim HSPICE with a standard simulation command (.AC, .DC, .TRAN) and monte=1 to produce an *.mc0 file, which lists all the independent variables (see [Parameter File on page 562](#)).
2. Create a data block outside PrimeSim HSPICE with the desired perturbations on the independent variables for global and local variations.
3. Run a PrimeSim HSPICE simulation with externally generated data block content.
4. Repeat steps 2 and 3, depending on the outcome of the previous experiments.

Syntax

The external sampling feature is defined in two parts in the Variation Block, a data block and an option.

The data block syntax is the same as for the regular PrimeSim HSPICE data block from .Data to .EndData. The first variable is always the index. All identifiers for the variables start with “I” because this is the only variable type which can be set externally. You invoke the feature itself by specifying the external sampling method, with the appropriate block name. For example:

```
.Variation
.Data test1
Index p1:@:IGN      snps20p:@:vth0:@:ILN  x1.mn1:@:q1:@:ILN  x2.mn1:@:vth0:@:ILN
1      .0            .0                  .0                .0
2      .0            0.1086              .0                .0
3      .3            .0                  .0                .0
4      .0            .0                  1.0                .0
5      .0            .0                  .0                1.0
6      .0            .0                  1.0               -1.0
.EndData

Option Sampling_Method=External Block_Name=test1
.End_Variation
```

To run a particular sample from the data block, use the following:

```
monte = list(num) or monte = list(<num1:num2> <num3> <num4:num5>)
monte=list(n1:n2) can exceed a limit in the data block.
```

If the netlist or the model, or both, are encrypted, the hash codes printed in the parameter file are recognized by PrimeSim HSPICE when reading in the external sampling data block.

Additional rules:

1. PrimeSim HSPICE does not check the range of values in the supplied data block against option value `Normal_Limit`.
2. Independent random variables which are not specified in the data block are assigned new random values generated by a new seed.
3. Monte number is based on index of external block, it is not based on how it is set in Monte command. If samples are missing in external block, then it is skipped in MC simulation.
4. External sampling supports list and firstrun. The selected Monte Carlo is supported for external sampling. But selected Monte index must be a subset of index of external block. Otherwise, PrimeSim HSPICE terminates with error message as **error* selected Monte Carlo index is out of the range of external sampling*.
5. Use the `OPTION SET_MISSING_VALUES = zero | random` (default random) command to allow zero variations in external sampling for missing IRVs
 - If `set_missing_values = zero`, then all missed IRVs in external block are of constant nominal value.
 - If `set_missing_values=random`, then the IRVs are not occurring in external block, SRS samples are used instead.

Controlling the Read-in of an External File

Use `Option External_File=filename` to enable read in of an external data block line-by-line during the simulation stage. This command distributes memory consumption and avoids overtaxing the front end with a data block containing large samples. This option is also available for DP+ DC Monte Carlo. For example:

```
Option Sampling_Method=External Block_Name=extern_data  
+ External_File=extern.mc0
```

Where `extern.mc0` contains:

```
.Data extern_data  
...  
.Enddata
```

Using the MC_File_Only Option

The `MC_File_Only` option enables you to generate a random value sample file without running an actual Monte Carlo simulation. Use the Variation Block `Option`

`MC_File_Only=yes|no` to circumvent the need to run a double Monte Carlo simulation.
The syntax is:

```
.Variation
  Option MC_File_Only=yes|no
.End_Variation
```

or

```
.OPTION MC_FILE_ONLY = yes | no
```

You can use this option in multiple applications during an external sampling flow. Consider a scenario where you expect that a few elements under local variation simulation must have the same variations on them. In a normal Monte Carlo simulation, every element gets a unique random value when local variations are defined on them. Thus you need to manually modify the assigned sample value to the selected elements before running the Monte Carlo simulation. Do this in PrimeSim HSPICE by using the External Sampling method.

The steps are:

1. Run the regular PrimeSim HSPICE simulation with `.OPTION MC_FILE_ONLY = Yes`.

This run generates an MC sampling file (`*.mc0`).

2. Manipulate the data in this file such that each random parameter corresponding to one element has the same value for each sample matching with that of other elements of interest.

This step is simplified by printing out the `*.mc0` file in `*.csv` format which can be directly read in an Excel sheet. You can manipulate columns in the Excel sheet easily. Set `.option MEASFORM=3` in the netlist to generate the file in CSV format.

Allowing Zero Variations in External Sampling for Missing IRVs

If you only want to vary a subset of transistor parameters in the netlist and avoid PrimeSim HSPICE creating an enormous table for devices that you do not care about, you can declare a sub-option to `SAMPLING_METHOD=External` — `OPTION SET_MISSING_VALUES=Zero`.

The syntax is: `OPTION SET_MISSING_VALUES = Random | Zero`

The default value is `Random` if the option is not specified in a netlist.

Use this option to control missing random values in a `.data` block for external sampling:

- `Set_Missing_Values=Random`: PrimeSim HSPICE generates its own random values for the missing random variables in a `.data` block.
- `Set_Missing_Values=Zero`: PrimeSim HSPICE generates zero values for those missing random variables in `.data` block in external sampling.

Syntax example:

```
option Sampling_Method = External Block_Name = XXXX  
+ File_Name = YYYY Set_Missing_Values = Random|Zero
```

Monte Carlo

Comparison of Sampling Methods

This section provides illustrations to describe the qualitative behavior of various sampling methods.

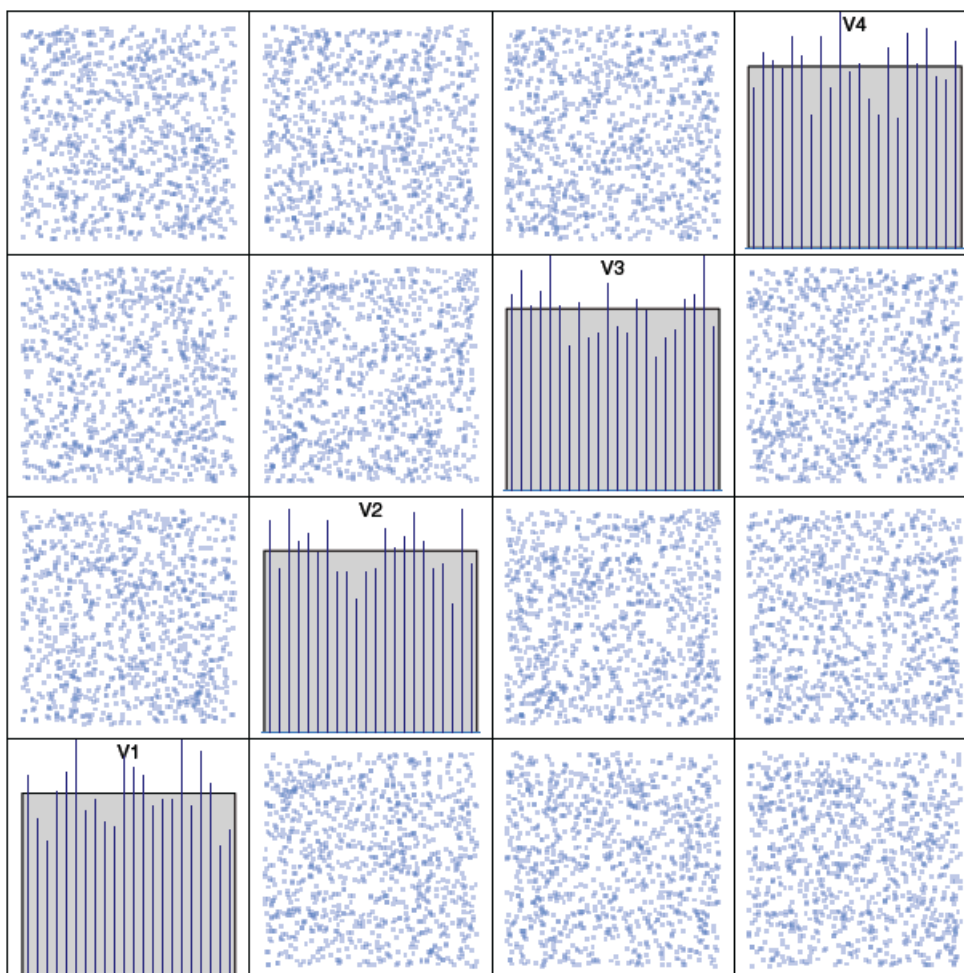
The sampling methods are described in detail in the following:

- [Gaussian Style Random Variable Definition](#)
- [Example 1: Variation Duplicated in Traditional Format and Variation Block](#)
- [Example 2: Subcircuit and Macro Models](#)

Pairs Plot for SRS Samples

[Figure 138](#) shows a pairs plot for 1024 samples from SRS in four dimensions with uniform distributions.

Figure 138 Sampling: SRS Distribution: uniform n ; 1024 maxCor; 7.2 percent



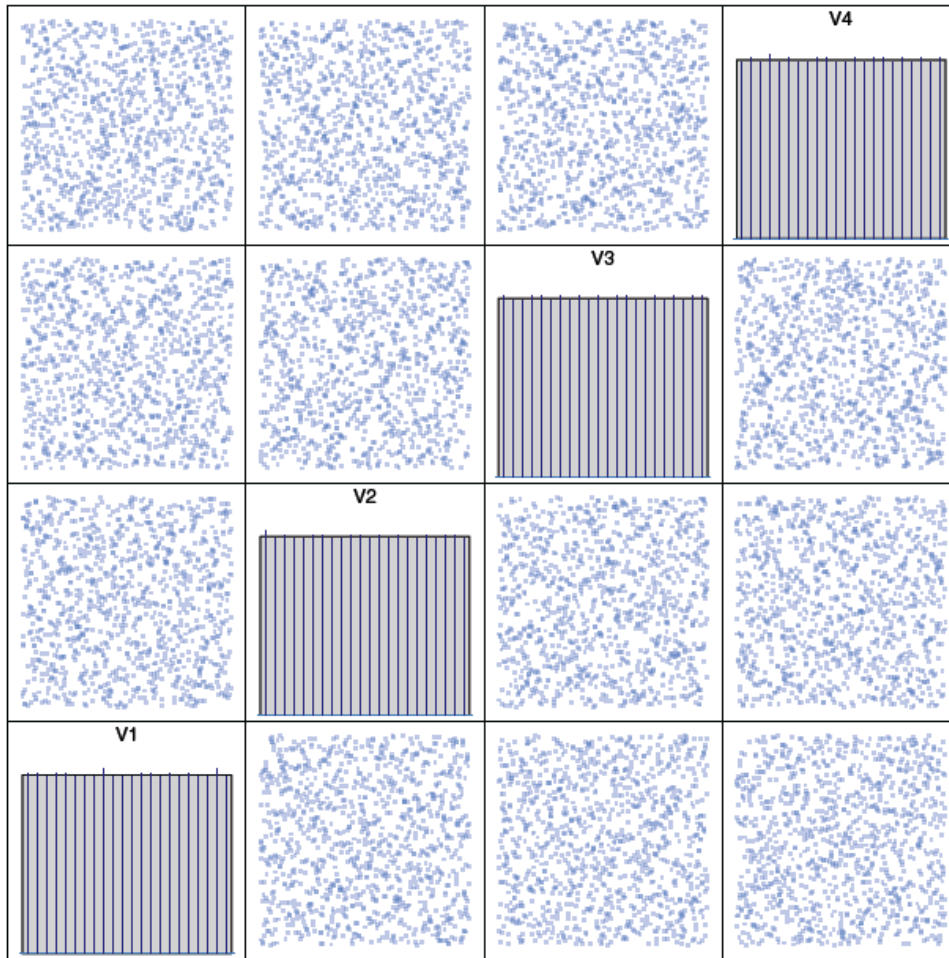
The diagonal subplots show the sample histogram together with the ideal uniform density as a shaded area for each of the random variables. The off-diagonal subplots give the two-dimensional projections of the sample points. Note that the 1-D projections depart from their ideal behavior. The correlation coefficients are also computed for the sample and the maximum absolute value is found to be 7 percent. As the samples are independent, the ideal value is zero. Such qualitative behavior is typical of small samples in traditional Monte Carlo.

1024 Points, Latin Hypercube Sample

Figure 139 shows 1024 LHS points, again in four dimensions and with uniform distributions. The 1-D histograms match the ideal shaded density—this is a consequence of the way in which LHS are constructed. The 2-D projections have a somewhat better

distribution with the maximum correlation being 4.5 percent for the particular sample. The correlation value changes with the random generator seed and small correlations are not currently enforced in constructing LHS in PrimeSim HSPICE. The correlation could be more or less than SRS in a particular experiment.

Figure 139 Sampling: LHS Distribution: uniform n ; 1024 maxCor; 4.5 percent

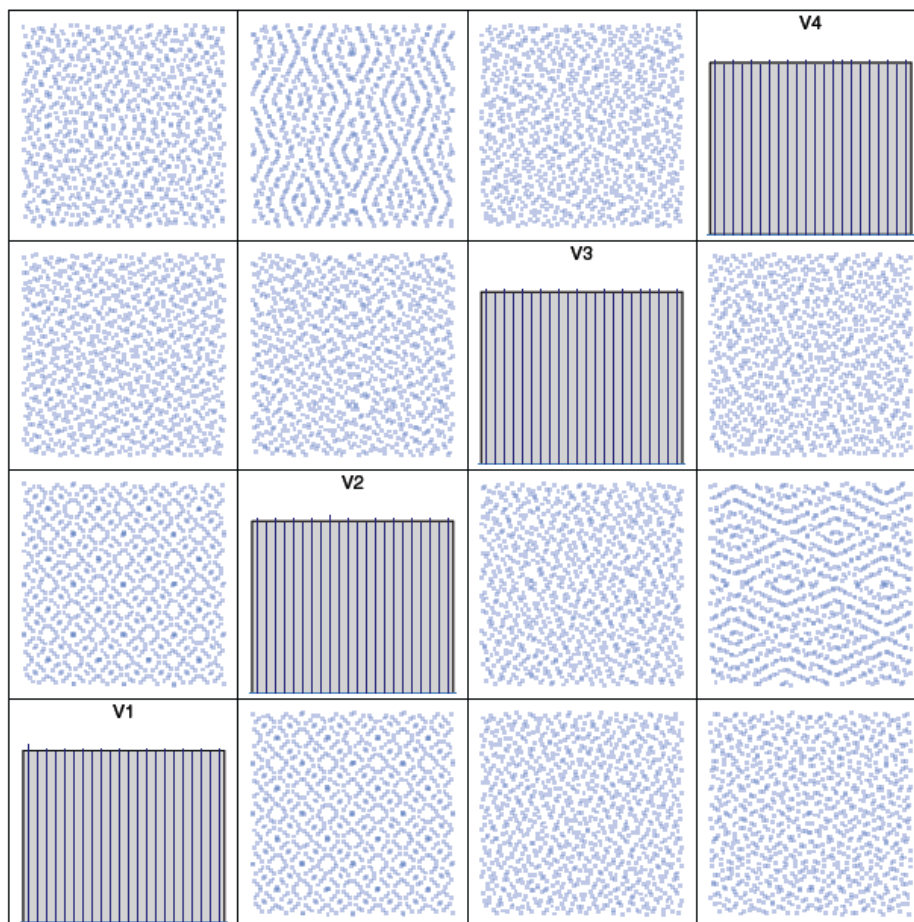


Four-Dimensional Sobol Sequence

Figure 140 shows the first 1024 sample points from the four-dimensional Sobol sequence. Like LHS, the 1-D histograms match the ideal shaded density — this is a property of low-discrepancy sequences. The uniformity in two dimensions is also better and the largest absolute correlation coefficient is 0.5 percent. Two-dimensional uniformity is one of the criteria used in generating low-discrepancy sequences. Notice that the Sobol sequence has a well-defined pattern that does not look random, for example, between variables one

and two. The reason is that low-discrepancy points are selected by special algorithms that try to fill the space as uniformly as possible.

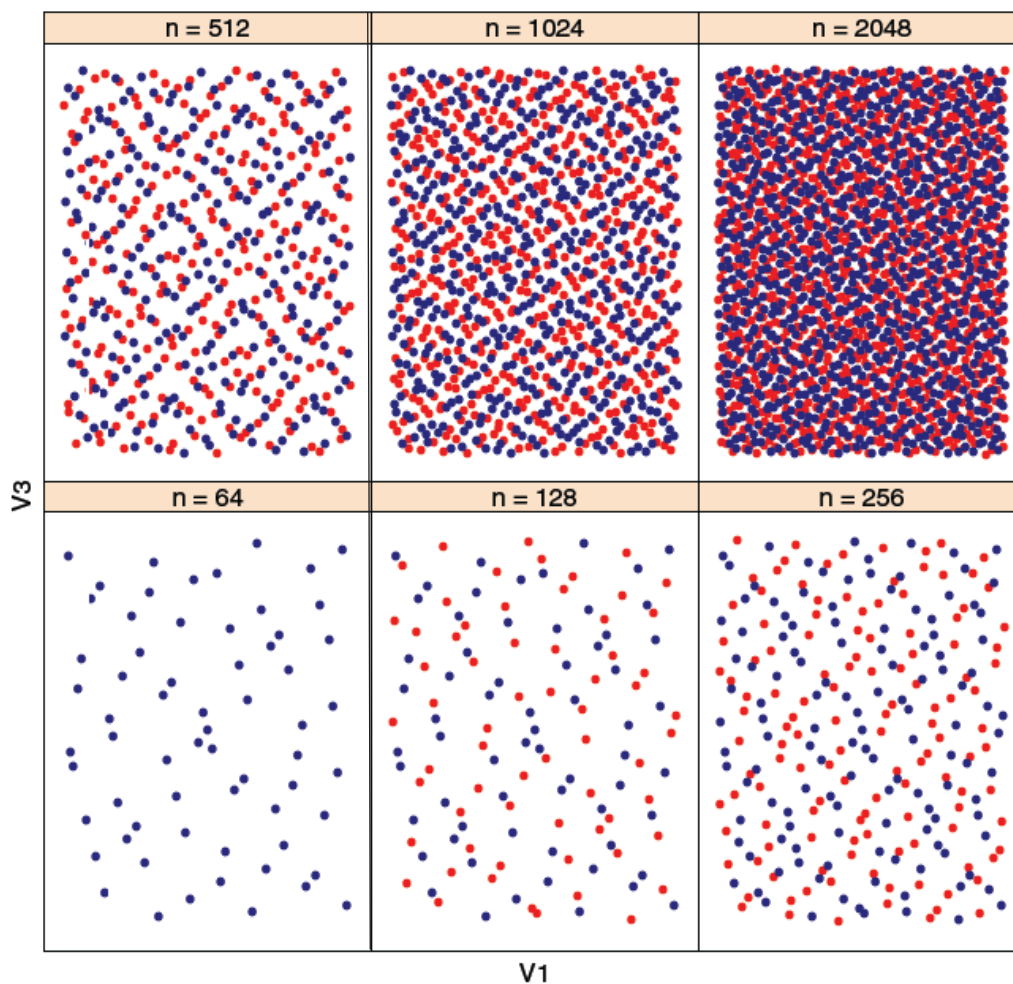
Figure 140 Sampling: Sobol Distribution: uniform n ; 1024 maxCor; 0.5 percent



Space Filling Properties for Sobol Samples

The space filling property is illustrated in [Figure 141](#). It shows the 2-D projection of samples for variables one and three for sample sizes ranging from 64 to 2048. Starting in the lower left, the subplot shows the first 64 Sobol points in blue. The subplot with $n = 128$ shows the first 64 points in red (which are the same as the blue points in the first subplot) and the samples from 65 through 128 shown in blue. Proceeding in a similar manner, other subplots are generated with the red points representing the previous samples and the blue points representing the new ones. The Sobol samples are generated in a structured manner with the new points filling the “holes” left behind by the prior samples.

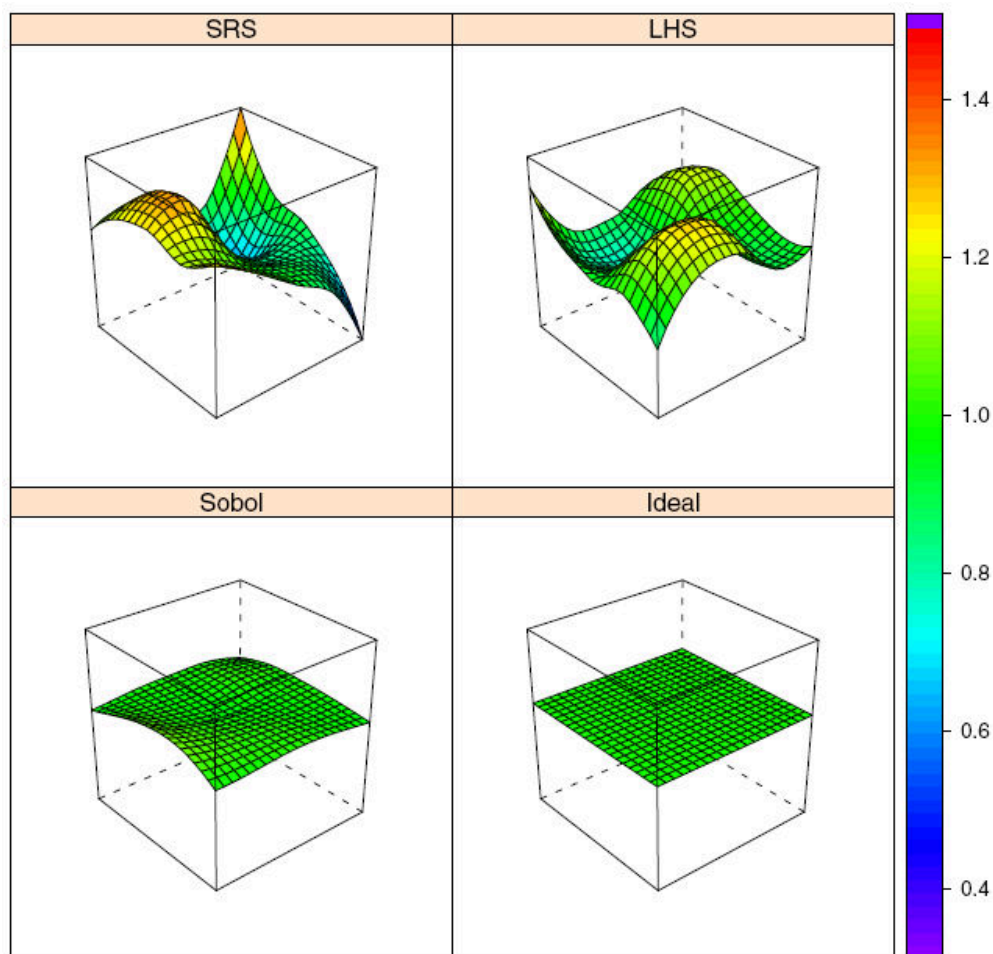
Figure 141 Space filling property of Sobol sampling



Smoothed Density Plots

While the qualitative behavior of the 1-D projections as histograms in [Figure 142](#) through [Figure 144](#) is clear, the quality of the 2-D projections is harder to visualize and the largest absolute correlation coefficient is not an intuitive measure. [Figure 142](#) shows the smoothed density plots for 128 samples. The samples are from a uniform density and the ideal plot is the one shown in the bottom right subplot. The traditional Monte Carlo, SRS, is shown in the top left subplot and departs from the ideal behavior for small sample sizes. The behavior of the LHS density depends on the specific random number generator seeds. The density for the Sobol sequence is close to the ideal and the property generally holds true for low-discrepancy sequences.

Figure 142 Comparison of smoothed density plots $n = 128$



Samples from Multivariate Normal Densities

Results from similar experiments are shown for samples from multivariate normal densities in four dimensions for SRS, LHS, and Sobol sequences in [Figure 143](#) through [Figure 145](#). The diagonal subplots show the 1-D histograms for each variable together with the ideal shaded density. The off-diagonal subplots show two-dimensional densities as topographic plots with color scale from deep blue to red with red indicating larger values. Both LHS and Sobol samples match the 1-D ideal density and are better than SRS. The largest absolute correlation coefficient is smaller for Sobol samples. However, the largest absolute correlation coefficient for LHS is more than that for SRS in this example.

Figure 143 Sampling: SRS Distribution: normal n ; 1024 maxCor; 3.8 percent

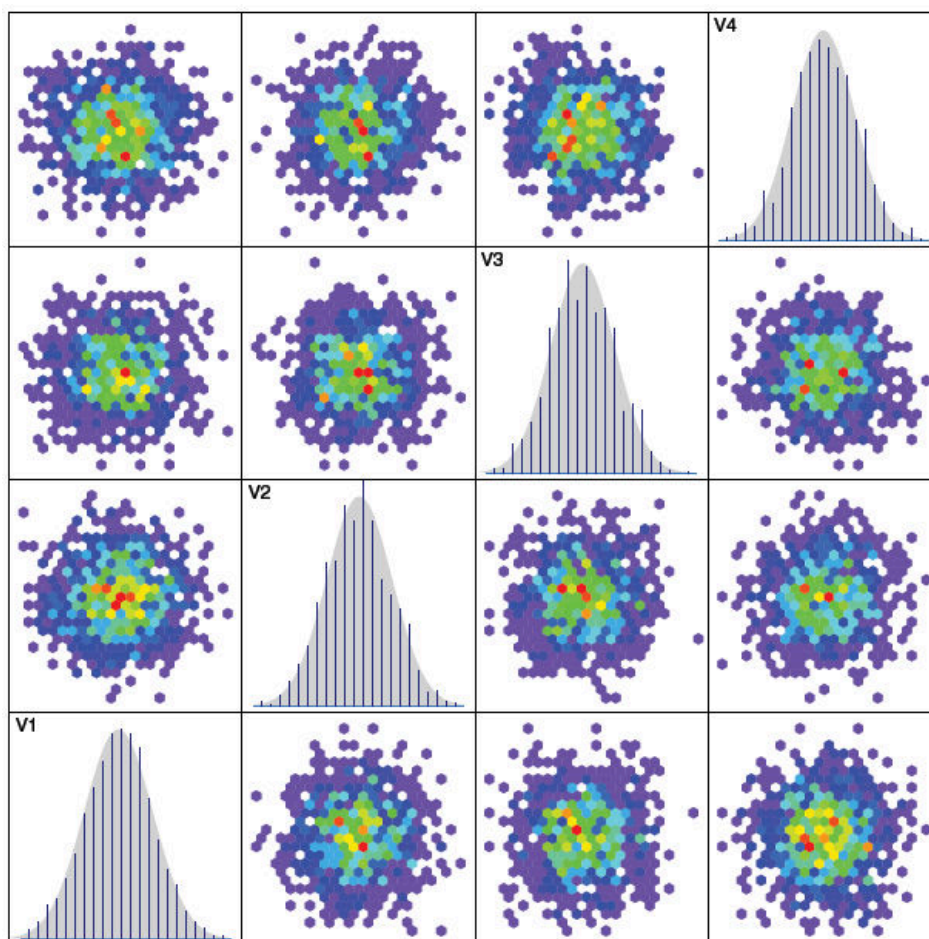


Figure 144 Sampling: LHS Distribution: normal n ; 1024 maxCor; 7.2 percent

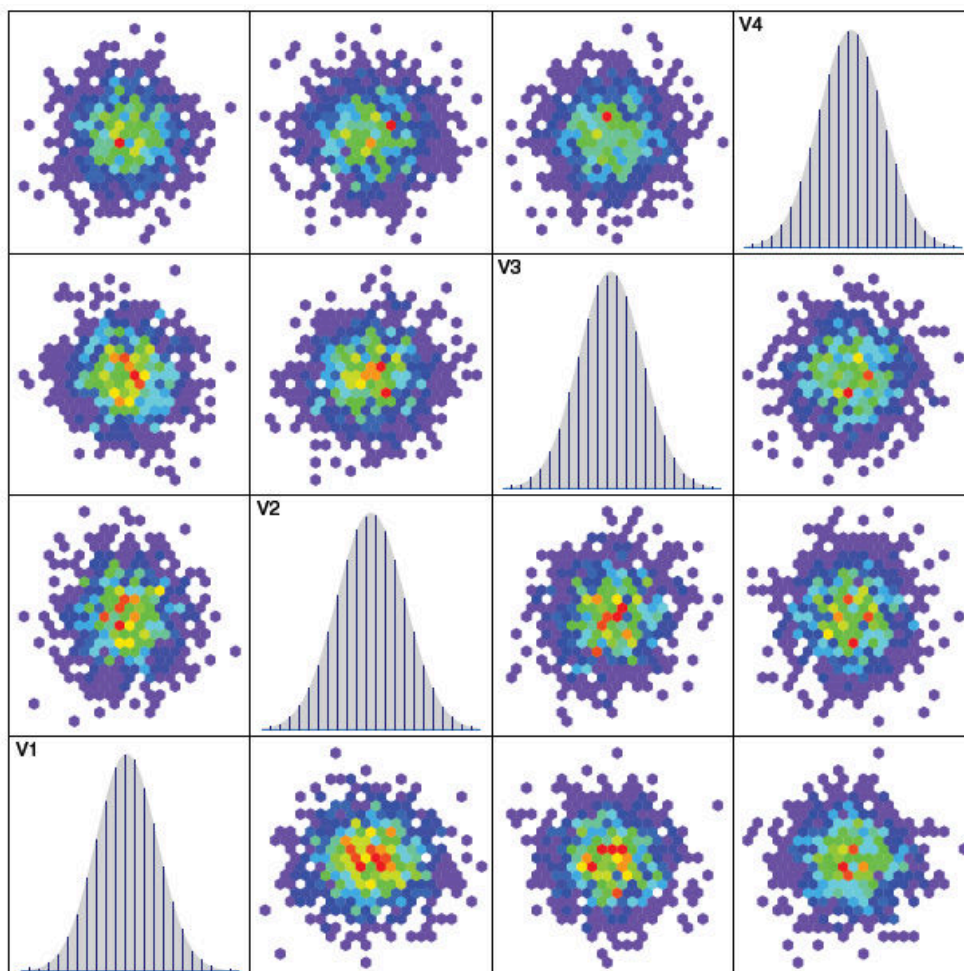
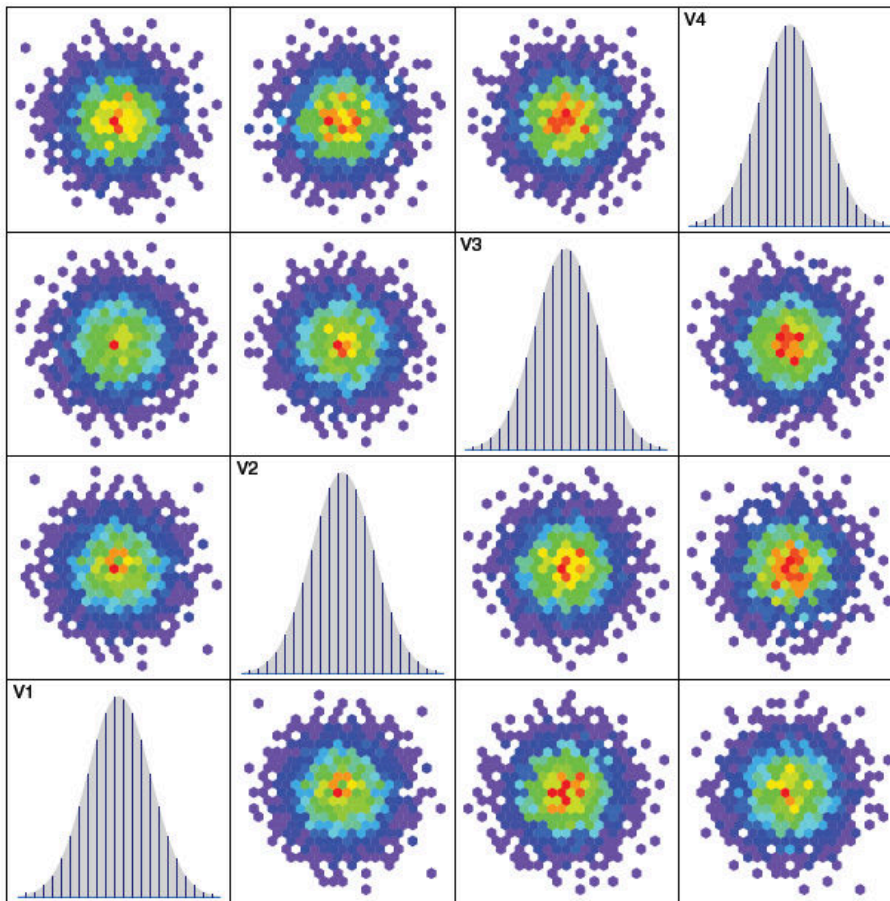
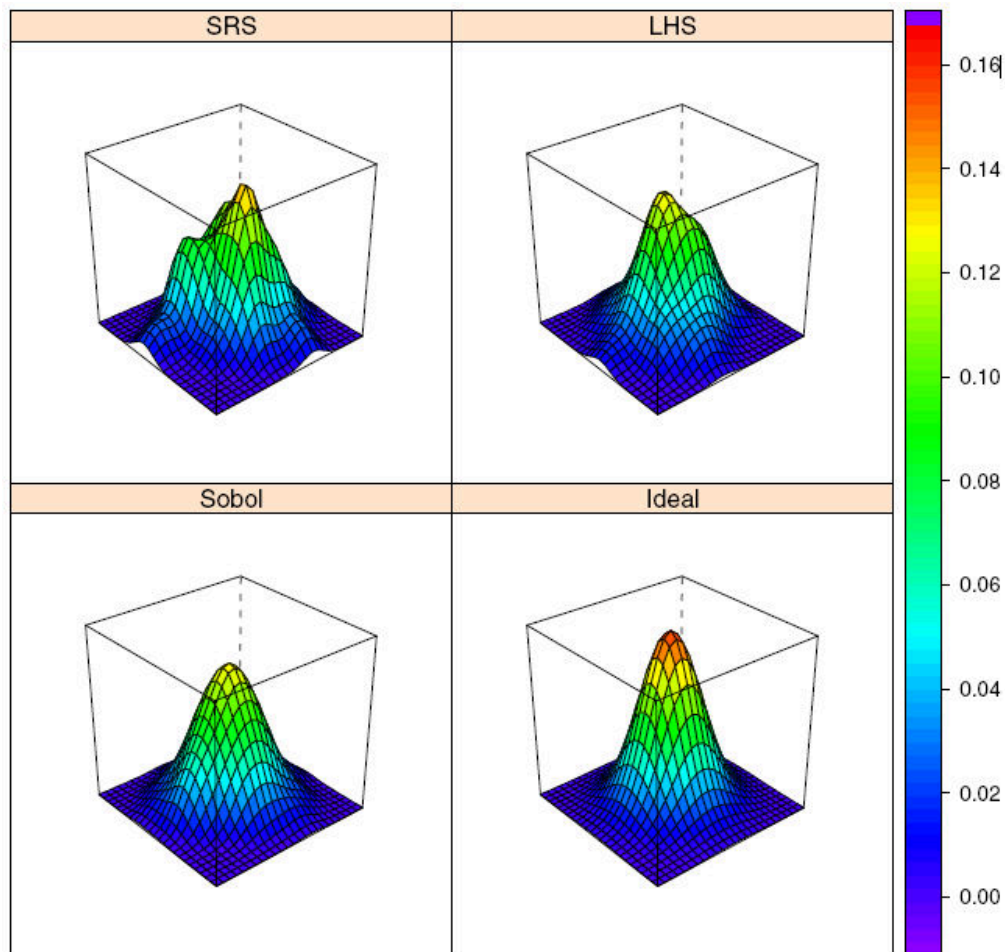


Figure 145 Sampling: Sobol Distribution: normal n ; 1024 maxCor; 0.9 percent



A better view of the 2-D projections can be seen in the smoothed density plots shown in [Figure 146](#) for 64 samples. As with uniform densities, we see that LHS looks better than SRS and that the Sobol sample is much closer to the ideal density.

Figure 146 Comparing sampling methods, smoothed density plots: $n = 64$



Gaussian Style Random Variable Definition

PrimeSim HSPICE permits independent random variables (IRV) to be defined by one of four probability distribution functions: GAUSS (relative normal distribution), AGAUSS (absolute normal distribution), UNIF (relative uniform distribution), AUNIF (absolute uniform distribution) with the following syntax:

```
.PARAM randpar1=GAUSS(nominal_val, rel_variation, sigma)
.PARAM randpar2=AGAUSS(nominal_val, abs_variation, sigma)
.PARAM randpar3=UNIF(nominal_val, rel_variation)
.PARAM randpar4=AUNIF(nominal_val, abs_variation)
```

Here, the default for sigma is 3, and the equivalent mathematical description is:

```
randpar1=N(nominal_val, nominal_val*rel_variation/sigma)
randpar2=N(nominal_val, abs_variation/sigma)
randpar3=U(nominal_val*(1-rel_variation), nominal_val*(1+rel_variation))
randpar4=U(nominal_val-abs_variation, nominal_val+abs_variation)
```

where $N(a, b) = a + b \cdot N(0, 1)$; $U(a, b) = a + (b - a) \cdot U(0, 1)$, $N(0, 1)$ and $U(0, 1)$ are the standard, normal, and uniform distributions, respectively.

Before each Monte Carlo trial, each IRV is assigned a different random number according to the corresponding probability density.

Example 1

```
.param randpar1=GAUSS(0, 1, 1) par2='0.1*randpar1' par3=par2
.model SYNOP_NMOS nmos vth0='0.1 + randpar1'
+ M1 d g s b SYNOP_NMOS w='randpar1' l='randpar1'
+ M2 d g s b SYNOP_NMOS w='par2' l='par3'
```

In Example 1, before each Monte Carlo trial, the IRV `randpar1` gets four different numbers from the random number stream and uses them to create `par2`, `vth0`, and M1's width and M1's length, respectively. As `par3` is equal to `par2`, M1's `vth0` is equal to M2's `vth0`, M2's `w` is equal to M2's `l`, but M1's `w` is different from M1's `l`. Thus direct use of `randpar1` represents a different random number while indirect use through `par2` and `par3` leads to common random numbers. This was an early attempt to model local and global variations in instance parameters. However, only global variation was possible on model parameters.

Example 2

For traditional Monte Carlo, the `modmonte=1` option enables local variation on model parameters, so that different instances with the same model can get different random numbers in a single Monte Carlo trial. In the following example, before each Monte Carlo trial, the IRV `randpar1` will get five different random numbers from the random number stream and allocate them to `par2`, M1's `vth0` and M2's `vth0`, M1's `w` and M1's `l`. The only difference from Example 1 is that M1's `vth0` is different from M2's `vth0`.

```
.option modmonte=1
.param randpar1=Gauss(0, 1, 1) par2='0.1*randpar1' par3=par2
.model SYNOP_NMOS nmos vth0='0.1+ randpar1'
+ M1 d g s b SYNOP_NMOS w='randpar1' l='randpar1'
+ M2 d g s b SYNOP_NMOS w='par2' l='par3'
```

Input/Output With New Capability

There is no change to measurement and Monte Carlo commands or to the measurement output files and only minimal changes to the listing file with additional informational messages in cases of dual variability definitions. For example, the *.mc0 files contain

additional fields for random variables defined in the Gaussian style and new suffix keys IRV are introduced.

The content in a *.mc0 file is similar to the Variation Block: the option settings are reported first, followed by the names of all requested models/devices/subcircuits, with their respective parameter names. Separators are used as follows:

- A single dot is a hierarchical separator between a subcircuit and an instance or device name.
- The special characters “:@:” separate model/device/subcircuit and parameter names.

Syntax

- For normal parameters that use random variables directly, the syntax is:

```
RandomVariable:@:SubcircuitName.InstanceName:@:IRV
```

For example:

```
globw:@:globwidth:@:IRV , locwidth:@:x1.width:@:IRV
```

- For devices with models, the syntax is:

```
RandomVariable:@:ModelName:@:SubcircuitName.InstanceName :@:IRV
```

For example:

```
res_dev:@:x1.res_sub@x1.rab:@:IRV, res_dev@resistor:@:r2:@:IRV
```

- For model parameters, the syntax is:

```
RandomVariable:@:ModelName:@:SubcircuitName:@:IRV
```

For example:

```
parl4@nch_mac.7:@:xmdut4:@:IRV
```

In the above syntax and examples, IRV can be 'IGNC', 'IGUC', 'IGLC', 'ILNC', 'ILUC', or 'ILLC'.

Where:

- IGNC: .param a = agauss()/gauss(), global variation
- IGUC: .param a = aunif()/unif(), global variation
- IGLC: .param a = limit(), global variation
- ILNC: .param a = agauss()/gauss(), local variation
- ILUC: .param a = aunif()/unif(), local variation
- ILLC: .param a = limit(), local variation

Example 1: Variation Duplicated in Traditional Format and Variation Block

```
.options OPTS LIST NOPAGE INGOLD=2 ALT999 PROBE POST=1 TNOM=25
.options MODMONTE=1
.model nch nmos LEVEL=53 VTH0='-0.4+vth0_nch*0.5'
+ TOX='4E-9+4e-11*tox_nch' VERSION=3.2
.param vth0_nch = AGAUSS (0 , 1 , 1.0 )
.param tox_nch = AGAUSS (0 , 1 , 1.0 )
m11 2 11 0 0 nch W=1E-6 L=0.15E-6
m12 2 11 0 0 nch W=1E-6 L=0.15E-6
m13 2 11 0 0 nch W=1E-6 L=0.15E-6
v01 2 0 1.5
v02 11 0 0.0
.Variation
Option Use_AGAUSS_Format=yes
Option Sampling_Method=LHS
.Local_Variation
nmos nch vth0 = 0.5 //also define vth0 variation in VB
.End_Local_Variation
.End_Variation
.dc v02 0 2.0 0.1 sweep monte=2
.meas dc ids_11 find par('i(m11)*1E3') when v(11)=1.5
.meas dc ids_12 find par('i(m12)*1E3') when v(11)=1.5
.meas dc ids_13 find par('i(m13)*1E3') when v(11)=1.5
.end
```

Sample output in the *.lis file for this simulation is as follows:

```
*** monte carlo index =      3 ***

MODEL PARAMETER MONTE CARLO DEFINITIONS
0:m11
tox_nch
nch = 4.0722E-09
0:m12
tox_nch
nch = 3.9584E-09
0:m13
tox_nch
nch = 4.0462E-09

MONTE CARLO PARAMETER DEFINITIONS
vth0_nch
tox_nch
ids_11= 1.6030E+00
ids_12= 5.8869E-01
ids_13= 8.1399E-01
```

The *.mc0 file includes the following characteristics using the \$ sign:

- The header section records lead with a \$ sign (which are read by PrimeSim HSPICE as comments).
- Title lines include an extra leading "\$", that is, \$\$.
- The external sampling option is recorded with a leading "\$", such as: \$.Option
Sampling_Method = External_Block_Name = extern_data
- Block definitions are recorded with a leading "\$" such as, \$.Data Extern_Data
- Column names, random variable data (if requested), and the enddata record are also represented with a leading "\$".
- The *.mc file is written only in .OPTION MEASFORM = 1|3| format for ease in exporting to Microsoft Excel. The default is 1. (See .OPTION MEASFORM in the *PrimeSim Continuum Reference Manual: Commands and Control Options*.)

The *.mc0 file output for Example 1 shows the following results. Note that the independent random variable vth0_nch is missing due to the duplicated variation definition on vth0.nch.

```
$DATA1 SOURCE='HSPICE' VERSION='N-2017.12 linux64'
$OPTION MEASFORM=1
$ This file format is subject to change
$ The separator is :@:
$index=1 is a nominal sample in Monte Carlo.

$option ignore_global_variation = no
$option ignore_local_variation = no
$option ignore_spatial_variation = no
$option ignore_variation = no
$option sampling_method = lhs
$option normal_limit = 20.0000
$option replicates = 1
$option random_generator = MCG
$option seed = 1
$option stream = 1
$option output_sigma_value = 1
$option use_agauss_format = yes
$option screening_method = pearson
$Global Variable number: 0
$Local Variable number: 6
.TITLE '.options opts list nopage ingold=2 alt999 probe post=1 tnom=25'
index m11:@:nch:@:tox_nch:@:ILNC
m12:@:nch:@:tox_nch:@:ILNC
m13:@:nch:@:tox_nch:@:ILNC m11:@:nch:@:vth0:@:ILN
m12:@:nch:@:vth0:@:ILN m13:@:nch:@:vth0:@:ILN
1 0. 0. 0. 0. 0. 0.
2 -1.2496 0.2578 0.5165 -0.5459 1.9872 -0.2346
3 0.8061 -0.1867 -2.0966 2.9925 -0.3265 0.1278
```


Example 2: Subcircuit and Macro Models

The following example demonstrates the combined syntax used for subckt and macro models:

```
*four resistors
.param bias=1m
.param globw=AGAUSS(1u,0.1u,3)
.param globwidth=globw
.param locwidth= AGAUSS(0.1u,0.02u,3)
.param res_dev = AGAUSS(10,10,1)
.option modmonte=1 numdgt=8 ingold=2
i1 0 1 bias
i2 0 2 bias
i3 0 3 bias
i4 0 4 bias
i5 0 4 bias
i6 0 4 bias
X1 1 0 res1 width='globwidth+locwidth'
X2 2 0 res1 width='globwidth+locwidth'
X3 3 0 res1 width='globwidth+locwidth'
X4 4 0 res1 width='globwidth+locwidth'
r1 5 0 resistor w='3u+locwidth'
r2 6 0 resistor w='3u+locwidth'
.subckt t res1 a b
rab a b res_sub w=width
.model res_sub R w=3u l='3u+res_dev*0.1u' rsh='100+res_dev'
.ends res1
.model resistor R w=3u l='3u+res_dev*0.1u' rsh='100+res_dev'
.Variation
Option Use_AGAUSS_Format=yes
.End_Variation
.op
.dc bias 1m 1m 1m monte=4
.print v(1) v(2) v(3) v(4)
.measure dc v1 find v(1) at=1m
.measure dc v2 find v(2) at=1m
.measure dc v3 find v(3) at=1m
.measure dc v4 find v(4) at=1m
.option measdgt=6
.end
```

The following sample is returned in the *.lis file for this statement:

```
MONTE CARLO DEFINITIONS
Option sampling_method was set, variation block and agauss format
variation are in effect.
Random number generator is MCG , and seed = 1, stream =
1
***** option summary *****
runlvl = 3 bypass = 2.00000000
```

```

*** monte carlo index = 1 ***
*four resistors
***** dc transfer curves tnom= 25.000 temp= 25.000 *****
*** monte carlo index = 1 ***
x
bias          voltage          voltage          voltage          voltage
              1                2                3                4
1.00000000e-03 4.0000000e-01 4.0000000e-01 4.0000000e-01
1.2000000e+00
y
v1= 4.000000e-01
v2= 4.000000e-01
v3= 4.000000e-01
v4= 1.200000e+00
*** monte carlo index = 2 ***
*** monte carlo index = 2 ***
x
bias          voltage          voltage          voltage          voltage
              1                2                3                4
1.00000000e-03 2.6669477e-01 4.6436176e-01 5.2809476e-01
1.1680637e+00
y
v1= 2.666948e-01
v2= 4.643618e-01
v3= 5.280948e-01
v4= 1.168064e+00
*** monte carlo index = 3 ***
*** monte carlo index = 3 ***
x
bias          voltage          voltage          voltage          voltage
              1                2                3                4
1.00000000e-03 2.1978322e-01 4.8798498e-01 2.8746940e-01
4.0459125e-01
y
v1= 2.197832e-01
v2= 4.879850e-01
v3= 2.874694e-01
v4= 4.045913e-01
*** monte carlo index = 4 ***
*** monte carlo index = 4 ***
x
bias          voltage          voltage          voltage          voltage
              1                2                3                4
1.00000000e-03 3.8207674e-01 2.9834458e-01 4.6294228e-01
1.5837948e+00
y
v1= 3.820767e-01
v2= 2.983446e-01
v3= 4.629423e-01
v4= 1.583795e+00
*** monte carlo index = 1 ***
meas_variable = v1
mean = 2.8951824e-01 varian = 6.9754803e-03

```

```

sigma = 8.3519341e-02  avgdev = 9.2558472e-02
max   = 3.8207674e-01  min    = 2.1978322e-01
  1-sigma = 8.3519341e-02 median = 2.6669480e-01
meas_variable = v2
mean   = 4.1689711e-01  varian = 1.0680541e-02
sigma = 1.0334670e-01  avgdev = 1.1855255e-01
max    = 4.8798498e-01  min    = 2.9834458e-01
  1-sigma = 1.0334670e-01 median = 4.6436180e-01
meas_variable = v3
mean   = 4.2616881e-01  varian = 1.5489356e-02
sigma = 1.2445624e-01  avgdev = 1.3869944e-01
max    = 5.2809476e-01  min    = 2.8746940e-01
  1-sigma = 1.2445624e-01 median = 4.6294230e-01
meas_variable = v4
mean   = 1.0521499e+00  varian = 3.5770726e-01
sigma = 5.9808633e-01  avgdev = 6.4755864e-01
max    = 1.5837948e+00  min    = 4.0459125e-01
  1-sigma = 5.9808633e-01 median = 1.1680637e+00
*****
*four resistors
***** operating point information tnom= 25.000 temp= 25.000 *****
***** operating point status is all      simulation time is
0.0000000e+00
  node    =voltage      node    =voltage      node    =voltage
+0:1      = 4.0000000e-01 0:2      = 4.0000000e-01 0:3      =
4.0000000e-01
+0:4      = 1.2000000e+00 0:5      = 0.0000000e+00 0:6      =
0.0000000e+00
**** current sources
subckt
element    0:i1      0:i2      0:i3      0:i4
volts     -4.0000000e-01 -4.0000000e-01 -4.0000000e-01 -1.2000000e+00
current    1.0000000e-03  1.0000000e-03  1.0000000e-03  1.0000000e-03
power      4.0000000e-04  4.0000000e-04  4.0000000e-04  1.2000000e-03
subckt
element    0:i5      0:i6
volts     -1.2000000e+00 -1.2000000e+00
current    1.0000000e-03  1.0000000e-03
power      1.2000000e-03  1.2000000e-03
  total current source power dissipation= 4.8000000e-03  watts
**** resistors
subckt
element    0:r1      0:r2      x1      x2
          1:rab      2:rab
r value    1.4193548e+02 1.4193548e+02 4.0000000e+02 4.0000000e+02
v drop     0.0000000e+00 0.0000000e+00 4.0000000e-01 4.0000000e-01
current     0.0000000e+00 0.0000000e+00 1.0000000e-03 1.0000000e-03
power       0.0000000e+00 0.0000000e+00 4.0000000e-04 4.0000000e-04
subckt     x3      x4
element    3:rab      4:rab
r value    4.0000000e+02 4.0000000e+02
v drop     4.0000000e-01 1.2000000e+00
current     1.0000000e-03 3.0000000e-03

```

```
power      4.0000000e-04  3.6000000e-03
***** job concluded *****
```

The following sample output is returned for the subcircuit/macro file example:

```
$DATA1 SOURCE='HSPICE' VERSION='N-2017.12-SP1-2 linux64'
$OPTION MEASFORM=1
$ This file format is subject to change
$ The seperator is :@:
$index=1 is a nominal sample in Monte Carlo.
$option ignore_global_variation = no
$option ignore_local_variation = no
$option ignore_spatial_variation = no
$option ignore_variation = no
$option sampling_method = srs
$option normal_limit = 2.0000000e+01
$option replicates = 1
$option random_generator = MCG
$option seed = 1
$option stream = 1
$option output_sigma_value = 1
$option use_agauss_format = yes
$option screening_method = pearson
$Global Variable number: 1
$Local Variable number: 18
.TITLE '*four resistors'

index globwidth:@:globw:@:IGNC r1:@:resistor:@:locwidth:@:ILNC
r2:@:resistor:@:locwidth:@:ILNC x1.width:@:res1:@:locwidth:@:ILNC
x2.width:@:res1:@:locwidth:@:ILNC x3.width:@:res1:@:locwidth:@:ILNC
x4.width:@:res1:@:locwidth:@:ILNC r1:@:resistor:@:res_dev:@:ILNC
r1:@:resistor:@:res_dev:@:ILNC r2:@:resistor:@:res_dev:@:ILNC
r2:@:resistor:@:res_dev:@:ILNC x1.rab:@:x1.res_sub:@:res_dev:@:ILNC
x1.rab:@:x1.res_sub:@:res_dev:@:ILNC
x2.rab:@:x2.res_sub:@:res_dev:@:ILNC
x2.rab:@:x2.res_sub:@:res_dev:@:ILNC
x3.rab:@:x3.res_sub:@:res_dev:@:ILNC
x3.rab:@:x3.res_sub:@:res_dev:@:ILNC
x4.rab:@:x4.res_sub:@:res_dev:@:ILNC
x4.rab:@:x4.res_sub:@:res_dev:@:ILNC

1  0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00
   0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00
   0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00
   0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00

2  -8.01418e-01 -8.29439e-01 -2.67651e-01  2.14974e-01  1.67567e+00
   8.94634e-01  2.01353e-01  1.04474e+00 -1.79870e+00  2.77419e+00
   6.55982e-01 -1.27190e+00 -4.93695e-01  4.48222e-01  3.20939e-01
   1.43785e+00 -5.18916e-01 -3.77951e-01  5.51789e-01

3  3.41452e-01  2.89983e-01 -1.84116e-01  8.48533e-01 -7.44553e-01
  -1.24384e+00  6.69724e-01  6.13081e-01 -1.19110e+00  3.82410e-01
```

```

9.69429e-01 -1.88489e+00 6.07289e-01 8.01334e-01 2.45112e-01
-1.32088e+00 8.36183e-01 -2.45022e+00 -1.28971e+00

4 4.52651e-01 7.24025e-01 -7.22316e-01 -2.17230e-01 -5.20381e-01
-1.57569e+00 1.49143e+00 1.84476e+00 -2.50903e-01 1.72629e+00
-6.47077e-01 4.23289e-01 -1.38055e+00 -6.95236e-01 -9.64623e-01
3.82951e-01 6.66992e-01 1.60989e+00 -4.12652e-01

```

Application Considerations

Due to the combinations of variation specified in the variation block and the value of `Normal_Limit`, variations that are applied may be too large. This can make some circuits display abnormal behavior and produce unrealistic results for certain samples. This, in turn, can distort the summary statistics reported by PrimeSim HSPICE at the end of the Monte Carlo simulation.

Output Files for Monte Carlo Simulations

Running Monte Carlo simulations is expensive and the raw outputs may not provide sufficient insight into the circuit behavior or guidance on how the circuit could be improved. Usually, designers import the data into external tools for graphical and analytical analyses.

For Monte Carlo simulations, the output files are automatically generated with the Variation Block style description. Use `.Option Sampling_Method=SRS` (or one of the other choices) with the traditional AGAUSS style.

For example cases and related `*.mcp0` demo files, see Variability Examples in the *PrimeSim HSPICE User Guide: Demonstration Netlists*.

The outputs created are listed in [Table 71](#). These files are created for releases before S-2021.09. [Table 67](#) lists the Monte Carlo output files starting from S-2021.09. These new outputs formats are supported for general Monte Carlo runs, distributed Monte Carlo runs and Standalone post processing Utility.

Note:

For Large Scale Monte Carlo, data block sweeps or Alters, the outputs remain same as [Table 71](#).

PrimeSim HSPICE generates different output files for different Monte Carlo simulations. See the following table.

Table 71 Unification of MC Output Files

MC Output	AGAUSS-style Variations (Traditional)	AGAUSS-style Variations With .option sampling_method	Variation Block (VB) Style Variations	Large Scale Monte Carlo (VB or AGAUSS)
.mpp#	Yes	Yes	Yes	Yes
.m?#	Yes	Yes	Yes	No
.tail#	Yes	Yes	Yes	Yes
.hm?#.csv	Yes	Yes	Yes	Yes
.qq?#.csv	Yes	Yes	Yes	Yes
.annotate	No	Yes	Yes	Yes
.mc#	No	Yes	Yes	No
.montelist	No	No	No	Yes
.ic0	Yes	Yes	Yes	No
Waveform files (.tr0, .fsdb, .wdf, and so on)	Yes	Yes	Yes	No

Content in *.mpp0 File

MC Output	Single + Gaussian-type Variation	Single + VB-type Variation	DP + Gaussian-type Variation	DP + VB-type Variation	LSMC + DP
Output all MC-related option values	yes	yes	yes	yes	yes
Global/local variable numbers	no	yes	no	yes	yes
Sample moments table	yes	yes	yes	yes	yes
Quantile table	yes	yes	yes	yes	yes
User-specified quantile table	yes	yes	yes	yes	yes

MC Output	Single + Gaussian-type Variation	Single + VB-type Variation	DP + Gaussian-type Variation	DP + VB-type Variation	LSMC + DP
Extremes and corners	yes	yes	yes	yes	yes
Response correlation matrix	yes	yes	yes	yes	yes
Selective simulation	yes	yes	yes	yes	yes
Variables screened by importance	no	yes	no	yes	yes
Important elements or models	no	yes	no	yes	yes
Instance parameters for elements in match groups	no	yes	no	yes	yes
Subcircuit encoding	no	yes	no	yes	yes

In the *.mpp0 file, all sections have a start and end marker '##'. There are no '\$' signs at the beginning of options.

Table 72 MC Output Files Using PrimeSim AVA Based Post-Processing

MC Output Files Starting S-2021.09 Version	MC Output Files prior to R-2020.12	Large Scale Monte Carlo - All Releases	File Description
.ava.report	.mpp#	.mpp#	Statistic reports : Moments, quantiles, sensitivity
.ava.qq?#.csv	.qq?#.csv	.qq?#.csv	Quantile data and plots in CSV format
.ava.qq?#.pdf	.qq?#.pdf	.qq?#.pdf	Quantile data and plots in PDF format
.ava.qq?#.wv_plt	.qq?#.wv_plt	.qq?#.wv_plt	Quantile data and plots in WaveView format

Table 72 MC Output Files Using PrimeSim AVA Based Post-Processing (Continued)

MC Output Files Starting S-2021.09 Version	MC Output Files prior to R-2020.12	Large Scale Monte Carlo - All Releases	File Description
.ava.log	.lis	.lis	Run logs including memory, time elapsed and so on.
.ava.annotate	.annotate	.annotate	Simulator output for interface
.ava.sens.csv	NA	NA	Sensitivity report in CSV format.
.ava.irv_corr.csv	.irv_corr.csv	.irv_corr.csv	Aggregated sensitivity report

Post-Processing of Monte Carlo Results

The PrimeSim HSPICE data mining report, available in `*.mpp#/.ava.report` and `*.annotate/.ava.annotate` files, provides extensive data including statistical summary, correlated contribution of IRVs on elements, design corner samples, and design environment back-annotation information.

PrimeSim HSPICE does post-processing on `*.ms0` (`*.ma0`, `*.mt0`) and `*.mc0` files, and generates output tables in a file with the suffix `*.mpp#/.ava.report`. After echoing the options used for the simulation, the output is displayed in a set of tables which are grouped into two sections: a summary statistics section and a variable screening section.

Starting from S-2021.09, the following files are created:

- `.ava.report`
- `.ava.details.json`
- `.ava.sens.csv`
- `.ava.irv_corr.csv`
- `.ava..annotate`

The [Table 72](#) shows the mapping of the files. Though the file extensions have changed, the content of the report remains constant and are listed appropriately to both versions throughout following sections in this chapter.

The following sections discuss these topics:

- [Summary Statistics](#)
- [Variable Screening](#)
- [Corner Analysis - DC Monte Carlo/Transient Analysis](#)
- [Back-Annotation — *.annotate\(.ava.annotate\) File](#)

Summary Statistics

The Summary Statistics section of the `*.mpp0/.ava.report` file lists the number of sample, number of failures and a listing of responses with such a minor variation that they are dropped from further analysis. See the top section in [Figure 147 on page 599](#).

Measure Failures

PrimeSim HSPICE categorizes the measure failures for Monte Carlo simulation into three types in the `*.mpp0/.ava.report` file:

1. Simulation is successful, but there are measure failures because some measures cannot reach the target and are marked as *failed* for some samples.
2. Measure failure due to job abortions during simulation.
3. In the DP MC mode, the simulation is not run by a worker due to resource (machine, disk, or network) issue.

See the following example for a summary statistics and simulation failures section in a `*.mpp0/.ava.report` file.

```
##Summary_Statistics
#-----
Total number of independent random variables = 1
Number of independent global random variables = 0
Number of independent local random variables = 1
Number of Samples: 1000
Number of Meas Failure: 1
Number of Job Abortion: 7
Number of Missed Samples: 400
....
##end_of_Sumary_Statistics
..
..
##Simulate_Failures
#-----
Measure Failure: ... Monte = list(4)
Job Aborted: ... Monte = list(38, 234, 253, 462, 475, 626, 879)
Missed Samples ... Monte = list(201:400, 801:1000)
##end_of_Simulate_Failures
```

The subsections consist of:

- [Sample Moments and Robust Statistics](#)
- [Sample Quantiles and Quartiles](#)
- [Extremes and Corners](#)

Note:

The `*.mpp0/.ava.report` file is generated with only a statistical summary in the case of the traditional flow (Monte Carlo simulation without the `.OPTION SAMPLING_METHOD`).

A sample summary statistics section is shown in [Figure 147 on page 599](#) and [Figure 148 on page 600](#).

Figure 147 Summary statistics section of *.mpp0/.ava.report file

```
##Summary_Statistics
#-----
Total number of independent random variables = 24
Number of independent global random variables = 4
Number of independent local random variables = 20
Number of Samples: 1000
Number of Failures: 0
##end_of_Summary_Statistics

##Sample_Moments
#-----
#CI: 95% Bootstrap confidence intervals based on 500 re-samples

```

	CI	mean	median	stdDev	mad	skewness	kurtosis
systoffset1	2.5%	2.126e-03	2.065e-03	9.931e-04	9.740e-04	-8.736e-02	2.625e+00
	97.5%	2.190e-03	2.182e-03	1.039e-03	1.065e-03	3.326e-02	2.867e+00
systoffset2	2.5%	1.588e-03	1.570e-03	5.127e-04	4.889e-04	-7.882e-02	2.656e+00
	97.5%	1.621e-03	1.616e-03	5.367e-04	5.345e-04	4.464e-02	2.930e+00
systoffset3	2.5%	1.240e-03	1.230e-03	3.496e-04	3.273e-04	-3.129e-02	2.730e+00
	97.5%	1.263e-03	1.256e-03	3.661e-04	3.551e-04	9.558e-02	3.018e+00
systoffset4	2.5%	1.047e-03	1.036e-03	2.697e-04	2.495e-04	3.587e-02	2.853e+00
	97.5%	1.064e-03	1.059e-03	2.826e-04	2.771e-04	1.834e-01	3.226e+00

```
##end_of_Sample_Moments

##Quantiles
#-----
#(Percentile table adds appropriate percentile points as the number of samples increases.)
#Percentile_Value Equivalent_Sigma_Value
#(Q15.86552539, Q84.134473607) (-1, 1)
#(Q2.275013195, Q97.724986805) (-2, 2)
#(Q0.134989803, Q99.865010197) (-3, 3)
#(Q3.167124183e-3, Q99.996832876) (-4, 4)
#(Q2.866515719e-5, Q99.999971335) (-5, 5)
#(Q9.865876450e-8, Q99.999999901) (-6, 6)

```

	CI	Q1.000000e-02	Q1.000000e-01	Q1.349898e-01	Q1	Q2.275013195	Q5
systoffset1	2.5%	-5.981000e-04	-5.981000e-04	-5.981000e-04	-3.960090e-04	-1.424637e-04	3.793369e-04
	97.5%	-5.946135e-04	-5.632349e-04	-5.608996e-04	-2.586330e-04	9.556030e-05	5.446650e-04

```


```

Figure 148 Summary statistics section of *.mpp0/.ava.report file (continued)

systoffset2	2.5%	1.684000e-04	1.684000e-04	1.684000e-04	2.872100e-04	4.208751e-04
		1.698286e-04	1.826857e-04	1.853141e-04	3.337990e-04	5.261912e-04
	97.5%	2.119626e-04	2.871993e-04	2.898095e-04	4.871380e-04	6.745466e-04
systoffset3	2.5%	2.686000e-04	2.686000e-04	2.686000e-04	3.620720e-04	4.614493e-04
		2.713872e-04	2.964721e-04	2.993232e-04	3.970670e-04	5.267925e-04
	97.5%	3.264463e-04	3.620387e-04	3.631108e-04	4.980870e-04	6.184642e-04
systoffset4	2.5%	3.014000e-04	3.014000e-04	3.014000e-04	3.782550e-04	4.416059e-04
		3.043670e-04	3.310703e-04	3.345158e-04	4.096670e-04	5.091456e-04
	97.5%	3.680394e-04	3.782579e-04	3.789971e-04	4.807840e-04	5.704189e-04
systoffset1	2.5%	Q1.586553e+01	Q25	Q75	Q84.134473607	Q95
		1.030471e-03	1.394769e-03	2.812963e-03	3.144503e-03	3.783880e-03
	97.5%	1.148463e-03	1.472750e-03	2.908500e-03	3.255524e-03	3.877450e-03
systoffset2	2.5%	1.023471e-03	1.208937e-03	1.936750e-03	2.117524e-03	2.442350e-03
		1.081993e-03	1.268500e-03	1.980250e-03	2.167000e-03	2.491000e-03
	97.5%	1.137487e-03	1.306262e-03	2.028775e-03	2.224768e-03	2.563376e-03
systoffset3	2.5%	8.528622e-04	9.804106e-04	1.472500e-03	1.596479e-03	1.821419e-03
		8.967435e-04	1.020750e-03	1.502250e-03	1.637503e-03	1.852200e-03
	97.5%	9.279442e-04	1.052500e-03	1.537775e-03	1.671032e-03	1.919152e-03
systoffset4	2.5%	7.537330e-04	8.533625e-04	1.221750e-03	1.318739e-03	1.491350e-03
		7.833844e-04	8.759000e-04	1.247500e-03	1.349503e-03	1.528100e-03
	97.5%	8.056986e-04	9.053831e-04	1.271062e-03	1.369529e-03	1.570150e-03
systoffset1	2.5%	Q99	Q99.865010197	Q99.9	Q99.99	
		4.322330e-03	4.847207e-03	4.957039e-03	5.015507e-03	
	97.5%	4.491720e-03	5.263054e-03	5.384322e-03	5.673832e-03	
systoffset2	2.5%	2.699300e-03	3.015967e-03	3.032004e-03	3.082805e-03	
		2.856190e-03	3.321502e-03	3.394006e-03	3.399401e-03	
	97.5%	3.019170e-03	3.397909e-03	3.394006e-03	3.399401e-03	
systoffset3	2.5%	2.004190e-03	2.246071e-03	2.275003e-03	2.306803e-03	
		2.184010e-03	2.439303e-03	2.440075e-03	2.507508e-03	
	97.5%	2.249290e-03	2.488859e-03	2.440075e-03	2.507508e-03	
systoffset4	2.5%	1.658210e-03	1.880589e-03	1.890042e-03	1.941001e-03	
		1.754210e-03	2.055029e-03	2.062059e-03	2.115106e-03	
	97.5%	1.888440e-03	2.100436e-03	2.062059e-03	2.115106e-03	
##end_of_Quantiles						
##Extremes_and_Corners						
#-----#						
	min	max	minIndex	maxIndex		
systoffset1	-5.981e-04	5.706e-03	648	969		
systoffset2	1.684e-04	3.400e-03	34	482		
systoffset3	2.686e-04	2.515e-03	34	482		
systoffset4	3.014e-04	2.121e-03	34	393		
##end_of_Extremes_and_Corners						

Sample Moments and Robust Statistics

The section called Sample Moments and Robust Statistics includes the sample mean, standard deviation, skewness, and kurtosis (Figure 150 on page 602).

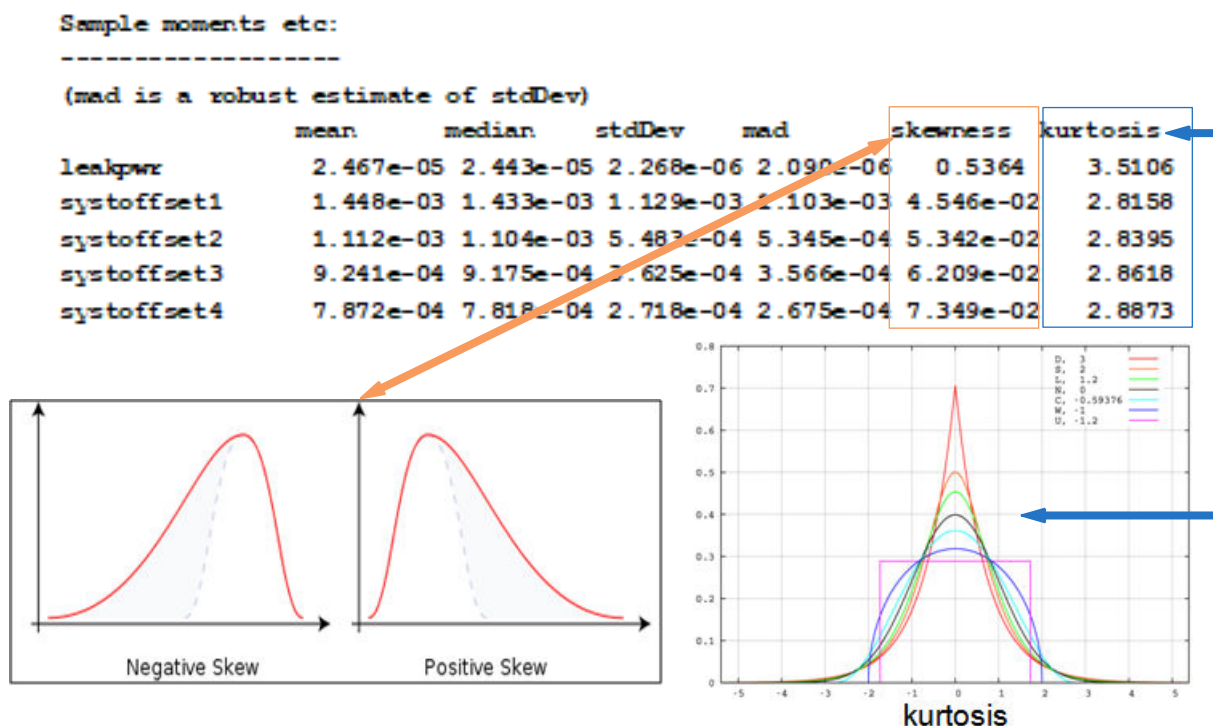
The skewness and kurtosis are the third and fourth central moments of the data. Skewness is zero for symmetric distributions (like the normal distribution) and the normal distribution has a kurtosis of three. Significant departures from (0, 3) for the skewness and kurtosis indicate non-symmetry and departure from normality for the distribution.

The median and MAD statistics are also reported in this section. These are the robust estimates of the mean and standard deviation, respectively. Unlike the `mean` and `standard deviation`, the `median` and `mad` are insensitive to outliers in the data. For more information on MAD, See https://en.wikipedia.org/wiki/Median_absolute_deviation.

Figure 149 Sample moments section of the .ava.report file

```
##Sample_Moments
#-----
Nominal    log_pwr  offset1  offset2  offset3  offset4  pwr
Mean       3.195    1.4      1.086    0.9047   0.7709   24.41
StdDev     3.202    1.359    1.055    0.8828   0.7548   24.73
Skewness   0.1044    38.27    18.5     12.2     9.108    2.614
Kurtosis   0.2061  0.002936 0.000401 -0.0002697 -0.001364 0.5471
Kurtosis   3.127    2.994    2.992    2.993    2.993    3.65
```

Figure 150 Sample moments section of the *.mpp0 file



Note:

From the 2015.06-SP1 version, nominal results are excluded from Monte Carlo data mining. This change is applicable to SRS and LHS sampling methods.

For the low discrepancy sampling methods (Sobol and Niederreiter), there is no change, as the sample is part of the number sequence. There is also no change for OFAT and Factorial as the samples are not random. The same is the case with external sampling which is under user control.

The traditional Monte Carlo (.option random_generator=3lc) does not run the nominal simulation. Hence this change does not affect traditional Monte Carlo analysis.

Confidence Interval Levels

(F-2011.09-SP1) The Sample Moments section of the *.mpp0 file includes the confidence interval (CI) percentage on the top line of the section and as the first column in the Summary Statistics portion of the file.

A confidence interval is an interval in which a measurement or trial falls corresponding to a given probability. *Bootstrapping* is a computer-based method for assigning measures of accuracy to sample estimates. The process is as follows:

After running Monte Carlo with 20 samples, PrimeSim HSPICE produces a measurement output variable 'systoffset1' with a set of values:

```
systoffset1 =
1.400e-03  1.254e-03  7.500e-04  4.476e-03  2.584e-03  9.569e-04
6.650e-04  -1.410e-04  3.142e-03  1.753e-03  -1.695e-04  7.377e-04
3.634e-03  -6.850e-04  1.632e-03  1.690e-03  6.391e-04  -4.123e-04
2.571e-03  2.506e-03
```

The sample moments based on above samples are:

	mean	median	stdDev	mad	skewness
kurtosis systoffset1	1.449e-03	1.327e-03	1.383e-03	1.384e-03	4.142e-01
	2.397e+00				

To calculate the confidence intervals of above moments, PrimeSim HSPICE resamples `systoffset1`'s 20 results 500 times.

For each resample, PrimeSim HSPICE calculates the sample:

- Mean
- Median
- stdDev
- MAD
- Skewness
- Kurtosis

Generate a histogram of these 500 moments, then pick out the 2.5th and 97.5th percentiles. (This is the same as sorting the 500 moments and selecting the 12th~13th avg and 488th sorted moments. These percentiles do actually estimate the percentiles of the true distribution of moments.)

Take mean value for example: after the bootstrapping resampling method, the 95% confidence interval of the mean is (9.060e-04, 2.041e-03), which means that you can be fairly confident that the true `systoffset1` mean value is between (9.060e-04, 2.041e-03).

Current limitations:

- Resampling size is a fixed value (500 samples).
- Confidence interval value is a fixed value (95%).

See [Figure 151 on page 604](#) for listing of Confidence intervals (CI) in Summary Statistics section of the *.mcp0 file.

Figure 151 Confidence intervals recorded in Summary Statistics

```
##Summary_Statistics
#-----
Total number of independent random variables = 20
Number of independent global random variables = 4
Number of independent local random variables = 16
Number of Samples: 1000
Number of Meas Failure: 0
Number of Job Abortion: 0
##sampling_method = SRS|LHS, the first MC sample is nominal value and is excluded in Monte Carlo data mining.
##end_of_Sumary_Statistics

##Sample_Moments
#-----
#CI: 95% Bootstrap confidence intervals based on 500 re-samples
      CI      mean      median      stdDev      mad      skewness      kurtosis
2.5%      2.557e-03  2.530e-03  1.059e-03  9.776e-04 -1.196e-03  2.802e+00
systoffset1      2.626e-03  2.600e-03  1.116e-03  1.055e-03  2.457e-01  3.837e+00
97.5%      2.695e-03  2.713e-03  1.170e-03  1.146e-03  5.821e-01  5.413e+00
##end_of_Sample_Moments

##Quantiles
#-----
#(Percentile table adds appropriate percentile points as the number of samples increases.)
#Percentile_Value      Equivalent_Sigma_Value
#(Q15.86552539, Q84.134473607)      (-1, 1)
#(Q2.275013195, Q97.724986805)      (-2, 2)
#(Q0.134989803, Q99.865010197)      (-3, 3)
#(Q3.167124183e-3, Q99.996832876)      (-4, 4)
#(Q2.866515719e-5, Q99.999971335)      (-5, 5)
#(Q9.865876450e-8, Q99.999999901)      (-6, 6)
      CI      Q1.000000e-02  Q1.000000e-01  Q1.349898e-01  Q1      Q2.275013195  Q5
2.5%      -6.063828e-04  -6.063828e-04  -6.063828e-04  -9.053460e-05  2.593649e-04  7.100242e-04
systoffset1      -6.019802e-04  -5.623572e-04  -5.305975e-04  9.402366e-05  4.573342e-04  8.375638e-04
97.5%      -3.629461e-04  -8.785455e-05  -8.250946e-05  3.903430e-04  6.737058e-04  9.406086e-04

##end_of_Quantiles
```

Sample Quantiles and Quartiles

The sample quantiles and quartiles are displayed next ([Figure 152](#) and [Figure 153 on page 606](#)) . These measures are also robust to outliers and provide better visibility into the distributional behavior in the tails. Quantiles are points taken at regular intervals

from the cumulative distribution function (CDF) of a random variable. They are found by counting the number of samples that fall to the left of the particular point in the empirical cumulative distribution function. Thus 25 percent of the samples fall below the Q25 quartile.

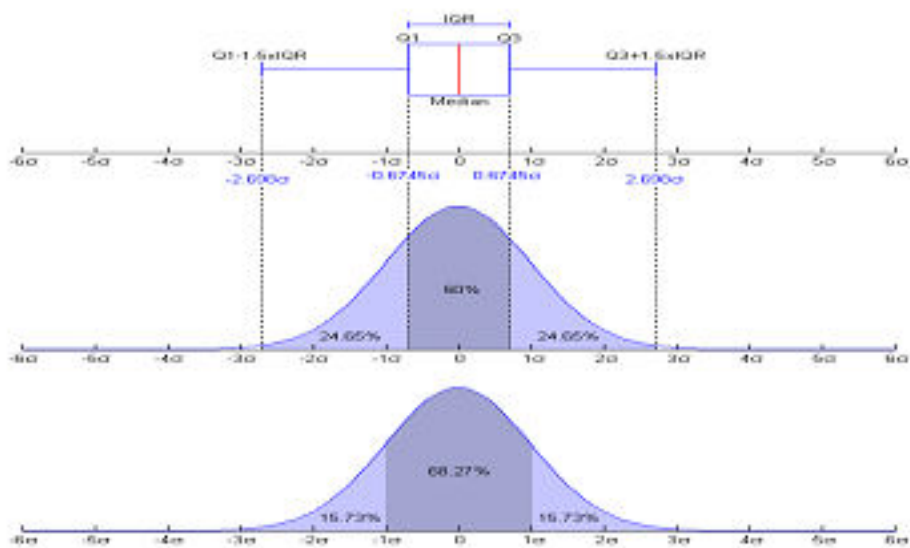
Figure 152 Quantiles section of the .ava.report file

```
##Quantiles
#-----
log_pwr offset1 offset2 offset3 offset4 pwr
Q1e-03 (-4.265) 2.793 -166.9 -80.22 -52.81 -39.36 16.42
Q3.17e-03 (-4) 2.818 -150.6 -72.82 -47.93 -35.7 16.81
Q1e-02 (-3.719) 2.846 -142.6 -68.83 -45.23 -33.67 17.2
Q1e-01 (-3.090) 2.906 -116.9 -56.1 -36.83 -27.39 18.31
Q1.35e-01 (-3) 2.914 -113.1 -54.4 -35.71 -26.55 18.48
Q1 (-2.326) 2.974 -87.32 -41.78 -27.38 -20.34 19.57
Q2.28 (-2) 3.004 -75.1 -35.92 -23.5 -17.45 20.17
Q5 (-1.645) 3.037 -61.61 -29.42 -19.23 -14.26 20.85
Q15.9 (-1) 3.098 -36.98 -17.49 -11.35 -8.375 22.17
Q25 (-0.674) 3.13 -24.5 -11.44 -7.358 -5.395 22.88
Q75 (0.674) 3.27 27.24 13.58 9.14 6.918 26.32
Q84.1 (1) 3.306 39.62 19.56 13.09 9.866 27.28
Q95 (1.645) 3.379 64.4 31.52 20.99 15.76 29.34
Q97.72 (2) 3.421 77.85 37.94 25.21 18.91 30.61
Q99 (2.326) 3.459 90.28 44.01 29.22 21.91 31.8
Q99.865 (3) 3.552 115.9 56.38 37.42 27.99 34.9
Q99.9 (3.090) 3.565 119.4 58.23 38.57 28.87 35.31
Q99.99 (3.719) 3.653 144.3 69.96 46.39 34.76 38.48
Q99.99683 (4) 3.686 155.6 74.73 49.34 36.86 39.78
Q99.999 (4.265) 3.745 169.7 82.61 54.76 40.98 42.24
```

Figure 153 Quantiles section of the *.mpp0 file

```
Quantiles:
-----
(Q50 is median, see above)

      Q01      Q05      Q25      Q75      Q95      Q99
leakpwr  2.032e-05 2.134e-05 2.311e-05 2.592e-05 2.865e-05 3.060e-05
systoffset1 -1.155e-03 -3.790e-04 7.043e-04 2.200e-03 3.345e-03 4.006e-03
systoffset2 -1.263e-04 2.242e-04 7.511e-04 1.480e-03 2.039e-03 2.371e-03
systoffset3  9.488e-05 3.390e-04 6.867e-04 1.161e-03 1.528e-03 1.762e-03
systoffset4  1.603e-04 3.521e-04 6.086e-04 9.656e-04 1.246e-03 1.417e-03
```



Using the Other_Percentile Option

The Variation Block's Option `Other_Percentile` provides you with the ability to specify quantiles lower than 1 percent. For example:

```
.Variation
Option Other_Percentile = percentile
.end_variation
.data percentile
    q
    0.001
    0.1
    0.25
    0.75
    0.9
    0.999
.enddata
```

This option allows you to help to see how much impact there is from trailing data points, or to count samples near the absolute minimum for a sample set. You can set Q to gauge the true population of values near or around the absolute minimum.

In this case, the data value range is (0.0 to 1.0). PrimeSim HSPICE issues a warning message if any values exceed the range and the data mining feature ignores these values. Because a data block can have more than one column, only the first column is processed when you use `Option Other_Percentile`.

You can also provide the percentile information as either a cumulative distribution or sigma values for the standard normal distribution.

Examples:

1. `Option other_Percentile = list(0.0013, 0.0228, 0.1587, 0.8413, 0.9772, 0.9987) or`

2. `Option other_Percentile = list_sigma(-3.0, -2.0, -1.0, 1.0, 2.0, 3.0)`

The `list_sigma` keyword indicates that the percentiles are provided in terms of the standard normal distribution.

Starting from S-2021.09, PrimeSim HSPICE uses PrimeSim AVA based post-processing. In such case, the `other_percentile` options are supported under the utility of Monte Carlo applications. For more information, see Configuring Basic Settings section in the *PrimeSim AVA User Guide*.

The following rows are added in the `*.mpp0/.ava.report` file for user-defined percentiles; the confidence intervals are calculated and displayed, as well:

```
.....
User Specified Quantiles:
-----
      Q(99.9%)      Q(0.1%)      Q(10.0%)      Q(25.0%)      Q(75.0%)      Q(90.0%)
2.750e-05      2.5%      1.689e-05      2.177e-05      2.295e-05      2.578e-05
3.214e-05
leakpwr      1.992e-05      2.196e-05      2.311e-05      2.592e-05
2.780e-05
3.266e-05
      97.5%      2.010e-05      2.211e-05      2.322e-05      2.609e-05
2.804e-05
3.266e-05
      2.5%      -1.669e-03      -8.952e-05      5.768e-04      2.114e-03
2.805e-03
4.402e-03
systoffset1      -1.620e-03      4.004e-05      7.043e-04      2.200e-03
2.914e-03
4.643e-03
```

3.045e-03 97.5% -1.323e-03 1.139e-04 7.738e-04 2.286e-03
4.643e-03
.....

Extremes and Corners

The minimum and maximum values for the measures, together with the sweep indices at which they occur, are given next (Figure 154 on page 608).

Figure 154 Extremes and corners section of the *.mpp0/.ava.report file

Extremes and corners:

	min	max	minIndex	maxIndex
leakpwr	1.689e -05	3.409e-05	729	790
systoffset1	-1.669e-03	5.049e-03	648	969
systoffset2	-4.006e-04	2.865e-03	648	969
systoffset3	-7.319e-05	2.075e-03	648	969
systoffset4	4.285e -05	1.689e-03	648	661

Variable Screening

The table in this section is similar to the DC and ACMatch tables shown in Chapter 27, Mismatch Analyses. The contribution of each independent variable to the response variability is computed. PrimeSim HSPICE uses correlation to measure the variability contribution.

See Table 74 for a listing and description of the column headings.

Variable Screening: Response and Important Elements/Models

By default, PrimeSim HSPICE uses the Pearson correlation coefficient for screening. See http://en.wikipedia.org/wiki/Pearson_product-moment_correlation_coefficient. The Pearson correlation is a suitable measure when the responses are approximately linearly related to the dominant sources of variation. The more general Spearman rank correlation coefficient (see http://en.wikipedia.org/wiki/Spearman%27s_rank_correlation_coefficient), which only requires monotone behavior, can also be used by specifying the Variation Block command `Option Screening_Method=Spearman`. The contributions from the independent variables are then aggregated to the element level and sorted by importance.

Starting from S-2021.09, PrimeSim HSPICE uses PrimeSim AVA based post-processing. In such case, the screening methods are supported under the Advanced Settings of Utility or Monte Carlo applications. For more information, see Configuring Advanced Settings section in the *PrimeSim AVA User Guide*.

Where possible, matched groups are identified. Careful layout techniques like common centroid can be used to reduce the influence of spatial variation and thermal/voltage gradients for matched groups. See [Figure 155 on page 609](#).

Note:

- In cases where all variation contributions are smaller than 0.05%, at least 10 element variations are reported. PrimeSim HSPICE also issues a warning noting that the results are below the cutoff point: `**warning** not all element variation contributions are above 0.05%`
- With an encrypted netlist, you will get a statistical summary. However, the variable screening report that uses the model parameters and elements may not be available, as they are encrypted.

Figure 155 Response subsection of the Variable Screening portion of the *.mpp0/.ava.report file

Response: systoffset1

Global Variation = 0.8%, Local Variation = 99.2%, IRV Variation = 0.%

Screening_method = Pearson|Spearman

Element	Model	EquivVar	CumVar	W(nm)	L(nm)	MatchGrp	Par	ID	Corr	ParVar
1:mr1	snps20n	30.8%	58.4%	1.39e+04	1.95e+03	1				
								vth0	ILN	-0.425 18.0%
								u0	ILN	0.360 12.9%
1:mr2	snps20n	27.6%	27.6%	1.39e+04	1.95e+03	1				
								vth0	ILN	0.428 18.3%
								u0	ILN	-0.306 9.3%
1:mp3	snps20p	21.7%	80.1%	1.79e+04	1.93e+03	2				
								vth0	ILN	-0.356 12.6%
								u0	ILN	-0.302 9.1%
1:mp4	snps20p	17.9%	98.0%	1.79e+04	1.93e+03	2				
								vth0	ILN	0.314 9.8%
								u0	ILN	0.284 8.0%

Subcircuit number Identify matched groups

Variable screening Using Fast Sensitivity

The fast sensitivity method uses PrimeSim AVA for computing sensitivities using machine learning techniques. When the sample size is small, the results of the sensitivity analysis cannot be exact. In such cases, the fast sensitivity method is helpful. In other cases, even though the sample size is relatively large, fast sensitivity method can be used when the simulation runtime of each sample is long or if there are a large number of IRVs. This method helps in creating the sensitivity contribution table in less time.

By default, Pearson correlation is used as the sensitivity method. This is also called classic sensitivity method.

To enable the fast sensitivity, use the following options:

```
.option fast_sens_method = 1
```

or

Sensitivity Method: Use the **Fast** option under **Basic Settings** of Monte Carlo or Utility applications in PrimeSim AVA. For more information refer, Configuring Basic Settings section in the *PrimeSim AVA User Guide*.

Important elements or models

The information in this section helps you focus on corner cases and extracts data on subcircuit encoding and selective simulation.

Figure 156 Information to simulate corners

```
Important elements or models
=====
1:r0 snps20n 1:mn2 1:mn1 1:mp3 1:mp4

subcircuit encoding
=====
Key Subckt
1: xi82

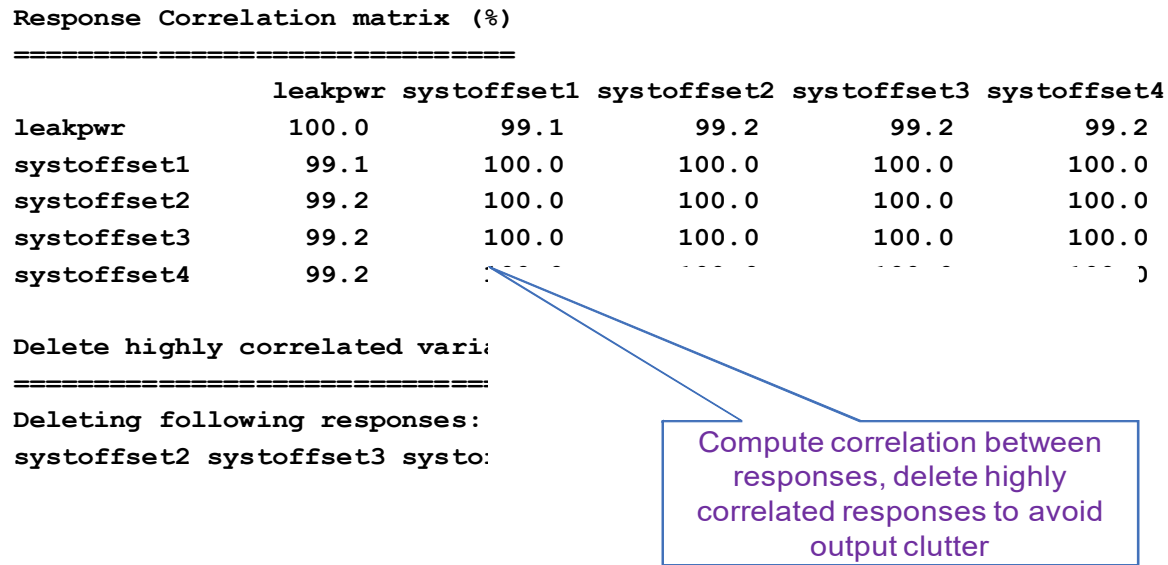
Selective simulation -----> Simulate these "corners" to validate
                                Robustness after device size changes

To simulate corners only:
Use: ... Monte = list(648, 729, 790, 969)
```

Response Correlation Matrix

PrimeSim HSPICE computes the correlation between responses and removes highly correlated responses to lessen output file size.

Figure 157 Response Correlation matrix



Variable Screening Table Headings

Column Heading	Description
Element	Name, if applicable.
Model	Name of model.
EquivVar	Normalized variance associated with an element or model; it is the sum of all local/global variation terms associated; uses normalized values for ease of comprehension. (Elements with EquivVar smaller than 0.05 percent are not reported.)
CumVar	The elements and models ordered by their contribution and shown as cumulative contributions.
W	Width of element in nanometers.
L	Length of element in nanometers.
MatchGrp	Mainly for analog and SOC designs. The influence of spatial variation, including thermal and bias gradients, is reduced if devices with the same MatchGrp index are close together and in the same orientation on the layout, preferably common-centroid. The *.mcp0 file reports for all instance parameters for matched group MOSFETs for the following models: PSP/BSIM/BSIMSOI and instance parameters W and L for all other models. See Figure 158 on page 612 .
Par	Parameters associated with the raw correlations.
ID	Form of variation (local, element, global, and so forth.)

Column Heading	Description
ParVar	Contents are related to the square of the Corr column and gives the equivalent normalized variance associated with the Corr column. The sum of this column is 100 percent if all sources of variation are displayed. The values in the Corr column are aggregated to the element level for local variation and to the model level for global variation. This drives the values in the left half of the table. (Parameters with ParVar smaller than 0.01 percent are not reported.) By default, PrimeSim HSPICE uses the Pearson correlation coefficient to calculate the ParVar items (you can optionally use the Spearman method, controlled by Variation Block Option Screening_Method). The Pearson equations are: Pearson correlation coefficient (default) $\rho_{x,y} = \frac{\text{cov}(X,Y)}{\sigma_x \sigma_y} = \frac{E[(X-\mu_x)(Y-\mu_y)]}{\sigma_x \sigma_y}$ Spearman correlation coefficient, $d_i = x_i - y_i$ $\rho = 1 - \frac{6 \sum d_i^2}{n(n^2-1)}$
Corr	The raw correlations are shown in the Corr column. These are associated with parameters in the Par column. Note: If the netlist has only one or more than 50 measure statements, no response correlation matrix is reported.

Instance parameters listing for elements in match groups

Figure 158 on page 612 breaks out instance parameters for elements in match groups.

Figure 158 Instance parameters for PSP/BSIM4/BSIMSOI models (top) and all other models — W and L instance parameters, only (bottom)

```

=====
Instance parameters for elements in match groups
=====
The following are elements defined with PSP/BSIM4/BSIMSOI models.
1.mn1 l=1E-06 w=7E-06 pd=5.5E-07 ps=4.8E-07 ad=1.89E-14 as=1.4175E-14
1.mn2 l=1E-06 w=7E-06 pd=5.5E-07 ps=5.5E-07 ad=1.89E-14 as=1.89E-14
1.mn6 l=3E-06 w=1.8E-05 pd=7.5E-07 ps=8.5E-07 ad=2.835E-14 as=3.51E-14
1.mn8 l=3E-06 w=1.8E-05 pd=0.39 ps=5.4E-07 ad=9.45E-15 as=1.54E-14
1.mp3 l=1E-06 w=9E-06 pd=4.8E-07 ps=5.5E-07 ad=1.4175E-14 as=1.89E-14
1.mp4 l=1E-06 w=9E-06 pd=5.4E-07 ps=4.7E-07 ad=1.82E-14 as=1.365E-14

=====
Instance parameters for elements in match groups
=====
Element          W (nm)      L (nm)
1.mn6             1.8e+04    3.0e+03
1.mn8             1.8e+04    3.0e+03
1.mp3             9.0e+03    9.3e+02
1.mp4             9.0e+03    9.3e+02

```


Corner Analysis - DC Monte Carlo/Transient Analysis

The PrimeSim HSPICE corner analysis solution combines a DC Monte Carlo run followed by a transient Monte Carlo trial with the corners extracted from the DC analysis. Corners are defined as the Monte Carlo samples that lead to extreme values of the measures. There are two "corners" for any measure, the *min* and the *max*.

Rather than focus on only the min/max values, PrimeSim HSPICE enables selection of outlying values from the tails. These are specified as a fraction of the total sample size, for example, 0.1 percent. The more general corner definition is preferable to reduce design risk.

The following sections discuss these topics:

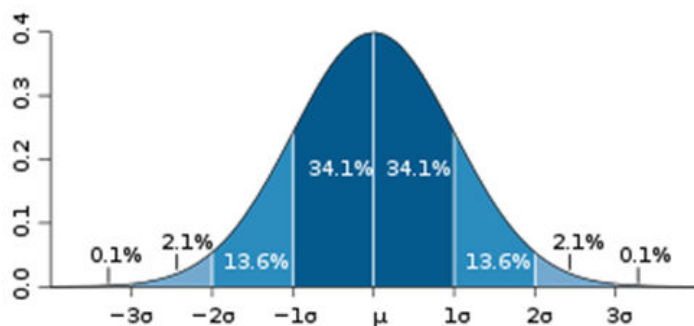
- [DC Monte Carlo Syntax](#)
- [Syntax for Transient Monte Carlo](#)
- [Example](#)

DC Monte Carlo Syntax

```
.DC var1 type np start1 stop1  
+[SWEEP MONTE=MCcommand] [corner_percentile=val]
```

where: `corner_percentile=val` specifies the percentiles used to find corners. The value is a non-negative number in the range (0.0~0.5). For example, if `value=0.1`, then PrimeSim HSPICE sorts the measure results: the points below the 10th percentile and those above the 90th percentile are chosen as corners. If the value = 0.0, then PrimeSim HSPICE uses the maximum and minimum values as corners. The default is 0.0. The following chart explains the corner percentiles for a response variable that has a Gaussian distribution:

Figure 159 Bell-curve to determine corners



- The dark blue zone at the center represents observations within one standard deviation on either side of the mean, which accounts for 68.2 percent of the population.
- Two standard deviations from the mean (dark and medium blue) account for 95.4 percent, and 3-sigma (dark, medium, and light blue) for about 99.7 percent.
- The sample points with 0.1 percent `corner_percentile` represent extreme points which have variations larger than 3.1-sigma.

If multiple DC measures use the `corner_percentile` option, the union of all the corners is used in the next transient simulation.

When PrimeSim HSPICE finds corners during DC analysis:

- PrimeSim HSPICE only generates `*.ms#` files to locate the corners during data mining. Other output files (`*.sw0`, `*.pa0`, and so forth) are not generated.
- The `*.lis` file is simplified and no Monte Carlo-related information is printed.
- Data mining files (`*.mpp0` and `*.annotate`), which are relatively small, are created. Large files such as `*.mc0` are suppressed.

During DC Monte Carlo, a `*.corner` file is also generated with content as follows:

```
*=====
* corner percentile = value
*Selective simulation for extreme corners:
*Monte= list ( corner_number1, corner_number2, ..., corner_numberN)
*Number of samples = K
```

Syntax for Transient Monte Carlo

```
.TRAN tstep1 tstop1 [tstep2 tstop2 ...tstepN tstopN]
+ [START=val] [UIC] [SWEEP MONTE=dc_corner]
```

where: `dc_corner` is a keyword for Monte Carlo simulation only. With this option, PrimeSim HSPICE reuses the corners generated in DC Monte Carlo and runs transient analysis with these corners as random values.

Example

The following is a complete netlist to illustrate running a transient simulation with DC corners.

```
* run Tran simulation with DC corners
*.option mcfast
.options POST OPTS LIST MODMONTE=1 numdgt=10 measdgt=10
.model nch nmos LEVEL=54 TOXM=4E-9 wint=wint_nch VERSION=4.4
.param wint_nch = agauss ( 3e-7 , 1e-7 , 3.0 )

m11 2 11 0 0 nch W=1E-6 L=0.15E-6 dtemp=10
m12 2 11 0 0 nch W=1E-6 L=0.15E-6 dtemp=10
```

```
m13 2 11 0 0 nch W=1E-6 L=0.15E-6 dtemp=10
v01 2 0 1.5
v02 11 0 0.0 pulse (0 1 0 1n 1n 10n 20n)

.dc v02 0 2.0 0.1 sweep monte=500 corner_percentile=0.001
.meas dc ids_1 find par('i(m11)*1E3') at = 1.5
.meas dc ids_2 find par('i(m12)*1E3') at =1.5
.meas dc ids_3 find par('i(m13)*1E3') at =1.5

.tran 1n 1u sweep monte= dc_corner
.meas tran i1 find i(m11) at=1u
.meas tran i2 find i(m12) at=1u
.meas tran i3 find i(m13) at=1u

.end
```

Back-Annotation — *.annotate(.ava.annotate) File

PrimeSim HSPICE provides a file that contains the key local variability contributors and matched pairs for display on the schematic in the Custom Compiler tool. A simple tabular layout is provided for information exchange. For each response, there is the full hierarchic element name, the variability it contributes, and the Match Group value.

The Tail Samples File - .ava.details.json

PrimeSim HSPICE provides a file that contains the details of the tail samples. It provides the indices, measure values, and some additional information to help debug the sample. For more information, see *.ava.details.json File section in the *PrimeSim AVA User Guide*.

More on Debugging - .ava.dsvc.json, .ava.critical and .ava.critical.m(s/d/t)0

PrimeSim HSPICE provides additional information on critical points such as when a measure is at maximum or minimum, left or right most quantiles and tail samples on both sides. These indices are listed separately for each measure in .ava.dsvc.json. For more information, see *.ava.dsvc.json File section in the *PrimeSim AVA User Guide*.

A more detailed analysis can also be done using .ava.critical and .ava.critical.m(s,d,t) 0 files. These files contain the IRV information of the critical corner sweep indices and their corresponding IRVs. To enable this option, use the **Critical Sweep Ext** option in **General Settings**. For more information, see *.ava.critical File section in the *PrimeSim AVA User Guide*.

Alternatively, you can opt to list and run these critical points to generate their waveforms. These waveform files can be helpful in debugging how these corners affect the

distribution. To enable this option, use the **Critical Sweep Waveform** option in **General Settings**.

Standalone Data Mining in PrimeSim HSPICE

Sometimes, users create additional “measurements” outside PrimeSim HSPICE and wish these to be included in the data mining. This requires the new measurements to be formatted in additional files.

You can enter the following on the command line to have PrimeSim HSPICE skip netlist readin, errchk, and simulation, and do standalone data mining:

```
hspice -datamining -i datamining.cfg [-o outname]
```

The configuration file content includes:

- *comments/description
- *Required records
- .sampleFile input.mc0
- .measFile input.mt0 input.mt0A input.mt0B ...
- .Option Screening_Method = Pearson | Spearman
- Option Other_Percentile=list(val1, val2,...,valn), where the value range for val1 to valn is between 0.0 and 1.0.

Or Option Other_Percentile=list_sigma(val1, val2,...,valn), where the value range for val1 to valn is between -10.0 and 10.0.

- .Option datamining_method = dp | scalar (the default value is scalar).

To perform standalone data mining with Monte Carlo in DP, the configuration file only needs to include the following lines:

```
.option datamining_method = dp  
.measfile output_dp
```

Post Processing Utility in PrimeSim HSPICE

Starting from S-2021.09, the standalone data mining feature uses PrimeSim AVA based application using post-processing feature. In this case, all the details like sample file, IRV file are included in the .jsonfile. Input netlist is not required.

Use the following syntax to invoke Post-Processing Utility in PrimeSim AVA is:

```
hspice -adv user.json -o outname
```

The contents of the .json file:

```
{
  "Active_Application": "Utility",
  "Utility": {
    "Method": "Post_Datamining",
    "Sens_Screening_Method": "Pearson",
    "Measure_Files" : ["input.ms0", "input.ms1", ... ],
    "Sample_Files": ["input.mc0", "input.mc1", ...],
    ...
  }
}
```

The extensive MC outputs are created similar to regular Monte Carlo simulation. For more information, see Monte Carlo Outputs section in the *PrimeSim AVA User Guide*. For more detailed sensitivity information, enable **Output Detailed Sensitivity File** under **General Settings** of Monte Carlo or Utility applications.

Incremental Monte Carlo Simulation in Standalone Data Mining Mode

PrimeSim HSPICE now provides solution to reuse previous Monte Carlo results and perform data mining in an incremental mode. You only need to indicate the location of the results after several runs; the standalone data mining merges these results to report new data mining results.

PrimeSim HSPICE provides two ways of running Monte Carlo, one in scalar mode and the other in DP mode. It stores their measure results in different ways. For example, in DP mode, the measure files are usually stored in **_dp/worker*/*. With the enhanced standalone data mining, PrimeSim HSPICE automatically finds these measure files and merges them to perform the data mining.

To use the merging mode, use the following syntax:

Scalar mode:DP mode:

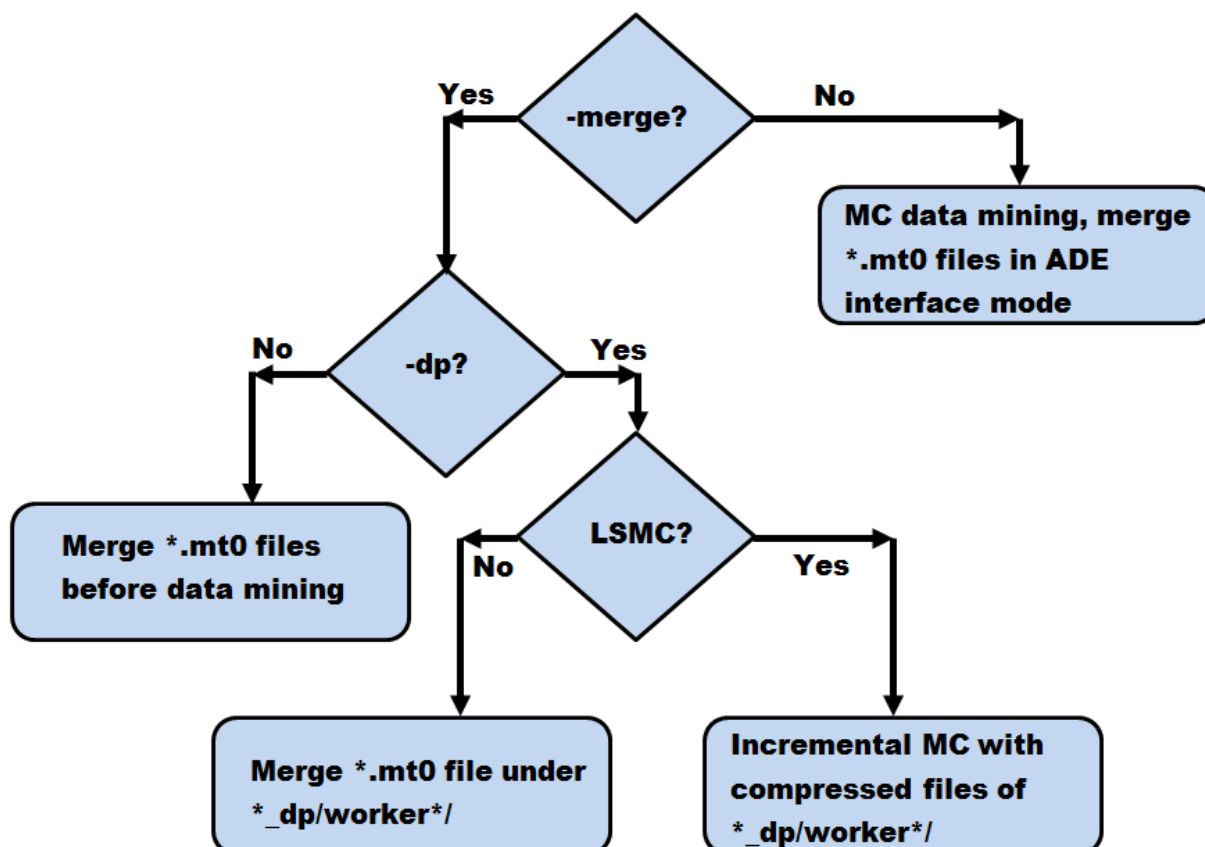
```
.measfile -merge output1.mt0, output2.mt0,..., outputn.mt0

.option datamining_method = dp
.measfile -merge output1_dp/, output2_dp/ ... outputn_dp/
```

The new keyword `-merge` helps PrimeSim HSPICE to distinguish the merging mode from the previous ADE mode. It indicates that all the measure outputs need to be merged before data mining.

With the option `dp`, PrimeSim HSPICE uses the **_dp/* paths to find the measure results. The measure results can be stored in either raw data mode (**.mt0* files) or compressed mode (**.ecdf.gz*). Data mining automatically identifies the raw/compressed modes and processes the data accordingly.

The following flow diagram illustrates how incremental Monte Carlo works in PrimeSim HSPICE.



Note:

To run incremental MC, you should set unduplicated samples between each runs.

In scalar mode, the PrimeSim HSPICE data mining algorithm can identify duplicated samples and process them correctly. In DP mode, PrimeSim HSPICE displays a warning message and stops data mining, when duplicated samples are detected.

CFG File Configuration for Data Mining in DP Mode

As mentioned earlier, you will use a command such as the following for incremental data mining in PrimeSim HSPICE:

```
hspice -datamining -i increm.cfg
```

You can configure the input *.cfg file for incremental data mining in the following four ways:

1. `.option datamining_method = dp`
`.measfile -merge output1_dp,output2_dp`

In this case, PrimeSim HSPICE looks to merge the two measure results files `output1` under `output1_dp/` and `output2` under `output2_dp/`. That is, the default measure results file names under the specified *_dp/ DP folder names.

Note:

By default, the measure results file names under the DP path are the same as the DP path name without _dp/.

2. `.option datamining_method = dp`
`.measfile -merge output1_dp/out1 output2_dp/out2`

In this case, PrimeSim HSPICE looks to merge two measure results files `out1` under `output1_dp/` and `out2` under `output2_dp/`. That is, the file names are not their default names.

3. `.option datamining_method = dp`
`.measfile -merge path1/output1_dp, path2/output2_dp`

Here, the two DP measure results are not in the same path. `path1` and `path2` are the relative or the absolute path to the `output1_dp/` and `output2_dp/` DP folders. `path1` and `path2` are not the same. Also, the file names are their default file names under the *_dp/ DP folder names.

4. `.option datamining_method = dp`
`.measfile -merge path1/output1_dp/out1, path2/output2_dp/out2`

Here, the two DP measure results are not in the same path. `path1` and `path2` are the relative or the absolute path to the `output1_dp/` and `output2_dp/` DP folders. `path1` and `path2` are not the same. Also, the file names are not their default names.

Large Scale Monte Carlo Simulation – Traditional and Variation Block

High replication circuits require small failure probability. The circuits include bit cells, sense amps, and digital standard cells (flip-flops and latches). Low probability of failure means high sigma simulation. However, collecting data and statistical analysis with one billion samples is expensive in disk space, memory, and computing time. PrimeSim HSPICE supports Monte Carlo simulation up to one billion samples.

Setup Syntax:

```
.Variation  
Option Large_Scale_MC=Yes  
.End_Variation
```

The statistical results are printed to the *.mpp file only. In addition to the regular information, you also can find the tail samples and values for the measured variables.

Measuring Output in Histogram Format:

To save disk space while streamlining statistical analysis, by default, there is no *.mt# (*.ms#, *.ma#) file generated. One extra *.hmt#.csv (*.hms#.csv, *.hma#.csv) file is created to display the distribution shape (histogram table) of each measure variable. This means measure results are stored and displayed in a compact format.

Plotting the Histogram in an Excel Spreadsheet:

To display the histogram with an Excel sheet, follow these steps:

1. Delete the title lines.
2. Select the whole columns of *_count.
3. Choose **Insert > line**, then this histogram was drawn; you can select several *_count columns and draw the histogram in the same plot.
4. The column of measure variables are response to the x-axis of the histogram.

Limitations and Assumptions:

The following limitations and assumptions apply to large scale Monte Carlo (LSMC):

- Running LSMC with ALTER statements is not recommended because the results are not reliable. When ALTER and Monte commands are in the netlist, DP will parallelize only the ALTER blocks.
- To verify LSMC results with the LIST command for specific samples, set the following options in the netlist;
 - .Variation
 - Option Use_Agauss_Format = Yes
 - Option Random_Generator = MCG
 - .End_Variation

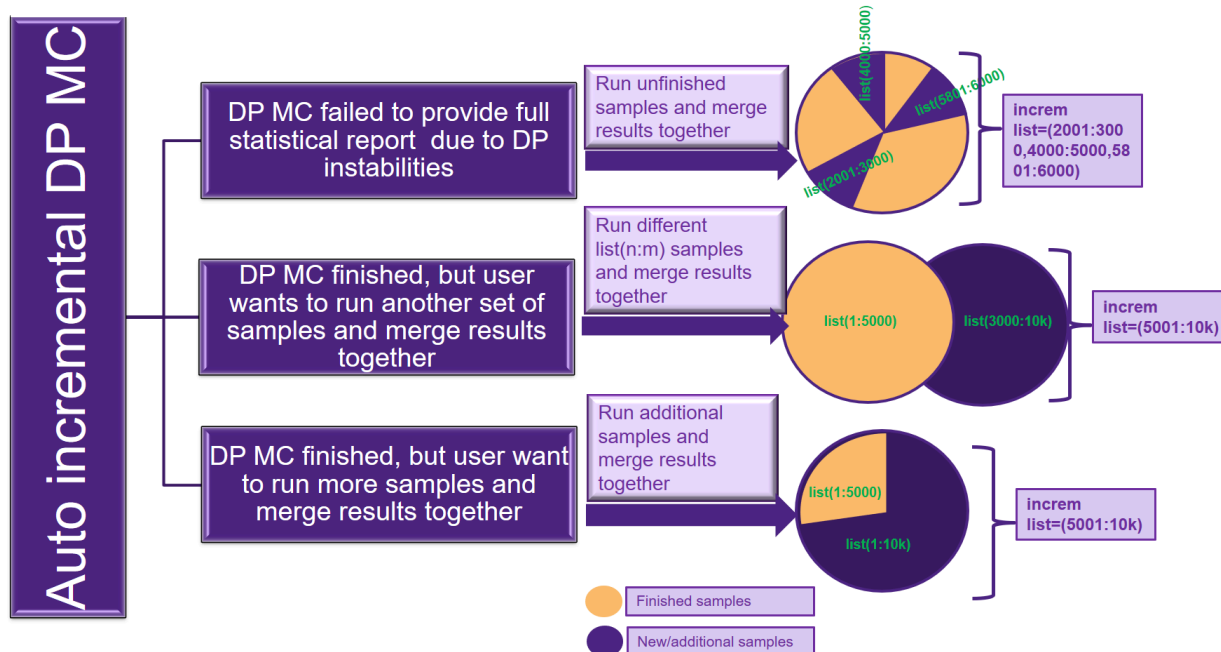
These options ensure the both the sampling method (SRS in this case) and the pseudo random number generator (MCG) are matched between LSMC and Monte=LIST() runs.

Automatic Incremental DP Monte Carlo

PrimeSim HSPICE now provides an automatic incremental distributed processing (DP) Monte Carlo (MC) solution which allows you to re-run DP MC many times on unfinished samples and new/additional samples. It then combines the results from the original and the new runs to provide a complete statistical MC analysis report. [Figure 160](#) illustrates the usage models for this solution.

Once you indicate the location of the previous results, PrimeSim HSPICE will recognize the unfinished samples and run them automatically in the DP mode. The simulator then collects these simulation results and merges them with the previous results. The merging includes measure results, waveform group files, and data-mining results.

Figure 160 Usage Models for Automatic Incremental DP MC



Syntax

Automatic incremental DP MC requires the following syntax in the DP command line:

```
-dpincremental original_path
```

Where `original_path` is the `sub_dpdir` of the original run.

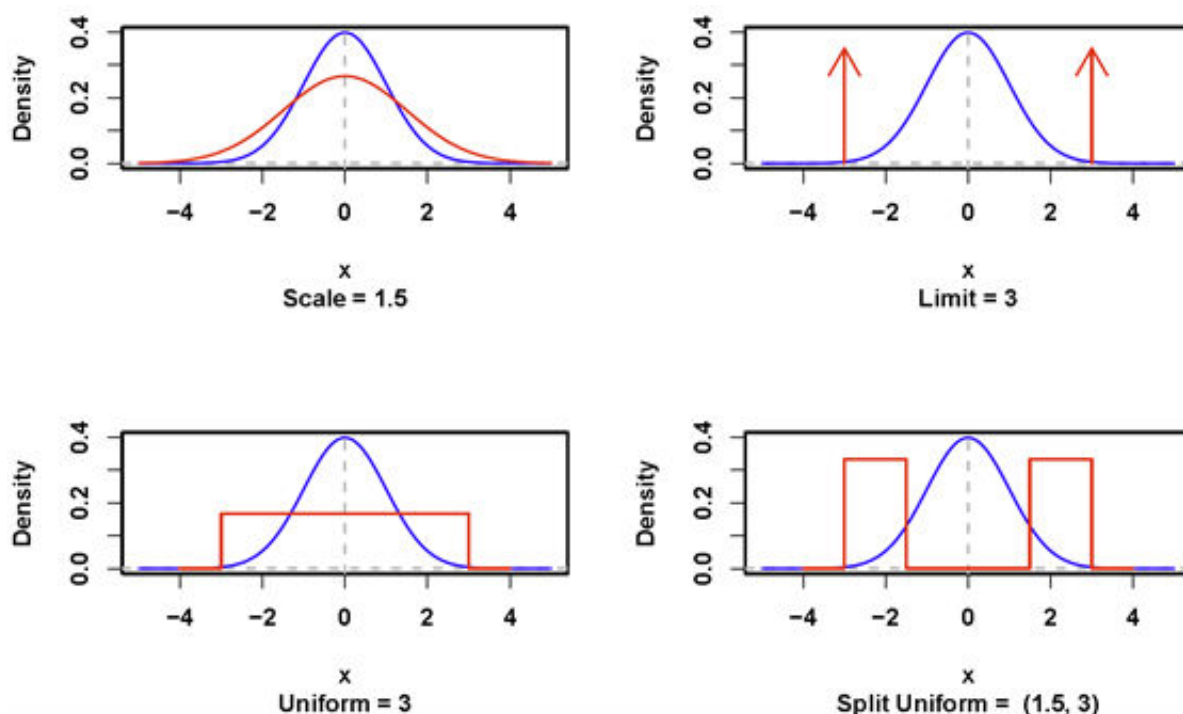
For more information, see [Running Multiple Simulations on page 102](#).

Modifying the Built-in Distribution for Monte Carlo Analysis

Monte Carlo analysis is generally used for estimating response distributions and yield. To get a relative accurate distribution and yield results, a large Monte Carlo sample size is required to ensure sufficient coverage, but this approach faces the performance issue.

PrimeSim HSPICE now provides another solution by modifying the device distributions to get a larger fraction of samples in the tails. With simple options, you can change the distributions to *extreme* values without touching any part of the variation definition.

The following plots show examples of modified input distributions on standard normal distribution functions. The blue curves are standard normal distributions in model files. The red curves are the modified distributions.



Syntax for Modifying the Distribution

PrimeSim HSPICE provides the option `Modified_Distribution` to modify/transform the built-in distribution for Monte Carlo analysis.

- **For Variation Block Definition**

```
.Variation  
Option Modified_Distribution =
```

```
+Scale(val)|Uniform(val)|Limit(val)|CDF(XYvalue_pairs)
.end_Variation
```

• For Gaussian-style Definition

```
.Variation
Option Use_Agauss_Format = Yes
Option Modified_Distribution =
+Scale(val)|Uniform(val)|Limit(val)|CDF(XYvalue_pairs)
.end_Variation
```

Caution:

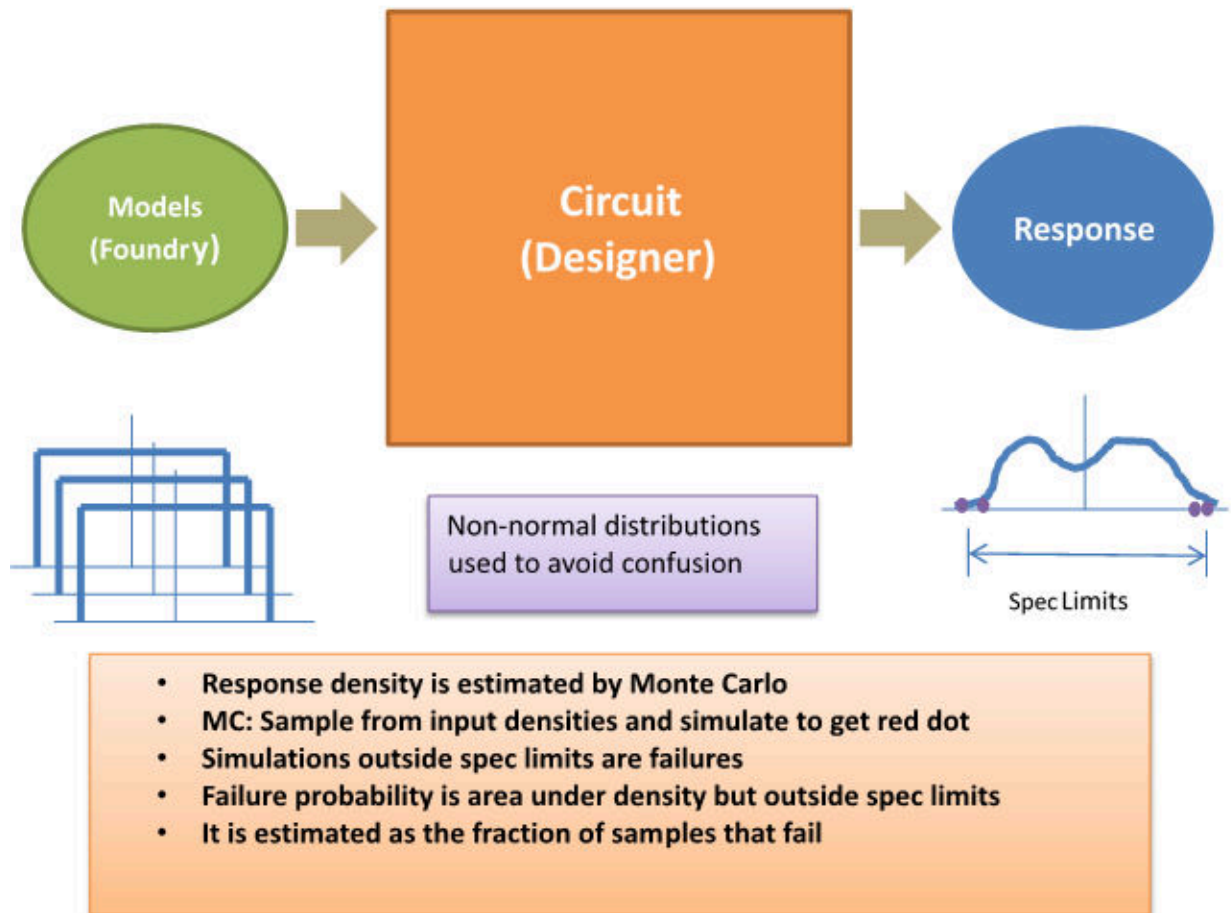
Option Modified_Distribution=Scale(val) affects only the scaling of Normal/Gaussian random parameters (while not affecting LIMIT/UNIFORM distributions).

Table 73 Description of Syntax

Syntax	Description	Examples
Scale(S)	$S > 0$; the sample are points multiplied by S	Option Modified_Distribution = Scale(1.5)
Limit(L)	$L > 0$; limits the distribution with samples at $\pm L$	Option Modified_Distribution = Limit(3)
Uniform(A)	$A > 0$; uniform distribution from $-A$ to $+A$	Option Modified_Distribution = Uniform(3)
CDF(XY_Pairs)	As in the CDF definition	Option Modified_Distribution = CDF(-3,0, -2,0.5, 2,0.5, 3,1)

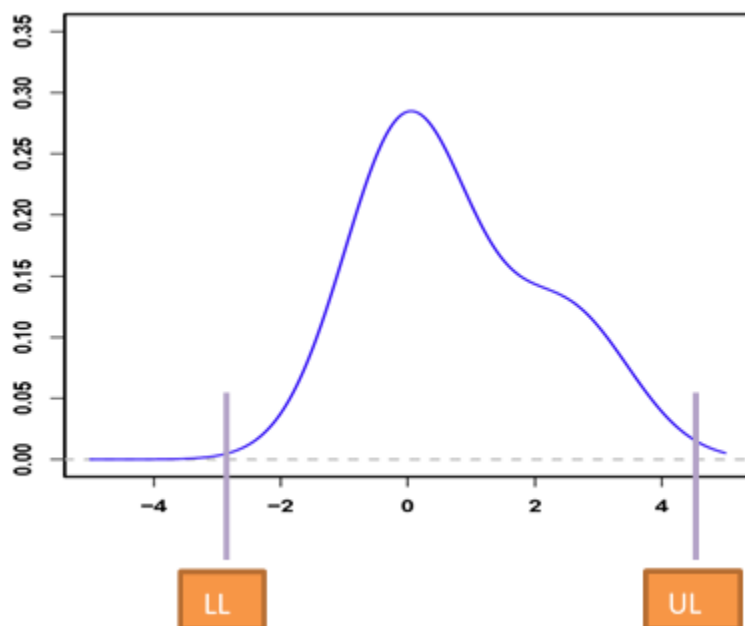
Yield Computation in Monte Carlo Analysis

Monte Carlo analysis is generally used for estimating response distributions and yield. The following figure describes the response yield in circuit design.



PrimeSim HSPICE now provides a specification block syntax for yield computation during Monte Carlo analysis.

To understand the yield specification, consider a measure with density as shown in the following figure:



For the circuit to function correctly, the measure must lie within the range defined by the lower limit (LL in the figure) and the upper limit (UL). This range is called the specification for that measure. At times, for some measures, LL can be $-\infty$ and UL can be $+\infty$. One-sided constraints are also possible.

Circuits with measure values within such specification are said to *pass*. In the case of multiple measures, all the measures must meet their specifications. *Yield* is the probability of passing all such measure specifications.

Syntax for Yield Specification

PrimeSim HSPICE supports yield specifications defined through a combination of `.DATA` and `.Variation` block format, such as the following:

```
.Data SPEC
  Meas1_name   Meas2_name   Meas3_name Meas4_name [dataform]
      LL1       LL2         LL3       LL4
      UL1       UL2         UL3       UL4
.EndData
.Variation
Option Yield_Spec_Block = SPEC
.end_Variation
```

Note:

- Some of the LL and UL entries can be `-Infinity` or `+Infinity`. You can also use the short form `-Inf` and `Inf`.
- Usually, $LL < UL$. PrimeSim HSPICE yield specification allows $LL > UL$. However, the interpretation for $LL > UL$ is different from that for $LL < UL$.
- The specifications define as $LL < \textit{measure_name} < UL$.
- Measures defined in the data block but missing in the netlist are ignored. However, a warning message is provided.
- Measures defined in the netlist but with no entries in the data block are assumed to have $LL=-Inf$ and $UL=Inf$, and are ignored for yield computation.
- Use the tail keyword `dataform` is used to identify the `-Inf` and `Inf` short forms in the data values.

PrimeSim HSPICE outputs the yield analysis results in the Yield section of the `*.mpp0` file. See [Output of Yield Analysis](#).

Yield Specification for Standalone Data Mining

PrimeSim HSPICE standalone data mining for Monte Carlo post processing supports the yield specification syntax (see [Syntax for Yield Specification](#)) in the `*.cfg` file, for yield analysis.

You can then execute the standalone data mining command as in the following example

```
HSPICE -datamining -i input.cfg
```

PrimeSim HSPICE performs the yield analysis and outputs the results in the Yield section of the `*.mpp0` file. See [Output of Yield Analysis](#).

Input Example for Yield Analysis

The following is a simple example of input for yield analysis:

```
* DAC for PT test
.param random1=agauss(0,1,1)
.param par1=random1
.param par11=0 $agauss(0,1,1)

i1 0 1 1
x1 1 2 sub1 w=1u l=20u
x2 2 3 sub1 w=1u l=20u
x3 3 4 sub2 w=1u l=20u
```

```
r4 4 0 10

.subckt sub1 1 0
.param p1=par1 p2=par11
r1 1 0 res1
.model res1 r res='10+p1+p2'
.ends

.subckt sub2 1 0
.param p3=par1 p4=par11
r1 1 0 res2
.model res2 r res='10+p3+p4'
.ends

.option sampling_method
.param dummy=1
.dc dummy 1 1 1 sweep monte=1000
.variation
option yield_spec_block =yield_spec
.end_variation
.data yield_spec
r1 r2 r3 v1
9.0 9.0 9.0 36
11.0 11.0 11.0 42
.enddata
.option post
.meas dc r1 find lv2(x1.r1) at=1
.meas dc r2 find lv2(x2.r1) at=1
.meas dc r3 find lv2(x3.r1) at=1
.meas dc v1 find v(1) at=1
.end
```

Output of Yield Analysis

Based on the yield specification, there are four forms of Pass/Fail regions as shown in the following figure:

#	LL	UL	Pass/Fail Regions		
1	-1	1			
2	1	-1			
3	- Inf	1			
4	1	Inf			

In the preceding figure, the green regions are passing while red ones are failing. In form 2, the lower limit is larger than the upper limit and the interior region is considered the failing region.

One example of PrimeSim HSPICE yield analysis output in the *.mpp0 file is shown in the following figure:

```
##Yield
#-----
```

	LL	UL	CI	Pf	Pf_L	Pf_U
			5%	25.726%	11.612%	14.114%
r1	9.000e+00	1.100e+01	95%	30.130%	13.714%	16.416%
				34.840%	15.968%	18.871%
			5%	25.726%	11.612%	14.114%
r2	9.000e+00	1.100e+01	95%	30.130%	13.714%	16.416%
				34.840%	15.968%	18.871%
			5%	25.726%	11.612%	14.114%
r3	9.000e+00	1.100e+01	95%	30.130%	13.714%	16.416%
				34.840%	15.968%	18.871%
			5%	29.630%	6.006%	23.624%
v1	3.600e+01	4.200e+01	95%	34.234%	7.808%	26.426%
				38.539%	9.409%	29.129%
5% CI:	5.706e+01%					
Circuit Yield:	5.986e+01%					
95% CI:	6.292e+01%					

```
##end_of_yield
```

Note:

- The confidence intervals are computed by bootstrap.
- Pf_L and Pf_U are the failure probabilities for the lower and upper limits for each measure. Pf is the failure probability of the measure.
- Only form 1 will have values for Pf_L and Pf_U.

When .option mppfile_form=1, the tables in *.mpp0 are transposed and printed in the Excel format.

If number of measures with yield specifications exceeds 50 or if option large_scale_mc=yes, PrimeSim HSPICE skips the computation of the confidence interval.

Yield specifications permit LL>UL. Then the yield computation follows the second form of Pass/Fail regions.

Consider the following yield specification:

```
.data yield_spec1
r1  r2  r3  v1
10.5 9.0  9.0 36
```

```
10.0 11.0 11.0 42
.enddata
```

For such a definition, the Yield section output in the *.mmp0 file is shown in the following figure:

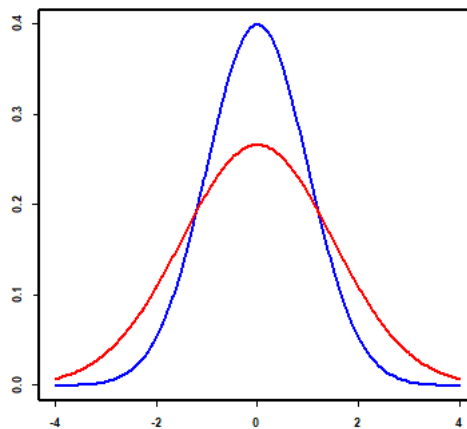
```
##Yield
#-----
                LL                UL                CI                Pf                Pf_L                Pf_U
                LL                UL                CI                Pf                Pf_L                Pf_U
r1              1.050e+01          1.000e+01          5%                17.165%                ***                ***
                19.720%                ***                ***
                95%                22.175%                ***                ***
r2              9.000e+00          1.100e+01          5%                25.726%                11.612%                14.114%
                30.130%                13.714%                16.416%
                95%                34.840%                15.968%                18.871%
r3              9.000e+00          1.100e+01          5%                25.726%                11.612%                14.114%
                30.130%                13.714%                16.416%
                95%                34.840%                15.968%                18.871%
v1              3.600e+01          4.200e+01          5%                29.630%                6.006%                23.624%
                34.234%                7.808%                26.426%
                95%                38.539%                9.409%                29.129%

5% CI:          3.718e+01%
Circuit Yield:  4.014e+01%
95% CI:         4.324e+01%
##end_of_yield
```

Sigma Amplification MC and Outputs

Sigma Amplification Monte Carlo analysis is a special form of importance sampling that can be applied to independent random variables (IRVs) whose distributions are Gaussian or Normal. This type of sampling amplifies the standard deviations of all IRVs by a common scaling factor to increase the number of samples in the tail regions (see [Figure 161 on page 631](#)). This enables designers to have better idea about the circuit behavior in the tails.

Figure 161 Sigma Amplification Principle



Blue curve indicates the density defined in the models.

Red curve indicates the density used in the simulation.

The following sections discuss these topics:

- [Activating Sigma Amplification MC](#)
- [Determining the Amplification Factor](#)
- [Sigma Amplification MC Outputs](#)
- [Frequently-Asked Questions](#)

Activating Sigma Amplification MC

The PrimeSim HSPICE simulator supports Sigma Amplification in Monte Carlo analysis through the `Modified_Distribution` option in the variation block. The syntax supported are the following:

```
.Variation  
Option Modified_Distribution = Scale(A)  
.End_Variation
```

Where *A* is the amplification factor. Generally *A* is between 1.1 and 1.5, and is never larger than 2. See [Determining the Amplification Factor on page 632](#).

Note:

The `Scale(A)` syntax works for all IRVs, including global and local IRVs.

For more information on using the `Modified_Distribution` option, see [Modifying the Built-in Distribution for Monte Carlo Analysis on page 622](#).

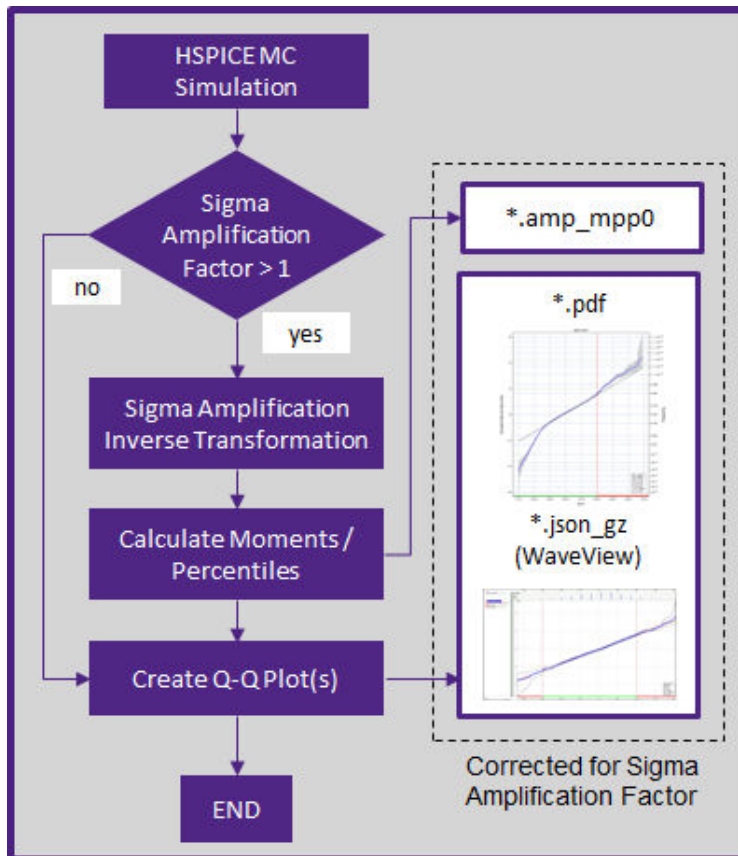
Certain values of the `MCBrief` option will inhibit Sigma Amplification. It is recommended that `.option MCBrief=5` be used for Sigma Amplification MC.

Caution:

Sigma Amplification affects the scaling of only Gaussian-distributed random variables.

After the simulation, the results are post-processed to scale them back to correspond to the original IRVs for statistical inference.

The following figure illustrates the Sigma Amplification MC flow in PrimeSim HSPICE:



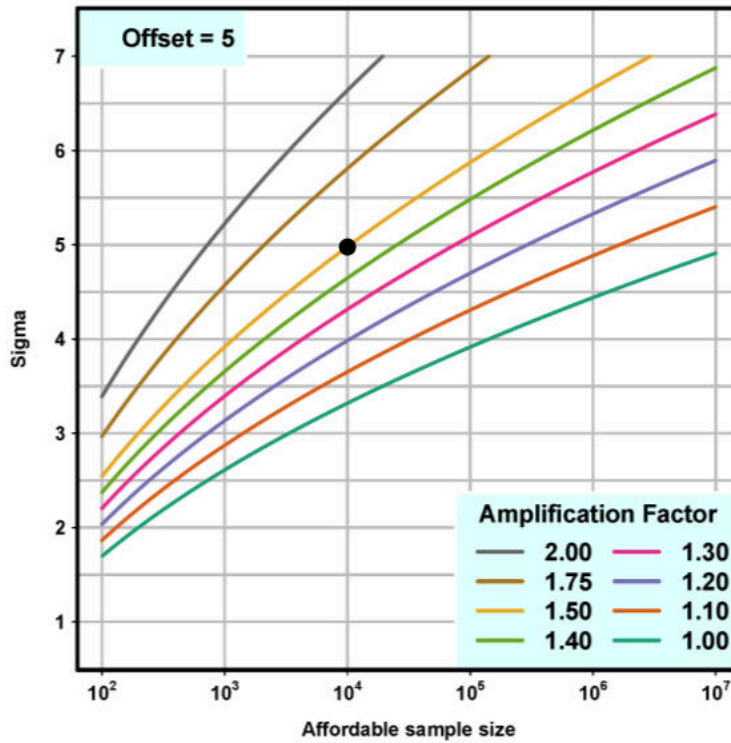
Determining the Amplification Factor

The following factors influence the choice of the amplification factor for Sigma Amplification:

- How many samples can be simulated (computational budget).
- The required/target accuracy in terms of percentiles or sigmas.

The following figure relates these quantities. In the figure, the X-axis represents the affordable sample size and the Y-axis is the desired sigma. Each colored curve

corresponds to an amplification factor. The figure assumes an offset (order statistics) of 5, which provides a reasonable tradeoff between accuracy and computation cost.



Such a figure can be used to determine the amplification factor. Consider the scenario where a sigma of 5 is required and the user can only afford 10^4 samples. The intersection of these values is on the yellow curve and corresponds to an amplification factor of 1.5.

Recommendations:

- Minimum sample size: 300.
- Maximum amplification factor: 2; it is recommended that you use an amplification factor between 1.1 and 1.5.

Sigma Amplification MC Outputs

When the Sigma Amplification factor is set, PrimeSim HSPICE adds one comment to indicate the Sigma Amplification factor in the head of the *.mt#, *.ms#, *.ma#, *.mpp0, and *.qq0.csv files. The *.qq0.csv file also displays more columns for confidence intervals. The new columns are added without the Sigma Amplification factor.

The post-processed outputs of Sigma Amplification MC include the following:

- [Q-Q Plot Files in PDF and WaveView Formats](#)
- [The *.amp_mpp0 File](#)

Q-Q Plot Files in PDF and WaveView Formats

The PrimeSim HSPICE simulator generates Q-Q plots in PDF (*.pdf file) and WaveView (*.json_gz file) formats for all Monte Carlo analyses (except when `.option MCBrief = 6`), including Sigma Amplification MC.

[Figure 162 on page 635](#) and [Figure 163 on page 636](#) show Q-Q plots in PDF and WaveView formats respectively.

In the Q-Q plot, the X-axis corresponds to a particular measurement. The Q-Q plot displays dual Y-axes. The left Y-axis shows the probabilities in terms of the standard normal quantiles. The right Y-axis shows the corresponding cumulative probabilities.

The blue line is the Q-Q plot curve for the given measure. The grey shaded area in the PDF plot, or the area between the two dotted curves in the WaveView plot, gives the 95% confidence intervals for the Q-Q plot. The diagonal line, called the Q-Q line, is a visual guide showing where data from a normal distribution would lie.

The bottom right box provides the values for the sample size N , the Sigma Amplification factor A , the nominal value x_0 , and the first four central moments (mean μ , standard deviation σ , skewness γ , and kurtosis κ).

Figure 162 Example of a PDF Q-Q Plot

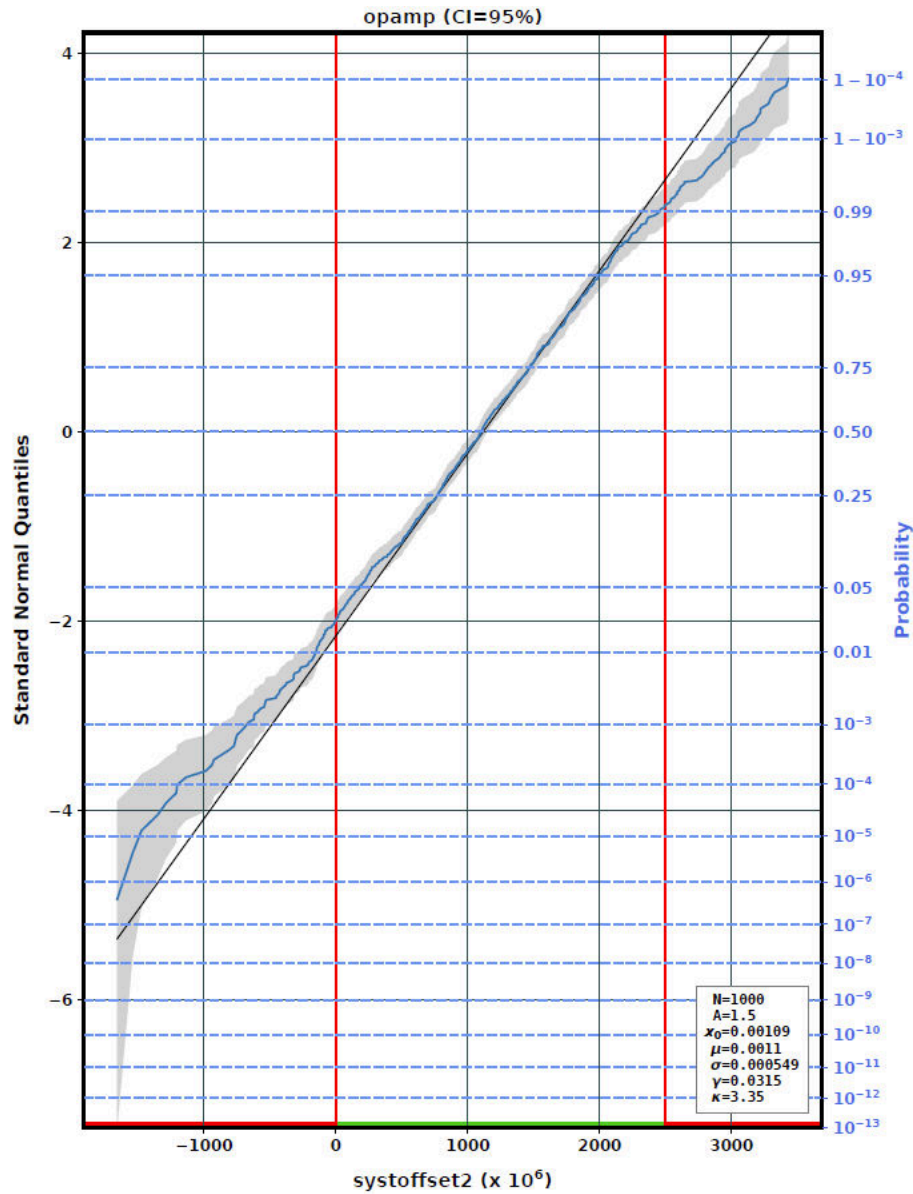
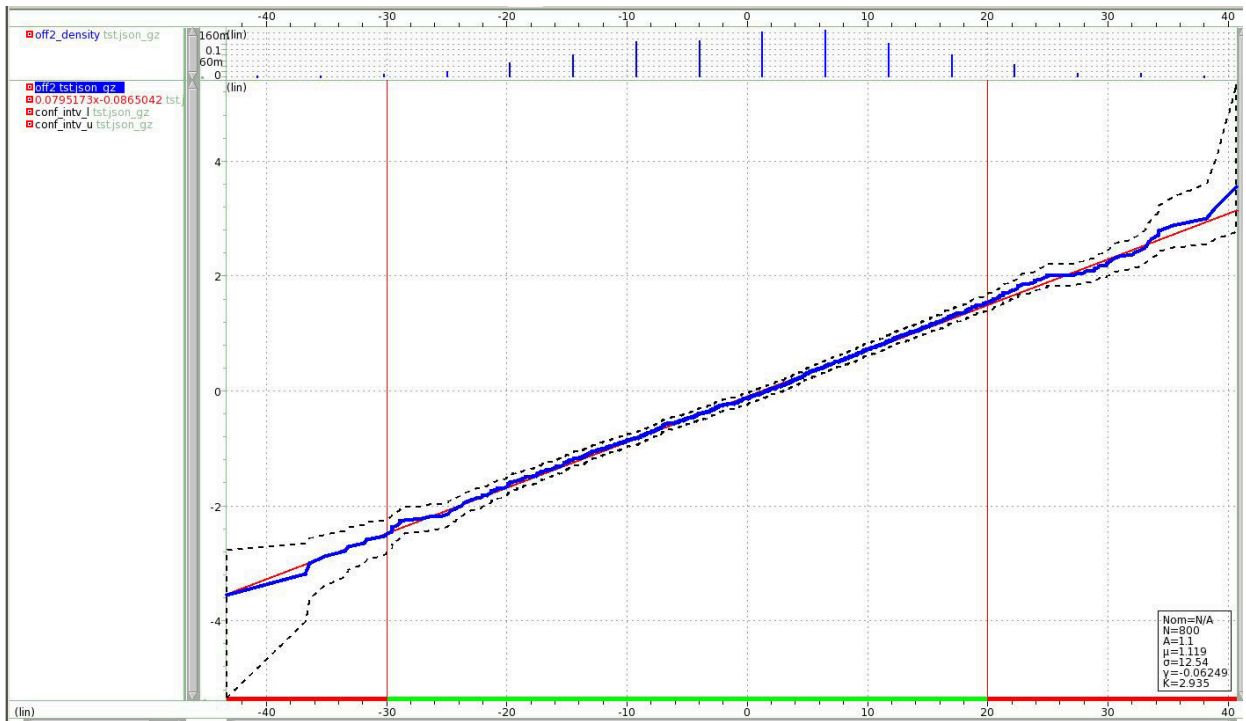


Figure 163 Example of a Q-Q Plot in WaveView



If a yield specification block (see [Syntax for Yield Specification on page 625](#)) is provided in the netlist, the yield specification limits are displayed in the Q-Q plot. The vertical red lines represent the upper and lower yield specification limits. The green and red horizontal bars just above the X-axis represent the yield specification limit regions of the given measure. The green region(s) represent the within-specification condition, while the red region(s) represent the out-of-specification condition.

If the yield specification block is not defined in the netlist, no yield limit information is displayed on the Q-Q plot.

When working with the *.json_gz file in WaveView, you can interactively select measures for the Q-Q plot display. WaveView supports pan, zoom, and dynamic measurements based on the cursor location. In addition, appearing above the Q-Q plot panel in WaveView, is the corresponding probability density plot, displayed for reference purposes.

The *.amp_mpp0 File

For Sigma Amplification MC, moments and percentiles corresponding to the original variables are printed to the *.amp_mpp0 file.

Example 8 The contents of *.amp_mpp0 file

...


```
#Number of Samples
1000
#Number of Measures
4
```

	leakpwr	logleakpwr	systoffset1	systoffset2
Nominal	2.441e-05	-10.62	0.0014	0.001086
Mean	2.452e-05	-10.62	0.001422	0.001099
StdDev	1.938e-06	0.07852	0.001046	0.0005078
Skewness	0.3657	0.1198	-0.01345	-0.00949
Kurtosis	3.361	3.038	2.976	2.978
Q1e-01	1.928e-05	-10.86	-0.001803	-0.0004812
Q1.35e-01	1.955e-05	-10.84	-0.001636	-0.000389
Q1	2.045e-05	-10.8	-0.00116	-0.0001429
Q2.28	2.099e-05	-10.77	-0.0007014	6.746e-05
Q5	2.149e-05	-10.75	-0.0002852	0.0002686
Q15.9	2.266e-05	-10.7	0.0003755	0.0005938
Q25	2.318e-05	-10.67	0.0006959	0.0007439
Q75	2.584e-05	-10.56	0.002124	0.001441
Q84.1	2.653e-05	-10.54	0.002481	0.001611
Q95	2.78e-05	-10.49	0.003132	0.001922
Q97.72	2.853e-05	-10.46	0.003543	0.002143
Q99	2.912e-05	-10.44	0.003888	0.002302
Q99.865	3.173e-05	-10.36	0.004525	0.002592
Q99.9	3.199e-05	-10.35	0.004568	0.002611

Frequently-Asked Questions

- Does Sigma Amplification depend on the technology node?

No. It is independent of the technology node.

- Does Sigma Amplification depend on the number of measures?

It is independent of the number of measures.

The post-processing time and the file size are acceptable even for circuits with ~3K measures.

- Does Sigma Amplification work with all circuit types and measures?

It works very well when the response distribution is close to Gaussian.

It has been found to work well in many cases even when the distribution departs significantly from Gaussian but is continuous.

It currently does not work for discrete measures like pass/fail. As far as possible, use continuous measures.

- Does Sigma Amplification depend on circuit size?

For the most part, it is independent of circuit size and the number of IRVs, but care is required to avoid excessive number of parameter out-of-bound errors.

Also, larger circuits take longer to simulate.

- How are the confidence bands created on the Q-Q plot in the *.json_gz file?

The confidence bands are based on the relation between order statistics and the beta distribution, and are exact.

- Do the confidence bands depend on the technology node and circuit type?

The confidence bands depend only on sample size and the amplification factor.

- Can I use a very small sample size, say 30?

Small sample size is not recommended because the confidence bands will be very large and the simulation results will not be reliable.

- Can I use a large Sigma Amplification factor to reduce the sample size drastically?

While the Y-axis range on the Q-Q plot increases with a higher amplification factor, the confidence interval width is also a function of the amplification factor and will be wider for a large amplification factor.

Defining Correlated Normal Distributed Random Variables in Monte Carlo Simulation

The `.COVAR` command defines the correlated random variables in statistic model by correlation coefficients for Monte Carlo simulation.

```
.COVAR [model = <model/subckt>] [instance = <list of instances>] param =  
  <list of parameters> value = <value>
```

This command addresses three scenarios:

- `.covar param = <list of parameters> value = <value>`
- `.covar model = <model/subckt> param = <list of parameters> value = <value>`
- `.covar instance = <list of instances> [param = <list of parameters>] value = <value>`

Arguments	Description
param	Random parameter name of AGAUSS/GAUSS function. This parameter is optional only for instance keyword. Otherwise, this is required.

Arguments	Description
value	Real value of correlation coefficient with absolute value not more than 1.
model	Device model name or subckt name of macro statistic model.
instance	Device name or x element name

Note:

- The parameter listed in the `.cover` command should be called only once in certain expression. For example, if `.param rand1=agauss(0,1,1)`, then `rand1` should not be used in other expression.

- `param` is optional only when instance is set. This is applicable in scenario 3.

- The command line of the first scenario can be defined in top level or inside `subckt`. If it is defined in top level, then random parameters should also be used in top level. It means correlation between global random variables.

If it is defined inside the `subckt`, random parameters should be called inside `subckt`. then it is equivalent to the second scenario with model name of the `subckt` name.

- The Command line of the second scenario is used in top level for local variation. Random parameters should be called inside `subckt` or model card with `modmonte=1`.
- For first and second scenarios, if the parameter number N is more than two, then it means correlations between all pairs of such parameters, that is $C(N,2)$ pairs. If model/subckt name is used, then such parameter pairs are for each instance of the model/subckt.
- For third scenario, correlation is between instances on the same parameters. If the instance (including X instance) number N is more than two, then it means correlations between all pairs of such instances, that is $C(N,2)$ pairs. If no following parameter is listed, then it means correlation between instances on each random parameter of corresponding model/subckt.

Examples

Example 1

```
.param rand1=agauss(0,1,1) rand2=agauss(0,1,1) rand3=agauss(0,1,1)
.param par1=rand1 par2=rand2 par3=rand3
.cover param=rand1,rand2 value=0.5
```

This refers to correlation between global random variables of `par1:@:rand1:@:IGNC` and `par2:@:rand2:@:IGNC`.

If `.covar param=rand1,rand2,rand3 value=0.5`, then

It refers to correlations of 3 global random variable pairs with the same correlation coefficient 0.5:

```
par1:@:rand1:@:IGNC and par2:@:rand2:@:IGNC
par1:@:rand1:@:IGNC and par3:@:rand3:@:IGNC
par2:@:rand2:@:IGNC and par3:@:rand3:@:IGNC
```

Example 2

```
.param rand1=agauss(0,1,1) rand2=agauss(0,1,1)
.subckt nch_mac d g s b
.param par1=rand1 par2=rand2
.covar param=rand1,rand2 value=0.5
...
.ends
x1 1 2 3 4 nch_mac
$.covar model=nch_mac param=rand1,rand2 value=0.5
```

It refers to correlation between local random variables of

`x1.par1:@:nch_mac:@:rand1:@:ILNC` and `x1.par2:@:nch_mac:@:rand2:@:ILNC`

Here, the `.covar` command can be defined inside `subckt`, or moved outside `subckt` with model name added.

Example 3

```
.option modmonte=1
.param rand1=agauss(0,1,1) rand2=agauss(0,1,1)
.model nch_mac nch vth0='rand1' u0='rand2' ...
.covar model=nch_mac param=rand1,rand2 value=0.5
m1 1 2 3 4 nch_mac
```

It refers to correlation between local random variables of

`m1:@:nch_mac:@:rand1:@:ILNC` and `m1:@:nch_mac:@:rand2:@:ILNC`

Example 4

```
.param rand1=agauss(0,1,1) rand2=agauss(0,1,1)
.subckt nch_mac d g s b
.param par1=rand1 par2=rand2
...
.ends
x1 1 2 3 4 nch_mac
x2 5 6 7 8 nch_mac
.covar instance=x1,x2 param=rand1,rand2 value=0.5
```

It refers to correlations of two local random variable pairs:

```
x1.par1:@:nch_mac:@:rand1:@:ILNC and x2.par1:@:nch_mac:@:rand1:@:ILNC  
x1.par2:@:nch_mac:@:rand2:@:ILNC and x1.par2:@:nch_mac:@:rand2:@:ILNC
```

Output

In the mc0 file, random variable names are not changed. But the correlated samples are printed. So the mc0 file does not show only samples of independent random variables any more.

If the correlation matrix is not positive semi-definite (PSD), PrimeSim HSPICE errors out with message of PSD failure.

External Sampling

The data of the mc0 file can still be used for external sampling. The `.covar` command in the netlist are ignored in Monte Carlo simulation with external correlated sampling.

Troubleshooting Monte Carlo-VB Issues

Troubleshoot this known issues as described in the following section:

- [Independent Random Variable Assignments](#)

Independent Random Variable Assignments

Users may come across differences in the results following multiple Monte Carlo simulations without changing the `SEED` setting if they have made small changes to the netlist without actually changing the circuit. The following describes the way PrimeSim HSPICE assigns random values to the independent random variables to explain how those differences can arise.

Let two random variables be defined in case 1:

```
.Variation  
  .Spatial_Variation  
    Parameter a = N( )  
    Parameter b = N( )  
    Parameter Slope = 'a/50u'  
    Parameter Pi = 3.14159265  
    Parameter Angle = 'Pi*2*b'  
R rmodel rsh=Perturb('Slope*sqrt(Get_E(x) * Get_E(x) + Get_E(y) * Get_E(y))  
  \\  
    *cos(Angle-atan(Get_E(y)/Get_E(x)) - (Get_E(x)<0?Pi:0))')
```

```
.End_Spatial_Variation
.End_Variation
```

Random variables *a* and *b* are used to calculate variations in sheet resistivity as a function of a resistor's coordinates.

Let case 2 have four random variables defined:

```
.Variation
.Spatial_Variation
    Parameter a = N( )
    Parameter b = N( )
    Parameter c = N( )
    Parameter d = N( )
    Parameter Slope = 'a/50u'
    Parameter Pi = 3.14159265
    Parameter Angle = 'Pi*2*b'
R rmodel rsh=Perturb('Slope*sqrt(Get_E(x)* Get_E(x)+ Get_E(y)*
Get_E(y))\\
    *cos(Angle-atan(Get_E(y)/Get_E(x))-(Get_E(x)<0?Pi:0))')
.End_Spatial_Variation
.End_Variation
```

Random variables *a* and *b* are used in the Variation Block; *c* and *d* are not used.

During the Monte Carlo sweep, a pseudo random number generator creates an array of random values (*Random1*, *Random2*, ..., *RandomN*) and assigns them to each an independent random variable.

In case 1, the random number assignment is:

```
Monte=1 -- a=Random1 b=Random2
Monte=2 -- a=Random3 b=Random4
...
Monte=N -- a=Random2N-1 b=Random2N
```

In case 2, the random number assignment is:

```
Monte=1 -- a=Random1 b=Random2 c=Random3 d=Random4
Monte=2 -- a=Random5 b=Random6 c=Random7 d=Random8
...
Monte=N -- a=Random4N-3 b= Random4N-2 c=Random4N-1 d=Random4N
```

Here, although *Random1* through *RandomN* are the same in both cases, the sequence of assignment to independent random variables differs. Hence, the individual samples of *a* (or *b*) differ between the two simulations. As a consequence, at low sample numbers the difference in the standard deviation (*sigma*) of the distributions of *a* (or *b*) might be quite large. For higher sample numbers, the differences get smaller, according to the general convergence rate of Monte Carlo results of $\frac{1}{\sqrt{n}}$ where *n* is the number of samples.

Because a pseudo random number generator is used in PrimeSim HSPICE, repeated simulations generate the same set of statistical results for a given set of independent random variables. The user can change the random number sequences at each run by defining in the Variation Block: `Option Stream = val` where *val* is an integer between one and twenty.

In the traditional Monte Carlo style, this is the similar setting to:

```
.option seed=val
```

When using Monte Carlo simulation, you should keep in mind that there is always uncertainty associated with this method in the relationship of one sample to the overall population.

References

- [1] V. Czitrom: One-Factor-at-a-Time Versus Designed Experiments. The American Statistician, pp.126-131, May 1999.
- [2] M.D. McKay, R.J. Beckman, and W.J. Conover: A Comparison of Three Methods for Selecting Values of Input Variables in the Analysis of Output from a Computer Code. Technometrics, pp. 239-245, 1979.
- [3] M. Stein: Large Sample Properties of Simulations Using Latin Hypercube Sampling. Technometrics, pp. 143-151, 1987.
- [4] I.M. Sobol: On the Systematic Search in a Hypercube. SIAM J. Numerical Analysis, pp. 790-793, 1979.
- [5] H. Niederreiter: Random Number Generation and Quasi-Monte Carlo Methods. Society for Industrial and Applied Mathematics, Philadelphia, PA, 1992.
- [6] A. Singhee and R.A. Rutenbar: Why Quasi-Monte Carlo is Better than Monte Carlo or Latin Hypercube Sampling for Statistical Circuit Analysis. IEEE Trans. on Computer Aided Design of Circuits and Systems, pp. 1763-1776, November, 2010.

27

Mismatch Analyses

Describes the use of DC and AC mismatch analyses in PrimeSim HSPICE.

DCMatch and ACMatch analyses are efficient techniques for computing the effects of variations on a circuit's DC or AC response. The variation definitions are taken from the Variation Block or from the traditional format (commonly known as “AGAUSS” format). Both methods are small signal analyses, similar to noise analysis. Unlike the traditional Monte Carlo analysis, these methods do not rely on sampling, and are therefore significantly faster. The Monte Carlo results converge to those from DCMatch or ACMatch analysis for a large number of samples, provided that the circuit characteristics are close to linear in the parameter perturbations. Matching analyses are supported with the Variation Block format and the AGAUSS style.

DCMatch and ACMatch analyses are affected by the control options specified in the Variation Block (see [Control Options and Syntax](#) in [Statistical Representation Using the Variation Block](#)).

PrimeSim HSPICE ships many examples for your use. See Variability Examples in the *PrimeSim HSPICE User Guide: Demonstration Netlists* for paths to the DCMatch and ACMatch demo files.

This chapter discusses the following topics:

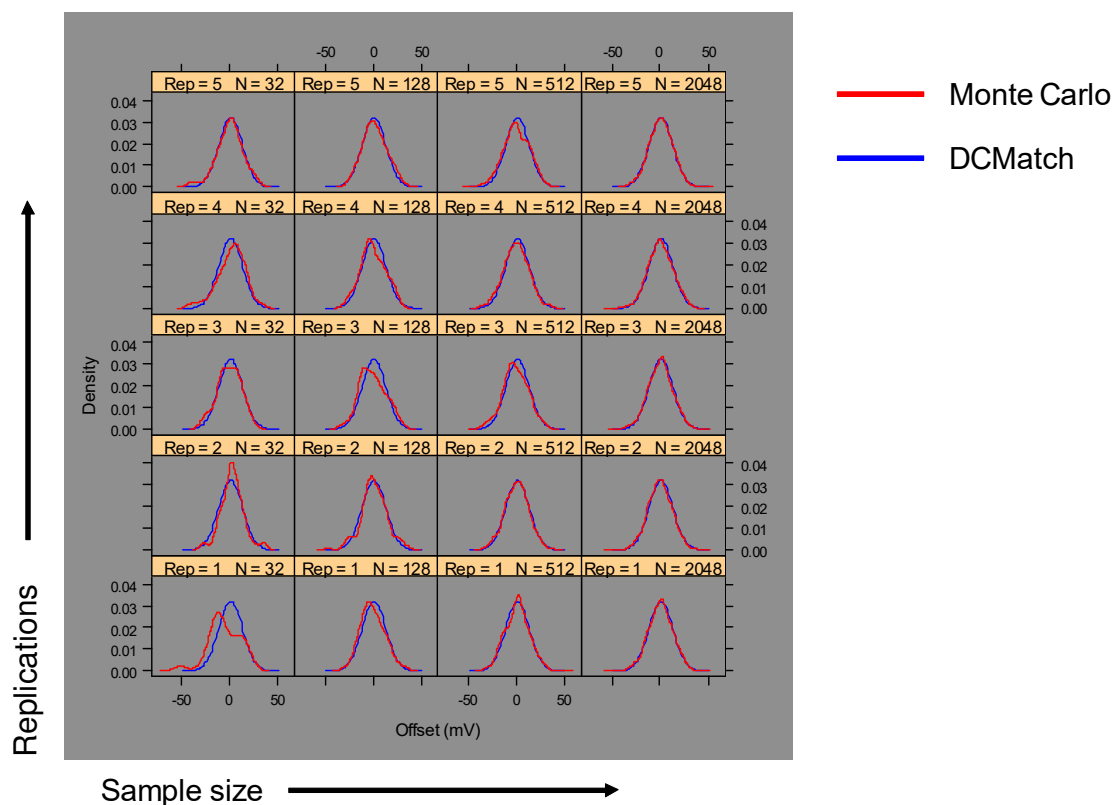
- [Mismatch](#)
- [DCMatch Analysis](#)
- [DC Sensitivity Analysis](#)
- [ACMatch Analysis](#)
- [Application Considerations](#)
- [Mismatch Compared to Monte Carlo Analysis](#)
- [Supported Models with DCMatch and ACMatch](#)
- [References](#)

Mismatch

Variations in materials and processing steps are the source of differences in the characteristics of identically designed devices in close proximity on the same integrated circuit. These are random time-independent variations by nature and are collectively called *mismatch*.

Mismatch is one of the key limiting factors in analog signal processing. It affects more and more circuit types as device dimensions and signal swings are reduced. Mismatch is a function of the geometry of the devices involved, their spatial relationship (distance and orientation), and their environment.

Figure 164 DCMatch and Monte Carlo Comparative Results



DCMatch Analysis

To observe the effects of variation on the DC response of a circuit, you can use a method called DC mismatch (DCMatch) analysis.

In DCMatch analysis, the combined effects of variations of all devices on a specified node voltage or branch current are determined. The primary purpose is to consider the effects of Local variations (that is, for devices in close proximity). DCMatch analysis also allows for identifying groups of matched devices (that is, devices that should be implemented on the layout according to special rules). A secondary set of results is calculated from the influences of Global and Spatial Variations, which is useful for investigating whether their effects on circuit response are much smaller than the effects of Local variations, when optimizing a design.

DCMatch analysis is based on the following dependencies and assumptions:

- Variations in device characteristics are modeled through variations in the underlying model parameters.
- Effects on a circuit's DC solution are small and can be modeled as a linear combination of the variations in the random variables.

In PrimeSim HSPICE, the variations in model parameters are defined in the Variation Block (see [Chapter 25, Statistical Representation Using the Variation Block](#)) or through the traditional Agauss style variation representation. Those definitions are used to calculate the variation in DC response. DCMatch analysis runs either from a default operating point or for each value of the independent variable in a DC sweep. The default output is in the form of tables containing the sorted contributions of the relevant devices to the total variation, as well as information on matched devices. In the current implementation, a heuristic algorithm makes a best guess effort to identify matched devices. This means that the results are suggestions only. In addition to the table, the total variation and contributions of selected devices can be output using `.PROBE` and `.MEASURE` commands.

The following sections discuss these topics:

- [Input Syntax](#)
- [Examples](#)
- [DCMatch Table Output](#)
- [PrimeSim HSPICE Ignores Limit Distributions in DCMatch](#)
- [Output Using `.PROBE` and `.MEASURE` Commands](#)
- [DCMatch Example Netlist](#)

Input Syntax

```
.DCMatch OUTVAR [THRESHOLD=T] [FILE=string] [INTERVAL=Int]
```

Parameter	Description
OUTVAR	One or more node voltages, voltage differences for a node pair, currents through an independent voltage source or a resistor.
THRESHOLD	Report devices with a relative variance contribution above Threshold in the summary table. • T=0: reports results for all devices • T<0: suppresses table output; however, individual results are still available through .PROBE or .MEASURE statements. The upper limit for T is 1, but at least 10 devices are reported, or all if there are less than 10. Default value is 0.01.
FILE	Valid file name for the output tables. Default is <code>basename.dm#</code> where “#” is the usual sequence number for PrimeSim HSPICE output files.
INTERVAL	Applies only if a DC sweep is specified. <i>Int</i> is a positive integer. A summary is printed at the first sweep point, then for each subsequent increment of <i>Int</i> , and then, if not already printed, at the final sweep point.

Note:

If more than one DCMatch analysis is specified per simulation, only the last statement is used.

Examples

Example 1

In this example, PrimeSim HSPICE reports DCMatch variations on the voltage of node 9, the voltage difference between nodes 4 and 2, on the current through the source `VCC`, and the current through resistor `x1.r1`.

```
.DCMatch V(9) V(4,2) I(VCC) I(x1.r1)
```

Example 2

In this example, the variable `XVal` is being swept in the DC command from 1k to 9k in increments of 1k. DCMatch variations are calculated for the voltage on node `out`. Tables with DCMatch results are generated for the set `XVal={1K, 4K, 7K, 9K}`.

```
.DC XVal Start=1K Stop=9K Step=1K
.DCMatch V(out) Interval=3
```

DCMatch Table Output

For each output variable and sweep point, PrimeSim HSPICE generates a result record that includes setup information, total variations, and a table with the sorted contributions of the relevant devices. The individual entries are:

- Sweep or operating points for which the table is generated
- Name of the output variable
- DC value of this output variable
- Values used for DCMatch options
- Output sigma due to combined Global, Local, and Spatial variations

$$\sqrt{\sigma_{\text{global}}^2 + \sigma_{\text{local}}^2 + \sigma_{\text{spatial}}^2}$$

- Results for Global variations (similar to the specifics of Local Variation)
- Results for Local variations:
 - Number of devices that had no local variability specified
 - Output sigma due to Local variations
 - Number of devices with local variance contributions below the threshold value and not included in the table
 - Table with sorted device contributions

Contribution sigma (in volts or amperes). Values below 100 nV or 1 pA are rounded to zero to avoid reporting numerical noise.

- Contribution variance for i th parameter (in percent)

$$\frac{\sigma(i)^2}{\sum_1^n \sigma(k)^2} \times 100$$

- The parameter “Threshold” applies to this column.
- Cumulative variance through i th parameter (in percent)

$$\frac{\sum_1^i \sigma(k)^2}{\sum_1^n \sigma(k)^2} \times 100$$

- Results for Spatial variations are similar to the previous item, Local Variation.

Example: Simple Op-Amp and DCMatch Output Table Described

Figure 165 Simple Op-Amp

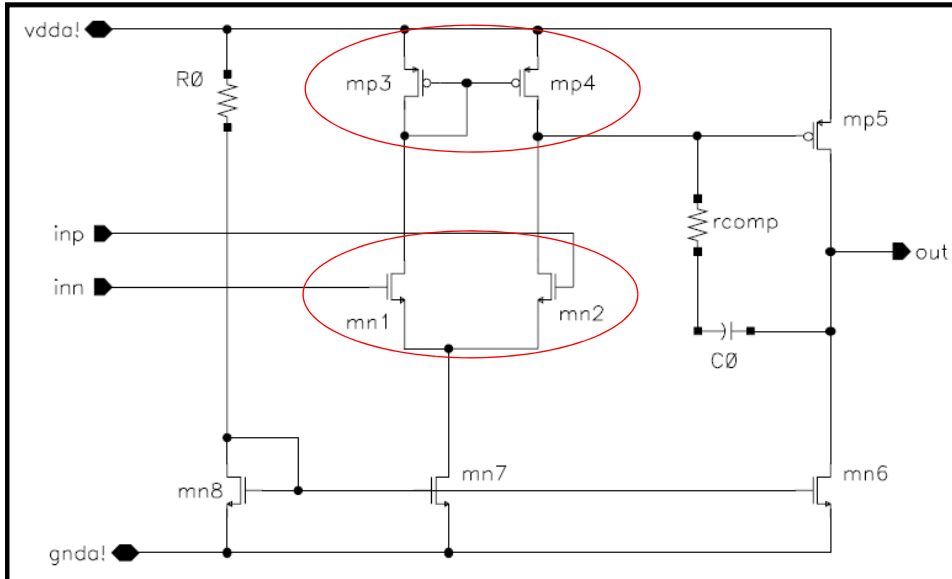


Figure 166 Sample Output Table

Output sigma due to global and local variations = 619.62uV				
DCMATCH GLOBAL VARIATION				
Output sigma due to global variations = 289.66uV				
Contribution	Contribution	Cumulative	Independent	
Sigma (V)	Variance (%)	Variance (%)	Variable	
227.94u	61.92	61.92	snps20p:@:u0	
139.48u	23.19	85.11	snps20p:@:vth0	
109.93u	14.40	99.51	snps20n:@:u0	
20.19u	485.62m	100.00	snps20n:@:vth0	
DCMATCH LOCAL VARIATION				
Output sigma due to local variations = 547.74uV				
Contribution	Contribution	Cumulative	Matched	Device
Sigma (V)	Variance (%)	Variance (%)	pair	Name
297.91u	29.46	29.46	1	mn1
296.38u	29.16	58.61	1	mn2
252.37u	21.14	79.75	2	mp3
246.91u	20.23	99.99	2	mp4

The output table in [Figure 166](#) shows the major sections of a table created by DCmatch analysis in the following order:

- The DCmatch results from an operating point, for the node called “out”, which has an operating point voltage of 1.25 V.
- The standard deviation (sigma) of the variation on the specified output, due to combined global and local variations.
- A section with detailed results for the global variations. First is the output sigma due to global variations.
- The contribution of the different variables and device parameters, as specified in the Variation Block.

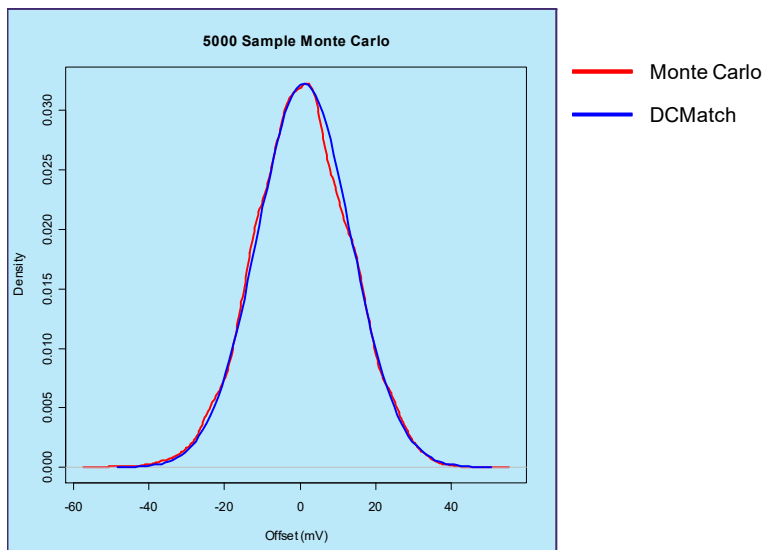
Note:

The sum of squares of the contribution sigmas adds up to the square of the total output sigma.

- The results for the contribution and cumulative variance are reported as percent of the total variance (which is the square of the total output sigma). In the example shown, the variation in Vth0 for the model snps20p is the largest contributor to the variation of the output.
- The detailed results for local variations, starting with the output sigma due to local variations.
- The contribution of the different devices. A column with matched pair information indicates that mn1 and mn2 are matched, as well as mp3 and mp4.

The table also includes a suggestion on matched devices that should be verified independently. Devices with the same number in the column “Matched pair” are likely to be matched. Their layout should be reviewed for conformity to established matching rules.

Figure 167 DCMatch and Monte Carlo



PrimeSim HSPICE Ignores Limit Distributions in DCMatch

PrimeSim HSPICE calculates the output sigma using the following equation:

$$\sigma^2 f = \sum \left((\partial f) / (\partial p_i) \right)^2 \sigma^2 p_i$$

In addition to this equation, DCMatch assumes that the metric is differentiable because it is calculus-based. The `Limit` function is often used to drive conditional statements. This makes the metric non-differentiable.

To avoid listing improper results for DCMatch, PrimeSim HSPICE issues a warning message and ignores DC/ACMatch, if variations have limit distributions. The advantages of DCMatch are (a) it is fast, and (b) results are expected to be close to what one would get with Monte Carlo. In the case of the limit distribution, condition (b) is likely to be violated.

Output Using .PROBE and .MEASURE Commands

Depending on the output variable specified on the `.DCMatch` command, results produced by DCMatch analysis can be saved by using `.PROBE` and `.MEASURE` commands (see syntax and examples that follow). If multiple output variables are specified, a result is produced for the last variable only. A DC sweep needs to be specified to produce these kinds of outputs; a single point sweep is sufficient.

The keywords available for saving specific results from DCMatch analysis are:

Table 74 *Keyword descriptions from DCMatch Analysis*

Keyword	Description
DCM_Total	Output sigma due to Global and Local variations.
DCM_Global	Output sigma due to Global variations.
DCM_Global (par)	Contribution of parameter “par” to output sigma due to Global variations. Here, 'par' can be an independent variable or a model parameter.
DCM_Local	Output sigma due to Local variations.
DCM_Local (dev)	Contribution of device “dev” to output sigma due to Local variations.
DCM_Spatial	Output sigma due to Spatial Variations.
DCM_Spatial (var)	Contribution of independent variable “var” to output sigma due to Spatial Variations.

Syntax for .PROBE Command for DCMatch

A `.PROBE` statement in conjunction with `.OPTION POST` creates a data file with waveforms that can be displayed in WaveView.

```
.PROBE DC DCM_Total
.PROBE DC DCM_Global
.PROBE DC DCM_Local
.PROBE DC DCM_Global (VariableName)
.PROBE DC DCM_Global (ModelType, ModelName, ParameterName)
.PROBE DC DCM_Local (InstanceName)
.PROBE DC DCM_Spatial
.PROBE DC DCM_Spatial (VariableName)
```

This type of output is useful for plotting the effects of mismatch as a function of bias current, temperature, or a circuit parameter.

Examples

In the first example, the contribution of the variations on vth0 (threshold) of the nmos devices with model SNPS20N is saved. In the second example, the contribution of device mn1 in subcircuit X8 is saved.

```
.Probe DCM_Global (nmos, SNPS20N, vth0)
.Probe DCM_Local (X8.mn1)
```

Syntax for .MEASURE Command

With `.MEASURE` statements, PrimeSim HSPICE performs measurements on the simulation results and saves them in a file with an `*.ms#` extension.


```
.MEAS DC res1 max DCM_Total
.MEAS DC res2 max DCM_Global
.MEAS DC res3 max DCM_Local
.MEAS DC res4 max DCM_Global(VariableName)
.MEAS DC res5 max DCM_Global(ModelType,ModelName,ParameterName)
.MEAS DC res6 max DCM_Local(InstanceName)
.MEAS DC res7 find DCM_Local at=SweepValue
.MEAS DC res8 find DCM_Local(InstanceName) at=SweepValue
.MEAS DC res9 max DCM_Spatial
.MEAS DC res10 find DCM_Spatial(VariableName) at=SweepValue
```

The keywords `DCM_total`, `DCM_global`, and `DCM_local` also support use of AGAUSS type macro models. For example:

```
.MEAS DC dcm_mn2 find DCM_local(xi82.xmn2.main) at=2
```

Example

In this example, the result `systoffset` reports the systematic offset of the amplifier, the result `matchoffset` reports the variation due to local mismatch, and the result `maxoffset` reports the maximum (3-sigma) offset of the amplifier.

```
.MEAS DC systoffset avg V(inp,inn)
.MEAS DC matchoffset avg DCM_Local
.MEAS DC maxoffset param='abs(systoffset)+3.0*matchoffset'
```

DCMatch Example Netlist

An example netlist for running DCMatch analysis using a classic 8-transistor CMOS operational amplifier is available in the PrimeSim HSPICE demo directory as `$installdir/demo/hspice/variability/opampdcm.sp`.

In this netlist, device sizes are set up as a function of a parameter `k`, which allows for investigating the effects of the Global and Local Variations as a function of device size.

The following lines relate to DCMatch analysis:

```
...
.param k=2
...
mn1 net031 inn net044 nmosbulk snps20N L='k*0.5u' W='k*3.5u' M=4
mn2 net18 inp net044 nmosbulk snps20N L='k*0.5u' W='k*3.5u' M=4
mp3 net031 net031 vdda pmosbulk snps20P L='k*0.5u' W='k*4.5u' M=4
mp4 net18 net031 vdda pmosbulk snps20P L='k*0.5u' W='k*4.5u' M=4
...
.DCMatch v(out)
.dc k start=1 stop=4 step=0.5
...
.meas DC systoffset find V(in_pos,in_neg) at=2
.meas DC dcmoffset find DCM_Local at=2
.meas DC maxoffset param='abs(systoffset)+3.0*dcmoffset'
```

```
.meas DC dcm_mn2 find DCM_Local(xi82.mn2) at=2
.meas DC gloffset find DCM_Global at=2
.option post
...
```

The DCMATCH analysis produces four types of output from this netlist:

- Table from operating point with $k=2$ in the output listing
- Table from DC sweep for $k=1$ to 4 in file `opampdcm.dm0`
- Waveform for output variation as a function of k in file `opampdcm.sw0`
- In file `opampdcm.sw0` for $k=2$:
 - Values for systematic offset
 - Output sigma due to Local Variation
 - 3-sigma amplifier offset
 - Contribution of device `mn2` to output sigma due to Local Variation
 - Output sigma due to Global Variation

DC Sensitivity Analysis

DC sensitivity analysis allows you to do the following tasks:

- Compute sensitivity of a model parameter.
- Compute parameter sensitivity of many more models than the traditional PrimeSim HSPICE `.SENS` command.
- Generate sensitivity for `.PROBE` and `.MEASURE` output commands.
- Generate sensitivity for DC sweeps.

DC sensitivity analysis is supported in conjunction with a single DC sweep, or DC sweep and Monte Carlo.

This section covers the following topics:

- [Sensitivity Block Using the Variation Block Construct](#)
- [Input Syntax](#)

Sensitivity Block Using the Variation Block Construct

The Variation Block serves as the default sensitivity block. In other words, the DC sensitivity analysis calculates the sensitivity for model and element parameters, which are specified in the Variation block.

Sensitivity to Local Variations

Sensitivity is reported by varying one parameter on one device at a time. So it is identified by device and parameter:

$$\text{SENS_Local}(p_1, k)$$

where, k is the device and p_1 is the parameter.

Sensitivity to Global Variations

Sensitivity is reported for varying one parameter simultaneously on all devices. So it is identified by model type, model, and parameter according to this equation:

$$\text{SENS_Global}(T, M, p_1) = \text{SUM}_k \text{SENS_Local}(p_1, k)$$

where k loops over all devices with the same model type T and model M . p_1 is the parameter.

Input Syntax

```
.DCSENS Output_Variable [File=string] [Perturbation=x]  
+ [Interval=SweepValue] [Threshold=x] [GroupByDevice=0|1]
```

Here, `Output_Variable` is the response with respect to the parameters designated in the Sensitivity Block. It can be a node voltage or branch current in the circuit.

Argument	Description
Output_Variable	Response with regard to the parameters designated in Sensitivity Block. Similar to the <code>.DCMATCH</code> command, the <code>Output_Variable</code> can be a node voltage or a branch current in the circuit.
File=string	Valid file name for the output tables. Default= <code>basename.ds#</code> where “#” is a number in the style of <code>ds0</code> , <code>ds1</code> , and so on. If multiple <code>dcswEEP</code> commands are specified in the netlist, then the sensitivity analysis table results for each <code>dcswEEP</code> are listed in <code>*.ds#</code> files. If <code>.OPTION OFFILE</code> is specified, sensitivity result tables on operating points are listed in <code>*.dp#</code> files. Otherwise, these tables are in the <code>*.lis</code> file.
Perturbation=x	Perturbations of x standard deviation used in computing the finite difference approximations to device derivatives. The valid range for the parameter is 0.0001 to 1.0; default =0.05.

Argument	Description
Interval=SweepValue	Positive integers for SweepValue. This option only applies to one-dimensional sweeps. A summary is printed at the first sweep point, and for each subsequent increment of SweepValue. The Interval key is ignored with a warning if a sweep is not being carried out. The option only controls the printed summary table. The analysis may be carried out at additional sweep values if required by other forms of output such as <code>.PROBE</code> and <code>.MEASURE</code> commands.
Threshold=x	Minimum value for reporting of absolute sensitivity. Only devices with absolute sensitivity value above x are reported. Results for all devices are displayed if Threshold=0 is set; default=10u.
GroupByDevice = 0 1	• 0: (Default) Alternate mode of generating sensitivity result tables • 1: Table form follows the <code>.SENS</code> command output.

ACMatch Analysis

In ACMatch analysis, the combined effects of variations of device characteristics on the frequency response of a circuit are determined. The variation definitions are taken from the Variation Block or from the traditional GAUSS style variation specification.

The main application for ACMatch analysis is in the simulation of circuits which are sensitive to parasitics or require matching of parasitics, for characteristics such as delays and power supply rejection.

ACMatch analysis takes the changes in frequency response due to variations in DC parameters (which affect operating point and low frequency response, as well as bias-dependent capacitors) and due to variations in AC parameters. Note that variation on the stimuli (voltage and current sources) can be specified on the DC and AC parameters, and both types are considered in the ACMatch analysis.

ACMatch analysis is similar to DCMatch analysis in that:

- It is efficient compared to Monte Carlo analysis because there is no sampling involved.
- Variations in component characteristics are modeled through variations in the underlying model parameters.
- Effects on a circuit's DC solution are small, and can be modeled as a linear combination of the variation in independent random variables. This is relevant for ACMatch analysis because the changes in the DC solution affect the circuit's AC characteristics.

ACMatch analysis is specified with an AC analysis, which defines the frequencies for which the circuit is analyzed; this can be at single or multiple sweep points. At least one measure or other output statement is required for this AC analysis, and subsequently ACMatch analysis, to run. The primary output of ACMatch analysis is a table with the sorted parameter and device contributions.

The following sections discuss these topics:

- [Parasitic Capacitor Sensitivity](#)
- [Input Syntax](#)
- [Examples](#)
- [ACMatch Table Output](#)
- [Output from .PROBE and .MEASURE Commands for ACMatch](#)
- [ACMatch Analysis Examples](#)

Parasitic Capacitor Sensitivity

The design of a network usually begins with a study in which all the parasitic elements are neglected. Later, if needed, the network is re-simulated with the parasitic elements and its behavior compared with the original design. Such analysis is useful for high precision (differential) analog circuits and switched capacitor filters, because they are quite sensitive to layout parasitics, but their values are not known at the pre-layout stage. Just as DC and ACMatch are useful for identifying critical devices that can then be re-sized or their layout constructed carefully, designers can evaluate similar feedback about parasitic components.

The Virtual Capacitance table calculates such parasitic capacitor sensitivities whose nominal values are 'zero' in the original design.

Input Syntax

```
.ACMatch OUTVAR [THRESHOLD=T] [FILE=string] [INTERVAL=Int]
+ [Virtual_Sensitivity=Yes|No] [Sens_threshold=x]
+ [Sens_Node=(nodei_name,nodej_name),..., (nodem_name,noden_name)]
```

Parameter	Description
OUTVAR	Output Variable can be one or several output voltages, voltage differences, branch current through an independent voltage source, or currents through a resistor, a capacitor, or an inductor. An identifier of the AC quantity of interest is followed by the voltage/current specifier: • M magnitude • P phase • R real part • I imaginary part

Parameter	Description
THRESHOLD	Report devices with a relative variance contribution above Threshold in the summary table. - T=0: reports results for all devices - T<0: suppresses table output; however, individual results are still available through .PROBE or .MEASURE statements. The upper limit for T is 1, but at least 10 devices are reported, or all if there are less than 10. Default: 0.01.
FILE	Valid file name for the output tables. Default is <code>basename.am#</code> where “#” is the usual sequence number for PrimeSim HSPICE output files.
INTERVAL	Relates to the associated AC sweep. <i>Int</i> is a positive integer. A summary is printed at the first sweep point, then for each subsequent increment of <i>Int</i> , and then, if not already printed, at the final sweep point.
Virtual_Sensitivity	Invokes ACmatch computation and output of virtual sensitivity. Default: Yes
Sens_Threshold=x	Only nodes with sensitivity above x are reported. At least 10 sensitivities (or all) are displayed. This avoids generation of null output if you specify too large a value for x. Default: 1e-6
Sens_Node	Output all sensitivities associated with the requested nodes. The node name should appear in pairs. (See examples below.)

If more than one ACMatch analysis is specified per simulation, only the last statement is executed.

Examples

```
.ACMatch VM(out) VP(out) IM(x1.r1) IP(x1.r1) IM(c1) IP(c1)
.AC dec 10 1k 10Meg interval=10
```

When using the virtual sensitivity option `Sens_Node`, multiple name pairs are supported with a comma between node names, and between node name pairs.

```
.ACmatch v(out) virtual_sens=yes
+ sens_node= (out, xi82.net044),
+ (0,out), (xi82.net044,xi82.net031) sens_threshold=1e-6
```

ACMatch Table Output

For each output variable and sweep point, PrimeSim HSPICE generates a result record in a file with default extension `.am#`. This file includes setup information, a main result with the total variations, and two tables. One table includes the sorted contributions of the relevant devices and parameters, and the other table includes virtual capacitance data.

The individual entries for the mismatch table include:

- Frequency sweep value
- Name of the output variable
- AC magnitude of this output variable
- ACMatch option values
- Number of devices which had no variability specified
- Output sigma values due to combined Global and Local Variations
- Result for Global Variations
- Contributions of parameters to Global Variations
- Results for Local Variations
- Contributions of devices to Local Variations

The entries in the different columns correspond to those described in the section on [DCMatch Table Output on page 648](#).

To avoid printing unreliable results due to precision issues, phase output is not available in the table if the associated magnitude of the same variation type is less than 1 uV for voltage or 1 nA for current output. A warning is printed instead.

The entries for the virtual capacitance table include:

- The output value is printed with Re and Im components regardless of the request on the ACMatch command.
- The section header is a virtual capacitance table (per femto farad).
- The sensitivities are multiplied by 1e-15. This is the “natural” unit for measuring typical parasitic capacitors in nanometer technologies.

Example 1: Parasitic Capacitance Sensitivity

```
1 ***** HSPICE -- B-2008.09-TST 32-BIT (May 23 2008) sunos *****
          ***** AC mismatch analysis                      tnom= 25.000
temp= 25.000
frequency = 1.000000D+03
=====
output = v(out) node voltage =      1.82kV threshold = 1.000E-02
perturbation = 2.00      interval = 1 virtual_sensitivity = yes
sensitivity_threshold = 1.000E-06
Output 1-sigma due to total variations = 148.43 V
ACMATCH LOCAL VARIATION
      15 Devices had no Local Variability specified
Output 1-sigma due to local variations = 148.43 V
```

```

1 Devices with Local Contribution Variance larger than Threshold
-----
Contribution      Contribution      Cumulative      Device
1Sigma (V)        Variance (%)      Variance (%)    Name
48.43             100.00            100.00          xi82.ccomp
745.32m           2.52m             100.00          c1
4.82m             105.39n           100.00          c0
-----
Virtual capacitance table (per femto farad)
sens(real)        sens(imaginary)    (nodei, nodej)
196.06u           208.38u            (out, xi82.net058)
196.04u           208.40u            (out, xi82.net18)
-191.73u          -203.47u           (out, xi82.net031)
7.29u             8.21u              (xi82.net031, xi82.net18)
7.31u             8.19u              (xi82.net031, xi82.net058)
3.63u             4.19u              (in_pos, xi82.net18)
3.64u             4.18u              (in_pos, xi82.net058)
3.69u             4.05u              (xi82.net18, xi82.net0148)
3.70u             4.04u              (xi82.net0148, xi82.net058)
3.69u             4.05u              (gnda, xi82.net18)
3.69u             4.05u              (vdda, xi82.net18)
3.69u             4.05u              (xi82.net18, 0)
3.69u             4.05u              (in_neg, xi82.net18)
3.70u             4.04u              (gnda, xi82.net058)
3.70u             4.04u              (vdda, xi82.net058)
3.70u             4.04u              (xi82.net058, 0)
3.70u             4.04u              (in_neg, xi82.net058)
3.64u             4.06u              (xi82.net044, xi82.net18)
3.65u             4.06u              (xi82.net044, xi82.net058)
1.38u             1.52u              (out, xi82.net044)
171.31n           202.01n            (out, 0)
-12.88n           17.84n             (xi82.net031, xi82.net044)
*** ACmatch end for this simulation***

```

Note that the sensitivities of the last two rows are smaller than the threshold. However, they are still printed because these nodes are selected with the option `sens_node = (0,out) (xi82.net044,xi82.net031)`.

Example 2: Sensitivity Not Calculated

```

frequency = 1.00000D+06
=====
output = v(out) node voltage = 1.87 V threshold = 1.000E-02
perturbation = 2.00 interval = 1 virtual sensitivity = no

Output 1-sigma due to Global and local Variations = 48.68mV

ACMatch GLOBAL VARIATION
10 Devices had no Global Variability specified
Output 1-sigma due to Global Variations = 46.97mV

```


Contribution Independent	Contribution	Cumulative	
1Sigma (V)	Variance (%)	Variance (%)	Variable
38.89m	68.57	68.57	
snps20n:@:vth0			
15.78m	11.28	79.85	
snps20p:@:vth0			
15.08m	10.31	90.16	snps20n:@:tox
14.56m	9.61	99.77	snps20n:@:u0
1.80m	146.80m	99.91	snps20p:@:u0
1.38m	86.49m	100.00	snps20p:@:tox

ACMatch LOCAL VARIATION

6 Devices had no Local Variability specified
Output 1-sigma due to Local Variations = 12.79mV
7 Devices with Local Contribution Variance larger than Threshold

Contribution	Contribution	Cumulative	Device
1Sigma (V)	Variance (%)	Variance (%)	Name
7.43m	33.77	33.77	xi82.mn7
6.26m	23.97	57.73	xi82.mp4
6.20m	23.53	81.26	xi82.mp3
4.87m	14.49	95.75	xi82.mn8
1.53m	1.43	97.18	xi82.mn2
1.49m	1.36	98.54	xi82.mn1
1.40m	1.20	99.74	r1
563.27u	193.90m	99.93	xi82.mp5
239.10u	34.94m	99.97	xi82.rcomp
184.24u	20.75m	99.99	xi82.mn6

Output from .PROBE and .MEASURE Commands for ACMatch

The syntax of .MEASURE and .PROBE commands for ACMatch analysis is similar to the syntax for DCMatch analysis.

Syntax for .PROBE Command

A .PROBE statement in conjunction with .OPTION POST creates a data file with waveforms that can be displayed in WaveView.

```
.PROBE AC ACM_Total
.PROBE AC ACM_Global
.PROBE AC ACM_Local
.PROBE AC ACM_Global (VariableName)
.PROBE AC ACM_Global (ModelType, ModelName, ParameterName)
.PROBE AC ACM_Local (InstanceName)
```

Syntax for .MEASURE Command

With `.MEASURE` statements, PrimeSim HSPICE performs measurements on the simulation results and saves them in a file with a `*.ma#` extension.

```
.MEAS AC res1 max ACM_Total
.MEAS AC res2 max ACM_Global
.MEAS AC res3 max ACM_Local
.MEAS AC res5 max ACM_Global (VariableName)
.MEAS AC res6 max ACM_Global (ModelType,ModelName,ParameterName)
.MEAS AC res7 max ACM_Local (InstanceName)
.MEAS AC res8 find ACM_Local at=SweepValue
.MEAS AC res9 find ACM_Local (InstanceName) at=SweepValue
```

ACMatch Analysis Examples

Example 1

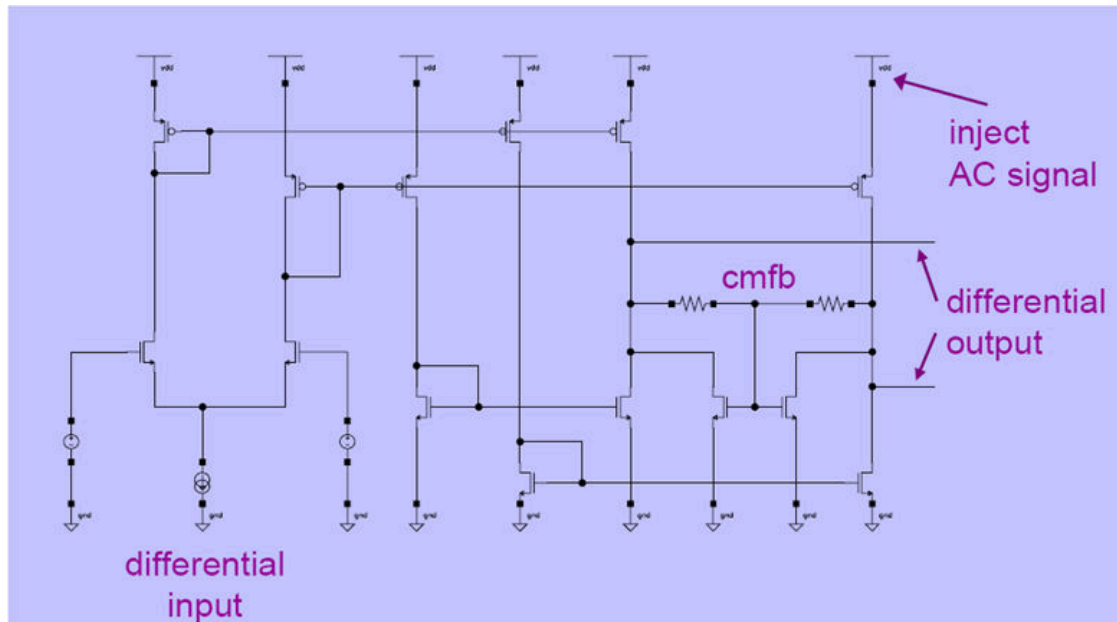
An example netlist for running ACMatch analysis using a classic 8-transistor CMOS operational amplifier is available in the PrimeSim HSPICE demo directory as `$installdir/demo/hspice/variability/opampacm.sp`. The following lines relate to ACMatch analysis:

```
.Variation
.Global_Variation
    Nmos snps20N vth0=0.07 u0=10 %   tox=3 %
    Pmos snps20P vth0=0.08 u0=8 %    tox=3 %
.End_Global_Variation
.Local_Variation
    Nmos snps20N vth0='1.234e-9 sqrt(Get_E(W)*Get_E(L)*Get_E(M))'
+      u0='2.345e-6/sqrt(Get_E(W)*Get_E(L)*Get_E(M))' %
+      tox='3.456e-6/sqrt(Get_E(W)*Get_E(L)*Get_E(M))' %
    Pmos snps20P
vth0='1.234e-9/          sqrt(Get_E(W)*Get_E(L)*Get_E(M))'
+      u0='2.345e-6/sqrt(Get_E(W)*Get_E(L)*Get_E(M))' %
+      tox='3.456e-6/sqrt(Get_E(W)*Get_E(L)*Get_E(M))' %
.Element_Variation
    R r=10 %
.End_Element_Variation
.End_Local_Variation
.End_Variation
.ACMatch v(out)
.ac dec 1 1k 10Meg
.meas ac res1 find acm_local at=1k
```

In this example, ACMatch analysis runs at 1 kHz, 10 kHz, 100 kHz, 1 MHz, and 10 MHz. After simulation, the results in `opampacm.am0` show the contributions of devices and parameters, and their different relative importance for the different frequencies. The measurement result is printed in `opampacm.ma0`.

Example 2

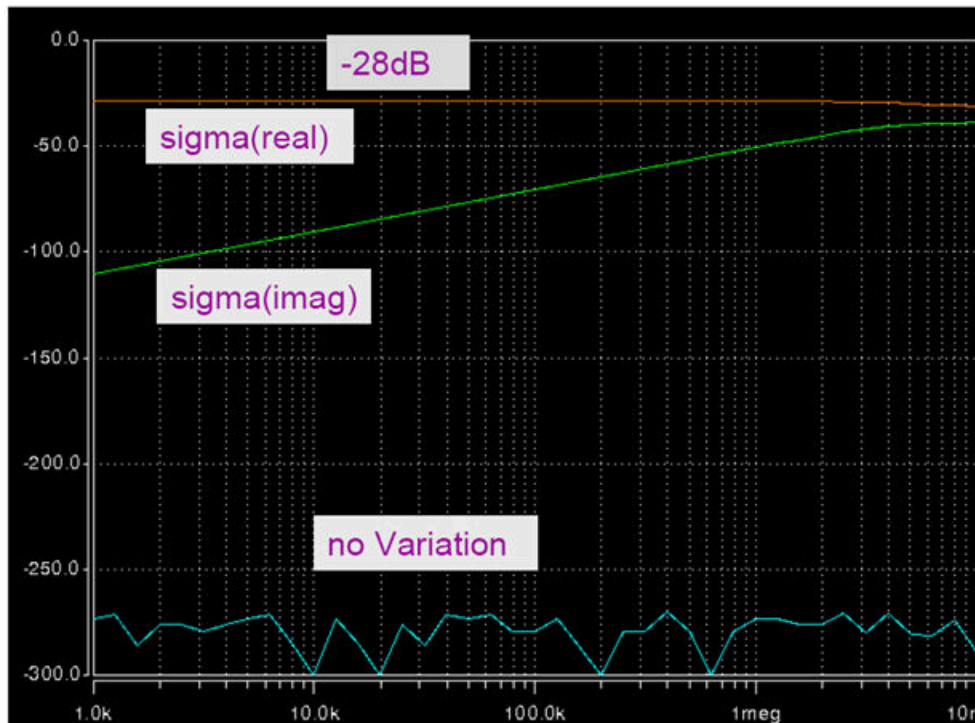
Figure 168 Fully Differential Amplifier: Calculate Effect of Variations on Power Supply Rejection and Feedthrough



A useful application of ACMatch analysis is the analysis of the effects of variation on the power supply rejection (or feedthrough) of differential circuits. The idea of differential circuits is to reduce the effects of common mode disturbances, due to variations in global device characteristics, bias currents, or supply variations. A major issue in mixed signal designs is the coupling of digital *noise* into the analog signal path.

The schematic in [Figure 168](#) shows a fully differential amplifier, with a simple common mode feedback circuit. For the ACMatch analysis, the AC signal is injected into the positive supply rail. Since the circuit is fully symmetrical, the differential output signal is zero with ideal components, as demonstrated in [Figure 169](#).

Figure 169 Power Supply Feedthrough



In Figure 169, the blue line shows virtually zero feedthrough of the supply noise, or infinite rejection; this result is from a regular AC analysis. The variations in device characteristics cause imbalance in the circuit. For low frequencies, the sigma of the real part of the feedthrough signal (the orange line in the figure) is -28dB, or 4%. The imaginary part (the green line) becomes noticeable for frequencies higher than 1MHz.

Application Considerations

ACMatch analysis results match those of a small signal Monte Carlo Analysis. Discrepancies arise with certain test setups, if the operating point in Monte Carlo analysis varies by a large amount. For example, the output of an amplifier might saturate at one of the supply rails for some samples, if it is configured for high gain at DC. If such conditions exist, and the amplifier is used with the same gain configuration in the real application, then they point to issues which need to be investigated with DCMatch analysis and resolved first. Otherwise, the DC gain configuration of the amplifier needs to be changed in the test setup.

Mismatch Compared to Monte Carlo Analysis

DCMatch and ACMatch analyses use calculus of probability instead of sampling. The following table shows a comparison of the two types.

Feature	Mismatch	Monte Carlo
Analysis type	DC or AC	AC, DC, Transient
Device/parameter contribution report	Yes	Yes, with the PrimeSim HSPICE data mining flow
Relative run time	Fast	Slow
Accuracy	Good	Dependent on number of samples
Distributions	Normal preferred	Any
Variation result report	Global, Local, Spatial, and Interconnect separate	Global, Local, Spatial, and Interconnect combined

Supported Models with DCMatch and ACMatch

The following models are supported with DCMatch and ACMatch:

- **MOSFET:** LEVEL <=61, LEVEL=69, LEVEL>=70, LEVEL=66, and CMI models
- **BJT:** 1, 2, 6, 8, and 9
- **R:** No limitation
- **C:** No limitation

References

- [1] M.Pelgrom, A.Duinmaijer, and A.Welbers: "Matching Properties of MOS Transistors", IEEE J. Solid-State Circuits, pp. 1433-1439, May 1989.
- [2] P.R.Kinget: "Device Mismatch and Tradeoffs in the Design of Analog Circuits", IEEE J. Solid-State Circuits, pp. 1212-1224, June 2005.

28

Exploration Block

Describes the use of the Exploration Block in PrimeSim HSPICE.

The Exploration Block addresses the need to study the behavior and sensitivities of circuits to come up with an optimum design. During this early design phase, you may want to explore ranges of device sizes for a given circuit topology. The Exploration Block feature allows you to describe a set of experiments with different geometries, without changes to the original netlist. The Exploration Block is closely related to the Variation Block with external sampling (see [Chapter 25, Statistical Representation Using the Variation Block](#)).

PrimeSim HSPICE ships hundreds of examples for your use; see Listing of Demonstration Input Files in the *PrimeSim HSPICE User Guide: Demonstration Netlists* for paths to the demo files.

This chapter discusses the following topics:

- [Exploration Block Functions](#)
- [Usage Guidelines](#)
- [Flow Using an External Exploration Tool](#)
- [Exploration Block Structure](#)
- [Export File Syntax](#)
- [Execution of Exploration in PrimeSim HSPICE](#)
- [Exploration Data Block Syntax](#)
- [Exploration and Variation Block Interactions](#)
- [Limitations](#)

Exploration Block Functions

The Exploration Block extracts the parameters suitable for exploration from a netlist, and thus it eliminates parsing by the Exploration tool. The parameters are presented in a normalized format. This solution eliminates the exploration tool's need to rewrite the

netlist with new parameter values. Use of the Exploration Block returns only the updated values of the parameters which need to be changed in the course of a set of exploration simulations. PrimeSim HSPICE finds all the places where they need to be applied.

The Exploration Block contains a section with options and constraints, and it may include a data block with instructions on how to change certain parameters on individual devices or device groups. The data block must be created outside the simulator, based on information from the simulator and considerations specific to the particular design, possibly from an optimization program.

Usage Guidelines

To accommodate time restriction, exploration needs to be applied in an organized manner, with the smallest number of unrelated variables. A good approach for best results (partial and full matching) is to consider that integrated circuits are built with hierarchy, and that known relationships exist between devices. In essence, your knowledge about the circuit is encapsulated in the way exploration is carried out. Experience with optimization tools has shown that exploration methodology is crucial for your success, but is often difficult to set up correctly.

The following sections discuss these topics:

- [Multiple Instantiations of the Same Cell or Subcircuit](#)
- [Specifying Relationships between Devices](#)
- [Specifying Relationships between Device Properties](#)
- [Subcircuits and Elements Supported for Exploration](#)

Multiple Instantiations of the Same Cell or Subcircuit

In a typical design process, a large circuit is assembled from cells out of existing libraries. Each cell has different descriptions, for different applications: subcircuit, layout, behavioral model, etc. The circuit netlist describes how the cells are connected with other cells, and it contains a description of their content. Exploration of PrimeSim HSPICE is currently restricted to the hierarchical mode only.

In the hierarchical mode, exploration is cell-oriented, meaning that all instantiations of a particular cell are affected the same way. With this usage model, if you wish to explore separate instantiations of the same cell in a different manner, then you need to create new cell names with their content definitions repeated, before exploration can start. This renaming needs to be done anyhow for the final design, if you accept new device sizes coming out of the exploration because a basic rule of a circuit description is that multiple cell definitions with same name (and possibly different content) are not allowed.

Specifying Relationships between Devices

The following relationships between different devices can be specified in the Exploration Block to force matching:

```
device1Property1=expression of device2Property1
```

Such expressions reduce the number of variables for exploration because derived properties are processed inside of PrimeSim HSPICE. These relationship rules will be applied to all the devices subject to exploration. Therefore even if no change is requested from the Exploration tool, PrimeSim HSPICE executes these rules. So, if for example the lengths of devices opamp.mn1 and opamp.mn2 are different in the netlist, they will be the same in a simulation which contains an Exploration Block with the rule that they should be the same.

A simplified syntax expresses relationships of a whole set of device properties.

Examples

```
length(opamp.mn1)=length(opamp.mn2)
```

```
opamp.mp4=opamp.mp3
```

mp4 will be identical to mp3, in all properties

```
bias.mn5=2*bias.mn6
```

mn5 consists of 2 devices in parallel, which are identical to mn6

Specifying Relationships between Device Properties

The following relationships can be specified to cover appropriate scaling of secondary properties on the same device:

```
deviceProperty2=expression of deviceProperty1.
```

Example

```
ad='120n*W' as='120n*W' ps='240n+W' pd='240n+W'
```

Subcircuits and Elements Supported for Exploration

The exploration feature is primarily designed for design work on integrated circuits in CMOS technology. Exploration is supported for the following subcircuits and elements:

- Independent sources: DC value
- MOS devices: W, L, M, dtemp

- Resistors: R or W, L, M, dtemp
- Capacitors: C or W, L, M, dtemp

When designing circuits, the multiplicity factor *M* is always a positive integer, but the Exploration tool can request arbitrary positive values.

To preserve relationships which have been previously defined through expressions, exploration can only be applied to parameters which are defined with numerical values.

Examples

Example 1

```
m1 out in1 vdd vdd pch w=wp l=100n m=3
```

Exploration can be applied to element parameters *l* and *m*, but not to *w* directly.

Example 2

```
subckt nand in1 in2 out wp=100n wn=50n len=100n
m1 out in1 vdd vdd pch w=wp l=len
m2 out in2 vdd vdd pch w=wp l=len
m3 out in1 mid gnd nch w=wn l=len
m4 mid in2 gnd gnd nch w=wn l=len
.ends nand
```

Exploration can be applied to subcircuit parameters *wp*, *wn*, and *len*. The application envisioned here is for leaf cells with programmable layout: separate width and common length of pmos and nmos devices.

Example 3

```
.subckt onebit in1 in2 carry-in out carry-out
x1 in1 in2 7 nand
x2 in1 7 8 nand wp=100n wn=100n
x3 in2 7 9 nand wp=300n wn=150n
.ends onebit
subckt nand in1 in2 out wp=200n wn=100n
m1 out in1 vdd vdd pch w=wp l=100n
m2 out in2 vdd vdd pch w=wp l=100n
m3 out in1 mid gnd nch w=wn l=100n
m4 mid in2 gnd gnd nch w=wn l=100n
.ends nand
```

The subcircuit named *onebit* can be used for exploration because it instantiates other subcircuits using parameters with numerical values: *wn* and *wp* of *nand* gates *x2* and *x3*. The subcircuit named *nand* can be used for exploration on the default values *wn* and *wp* (exploration only affects instantiation *x1* because *x2* and *x3* parameters override the default values). The devices *m1* to *m4* can be used for exploration on their length but not on the width. This preserves the imposed relationship of equal width for *m1* and *m2*, and for *m3*

and `m4`. Exploration supports variation in temperature, in addition to element and subcircuit parameters. Encrypted sections of a netlist are not available for exploration.

Flow Using an External Exploration Tool

The design flow consists of:

- An [Information Extraction and Export Phase](#) in PrimeSim HSPICE
- A [Definition Phase](#) for the Exploration Block outside of PrimeSim HSPICE
- An [Exploration Phase](#) in PrimeSim HSPICE

Information Extraction and Export Phase

PrimeSim HSPICE creates an output file with the Export Block. The file's first section contains the names of the variables suitable for exploration, parameters, element and subcircuit parameters, along with appropriate identifiers, which include subcircuit name, device instance names, models and properties. In the second section, the corresponding values are listed.

From [Example 3 on page 669](#):

- Subcircuit `onebit` with properties `wn` and `wp` for instantiations `x2` and `x3`
- Subcircuit `nand` with properties `wp` and `wn` (only useful if an instantiation exists where `wp` and `wn` are not defined, as in `onebit.x1`)
- Devices `m1` to `m4` with property `l`

In the export phase, PrimeSim HSPICE runs a simulation from the originally supplied netlist, ignoring any Exploration Block content other than options.

Definition Phase

You must create or adapt the external utility described in the following items; it is not provided by PrimeSim HSPICE.

- An external utility reads the files created by PrimeSim HSPICE with the device information and any constraints.
- Supplemental information to the external utility consists of technology and details of the experiment.

- The utility creates a set of experiments and formulates them as a data block, with some or all variables contained in the Export Block, and one or more sets of exploration data.
 - The utility submits the netlist with the Exploration Block to PrimeSim HSPICE.
-

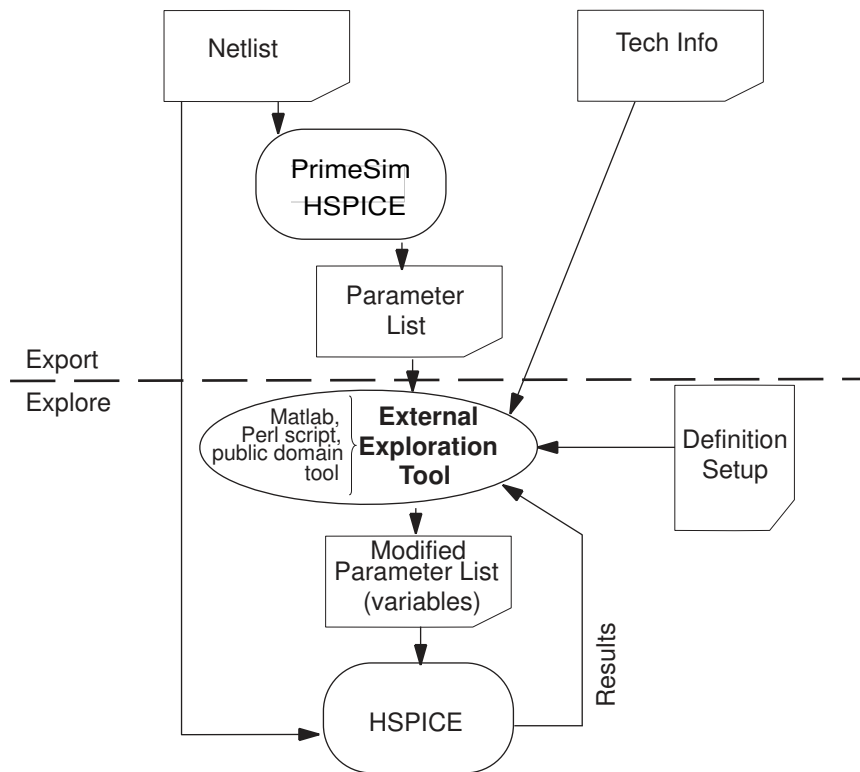
Exploration Phase

These items are done in the exploration phase:

- PrimeSim HSPICE applies the content of the data block, calculates the secondary parameters and constraints, and runs a set of simulations with the updated device geometries as specified in the Exploration Block. PrimeSim HSPICE produces measurement results and a file with all the parameter values used for each exploration simulation.
- The external utility analyzes and combines the simulation results.
- Based on the results, the utility might specify another set of experiments, with a new set of simulations, and run through these steps until some predefined goal is reached.

Refer to the [Figure 170](#), and notice the flow difference before and after adding Exploration Block.

Figure 170 Exploration Block Flow



Exploration Block Structure

Because the Exploration Block is closely related to the Variation Block, the internal structure is similar, in particular when compared with external sampling. The major differences include:

- The Variation Block specifies variation on the model parameters, whereas the Exploration Block deals with the values as defined in the netlist.
- Perturbations specified in the Variation Block are applied in a flat manner, whereas those from the Exploration Block apply to subcircuits.
- The Exploration Block includes:
 - options
 - parameters
 - relationships between devices

- relationships between properties
- area calculation
- data block characteristics
- A period is used as separator between a subcircuit name and an element name. For example: `opamp.rbias` refers to the resistor **rbias** instantiated in subcircuit **opamp**. A period is also used as separator between subcircuit names, if one subcircuit is defined within another. For example: `opamp.bias.rbias` refers to the resistor **rbias** in subcircuit **bias**, nested within subcircuit **opamp**.

The following sections discuss these topics:

- [Syntax](#)
- [Area Measurement](#)
- [The Processing of Netlist Parameters](#)

Syntax

The following syntax shows the parts and sequence of an Exploration Block:

```
.Design_Exploration
  Options
    Parameter Parameter_Name = value
    Parameter Parameter_Name = expression
    .Data BlockName
      Index    Name    Name, ...
      ...
    .EndData
.End_Design_Exploration
```

The following sections discuss these topics:

- [Control Options](#)
- [Example: Option secondary_param=yes](#)
- [Parameters Section](#)
- [Device Relationships](#)
- [Property Relationships](#)
- [Get_E\(\) Function In Exploration Block](#)
- [Parameters Defined Outside the Exploration Block](#)

Control Options

The options below are described in the tables that follow.

- Option Explore_only|Do_not_explore
- Option Export
- Option Exploration_method
- Option Ignore_exploration
- Option Secondary_param

If you want to explore only certain cells or subcircuits use:

Option	Description
Option Explore_only Subckts=SubcktList	This command is executed hierarchically — the specified subcircuits and all instantiated subcircuits and elements underneath are affected. Thus, if an inverter with name INV1 is placed in a digital control block called DIGITAL and in an analog block ANALOG, and Option Explore_only Subckts = ANALOG, then the perturbations only affect the INV1 in the analog block. You must create a new inverter INV1analog, with the new device sizes.
Option Do_not_explore Subckts=SubcktList	Excludes listed subcircuits.

The two modes of exploration are distinguished by setting either:

Option	Description
Option Export=yes	Exports extraction data and runs a simulation with the original netlist
Option Export=no	(Default) Runs a simulation with Exploration data

If you specify Option Export=yes followed by Option Mex_File_Only=yes PrimeSim HSPICE generates a *.mex file without running any simulations.

Option	Description
Option Mex_File_Only=yes	Generates a *.mex file without running any simulations.
Option Mex_File_Only=no	(Default) No *.mex file is generated.

The perturbation types are selected by setting either:

Option	Description
Option Exploration_method= external Block_name= Block_name	The Block_name is the same as the name specified in the .DATA block; PrimeSim HSPICE sweeps the row content with the EXCommand (see EXCommand Option: Export Data Block Action).
Option Ignore_exploration= yes no	(Default=no) PrimeSim HSPICE ignores the content in the design_exploration block, when Ignore_exploration=yes.
Option Secondary_param= yes no	(Default = no) If Secondary_param= yes, PrimeSim HSPICE exports the MOSFET secondary instance parameters to a *.mex file (created when option export=yes), and also permits the secondary parameters to be imported as a column header in the .DATA block (option export=no). See the example, following this section.

Example: Option secondary_param=yes

```
.design_exploration
  option secondary_param=yes
  parameter asad = '1.5e-7'
  option exploration_method=external block_name=dat2
  *nmos snps20n design_area='(Get_E(l)+1u)*(Get_E(W)+0.8u) '
  pmos snps20p as = 'asad*Get_E(W) ' ad = '2*asad*Get_E(W) '
  .data dat2
    index vdd@p k@p
    opamp.rcomp@r@e
    opamp.r0@r@e      r1@r@e r0@r@e
    opamp.ccomp@c@e c0@c@e c1@c@e
    opamp.mn1@snps20n@m@e
    opamp.mn1@snps20n@ad@e
    opamp.mn2@snps20n@m@e
    opamp.mn2@snps20n@ad@e
    opamp.mp3@snps20p@m@e
    opamp.mp4@snps20p@m@e
    opamp.mp5@snps20p@l@e      opamp.mp5@snps20p@w@e
    opamp.mp5@snps20p@m@e      opamp.mn8@snps20n@l@e
    opamp.mn8@snps20n@w@e      opamp.mn8@snps20n@m@e
    opamp.mn7@snps20n@l@e      opamp.mn7@snps20n@w@e
    opamp.mn7@snps20n@m@e      opamp.mn6@snps20n@l@e
    opamp.mn6@snps20n@w@e      opamp.mn6@snps20n@m@e
    v2@v@e
    1.000      2.5000      2.0000      7.000e+03      1.000e+06      1.000e+06
    1.000e+07      9.000e-13      1.000e-03      5.000e-12
    4.0000      1.000e-08      4.0000      1.000e-08      4.0000
    4.0000      4.000e-07      1.000e-05      3.0000      6.000e-06
    3.600e-05      10.0000      6.000e-06      3.600e-05      4.0000
    6.000e-06      3.600e-05      6.0000      0.
  .enddata
.End_Design_Exploration
```

Notice that column header `opamp.mn1@snps20n@ad@e` can be recognized by PrimeSim HSPICE only if `Option Secondary_param=yes`. In the netlist for the opamp (not shown above), only the devices `mn1` and `mn2` have secondary element parameter `AD` defined.

The supported MOSFET secondary parameters are: `AS`, `AD`, `PS`, `PD`, `NRD`, `NRS`, `RDC`, `RSC`.

Parameters Section

Parameters can be defined here, which are used in subsequent definitions within the Exploration Block. The name space is separate from the netlist. Parameters specified with numerical values are exported; derived parameters are not exported and are not available for exploration.

The following sections discuss these topics:

- [Secondary Element Parameters](#)
- [Secondary Device Parameters](#)
- [Same-Circuit Parameters](#)

Secondary Element Parameters

To calculate secondary element parameters on a single device:

```
Element subcircuitName.ElementName parameterName=  
'expression of parameterName'
```

The element parameters here include `AD`, `AS`, `PD`, `PS`, `NRS`, and `NRD` on the left side and expressions of `L` and `W` of the same element on the right hand side. For example:

```
Element opamp.nm1 AD='1e-7*Get_E(W)'
```

This relationship is enforced on all instantiations of subcircuit `opamp` (unless specifically excluded from exploration). Also, the property `AD` of `opamp.mn1` is not exported, and it is not available for exploration.

Secondary Device Parameters

Expressions for calculating the values of secondary device parameters for all devices with a certain model can be defined. Default values for `AD`, `AS`, `PD`, `PS`, `NRS` and `NRD` are often specific for devices which share the same model, as a function of `W` and `L`.

```
ModelType modelName instanceParameterName='expression of parameterName'
```

For example:

```
nmos snps65n as='asad*Get_E(W)' ad='asad*Get_E(W)'
```

This directive means that all `nmos` devices subject to exploration, with model `snps65n`, and have `AS` and/or `AD` specified, have their source and drain areas re-calculated by this

equation prior to simulation. If the secondary parameter is not specified on the device, then it is not added.

Note that PrimeSim HSPICE simulation results can change when such a definition is added to the Exploration Block, if the original values for AS and AD are different from the values calculated using the expression. While the secondary parameters are not exported, they are available for exploration when defined in the data block (expressions are not supported):

```
Opamp.mn1 AD=1e-12
```

Same-Circuit Parameters

To force relationship between parameters of the same subcircuit, use the syntax

```
Subckt subcircuitName parameterName='expression of parameterName'
```

Note that this function supports only relationship within the same subcircuit.

Device Relationships

Relationships between element properties exist, which must be respected when changing device size. To reduce the amount of time required by the Exploration Tool to calculate these dependencies, such relationships can be defined directly in the Exploration Block.

To force a relationship between two different elements, use the syntax:

```
Element subcircuitName.ElementName parameterName=  
'expression of Get_E([subcircuitName.]ElementName@parameterName)'
```

The element parameter names here include W and L for NMOS and PMOS devices. The subcircuit name on the right side of the definition is optional, if it is the same as the one on the left side.

```
Element opamp.mn1 l='Get_E(mn2@l)'  
Element inv4.mp1 w='2*Get_E(inv2.mp1@w)'
```

These relationships are enforced on all instantiations of subcircuits opamp and inv4 (unless specifically excluded from exploration).

Property Relationships

Properties L of opamp.mn1 and W of inv4.mp1 are not exported, and are not available for exploration.

Derived Device Properties

Derived device properties, as defined in the Exploration Block, are not exported. While specifying device relationship in a direct way is not supported, you can do this through parameter transformations.

Element Relationship: This relationship is enforced on all instantiations of subcircuit `opamp` (unless specifically excluded from exploration). Also, the property `AD` of `opamp.mn1` is not exported, and it is not available for exploration. The rules for the element relationship are:

1. If Element relationship equations are defined, then no related element parameters be exported in `*.mex` file, and such parameters do not appear in the `.DATA` block, as well.
2. If such parameters appear in a `.DATA` block, a warning message such as `**warning** unsupported statement c0@c@e` will be ignored, but will be reported in the list file (PrimeSim HSPICE ignores this column in the `.DATA` block).
3. Considering the implementation complex, netlist-defined subcircuits are not supported.
4. Such relationship equations can change element parameters no matter such element Parameters are defined through numbers or through expressions in the netlist.

Examples

Example 1:

```
Element opamp.mn1@L = 'Get_E(opamp.mn2@L)'
```

Only `opamp.mn2@L` from the netlist is exported; here, the device property of `opamp.mn1@L` is not exported.

Example 2:

```
nmos nch ad='120n*Get_E(W)'  
+ as='120n*Get_E(W)'  
+ ps='240n+Get_E(W)'  
+ pd='240n+Get_E(W)'
```

The properties `AD`, `AS`, `PS`, and `PD` are not exported.

Get_E() Function In Exploration Block

The Exploration Block deals strictly with netlist values, not final values. So the `Get_E( )` function returns the value that the user specifies in the netlist, and BEFORE adding any scale, `xw` and `wint` calculations. This is distinct from the `Get_E( )` in Variation Block, which returns the effective values.

`Get_E( )` is used to return a specific instance parameter value supported in the Exploration Block. The syntax is:

```
Get_E(SubcircuitName.InstanceName@ParameterName)
```

This function allows you to conveniently specify the relationship between instances. For example:

```
.design_exploration  
element op5.mn1 l='Get_E(op5.mn2@l)'
```

```
element op5.mn1 w='Get_E(op5.mn2@w) '  
.end_design_exploration
```

Parameters Defined Outside the Exploration Block

The parameters defined outside the Exploration Block can be referenced using the syntax:

```
Get_P(parameterName)
```

Area Measurement

While an exact area is only available after layout in integrated circuit design, following certain rules can provide a good estimate.

The complete measurement consists of three steps:

1. Calculate area of each device, according to model specific expressions
2. Calculate total area of top circuit or specified subcircuit.
3. Make results available to built-in measurement processor for output.

The calculation is performed as part of the operating point for AC and TRAN, but executed for each step of a DC transfer characteristics. This allows for reporting area at a certain value of a design parameter, which affects circuit area. However, area is not recalculated if it changes during an AC or transient sweep.

The following sections discuss these topics:

- [Syntax for Device Area Calculation](#)
- [Rules for Area Measurement using the design_area Keyword](#)
- [Specifying Constraints](#)

Syntax for Device Area Calculation

```
Modeltype Modelname design_area = expression
```

For example,

```
nmos nch design_area='(Get_E(L)+1u)*(Get_E(W)+0.8u) '
```

The measurement syntax allows for reporting the area of the whole circuit or a subcircuit, and has the following structure:

```
.measure analysisType measName Function  
+ DESIGN_AREA(HierarchicalName DeviceName)
```

Where, *analysisType* is DC|AC|Tran, *Function* can be min, max, find -at.

For example,

```
.measure top_area max design_area(x1)
```

Rules for Area Measurement using the design_area Keyword

The following rules apply for area measurement for typical cases.

1. Area computation only supports resistors, capacitors, and MOSFETs, currently. There is no geometry parameter for L, so this element is ignored in area computation.
2. The `design_area` keyword only supports those instances in which W/L is defined explicitly in netlist.
3. Compute total circuit area:

```
.measure |dc|ac|tran output_name1 find design_area at=val
```

Hierarchically based, compute the sum area of subcircuit x1:

```
.measure |dc|ac|tran output_name2 find design_area(x1) at=val
```

Compute area of x1.mn1

```
.measure |dc|ac|tran output_name3 find design_area(x1.mn1) at=val
```

For the equations defining area inside `.design_exploration` block, only model specific expressions are currently supported: `Modeltype Modelname design_area = expression`

4. The priority of computing one device area is
 - a. For resistors and capacitors:
 - (i) Expressions defined inside `exploration_block`
 - (ii) $W * L * M$ (W and L is defined as instance parameter)
 - (iii) $W_{model} * L_{model} * M$ (W_{model} and L_{model} are the model geometry values)
 - (iv) Otherwise, their area is zero.
 - b. For MOSFETs:
 - (i) Expressions defined inside `exploration_block`
 - (ii) $W * L * M$ (W and L are defined as instance parameters)

(iii) `wdefault*ldefault *mdefault` (`wdefault` and `ldefault` are the default geometry values for MOSFET)

(iv) `M` is the multiplier parameter.

5. If 'scale' is defined, then `design_area = area*scale*scale` Such measurement works with the `.DC` `.AC` `.TRAN` command, whether `.design_exploration` block is defined or not.

Specifying Constraints

While working through the device relationships, designers may want to specify constraints in the Exploration Block. The IF-ELSE structure can be used to constrain the circuit topology in PrimeSim HSPICE only. Each device subject to exploration is checked to verify whether the condition applies, and then the specified action or actions are executed.

The following sections discuss these topics:

- [Syntax](#)
- [Guidelines for using IF-ELSE Blocks](#)

Syntax

```
if (condition1){  
statement_block1  
}
```

The following statement block is optional, and can be repeated multiple times:

```
elseif (condition2)  
{statement_block2  
}
```

The following statement block is optional, and cannot be repeated:

```
else{  
statement_block3  
}
```

Guidelines for using IF-ELSE Blocks

The following guidelines aid in usage of the `.IF`, `.ELSE-IF`, or `.ELSE`:

- In an IF, ELSEIF, or ELSE statement block, you can reset certain elements geometries or issue a warning, or an error, to cause the simulator to abort. This example checks all MOSFETs that use model `nch`, if the length `l` is larger than `10u`, then a warning message is issued and the command resets it to `10u`.

```
If(nmos@nch@l>1e-5) {  
Reset_parameter l=10u
```

```
Warning
}
```

- The following functions can be used in specifying constraints:

- < Relational operator (less than)
- <= Relational operator (less than or equal to)
- > Relational operator (greater than)
- >= Relational operator (greater than or equal to)
- && Logical AND
- || Logical OR

- Supported CONDITION clauses include:

```
ModelType@ModelName@ModelParameter
Subckt@SubcktName@SubcktParameter
```

When logical operators (&& and ||) are used, the `ModelType`, `ModelName`, `Subckt`, `SubcktName` within the condition clauses must be the same. Otherwise, an error message is issued and PrimeSim HSPICE aborts.

For example:

```
If(nmos@snps20n@w<2e-7 && nmos@snps20n@l>10e-6) {
    action1
    action2
}
elseif(subckt@opamp@k>3 || subckt@opamp@n<2) {
    action
}
```

- You can include an unlimited number of ELSE-IF statements within an IF-ELSE block. If one element is found to satisfy the IF condition, PrimeSim HSPICE executes the action statements in the IF constraint for this element, and ignores any later ELSEIF or ELSE blocks.
- Commands for action statements are: `reset_parameter`, `warning`, and `abort`.
 - The `reset_parameter` statement can be specified several times in a constraint block. It can be used to limit or round off model or subcircuit parameters. For example:

```
If(nmos@nch@l>1e-5) {
    reset_parameter l=10u
    reset_parameter w=5u
    ...
    warning
}
```

- An `abort` keyword would print a message to indicate that the condition was satisfied and terminate the simulation.
- The `warning` keyword does not abort the simulation, but generally remedies the condition. In the example below, a warning message is generated when the condition is met.

```
Warning: Condition "nmos@nch@l > 10u" encountered for device  
"DeviceName".
```

- Since the `ELSE` constraint block does not contain any `CONDITION` clauses, its action statements are executed only if no element satisfies the `CONDITON` in `'IF'` constraint.

The Processing of Netlist Parameters

As shown in the section [“Flow Using an External Exploration Tool,”](#) you can have a special case of passing parameter values down one level of hierarchy. In a general case, when PrimeSim HSPICE finds a parameter definition with numerical value (`.param paramName=value`), it is exported with its name and value in the appropriate section. Parameters which are defined with other parameters instead of numerical values, or expressions of other parameters and numerical values, are not included in the Export file. This preserves relationships between devices, which have been set up by the designer in the original netlist.

Example of `diffpair` in netlist:

```
.subckt diffamp in1 in2 out lpair=2u wpair=2u mpair=4  
mn1 d1 in1 s b modelName l=lpair w=wpair m=mpair  
mn2 d2 in2 s b modelName l=lpair w=wpair m=mpair  
....  
.ends diffamp
```

The subcircuit `diffamp` and its parameters `lpair`, `wpair`, and `mpair` will be in the Export file along with their local values. The devices `mn1` and `mn2` are not available for exploration.

Export File Syntax

PrimeSim HSPICE writes the extracted data from the circuit to a file with the extension `.mex?` with syntax similar to the `*.mcx?` file, which lists the perturbations created from the Variation Block content. The option settings are reported first, followed by the names of all requested subcircuits and devices with their respective parameter names.

Separators are used as follows:

- A single period is used as a hierarchy separator between a subcircuit and an instance or device name, and as a separator between one or more subcircuit names, if their definitions are nested
- The @ character is used as a separator between model and parameter names.
- Additionally, identifiers are appended as follows to identify the proper owner if an element and a nested subcircuit have the same name:
 - E for element parameters
 - P for global parameters, and parameters used in subcircuit definitions and instantiations

The following sections discuss these topics:

- [Syntax Structure](#)
- [Example of an Export File](#)

Syntax Structure

The following constructs are provided:

- For primitives (R,C, without model):

```
[SubcircuitName.] InstanceName@ParamName@E
```

Example:

```
Opamp.rbias@r@E
```

- For devices with model (in NMOS, PMOS)

```
[SubcircuitName.] InstanceName@ModelType@ModelName@ParamName@E
```

Example:

```
Opamp.mn1@snps65n@L@E
```

- For standalone parameters:

```
[SubcircuitName@]=ParamName@P
```

Example:

```
(.param factor) Opamp@factor@P
```

- For parameters declared on subcircuit definition line:

```
[SubcircuitName@] ParamName@P
```


Example:

```
nand@wp@P
```

- For parameters appended to subcircuit instantiation:

```
[SubcircuitName.] InstanceName@ParamName@P
```

Example:

```
onebit.x2@wp@P
```

Whenever the optional *SubcircuitName* is not specified, the top level is assumed (implicit definition). For nested subcircuits, several *SubcircuitName* entries separated with a period are used.

Example of an Export File

```
onebit.x2@wp@P      onebit.x2@wn@P
onebit.x3@wp@P      onebit.x3@wn@P
opamp.mn1@snps65n@L@E opamp.bias.rbias@r@E
diffamp@lpair@P     diffamp@wpair@P     diffamp@mpair@P
index1 value1 value2 value3 value4 value5 etc
```

If option `export=yes` is set, then the output file contains a single data set with the original design values from the netlist. If option `export=no` (or default), then one data set is written per exploration step with all the parameters suitable for exploration, not only the ones which were changed through an Exploration Data Block (see [Exploration Block Structure](#)).

Execution of Exploration in PrimeSim HSPICE

Exploration is considered a second sweep. The following syntax of the sweep with the data block command is used with `EXCommand` using the keyword `explore`, otherwise it has the same syntax as `MCCCommand` for Monte Carlo.

```
.DC|.TRAN|.AC analysisDetail sweep EXCommand
```

The sample number is optional (and ignored if specified) when data export is requested. The following table shows the tasks performed by the simulator with the different combinations of `EXCommand`, option `Export`, and Data Block definition (valid meaning here: defined and having at least one set). Simulation with relationships means that the relationships described in the section [Device Relationships](#) are enforced.

EXCommand Option: Export Data Block Action

EXCommand	Option Export	Data Block	Action
Ignored	Yes	ignored	Export and run simulation with original netlist
<code>explore</code>	No or undefined	valid	Simulate all sets with relationships in Data Block
<code>explore=5</code>	No or undefined	valid	Simulate sets 1 to 5 from Data Block, with relationships
<code>explore list=3</code>	No or undefined	valid	Simulate set 3 from Data Block, with relationships
Ignored	No or undefined	not defined	Simulate with relationships enabled

Exploration Data Block Syntax

The exploration tool output is returned back into PrimeSim HSPICE as a data block, which is referenced in the Exploration Block as “Exploration_Data”. Data blocks can contain, for example, information such as the variable `temper`, to enable temperature sweeps.

The content is as follows:

```
variableName1 variableName2 variableName3
index1 value11 value12 value13
index2 value21 value22 value23
.....
```

`index` is an integer, monotonically increasing.

It is sufficient here to include only the cumulative set of parameters which change for the exploration run. Parameter names and values not specified here are left at their original values.

Exploration and Variation Block Interactions

When an Exploration Block and Variation Block are both present, PrimeSim HSPICE can currently handle sweeps in up to two dimensions, but the following rules apply:

- There is a `monte` command only: Exploration Block is ignored
- There is an `explore` command only: Variation Block is ignored
- There is neither `monte` nor `explore` command: Variation and Exploration blocks ignored

- When there is both a `monte` and `explore` command and there is/are:
 - Single Monte Carlo sample specified: execute the Exploration Block content, according to the `explore` command and option settings.
 - Several Monte Carlo samples specified, single Exploration request: execute the requested Exploration Data set, with specified Monte Carlo samples.
 - Several Monte Carlo samples and several `explore` requests: abort, with appropriate message.
- For multiple Exploration Blocks:
 - Options are cumulative, the last definition prevails.
 - Only one named data block can be executed.

Limitations

The following feature is not supported:

Netlist Export

At the end of the exploration procedure, a valid netlist needs to be generated which reflects the final choices for the device sizes, in order to be able to drive a layout tool and run a successful LVS (layout versus schematic) verification.

29

Optimization

Describes optimization in PrimeSim HSPICE for optimizing electrical yield.

PrimeSim HSPICE ships numerous examples for your use; see Device Optimization Examples in the *PrimeSim HSPICE User Guide: Demonstration Netlists* for paths to the demo files relating to the techniques discussed in this chapter. See also Circuit Optimization Examples, and Cell Characterization Examples for `.MODEL opt passfail` and `bisection` methods.

These topics are covered in the following sections:

- [Overview](#)
- [Optimization Statements](#)
- [Optimization Examples](#)

Overview

Optimization automatically generates model parameters and component values from a set of electrical specifications or measured data. When you define an optimization program and a circuit topology, PrimeSim HSPICE automatically selects the design components and model parameters to meet your DC, AC, and transient electrical specifications.

The circuit-result targets are part of the `.MEASURE` command structure and you use a `.MODEL` statement to set up the optimization.

Note:

PrimeSim HSPICE uses post-processing output to compute the `.MEASURE` statements. If you set `INTERP=1` to reduce the post-processing output, the measurement results might contain interpolation errors. See the *PrimeSim™ Continuum Reference Manual: Commands and Control Options* for more information about these options.

PrimeSim HSPICE employs an incremental optimization technique. This technique solves the DC parameters first, then the AC parameters, and finally the transient parameters. A

set of optimizer measurement functions not only makes transistor optimization easy, but significantly improves cell and circuit optimization.

To perform optimization, create an input netlist file that specifies:

- Minimum and maximum parameter and component limits.
- Variable parameters and components.
- An initial estimate of the selected parameter and component values.
- Circuit performance goals or a model-versus-data error function.

If you provide the input netlist file, optimization specifications, component limits, and initial guess, then the optimizer reiterates the circuit simulation until it either meets the target electrical specification, or finds an optimized solution.

For improved optimization, reduced simulation time, and increased likelihood of a convergent solution, the initial estimate of component values should produce a circuit whose specifications are near those of the original target. This reduces the number of times the optimizer reselects component values and resimulates the circuit.

The following sections discuss these topics:

- [Optimization Control](#)
- [Simulation Accuracy](#)
- [Curve Fit Optimization](#)
- [Goal Optimization](#)
- [Timing Analysis](#)

Optimization Control

How much time an optimization requires before it completes depends on:

- Number of iterations allowed.
- Relative input tolerance.
- Output tolerance.
- Gradient tolerance.

The default values are satisfactory for most applications. Generally, 10 to 30 iterations are sufficient to obtain accurate optimizations.

Simulation Accuracy

For optimization, set the simulator with tighter convergence options than normal. The following are suggested options.

For DC MOS model optimizations:

```
absmos=1e-8  
relmos=1e-5  
relv=1e-4
```

For DC JFET, BJT, and diode model optimizations:

```
absi=1e-10  
reli=1e-5  
relv=1e-4
```

For transient optimizations:

```
relv=1e-4  
relvar=1e-2
```

Curve Fit Optimization

Use optimization to curve-fit DC, AC, or transient data:

1. Use the `.DATA` statement to store the numeric data for curves in the data file as in-line data.
2. Use the `.PARAM xxx=OPTxxx` statement to specify the variable circuit components and the parameter values for the netlist. The optimization analysis statements use the `DATA` keyword to call the in-line data.
3. Use the `.MEASURE` statement to compare the simulation result to the values in the data file. In this statement, use the `ERR1` keyword to control the comparison.

If the calculated value is not within the error tolerances specified in the optimization model, PrimeSim HSPICE selects a new set of component values. PrimeSim HSPICE then simulates the circuit again and repeats this process until it obtains the closest fit to the curve or until the set of error tolerances is satisfied.

Goal Optimization

Goal optimization differs from curve-fit optimization because it usually optimizes only a particular electrical specification, such as rise time or power dissipation.

To specify goal optimizations, do the following:

1. Use the GOAL keyword.
2. In the .MEASURE statement, select a relational operator where GOAL is the target electrical specification to measure.
3. In addition to the '=' operator, the '>' and '<' conditional operators can be used to specify a GOAL in the .MEASURE statement.

For example, you can choose a relational operator in multiple-constraint optimizations when the absolute accuracy of some criteria is less important than for others.

Timing Analysis

To analyze circuit timing violation, PrimeSim HSPICE uses a binary search algorithm. This algorithm generate a set of operational parameters, which produce a failure in the required behavior of the circuit. When a circuit timing failure occurs, you can identify a timing constraint, which can lead to a design guideline. Typical types of timing constraint violations include:

- Data setup time before a clock.
- Data hold time after a clock.
- Minimum pulse width required to allow a signal to propagate to the output.
- Maximum toggle frequency of the component(s).

Bisection Optimization finds the value of an input variable (target value) associated with a goal value for an output variable. To relate them, you can use various types of input and output variables, such as voltage, current, delay time, or gain, and a transfer function.

You can use the bisection feature in either a PASSFAIL mode or a bisection mode. In each case, the process is largely the same.

Optimization Statements

Optimization requires several statements:

- .MODEL *modname* OPT ...
- .PARAM *parameter*=OPTxxx (*init*, *min*, *max*)

Use .PARAM statements to define initial, lower, and upper bounds.

- A .DC, .AC, or .TRAN analysis statement, with:

MODEL=*modname*

```
OPTIMIZE=OPTxxx  
  
RESULTS=measurename
```

Use `.PRINT` and `.PROBE` output statements, with the `.DC`, `.AC`, or `.TRAN` analysis statements.

Only use an analysis statement with the `OPTIMIZE` keyword for optimization. To generate output for the optimized circuit, specify another analysis statement (`.DC`, `.AC`, or `.TRAN`), and the output statements.

- `.MEASURE measurename ... [GOAL=val]`

Include a space on either side of the relational operator: `= <space> <space> <space>`

For a description of the types of `.MEASURE` statements that you can use in optimization, see [Chapter 9, Simulation Output](#).

The proper specification order is:

1. Analysis statement with `OPTIMIZE`.
2. `.MEASURE` statements specifying optimization goals or error functions.
3. Ordinary analysis statement.
4. Output statements.

Syntax for Optimizing Analysis (.DC, .TRAN, .AC)

The following syntax optimizes PrimeSim HSPICE simulation for a DC, AC, and Transient analysis.

```
.DC [DATA=filename] SWEEP OPTIMIZE=OPTxxx  
+ RESULTS=meas1,...,measN MODEL=optmod  
  
.AC [DATA=filename] SWEEP OPTIMIZE=OPTxxx  
+ RESULTS=meas1,...,measN MODEL=optmod  
  
.TRAN [DATA=filename] SWEEP OPTIMIZE=OPTxxx  
+ RESULTS=meas1,...,measN MODEL=optmod
```

Argument	Description
DATA	Specifies an in-line file of parameter data to use in optimization.
MODEL	The optimization reference name, which you also specify in the <code>.MODEL</code> optimization statement.

Argument	Description
OPTIMIZE	Indicates that the analysis is for optimization. Specifies the parameter reference name used in the .PARAM optimization statement. In a .PARAM optimization statements, if OPTIMIZE selects the parameter reference name, then the associated parameters vary during an optimization analysis.
RESULTS	Measure names used to specify the optimization goals or error functions from one or more .MEASURE commands. Multiple results are written as a comma-separated list.

Optimization Examples

This section contains examples of PrimeSim HSPICE optimizations. (For information on PrimeSim HSPICE advanced analog analyses optimization, see the Optimization section in the *PrimeSim HSPICE User Guide: Advanced Analog Simulations and Analysis*.)

- [MOS Level 3 Model DC Optimization](#)
- [MOS Level 13 Model DC Optimization](#)
- [RC Network Optimization](#)
- [Optimizing CMOS Tristate Buffer](#)
- [BJT S-parameters Optimization](#)
- [BJT Model DC Optimization](#)
- [Optimizing GaAsFET Model DC](#)
- [Optimizing MOS Op-amp](#)

MOS Level 3 Model DC Optimization

This example shows an optimization of I-V data to a Level 3 MOS model. The data consists of gate curves (ids versus vgs) and drain curves (ids versus vds).

This example optimizes the Level 3 parameters:

- VTO
- GAMMA
- UO
- VMAX
- THETA
- KAPPA

After optimization, PrimeSim HSPICE compares the model to the data for the gate, and then to the drain curves. `.OPTION POST` generates waveform files for comparing the model to the data.

Input Netlist File for Level 3 Model DC Optimization

You can find the sample netlist for this example in the following directory:

```
$installdir/demo/hspice/devopt/ml3opt.sp
```

The PrimeSim HSPICE input netlist shows:

- Using `.OPTION` to tighten tolerances, which increases the accuracy of the simulation. Use this method for I-V optimization.
- `.MODEL optmod OPT itropt=30` limits the number of iterations to 30.
- The circuit is one transistor. The `VDS`, `VGS`, and `VBS` parameter names, match names used in the data statements.
- `.PARAM` statements specify `XL`, `XW`, `TOX`, and `RSH` process variation parameters, as constants. The device characterizes these measured parameters.
- The model references parameters. In `GAMMA= GAMMA`, the left side is a Level 3 model parameter name; the right side is a `.PARAM` parameter name.
- The long `.PARAM` statement specifies initial, min and max values for the optimized parameters. Optimization initializes `UO` at 480, and maintains it within the range 400 to 1000.
- The first `.DC` statement indicates that:
 - Data is in the in-line `.DATA` all block, which contains merged gate and drain curve data.
 - Parameters that you declared as `OPT1` (in this example, all optimized parameters) are optimized.
 - The `COMP1` error function matches the name of a `.MEASURE` statement.
 - The `OPTMOD` model sets the iteration limit.
- The `.MEASURE` statement specifies least-squares relative error. PrimeSim HSPICE divides the difference between data `par(ids)` and model `i(m1)` by the larger of:
 - the absolute value of `par(ids)`, or
 - `minval=10e-6` If you use `minval`, low current data does not dominate the error.

- Use the remaining `.DC` and `.PRINT` statements for print-back after optimization. You can place them anywhere in the netlist input file because parsing the file correctly assigns them.
- The `.PARAM VDS=0 VGS=0 VBS=0 IDS=0` statements declare these data column names as parameters.

The `.DATA` statements contain data for `IDS` versus `VDS`, `VGS` and `VBS`. Select data that matches the model parameters to optimize.

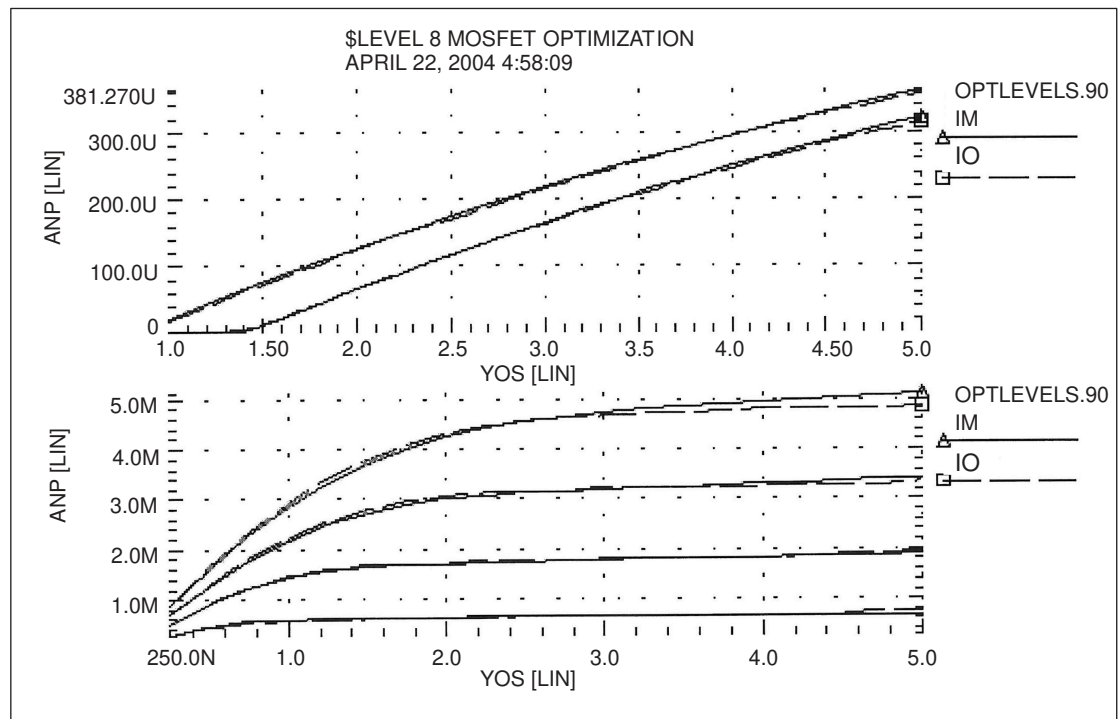
Results

To optimize GAMMA, use data with back bias (`VBS = -2` in this case). To optimize KAPPA, the saturation region must contain data. In this example, the all data set contains:

- Gate curves: `vds=0.1 vbs=0,-2 vgs=1 to 5` in steps of 0.25.
- Drain curves: `vbs=0 vgs=2,3,4,5 vds=0.25 to 5` in steps of 0.25.

Figure 171 on page 695 shows the results.

Figure 171 Level 3 MOSFET Optimization



MOS Level 13 Model DC Optimization

This example shows I-V data optimization to a Level 13 MOS model. The data consists of gate curves (ids versus vgs) and drain curves (ids versus vds). This example demonstrates two-stage optimization.

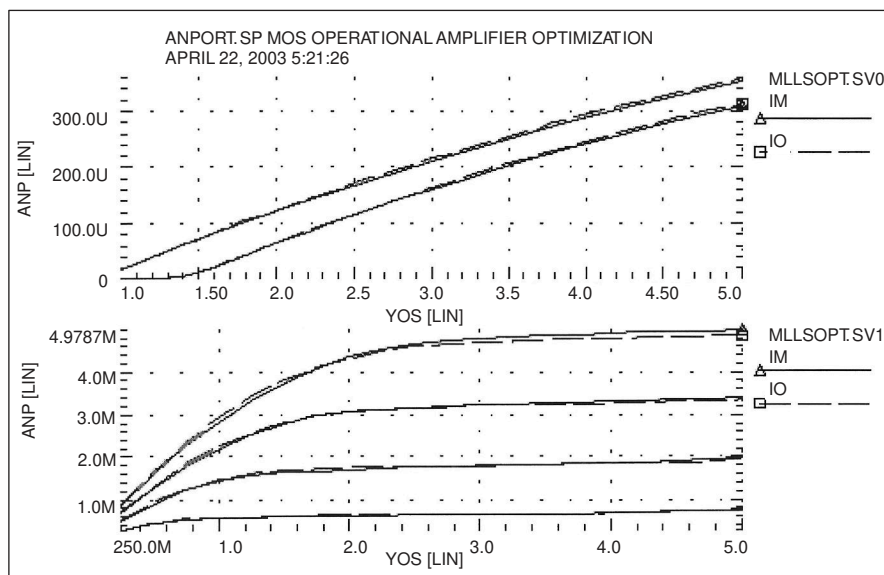
1. PrimeSim HSPICE optimizes the `vfb0`, `k1`, `muz`, `x2m`, and `u00` Level 13 parameters to the gate data.
2. PrimeSim HSPICE optimizes the `MUS`, `X3MS`, and `U1` Level 13 parameters, and the `ALPHA` impact ionization parameter to the drain data.

After optimization, PrimeSim HSPICE compares the model to the data. The `POST` option generates waveform files to compare the model to the data. [Figure 172 on page 696](#) shows the results.

DC Optimization Input Netlist File for Level 13 Model

You can find the sample netlist for this example in the following directory: `$installdir/demo/hspice/mos/ml13opt.sp`.

Figure 172 Level 13 MOSFET Optimization



RC Network Optimization

For a full demonstration example of optimizing speed or power for an RC circuit, follow the path to `rcopt.sp` in Circuit Optimization Examples in the *PrimeSim HSPICE User Guide: Demonstration Netlists*.

The following example optimizes the power dissipation and time constant for an RC network. The circuit is a parallel resistor and capacitor. Design targets are:

- 1 s time constant.
- 50 mW rms power dissipation through the resistor.

The PrimeSim HSPICE strategy is:

- RC1 `.MEASURE` calculates the RC time constant, where the GOAL of .3679 V corresponds to 1 s time constant e-rc.
- RC2 `.MEASURE` calculates the rms power, where the GOAL is 50 mW.
- OPT_{RC} identifies `RX` and `CX` as optimization parameters, and sets their starting, minimum, and maximum values.

Network optimization uses these PrimeSim HSPICE features:

- Measure voltages and report times that are subject to a goal.
- Measure device power dissipation subject to a goal.
- Measure statements replace the tabular or plot output.
- Parameters used as element values.
- Parameter optimizing function.
- Transient analysis with SWEEP optimizing.

The following sections discuss these topics:

- [Optimization Results](#)
- [Optimized Parameters OPTRC](#)

Optimization Results

```
RESIDUAL SUM OF SQUARES      = 4.291583E-16
NORM OF THE GRADIENT         = 5.083346E-04
MARQUARDT SCALING PARAMETER  = 2.297208E-04
NO. OF FUNCTION EVALUATIONS  = 20
NO. OF ITERATIONS            = 9
```

Residual Sum of Squares: The residual sum of squares is a measure of the total error. The smaller this value, the more accurate the optimization results.

$$\text{residual sum of squares} = \sum_{i=1}^{ne} E_i^2$$

In this equation, E is the error function, and ne is the number of error functions.

Norm of the Gradient: The norm of the gradient is another measure of the total error. The smaller this value is the more accurate the optimization results are. The following equations calculate the G gradient:

$$G_j = \sum_{i=1}^{ne} E_i \cdot (\Delta E_i / \Delta P_j)$$

$$\text{norm of the gradient} = 2 \cdot \sqrt{\sum_{j=1}^{np} G_j^2}$$

In this equation, P is the parameter, and np is the number of parameters to optimize.

Marquardt Scaling Parameter: The Levenburg-Marquardt algorithm uses this parameter to find the actual solution for the optimizing parameters. The search direction is a combination of the Steepest Descent method and the Gauss-Newton method.

The optimizer initially uses the Steepest Descent method as the fastest approach to the solution. It then uses the Gauss-Newton method to find the solution. During this process, the Marquardt Scaling Parameter becomes very small, but starts to increase again if the solution starts to deviate. If this happens, the optimizer chooses between the two methods to work toward the solution again.

If the optimizer does not attain the optimal solution, it prints both an error message, and a large Marquardt Scaling Parameter value.

Number of Function Evaluations: This is the number of analyses (for example, finite difference or central difference) needed to find a minimum of the function.

Number of Iterations: This is the number of iterations needed to find the optimized or actual solution.

Optimized Parameters OPTRC

```
.param rx= 7.4823 $ 55.6965 5.7945m
.param cx=133.9934m $ 44.3035 5.1872m
```

Figure 173 Power Dissipation and Time Constant (VOLT) RCOPT.TR0=Before Optimization, RCOPT.TR1=Optimized Result

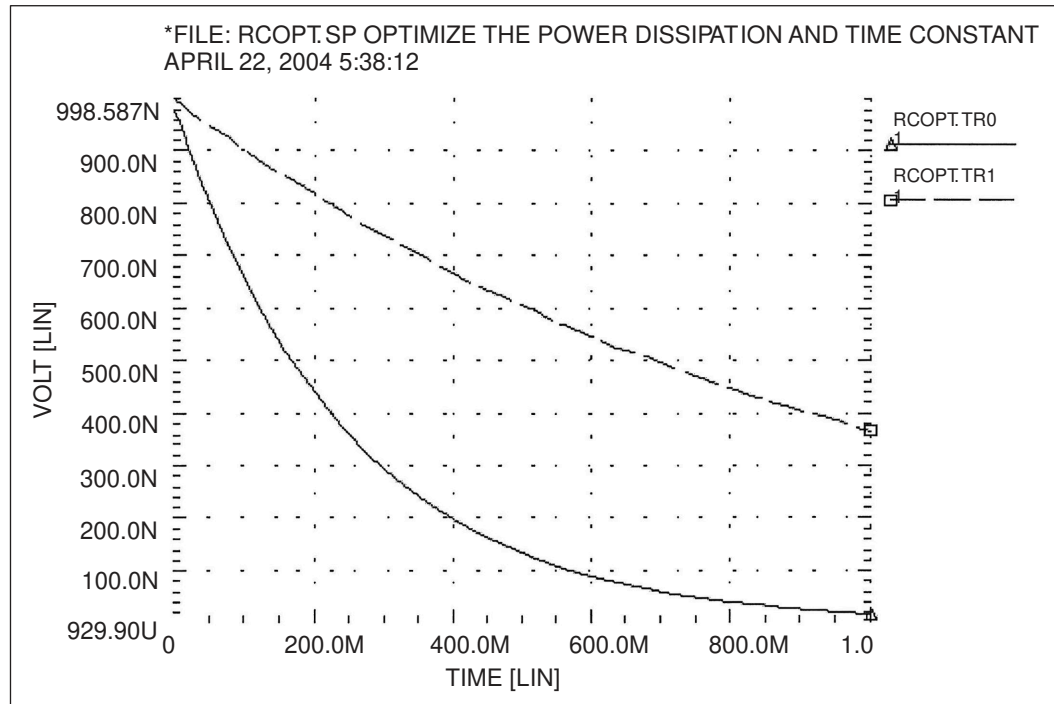
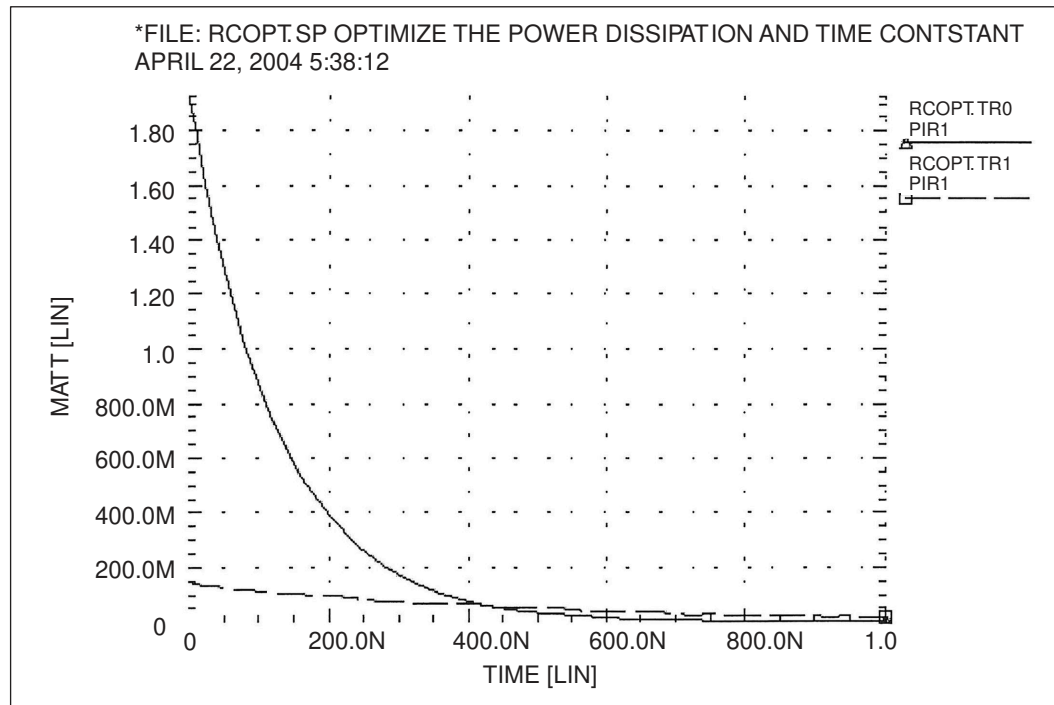


Figure 174 Power Dissipation and Time Constant (WATT) RCOPT.TR0=Before Optimization, RCOPT.TR1=Optimized Result



Optimizing CMOS Tristate Buffer

The example circuit is an inverting CMOS tristate buffer. The design targets are:

- Rising edge delay of 5 ns (input 50 percent voltage to output 50 percent voltage).
- Falling edge delay of 5 ns (input 50 percent voltage to output 50 percent voltage).
- RMS power dissipation should be as low as possible.
- Output load consists of:
 - pad capacitance
 - leadframe inductance
 - 50 pF capacitive load

The PrimeSim HSPICE strategy is:

- Simultaneously optimize both the rising and falling delay buffer.
- Set up the internal power supplies, and the tristate enable as global nodes.

- Optimize all device widths except:
 - Initial inverter (assumed to be standard size).
 - Tristate inverter and part of the tristate control (optimizing is not sensitive to this path).
- Perform an initial transient analysis for plotting purposes. Then optimize and perform a final transient analysis for plotting.
- To use a weighted RMS power measure, specify unrealistically low power goals. Then use MINVAL to attenuate the error.

Input Netlist File to Optimize a CMOS Tristate Buffer

You can find the sample netlist for this example in the following directory:

```
$installdir/demo/hspice/apps/trist_buf_opt.sp
```

Figure 175 Tristate Buffer Optimization Circuit

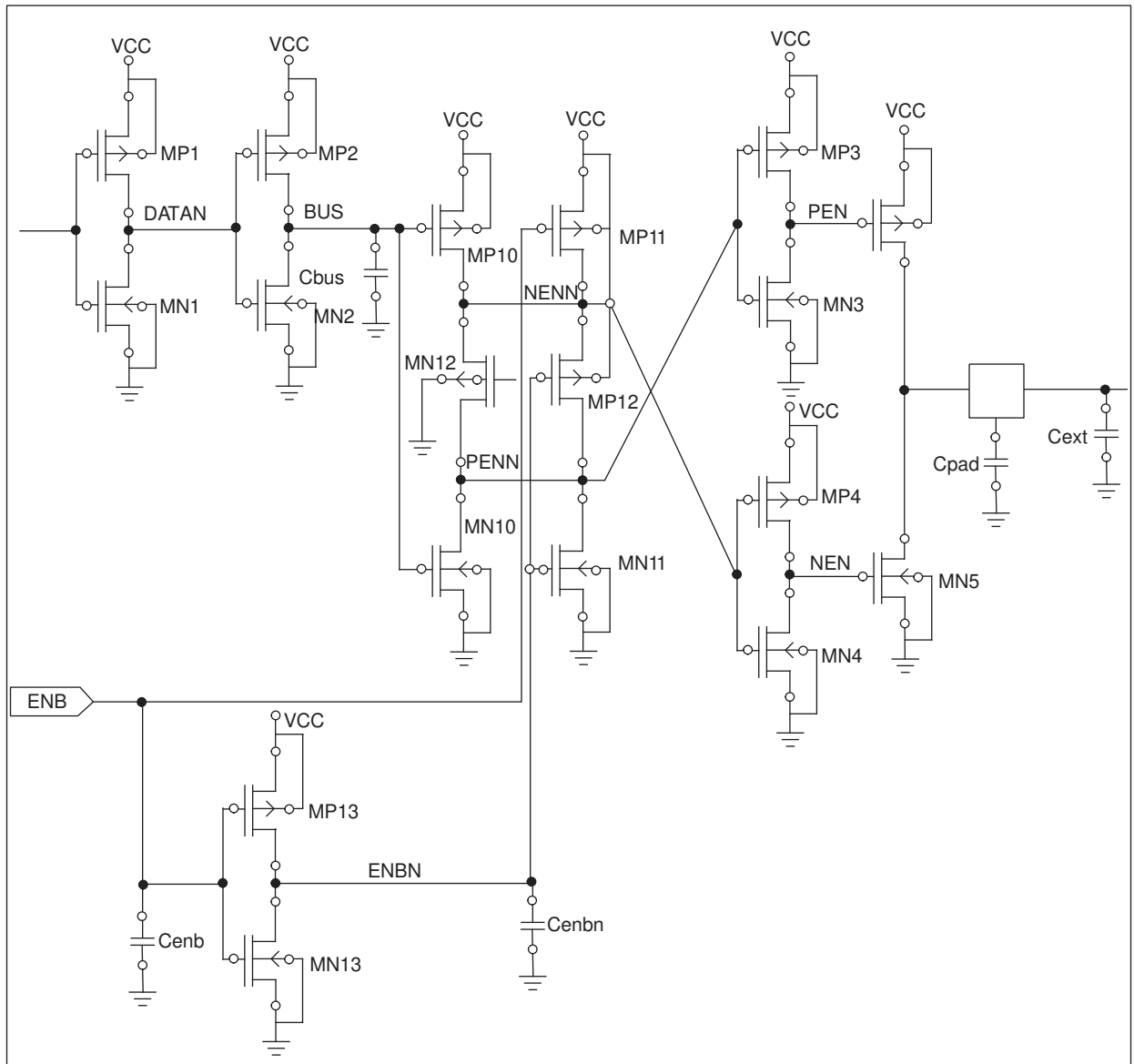
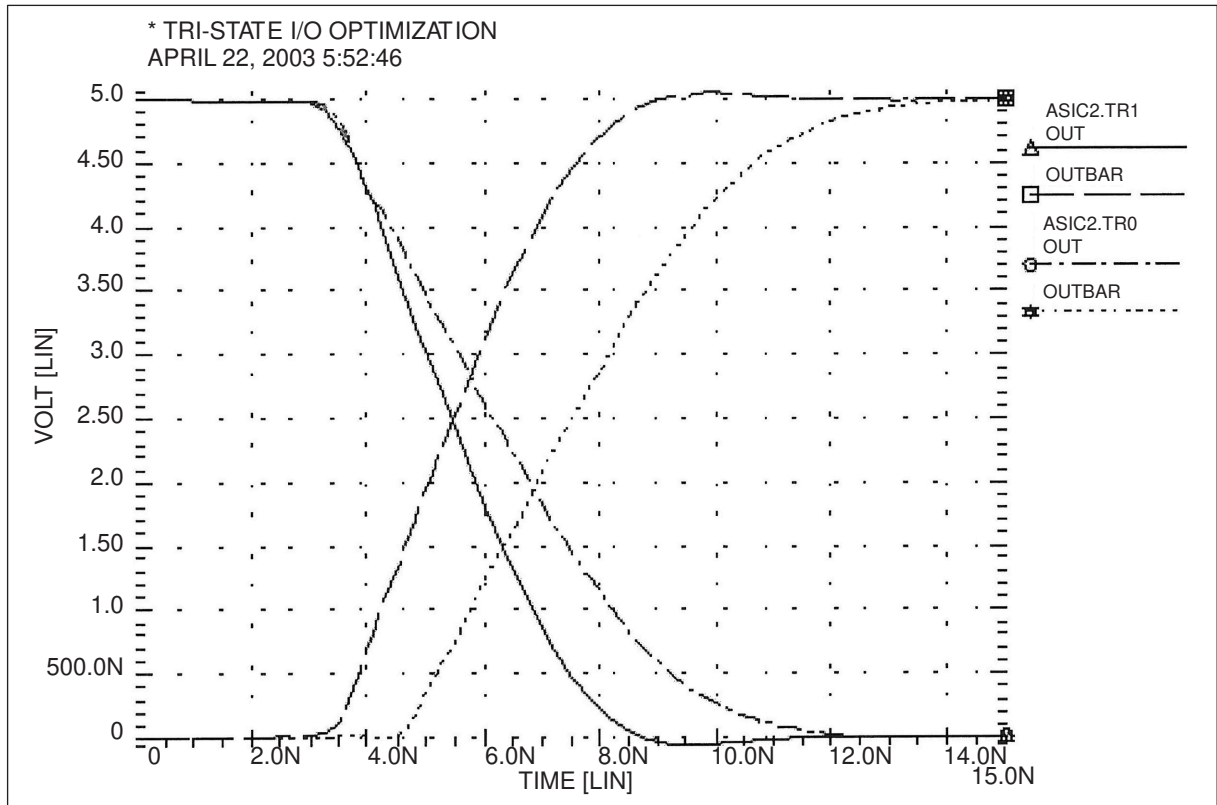


Figure 176 Tristate Input/Output Optimization ACIC2B.TR0 = Before Optimization, ACIC2B.TR1=Optimized Result



BJT S-parameters Optimization

The following example optimizes the S-parameters to match those specified for a set of measurements. The `.DATA` measured in-line data statement contains these measured S-parameters as a function of frequency. The model parameters of the microwave transistor (`LBB`, `LCC`, `LEE`, `TF`, `CBE`, `CBC`, `RB`, `RE`, `RC`, and `IS`) vary. As a result, the measured S-parameters (in the `.DATA` statement) match the calculated S-parameters from the simulation results.

This optimization uses a 2n6604 microwave transistor, and an equivalent circuit that consists of a BJT, with parasitic resistances and inductances. The BJT is biased at a 10 mA collector current (0.1 mA base current at DC bias and `bf=100`).

The following sections discuss these topics:

- [Key PrimeSim HSPICE Features Used](#)
- [Input Netlist File for Optimizing BJT S-parameters](#)
- [Optimization Results](#)

Key PrimeSim HSPICE Features Used

- `.NET` command to simulate network analyzer action.
- `.AC` optimization.
- Optimized element and model parameters.
- Optimizing, compares measured S-parameters to calculated parameters.
- S-parameters used in magnitude and phase (real and imaginary available).
- Weighting of data-driven frequency versus S Parameter table. Used for the phase domain.

Input Netlist File for Optimizing BJT S-parameters

BJT Equivalent Circuit Input

Use the `bjtopt.sp` netlist file located in your `$installdir/demo/hspice/devopt` directory for optimizing BJT S-parameters.

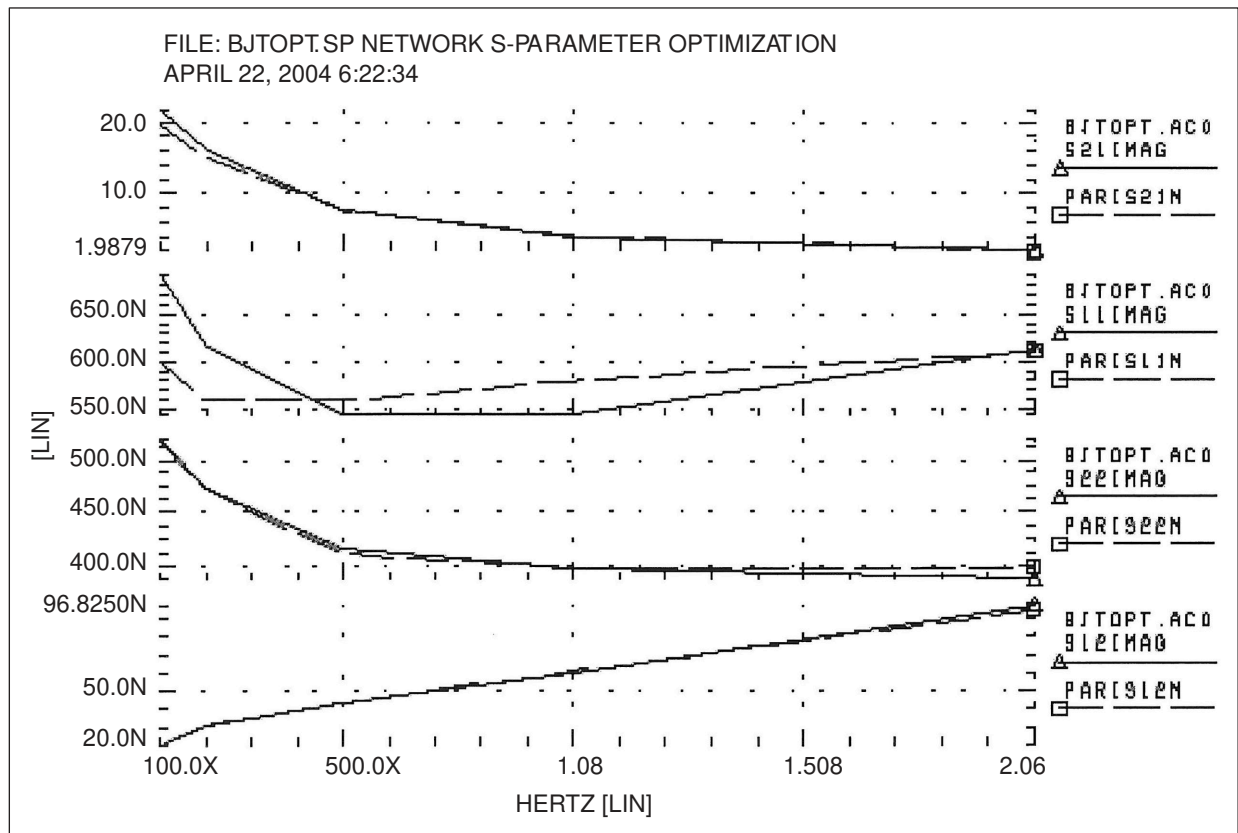
Optimization Results

```
RESIDUAL SUM OF SQUARES      =5.142639e-02
NORM OF THE GRADIENT          =6.068882e-02
MARQUARDT SCALING PARAMETER=0.340303
CO. OF FUNCTION EVALUATIONS=170
NO. OF ITERATIONS             =35
```

The maximum number of iterations (25) was exceeded. However, the results probably are accurate. Increase ITROPT accordingly.

```
Optimized Parameters OPT1- Final Values
***OPTIMIZED PARAMETERS OPT1 SENS  %NORM-SEN
.PARAM  LBB = 1.5834N  $ 27.3566X  2.4368
.PARAM  LCC = 2.1334N  $ 12.5835X  1.5138
.PARAM  LEE  =723.0995P  $254.2312X 12.3262
.PARAM  TF   =12.7611P  $  7.4344G 10.0532
.PARAM  CBE  =620.5195F  $ 23.0855G  1.5300
.PARAM  CBC  = 1.0263P  $346.0167G 44.5016
.PARAM  RB   =  2.0582  $ 12.8257M  2.3084
.PARAM  RE   =869.8714M  $ 66.8123M  4.5597
.PARAM  RC   =54.2262  $  3.1427M 20.7359
.PARAM  IS   =99.9900P  $  3.6533X 34.4463M
```

Figure 177 BJT-S Parameter Optimization



BJT Model DC Optimization

The goal is to match forward and reverse Gummel plots obtained from a HP4145 semiconductor analyzer by using the PrimeSim HSPICE `LEVEL=1` Gummel-Poon BJT model. Because Gummel plots are at low base currents, PrimeSim HSPICE does not optimize the base resistance. PrimeSim HSPICE also does not optimize forward and reverse Early voltages (VAF and VAR) because simulation does not measure VCE data.

The key feature in this optimization is incremental optimization.

1. PrimeSim HSPICE first optimizes the forward-Gummel data points.
2. PrimeSim HSPICE updates forward-optimized parameters into the model. After updating, you cannot change these parameters.
3. PrimeSim HSPICE next optimizes the reverse-Gummel data points.

BJT Model DC Optimization Input Netlist File

You can find the sample netlist for this example in the following directory:

```
$installdir/demo/hspice/devopt/opt_bjt.sp
```

Results

Figure 178 BJT Optimization Forward Gummel Plots

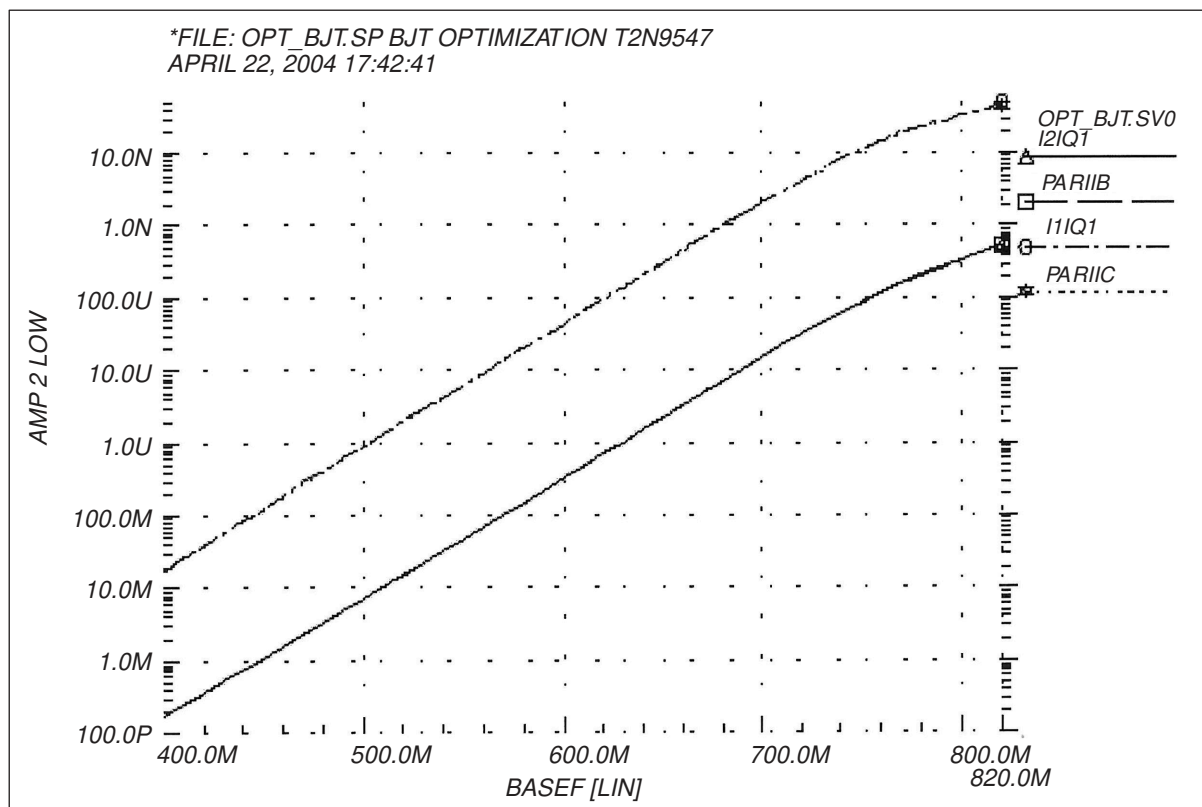
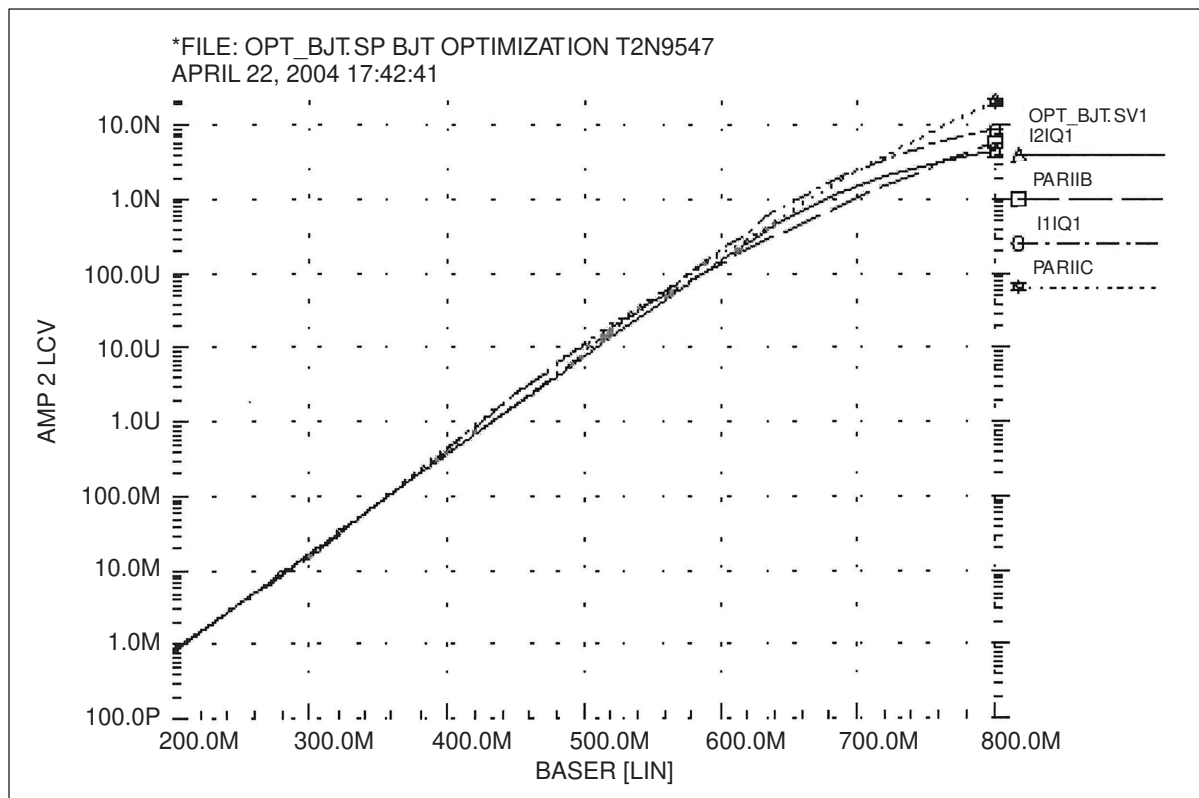


Figure 179 BJT Optimization Reverse Gummel Plots



Optimizing GaAsFET Model DC

This example circuit is a high-performance, GaAsFET transistor. The design target is to match HP4145 DC measured data to the PrimeSim HSPICE LEVEL=3 JFET model.

The PrimeSim HSPICE strategy is:

- `.MEASURE IDSERR` is an `ERR1` type function. It provides linear attenuation of the error results starting at 20 mA. This function ignores all currents below 1 mA. The high-current fit is the most important for this model.
- The `OPT1` function simultaneously optimizes all DC parameters.
- The `.DATA` statement merges TD1.dat and TD2.dat data files.
- The graph plot model sets the `MONO=1` parameter to remove the retrace lines from the family of curves.

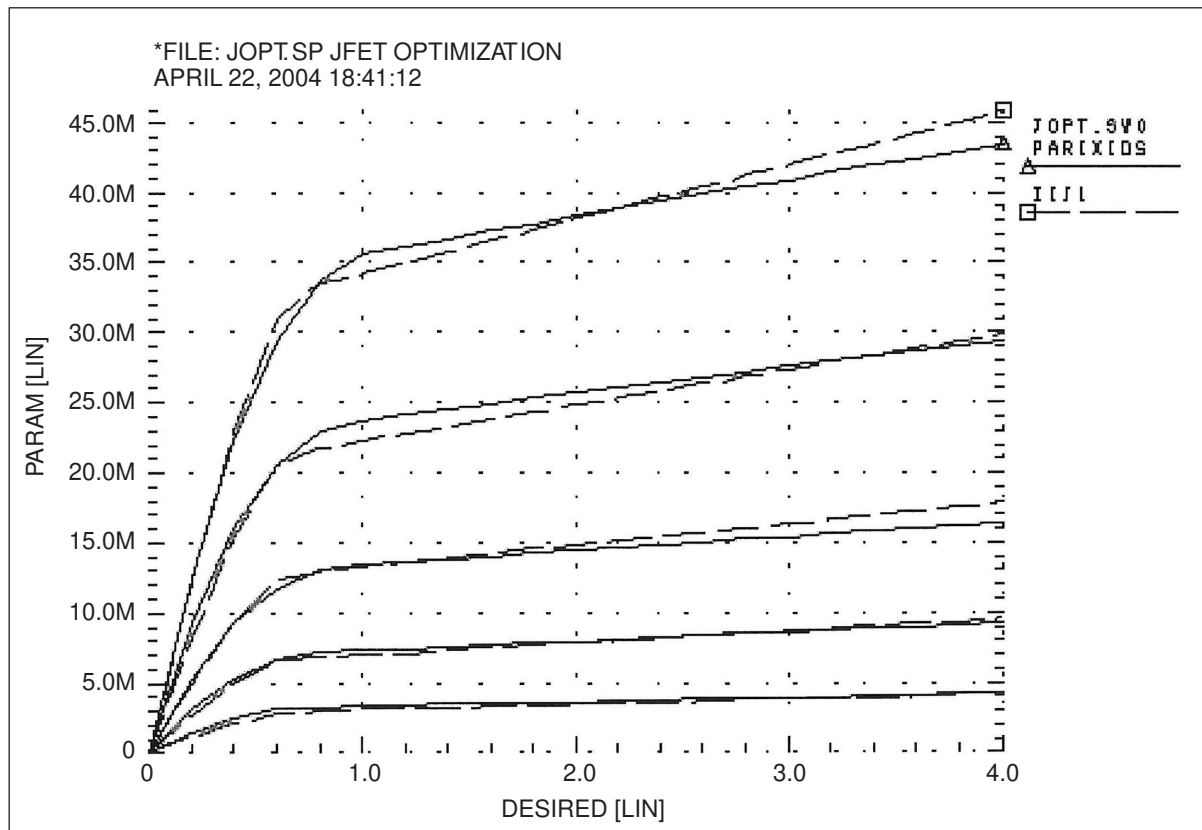
GaAsFET Model DC Optimization Input Netlist File

You can find the sample netlist for this example in the following directory:

```
$installdir/demo/hspice/devopt/jopt.sp
```

Results

Figure 180 JFET Optimization



Optimizing MOS Op-amp

The design goals for the MOS operational amplifier are:

- Minimize the gate area (and therefore the total cell area).
- Minimize the power dissipation.
- Open-loop transient step response of 100 ns for rising and falling edges.

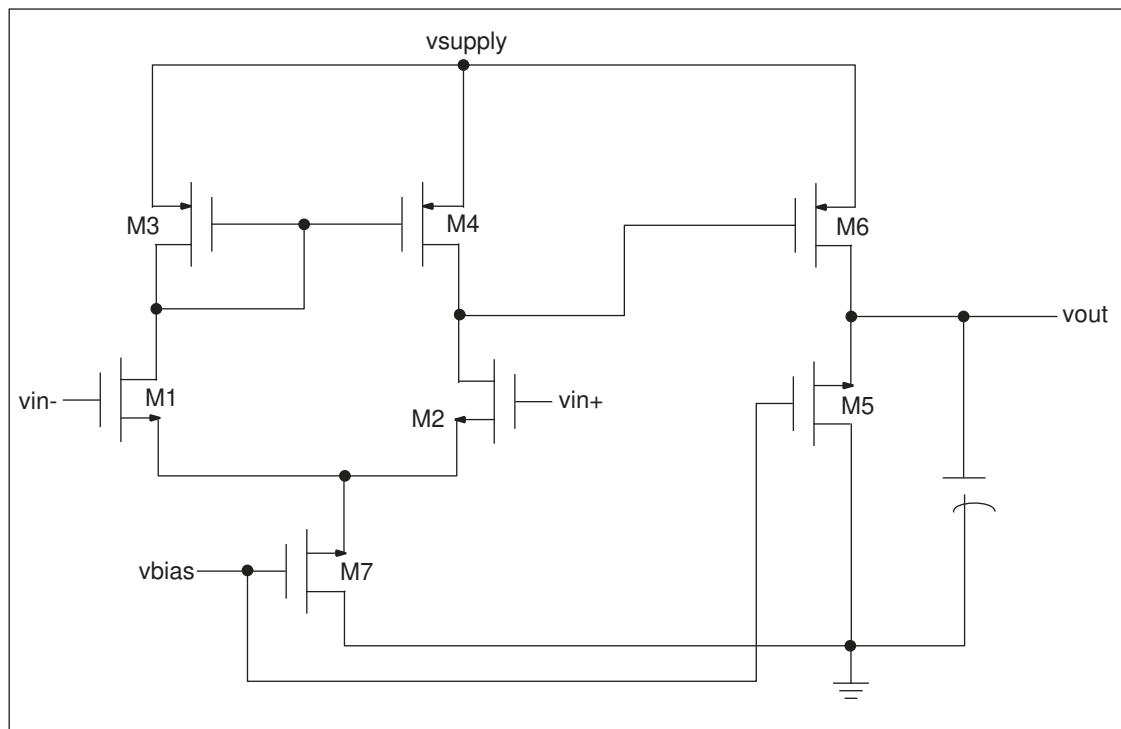
The PrimeSim HSPICE strategy is:

- Simultaneously optimize two amplifier cells for rising and falling edges.
- Total power is power for two cells.
- The optimization transient analysis must be longer to allow for a range of values in intermediate results.
- All transistor widths and lengths are optimized.
- Calculate the transistor area algebraically use a voltage value and minimize the resulting voltage.
- The transistor area measure statement uses `MINVAL`, which assigns less weight to the area minimization.
- Optimizes the bias voltage.

MOS Op-amp Optimization Input Netlist File

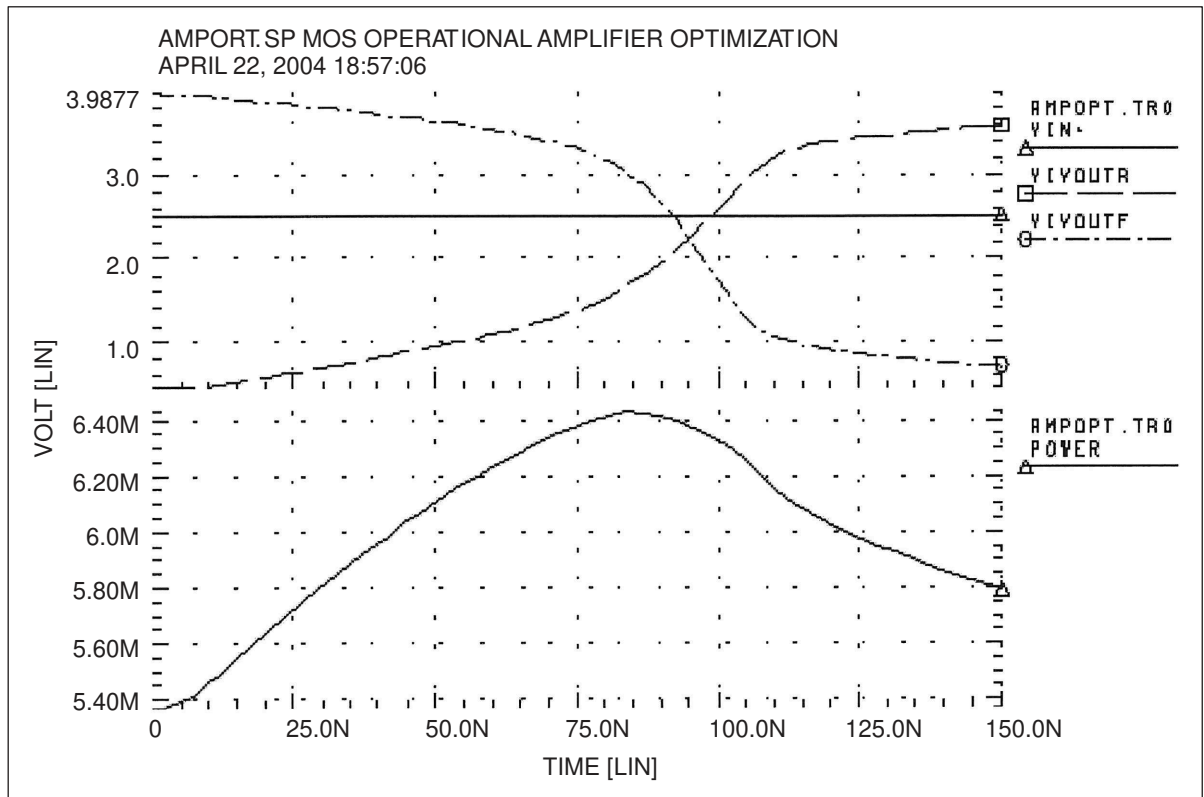
You can find the sample netlist for this example in the following directory: `$installdir/demo/hspice/cirop/ampopt.sp`

Figure 181 CMOS Op-amp



Results

Figure 182 Operational Amplifier Optimization



30

Model Parameter Optimization

Describes an utility in PrimeSim HSPICE for model parameter optimization.

These topics are covered in the following sections:

- [Overview](#)
- [Examples](#)

Overview

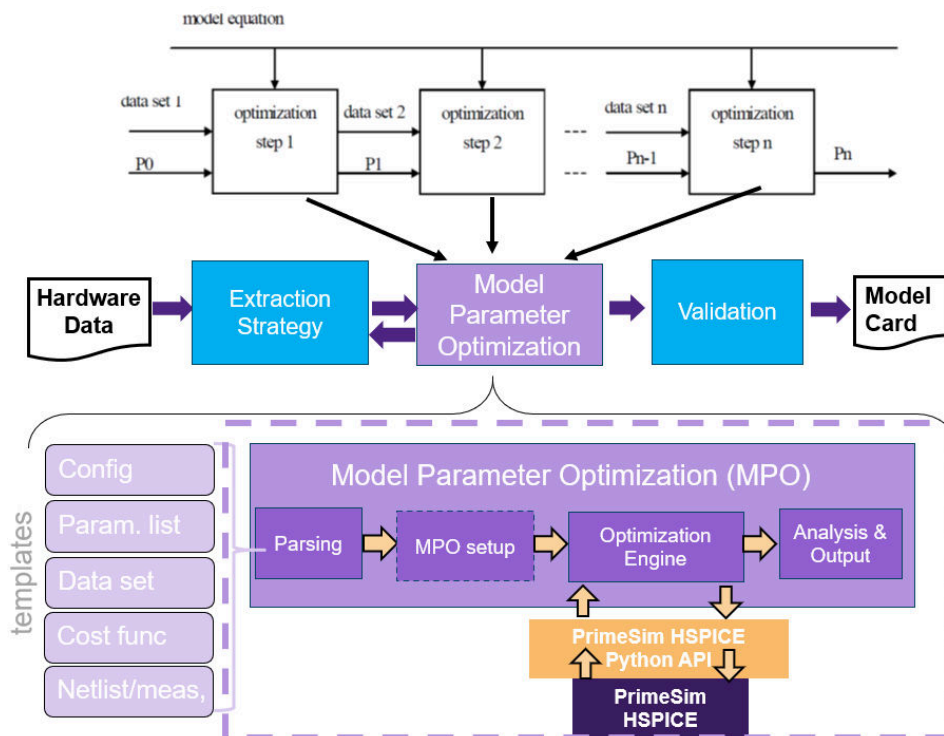
Model Parameter Optimization (MPO) is the enhanced model parameter optimization utility based on PrimeSim HSPICE.

It is released as a separate Python module and can be imported using Python script. Contact Synopsys Technical Support for full MPO manual and demo package.

MPO supports:

- Genetic Algorithm(GA) and Levenberg-Marquardt(LM) optimization algorithm
- Optimization of hardware measured targets (for example, IV/CV target data with its derivatives)
- Optimization of user defined targets (for example, targets defined by the PrimeSim HSPICE .MEAS commands)
- Optimization of mixed targets
- Customized weighting factor and optimization tolerance control
- User-defined loss function
- Parallelization

Figure 183 Workflow of MPO



MPO requires user-input optimization targets, parameters to be optimized, optimization algorithm, and PrimeSim HSPICE netlist. These inputs are configured into a JSON format configuration file. Output of MPO include optimized parameters, fitting result, and a log file of extraction statistics.

Examples

Two simple examples are provided in this section. For further details on MPO setup and usage, contact Synopsys Technical Support for full MPO manual.

- [Example 1: IV Data Fitting](#)
- [Example 2: Measurement Data Fitting](#)

Example 1: IV Data Fitting

This example shows model parameter optimization to fit measured IV data. The target data is two I_d - V_g curves with $V_d = 0.1V$, $V_b = 0$, and $-2V$ respectively.

Table 75 *Model Parameter Optimization for IV Data Fitting*

Vd	Vg	Vb	Id
1.000000e-01	1.000000e+00	0.000000e+00	2.27433E-05
1.000000e-01	1.400000e+00	0.000000e+00	7.39352E-05
...			
1.000000e-01	5.000000e+00	-2.000000e+00	3.60033E-04

[Table 76](#) lists the 8 BSIM model(level=13) parameters selected for optimization with initial value and range.

Table 76 *Parameters for Optimization: 8 BSIM Model(level=13)*

Parameter Name	Initial Value	Minimum Value	Maximum Value
vfb0	-0.5	-2	1
K1	0.9	0.1	1
muz	900	100	1500
u00	0.1	0	0.5
mus	500	100	1000
x3ms	8	1	10
u1	0.5	0	1
alpha	5	1	10

GA algorithm is used for the parameter optimization in this example.

MPO Input Files

Input files include target data file, SPICE netlist file(including model card), parameter file and configuration file.

The target data file is stored in the `target.dat` and `target.txt` format. The ASCII format file contains target data.

Sample of the `target.dat` file:

```
Definition
im
```

```
Value
2.27433E-05
7.39352E-05
...
3.60033E-04
```

Sample of the `target.txt` file:

```
./target.dat
```

In the `target.dat` file, *Definition* is reserved section for signal name. The signal name must be consistent with the signal name in SPICE netlist. Data value is defined in *Value* section. Support multiple target data files.

Netlist (test.sp): SPICE Format Netlist

```
...
Vds 30 0 vd
Vgs 30 0 vg
Vbs 40 0 vb
.param
+ vfb0 = -0.5
+ k1 = 0.9
...
.model nch nmos level=13
+ vfb0 = vfb0
+ k1 = k1
...
m1 30 20 0 40 nch w=50u l=4u
*..optimization sweeps
.dc data=gate
.print dc vb= par(vb) vg=par(vg) im=i(m1)
.data gate vd vg vb
1.000000e-01 1.000000e+00 0.000000e+00
1.000000e-01 1.400000e+00 0.000000e+00
...
1.000000e-01 5.000000e+00 -2.000000e+00
.enddata
.end
```

Parameter File (param.txt)

ASCII format file that contains model parameters with initial value and range for optimization.

```
vfb0 = (-0.5, -2,1)
k1 = (0.9,0.1,1)
muz = (900,100,1500)
u00 = (0.25,0,0.5)
mus = (500,100,1000)
x3ms = (8,1,10)
u1 = (0.5,0,1)
alpha= (5,1,10)
```

Configuration File (config.json)

JSON format file for MPO run, include input files, optimization algorithm, and optimization engine related settings.

```
{
  "casePath" : "./test.sp",
  "paramPath" : "./param.txt",
  "targetFile" : "./target.txt",
  "outDir" : "./out_demo_ga",
  "coreNum": 16,
  "sampleSize": 15,
  "seed": 8,
  "warnMsg": 1,
  "tol": 0.1,
  "optAlg": "ga"
}
```

A simple python script (test_mpo.py) to import MPO

```
from sys import argv
import mpo as test_mpo

if __name__=="__main__":
    mpo = test_mpo.MPO(argv[1])
    mpo.Run()
```

Use Python 3.8.3 or later version to run this script:

```
python test_mpo.py ./config.json
```

All files of example 1 is placed under PrimeSim HSPICE demo directory.

```
$installdir/demo/hspice/mpo
```

MPO output of example 1.

paramOpt.cfg: Optimized parameters

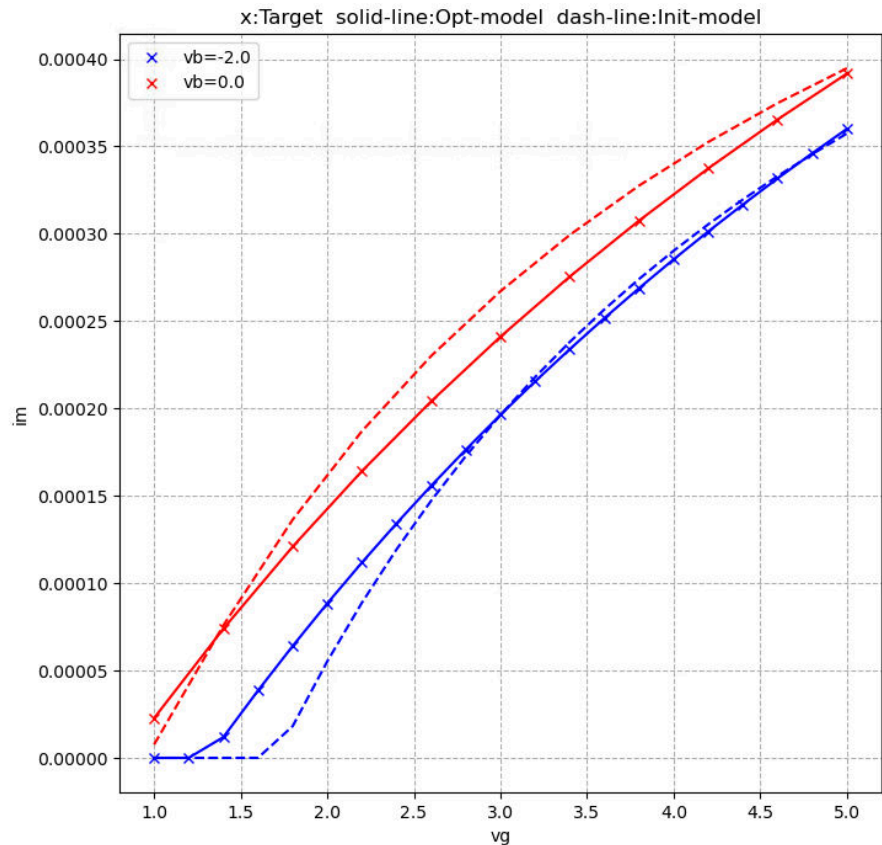
```
.param vfb0 = -3.9999926537026165e-01
.param k1 = 6.0000011494071070e-01
.param muz = 6.0003878585495011e+02
.param u00 = 9.9999254458312509e-02
.param mus = 6.6123158469197085e+02
.param x3ms = 5.2548814917487929e+00
.param u1 = 4.9890124947118419e-03
.param alpha = 9.1745065971343021e+00
```

simResult_im.txt: data fitting result

```
2.27433e-05    2.27434e-05    0.0%
 7.39352e-05    7.39351e-05    0.0%
...
3.60033e-04    3.60033e-04    0.0%
```

simResult_im.png: fitting result shown as IV curve.

Figure 184 Fitting Result



Example 2: Measurement Data Fitting

This example shows model parameters optimization to fit user-defined targets.

In this example, `IGateOn` and `IGateOff` are from PrimeSim HSPICE measurement statement. `Aigc` and `Aigd` are model parameters to be optimized.

Table 77 Parameter Information

Parameter Name	Initial Value	Minimum	Maximum
Aigc	1.5e-02	1.0e-03	1
Aigd	1.0e-02	1.0e-03	1

Target:

- IGateOn = -54.85p
- IGateOff = 3.273p

Simulation result with initial model parameter value:

- IGateOn = -0.941p
- IGateOff = 0.334p

LM algorithm is used to do the optimization.

Input File

Target data file (targetMeas.txt) setup: ASCII format file.

```
igateon::value=54.85  
igateoff::value=3.273
```

In this example, `igateon` and `igateoff` are PrimeSim HSPICE measurement name, must be consistent with the measurement name defined in SPICE netlist. `value` is reserved keyword. `::` is the delimiter.

Parameter File (param.txt)

```
aigc = (1.5e-02, 1.0e-03, 1)  
aigd = (1.0e-02, 1.0e-03, 1)
```

SPICE Netlist (test.sp)

```
.param vdd=0.7  
.param  
+ aigc = 1.5e-02  
+ aigd = 1.0e-02  
.model nfet nmos  
+ aigc=aigc  
+ aigd=aigd  
+ ...  
...  
**** Measurements  
.meas dc igateon find par('-1*i(vgs1)*1e12') at='vdd'  
.meas dc igateoff find par('i(vgs2)*1e12') at=0  
.end
```

Configuration File (config.json)

JSON format file for MPO run, include input files, optimization algorithm, and optimization engine related settings.

```
{  
  "casePath" : "./test.sp",
```

```
"paramPath" : "./param.txt",  
"targetMeasFile" : "./targetMeas.txt",  
"outDir" : "./out_demo_lm",  
"coreNum": 16,  
"warnMsg": 1,  
"gtol": 0,  
"stol": 0,  
"etol": 0.001,  
"optAlg": "lm"  
}
```

Same `test_mpo.py` script and running command as example 1 required to start MPO.

MPO output of example 2.

`paramOpt.cfg`: Optimized parameters

```
.param aigc = 1.1586521124706234e-02  
.param aigd = 8.4840846269167713e-03
```

`simMeasResult.txt`: Fitting Result

meas_name	target	simResult	diff
igateon	5.48500e+01	5.48892e+01	0.07%
igateoff	3.27300e+00	3.27307e+00	0.0%

Part 5: Encryption and Error and Warning Messages

Describes the encryption and error and warning messages.

Part 5 presents the following chapters/topics:

- Library and Data Encryption
- [Chapter 32, Warning/Error Messages](#)

31

Library and Data Encryption

This chapter has been moved to the PrimeSim Continuum Overview. For more information , see Library and Data Encryption in the *PrimeSim™ Continuum Overview* guide.

32

Warning/Error Messages

Provides an overview of the type of warnings and error messages that PrimeSim HSPICE prints and troubleshooting measures to take when possible.

Users can exercise control over the number of occurrences of warning or error messages and escalate the severity of certain messages by using `.OPTIONS MESSAGE_LIMIT` and `STRICT_CHECK`. See `.OPTION MESSAGE_LIMIT` and `.OPTION STRICT_CHECK` in the *PrimeSim HSPICE Reference Manual: Commands and Control Options*.

PrimeSim HSPICE ships hundreds of examples for your use; see Listing of Demonstration Input Files in the *PrimeSim HSPICE User Guide: Demonstration Netlists* for paths to the demo files.

This chapter discusses the following topics:

- [Warning Messages](#)
- [Error Messages](#)
- [Transient Analysis Errors and Solutions](#)
- [Safe Operating Area \(SOA\) Warnings](#)
- [Verilog-A \(pVA\) Messages](#)
- [Warning Message Index \[00001-11146\]](#)
- [Error Message Index \[20001-20084\]](#)
- [Exit Codes](#)

Warning Messages

The following sections present these topics:

- [Topology Warnings](#)
- [Model Warnings](#)
- [Control Option Warnings](#)

- [Device Warnings](#)
- [Analysis Warnings](#)

Topology Warnings

Note:

To suppress the netlist topology checks and cause no topology warnings or errors to be reported set `.OPTION NOTOP`.

The following sections discuss these topics:

- [Topology Integrity](#)
- [No DC Path to Ground](#)
- [Duplicate Initialization](#)

Topology Integrity

When PrimeSim HSPICE encounters topology integrity issues, it reports warning messages similar to the four types shown:

```
**warning** only 1 connection at node 1:net0107 defined in subckt bg:
called in element 12:mn0 defined in subckt bg at line 161 within the
hspice source, library or include file.

**warning** both nodes of resistor 1:rinp defined in subckt opa350 are
connected. together

**warning** 2:r11 defined in subckt pwdr resistance limited to 1.000E-05

**warning** the following singular supplies were terminated to 1 meg
resistor
supply    node1                      node2
vddl8     0:dvddl8 defined in subckt 0  0:0 defined in subckt0
vddlp8     0:dvddlp8 defined in subckt 0  0:0 defined in subckt 0
```

No DC Path to Ground

The warning for no DC path to ground, effective from 2007.09, is:

```
**warning** no dc path to ground from node 13:fl defined in subckt
d****01 now it is connected with gdcpath.
```

Duplicate Initialization

If a node is initialized using a `.ic` or `.nodeset` more than once, the following warning is issued:

```
**warning** a duplicate initialization for node=l620:ram***, keeping last  
value 0.900 only.
```

Model Warnings

The following sections discuss these topics:

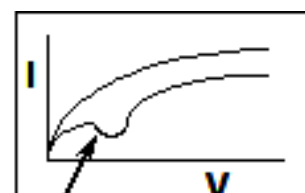
- [Zero or Negative Conductance](#)
- [Encryption-Related Warnings](#)
- [Model Binning Warnings](#)
- [Key Model Parameter Checking](#)
- [Parameter Expression Warning](#)

Zero or Negative Conductance

The following two examples show sample warning messages for negative or zero conductance:

```
**warning** negative-mos conductance = 0:m1 iter= 2  
vds,vgs,vbs =      4.22      2.12      0.925  
gm,gds,gmbs,ids=  1.707E-03  9.366E-05 -1.380E-04  5.040E-04  
  
**warning** conductance of 0. on node 0:net4823 iter= 1
```

The typical causes for these warnings are modeling problems in the subthreshold equations (if in cutoff) or channel length modulation equations (if in saturation). The magnitude reported in the warnings indicates the magnitude of the conductance (leakage) that must be placed across the drain and source to offset the effect of the negative conductance. Typically, `.option GMINDC` and `GMIN` can be used to do this.



**Region of Negative
Conductance
(Negative slope)**

Encryption-Related Warnings

```
**warning** Data associated with encrypted blocks were suppressed due to  
encrypted content
```

```
**warning** Some parameters in encrypted block are defined as an  
expression containing output signals. which may cause incorrect result.  
Suggest to use user defined functions to replace.
```

Model Binning Warnings

```
**warning** (L65***.mdl:2) model n_10_rvl device geometries will  
not be checked against the limits set by lmin, lmax, wmin and wmax. To  
enable this check, add a period(.) to the model name (i.e. enable model  
selector).
```

Key Model Parameter Checking

```
*** warning ***: area for diode can not be 0.0, reset to 1e-12 (default  
value)
```

Parameter Expression Warning

```
**warning** parameter pdt is defined as an expression containing output  
signals, which may cause incorrect result. Suggest to use user defined  
functions to replace.
```

For example:

```
.Param pdt = "rs*pi*i(node2)+v(out)"
```

Control Option Warnings

ACCURATE

```
runlvl smaller than 5, reset to 5 when accurate turned on
```

GMIN, GMINDC

```
**warning** pivtol too large ** reset to half minimum value of  
(gmindc,gmin)
```

Device Warnings

Device warnings are specific to each model.

Device Geometry Check

```
Warning: Pd = 1.36e-06 is less than W.  
Model: 0:nch  
W = 4.444e-06, L = 5.3e-07
```


Device Parameter Check

Warning: Moin = 1568.2 is too large.
Warning: Acde = 0.0350921 is too small.

Analysis Warnings

The following sections discuss these topics:

- [Transient](#)
- [Bisection](#)
- [Measure](#)
- [.DC and .OP Analysis Warnings](#)

Transient

****warning**** the third value 0.00000D+00 and the fourth value 1.00000D-12 are both smaller than the second value 5.00000D-10, so the transient statement is interpreted as `'.tran tstep tstop tstart delmax'`.

Example:

```
.tran 1n 1u 0 1p  
.tran 1n 1u 1p 2u
```

Bisection

With option OPTCON=1

****warning**** endpoints have same sign in bisection
For x = 0.0000 , y = 0.0000 .
For x = 1.0000 , y = 1.0000 .
Both of these are on the same side of the goal value y = 0.30000 .

Multiple Results

****warning**** multiple results used in bisection

Example:

```
.tran 1.0e-9 8.0e-9 sweep optimize=opt1 results=y,z model=opt_model
```

Measure

****warning**** measure results may be incorrect since initial start time is non-zero.
vin_pp= 1.4855E-01 from= 1.5000E-05 to= 2.0000E-05

```
**warning** the Equation Evaluation form of the .MEASURE statement  
must not be a function of node voltages or branch currents. Unexpected  
results may incur.
```

Example:

```
.MEAS VARG PARAM='(V(2) + V(3))/2'
```

.DC and .OP Analysis Warnings

When both DC and TRAN source are defined

```
**warning** dc voltage reset to initial transient source value in source  
0:vclk new dc= 0.0000D+00
```

Example:

```
vlo2 in gnd dc vhaf sin(0 '(pwr(10,((toin)/20)))*(1e-6)*SQ2' fq 0 0 180)
```

Character line limit warning: The PrimeSim HSPICE line limit is 1024 characters.

```
**warning** node full pathname length in .ic file greater than limit,  
node NOT initialized in the save file nodeset.ic
```

Autoconverge overflow message

```
**warning** Due to a floating point overflow problem, the damped  
pseudo-tran method was used. Also, gmindc was set to 1.0000E-11
```

Difficult operating point calculation warning message

```
**warning** This was a difficult operating point. You can speed up your  
simulation by specifying:.OPTION CONVERGE=4
```

Auto-convergence flow messages

```
convergence problems in dc sweep curves at 15.894 resimulating with dc  
convergence controls
```

```
**diagnostic** dc convergence failure, resetting dcon option to 1 and  
retrying with dcon=1, it converged for gmindc= 5.500E-14
```

```
**diagnostic** dc convergence failure,  
resetting dcon option to 2 and retrying.
```

```
**diagnostic** although this circuit has failed to converge to gmindc=  
1.000E-15, it did converge to a gmindc= 7.662E-15 for most circuits a  
value of gmindc 1e-7 or less, is acceptable
```

Operating point diagnostic failure messages

```
**diagnostic** number of iteration exceeds min (7000, 20*itl1)=7000 in  
pseudo tran process (converge=1 process). Usually this happens when the  
models are discontinuous, or there are uninitialized bi-stable cells
```

(flip-flop) in the circuit. By setting options `dcon=-1` and `converge=-1` you can disable auto convergence process. Retry the run, non-convergence diagnostics will provide useful information about the nodes and devices which can be used to work around the non-convergence problems.

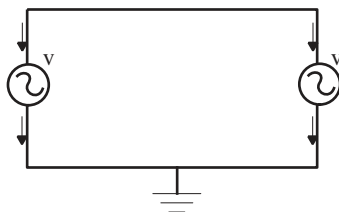
Error Messages

The following sections present these topics:

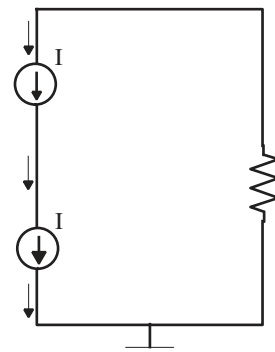
- [Topology Errors](#)
- [Model Errors](#)
- [Analysis Errors](#)

Topology Errors

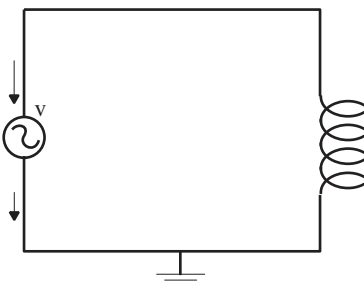
When constructing the circuit description PrimeSim HSPICE does not allow certain topologies. Topology errors will be reported according the following circumstances:



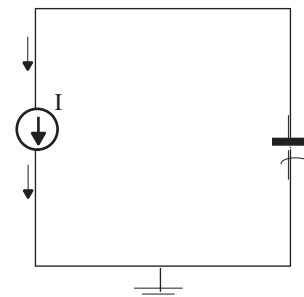
No voltage loops: no voltage sources in parallel with no other elements



No stacked current sources: no current sources in series



No ideal voltage source in closed inductor loop



No ideal current source in closed capacitor loop

Negative or 0 multiplier is not allowed

```
**error** Value of multiplier parameter in 1:x1 is less than or equal to 0
```

Model Errors

Undefined Model

```
**error** model name pch in the element 0:mp is not defined.
```

Redundant Model Definition

```
**error** above line attempts to redefine tnl  
**error** (../models/res2:4) difficulty in reading input
```

Undefined parameter

```
**error** no definition for 0:rsit was called by 0:rin  
**error** no definition for 0:toxn it was called by 0:n
```

Analysis Errors

The following sections describe:

- [.DC and Operating Convergence Errors](#)
- [Convergence Termination Criteria](#)
- [Non-Convergence: Possible Causes](#)
- [DC/OP Convergence Aids](#)
- [DC Bias Point Convergence Actions](#)
- [DC Bias Point Troubleshooting with .NODESET and .IC](#)
- [Troubleshooting Model-Related DC Bias Point Issues](#)
- [Impact of Conductance on Convergence](#)

.DC and Operating Convergence Errors

No convergence error

```
**error** no convergence in operating point
```

No convergence at a .DC sweep point

```
**error** no convergence in dc sweep curves at 15.851
```

Operating Point Debugging Information

```
*** hspice diagnostic *** nonconvergent voltage failures= 33803
nonconvergent element current failures= 1
```

Convergence Termination Criteria

NC is the # of non-convergent nodes, currents, or MOSFETs

Other Iteration: Iteration_Number = Iteration_Number + 1

NC = 0

I=1, # Circuit Nodes

if ($|V(n) - V(n-1)| > RELV * V(n) + ABSV$) then NC = NC + 1

I=1, # Branch Currents

if ($|I(n) - I(n-1)| > RELI * I(n) + ABSI$) then NC = NC + 1

I=1, # MOSFETs

if ($|Ids(n) - Ids(n-1)| > RELMOS * Ids(n) + ABSMOS$) then NC = NC + 1

NC = 0

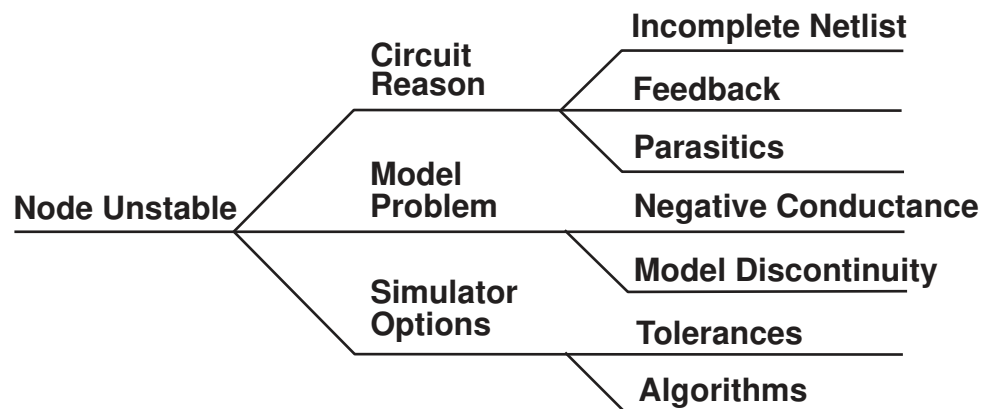
Save Solution

end

If (Iteration_Number < Iteration_Limit) do Another_Iteration

else Failed to Converge

Non-Convergence: Possible Causes



DC/OP Convergence Aids

Aids to Remedy DC Bias Non-Convergence

- Auto-convergence process
- .NODESET/.IC Commands (see the following sections)
- Model-related solutions
- Others, with less impact
 - GDCPATH and GMINDC ramping
 - Source stepping/ramping
 - GSHUNT/CSHDC
 - DV

DC Bias Point Convergence Actions

Take the following actions to resolve issues dealing with DC bias point non-convergence:

- Remove all options *except* `node`, `list`, `post`, and `opts`
- Allow the auto converge process to proceed
- Review the `.lis` file for convergence hints
- Search for “warning” and “error” messagesRerun the simulation

DC Bias Point Troubleshooting with .NODESET and .IC

Non-convergence can occur due to poor initial conditions. Set initial conditions and/or nodesets. For example:

```
.IC v(1)=5v v(abc)=0v v(12)=VDD
```

```
.NODESET v(x1.87)=5v
```

Identify the problem nodes by:

- Reviewing the non-convergent diagnostic table in the listing file
- Identifying non-convergent nodes with unusually high voltages, branch currents, or high error tolerances
- Initializing these nodesReviewing the circuit for un-initialized feedback paths (flip-flops, oscillators, etc.)

Because it is inefficient to manually add `.NODESET` and/or `.IC` for a large number of nodes, to set a large number of nodes:

1. Comment out all analysis commands except `.TRAN`.
2. Add `UIC` to the end of the `.TRAN` command.
3. Disable auto-convergence process by setting:

```
.option DCON=-1 CONVERGE=-1
```
4. Use `.SAVE [TYPE=<nodeset|ic>] [TIME=<x>]` to store the calculated operating point as a `.ic` or `.nodeset` file.
5. Simulate the circuit.
6. Use `.LOAD` for loading the file from the `.SAVE` command.
7. Enable the auto-converge process by removing the `DCON` and `CONVERGE` options.
8. Remove `UIC` from the `.TRAN` command.
9. Re-simulate the circuit.

Troubleshooting Model-Related DC Bias Point Issues

Inappropriate model parameters are usually the cause having to do with units or negative/zero conductance:

- If the issue is units:
 - `.OPTION SCALM` (global)
 - `.MODEL SCALM` factor (local value within `.MODEL` statements)
- Is a global `.OPTION SCALM` needed?

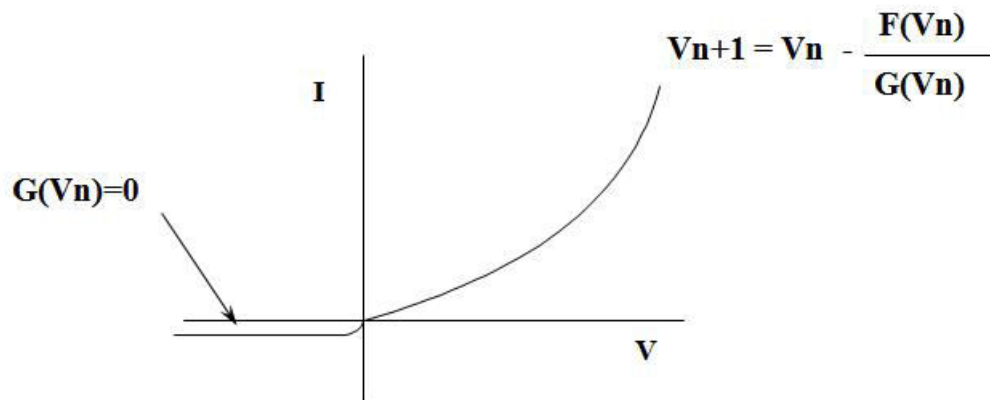
Look at the listing file and review element values.

- Swap in a known good model.

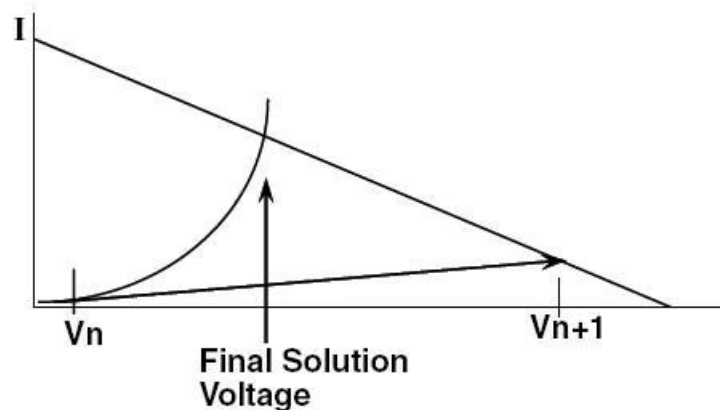
Impact of Conductance on Convergence

This section describes issues and possible solutions when conductance values impact on convergence. For example: Using a conductance term to predict the next voltage value, can create the problem that if conductance becomes small, the 2nd term becomes large:

- Next voltage value unrealistic
- Causes extra iterations
- Worse: Conductance of zero!



- Since all semiconductor device models contain regions of zero conductance:
 - Shunt R placed in parallel with every PN junction and drain to source
 - Determine smallest parasitic R_p that can be placed across any 2 nodes without influencing circuit behavior
 - $G = 1/R_p$
 - Try setting `.OPTION GMINDC=1e-9 GMIN=1e-9`
 - Default for both $GMIN$ and $GMINDC$ is $1e-12$
- You must ask, “How much leakage is acceptable?”
 - Typically, a setting $GMIN=1e-10$ does not affect CMOS circuit accuracy
 - Larger values of $GMIN$ will affect accuracy and indicate that there may be a model problem



The following sections describe:

- [Convergence/Diode Resistance](#)
- [Analysis Options: DIAGNOSTIC](#)

Convergence/Diode Resistance

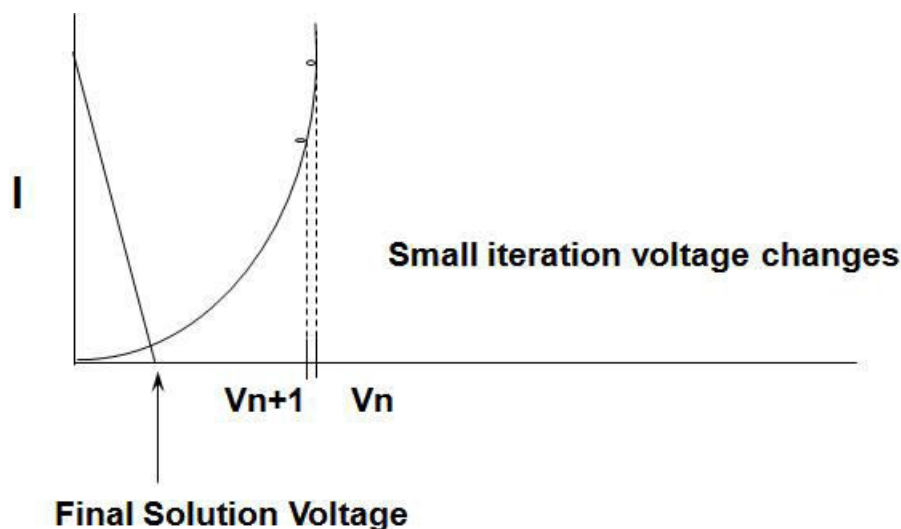
High conductance is troublesome to the algorithm:

Problem: Highly forward-biased diodes (greater than 0.8V)

- Lead to very small iteration-to-iteration voltage changes
- Can cause PrimeSim HSPICE to reach iteration limit before reaching the proper solution voltage

Solution:

- Always specify the series-resistance model parameter for all diodes, bipolar devices, and MOSFETs in the circuit (Default is ZERO ohms).
- At high forward bias, the series resistance dominates the conductance of the device and helps reduce the occurrence of non-convergence.



- Series resistance model parameters for active devices:
 - Diode R_S
 - Bipolar transistor R_E and R_C

- JFET RD and RS
- MOSFET RD and RS

Analysis Options: DIAGNOSTIC

PrimeSim HSPICE automatically prints out the first occurrence of “negative-mos conductance” in the `.lis` file.

The `.option DIAGNOSTIC`:

- Causes all occurrences of negative model conductances to be printed in the `.lis` file
- If the magnitude of the negative conductance is $> -1\text{e-}8$, consult your modeling department or foundry

Transient Analysis Errors and Solutions

The following sections discuss these topics:

- [Transient Analysis Error](#)
- [Transient Non-Convergence](#)
- [Transient Convergence Aids](#)

Transient Analysis Error

- Most frequent error message:

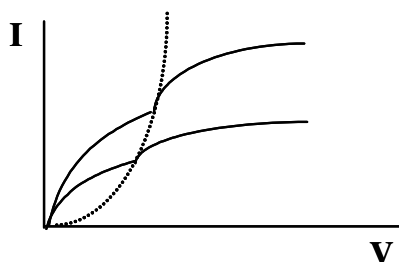
```
**error**  internal timestep too small in transient analysis
```
- Occurs when: $\text{Internal timestep} < \text{RMIN} * \text{TSTEP}$ TSTEP is from `.TRAN` statement.

Transient Non-Convergence

Rapid Voltage Transitions:

- Dynamic timestep control automatically reduces the timestep size
- As the circuit approaches a voltage transition, two potentially conflicting events occur:
 - Semiconductor devices are switching from one region of operation to another.
 - Timestep is reduced.

Model Discontinuities:



- Separate equations used for the different operating regions of an active device
- Most common discontinuities are at the intersection of the linear and saturation regions
- Failure Mechanism
 - Newton-Raphson can oscillate back and forth across the discontinuity
 - Oscillations use up iterations without progressing toward a solution
 - A sweep increases likelihood of hitting model discontinuities

Transient Convergence Aids

Corrective actions include:

- Device model capacitance
- GEAR integration
- Use RUNLVL option

Device model capacitance—Transient non-convergence is primarily caused by a combination of model discontinuities and a reduced step size brought on by voltage transitions within the circuit:

- All simulation models should have their associated capacitance terms set to a non-zero value.
- Real models have real capacitances.
- Capacitive Model Parameters:
 - Diode CJO
 - Bipolar CJE, CJC, CJS
 - JFET CGD, CGS
 - MOSFET CGDO, CGSO, CGDO, CBD, CBS, CJ, CJSW

GEAR Integration:

- Numeric integration of time varying currents and voltages are accomplished through Trapezoidal and Gear linearization.
- GEAR integration acts as a filter, removing oscillations that can occur due to the trapezoidal algorithm.
- Circuits that are non-convergent with TRAP will often converge with GEAR.

Why GEAR sometimes converges where TRAP fails:

- Gear integration uses a “weighted average” of past timesteps to determine the next time step.
- This past history helps to project over model discontinuities that exist.

Use of the RUNLVL option:

- This option has an enhanced convergence algorithm providing less chance of encountering “time step too small” error.
- RUNLVL=3 (default setting) is similar to default PrimeSim HSPICE setting.
- RUNLVL=5 is similar to setting ACCURATE option.

Safe Operating Area (SOA) Warnings

You can set `.option warn` and `.option maxwarns` to have PrimeSim HSPICE issue warnings when terminal voltages of a device (MOSFET, HV, BJT, diode, capacitor, resistor, etc.) exceed the SOA (Safe Operating Area). All warning message parameters (e.g., `Bv_max`, `Vbe_max`, etc.) are positive with default value of infinity.

See the following control options for details:

- `.OPTION WARN`
- `.OPTION MAXWARNS`

For a listing of the checking criteria, see the following topics:

- [MOSFET / JFET](#)
- [SOI MOSFET \(LEVEL 70\)](#)
- [BJT](#)
- [Diodes](#)

- [Resistor](#)
- [Capacitor](#)

MOSFET / JFET

Terminal voltages checked: Vgs, Vgd, Vgb, Vds, Vbs, and Vbd.

Model parameters: Vgs_max, Vgd_max, Vds_max, Vbd_max, Vbs_max, Vgb_max, Vgsr_max, Vgdr_max, Vgbr_max, Vbsr_max, and Vbdr_max.

Terminal Voltages Checked	Model Params	Check criteria (i.e., the device under checking is called M1) and Warning Issued
The device under checking is called M1, for example.		
Vgs	Vgs_max Vgsr_max	If Vgsr_max is not given if $ V_{gs} > V_{gs_max}$, issue "Warning: Vgs(=xxx) of M1 has exceeded Vgs_max(=yyy)" If Vgb_max is not given if $ V_{gb} > V_{gs_max}$, issue "Warning: Vgb(=xxx) of M1 has exceeded Vgs_max(=yyy)" Else if $ V_{gb} > V_{gb_max}$, issue "Warning: Vgb(=xxx) of M1 has exceeded Vgb_max(=yyy)" Else NMOS/NJF: if $V_{gs} > V_{gs_max}$, issue "Warning: Vgs(=xxx) of M1 has exceeded Vgs_max(=yyy)" if $-1 * V_{gs} > V_{gsr_max}$, issue "Warning: Vgs(=xxx) of M1 has exceeded Vgsr_max(=yyy)" PMOS/PJF: if $V_{gs} > V_{gsr_max}$, issue "Warning: Vgs(=xxx) of M1 has exceeded Vgsr_max(=yyy)" if $-1 * V_{gs} > V_{gs_max}$, issue "Warning: Vgs(=xxx) of M1 has exceeded Vgs_max(=yyy)"
Vgb	Vgb_max Vgbr_max	If Vgbr_max is not given if $ V_{gb} > V_{gb_max}$, issue "Warning: Vgb(=xxx) of M1 has exceeded Vgb_max(=yyy)" Else NMOS/NJF: if $V_{gb} > V_{gb_max}$, issue "Warning: Vgb(=xxx) of M1 has exceeded Vgb_max(=yyy)" if $-1 * V_{gb} > V_{gbr_max}$, issue "Warning: Vgb(=xxx) of M1 has exceeded Vgbr_max(=yyy)" PMOS/PJF: if $V_{gb} > V_{gbr_max}$, issue "Warning: Vgb(=xxx) of M1 has exceeded Vgbr_max(=yyy)" if $-1 * V_{gb} > V_{gb_max}$, issue "Warning: Vgb(=xxx) of M1 has exceeded Vgb_max(=yyy)"
Vgd	Vgd_max Vgdr_max	If Vgdr_max is not given if $ V_{gd} > V_{gd_max}$, issue "Warning: Vgd(=xxx) of M1 has exceeded Vgd_max(=yyy)" Else NMOS/NJF: if $V_{gd} > V_{gd_max}$, issue "Warning: Vgd(=xxx) of M1 has exceeded Vgd_max(=yyy)" if $-1 * V_{gd} > V_{gdr_max}$, issue "Warning: Vgd(=xxx) of M1 has exceeded Vgdr_max(=yyy)" PMOS/PJF: if $V_{gd} > V_{gdr_max}$, issue "Warning: Vgd(=xxx) of M1 has exceeded Vgdr_max(=yyy)" if $-1 * V_{gd} > V_{gd_max}$, issue "Warning: Vgd(=xxx) of M1 has exceeded Vgd_max(=yyy)"

Terminal Voltages Checked	Model Params	Check criteria (i.e., the device under checking is called M1) and Warning Issued
Vds	Vds_max	If $ Vds > Vds_max$, issue "Warning: Vds (=xxx) of M1 has exceeded Vds_max (=yyy)"
Vbd	Vbd_max	If Vbdr_max is not given if $Vbd > Vbd_max$, issue "Warning: Vbd (=xxx) of M1 has exceeded Vbd_max (=yyy) Else NMOS/NJF: if $-1 \cdot Vbd > Vbd_max$, issue "Warning: Vbd(=xxx) of M1 has exceeded Vbd_max (=yyy). if $Vbd > Vbdr_max$, issue "Warning: Vbd(=xxx) of M1 has exceeded Vbdr_max(=yyy). PMOS/PJF: if $-1 \cdot Vbd > Vbdr_max$, issue "Warning: Vbd(=xxx) of M1 has exceeded Vbdr_max (=yyy). if $Vbd > Vbd_max$, issue "Warning: Vbd(=xxx) of M1 has exceeded Vbd_max (=yyy).
Vbs	Vbs_max	If Vbsr_max is not given If Vbs_max is not given if $Vbs > Vbd_max$, issue "Warning: Vbs (=xxx) of M1 has exceeded Vbd_max (=yyy). Else if $Vbs > Vbs_max$, issue "Warning: Vbs (=xxx) of M1 has exceeded Vbs_max (=yyy). Else If Vbs_max is not given NMOS/NJF: if $-1 \cdot Vbs > Vbd_max$, issue "Warning: Vbs (=xxx) of M1 has exceeded Vbd_max (=yyy). if $Vbs > Vbsr_max$, issue "Warning: Vbs(=xxx) of M1 has exceeded Vbsr_max (=yyy). PMOS/PJF: if $-1 \cdot Vbs > Vbsr_max$, issue "Warning: Vbs(=xxx) of M1 has exceeded Vbsr_max (=yyy). if $Vbs > Vbd_max$, issue "Warning: Vbs (=xxx) of M1 has exceeded Vbd_max (=yyy). Else NMOS/NJF: if $-1 \cdot Vbs > Vbs_max$, issue "Warning: Vbs (=xxx) of M1 has exceeded Vbs_max (=yyy). if $Vbs > Vbsr_max$, issue "Warning: Vbs (=xxx) of M1 has exceeded Vbsr_max (=yyy). PMOS/PJF: if $-1 \cdot Vbs > Vbsr_max$, issue "Warning: Vbs(=xxx) of M1 has exceeded Vbs_max (=yyy). if $Vbs > Vbs_max$, issue "Warning: Vbs(=xxx) of M1 has exceeded Vbs_max (=yyy).

Note:

Special considerations:

1. Vgd has different warning parameter for HV device considerations.
2. If Vbs_max is not given, Vbs and Vbd share the same model warning parameter "Vbd_max".
3. 5T, 6T, 7T models are implemented through macro models. The junction breakdown warnings will be issues by the diodes in the macro model.

4. If Vgb_max is not given, Vgs and Vgb share the same model warning parameter "Vgs_max".
5. If Vgsr_max is not given, Vgsr_max share the same model warning parameter "Vgs_max".
6. If Vgdr_max is not given, Vgdr_max share the same model warning parameter "Vgd_max".
7. If Vgbr_max is not given, Vgbr_max share the same model warning parameter "Vgb_max".
8. If Vbsr_max is not given, Vbsr_max share the same model warning parameter "Vbs_max".
9. If Vbdr_max is not given, Vbdr_max share the same model warning parameter "Vbd_max".
10. Not include SOI MOSFET (LEVEL 70)

SOI MOSFET (LEVEL 70)

Terminal voltages checked: Vgs, Vgd, Vgp, Vds, Vps, Vpd, Vge, Ved, Ves, Vpe, Vgb, Vbs, Vbd, and Vbe.

Model parameters: Vgs_max, Vgd_max, Vds_max, Vpd_max, Vps_max, Vgp_max, Vgsr_max, Vgdr_max, Vgpr_max, Vpsr_max, Vpdr_max, Vge_max, Ved_max, Ves_max, Vpe_max, Vgb_max, Vbs_max, Vbd_max, Vbe_max, Vger_max, Vedr_max, Vesr_max, Vper_max, Vgbr_max, Vbsr_max, Vbdr_max, and Vber_max.

Terminal Voltages Checked	Model Parameters	Check Criteria (that is, the device under checking is called M1) and Warning Issued
Vgs	Vgs_max	If Vgsr_max is not given
	Vgsr_max	if $ Vgs > Vgs_max$, issue "Warning: Vgs (=xxx) of M1 has exceeded Vgs_max (=yyy) "
		if Vgp_max is not given
		if $ Vgp > Vgs_max$, issue "Warning: Vgp (=xxx) of M1 has exceeded Vgs_max (=yyy)"
		Else
		if $ Vgp > Vgp_max$, issue "Warning: Vgp (=xxx) of M1 has exceeded Vgp_max (=yyy)"
		Else

Terminal Voltages Checked	Model Parameters	Check Criteria (that is, the device under checking is called M1) and Warning Issued
		NMOS: if $V_{gs} > V_{gs_max}$, issue "Warning: $V_{gs}(=xxx)$ of M1 has exceeded $V_{gs_max}(=yyy)$" if $-1 \cdot V_{gs} > V_{gsr_max}$, issue "Warning: $V_{gs}(=xxx)$ of M1 has exceeded $V_{gsr_max}(=yyy)$" PMOS: if $V_{gs} > V_{gsr_max}$, issue "Warning: $V_{gs}(=xxx)$ of M1 has exceeded $V_{gsr_max}(=yyy)$" if $-1 \cdot V_{gs} > V_{gs_max}$, issue "Warning: $V_{gs}(=xxx)$ of M1 has exceeded $V_{gs_max}(=yyy)$"
Vgp	Vgp_max	If Vgpr_max is not given
	Vgpr_max	if $V_{gp} > V_{gp_max}$, issue "Warning: $V_{gp}(=xxx)$ of M1 has exceeded $V_{gp_max}(=yyy)$" Else NMOS: if $V_{gp} > V_{gp_max}$, issue "Warning: $V_{gp}(=xxx)$ of M1 has exceeded $V_{gp_max}(=yyy)$" if $-1 \cdot V_{gp} > V_{gpr_max}$, issue "Warning: $V_{gp}(=xxx)$ of M1 has exceeded $V_{gpr_max}(=yyy)$" PMOS: if $V_{gp} > V_{gpr_max}$, issue "Warning: $V_{gp}(=xxx)$ of M1 has exceeded $V_{gpr_max}(=yyy)$" if $-1 \cdot V_{gp} > V_{gp_max}$, issue "Warning: $V_{gp}(=xxx)$ of M1 has exceeded $V_{gp_max}(=yyy)$"
Vgd	Vgd_max	If Vgdr_max is not given
	Vgdr_max	if $V_{gd} > V_{gd_max}$, issue "Warning: $V_{gd}(=xxx)$ of M1 has exceeded $V_{gd_max}(=yyy)$" Else NMOS:

Terminal Voltages Checked	Model Parameters	Check Criteria (that is, the device under checking is called M1) and Warning Issued
		if $V_{gd} > V_{gd_max}$, issue "Warning: $V_{gd}(=xxx)$ of M1 has exceeded $V_{gd_max}(=yyy)$" if $-1 \cdot V_{gd} > V_{gdr_max}$, issue "Warning: $V_{gd}(=xxx)$ of M1 has exceeded $V_{gdr_max}(=yyy)$" PMOS: if $V_{gd} > V_{gdr_max}$, issue "Warning: $V_{gd}(=xxx)$ of M1 has exceeded $V_{gdr_max}(=yyy)$" if $-1 \cdot V_{gd} > V_{gd_max}$, issue "Warning: $V_{gd}(=xxx)$ of M1 has exceeded $V_{gd_max}(=yyy)$"
V_{ds}	V_{ds_max}	if $ V_{ds} > V_{ds_max}$, issue "Warning: $V_{ds} (=xxx)$ of M1 has exceeded $V_{ds_max} (=yyy)$ "
V_{pd}	V_{pd_max}	If V_{pdr_max} is not given
	V_{pdr_max}	if $V_{pd} > V_{pd_max}$, issue "Warning: $V_{pd} (=xxx)$ of M1 has exceeded $V_{pd_max} (=yyy)$" Else NMOS: if $-1 \cdot V_{pd} > V_{pd_max}$, issue "Warning: $V_{pd}(=xxx)$ of M1 has exceeded $V_{pd_max} (=yyy)$" if $V_{pd} > V_{pdr_max}$, issue "Warning: $V_{pd}(=xxx)$ of M1 has exceeded $V_{pdr_max}(=yyy)$" PMOS: if $-1 \cdot V_{pd} > V_{pdr_max}$, issue "Warning: $V_{pd}(=xxx)$ of M1 has exceeded $V_{pdr_max} (=yyy)$" if $V_{pd} > V_{pd_max}$, issue "Warning: $V_{pd}(=xxx)$ of M1 has exceeded $V_{pd_max} (=yyy)$"
V_{ps}	V_{ps_max}	If V_{psr_max} is not given
	V_{psr_max}	If V_{ps_max} is not given if $V_{ps} > V_{pd_max}$, issue "Warning: $V_{ps} (=xxx)$ of M1 has exceeded $V_{pd_max} (=yyy)$"

Terminal Voltages Checked	Model Parameters	Check Criteria (that is, the device under checking is called M1) and Warning Issued
		Else if $V_{ps} > V_{ps_max}$, issue "Warning: V_{ps} (=xxx) of M1 has exceeded V_{ps_max} (=yyy)" Else If V_{ps_max} is not given NMOS if $-1 * V_{ps} > V_{pd_max}$, issue "Warning: V_{ps} (=xxx) of M1 has exceeded V_{pd_max} (=yyy)" if $V_{ps} > V_{psr_max}$, issue "Warning: V_{ps}(=xxx) of M1 has exceeded V_{psr_max} (=yyy)" PMOS if $-1 * V_{ps} > V_{psr_max}$, issue "Warning: V_{ps}(=xxx) of M1 has exceeded V_{psr_max} (=yyy)" if $V_{ps} > V_{pd_max}$, issue "Warning: V_{ps} (=xxx) of M1 has exceeded V_{pd_max} (=yyy)" Else NMOS if $-1 * V_{ps} > V_{ps_max}$, issue "Warning: V_{ps} (=xxx) of M1 has exceeded V_{ps_max} (=yyy)" if $V_{ps} > V_{psr_max}$, issue "Warning: V_{ps} (=xxx) of M1 has exceeded V_{psr_max} (=yyy)" PMOS if $-1 * V_{ps} > V_{psr_max}$, issue "Warning: V_{ps}(=xxx) of M1 has exceeded V_{ps_max} (=yyy)" if $V_{ps} > V_{ps_max}$, issue "Warning: V_{ps}(=xxx) of M1 has exceeded V_{ps_max} (=yyy)"
Vge	Vge_max	If Vger_max is not given
	Vger_max	if $ V_{ge} > V_{ge_max}$, issue "Warning: V_{ge} (=xxx) of M1 has exceeded V_{ge_max} (=yyy)"

Terminal Voltages Checked	Model Parameters	Check Criteria (that is, the device under checking is called M1) and Warning Issued
		Else NMOS: if $V_{ge} > V_{ge_max}$, issue "Warning: $V_{ge}(=xxx)$ of M1 has exceeded $V_{ge_max}(=yyy)$" if $-1 \cdot V_{ge} > V_{ger_max}$, issue "Warning: $V_{ge}(=xxx)$ of M1 has exceeded $V_{ger_max}(=yyy)$" PMOS: if $V_{ge} > V_{ger_max}$, issue "Warning: $V_{ge}(=xxx)$ of M1 has exceeded $V_{ger_max}(=yyy)$" if $-1 \cdot V_{ge} > V_{ge_max}$, issue "Warning: $V_{ge}(=xxx)$ of M1 has exceeded $V_{ge_max}(=yyy)$"
Ved	Ved_max	If Vedr_max is not given
	Vedr_max	if $V_{ed} > V_{ed_max}$, issue "Warning: $V_{ed}(=xxx)$ of M1 has exceeded $V_{ed_max}(=yyy)$" Else NMOS: if $-1 \cdot V_{ed} > V_{ed_max}$, issue "Warning: $V_{ed}(=xxx)$ of M1 has exceeded $V_{ed_max}(=yyy)$" if $V_{ed} > V_{edr_max}$, issue "Warning: $V_{ed}(=xxx)$ of M1 has exceeded $V_{edr_max}(=yyy)$" PMOS: if $-1 \cdot V_{ed} > V_{edr_max}$, issue "Warning: $V_{ed}(=xxx)$ of M1 has exceeded $V_{edr_max}(=yyy)$" if $V_{ed} > V_{ed_max}$, issue "Warning: $V_{ed}(=xxx)$ of M1 has exceeded $V_{ed_max}(=yyy)$"
Ves	Ves_max	If Vesr_max is not given
	Vesr_max	If Ves_max is not given if $V_{es} > V_{ed_max}$, issue "Warning: $V_{es}(=xxx)$ of M1 has exceeded $V_{ed_max}(=yyy)$"

Terminal Voltages Checked	Model Parameters	Check Criteria (that is, the device under checking is called M1) and Warning Issued
		Else if $V_{es} > V_{es_max}$, issue "Warning: V_{es} (=xxx) of M1 has exceeded V_{es_max} (=yyy)" Else If V_{es_max} is not given NMOS if $-1 * V_{es} > V_{ed_max}$, issue "Warning: V_{es} (=xxx) of M1 has exceeded V_{ed_max} (=yyy)" if $V_{es} > V_{esr_max}$, issue "Warning: V_{es} (=xxx) of M1 has exceeded V_{esr_max} (=yyy)" PMOS if $-1 * V_{es} > V_{esr_max}$, issue "Warning: V_{es} (=xxx) of M1 has exceeded V_{esr_max} (=yyy)" if $V_{es} > V_{ed_max}$, issue "Warning: V_{es} (=xxx) of M1 has exceeded V_{ed_max} (=yyy)" Else NMOS if $-1 * V_{es} > V_{es_max}$, issue "Warning: V_{es} (=xxx) of M1 has exceeded V_{es_max} (=yyy)" if $V_{es} > V_{esr_max}$, issue "Warning: V_{es} (=xxx) of M1 has exceeded V_{esr_max} (=yyy)" PMOS if $-1 * V_{es} > V_{esr_max}$, issue "Warning: V_{es} (=xxx) of M1 has exceeded V_{es_max} (=yyy)" if $V_{es} > V_{es_max}$, issue "Warning: V_{es} (=xxx) of M1 has exceeded V_{es_max} (=yyy)"
Vpe	Vpe_max	If Vper_max is not given
	Vper_max	if $ V_{pe} > V_{pe_max}$, issue "Warning: V_{pe} (=xxx) of M1 has exceeded V_{pe_max} (=yyy)"

Terminal Voltages Checked	Model Parameters	Check Criteria (that is, the device under checking is called M1) and Warning Issued
		Else NMOS: if $V_{pe} > V_{pe_max}$, issue "Warning: $V_{pe}(=xxx)$ of M1 has exceeded $V_{pe_max}(=yyy)$" if $-1 \cdot V_{pe} > V_{per_max}$, issue "Warning: $V_{pe}(=xxx)$ of M1 has exceeded $V_{per_max}(=yyy)$" PMOS: if $V_{pe} > V_{per_max}$, issue "Warning: $V_{pe}(=xxx)$ of M1 has exceeded $V_{per_max}(=yyy)$" if $-1 \cdot V_{pe} > V_{pe_max}$, issue "Warning: $V_{pe}(=xxx)$ of M1 has exceeded $V_{pe_max}(=yyy)$"
Vgb	Vgb_max	If Vgbr_max is not given
	Vgbr_max	if $V_{gb} > V_{gb_max}$, issue "Warning: $V_{gb}(=xxx)$ of M1 has exceeded $V_{gb_max}(=yyy)$" Else NMOS: if $V_{gb} > V_{gb_max}$, issue "Warning: $V_{gb}(=xxx)$ of M1 has exceeded $V_{gb_max}(=yyy)$" if $-1 \cdot V_{gb} > V_{gbr_max}$, issue "Warning: $V_{gb}(=xxx)$ of M1 has exceeded $V_{gbr_max}(=yyy)$" PMOS: if $V_{gb} > V_{gbr_max}$, issue "Warning: $V_{gb}(=xxx)$ of M1 has exceeded $V_{gbr_max}(=yyy)$" if $-1 \cdot V_{gb} > V_{gb_max}$, issue "Warning: $V_{gb}(=xxx)$ of M1 has exceeded $V_{gb_max}(=yyy)$"
Vbs	Vbs_max	If Vbsr_max is not given
	Vbsr_max	If V_{bs_max} is not given if $V_{bs} > V_{bd_max}$, issue "Warning: $V_{bs}(=xxx)$ of M1 has exceeded $V_{bd_max}(=yyy)$"

Terminal Voltages Checked	Model Parameters	Check Criteria (that is, the device under checking is called M1) and Warning Issued
		Else if $V_{bs} > V_{bs_max}$, issue "Warning: V_{bs} (=xxx) of M1 has exceeded V_{bs_max} (=yyy)" Else If V_{bs_max} is not given NMOS if $-1 \cdot V_{bs} > V_{bd_max}$, issue "Warning: V_{bs} (=xxx) of M1 has exceeded V_{bd_max} (=yyy)" if $V_{bs} > V_{bsr_max}$, issue "Warning: V_{bs}(=xxx) of M1 has exceeded V_{bsr_max} (=yyy)" PMOS if $-1 \cdot V_{bs} > V_{bsr_max}$, issue "Warning: V_{bs}(=xxx) of M1 has exceeded V_{bsr_max} (=yyy)" if $V_{bs} > V_{bd_max}$, issue "Warning: V_{bs} (=xxx) of M1 has exceeded V_{bd_max} (=yyy)" Else NMOS if $-1 \cdot V_{bs} > V_{bs_max}$, issue "Warning: V_{bs} (=xxx) of M1 has exceeded V_{bs_max} (=yyy)" if $V_{bs} > V_{bsr_max}$, issue "Warning: V_{bs} (=xxx) of M1 has exceeded V_{bsr_max} (=yyy)" PMOS if $-1 \cdot V_{bs} > V_{bsr_max}$, issue "Warning: V_{bs}(=xxx) of M1 has exceeded V_{bs_max} (=yyy)" if $V_{bs} > V_{bs_max}$, issue "Warning: V_{bs}(=xxx) of M1 has exceeded V_{bs_max} (=yyy)"
Vbd	Vbd_max	If Vbdr_max is not given
	Vbdr_max	if $ V_{bd} > V_{bd_max}$, issue "Warning: V_{bd} (=xxx) of M1 has exceeded V_{bd_max} (=yyy)"

Terminal Voltages Checked	Model Parameters	Check Criteria (that is, the device under checking is called M1) and Warning Issued
		Else NMOS: if $-1 \cdot V_{bd} > V_{bd_max}$, issue "Warning: $V_{bd}(=xxx)$ of M1 has exceeded $V_{bd_max}(=yyy)$" if $V_{bd} > V_{bdr_max}$, issue "Warning: $V_{bd}(=xxx)$ of M1 has exceeded $V_{bdr_max}(=yyy)$" PMOS: if $-1 \cdot V_{bd} > V_{bdr_max}$, issue "Warning: $V_{bd}(=xxx)$ of M1 has exceeded $V_{bdr_max}(=yyy)$" if $V_{bd} > V_{bd_max}$, issue "Warning: $V_{bd}(=xxx)$ of M1 has exceeded $V_{bd_max}(=yyy)$"
Vbe	Vbe_max	If Vber_max is not given
	Vber_max	if $V_{be} > V_{be_max}$, issue "Warning: $V_{be}(=xxx)$ of M1 has exceeded $V_{be_max}(=yyy)$" Else NMOS: if $V_{be} > V_{be_max}$, issue "Warning: $V_{be}(=xxx)$ of M1 has exceeded $V_{be_max}(=yyy)$" if $-1 \cdot V_{be} > V_{ber_max}$, issue "Warning: $V_{be}(=xxx)$ of M1 has exceeded $V_{ber_max}(=yyy)$" PMOS: if $V_{be} > V_{ber_max}$, issue "Warning: $V_{be}(=xxx)$ of M1 has exceeded $V_{ber_max}(=yyy)$" if $-1 \cdot V_{be} > V_{be_max}$, issue "Warning: $V_{be}(=xxx)$ of M1 has exceeded $V_{be_max}(=yyy)$"

Note:

Special considerations:

1. If V_{ps_max} is not given, V_{ps} and V_{pd} share the same model warning parameter " V_{pd_max} "
2. If V_{bs_max} is not given, V_{bs} and V_{bd} share the same model warning parameter " V_{bd_max} "
3. If 5T, 6T, 7T models are implemented through macro models, the junction breakdown warnings will be issues by the diodes in the macro model.
4. If V_{gp_max} is not given, V_{gs} and V_{gp} share the same model warning parameter " V_{gs_max} "
5. If V_{gb_max} is not given, V_{gs} and V_{gb} share the same model warning parameter " V_{gs_max} "
6. If V_{gsr_max} is not given, V_{gsr_max} share the same model warning parameter " V_{gs_max} "
7. If V_{gdr_max} is not given, V_{gdr_max} share the same model warning parameter " V_{gd_max} "
8. If V_{gpr_max} is not given, V_{gpr_max} share the same model warning parameter " V_{gp_max} "
9. If V_{gbr_max} is not given, V_{gbr_max} share the same model warning parameter " V_{gb_max} "
10. If V_{psr_max} is not given, V_{psr_max} share the same model warning parameter " V_{ps_max} "
11. If V_{bsr_max} is not given, V_{bsr_max} share the same model warning parameter " V_{bs_max} "
12. If V_{pdr_max} is not given, V_{pdr_max} share the same model warning parameter " V_{pd_max} "
13. If V_{bdr_max} is not given, V_{bdr_max} share the same model warning parameter " V_{bd_max} "

BJT

Terminal voltages checked: V_{be} , V_{bc} , V_{ce} , and V_{cs} .

Model parameters: V_{be_max} , V_{bc_max} , V_{ce_max} , and V_{cs_max} .

Terminal Voltages Checked	Model Params	Check criteria (i.e., the device under checking is called Q1) and Warning Issued
The device under checking is called Q1, for example.		
Vbe	Vbe_max	if $ V_{be} > V_{be_max}$: "Warning: Vbe (=xxx) of Q1 has exceeded Vbe_max (=yyy)"
Vbc	Vbc_max	if $ V_{bc} > V_{bc_max}$: "Warning: Vbc (=xxx) of Q1 has exceeded Vbc_max (=yyy)"
Vce	Vce_max	if $ V_{ce} > V_{ce_max}$: "Warning: Vce (=xxx) of Q1 has exceeded Vce_max (=yyy)"
Vcs	Vcs_max	if $ V_{cs} > V_{cs_max}$: "Warning: Vcs (=xxx) of Q1 has exceeded Vcs_max (=yyy)"

Diodes

Terminal voltage: V_j (from N to P) and V_f (forward).

Model parameters: Bv_max and Fv_max .

Terminal Voltages Checked	Model Params	Check criteria (i.e., the device under checking is called D1) and Warning Issued
The device under checking is called D1, for example.		
V_j (from N to P)	Bv_max	if $ V_j > Bv_max$: "Warning: V_j (=xxx) of D1 has exceeded Bv_max (=yyy)"
V_f (forward)	Fv_max	if $ V_f > Fv_max$: "Warning: V_f (=xxx) of D1 has exceeded Fv_max (=yyy)"

Note:

V_f means the junction voltage in forward bias and V_j means the junction voltage in reverse bias.

Resistor

Needed for both model and instance, instance Bv_max overrides model.

Terminal voltage: V_r .

Model parameters: Bv_max (a positive number).

Terminal Voltages Checked	Model Params	Check criteria (i.e., the device under checking is called D1) and Warning Issued
The device under checking is called R1, for example.		
V_r	Bv_max	if $ V_r > Bv_max$: "Warning: V_r (=xxx) of R1 has exceeded Bv_max (=yyy)"

Capacitor

Needed for both model and instance, instance Bv-max and Bv-min overrides model.

Terminal voltage: Vc.

Model parameters: Bv_max (a positive number), Bv_min.

Terminal Voltages Checked	Model Params	Check criteria (i.e., the device under checking is called C1) and Warning Issued
The device under checking is called C1, for example.		
Vc	Bv_max Bv_min	If Bv_min is not given if Vc > Bv_max, issue "Warning: Vc (=xxx) of C1 has exceeded Bv_max (=yyy) Else if Vc > Bv_max, issue "Warning: Vc (=xxx) of C1 has exceeded Bv_max (=yyy) if Vc < Bv_min, issue "Warning: Vc (=xxx) of C1 has exceeded Bv_min (=yyy)

Verilog-A (pVA) Messages

When compiling a Verilog-A module using the pVA compiler, you may see pvaI, pvaW, pvaE, or pvaNIY messages.

These messages have the following meanings:

- pvaI: Informational message that has no effect on compilation and simulation.
- pvaW: Warning message that possibly could affect compilation and simulation.
- pvaE: Error detected by pVA. The compilation will be aborted.
- pvaNIY: A “Not Implemented Yet” message for Verilog-A functions.

These messages provide useful information and help in debugging the Verilog-A module. For example:

```
*pvaI* #### Total 131 line-size(s), 29 expr(s), 2 contr(s), 4 init(s), 4  
behav(s), 2 port(s)  
*pvaW* macro `P_Q redefined at (constants.vams:34)
```

Topology Check Warning

In cases where the competing voltage sources have the same value the following warning message is written to the *.lis file.

```
**warning** inductor/voltage loop found  
Instance from to
```

```
0:x1      n1  0
0:v2      n1  0
```

Warning Message Index [00001-11146]

The following table lists an index of warning messages that can be encountered in PrimeSim HSPICE usage. The table is organized according to the index number, netlist line, sample warning message, and example of what triggered the warning.

Index	**warning** Message and Example
00043	R res_name is recovered to avoid V loop error. Example: <pre>.option warnlimit=1 message_limit '00043:2' v 1 0 1 r 1 0 1 r1 1 0 1 .option rm_rmin=10 .op .end</pre>
10001	(10001.sp:4) No independent source value specified. Reset to zero. Example: v1 1 0
10002	(10002.sp:5) No resistance value specified. Reset to the value of resmin. Example: R1 1 2
10003	(10003.sp:11) No capacitance value specified. Reset to zero Example: C11 1 2
10004	(10004.sp:21) No inductance value specified. Reset to 1e-12. Example: L1 3 2
10005	(10005.sp:8) Syntax error while using .print/.probe, missing output variable. Line ignored. Example: .probe \$no output variable is specified.
10006	(10006.sp:9) Missing parameters for .ic. Enter parameters for initial condition with their respective values; Line ignored. Example: .ic
10007	(10007.sp:12) Model nch device geometries will not be checked against the limits set by lmin, lmax, wmin and wmax. To enable this check, add a period(.) to the model name(i.e. enable model selector). Example: .model nch nmos level=1 lmin=1n lmax=2n
10008	(10008.sp:32) Fundamental frequency cannot be zero or negative for fourier analysis. Specify a positive frequency value; Fourier analysis ignored. Example: .four 0 v(6)

Index	**warning** Message and Example
10009	(10009.sp:32) Number of repeating operation (R) for PAT voltage source v should be integer greater than or equal to -1. Reset to default(R=0). Example: <code>v 1 0 PAT (5 1 1n 0.5n 0.5n 5n b1011 r=-2 rb=1 b01m1z)</code>
10010	(10010.sp:32) Number to specify the starting bit of repeating operation (RB) for PAT voltage source v cannot be less than 1. Please enter a positive number. Reset to default (RB= 1). Example: <code>v 1 0 PAT (5 1 1n 0.5n 0.5n 5n b1011 r=-1 rb=0 b01m1z)</code>
10011	(10011.sp:4) First TAP value must be largest of all TAP values for Pseudo Random Bit Generator source v. Assumes a descending order sort. Example: <code>.param a=4 vlow=1 vhigh=5 tdelay=1n trise=0.5n tfall=0.5n + rate=0.1g seed=10 rout=10k v 1 0 vlow LFSR (vlow vhigh tdelay trise tfall rate seed [2,5] + rout)</code>
10012	(10012.sp:11) Mutual inductance value has not been specified for mutual inductor. Reset to zero. Enter value of mutual inductance coefficient. Example: <code>k1 l1 l2 l3 tsat MAG=2 \$coefficient is not specified.</code>
10013	(10013.sp:14) Magnetization (MAG) of mutual inductor can only be -1 0 1, assumes the value is -1. Enter a valid number for MAG. Example: <code>k1 l1 l2 l3 tsat MAG=2 .model tsat L(ac=1e4 lc=100 hc=.1 tc=1u br=6.4k bs=6.75k hs=.6 + hcr=0)</code>
10014	(10014.sp:3) Attempt to reference undefined pin x1.mid1 in isub(); branch output ignored. Specify a valid pin name. Example: <code>x1 1 0 aa .subckt aa in out r1 in mid aa .probe tran isub(x1.mid1) .ends</code>
10015	(10015.sp:13) Attempt to reference undefined pin x1.1; biaschk output ignored. Specify a valid pin name. Example: <code>x1 1 0 aa .subckt aa in out r1 in mid aa .ends .biaschk subckt terminal1=1 simulation=tran monitor=i max=1 min=0.1 sname=x1</code>
10016	(10016.sp:3) ISUB() unsupported for top-level node 1; branch output ignored. Example: <code>.probe tran isub(1)</code>
10017	(10017.sp:12) Unable to find referenced node 100; Output variable ignored. Specify a valid node. Example: <code>.probe tran v(100)</code>

Index	**warning** Message and Example
10018	(10018.sp:8) Inductance for the inductor lout >= 0.1 henry, please verify it. Example: lout out 0 1
10019	(10019.sp:17) Skin Effect Coefficient parameter (Rs) for the element rout cannot be negative. Parameter has been ignored. Enter a valid Rs value. Example: rout out 0 1K rs=-1
10020	(10020.sp:6) Frequency "FMAX" cannot be negative for frequency dependent resistor r11. Parameter has been ignored. Enter positive value of "FMAX". Example: R11 1 2 1 Rs=1 FMAX=-100 FBASE=10 CONVOLUTION=1
10021	(10021.sp:6) Frequency "FBASE" cannot be negative for frequency dependent resistor r11. Parameter has been ignored. Enter positive value for "FBASE". Example: R11 1 2 1 Rs=1 FMAX=100 FBASE=-10 CONVOLUTION=1
10022	(10022.sp:6) "CONVOLUTION" can ONLY have value of 0 1 2 for frequency dependent resistor r11. Reset to default(CONVOLUTION=0). Enter valid value of CONVOLUTION. Example: R11 1 2 1 Rs=1 FMAX=100 FBASE=-10 CONVOLUTION=1
10023	(10023.sp:3) Propagation Delay (TD) cannot be negative for dependent source e11. Reset it to default (TD=0). Enter positive value of "TD". Example: E11 out 0 VCVS DELAY 1 0 td=-1n
10024	(10024.sp:6) "FBASE" cannot be > "FMAX" for element r11. Parameters have been ignored. Enter FBASE value < FMAX. Example: R11 1 2 1 Rs=1 FMAX=1k FBASE=1MEG CONVOLUTION=11
10025	(10025.sp:8) Scaling parameter (SCALE) for the x1.r1 cannot be < or = zero. Parameter has been ignored. Enter a valid value of "SCALE". Example: r1 in mid aa scale=0
10026	(10026.sp:5) Frequency (freq) of sine voltage source v1 cannot be negative. Reset to freq= "1/TSTOP". Enter a positive value. Example: v1 1 0 5 sin (5 1 -1g 0.5n 0.2 60)
10027	(10027.sp:5) Damping factor (theta) for sine voltage source v1 cannot be negative. Please enter a positive value for "theta". Reset to default (theta=0). Example: v1 1 0 5 sin (5 1 1g 0.5n -0.2 60)
10028	(10028.sp:4) Rise delay time for exponential voltage source v cannot be negative. Reset to default (rise delay time=0). Enter positive rise delay time value. Example: v 1 0 v0 exp(4 1 -2n 30n 40n 80n)
10029	(10029.sp:4) Fall delay time for exponential voltage source v cannot be negative. Reset to default (fall delay time= Rise delay time +TSTEP). Enter positive fall delay time value. Example: v 1 0 v0 exp(4 1 2n 30n -40n 80n)
10030	(10030.sp:4) Rise time constant for exponential voltage source v cannot be negative. Reset to default (rise time constant=TSTEP). Enter positive rise time constant value. Example: v 1 0 v0 exp(4 1 2n -30n 40n 80n)

Index	**warning** Message and Example
10031	(10031.sp:4) Fall time constant for exponential voltage source v cannot be negative. Reset to default (fall time constant=TSTEP). Enter positive fall time constant value. Example: <code>v 1 0 v0 exp(4 1 2n 30n 40n -80n)</code>
10032	(10032.sp:4) Frequency cannot be negative for FM voltage source v. Reset to default (freq= "1/TSTOP"). Enter a positive frequency value. Example: <code>v 1 0 4 SFFM (4 1 -1 2 20K)</code>
10033	(10033.sp:4) Carrier frequency cannot be negative for AM voltage source v. Reset to default (freq= 0). Enter a positive frequency value. Example: <code>v 1 0 AM (4 1 1 -2 20K)</code>
10034	(10034.sp:4) Modulation frequency cannot be negative for AM voltage source v. Reset to default (freq= 1/TSTOP). Enter a positive frequency value. Example: <code>v 1 0 AM (4 1 -1 2 20K)</code>
10035	(10035.sp:4) Offset coefficient cannot be negative for AM voltage source v. Reset to default (offset coefficient= 0). Enter a positive offset coefficient value. Example: <code>v 1 0 AM (4 -1 1 2 20K)</code>
10036	(10036.sp:4) Propagation delay cannot be negative for AM voltage source v. Reset to default (Propagation delay= 0). Enter a positive propagation delay value. Example: <code>v 1 0 AM (4 1 1 2 -20K)</code>
10037	(10037.sp:3) Rise or fall time cannot be negative for voltage source v. Please enter positive value of rise or fall time. Reset time to TSTEP. Example: <code>v 1 0 0 PULSE (0 5 1p -1p -1p 49p 50p)</code>
10038	(10038.sp:9) Number of turns (NT) for the element x1.l1 cannot be <= zero. Parameter has been ignored. Enter a valid value of NT. Example: <code>l1 mid 0 1n nt=-1</code>
10039	(10039.sp:14) Value of "TD" cannot be negative in .measure of tdly. Reset to absolute value. Example: <code>.measure tran tdly TRIG v(in) VAL=2.5 td=-1n RISE=1 TARG v(out) VAL=1 CROSS=1</code>
10040	(10040.sp:13) .MEASURE tdly never reached the trigger value, Measurement failed. Example: <code>.measure tran tdly TRIG v(in) VAL=9.5 td=1n RISE=1 TARG v(out) VAL=10 CROSS=1 \$the maximum value of v(out) is 2</code>
10041	(10041.sp:13) .MEASURE tdly never reached the target value, Measurement failed. Example: <code>.measure tran tdly TRIG v(in) VAL=9.5 td=1n RISE=1 TARG v(out) VAL=10 CROSS=1 \$the maximum value of v(out) is 2</code>
10042	(10042.sp:4) Both nodes of element v2 are connected together; Line ignored. Example: <code>v2 1 1 2</code>
10043	(10043.sp:32) Variable v11 does not exists in the netlist for DC analysis. Please specify the variable which is present in the netlist. v11 assumed to be new variable for DC analysis however results may not be desired. Example: <code>.DC v11 0v 5v 0.1v \$v11 does not exist</code>

Index	**warning** Message and Example
10044	(10044.sp:33) FIRSTRUN value -1 cannot be less than 1 for monte-carlo sweep in Transient Analysis. Please specify correct value of FIRSTRUN. Parameter has been ignored. Example: <code>.DC v1 0.5 1 0.5 monte=2 firstrun=-1</code>
10045	(10045.sp:27) The third value 0.00D+00 and the forth value 1.00D-12 both are smaller than the second value 1.00D-06, so the transient statement is interpreted as: <code>.tran tstep tstop tstart delmax</code> . Example: <code>.tran 1n 1u 0 1p</code>
10046	(10046.sp:12) Duplicate <code>.ic</code> declaration for node 1. Taking the last <code>.ic</code> value 1.00 Example: <code>.ic v(1)=0</code> <code>.ic v(1)=1</code>
10047	(10047.sp:6) Area for diode cannot be 0.0, reset to 1e-12 (default value). Enter valid value for area. Example: <code>d1 1 0 dml area=-100p</code>
10048	(10048.sp:6) Geometry parameter "pj" cannot be negative for diode. Please specify positive value to "pj". Reset to zero. Example: <code>d1 1 0 dml area=100p pj=-10n</code>
10049	(10049.sp:8) Noise parameter (noise) for the resistance can ONLY be 1 or 0. Please enter a valid value of "noise". Parameter has been ignored. Example: <code>r1 in mid aa noise=2</code>
10050	(10050.sp:5) Argument of asin can range from -1 to 1. Reset value of asin(5.00) to zero. Enter a valid argument for asin. Example: <code>.param aa=asin(5)</code>
10051	(10051.sp:4) Argument of acos can range from -1 to 1. Reset value of acos(5.00) to zero. Enter a valid argument for acos. Example: <code>.param aa=acos(5)</code>
10052	(10052.sp:5) Argument of log cannot be zero. The value of log(0) is replaced by the value of log(epsmin), set by option EPSMIN; Default = 1e-28. Example: <code>.param a='-log(0)'</code>
10053	(10053.sp:82) Node name tn in <code>.IC</code> or <code>.NODESET</code> cannot be found. Nodal initial condition is ignored. Enter a valid node. Example: <code>.IC V(TN)=0 \$node TN does not exist</code>
10054	(10054.sp:87) Invalid node pathname on output variable or initialized node net0103; this statement is ignored. Example: <code>.probe ac vdb(xeq_filter.net0103) \$ pathname xeq_filter does not exist</code>
10055	(10055.sp:137) Multiple ACmatch/DCmatch commands found, only the last one is used. Example: <code>.acmatch v(out) threshold=0 perturb=1</code> <code>.acmatch v(xi82.net18)</code>
10056	(10056.sp:116) Maximum 30 output variables are supported in acmatch/dcmatch analysis. Taking first 30 variables defined and ignoring the rest. Example: <code>.acmatch v(out) v(in_neg) v(gnda) v(vdda) v(in_pos)</code> <code>+ v(out) v(in_neg) v(gnda) v(vdda) v(in_pos) ... more than 30 output variables</code>

Index	**warning** Message and Example
10057	(10057.sp:29) DCmatch/ACmatch analysis only supports independent voltage source. i(xmdut0) output ignored. Enter a valid output source. Example: <code>.dcmatch I(xmdut0)</code>
10058	(10058.sp:119) Threshold -10.000 specified in ACmatch/DCmatch analysis is negative. Table has not been generated. Example: <code>.dcmatch v(out) threshold=-10 perturb=1</code>
10059	(10059.sp:119) Perturbation 7.0000 specified in ACmatch/DCmatch analysis exceeds valid range from 0.01 to 6.0. Reset to default (2.0). Enter a valid value. Example: <code>.dcmatch v(out) threshold=0 perturb=7</code>
10060	(10060.sp:17) Interval specified in ACmatch/DCmatch analysis cannot be negative. Parameter ignored. Enter a valid value. Example: <code>.acmatch v(12) perturb=3.0 interval=-3 threshold=0.9 matched=0.95</code>
10061	(10061.sp:119) Unrecognized Output variable specified in DCmatch/DCsens. Analysis ignored. Enter a valid output variable. Example: <code>.dcmatch gv(out) threshold=10 perturb=1</code>
10062	(10062.sp:49) Threshold -6.00000E-02 cannot be negative for DCsens analysis. value Reset to zero. Enter a valid vale for threshold. Example: <code>.DCsens v(2) file='2357' pertur=1.0 threshold=-60m interval = 2 _dsdbg=1</code>
10063	(10063.sp:49) Perturbation 7.0000 specified in DCsens analysis exceeds valid range from 0.0001 to 1.0. Reset it to default (0.05). Enter a valid value. Example: <code>.DCsens v(2) file='2357' pertur=7.0 threshold=60m interval = 2 _dsdbg=1</code>
10064	(10064.sp:49) Interval specified in DCsens analysis cannot be negative. Parameter reset to default(1). Enter a valid value. Example: <code>.DCsens v(2) file='2357' pertur=1.0 threshold=60m interval = -2 _dsdbg=1</code>
10065	(10065.sp:49) Groupbydevice (groupbydev) for DCsens analysis can be 0 or 1. Reset to default (0). Enter a valid value. Example: <code>.DCsens v(2) file='2357' pertur=1.0 threshold=60m interval = 2 _dsdbg=1 groupbydev=2</code>
10066	(10066.sp:6) LEVEL cannot be used along with keyword LAPLACE for element lowpass. Parameter has been ignored. Example: <code>Glowpass 0 out LAPLACE in 0 1.0 / 1.0 2.0 2.0 1.0 level=0</code>
10067	(10067.sp:116) Measurement time value specified exceeds analysis limit in measure variable vcp. .measure has been ignored. Enter a valid value in .measure. Example: <code>.tran `ln` `10n` start=`0` .measure TRAN vcp FIND v(vcp) AT=780u</code>
10068	(10068.sp:2) noise is not level= 49 model parameter. Parameter has been ignored. Example: <code>.model nch nmos version=3.22 level=49 noise=1</code>

Index	**warning** Message and Example
10069	(10069.sp:96) Multiple analysis statements are not allowed. The latter statement ignored. Example: <code>.op all</code> <code>.op</code>
10070	(10070.sp:16) Capacitance between second node and BULK node cannot be negative for resistor instance r1. Please enter a valid value of "C". Parameter has been ignored. Example: <code>r1 out 0 5 c=-1p</code>
10071	(10071.sp:21) Capacitance of 1.00 for c0 is too high. Please verify capacitance value. Example: <code>c0 in_neg 0 1</code>
10072	(10072.sp:5) Value of resistance r1 defined is limited to 1.000E-05 (RESMIN). Please modify RESMIN to incorporate resistances. Example: <code>r1 1 3 1e-10</code>
10073	(10073.sp:336) Measure results may be incorrect since initial start time is non-zero. Example: <code>.TRAN 0.001NS 9NS START=2NS</code> <code>.measure tran inv_delay trig v(01) val=1.25 rise=1 targ v(02) val=1.25 + fall=1</code>
10074	(10074.sp:24) Parameter weff is defined as an expression containing output signals, which may cause incorrect result. Recommend replacing output signals with user-defined functions. Example: <code>.param Weff='(w-(dW+pdWb*MAX((V(B,D)),(V(B,S)))+pdWd*ABS(V(D,S))))'</code>
10075	(10075.sp:0) Parameter(s), within encrypted block, contain output signals in expression which may cause incorrect result. Recommended: Replace output signals with user-defined functions. Example: <code>.prot</code> <code>.param Weff='(w-(dW+pdWb*MAX((V(B,D)),(V(B,S)))+pdWd*ABS(V(D,S))))'</code> <code>.unprot</code>
10076	(10076.sp:32) Number of monte-carlo sweep cannot be negative number for DC analysis. Please enter positive value of sweeps. Monte= -1.00 has been ignored. Example: <code>.DC v1 0.5 1 0.5 monte=-1</code>
10077	10077 Syntax error for output type xxx the left parenthesis is expected. This output is ignored. Example: <code>.probe tran v</code>
10078	10078 Invalid monitor type xxx. The .biaschk is ignored. Example: <code>.biaschk mos monitor=ttt ...</code>
10079	10079 Keyword xxx not supported for E(VCVS). Keyword is ignored. Example: <code>Ekomp fout 0 VOL='A0*v(vmid,load)' max=2 min=1 dmax=0 dmin=0</code>
10080	10080 Keyword xxx not supported for G(VCCS). Keyword is ignored. Example: <code>g1 1 2 cur='v(1,2)*0.001' dmax=0</code>
10081	10081 Unrecognized output type for xxx in .print or .probe; The line is ignored. Example: <code>.print tran t(v1)</code>

Index	**warning** Message and Example
10082	10082 Parameter xxx=xxxx overrides previously declared parameter xxx (filename:line_number). Example: <code>.param a=1</code> <code>.param b=2 a='b+1'</code>
10083	10083 Parameter xxx=xxx x overrides previously declared parameter xxx in encrypted file. Example: <code>"T1.incl"</code> <code>.prot</code> <code>.param a=1</code> <code>.unprot</code> <code>T1.sp</code> <code>.param b=2 a=3</code>
10084	10084 Unknown model parameter xxx. Parameter is ignored. Example: <code>.model pnh PNP LEVEL = 4</code> <code>isrr = 1.000 \$ isrr is unknown</code>
10085	10085 Multiple output options specified, using xxx Example: <code>.option post</code> <code>.option wdf</code>
10086	Invalid device name for .option subckt_union_device.
10087	Invalid device name for .option subckt_union_device. The name must start with a character.
10088	unrecognized output variable xxx on above line Example: <code>.meas tran m1 avg x(m1)</code>
10089	PWLZ source xxx has only one timepoint. Example: <code>VXD 5 0 pwlz (0 0.3)</code>
10090	No subckt name for .option etmif_noshare_subckt. Set to default null. Example: <code>.option etmif_noshare_subckt</code>
10091	Keyword "TD" in E, F, G or H element only works for DELAY Element. Keyword "TD" is invalid. Example: <code>g1 1 2 cur='v(1,2)*0.001' td=1n</code>
10092	Invalid keyword xxx in .MEASURE Equation Evaluation. It is ignored. Example: <code>.meas tran m1 param='m2*2' at=2n</code>
10093	xxx not a valid Monte Carlo, distribution, parameter will default to the nominal value. If not used for Monte Carlo, please ignore this warning.
10094	Cannot find string after "str". Example: <code>.param a=str(</code>
10095	Invalid encoding method xxx. It is ignored. Example: <code>v1 1 0 PAT (1 0 0p 1p 1p 1n [b1011 r=1 rb=0 b0110] r=2 rb=0 encode=DW128</code> <code>rd_init=1)</code>

Index	**warning** Message and Example
10096	Invalid value for keyword PRINT -- Ignored. The value only can be 0 or 1. Example: <code>.probe tran v(1) print=2</code>
10097	No subckt instance name for <code>.option skip_xinst</code> . Set to default null. Example: <code>.option skip_xinst</code>
11003	In <code>pwl(x)</code> , x cannot be >1 . Please specify x=1. Example: <code>E1 n1 n2 VCVS PWL(2) in1 in2</code>
11004	Capacitance value for element xxx is Cxxx. Capacitance should be a positive, non-zero value. Example: <code>C1 n1 n2 -2p</code>
11005	Unable to find referenced pin xxx. Node reset to ground. Specify a valid pin. Example: <code>.biaschk subckt terminal1=in max=2 \$ in is not a valid subckt pin node</code>
11007	Inductance value for element Lxxx <=0; check for M/Scale<=0 or negative value. Example: <code>L1 n1 n2 l=-2u</code>
11009	Value of RISE cannot be negative in <code>.measure</code> of xxx . Reset to default. Example: <code>.meas tran m1 trig v(1) val=0.5 rise=-3 targ v(2) val=2 rise=3 \$ rise=-3 is illegal</code>
11010	Value of FROM cannot be negative in <code>.measure</code> of xxx Example: <code>.meas tran mavg avg v(2) from=-3n \$ -3n is illegal</code>
11011	Value of TO cannot be negative in <code>.measure</code> of xxx. Exchange the value of TO and the value of FROM to let the value of TO bigger. Example: <code>.meas tran mavg avg v(2) to=-2n \$ -2n is illegal</code>
11012	xxx defined in subckt xxx resistance is negative.it may cause instability problem. Example: <code>r1 1 0 -2</code>
11016	Multiple <code>dcsens</code> commands found, only the last one is used. Example: <code>.DCsens V(vcc) Interval=3</code> <code>.DCsens V(vin) Interval=2</code>
11022	Only 1 connection at node xxx called in element xxxx.
11023	Parameter name is not defined in <code>.param</code> . Please enter parameter variable with their respective value/expression. Example: <code>.param \$ no parameter name defined</code> <code>R1 n1 n2 1</code> <code>.param a=1</code>
11024	Mosfet substrate nodes xxx are connected together but have no DC path to ground.
11025	No matched subckt ignore this <code>ic/nodeset</code> statement.
11027	Model xxx defined in <code>.biaschk</code> cannot be found in netlist.

Index	**warning** Message and Example
11028	Subckt xxx defined in .biaschk cannot be found in netlist.
11029	Element xxx defined in .biaschk cannot be found in netlist. Example: .biaschk nmos terminal1=ng terminal2=ns simulation=tr name=m1 \$ m1 is not defined in the netlist
11030	This device xxx cannot be found in VA.
11031	Element name must start with a letter "x" or "m". DELTAGD=0.1 is apply to all elements. Example: .iVdmargin D3 DELTAGD=0.3 \$ D3 is illegal
11057	Trigger event is undefined. Please specify a valid event. Measure failed. Example: .meas tran m1 trig v(n1) val=0.5 rise=1 targ v(2) val=2 rise=2 \$ node n1 is not defined in the netlist
11058	Target event is undefined. Please specify a valid event. Measure failed. Example: .meas tran m1 trig v(1) val=0.5 rise=1 targ v(2) val=2 rise=2 \$ node 2 is not defined in the netlist
11059	"FIND" function event is undefined. Please specify a valid event. Measure failed. Example: .meas tran m1 find v(vdd) at=2n \$ node vdd is not defined in the netlist
11060	"WHEN" function event is undefined. Please specify a valid event. Measure failed. Example: .meas tran m1 find v(vin) when v(out)=2 \$ node out is not defined in the netlist
11061	.print/.probe statement contains unused output; the unused output is ignored. Example: .print tran v(n2) i(r2) \$ n2 and r2 are not defined in the netlist
11067	Could not find branch element xxx ;branch output ignored Example: .print i(r3) \$ r3 is not defined in the netlist
11068	Invalid circuit pathname xxx found.
11069	Invalid keyword xxx for Noise Analysis. Parameter ignored.
11070	Incorrect value of LISTCKT for Noise Analysis. It should be LISTCKT=[1 0] . Parameter ignored. Example: .noise V(5) VIN 10 listckt=2
11071	Incorrect entry for LISTFREQ for Noise Analysis. It should be LISTFREQ=[all none freq1,freq2...]. Set as default format: LISTFREQ=ALL.
11072	Invalid interval entered. Value of interval for Noise Analysis should be >=0. Interval reset to 0.
11073	Value of LISTCOUNT for Noise Analysis cannot be less than 1. Parameter ignored. Enter a parameter value more than or equal to 1.
11074	Invalid value for "sparcalc" of .LIN Analysis. sparcalc can only be 0 or 1. Analysis has been ignored.
11075	Invalid value for "gdcalc" of .LIN Analysis. gdcalc can only be 0 or 1. Resetting it to default value 0.
11076	Invalid data format xxx entered for .LIN Analysis. Format only can be RI MA DB. Format set to xx as per output file format default.

Index	**warning** Message and Example
11077	Invalid value for "freqdigit" entered for .LIN Analysis. "freqdigit" should be greater than zero. Resetting to 6 (default).
11078	Invalid value for "spardigit" entered for .LIN Analysis. "spardigit" should be greater than zero. Resetting to 6 (default).
11079	Invalid value for "listsources" entered for .LIN Analysis. "listsources" value can be 0 1 yes no. Resetting value of listsources to zero.
11080	Invalid frequency value for "listfreq" entered for .LIN Analysis. "listfreq" value should be greater than zero. Resetting value of listfreq to none.
11081	Invalid entry to Method for Optimization model. Method can be PASSFAIL BISECTION.
11082	Parameter xxx cannot be negative value xxx for optimization using model xxx. Enter a positive value. The parameter reset to xxx. Example: <code>.model optmod opt relin=-0.2</code>
11083	The start sweep value for dec format should be greater than zero. Enter a non-zero positive value. It has been reset to 1e-6*(final sweep value)= xxx. Example: <code>.DC par1 DEC 10 0 100MEG</code>
11084	The start sweep value for oct format should be greater than zero. Enter a non-zero positive value. It has been reset to 1e-3*(final sweep value)= xxx. Example: <code>.DC par1 Oct 10 0 10</code>
11087	The start frequency value xxx should be greater than zero. Enter a non-zero positive value. It has been reset to xxxx.Example: <code>.AC DEC 10 0 100MEG</code>
11088	Above line is not allowed within subcircuit. The line is ignored. Example: <code>.subckt sub1 n1 n2</code> <code>.option post \$.option is not allowed within .subckt</code> <code>R1 n1 n2 1</code> <code>.ends</code>
11089	Global net name xxx in subckt pin list. The pin will be connected to the local net. Recommend to not use global net names in subckt pin lists. Example: <code>.global vdd</code> <code>.subckt sub1 n1 vdd</code> <code>R1 n1 vdd 1</code> <code>.ends</code>
11090	There is no matched node or element to the command "PRINT/.PROBE OP". The .dp# and .ic# will have invalid operating point information. Check the variables of the command "PRINT/.PROBE OP".
11091	Instance_name/macromodel_name is required. Correct syntax: <code>.ivdmargin instance_name/macromodel_name DELTAGD=val</code> ; DELTAGD=0.1 is apply to all elements.
11092	Value of DELTAGD is required. Reset DELTAGD to default 0.1. Enter a valid value. Example: <code>.ivdmargin m1</code>

Index	**warning** Message and Example
11093	Illegal value is assigned to option xxx -- Resetting to default value 0. Example: <code>.option gen_cur_pol=aaa \$ aaa is illegal</code>
11094	Unknown option xxx. Check the option name again.
11097	Above line is not allowed within module/modulevar. The line is ignored. Example: <code>.module mod1</code> <code>.tran ln 100n \$.tran is not allowed in .module block</code> <code>.subckt subl n1 n2</code> <code>...</code> <code>.ends</code> <code>.endmodule</code>
11098	Invalid syntax. Specific MOSFET instance follow the command <code>.MODEL_INFO</code> ; the <code>.model_info xxx</code> is ignored. Example: <code>.model_info main \$ main is not defined in netlist</code>
11099	Mosfet xxx Instance length or width does not fit the lmin/lmax, wmin/wmax range for the model xxx . The instance Length=xxx Width=xxx. The model Lmin=xxx Lmax=xxx Wmin=xxx Wmax=xxx.
11100	Expected a keyword in LSTB statement.', ' Analysis is omitted.
11101	The v-source name in LSTB statement is invalid. Analysis is omitted.
11102	The mode type is missing in LSTB statement. Analysis is omitted.
11103	The mode type in LSTB statement is invalid. Analysis is omitted.
11104	The v-sources are missing in LSTB statement. Analysis is omitted.
11105	The v-source for single mode is missing in LSTB statement. Analysis is omitted.
11106	The first v-source for diff/comm mode testing is missing in LSTB statement. Analysis is omitted.
11107	Expected a number after method in LSTB statement. Reset to default method.
11108	The number assigned to method in LSTB statement is invalid. Reset to default method.
11109	The second v-source for diff/comm mode in LSTB statement is missing. Analysis is omitted.
11110	The second v-source for diff/comm mode in LSTB statement is invalid. Analysis is omitted.
11111	Found an invalid keyword xxx for LSTB analysis. Ignored it and continued.
11112	The second v-source for diff/comm mode testing is missing in LSTB statement. Analysis is omitted.
11113	AC analysis is missing. LSTB', ' analysis is omitted.
11114	The v-source xxx in LSTB statement is invalid. Analysis is omitted.
11115	Invalid subcircuit path xxx in LSTB v-source's definition. Analysis is omitted.

Index	**warning** Message and Example
11116	Attempt to reference undefined modulevar xxx. Example: <code>Xmod1 n1 n2 mod1::sub modulevar=mvar \$ mvar is not defined by .modulevar</code>
11117	Since <goalmax/goalmin> specified, switch to bisection method automatically. Example: <code>.measure tran Tsense when PAR('ABS(V(SA)-V(SAB))') = '0.8*VALVDD' + GOALMAX</code>
11118	The equation in .MEASURE cannot be a function of node voltages or branch currents. Unexpected results may incur. Example: <code>.measure tran m1 param='v(2)+2'</code>
11119	Cannot find model xxx at the same level of .appendmodel command. Verify that it is defined or used. Example: <code>.option appendmodel=all .appendmodel hci_1 mosra b3_nch nmos \$ hci_1 is not defined</code>
11120	Cannot find model xxx at any level of .appendmodel command. Verify that it is defined or used. Example: <code>.appendmodel hci_1 mosra b3_nch nmos \$ hci_1 is not defined</code>
11121	Could not find current for verilog-A port or branch xxx; output ignored
11127	Scaling parameter (SCALE) for the xxx cannot be < or = zero. Reset to default(1.0). Example: <code>R1 n1 n2 r=2 scale=-2</code>
11131	Unexpected negative parameter in model xxx parameter =xxx value= xxx.
11132	The measured variable for xxx is not defined.
11133	Maximum 2e9 random values are supported for Monte Carlo of LHS sampling method. Here, the number is xxx.
11134	Multiplier param M=xxx on subckt instance xxx is less than one. Reset M to one. Example: <code>X1 n1 n2 inv m=0.2</code>
11135	Multiplier param M=xxx on instance xxx is less than one. Example: <code>R1 n1 n2 1 m=0.5</code>
11136	Element is ignored in .OPTION MODPRT output because of encrypted block.
11137	Element xxx in .print, .plot, .probe or .measure statements is invalid. Output is ignored. Example: <code>.probe tran par('v1(x1.m4)*2') \$ m4 is not defined in the netlist</code>
11138	Constant value for xxx is expected. It will be ignored. Example: <code>.defparam x1.p1='a+1'</code>
11139	The Expression Evaluation of .MEASURE statement contains output variables which may have different values for different time points. The value in the last time point is used. Example: <code>.meas tran m1 param='v(2)+3'</code>
11140	Unsupported keyword xxx found in .measure statement. The statement will be ignored. Example: <code>.meas tran m1 param='m0+2' at=2 \$ at is not supported</code>

Index	**warning** Message and Example
11141	No matched subckt xxx ignore this biaschk statement. Example: <code>.biaschk subckt mname=x2 expr='abs(v(POS,NEG)*i(POS))/(MX*(W+0.64u)*L)/1.0e-12'</code> max=180u \$ x2 is not defined in the netlist
11142	This node xxx cannot be found in VA.
11143	MAX value cannot be < MIN value for element xxx. Ensure MIN<MAX. Example: <code>F1 13 5 VSENS MAX=3 MIN= 5</code>
11145	The value of the expression defined in xxx is NaN. Please check this expression.
11146	Model xxx device geometries will not be checked against the limits set by lmin, lmax, nfinmin and nfinmax. To enable this check, add a period(.) to the model name (that is, enable model selector). Example: <code>.model nch nmos level=54 lmin=1u lmax=2u</code>

Error Message Index [20001-20084]

The following table lists an index of error messages that can be encountered in PrimeSim HSPICE usage. The table is organized according to the index number, netlist line, sample warning message, and example of what triggered the warning.

Index	**error** Message and Example
20001	(20001.sp:19) Unbalanced parentheses. Example: <code>.param aa = '5*(1+bb'</code>
20002	(20002.sp:38) Successive time point 5.000E-12, 4.900E-12 must increase for piecewise linear voltage source v8. Example: <code>v8 8 0 0 PWL (0 0 1p 1 5p 1 4.9p 0 10n 0)</code>
20003	(20003.sp:30) Cannot find table of data v11 for DC analysis. Please specify data table which has valid definition. Example: <code>.DC DATA=v11 \$ v11 does not exist</code>
20004	(20004.sp:25) TSTOP cannot be zero or negative for Transient Analysis. Please enter a positive value for TSTOP. Example: <code>.tran 0.1ns -50ns</code>
20005	(20005.sp:25) Sweep step cannot be 0 for Transient Analysis. Please enter positive value of sweep. Example: <code>.tran 0.1ns 50ns sweep v1 0 5 0</code>
20006	(20006.sp:5) Number of Stages (TAP) for Pseudo Random Bit Generator source v1 should lie between 2 and 30. Please enter a valid value of TAP. Example: <code>V1 1 0 LFSR (0 1 1u 1n 1n 10meg 1 [-5, 2] rout=10)</code>
20007	(20007.sp:150) For AC analysis STOP frequency cannot be less than START frequency. STOP frequency should be greater than START frequency. Example: <code>.AC lin 1000 10GHz 1GHz</code>

Index	**error** Message and Example
20008	(20008.sp:195) Number of points for AC analysis cannot be less than 1. Please enter number of points more than 1. Example: .AC lin 0 1Hz 1GHz
20009	(20009.sp:4) Definition not declared for rsh(1+b). Please enter a defined name. Example: .param rsh=100 b=10 Rbody 1 0 'rsh(1+b)'
20010	(20010.sp:22) Mutual Inductor declaration contains only one reference inductor. Please enter at least 2 reference inductors. Example: K1 L1
20011	(20011.sp:509) Definition of model/subckt "pch_hvt_mac" is not found for the element "xcut.xmmu7012345678901234567890". Please specify a defined model/subckt name. Example: xMMU7012345678901234567890 net132 MU70_GATE MU70_SRC MI33-M_u3_BULK pch_hvt_mac ad=0.0119p as=0.017044p dfm_flag=1 \$ pch_hvt_mac does not exist
20012	(20012.sp:15) Number of data points used in Delay (NPDELAY) cannot be negative for element Fd. Enter positive NPDELAY value. Example: Fd in 0 DELAY vcc TD=7ns SCALE=5 NPDELAY=-10
20013	(20013.sp:17) For W element the number of terminals exceeds 6. For the given number of signal conductors, N = 2, the number of terminals should be 2(N + 1) = 6. Enter correct number of terminals. Example: WTL1_0_0 1 2 7 0 3 4 0 TABLEMODEL=MODEL_5 N=2 L=LEN delayopt=3
20014	(20014.sp:17) The W-element has invalid number of terminals 5 for the given number of signal conductors N = 2. The number of terminals should be 2(N + 1). Enter valid number of terminals. Example: WTL1_0_0 1 0 3 4 0 TABLEMODEL=MODEL_5 N=2 L=LEN delayopt=3
20015	(20015.sp:12) Physical length defined for U-element u1 cannot be zero or negative. Please define a positive value to L. Example: U1 3 10 2 0 5 1 4 0 USTRIP L=-0.178
20016	(20016.sp:17) Number of conductors (N) parameter not found, for W element. Please define value of N. Example: WTL1_0_0 1 2 0 3 4 0 TABLEMODEL=MODEL_5 L=2 delayopt=3 \$ N=2
20017	(20017.sp:12) Number of nodes "7" of U-element "u1" does not match with number of nodes "8" specified for U model "ustrip". Number of nodes of instance should match with the model. Example: U1 3 10 2 0 5 1 0 USTRIP L=0.178 \$LUMPS=1 .Model USTRIP U LEVEL=3 PLev=1 Elev=1 Dlev=2 Nl=3 Ht=381u + Wd=305u Th=25u Sp=102u Ts=838u Kd=4.7 \$ LEVEL=3 required 8 nodes
20018	(20018.sp:32) Unknown setting for ComputeGo. ComputeGo can be YES or NO. Example: .Fsoptions opt1 Printdata=yes Computers=yes Computegd=yes + Computego=2

Index	**error** Message and Example
20019	(20019.sp:38) Physical length defined for W-element cannot be zero or negative. Please define a positive value. Example: W4 N=3 1 2 3 0 4 5 6 0 RLGCfile=wel4rs.rlc l=-1.2
20020	(20020.sp:25) TSTEP cannot be zero or negative for Transient Analysis. Please enter a positive value for TSTEP. Example: .tran -0.1ns 50ns
20021	(20021.sp:5) Denominator cannot be zero for E-element ehpass. Please enter a non-zero denominator. Example: Ehipass out 0 LAPLACE in 0 0.0,0.0,0.0,1.0 / 0.0,0.0,0.0,0.0
20022	(20022.sp:4) Seed for Pseudo Random Bit Generator source v1 should lie between 0 and 1073741823. Please enter a valid value of seed. Example: .param a=4 vlow=1 vhigh=5 tdelay=1n trise=0.5n tfall=0.5n rate=0.1g seed=-10 rout=10k v1 1 0 LFSR (vlow vhigh tdelay trise tfall rate seed [5,2] rout)
20023	(20023.sp:13) Number of nodes miss match between instance "xres" and subcircuit "ress ". Subcircuit definition has 2 node(s) whereas subckt instance was found with 3 node(s). Please specify same number of nodes. Example: .subckt ress 7 1 R1 7 1 50 .ends Xres 7 1 10 ress
20024	(20024.sp:3) Name of rlgcmodel cannot start with a number. Names should start with alphabet. Check/correct the parameter name. Example: W1 N=3 1 3 5 0 2 4 6 0 RLGCMODEL=11 l=0.97
20030	xxx is a reserved parameter name, please change the parameter name and rerun; if this is an initialization for a data statement, it can be removed. Example: .param temper=2 \$ 'temper' is a reserved word
20031	No algebraic for function definition. Example: .param f(x)
20041	Multiplication factor(M) for the element 'a,' cannot be < or = zero. Enter a valid M value. Example: R1 1 0 1 m=-2
20042	In Transient Analysis TSTEP xxx cannot be greater than TSTOP-TSTART xxxx- xxxxx . Please enter TSTEP < TSTOP-TSTART. This TSTEP STOP pair has been ignored. Syntax: .TRAN tstep1 tstop1 [tstep2 tstop2 ...tstepN tstopN] [START=val] [UIC] [SWEEP var type np pstart pstop]'. Example: .TRAN 10n 5n
20043	MIN and MAX are not supported for the delay element xxx. Example: Edelay in_delay 0 DELAY in 0 TD=2n max=2
20044	Parameter has been repeated. Enter parameter only once. Example: Qopamp1 c1 b3 e2 s Mod1stagepnp AREA=1.5 AREAB=2.5AREAB=2

Index	**error** Message and Example
20045	Duplicate parameter is set or the number of element nodes is not within the range of 3-7. Example: M1 d g s b nch l=1u w=2u l=2u
20046	Undefined parameter or function definition "xxx". Please enter a defined name. Example: .param a='b+1' \$ b is not defined in the netlist
20047	xxx settings have been parsed as,' Start:xxx Stop:xxx Step:xxxx Sweep size:xxx. The number of steps is larger than allowed 1E+9. Example: .tran 2.98362e-14 6.550567e-10 sweep __out_load 10 0 6e-12
20048	Reference xxx not found, or instance width or length does not fit into the wmax/wmin or lmax/lmin range. It was referenced in element xxx. The channel width=xxx and length=xxx. Please enter channel width and length that do not exceed the referenced maximal area Lmax=xxx Lmin=xxx Wmax=xxx Wmin=xxxx. Example: M1 g d s b nch l=3u w=6u .model nch.1 nmos level=54 lmax=2u lmin=0.5u wmax=4u wmin=1u
20049	Invalid output file format xxx entered for .LIN Analysis. Format only can be sele citi touchstone. Analysis has been ignored. Example: .LIN sparcalc=1 modelname=my_custom_model + filename=mydesign format=rlgc noisecalc=1
20051	Could not find parameter function "xxx". Enter a defined parameter function. Example: .tran 1.0e-10 1.0e-9 sweep optimize=opt results=y model=opt_model \$ opt is illegal
20052	Could not find measure variable "xxx" for optimization. Enter a valid measure variable. Example: .tran 1.0e-10 1.0e-9 sweep optimize=opt1 results=y model=opt_model \$ y is not defined in .measure
20053	Invalid number of parametric entry for optimization. Syntax is: .PARAM ParamName=OptParFun (Initial, Lower, Upper). Example: .param x=opt1(0,1)
20054	Missing AC analysis type. Enter one type of DEC/OCT/LIN/POI. Example: .AC 1K 100MEG
20056	Reference xxx not found, or instance width or length does not fit into the nfinmax/nfinmin or lmax/lmin range. It was referenced in element xxx. The channel nfin=xxx and length=xxx .Please enter channel nfin and length that do not exceed the referenced maximal area Lmax=xxx Lmin=xxx Nfinmax=xxx Nfinmin=xxx. Example: M1 g d s b nfet NF=5 NFIN=20model nfet.1 nmos level=72 VERSION = 106.1 ...
20057	Stop time should be larger than start time in the same pair in twindow. Example: .set_sample_time twindow 10u 5u
20058	Stop time should not be larger than start time in the next pair in twindow. Example: .set_sample_time twindow 10u 20u 15u 30u #define hsperror_sample_point_02_index 20058

Index	**error** Message and Example
20059	Reference xxx not found. It was referenced in element xxx. The channel width=xxx and length=xxx.
20060	Missing message content in biaschk. Example: <code>.biaschk mos terminal1=nd monitor=v max=1 message=</code>
20061	sname of biaschk is not supported in subckt. Example: <code>.subckt sub2 d g s b .biaschk mos terminal1=nd sname=sub1 max=1 monitor=v M1 d g s b nch l=1u w=2u .ends</code>
20062	Invalid output variable in element xxx. Enter a valid output. Example: <code>r1 n1 n2 r='i(v3)*2' \$ v3</code> is not defined in the netlist
20063	Too many arguments in function call. Example: <code>.param f(x,y)='x+y' a=f(1,2,3)</code>
20064	Too few arguments in function call. Example: <code>.param f(x,y)='x+y' a=f(1)</code>
20066	Source element, keyword "PWL" cannot be node name. Please specify a valid source node name. Example: <code>V1 n1 pw1 1</code>
20067	[lower upper] in .meas statements have to be identical to support multiple goals in pushout bisection. Example: <code>.Measure Tran pushout_4 When v(D_Output)='vih/4' rise=1 pushout_per=0.03 upper</code>
20068	Invalid key word "xxx" in .TRAN command. Example: <code>.tran 1n 'simtime' TSTART=0</code>
20069	No arguments in function call xxx. Please check .param xxx statements. Example: <code>.param a=3 a(x,y)='x+y' b=a</code>
20071	Element xxx does not support LEVEL = xxx. Specify a valid LEVEL. Example: <code>Qn10 VDD! qp outp npn_level0 .model npn_level0 NPN LEVEL = 0 BF=180 IS=1E-13</code>
20073	Invalid model level xxx. Specify a valid model level. Example: <code>.model nch nmos level=99</code>
20074	Cannot redefine parameter to itself=xxx. Example: <code>.param a=a</code>
20077	Inductor/voltage source loop found containing xxx defined in subckt xxx.
20078	Inductor/voltage source loop found on signal 0 :xxx defined in vec source.
20079	Values of a list must sorted from smallest to largest. Example: <code>.tran 1n 50n sweep monte=list (8:6)</code>

Index	**error** Message and Example
20080	The value of a list must be greater than 0. Example: <code>.tran 1n 50n sweep monte=list (0:5)</code>
20082	In <code>.param</code> statement, the argument "xxx" of function "xxx" cannot use quotation marks. Please remove the quotation marks. Example: <code>.param f('x')='x+2'</code>
20083	Start Frequency <value> for AC Analysis cannot be negative. Enter START frequency greater than zero. Example: <code>.AC DEC 10 -4K 400MEG</code>
20084	td is not specified or the value of td <= 0, please check it. Example: <code>Tin 14 0 10 0 ZO=50 TD=0</code>

Exit Codes

PrimeSim HSPICE prints these exit codes. The numerals below may be preceded by the word *SIGTERM* or *SIGABRT*. The corresponding meanings are as follows:

- 0: Simulation succeeded.
- 1: Simulation failed due to errors, e.g., syntax error, non-convergence, etc.
- 2: PrimeSim HSPICE stopped due to lack of a PrimeSim HSPICE license.
- 3: The simulation is terminated by **Ctrl+** (Control key plus backslash) on Linux.
- 6: The simulation is terminated by signal SIGABRT due to abnormal operation of the source code, such as: out of memory, invalid memory access, etc.
- 8: Floating-point exception.
- 11: Segmentation fault (invalid memory address access).
- 15: PrimeSim HSPICE stopped by a UNIX `kill` command.
- 24: CPU limit exceeded.
- 28: No space left on device. (Simulation cannot start.)
- 38: Error writing to file (either `.lis` or `.tr0`). (Simulation started.)
- 99: Error met during tool doing distribution simulation (`-dp`).
- 101: PrimeSim HSPICE stopped by **Ctrl+C**.